



AWS Solutions Architect з нуля: глибоке навчання

Підручник до іспиту AWS Certified Solutions Architect – Associate (SAA-C03).

46 розділів · 327 питань для самоперевірки з відповідями · схеми · практика в AWS.

Інтерактивна версія з підказками, картками і пробним іспитом:

<https://nexgen-fullstack.github.io/AWSSAAC03/deep/>

Оновлено 30.09.2026. Незалежний навчальний підручник. Не пов'язаний з Amazon Web Services і не схвалений AWS. AWS і назви сервісів — торговельні марки Amazon.com, Inc. або її афілійованих осіб. Практичні питання — оригінальні, це не реальні питання іспиту. «Хмаринка» — вигадана компанія.

Зміст

МОДУЛЬ 0 · СТАРТ З НУЛЯ

- 1  Як пройти цей курс і стати архітектором рішень
- 2  Основи ІТ, частина 1: комп'ютер, сервер, операційна система і віртуалізація
- 3  Основи ІТ, частина 2: мережі, IP-адреси, порти, DNS і HTTPS
- 4  Основи ІТ, частина 3: дані, бази даних, API і шифрування
- 5  Що таке хмара і навіщо вона бізнесу
- 6  Глобальна інфраструктура AWS — регіони, зони доступності, edge
- 7  Твій акаунт AWS — безкоштовний план, бюджет, root і Well-Architected

МОДУЛЬ 1 · БЕЗПЕКА І ДОСТУП

- 8  IAM — користувачі, групи, політики і принцип найменших привілеїв
- 9  Ролі, тимчасові ключі і вхід через корпоративні акаунти
- 10  Багато акаунтів — AWS Organizations, SCP, Control Tower
- 11  Шифрування, ключі, секрети й сертифікати — KMS, CloudHSM, ACM, Secrets Manager
- 12  Захист від атак і виявлення загроз — WAF, Shield, GuardDuty та компанія

МОДУЛЬ 2 · МЕРЕЖА

- 13  VPC з нуля — адреси, підмережі, маршрути, Internet Gateway і NAT
- 14  Охорона мережі — Security Groups, Network ACL і безпечний доступ до серверів
- 15  Як VPC говорить із сервісами й іншими VPC — endpoints, PrivateLink, Peering, Transit Gateway
- 16  Офіс ↔ AWS — Site-to-Site VPN, Direct Connect, Client VPN і гібридний DNS

- 17  Route 53 — DNS, який керує трафіком
- 18  Швидко для всього світу — Amazon CloudFront і AWS Global Accelerator

МОДУЛЬ 3 · ОБЧИСЛЕННЯ

- 19  Amazon EC2 — віртуальні сервери зсередини
- 20  Як платити за сервери — On-Demand, Savings Plans, Reserved, Spot і Dedicated
- 21  Диски серверів — Amazon EBS, instance store і снапшоти
- 22  Балансувальники навантаження — ALB, NLB і Gateway Load Balancer
- 23  Auto Scaling — сервери, що додаються й замінюються самі
- 24  Контейнери — Docker, Amazon ECS, Amazon EKS, AWS Fargate і ECR
- 25  AWS Lambda — код без серверів
- 26  API й платформи — Amazon API Gateway, Elastic Beanstalk, AWS Batch, Amplify

МОДУЛЬ 4 · СХОВИЩА

- 27  Amazon S3 — основи безрозмірного складу об'єктів
- 28  S3 глибше — класи, lifecycle, версії, реплікація, Object Lock і керування даними
- 29  Спільні файли — Amazon EFS і сімейство Amazon FSx
- 30  Гібридні сховища і перенесення даних — Storage Gateway, DataSync, Transfer Family, Snow

МОДУЛЬ 5 · ІНТЕГРАЦІЯ

- 31  Черги й розсилки — Amazon SQS і Amazon SNS
- 32  Події й оркестрація — EventBridge, Step Functions, Amazon MQ, AppFlow

МОДУЛЬ 6 · ПЕРЕЇЗД У ХМАРУ

- 33**  Переїзд у хмару — стратегії 7R, MGN, DMS, SCT і план міграції
-

МОДУЛЬ 7 · БАЗИ ДАНИХ

- 34**  Amazon RDS — реляційні бази без рутинного адміністрування
- 35**  Amazon Aurora — реляційна база, переосмислена для хмари
- 36**  Amazon DynamoDB — NoSQL для будь-якого масштабу
- 37**  Кеш і спеціалізовані бази — ElastiCache, MemoryDB, DocumentDB, Neptune, Keyspaces, Timestream, OpenSearch
-

МОДУЛЬ 8 · ДАНІ, АНАЛІТИКА Й AI

- 38**  Потоки даних у реальному часі — Kinesis Data Streams, Data Firehose, Flink, MSK
- 39**  Аналітика — озеро даних, Glue, Athena, Redshift, EMR, Lake Formation, Quick Sight
- 40**  Штучний інтелект — готові AI-сервіси, SageMaker AI і генеративний AI
-

МОДУЛЬ 9 · МОНІТОРИНГ, НАДІЙНІСТЬ І ГРОШІ

- 41**  Моніторинг і керування — CloudWatch, CloudTrail, Config, X-Ray, Systems Manager, CloudFormation
- 42**  Надійність і відновлення після катастроф — RPO, RTO, стратегії DR, AWS Backup
- 43**  Гроші — архітектура, що коштує менше, і інструменти контролю витрат
-

МОДУЛЬ 10 · ФІНАЛ

- 44**  Велика архітектура Хмаринки — усе разом
- 45**  Розбір іспиту SAA-C03 — формат, домени, слова-підказки і метод відповіді
-

46 🚀 Що далі — план підготовки, чек-лист готовності і шлях після сертифіката

Як читати цей підручник

Розділи йдуть від простого до складного: спершу основи ІТ і хмари, потім безпека, мережа, обчислення, сховища, інтеграція, бази даних, аналітика, моніторинг, надійність і вартість. Наприкінці — велика архітектура, розбір іспиту і план.

У кожному розділі: 🎬 історія «Хмаринки» (навіщо це потрібно), 🖼️ образ (щоб запам'ятати), пояснення, схеми, ✏️ практика в AWS, 💡 «Запам'ятай», 🎯 як питають на іспиті, ⚠️ пастки і ✅ питання «Перевір себе» з відповіддю й поясненням одразу під питанням — спершу прикрий відповідь рукою.

Практика в AWS завжди закінчується прибиранням створених ресурсів. Перед першою практикою прочитай розділ 7 про безкоштовний план і бюджет.



Як пройти цей курс і стати архітектором рішень

🎯 Безпека · 30%

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Вартість · 20%



Історія Хмаринки. П'ятниця, 23:40, «чорна п'ятниця». Інтернет-магазин подарунків «Хмаринка» зі Львова запустив найбільший розпродаж року — і за двадцять хвилин сайт перестав відкриватися. Увесь магазин — сайт, база замовлень, фото товарів — жив на одному сервері в підсобці офісу. Сервер «захлинувся» від тисяч покупців, а потім ще й перегрівся. Засновниця Олена рахує втрачені замовлення і вирішує: досить. Хмаринка переїжджає в AWS. А проєктувати переїзд доведеться тобі — новому **архітектору рішень** (Solutions Architect).



Образ. Архітектор будинку не кладе цеглу сам. Він слухає замовника («хочу теплий, надійний, світлий і недорогий дім»), знає сотні матеріалів і обирає ті, що разом дадуть потрібний результат. AWS — величезний будівельний магазин на 200+ «матеріалів»-сервісів. Архітектор рішень обирає й поєднує їх так, щоб система була **захищеною, надійною, швидкою і недорогою одночасно.**

Хто такий архітектор рішень

Solutions Architect — людина, яка перетворює бізнес-потребу на технічний план. Олена каже: «Хочу, щоб сайт витримав розпродаж, щоб дані клієнтів

ніхто не вкрав і щоб рахунок за хмару не з'їв прибуток». Архітектор відповідає: «Сайт стоятиме на кількох серверах у двох окремих дата-центрах, їх кількість сама зростатиме під час піку; дані — зашифровані; за постійне навантаження заплатимо зі знижкою, а піки покриємо дешевими тимчасовими серверами».

Щодня архітектор:

- **розпитує** про вимоги: скільки користувачів, скільки даних, що буде, якщо сайт ляже на годину, які закони діють;
- **обирає** сервіси і пояснює чому — зазвичай з кількох робочих варіантів;
- **малює схеми** — щоб розробники, безпека і бухгалтерія побачили одну картину;
- **шукає компроміси**: швидше майже завжди дорожче, надійніше — складніше;
- **перевіряє** рішення за кращими практиками і виправляє слабкі місця.

Архітектор не мусить пам'ятати кожну кнопку в консолі. Але він мусить розуміти, **як сервіси працюють усередині і чим вони відрізняються**. Саме цьому вчить цей курс — і саме це перевіряє іспит.

Чим архітектор відрізняється від сусідніх професій:

Роль	Головне питання	Приклад задачі
Розробник	Як написати код?	Зробити кошик на сайті
DevOps / SRE	Як автоматично випускати й підтримувати систему?	Налаштувати конвеєр релізів і аларми
Адміністратор	Як тримати сервери в порядку?	Оновити ОС, налаштувати бекапи
Архітектор рішень	З чого побудувати систему, щоб вона відповідала вимогам?	Спроектувати сайт, що витримає 10× трафіку за мінімальні гроші

Іспит SAA-C03 за п'ять хвилин

AWS Certified Solutions Architect – Associate — найпопулярніший сертифікат AWS. Код актуальної версії — **SAA-C03** (станом на вересень

2026 року; сайти, які пишуть про «C04», помиляються).

- **65 питань за 130 хвилин.** З них 50 оцінюються, а 15 — «пробні», які AWS тестує для майбутніх версій. Які саме пробні — невідомо, тож старайся на кожному.
- **Прохідний бал — 720 з 1000.** Бал масштабований: це не «72% правильних», але орієнтир схожий. Модель компенсаторна — не треба окремо «здати» кожную тему, рахується загальний результат.
- **Штрафу за помилку немає.** Порожнє питання — гарантований нуль, здогадка — шанс. Відповідай на все.
- **Два типи питань:** *multiple choice* — одна правильна з чотирьох; *multiple response* — дві чи більше правильних з п'яти-шести (у питанні написано, скільки обрати).
- **Ціна — \$150,** сертифікат діє **3 роки.** Після здачі в кабінеті з'являється знижка 50% на наступний іспит AWS.
- **Де складати:** центр тестування Pearson VUE або онлайн удома з веб-камерою і наглядачем.
- **Мова:** української немає, тож терміни вчимо англійською. Якщо англійська не рідна — **до запису** на іспит попроси +30 хвилин (*ESL +30*) у кабінеті сертифікації. Це безкоштовно і схвалюється автоматично.

Іспит має чотири **домени** — це ті самі чотири побажання Олени:



Безпека — 30%

Design Secure Architectures

«щоб не зламали»: доступи, шифрування, мережевий захист



Відмовостійкість — 26%

Design Resilient Architectures

«щоб не падало й не губило дані»: кілька зон, черги, DR



Продуктивність — 24%

Design High-Performing Architectures

«щоб було швидко»: кеш, масштабування, правильні сховища



Вартість — 20%

Design Cost-Optimized Architectures

«щоб недорого»: тарифи, класи зберігання, трафік

Майже кожне питання зачіпає кілька domenів одночасно: «найдешевше, але надійне», «найбезпечніше з найменшими зусиллями».

Питання іспиту — це міні-історії: «Компанія має... їй потрібно... Яке рішення відповідає вимогам з найменшими операційними зусиллями?». Часто **всі чотири варіанти технічно працюють**, але лише один виконує **всі** умови з тексту. Тому головна навичка — не пам'ять, а **уважне читання вимог і вибір**.



Ключові фрази, які вирішують відповідь

- **MOST cost-effective** — найдешевше з рішень, що виконують усі вимоги (не найдешевше взагалі!).
- **LEAST operational overhead** — найменше ручної роботи: керовані (managed) і serverless-сервіси замість власних серверів і скриптів.
- **highly available** — працює попри збій однієї зони; **fault tolerant** — працює взагалі без перерви.
- **without changing the application code** — рішення, яке не вимагає переписувати застосунок.

Як думає архітектор: чотири питання до будь-якого рішення

Перш ніж обрати сервіс, архітектор подумки ставить чотири запитання — ті самі, що й чотири домени іспиту, плюс п'яте, яке AWS дуже любить:

1. **Чи це безпечно?** Хто має доступ, чи зашифровані дані, чи не відкрито зайвого в інтернет?
2. **Чи це переживе збій?** Що буде, якщо впаде сервер, дата-центр, цілий регіон?
3. **Чи це швидко під навантаженням?** Що буде, якщо користувачів стане вдесятеро більше?
4. **Чи це недорого?** Чи не платимо за простій, чи обрано правильний тариф?

5. **Скільки ручної роботи це потребує?** Хто оновлюватиме, лататиме, стежитиме вночі?

AWS зібрав ці питання в **Well-Architected Framework** — шість «стовпів» якісної архітектури. Детально розберемо їх у розділі 7, а зараз запам'ятай, що іспит перевіряє саме це вміння: **зважити всі п'ять питань одночасно**.



Приклад мислення архітектора

Хмаринці треба зберігати фото товарів. Варіанти: диск сервера, спільна файлова система, об'єктне сховище S3.

- Диск сервера: якщо сервер помре — фото зникнуть (надійність **X**), і його не поділиш між кількома серверами (масштаб **X**).
- Файлова система: працює, але дорожча за гігабайт і потребує мережевих налаштувань.
- **S3**: фото зберігаються в кількох дата-центрах (надійність ✓), доступні будь-якій кількості серверів і користувачів (масштаб ✓), дешеві (вартість ✓), без адміністрування (операції ✓), доступ налаштовується політиками (безпека ✓).

Відповідь: S3 — бо тільки він закриває всі п'ять питань. Саме так ти розв'язуватимеш питання іспиту.

Як улаштований цей курс

Курс — це підручник з нуля. Ніяких знань ІТ не потрібно: перші розділи пояснюють, що таке сервер, IP-адреса, порт, база даних і шифрування. Далі — 11 модулів, які ведуть до рівня архітектора:

МОДУЛЬ 0

 Старт з нуля: основи ІТ, хмара, інфраструктура AWS, акаунт і бюджет



МОДУЛІ 1–2

 Безпека і доступ ·  Мережа: VPC, DNS, CDN



МОДУЛІ 3–5

 Обчислення ·  Сховища ·  Інтеграція: черги й події



МОДУЛІ 6–8

 Переїзд у хмару ·  Бази даних ·  Аналітика й AI



МОДУЛІ 9–10

 Моніторинг, надійність, гроші ·  Велика архітектура, розбір іспиту, план

Кожен модуль спирається на попередні. Якщо поспішаєш на іспит — усе одно не пропускай модулі 1–5: це половина питань.

Кожен розділ побудований однаково — так мозку легше:

-  **Історія Хмаринки** — яку проблему розв'язуємо і навіщо.
-  **Образ** — картинка з життя, яку легко згадати на іспиті.
- **Пояснення** — від простого до складного, з таблицями і схемами.
-  **Глибше** — розгортні блоки для допитливих (як це працює всередині).
-  **Спробуй у справжньому AWS** — покрокова практика в консолі, з вартістю і обов'язковим прибиранням.
-  **Запам'ятай** — коротке резюме.
-  **Як питають на іспиті** — тригери «бачиш у питанні → обираєш».
-  **Пастки** — де найчастіше помиляються.
-  **Перевір себе** — питання у стилі іспиту з поясненням кожної відповіді.

Інструменти сайту:

-  **Підказки** — торкнись підкресленого слова (або виділи будь-яке слово пальцем) — з'явиться картка: що це, образ, коли обирати, як питають на іспиті, з чим не плутати.

-  **Практика** — усі питання курсу в одному місці, **картки-тригери** для швидкого повторення і **пробний іспит** англійською з таймером.
-  **Словник** — сервіси AWS і терміни з основ IT.
-  **Схеми** — усі схеми курсу для повторення.
-  **PDF** — увесь підручник одним файлом; сайт можна встановити на телефон як застосунок, і він працюватиме без інтернету.

Як вчитися, щоб запам'ятати надовго

Дослідження пам'яті дають три прості правила, на яких побудований курс:

1. **Активне пригадування.** Перечитування створює ілюзію знання. Справжнє запам'ятовування — коли ти **витягаєш** відповідь з пам'яті: питання «Перевір себе», картки-тригери, пробний іспит. Спершу відповідай подумки, потім перевіряй.
2. **Інтервальні повторення.** Повертайся до теми через 1 день, 3 дні, тиждень. Картки самі частіше показують те, що ти «не знав».
3. **Практика руками.** Те, що клацнув сам, запам'ятовується в рази краще. Роби хоча б частину практик «Спробуй у справжньому AWS».

Скільки часу потрібно:

Темп	Скільки на день	Скільки тижнів
Спокійний	45–60 хвилин	8–10 тижнів
Звичайний	1–1,5 години	5–7 тижнів
Інтенсивний	2–3 години	3–4 тижні

Орієнтир готовності: **стабільно 80%+** на пробному іспиті й у «Перевір себе». Детальний план підготовки — у розділі 46.

Хмаринка — наш наскрізний приклад

Усі розділи розповідають одну історію. Ось із чого починаємо:

- **Хто:** інтернет-магазин подарунків і товарів для дому, Львів. Засновниця — Олена. Розробник — Тарас. Є бухгалтерія і склад.
- **Клієнти:** Україна, а незабаром — Польща і Німеччина.
- **Як усе працює зараз:** один сервер у підсобці: на ньому сайт, база даних замовлень і фото товарів. Бекап — раз на тиждень на зовнішній диск (якщо не забудуть). Склад працює в старій програмі на окремому комп'ютері.
- **Болі:** сайт падає під час розпродажів; якщо згорить сервер — зникне все; нові фото вантажаться повільно; ніхто не знає, хто і коли змінював налаштування; рахунки за обладнання непередбачувані.

До кінця курсу ми збудуємо для Хмаринки архітектуру, яка витримає будь-який розпродаж, переживе аварію цілого дата-центру, захистить дані клієнтів і коштуватиме розумних грошей. Кожен розділ додає до неї одну цеглинку, а в розділі 44 ти побачиш усю будівлю.

Зараз

Один сервер у підсобці: сайт + база + фото. Бекап на флешку. Падає під навантаженням.



Цеглинки

IAM, KMS, WAF

VPC, CloudFront

EC2, Auto Scaling, Lambda

S3

Aurora, DynamoDB

SQS, SNS



Після курсу

Сайт у кількох зонах, масштабується сам, дані зашифровані й продубльовані в іншому регіоні, витрати під контролем.

Не лякайся назв — кожен пояснимо з нуля. Підказка  працює вже зараз: торкнись будь-якої підкресленої назви.

Запам'ятай

Архітектор рішень **обирає і поєднує** сервіси під вимоги: безпека, надійність, швидкість, вартість, мінімум ручної роботи.

SAA-C03: **65 питань, 130 хвилин, 720 з 1000**, штрафу за помилку немає, \$150, діє 3 роки.

Домени: **безпека 30%, відмовостійкість 26%, продуктивність 24%, вартість 20%**.

Правильна відповідь виконує **всі** вимоги з питання одночасно.

Вчимося через **пригадування, повторення і практику**, а не перечитування.

Як питають на іспиті

«...with the LEAST operational overhead» → **керований або serverless-сервіс замість власних серверів і скриптів**

«...the MOST cost-effective solution» → **найдешевше з рішень, що виконують усі вимоги**

Варіант «напишемо скрипт на сервері й запускатимемо за розкладом» → **майже завжди програє керованому сервісу**

«...highly available» у питанні → **розгортання щонайменше у двох зонах доступності**

Пастки

«Найдешевше» ≠ «найдешевше взагалі»: дешевий варіант без потрібної надійності — неправильний.

Не обирай варіант лише тому, що він «найпотужніший» чи «найсучасніший»: зайва складність і ціна — теж помилка.

Порожня відповідь — гарантована помилка. Навіть не знаючи, відкинь явно неправильне і обери з решти.

✓ Перевір себе

ПИТАННЯ 1

Скільки питань на іспиті SAA-C03 впливають на твій бал?

- A** 50 з 65 — ще 15 неоцінювані, але невідомо які
- B** 60 з 65
- C** 40 з 65
- D** Усі 65

Правильна відповідь: A. На іспиті 65 питань: 50 оцінюються, 15 — пробні для майбутніх версій. Оскільки ти не знаєш, які саме пробні, відповідай на кожне якнайкраще.

ПИТАННЯ 2

У питанні сказано: «Яке рішення відповідає вимогам з LEAST operational overhead?» Що це означає?

- A** Рішення з найменшим обсягом ручної роботи з підтримки — зазвичай керований або serverless-сервіс
- B** Рішення з найменшою кількістю серверів у регіоні
- C** Найшвидше рішення
- D** Найдешевше рішення

Правильна відповідь: A. Operational overhead — це робота з обслуговування: оновлення, латання, моніторинг, масштабування вручну. Мінімум такої роботи дають керовані сервіси (AWS робить рутину за тебе) і serverless.

ПИТАННЯ 3

Ти не впевнений у відповіді, а час спливає. Що найкраще зробити на іспиті SAA-C03?

- A** Залишити питання порожнім, щоб не втратити бали
- B** Обрати найдовший варіант
- C** Відкинути явно неправильні варіанти, обрати найкращий із решти і позначити питання, щоб повернутися
- D** Завершити іспит достроково

Правильна відповідь: C. Штрафу за неправильну відповідь немає, тож порожнє питання — гарантований нуль. Відкидай неможливе, обирай найкраще з решти і використовуй позначку «Flag for review».

ПИТАННЯ 4

Який домен має найбільшу вагу на іспиті SAA-C03?

- A** Design Secure Architectures — 30%
- B** Design Cost-Optimized Architectures — 30%
- C** Design Resilient Architectures — 30%
- D** Design High-Performing Architectures — 30%

Правильна відповідь: A. Безпека — 30%, відмовостійкість — 26%, продуктивність — 24%, вартість — 20%. Тому модуль про безпеку в цьому курсі — один із найбільших.

ПИТАННЯ 5

Англійська для тебе не рідна. Коли потрібно подати запит на додаткові 30 хвилин (ESL +30)?

- A** Під час іспиту, натиснувши кнопку Help
- B** Надіслати лист у підтримку Pearson VUE в день іспиту
- C** Заздалегідь, у кабінеті сертифікації, до запису на іспит
- D** Після іспиту, щоб перерахували бал

Правильна відповідь: C. Запит ESL +30 подають один раз у кабінеті AWS Certification (Request Exam Accommodations) і обов'язково до запису на іспит. Якщо вже записався — скасуй запис, подай запит і запишись знову.



Основи ІТ, частина 1: комп'ютер, сервер, операційна система і віртуалізація



Відмовостійкість · 26%



Продуктивність · 24%



Історія Хмаринки. Наступного ранку після аварії Тарас відкриває кришку сервера з підсобки: пил, гарячий радіатор, лампочка диска блимає без упину. «Процесор був завантажений на сто відсотків, пам'ять закінчилась, диск не встигав», — пояснює він. Олена чесно зізнається: «Я не розумію жодного слова. Що таке сервер? Чим він відрізняється від мого ноутбука? І чому хмара — це не просто "чужий сервер"?» Щоб проєктувати в AWS, треба розуміти, з чого зроблений будь-який комп'ютер. Почнемо з цього.



Образ. Комп'ютер — це ресторанна кухня. **Процесор** — кухар: чим більше кухарів і чим вони швидші, тим більше страв за хвилину. **Оперативна пам'ять** — робочий стіл: усе під рукою, але стіл невеликий і ввечері його прибирають (після вимкнення пам'ять очищується). **Диск** — комора і холодильник: місця багато, продукти зберігаються довго, але по них треба сходити. **Мережева карта** — вікно видачі, через яке приходять замовлення і виходять страви. **Операційна система** — шеф-менеджер, що розподіляє роботу між кухарями, столом і коморою.

Із чого складається будь-який комп'ютер

Ноутбук, телефон, сервер у дата-центрі AWS — усі вони складаються з тих самих чотирьох частин. Від них залежить, що і як швидко комп'ютер може робити, і саме їх ти обиратимеш, коли замовлятимеш сервер в AWS.



Процесор (CPU)

Виконує обчислення. Має кілька ядер (cores) — кожне працює паралельно.
В AWS: vCPU — «віртуальні ядра» сервера



Оперативна пам'ять (RAM)

Дуже швидка, але тимчасова: зникає при вимкненні.
В AWS: GiB пам'яті в типі сервера



Диск (storage)

Зберігає дані надовго: ОС, програми, файли, базу даних.
В AWS: EBS-диски, instance store, S3



Мережа (NIC)

Обмін даними з іншими комп'ютерами.
Швидкість — у бітах за секунду.
В AWS: мережева пропускна здатність типу сервера (Gbps)

Коли в AWS обираєш тип сервера (наприклад, m7g.large), ти фактично обираєш комбінацію цих чотирьох ресурсів.

Процесор (CPU)

Процесор виконує інструкції програм: додає числа, порівнює, переміщує дані. Головні характеристики:

- **Ядра (cores).** Кожне ядро — окремий «кухар». 8 ядер можуть одночасно виконувати 8 задач. Вебсервер, що обслуговує тисячі відвідувачів, любить багато ядер.
- **Частота (GHz).** Скільки мільярдів тактів за секунду робить ядро. Вища частота — швидше виконується одна задача.
- **Архітектура.** Більшість серверів світу — **x86** (процесори Intel і AMD). Друга велика родина — **ARM**: так працюють телефони, ноутбуки Apple і власні процесори AWS **Graviton**. ARM-процесори зазвичай дають більше обчислень за ті самі гроші й електрику. Але програма має бути зібрана під потрібну архітектуру (для більшості сучасних мов це не проблема).



Глибше: що таке vCPU в AWS

—

В AWS потужність сервера вимірюють у **vCPU**. На процесорах Intel і AMD одне фізичне ядро вміє виконувати два потоки одночасно (технологія hyper-threading / SMT), і кожен потік AWS рахує як 1 vCPU. Тобто 4 vCPU на Intel — це 2 фізичні ядра по 2 потоки. На **Graviton** такої технології немає: 1 vCPU = 1 справжнє фізичне ядро. Тому Graviton часто показує кращу продуктивність «на один vCPU». Для іспиту достатньо знати: Graviton — ARM-процесори AWS з найкращим співвідношенням ціни й продуктивності; в назві типу сервера їх позначає літера **g** (m7**g**, c7**g**, r8**g**).

Оперативна пам'ять (RAM)

Пам'ять — це робочий простір, де лежать дані, з якими програма працює **зараз**: відкриті сторінки сайту, кошики покупців, частина бази даних. Вона в тисячі разів швидша за диск, але:

- **енергозалежна** (volatile): вимкнули живлення — вміст зник;
- **обмежена**: зазвичай від кількох гігабайтів до кількох терабайтів на сервер.

Коли пам'яті не вистачає, ОС починає тимчасово скидати її вміст на диск (swap) — і все різко гальмує. Саме це сталося з сервером Хмаринки в «чорну п'ятницю».

Програми, яким потрібно багато пам'яті (бази даних «в пам'яті», кеші, аналітика великих наборів даних), в AWS запускають на **memory optimized** серверах — сімейства R, X, z.

Диск (сховище)

Диск зберігає все, що має пережити вимкнення: операційну систему, програми, файли, базу даних. Типи дисків:

Тип	Як працює	Швидкість	Для чого
HDD (жорсткий диск)	Магнітні пластини обертаються, голівка шукає дані	Повільний для дрібних операцій, непоганий для великих послідовних файлів	Архіви, логи, великі файли, дешеве зберігання
SSD	Флеш-пам'ять, без рухомих частин	Швидкий для будь-яких операцій	Операційна система, бази даних, більшість застосунків
NVMe SSD	SSD, підключений напям до шини процесора	Найшвидший	Високонавантажені бази, кеші, аналітика

Швидкість диска описують трьома числами, які ти побачиш у кожному питанні про сховища:

- **IOPS** (input/output operations per second) — скільки **окремих** операцій читання чи запису диск робить за секунду. Важливо для баз даних, які читають і пишуть маленькі шматочки.
- **Пропускна здатність** (throughput, МБ/с) — скільки **даних** проходить за секунду. Важливо для великих файлів: відео, логів, аналітики.
- **Затримка** (latency, мс) — скільки чекати відповіді на одну операцію.

Як не плутати IOPS і throughput

Уяви комірника. **IOPS** — скільки **коробок** за секунду він встигає видати (багато маленьких замовлень — база даних). **Throughput** — скільки **кілограмів** за секунду він перевозить (кілька величезних палет — відео, бекапи, big data). Для бази даних важливіші IOPS, для потоку великих файлів — throughput.

Одиниці виміру: біти, байти і «чесні» гігабайти

- **1 байт = 8 біт**. Обсяг даних рахують у **байтах** (MB, GB, TB), а швидкість мережі — у **бітах** за секунду (Mbps, Gbps). Мережа 1 Gbps передає максимум ~125 МБ за секунду (ділимо на 8).

- **GB і GiB — не одне й те саме.** GB (гігабайт) = 1 000 000 000 байт. GiB (гібібайт) = 1 073 741 824 байт (2^{30}), приблизно на 7% більше. AWS рахує диски й пам'ять у **GiB і TiB**, а трафік — у GB. На іспиті різниця майже ніколи не важлива, але в документації не лякайся.

Що таке сервер

Сервер — це комп'ютер, який **обслуговує** (serves) запити інших комп'ютерів. Коли покупець відкриває сайт Хмаринки, його браузер — **клієнт** — надсилає запит, а сервер повертає сторінку. Ця модель називається **клієнт-сервер**, і на ній побудований майже весь інтернет. Технічно сервером може бути будь-який комп'ютер, навіть старий ноутбук. Але справжні сервери відрізняються:

- працюють **24/7** роками, без монітора й клавіатури — ними керують через мережу;
- мають **резервовані частини**: два блоки живлення, пам'ять з виправленням помилок (ECC), кілька дисків;
- стоять у **дата-центрах** — будівлях з потужним живленням, генераторами, охолодженням, охороною і кількома каналами інтернету;
- монтуються в **стійки** (racks) по 20–40 штук.

За роллю сервери бувають різні: **вебсервер** (віддає сторінки), **сервер застосунку** (виконує бізнес-логіку: кошик, оплата), **сервер бази даних**, **файловий сервер**, **поштовий** і так далі. У старій Хмаринці всі ролі виконував один комп'ютер — і коли він упав, упало все.



Що таке «продакшн»

Production (prod) — середовище, з яким працюють справжні клієнти.
Dev / test / staging — середовища для розробки й перевірки. Золоте

правило: ніколи не експериментуй у prod. В AWS різні середовища часто розносять навіть по різних акаунтах — про це в розділі 10.

Операційна система

Операційна система (ОС) — головна програма комп'ютера. Вона керує процесором, пам'яттю, дисками й мережею і дає іншим програмам зручний спосіб ними користуватися. Без ОС кожна програма мала б сама «говорити» із залізом.

Що робить ОС:

- **Процеси.** Кожна запущена програма — процес. ОС ділить між ними час процесора і пам'ять.
- **Файли.** Організовує дані на диску в папки і файли, стежить за правами доступу.
- **Користувачі і права.** Хто може читати, змінювати, запускати. Найсильніший користувач — адміністратор (у Linux — `root`).
- **Мережа.** Приймає і надсилає дані, відкриває **порти** для програм (про порти — у наступному розділі).
- **Драйвери.** Маленькі програми, що вміють працювати з конкретним залізом.

Серверні ОС бувають двох великих родин:

	Linux	Windows Server
Ліцензія	Переважно безкоштовний (open source)	Платна ліцензія (в AWS входить у ціну сервера)
Де поширений	Більшість вебсерверів, контейнерів, хмарних сервісів	Корпоративні застосунки Microsoft: .NET Framework, SQL Server, Active Directory, SharePoint
Як керувати віддалено	SSH , порт 22 — текстова командна строка	RDP , порт 3389 — віддалений робочий стіл
Приклади в AWS	Amazon Linux 2023, Ubuntu, Red Hat, SUSE, Debian	Windows Server 2019, 2022, 2025

Оновлення і патчі. В ОС постійно знаходять вразливості, і виробники випускають виправлення — **патчі**. Сервер без патчів — відчинені двері для злоумисників. Хто латає ОС у хмарі — ти чи AWS — залежить від сервісу. Це одна з найчастіших тем іспиту, розберемо її в розділі 5.



Як виглядає керування Linux-сервером

Адміністратор підключається по SSH і вводить команди:

```
ssh ec2-user@203.0.113.10      # підключитися до сервера
top                             # подивитися, які процеси їдять CPU
df -h                          # скільки місця на дисках
sudo dnf update -y              # встановити оновлення (Amazon Linux)
sudo systemctl restart nginx   # перезапустити вебсервер
```

Запам'ятовувати команди не треба — іспит їх не перевіряє. Але корисно розуміти: сервером керують через мережу, і ці доступи треба захищати.

Масштабування: вертикальне і горизонтальне

Що робити, коли сервер не справляється? Є два шляхи — і це одна з найважливіших ідей усієї архітектури.

↑ Вертикальне (scale up)

Замінити сервер на потужніший: більше CPU і пам'яті.

✓ Просто, не треба змінювати застосунок.

✗ Є стеля — найбільший сервер. Потрібна зупинка для заміни. Усе ще одна точка відмови.

→ Горизонтальне (scale out)

Додати ще кілька однакових серверів і розподіляти між ними запити балансувальником.

✓ Майже безмежно. Без зупинки. Відмова одного сервера не зупиняє сайт.

✗ Застосунок має бути «stateless» — не тримати дані користувача на конкретному сервері.

Хмара створена для горизонтального масштабування: сервери додаються й прибираються автоматично (Auto Scaling, розділ 23).

Звідки обмеження «stateless»? Уяви, що покупець поклав товар у кошик на сервері №1, а наступний його запит потрапив на сервер №2 — кошик зник. Тому в горизонтальній архітектурі **стан** (сесії, кошики) зберігають не на самих серверах, а в спільному місці: базі даних чи кеші. Тоді будь-який сервер може обслужити будь-якого покупця, і сервери можна вільно додавати й вимикати.

Надійність: все колись ламається

Диски зношуються, блоки живлення горять, пам'ять дає збої, у дата-центрі може зникнути світло. Архітектор виходить з того, що **відмова — це норма**, і проєктує систему, яка її переживе.

Головне поняття — **єдина точка відмови** (single point of failure, SPOF): компонент, поломка якого зупиняє всю систему. Сервер у підсобці Хмаринки був однією великою SPOF. Способи прибрати SPOF:

- **Резервування (redundancy)** — дублювати компоненти: два сервери замість одного, два блоки живлення, дзеркальні диски (RAID 1).
- **Рознесення** — ставити дублікати в різних місцях, щоб одна пожежа чи аварія живлення не вбила обидва. В AWS це кілька **зон доступності** (розділ 6).
- **Автоматичне перемикання (failover)** — коли основний компонент падає, резервний автоматично бере роботу на себе.

- **Бекапи** — копії даних, з яких можна відновитися після помилки чи атаки.

Доступність (availability) вимірюють у «дев'ятках» — частці часу, коли система працює:

Доступність	Простій за рік	Простій за місяць
99% («дві дев'ятки»)	~3,65 доби	~7,3 години
99,9% («три дев'ятки»)	~8,8 години	~44 хвилини
99,99% («чотири дев'ятки»)	~53 хвилини	~4,4 хвилини
99,999% («п'ять дев'яток»)	~5,3 хвилини	~26 секунд

Кожна додаткова дев'ятка коштує дорожче: більше дублікатів, складніша автоматика. Тому архітектор завжди питає: **скільки простою бізнес може собі дозволити?**

Доступність ≠ збереження даних

Availability — чи працює система **зараз** («двері відчинені»). **Durability** — чи **не загубляться** дані з часом («речі на складі цілі»). Сховище може бути тимчасово недоступним, але дані в ньому збережуться. S3 обіцяє durability 99,999999999% («11 дев'яток») — шанс втратити файл мізерний, хоча доступність у S3 нижча (99,99% для S3 Standard).

Віртуалізація: як з одного сервера зробити десять

Фізичний сервер у дата-центрі потужний: десятки ядер, сотні гігабайтів пам'яті. Одному застосунку стільки рідко потрібно. **Віртуалізація** дозволяє розділити один фізичний сервер на багато **віртуальних машин** (VM, virtual machines). Кожна VM думає, що вона — окремий комп'ютер зі своїм процесором, пам'яттю, диском і ОС.

Цим керує спеціальна програма — **гіпервізор** (hypervisor). Він:

- ділить ресурси фізичного сервера (хоста) між віртуальними машинами;

- **ізолює** їх одна від одної: VM одного клієнта не бачить пам'ять і диски іншої;
- дозволяє створювати, зупиняти й переносити VM за секунди — без походу в дата-центр.

Саме віртуалізація робить хмару можливою. Коли ти «запускаєш сервер» в AWS, ти отримуєш віртуальну машину на одному з мільйонів фізичних серверів AWS. У AWS вона називається **EC2 instance** (інстанс).



Глибше: AWS Nitro — гіпервізор, розібраний на мікросхеми

—

Класичний гіпервізор забирає частину ресурсів хоста на себе: він обробляє мережу, диски, безпеку. AWS пішов далі й створив **Nitro System**: мережеву роботу, доступ до дисків EBS і шифрування виконують **окремі спеціалізовані плати Nitro**, а сам гіпервізор став дуже легким. Результат: віртуальні машини отримують майже всю потужність заліза, продуктивність стабільніша, а безпека сильніша — навіть працівники AWS не мають інтерактивного доступу до пам'яті твого сервера. Nitro також дає **bare metal** інстанси — коли ти отримуєш увесь фізичний сервер без гіпервізора (для спеціального ПЗ чи ліцензій). На іспиті Nitro іноді згадують як вимогу: наприклад, «EBS Multi-Attach працює лише на Nitro-інстансах».

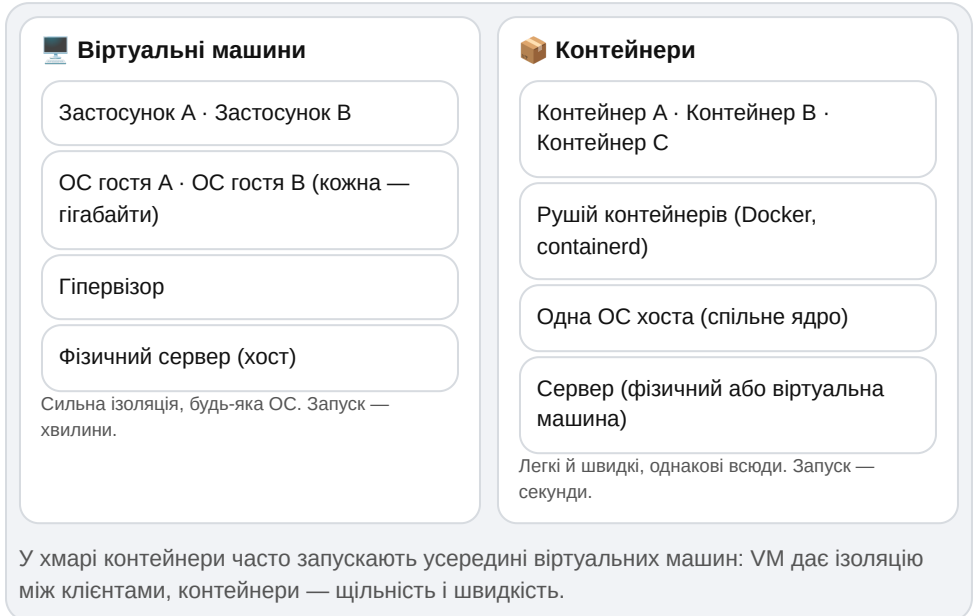
Два типи гіпервізорів. Тип 1 («bare-metal») працює прямо на залізі — так влаштовані дата-центри й хмари (KVM, VMware ESXi, Microsoft Hyper-V, Nitro). Тип 2 працює як звичайна програма всередині ОС (VirtualBox, VMware Workstation) — для експериментів на ноутбуці. Багато компаній мають власні дата-центри на **VMware** — і для них AWS пропонує спеціальні шляхи переїзду (про це — у розділі 33).

Контейнери: ще легше за віртуальні машини

Віртуальна машина містить **цілу ОС** — це гігабайти і хвилини на запуск. **Контейнер** пакує лише **застосунок і його залежності** (бібліотеки,

налаштування), а ядро ОС використовує спільне з хостом. Тому контейнер:

- важить мегабайти, а не гігабайти;
- стартує за секунди;
- працює **однаково всюди** — на ноутбучі розробника, на тестовому сервері й у хмарі («у мене працює» більше не аргумент).



Найвідоміший інструмент — **Docker**: розробник описує контейнер у файлі, збирає **образ** (image) і запускає його будь-де. Коли контейнерів сотні, ними керує **оркестратор** — наприклад, Kubernetes. В AWS для цього є ECS, EKS і Fargate (розділ 24).

І останній крок абстракції — **serverless** («без серверів»): ти віддаєш хмарі лише **код функції**, а коли і на чому її запустити, скільки копій створити і як оновлювати сервери — вирішує провайдер. Платиш тільки за час виконання. В AWS це Lambda (розділ 25).



Фізичний сервер

Купуєш, ставиш, ремонтуєш сам

Максимум контролю й турбот



Віртуальна машина

EC2: ОС і все вище — твоє

Залізо — турбота AWS



Контейнер

ECS / EKS / Fargate: тільки застосунок в образі

З Fargate — навіть без серверів



Функція

Lambda: лише код

Мінімум турбот, платиш за виклики

На іспиті «LEAST operational overhead» майже завжди тягне відповідь праворуч по цих сходах — якщо вона виконує решту вимог.



Спробуй: подивись на свій комп'ютер очима архітектора

Вартість: безкоштовно, AWS не потрібен. **Час:** 5 хвилин.

1. **Windows:** натисни Ctrl + Shift + Esc → вкладка «Продуктивність» (Performance). **macOS:** «Моніторинг системи» (Activity Monitor). **Телефон:** Налаштування → Про телефон.
2. Знайди: скільки **ядер і логічних процесорів** має CPU (логічних часто вдвічі більше — це hyper-threading, тобто «vCPU» за мірками AWS).
3. Подивись, скільки **пам'яті** зайнято зараз, і відкрий кілька важких вкладок браузера — спостерігай, як росте використання RAM.

4. Відкрий графік **диска** і скопіюй великий файл — побачиш пропускну здатність у МБ/с.
5. Подумай: якби твій комп'ютер був сервером Хмаринки, що закінчилося б першим під час розпродажу — CPU, пам'ять, диск чи мережа?

Прибирати нічого не треба.

Запам'ятай

Комп'ютер = **CPU** (обчислення) + **RAM** (швидка тимчасова пам'ять) + **диск** (довге зберігання) + **мережа**.

IOPS — кількість операцій диска за секунду (бази даних); **throughput** — обсяг даних за секунду (великі файли).

Сервер — комп'ютер, що обслуговує запити клієнтів; працює 24/7 у дата-центрі.

Вертикальне масштабування — більший сервер (має стелю і простій); **горизонтальне** — більше серверів (потрібен stateless-застосунок).

Єдина точка відмови — те, що треба прибрати резервуванням і рознесенням.

Віртуалізація ділить фізичний сервер на VM; **контейнер** — легший за VM, бо ділить ядро ОС; **serverless** — лише код.

Graviton (літера **g** у назві типу) — ARM-процесори AWS з кращою ціною за продуктивність.

Як питають на іспиті

Застосунок потребує великого обсягу оперативної пам'яті (in-memory база, кеш, аналітика) → **memory optimized інстанси (R, X)**

Найкраще співвідношення ціни й продуктивності для застосунку, сумісного з ARM → **інстанси на Graviton**

Сервери мають вільно додаватися й прибиратися, але сесії користувачів не повинні губитися → **stateless-сервери, сесії — у спільному сховищі (ElastiCache або DynamoDB)**

Система має витримати відмову одного сервера без простою → **кілька серверів за балансувальником (горизонтальне масштабування)**

Застосунок має однаково запускатися на ноутбуці й у хмарі, швидко стартувати → **контейнери**

Запускати код без керування серверами, платити лише за виконання → **serverless (AWS Lambda)**

Пастки

«Просто візьмемо більший сервер» — вертикальне масштабування не прибирає єдину точку відмови і має стелю.

Availability ≠ durability: «дані не загубляться» і «сервіс працює зараз» — різні вимоги.

Мережа рахується в **бітах**, дані — в **байтах**: 1 Gbps $\approx$ 125 МБ/с, а не 1 ГБ/с.

Контейнер — не віртуальна машина: у нього немає власного ядра ОС.

✓ Перевір себе

ПИТАННЯ 1

Сайт Хмаринки під час розпродажу гальмує, бо один сервер не справляється. Олена хоче рішення, яке не має «стелі» і переживе відмову сервера. Що обрати?

- A** Додати серверу більше оперативної пам'яті
- B** Запустити кілька однакових серверів за балансувальником і додавати нові під час піку (горизонтальне масштабування)
- C** Перенести базу даних на той самий сервер, щоб зменшити затримку
- D** Замінити сервер на найпотужніший доступний (вертикальне масштабування)

Правильна відповідь: B. Горизонтальне масштабування не має жорсткої стелі й прибирає єдину точку відмови: якщо один сервер впаде, інші продовжать працювати. Вертикальне масштабування упирається в найбільший доступний сервер, потребує простою, а сервер лишається один.

ПИТАННЯ 2

База даних робить десятки тисяч дрібних операцій читання й запису щосекунди. Яка характеристика диска для неї найважливіша?

- A** IOPS — кількість операцій за секунду
- B** Пропускна здатність у МБ/с для великих файлів
- C** Обсяг диска в терабайтах
- D** Частота процесора

Правильна відповідь: А. Бази даних читають і пишуть маленькі блоки, тому їх обмежує кількість операцій — IOPS. Пропускна здатність (throughput) важливіша для великих послідовних файлів: відео, логів, аналітики.

ПИТАННЯ 3

Чому для горизонтального масштабування застосунок має бути stateless?

- A** Бо stateless-сервери дешевші за годину роботи
- B** Щоб сервери не мали доступу до бази даних
- C** Щоб будь-який сервер міг обслужити будь-який запит: дані сесій зберігаються не на сервері, а в спільному сховищі
- D** Бо stateless-сервери не потребують операційної системи

Правильна відповідь: С. Якщо кошик покупця лежить у пам'яті сервера №1, а наступний запит потрапить на сервер №2, кошик «зникне». Коли стан винесено в спільне сховище (кеш або базу), сервери стають взаємозамінними — їх можна додавати й вимикати.

ПИТАННЯ 4

Чим контейнер відрізняється від віртуальної машини?

- A** Контейнер використовує спільне ядро ОС хоста, тому він легший і стартує за секунди
- B** Контейнер має власне ядро ОС і тому ізольованіший
- C** Контейнер може працювати лише на фізичному сервері, без віртуалізації
- D** Контейнер завжди дорожчий за віртуальну машину

Правильна відповідь: A. Віртуальна машина містить повну ОС (гігабайти, хвилини на старт). Контейнер пакує лише застосунок і залежності, а ядро ділить з ОС хоста — тому він легкий, швидкий і однаково запускається всюди.

ПИТАННЯ 5

Компанії потрібна доступність 99,99%. Скільки приблизно простою на рік це дозволяє?

- A** Близько 3,6 доби
- B** Близько 53 хвилин
- C** Близько 8,8 години
- D** Близько 5 хвилин

Правильна відповідь: B. 99,99% («чотири дев'ятки») — це приблизно 52–53 хвилини простою на рік. 99,9% — близько 8,8 години, 99,999% — близько 5 хвилин.

ПИТАННЯ 6

Мережевий канал має швидкість 1 Gbps. Скільки приблизно даних ним можна передати за секунду в ідеальних умовах?

A 8 ГБ

B Близько 1 МБ

C Близько 125 МБ

D 1 ГБ

Правильна відповідь: C. Швидкість мережі вимірюють у бітах, а дані — в байтах. 1 байт = 8 біт, тому $1 \text{ Gbps} = 1000 \text{ Мбіт/с} \div 8 \approx 125 \text{ МБ/с}$.



Основи ІТ, частина 2: мережі, IP-адреси, порти, DNS і HTTPS

🎯 Безпека · 30%

🎯 Продуктивність · 24%



Історія Хмаринки. Покупниця з Варшави пише в підтримку: «Сайт відкривається по 5 секунд, а фото — ще довше». Тарас зітхає: «Це мережа». Олена не здається: «А що саме відбувається між її браузером і нашим сервером? Де губляться ці секунди?» Щоб відповісти, треба пройти шлях одного кліку від телефона покупниці до сервера — і назад. Вся мережа AWS (VPC, DNS, балансувальники, CDN) побудована на тих самих ідеях, тож цей розділ — фундамент для половини іспиту.



Образ. Інтернет — це пошта. **IP-адреса** — адреса будинку. **Порт** — номер квартири: в одному будинку (сервері) живуть різні «мешканці» — вебсервер у квартирі 443, база даних у квартирі 3306. **Протокол** — мова й правила листування. **DNS** — довідник, що перетворює назву «khmarynka.ua» на адресу будинку. **Маршрутизатор** — сортувальний центр, що вирішує, куди везти лист далі. А великі посилки пошта розбиває на **пакети** — окремі конверти, які можуть їхати різними дорогами і збираються на місці.

Мережа і пакети

Мережа — це комп'ютери, з'єднані так, щоб обмінюватися даними. Локальна мережа (LAN) — офіс Хмаринки чи твоя квартира. Інтернет —

«мережа мереж», мільйони локальних мереж, з'єднаних між собою провайдерами.

Дані в мережі передаються не суцільним потоком, а **пакетами** — невеликими порціями (зазвичай до ~1500 байт). Кожен пакет має «конверт» з адресою відправника і одержувача. Сторінка сайту — це сотні пакетів, які можуть пройти різними шляхами і збираються в правильному порядку на комп'ютері одержувача.

Хто доставляє пакети:

- **Комутатор (switch)** — з'єднує пристрої в межах однієї локальної мережі.
- **Маршрутизатор (router)** — передає пакети між мережами, обираючи наступний крок за **таблицею маршрутів** (route table). В AWS таблиці маршрутів ти налаштовуватимеш сам — у розділі 13.
- **Шлюз (gateway)** — «двері» з однієї мережі в іншу, наприклад з офісної мережі в інтернет.

IP-адреса: адреса будинку

Кожен пристрій у мережі має **IP-адресу**. Найпоширеніша версія — **IPv4**: чотири числа від 0 до 255, розділені крапками, наприклад 203.0.113.25 . Кожне число — це 8 біт, разом 32 біти, тобто близько **4,3 мільярда** можливих адрес. Для всього світу цього давно замало.

Тому IPv4-адреси поділили на два світи:

- **Публічні** — унікальні в усьому інтернеті. Їх видають провайдери, вони дефіцитні й коштують грошей (в AWS кожна публічна IPv4-адреса платна з 2024 року).
- **Приватні** — для внутрішніх мереж, у різних компаніях можуть повторюватися. Три діапазони, які варто знати напам'ять:

Діапазон	Запис CIDR	Скільки адрес	Де зустрінеш
10.0.0.0 – 10.255.255.255	10.0.0.0/8	~16,7 млн	Великі корпоративні мережі, VPC в AWS

Діапазон	Запис CIDR	Скільки адрес	Де зустрінеш
172.16.0.0 – 172.31.255.255	172.16.0.0/16	~1 млн	Default VPC в AWS (172.31.0.0/16)
192.168.0.0 – 192.168.255.255	192.168.0.0/16	~65 тис.	Домашні й офісні роутери

Як приватні пристрої виходять в інтернет? Через **NAT** (Network Address Translation): роутер підмінює приватну адресу відправника на свою публічну, а відповіді повертає потрібному пристрою. Удома всі твої телефони й ноутбуки ходять в інтернет з однієї публічної адреси — це NAT. В AWS для цього є **NAT Gateway** (розділ 13).

IPv6 — нова версія з 128-бітними адресами на кшталт `2001:db8:1234::1`. Адрес вистачить на все: кожна — публічна й унікальна, NAT не потрібен. AWS підтримує IPv6 у більшості сервісів.

CIDR: як записують діапазони адрес

Мережі в AWS описують записом **CIDR**: адреса + скісна риска + число, наприклад `10.0.0.0/16`. Число після «/» показує, скільки перших бітів адреси **фіксовані** (це «номер мережі»), а решта бітів — вільні для пристроїв.

Формула кількості адрес: $2^{(32 - \text{число})}$. Чим **менше** число після «/», тим **більша** мережа.

CIDR	Вільних біт	Адрес	Образ
/16	16	65 536	Ціле місто — типовий розмір VPC
/20	12	4 096	Район
/24	8	256	Вулиця — типова підмережа
/28	4	16	Під'їзд — найменша підмережа в AWS
/32	0	1	Одна квартира — рівно одна адреса



Розберімо приклад

`10.0.0.0/24` — перші 24 біти (`10.0.0`) фіксовані, останні 8 біт вільні. Отже, адреси від `10.0.0.0` до `10.0.0.255` — рівно 256 штук.

Мережу `10.0.0.0/16` можна поділити на 256 підмереж по `/24` :
`10.0.0.0/24` , `10.0.1.0/24` , `10.0.2.0/24` ... `10.0.255.0/24` .

Нюанс AWS: у кожній підмережі AWS забирає собі **5 адрес** (перші чотири і останню) — під маршрутизатор, DNS і службові потреби. Тому в підмережі `/24` реально доступно **251** адреса, а в `/28` — лише **11**. На іспиті це люблять питати.

Навіщо архітектору CIDR? Дві мережі з **однаковими або перетинними** діапазонами не можна напряму з'єднати: маршрутизатор не зрозуміє, в яку з них відправляти пакет. Тому діапазони для VPC, офісів і партнерів плануй заздалегідь, щоб вони не перетиналися. Це правило ще не раз врятує тебе на іспиті.

Порти і протоколи: номер квартири і мова листування

IP-адреса довозить пакет до **сервера**. Але на сервері працює багато програм. Щоб пакет потрапив до потрібної, використовують **порт** — число від 0 до 65 535. Програма «слухає» свій порт: вебсервер — 443, база MySQL — 3306.

Порти, які треба знати напам'ять (вони постійно зустрічаються в правилах файрволів AWS):

Порт	Протокол / сервіс	Коментар
22	SSH	Керування Linux-сервером (і SFTP)
3389	RDP	Віддалений робочий стіл Windows
80	HTTP	Вебсайти без шифрування
443	HTTPS	Вебсайти з шифруванням — стандарт
53	DNS	Перетворення імен на адреси

Порт	Протокол / сервіс	Коментар
3306	MySQL / MariaDB / Aurora MySQL	Бази даних
5432	PostgreSQL / Aurora PostgreSQL	Бази даних
1433	Microsoft SQL Server	Бази даних
1521	Oracle	Бази даних
6379	Redis / Valkey	Кеш у пам'яті
2049	NFS	Спільні папки Linux (Amazon EFS)
445	SMB	Спільні папки Windows (FSx for Windows)

Порти 0–1023 — «відомі» (well-known), для стандартних сервісів. А коли твій браузер відкриває з'єднання, ОС виділяє йому тимчасовий порт з великих номерів — **ephemeral port** (приблизно 1024–65535). Відповідь сервера повертається саме на цей тимчасовий порт. Запам'ятай це слово: у розділі 14 воно пояснить одну з найвідоміших пасток іспиту про мережеві ACL.

TCP і UDP

Над IP працюють два головні транспортні протоколи:

- **TCP** — надійний: встановлює з'єднання («рукоштовування» з трьох кроків), нумерує пакети, перепитує загублені, збирає в правильному порядку. Повільніший, зате нічого не губиться. Вебсайти, API, бази даних, пошта, передача файлів.
- **UDP** — швидкий: просто надсилає пакети без з'єднання і перепитувань. Якщо пакет загубився — ну й добре. Відеодзвінки, онлайн-ігри, стрімінг, голос, DNS-запити. Для реального часу краще втратити кадр, ніж чекати.

На іспиті: «ігровий сервер на UDP» або «VoIP» — сигнал, що HTTP-орієнтовані сервіси не підходять, а потрібні ті, що працюють з TCP/UDP (балансувальник NLB, Global Accelerator).

Рівні мережі: чому балансувальники бувають L4 і L7

Мережеві протоколи складаються «шарами», як матрьошка. Повна модель OSI має 7 рівнів, але архітекторів достатньо трьох:

Рівень	Що на ньому	Що «бачить» пристрій цього рівня	Приклад в AWS
L3 — мережевий	IP-адреси, маршрутизація	Звідки і куди йде пакет	Gateway Load Balancer, таблиці маршрутів
L4 — транспортний	TCP / UDP, порти	Адреси й порти, але не вміст	Network Load Balancer, Security Groups
L7 — прикладний	HTTP, HTTPS, DNS	Увесь запит: URL, заголовки, cookies	Application Load Balancer, WAF, CloudFront

Що вищий рівень, то «розумніші» рішення може ухвалювати пристрій: балансувальник L7 бачить, що запит іде на `/api/orders`, і може відправити його на окрему групу серверів; балансувальник L4 бачить лише адресу й порт, зате працює швидше.

DNS: телефонний довідник інтернету

Люди запам'ятовують назви (`khmarynka.ua`), комп'ютери — IP-адреси. **DNS** (Domain Name System) перетворює одне на інше. Це розподілена ієрархія серверів:

1. Браузер і ОС

Шукають відповідь у своєму кеші



2. Резолвер

DNS-сервер провайдера (або 8.8.8.8, 1.1.1.1). Якщо не знає — питає далі



3. Кореневі сервери

«Про .ua питай сервери зони .ua»



4. Сервери зони .ua

«Домен khmarynka.ua обслуговують такі-то name servers»



5. Авторитетний сервер

Знає точну відповідь: «khmarynka.ua = 203.0.113.25». В AWS це Route 53

Відповідь кешується на час TTL. Наступні відвідувачі від того ж резолвера отримають її миттєво.

Основні типи DNS-записів:

- **A** — ім'я → IPv4-адреса. **AAAA** — ім'я → IPv6-адреса.
- **CNAME** — ім'я → інше ім'я (псевдонім): `www.khmarynka.ua` → `khmarynka.ua`. Обмеження: CNAME не можна ставити на «голий» домен (zone apex) `khmarynka.ua`. В AWS цю проблему розв'язує спеціальний **Alias**-запис Route 53 (розділ 17).
- **MX** — поштові сервери домену. **TXT** — довільний текст (підтвердження володіння доменом, захист пошти). **NS** — які сервери відповідають за домен.

TTL (time to live) — скільки секунд резолвери можуть кешувати відповідь. Великий TTL — менше запитів і швидше, але зміни (наприклад, перемикання на резервний регіон) доходять до людей повільніше. Малий TTL — навпаки. Цей компроміс з'явиться в темі аварійного відновлення.

HTTP і HTTPS: мова вебу

HTTP — протокол «запит — відповідь». Браузер надсилає запит з **методом** і адресою, сервер повертає **код стану** й вміст.

Методи: **GET** — отримати (відкрити сторінку); **POST** — створити (оформити замовлення); **PUT/PATCH** — змінити; **DELETE** — видалити.

Коди стану, які треба впізнавати:

Код	Значення	Коли бачиш в AWS
200 OK	Усе добре	Нормальна відповідь
301 / 302	Переадресація	HTTP → HTTPS, старий URL → новий
400	Поганий запит	Помилка в запиті клієнта
401 / 403	Не автентифікований / доступ заборонено	Немає прав: bucket приватний, політика IAM забороняє
404	Не знайдено	Немає такої сторінки чи файлу
429	Забагато запитів	Спрацювало обмеження (throttling), напр. в API Gateway
500	Помилка сервера	Застосунок «впав» на запиті
502 / 504	Поганий шлюз / таймаут шлюзу	Балансувальник не отримав нормальної відповіді від сервера вчасно
503	Сервіс недоступний	Немає здорових серверів за балансувальником, перевантаження

Групи легко запам'ятати: **2xx** — успіх, **3xx** — «йди туди», **4xx** — «ти помилився» (клієнт), **5xx** — «я помилився» (сервер).

HTTPS = HTTP + шифрування **TLS**. Сервер показує браузеру **сертифікат** — цифровий документ, підписаний довіреним центром сертифікації (CA),

який доводить: «я справді khmarynka.ua». Далі браузер і сервер домовляються про ключ і шифрують увесь обмін — ніхто посередині (публічний Wi-Fi, провайдер) не прочитає паролі й номери карток. Замочок в адресному рядку — це TLS. В AWS безкоштовні сертифікати видає **ACM** (розділ 11).



Глибше: що відбувається за одну секунду після Enter

—

1. **DNS:** браузер з'ясовує IP-адресу khmarynka.ua (кеш → резолвер → ... → Route 53).
2. **TCP-рукоштовування:** SYN → SYN-ACK → ACK. Один «обмін» туди-назад по мережі.
3. **TLS-рукоштовування:** сервер надсилає сертифікат, сторони узгоджують ключ шифрування. Ще один-два обміни.
4. **HTTP-запит:** GET / з заголовками й cookies.
5. **Сервер** виконує код: читає кошик з кешу, товари з бази, формує сторінку.
6. **Відповідь** 200 OK з HTML. Браузер бачить у ньому посилання на картинки, стилі й скрипти — і для кожного робить ще запити (часто паралельно, по вже відкритому з'єднанні).

Кожен «обмін туди-назад» (round trip) коштує час, що залежить від **відстані**. Якщо сервер у Франкфурті, а покупиця у Варшаві, один round trip — ~20–30 мс; якщо сервер в Америці — понад 100 мс. Помножимо на кілька обмінів і десятки файлів — ось і п'ять секунд. Ліки: ближчий регіон, кеш на краю мережі (CDN — CloudFront), менше й легші файли.

Затримка, пропускна здатність і чому відстань важлива

- **Затримка (latency)** — час, за який пакет долітає до мети і назад (round trip time, RTT), у мілісекундах. Світло в оптоволокну проходить ~200 км за мілісекунду, тож фізику не обдуриш: 1000 км — це щонайменше ~10 мс туди-назад, а реально більше.
- **Пропускна здатність (bandwidth)** — ширина «труби»: скільки даних за секунду канал може передати (Mbps, Gbps).
- **Throughput** — скільки реально передається зараз (зазвичай менше за bandwidth).

Образ: bandwidth — кількість смуг на трасі, latency — відстань між містами. Навіть на 10-смуговій трасі до далекого міста їхати довго. Тому для глобальних користувачів ставлять копії контенту ближче до людей (**CDN**, edge-локації) і обирають регіон біля клієнтів.

Файрвол, VPN, балансувальник, проксі

- **Файрвол (firewall)** — пропускає або блокує трафік за правилами: від кого (IP), куди, на який порт, яким протоколом. Буває **stateful** — «пам'ятає» дозволені з'єднання і сам пропускає відповіді на них, і **stateless** — перевіряє кожен пакет окремо, тож дозвіл для відповіді треба прописати явно. В AWS є обидва: Security Groups (stateful) і Network ACL (stateless).
- **VPN** — зашифрований «тунель» через інтернет між двома мережами (офіс ↔ хмара) або між ноутбуком і мережею. Дані йдуть публічним інтернетом, але ніхто не може їх прочитати.
- **Балансувальник навантаження (load balancer)** — приймає запити від клієнтів і розподіляє їх між кількома серверами, перевіряючи, які з них здорові. Серце горизонтального масштабування.

- **Проксі / зворотний проксі** — посередник. Зворотний проксі стоїть перед серверами, приймає запити замість них і може кешувати, шифрувати, фільтрувати. Балансувальник і CDN — різновиди зворотного проксі.

Спробуй: розбери мережу на своєму комп'ютері

Вартість: безкоштовно, AWS не потрібен. **Час:** 10 хвилин.

1. Відкрий командний рядок (Windows: Win + R → `cmd` ; macOS/Linux: Terminal).
2. `ipconfig` (Windows) або `ifconfig / ip addr` (macOS/Linux) — знайди свою **приватну** IP-адресу (найімовірніше 192.168.x.x або 10.x.x.x). Тепер відкрий у браузері будь-який сервіс «what is my IP» — побачиш **публічну** адресу. Різниця між ними — це NAT твого роутера.
3. `nslookup aws.amazon.com` — DNS покаже IP-адресу (і, можливо, ланцюжок CNAME).
4. `ping aws.amazon.com` — побачиш затримку в мілісекундах (деякі сервери не відповідають на ping — це нормально, фаїрвол).
5. `tracert aws.amazon.com` (Windows) або `traceroute` (macOS/Linux) — маршрут через маршрутизатори провайдерів.
6. У браузері натисни F12 → вкладка **Network** і онови будь-який сайт: подивись **коди стану** (200, 301, 304...), кількість запитів і час кожного. Клацни на замочок в адресному рядку — побачиш **сертифікат TLS**.

Прибирати нічого не треба.

Запам'ятай

IP-адреса — адреса пристрою; приватні діапазони: 10.0.0.0/8 , 172.16.0.0/12 , 192.168.0.0/16 . Вихід приватних адрес в інтернет — через **NAT**.

CIDR $/n \rightarrow 2^{(32-n)}$ адрес: $/16 = 65\,536$, $/24 = 256$, $/28 = 16$. В AWS у кожній підмережі мінус **5** адрес.

Мережі з перетинними CIDR **не можна** напряму з'єднати.

Порт — номер програми на сервері: 22 SSH, 3389 RDP, 80 HTTP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL.

TCP — надійно (веб, бази); **UDP** — швидко (ігри, голос, відео, DNS).

L7 бачить HTTP (URL, заголовки) $\rightarrow$ ALB; **L4** — лише IP і порти $\rightarrow$ NLB.

DNS перетворює імена на IP; TTL — час кешування; CNAME не можна на zone apex.

HTTPS = HTTP + TLS із сертифікатом.

Затримка залежить від відстані $\rightarrow$ ставимо ресурси ближче до користувачів.

Як питають на іспиті

Скільки IP-адрес доступно для серверів у підмережі $/24$ в AWS $\rightarrow$ **251 (256 мінус 5 зарезервованих)**

Дві мережі мають однаковий CIDR $10.0.0.0/16$, їх треба з'єднати напряму $\rightarrow$ **неможливо: спершу змінити адресацію (діапазони не повинні перетинатися)**

Онлайн-гра на UDP потребує балансування $\rightarrow$ **Network Load Balancer (L4), а не Application Load Balancer**

Маршрутизувати запити за шляхом URL ($/api$, $/images$) $\rightarrow$ **балансувальник рівня 7 (Application Load Balancer)**

Дозволити доступ до вебсервера з інтернету по HTTPS $\rightarrow$ **правило на вхідний TCP-порт 443**

Сервери в приватній мережі мають виходити в інтернет за оновленнями, але бути недоступними ззовні $\rightarrow$ **NAT**

Балансувальник повертає 503 → **немає здорових серверів у групі (перевір health checks)**

Пастки

Менше число після «/» — **більша** мережа: `/16` більша за `/24`.

Приватна IP-адреса не доступна з інтернету напряму — ні вхідним, ні вихідним трафіком без NAT чи шлюзу.

CNAME не можна створити для кореневого домену (zone apex) — в AWS для цього Alias-запис.

4xx — помилка клієнта (зокрема «немає доступу» 403), 5xx — помилка сервера. Не плутай 403 з 503.

Перевір себе

ПИТАННЯ 1

Скільки IP-адрес містить діапазон `10.0.4.0/22` ?

A 4 096

B 256

C 1 024

D 512

Правильна відповідь: C. Формула: $2^{(32 - 22)} = 2^{10} = 1\,024$ адреси.

Нагадування: `/24` = 256, кожне зменшення числа на 1 подвоює мережу:
`/23` = 512, `/22` = 1 024.

ПИТАННЯ 2

Архітектор створює в AWS підмережу /28 . Скільки адрес у ній можна призначити серверам?

A 14

B 16

C 8

D 11

Правильна відповідь: D. /28 — це 16 адрес, але AWS резервує 5 у кожній підмережі (перші чотири і останню). Лишається 11. Тому для підмереж з багатьма серверами обирають щонайменше /24 .

ПИТАННЯ 3

Яка з адрес належить до приватного діапазону і не маршрутизується в інтернеті напряму?

A 52.94.1.10

B 8.8.8.8

C 172.20.5.9

D 203.0.113.25

Правильна відповідь: C. Приватні діапазони: 10.0.0.0/8, 172.16.0.0/12 (172.16.x.x – 172.31.x.x) і 192.168.0.0/16. Адреса 172.20.5.9 входить у 172.16.0.0/12.

ПИТАННЯ 4

Компанія запускає сервіс голосових дзвінків, де важлива мінімальна затримка, а втрата окремих пакетів некритична. Який транспортний протокол найпридатніший?

- A** HTTP, бо він працює через порт 80
- B** UDP, бо він не встановлює з'єднання і не чекає повторної передачі
- C** TCP, бо він гарантує доставку всіх пакетів
- D** SMTP, бо він призначений для передачі повідомлень

Правильна відповідь: B. Для голосу, відео й ігор важливіша швидкість, ніж ідеальна доставка: UDP не витрачає час на рукошукання й перепитування загублених пакетів. TCP надійніший, але повторні передачі додають затримку.

ПИТАННЯ 5

Потрібен балансувальник, який відправлятиме запити з шляхом `/api/*` на одну групу серверів, а `/images/*` — на іншу. На якому рівні він має працювати?

- A** L7 — прикладний
- B** Рівень не важливий, будь-який балансувальник це вміє
- C** L4 — транспортний
- D** L3 — мережевий

Правильна відповідь: A. Шлях URL — частина HTTP-запиту, його бачить лише пристрій рівня 7. В AWS це Application Load Balancer. Балансувальник L4 (Network Load Balancer) бачить тільки IP-адреси й порти.

ПИТАННЯ 6

Команда хоче, щоб перемикання DNS на резервний сервер доходило до користувачів якомога швидше. Який параметр DNS-запису на це впливає?

- A** Кількість NS-записів
- B** Тип запису MX
- C** Номер порту 53
- D** TTL — чим він менший, тим швидше резолвери забудуть стару відповідь

Правильна відповідь: D. TTL визначає, скільки секунд резолвери й браузері кешують відповідь. Малий TTL означає швидше поширення змін, але більше DNS-запитів.

ПИТАННЯ 7

Користувачі бачать помилку 403 при спробі відкрити файл. Що це означає найімовірніше?

- A** Сервер перевантажений і не встигає відповідати
- B** Балансувальник не знайшов здорових серверів
- C** Файл не існує
- D** Доступ до ресурсу заборонено — бракує прав

Правильна відповідь: D. 403 Forbidden — клієнтська помилка «доступ заборонено»: запит зрозумілий, але прав немає (наприклад, приватний об'єкт у S3). 404 — не знайдено, 503 — сервіс недоступний.



Основи ІТ, частина 3: дані, бази даних, API і шифрування

🔒 Безпека · 30%

🚀 Продуктивність · 24%



Історія Хмаринки. Бухгалтерка Марта веде облік замовлень в Excel-таблиці, яку щодня пересилає поштою. Фото товарів лежать у папці «Фото_НОВІ_остаточні_2», а база сайту — на тому самому сервері в підсобці. Коли Тарас каже: «Замовлення зберігаємо в реляційній базі, фото — в об'єктному сховищі, сесії — в кеші, а паролі — лише хешовані», — Олена просить перекласти це людською мовою. Цей розділ — саме такий переклад. Без нього не зрозуміти половину сервісів AWS.



Образ. Уяви три способи зберігати речі. **Блочне сховище** — порожні полиці, які ти розставляєш як хочеш (диск: розмітиш і впорядкуєш сам). **Файлове сховище** — бібліотека з каталогом, папками і полицями, куди одночасно заходять багато читачів. **Об'єктне сховище** — величезний склад: здаєш коробку, отримуєш номерок, а по номерку будь-коли її забираєш; що всередині — склад не цікавить. Бази даних — це вже не склад, а **картотека з розумним бібліотекарем**, який за секунду знайде «всі замовлення Олени за березень дорожчі за 500 гривень».

Які бувають дані

- **Структуровані** — таблиці з чіткими колонками: замовлення (номер, дата, клієнт, сума), клієнти, товари. Живуть у реляційних базах.
- **Напівструктуровані** — мають структуру, але гнучку: JSON, XML, CSV, логи. Кожен запис може мати різні поля.
- **Неструктуровані** — фото, відео, аудіо, PDF, скани. Їх не розкладеш по колонках, їх зберігають як файли чи об'єкти.

Хмаринка має всі три: замовлення (структуровані), події з сайту в JSON (напівструктуровані), фото товарів (неструктуровані). Для кожного типу — свій найкращий сервіс.

Три способи зберігати: блоки, файли, об'єкти

**Блочне (block)**

Диск, поділений на однакові блоки. ОС сама розмічає його і створює файлову систему.
Доступ: один сервер, як до свого диска. Найшвидше для баз даних і ОС.
AWS: EBS, instance store

**Файлове (file)**

Спільні папки й файли, до яких одночасно звертаються багато серверів через мережу.
Протоколи: NFS (Linux), SMB (Windows).
AWS: EFS, FSx

**Об'єктне (object)**

Об'єкт = дані + метадані + унікальний ключ.
Доступ через HTTP API, без «монтування».
Необмежений обсяг, дуже надійно, дешево.
AWS: S3

Питання-підказка: «Що буде читати ці дані?» Один сервер як диск → блочне. Багато серверів як спільну папку → файлове. Застосунки й люди через інтернет → об'єктне.

Порівняймо детальніше:

	Блочне	Файлове	Об'єктне
Як виглядає	Диск (том)	Дерево папок і файлів	Плоский простір «ключ → об'єкт»
Хто підключає	Зазвичай один сервер	Багато серверів одночасно	Будь-хто через HTTP з правами

	Блочне	Файлове	Об'єктне
Зміна частини даних	Так, будь-який блок	Так, частину файла	Ні: об'єкт замінюють цілком
Обсяг	Обмежений розміром тому	Росте до петабайтів	Практично необмежений
Типові дані	ОС, бази даних	Спільні документи, контент для кількох серверів	Фото, відео, бекапи, логи, дата-лейк
AWS	EBS, instance store	EFS (NFS), FSx (SMB, Lustre, ONTAP, OpenZFS)	S3

Протоколи файлового доступу

NFS — «мова» спільних папок у Linux. **SMB** — у Windows (мережеві диски на кшталт `\\server\share`). **iSCSI** — диск по мережі: сервер бачить віддалене сховище як свій локальний блочний диск. Ці три слова на іспиті одразу підказують сервіс: NFS для Linux у кількох зонах → Amazon EFS; SMB з Active Directory → FSx for Windows File Server.

Бази даних

Чому не зберігати замовлення просто у файлі? Бо коли тисяча покупців одночасно оформлює замовлення, файл треба блокувати, шукати в ньому повільно, а збій посеред запису зіпсує дані. **База даних (БД)** — спеціальна програма, що вміє швидко шукати, безпечно змінювати дані паралельно і не губити їх при збоях.

Реляційні бази (SQL)

Дані лежать у **таблицях** з рядками і колонками. Кожен рядок має унікальний **первинний ключ** (primary key). Таблиці пов'язані між собою **зовнішніми ключами**: у таблиці «Замовлення» є колонка «id клієнта», що вказує на рядок у таблиці «Клієнти».

Мова запитів — **SQL**:

```
SELECT c.name, o.total
FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.created_at >= '2026-03-01' AND o.total > 500;
```

Головна суперсила реляційних баз — **транзакції** з гарантіями **ACID**:

- **Atomicity** (атомарність): усі кроки транзакції виконуються разом або жоден. Списали гроші — обов'язково створили замовлення; якщо щось зламалося — відкат усього.
- **Consistency** (узгодженість): дані завжди відповідають правилам (не буде замовлення неіснуючого клієнта).
- **Isolation** (ізоляція): паралельні транзакції не заважають одна одній.
- **Durability** (довговічність): підтверджене — не загубиться навіть при збої.

Популярні рушії: **MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server**. В AWS — Amazon RDS і Amazon Aurora (розділи 34–35).

NoSQL-бази

Реляційні бази чудові, але мають межі: жорстка схема (змінити колонки у величезній таблиці складно), а масштабувати запис на мільйони операцій за секунду важко. **NoSQL** — сімейство баз, що жертвують частиною можливостей (складні JOIN) заради швидкості, масштабу або особливої моделі даних:

Тип	Як зберігає	Для чого	AWS
Key-value	Ключ → значення	Кошики, сесії, профілі, лічильники — мільйони запитів за секунду	DynamoDB
Документна	JSON-документи	Каталоги, контент із різною структурою	DocumentDB (MongoDB-сумісна)
Графова	Вузли і зв'язки	Соцмережі, рекомендації, пошук шахрайських зв'язків	Neptune
Широкі колонки	Рядки з гнучкими колонками	Величезні обсяги запису (Cassandra)	Keyspaces
Часові ряди	Значення з мітками часу	Метрики, IoT-датчики	Timestream

Тип	Як зберігає	Для чого	AWS
У пам'яті	Усе в RAM	Кеш, лідерборди, сесії — мікросекунди	ElastiCache, MemoryDB

Правило вибору просте: **спершу подивись на запити**. Потрібні складні JOIN, звіти, транзакції між багатьма таблицями → реляційна. Відомі прості запити «за ключем», гігантський масштаб, мілісекунди → key-value. Зв'язки «друзі друзів» → графова.

OLTP і OLAP: касир і аналітик

- **OLTP** (online transaction processing) — багато маленьких швидких операцій: оформити замовлення, змінити адресу. Це «касир». Реляційні бази (RDS, Aurora) і DynamoDB.
- **OLAP** (online analytical processing) — рідкісні, але важкі запити по мільйонах рядків: «виручка по регіонах за три роки». Це «аналітик». Для OLAP дані зберігають **по колонках** — запит читає лише потрібні колонки. Це **сховище даних** (data warehouse) — в AWS **Redshift**.
- **Data lake** (озеро даних) — усі сирі дані компанії в дешевому об'єктному сховищі (S3) у різних форматах, з яких потім будують аналітику. Формати на кшталт **Parquet** зберігають дані по колонках і стиснено — запити по них у рази дешевші.

Ніколи не запускай важку аналітику на основній OLTP-базі: звіт Марти не повинен гальмувати оформлення замовлень. Для цього є репліки для читання, окремі сховища даних і озера даних.

Кеш: пам'ять, що економить час

Кеш — швидке тимчасове сховище для даних, які часто читають. Каталог товарів Хмаринки змінюється раз на день, а читають його тисячі разів на хвилину — навіщо щоразу питати базу?

- **Cache hit** — відповідь знайшлася в кеші (швидко). **Cache miss** — не знайшлася, йдемо в базу і кладемо результат у кеш.

- **TTL** — скільки часу запис живе в кеші. Задовгий TTL — ризик показати застарілі дані; закороткий — менше користі.
- Кеші бувають скрізь: у браузері, на **CDN** біля користувачів (CloudFront), у пам'яті перед базою (ElastiCache, DAX).

Синхронно чи асинхронно: черги і події

- **Синхронна** взаємодія: «подзвонив — чекаєш відповіді». Сайт викликає платіжний сервіс і чекає результат.
- **Асинхронна**: «залишив записку — працюєш далі». Сайт кладе замовлення в **чергу**, а склад забирає звідти, коли встигає.

Черга розв'язує (decouples) компоненти: якщо склад на годину впав, замовлення не губляться — вони чекають у черзі. А під час розпродажу черга працює як буфер: сайт приймає тисячі замовлень, склад обробляє їх у своєму темпі. Модель **pub/sub** (публікація — підписка): одна подія «нове замовлення» розсилається всім зацікавленим — складу, бухгалтерії, email-сервісу. В AWS: SQS, SNS, EventBridge (розділи 31–32).

API: як програми розмовляють між собою

API (application programming interface) — «меню», через яке одна програма замовляє щось в іншій. Мобільний застосунок Хмаринки не лізе в базу напряму, а викликає API: `GET https://api.khmarynka.ua/orders/1042`. Сервер відповідає даними, найчастіше у форматі **JSON**:

```
{
  "orderId": 1042,
  "status": "shipped",
  "total": 780.50,
  "items": [ { "sku": "MUG-01", "qty": 2 } ],
  "customer": { "name": "Ірина", "city": "Варшава" }
}
```

- **REST API** — найпоширеніший стиль: ресурси мають адреси (URL), дії — HTTP-методи (GET, POST, PUT, DELETE).
- **Автентифікація API**: ключі API, токени (наприклад, JWT), підписані запити.
- **Throttling** (обмеження частоти) захищає API від перевантаження: «не більше 100 запитів за секунду з одного ключа» — інакше код 429.

Уся AWS — це тисячі API. Коли ти натискаєш кнопку в консолі AWS, вона викликає API. Командний рядок (AWS CLI), SDK для мов програмування, інструменти «інфраструктура як код» — усе це різні способи викликати ті самі API. Тому кожну дію в AWS можна автоматизувати і **записати в журнал** (CloudTrail, розділ 41).

Бекап і реплікація: дві різні страховки

- **Бекап (резервна копія)** — знімок даних на певний момент, збережений окремо. Захищає від **помилки і атак**: випадково видалили таблицю, вірус-шифрувальник — відновлюєшся з учорашньої копії.
- **Реплікація** — постійне копіювання змін на іншу машину. Захищає від **відмови обладнання**: основна база впала — копія вже готова.

Реплікація буває:

- **синхронною** — запис вважається успішним, лише коли його підтвердили обидві копії. Даних не втратиш, але кожен запис трохи довший (тому копії ставлять неподалік — у сусідніх дата-центрах);
- **асинхронною** — основна копія підтверджує запис одразу, а на репліку він доходить із затримкою. Швидко і можна далеко (в інший регіон), але при аварії останні секунди змін можуть загубитися.

Важливо: реплікація **копіює і помилки**. Якщо хтось видалив таблицю, репліка слухняно видалить її теж. Тому реплікація не замінює бекапи.

Два числа, з яких починається будь-яка розмова про аварії:

- **RPO** (Recovery Point Objective) — скільки даних (за часом) можна втратити. RPO = 1 година → бекап щонайменше щогодини.

- **RTO** (Recovery Time Objective) — за скільки часу система має знову запрацювати.

Детально — у розділі 42.

Шифрування і хешування

Шифрування перетворює дані на «кашу», яку можна повернути назад лише з **ключем**.

- **Симетричне** — один ключ шифрує і розшифровує (алгоритм **AES-256**). Швидке, ідеальне для великих обсягів даних. Проблема — як безпечно передати ключ.
- **Асиметричне** — пара ключів: **публічний** (можна роздавати всім) шифрує, **приватний** (тримаєш у секреті) розшифровує. Алгоритми RSA, ECC. Повільніше, але розв'язує проблему передачі ключа. Той самий принцип дає **цифровий підпис**: підписуєш приватним ключем — будь-хто перевіряє публічним.

HTTPS поєднує обидва: асиметричне шифрування використовується, щоб безпечно домовитися про симетричний ключ, а далі дані шифруються швидким симетричним.

Де шифрують дані:

- **In transit** (при передачі) — TLS/HTTPS, VPN. Захищає від підслуховування в мережі.
- **At rest** (при зберіганні) — шифровані диски, бази, об'єкти. Захищає, якщо хтось отримає доступ до носія чи копії.

Найважче в шифруванні — **керування ключами**: де зберігати, хто має доступ, як міняти (ротація), як не загубити. В AWS для цього є **KMS** (розділ 11).

Хешування — інша річ. Хеш-функція (наприклад, **SHA-256**) перетворює будь-які дані на «відбиток» фіксованої довжини, і **назад повернути неможливо**. Зміни одну літеру — отримаєш зовсім інший відбиток. Навіщо:

- **паролі** зберігають не відкритими і не зашифрованими, а **хешованими** (з «сіллю»): навіть якщо базу вкрадуть, паролів там немає;
- **цілісність**: порівнявши хеші, легко перевірити, що файл не змінився при передачі.



Шифрування чи хеш?

Шифрування — сейф: поклав, зачинив ключем, відчинив ключем.

Хеш — м'ясорубка: з фаршу котлету назад не зробиш, але з того самого м'яса завжди вийде той самий фарш.

Автентифікація і авторизація

- **Автентифікація (authentication)** — «хто ти?»: логін і пароль, відбиток пальця, токен.
- **Авторизація (authorization)** — «що тобі можна?»: читати замовлення можна, видаляти — ні.

MFA (багатофакторна автентифікація) вимагає двох різних факторів: щось, що ти **знаєш** (пароль), щось, що ти **маєш** (телефон із кодом, ключ безпеки), щось, чим ти **є** (відбиток). Украдений пароль без другого фактора марний.

Федерація і SSO (єдиний вхід) — коли за тебе ручається інша система: «увійти через Google» або корпоративний каталог співробітників. Усе це в AWS — модуль про безпеку, починаючи з розділу 8.



Спробуй: справжній JSON від AWS і хеш своїми руками

Вартість: безкоштовно, акаунт AWS не потрібен. **Час**: 10 хвилин.

1. Відкрий у браузері <https://ip-ranges.amazonaws.com/ip-ranges.json> — це публічний JSON з усіма діапазонами IP-адрес AWS. Знайди (Ctrl + F) `eu-central-1` — побачиш CIDR-діапазони Франкфурта і назви сервісів. Так виглядає реальний API-відповідь.

2. Створи текстовий файл `test.txt` зі словом «Хмаринка».
3. Порахуй його хеш SHA-256. Windows: `certutil -hashfile test.txt SHA256` . macOS/Linux: `shasum -a 256 test.txt` .
4. Зміни одну літеру у файлі й порахуй ще раз — відбиток зміниться повністю. Поверни літеру — повернеться і старий відбиток.
5. Клацни замочок на будь-якому HTTPS-сайті → сертифікат: знайди, хто його видав (CA) і до якої дати він дійсний.

Прибери: видали `test.txt` .

Запам'ятай

Блочне (EBS) — диск одного сервера; **файлове** (EFS, FSx) — спільні папки для багатьох; **об'єктне** (S3) — необмежений склад через HTTP.

Реляційні бази — таблиці, SQL, JOIN, транзакції ACID (RDS, Aurora). **NoSQL** — масштаб і швидкість для простих запитів (DynamoDB) та спеціальні моделі (графи, документи, часові ряди).

OLTP — багато дрібних транзакцій; **OLAP** — важка аналітика в сховищі даних (Redshift).

Кеш прискорює повторні читання; **TTL** — баланс між свіжістю і швидкістю.

Черга розв'язує компоненти і згладжує піки.

Бекап — від помилок і атак; **реплікація** — від відмов (синхронна — без втрат поруч, асинхронна — далеко з можливою втратою останніх змін).

Шифрування **in transit** (TLS) і **at rest** (KMS); **хеш** — одностороння функція для паролів і цілісності.

Автентифікація — «хто ти», авторизація — «що можна»; **MFA** — два різні фактори.

Як питають на іспиті

Зберігати мільйони фото, відео чи бекапів дешево і надійно, доступ через інтернет → **об'єктне сховище (Amazon S3)**

Спільна файлова система для багатьох Linux-серверів → **файлове сховище по NFS (Amazon EFS)**

Диск для бази даних на одному сервері з максимальною швидкістю → **блочне сховище (Amazon EBS)**

Складні запити з JOIN і транзакції → **реляційна база (RDS / Aurora), а не key-value**

Мілісекундні відповіді на мільйони простих запитів за ключем, гнучка схема → **NoSQL key-value (DynamoDB)**

Важкі аналітичні звіти гальмують основну базу → **винести аналітику в окреме сховище даних (Redshift) або на репліку**

Компоненти мають не губити повідомлення, якщо один із них тимчасово недоступний → **черга між ними (асинхронна взаємодія)**

Пастки

S3 — не диск: його не «монтують» під базу даних і не змінюють частину об'єкта.

Реплікація не замінює бекапи: випадкове видалення реплікується миттєво.

Хеш не можна «розшифрувати»; шифрування — можна (з ключем). Паролі зберігають хешованими, а не зашифрованими.

Асинхронна репліка може не мати останніх змін — при аварії можлива невелика втрата даних.

Перевір себе

ПИТАННЯ 1

Хмаринці треба зберігати 2 мільйони фото товарів, які показуються на сайті й у мобільному застосунку. Обсяг постійно росте. Який тип сховища найкраще підходить?

- A** Об'єктне сховище з доступом через HTTP
- B** Блочне сховище, підключене до вебсервера
- C** Реляційна база даних з колонкою для фото
- D** Файлове сховище, змонтоване на кожному сервері

Правильна відповідь: A. Фото — неструктуровані дані, які читають застосунки й браузері через інтернет. Об'єктне сховище (S3) практично необмежене, дуже надійне, дешеве і доступне по HTTP. Зберігати великі файли в реляційній базі — повільно й дорого.

ПИТАННЯ 2

Кілька Linux-серверів у різних дата-центрах мають одночасно читати й записувати ті самі файли як у звичайній папці. Що обрати?

- A** Об'єктне сховище без монтування
- B** Блочний диск, підключений до одного сервера
- C** Кеш у пам'яті
- D** Файлове сховище з доступом по NFS

Правильна відповідь: D. Спільний одночасний доступ багатьох серверів до файлів як до папки — це файлове сховище. Для Linux протокол — NFS (в AWS — Amazon EFS). Блочний диск зазвичай підключають до одного сервера.

ПИТАННЯ 3

Банківський переказ має або повністю виконатися (списання й зарахування), або не виконатися зовсім. Яка властивість бази даних це гарантує?

- A** TTL у кеші
- B** Горизонтальне масштабування
- C** Atomicity (атомарність) з ACID
- D** Eventual consistency

Правильна відповідь: C. Атомарність гарантує, що всі кроки транзакції виконуються разом або жоден. Це частина гарантій ACID, якими славляться реляційні бази.

ПИТАННЯ 4

Чим хешування відрізняється від шифрування?

- A** Хеш не можна повернути до вихідних даних, а зашифровані дані можна розшифрувати ключем
- B** Хешування використовує публічний і приватний ключі
- C** Хешування швидше, тому ним захищають дані на дисках
- D** Різниці немає, це синоніми

Правильна відповідь: А. Хеш — одностороння функція: відбиток даних, з якого не відновити оригінал. Тому ним зберігають паролі та перевіряють цілісність. Шифрування оборотне — з правильним ключем дані розшифровуються.

ПИТАННЯ 5

Основна база реплікується в інший регіон асинхронно. Що може статися при раптовій аварії основної бази?

- A** Репліка автоматично стане синхронною
- B** Останні зміни, які ще не встигли дійти до репліки, можуть загубитися
- C** Дані на репліці будуть зашифровані іншим ключем
- D** Нічого: асинхронна репліка завжди має всі дані

Правильна відповідь: В. При асинхронній реплікації основна база підтверджує запис одразу, а на репліку він доходить із затримкою. Якщо аварія станеться в цей проміжок, останні зміни загубляться. Це і є ненульовий RPO.

ПИТАННЯ 6

Звіт бухгалтерії з підсумками за три роки виконується годину і сповільнює оформлення замовлень на сайті. Що змінити в архітектурі?

- A** Додати в основну базу більше колонок
- B** Перенести замовлення в кеш
- C** Вимкнути транзакції в основній базі
- D** Виконувати аналітику окремо від транзакційної бази — у сховищі даних або на репліці для читання

Правильна відповідь: D. OLTP-база (замовлення) і OLAP-навантаження (важкі звіти) заважають одне одному. Аналітику виносять на репліку для читання або в окреме сховище даних з колонковим зберіганням (Redshift).



Що таке хмара і навіщо вона бізнесу

🔒 Безпека · 30%

🔗 Вартість · 20%

 **Історія Хмаринки.** Тарас приносить Олені кошторис: новий потужний сервер — 8 000 доларів, ще один для резерву — стільки ж, плюс кондиціонер, безперебійник і налаштування. Доставка — через шість тижнів. «А якщо наступна "чорна п'ятниця" буде вдвічі більшою? А якщо меншою — і сервери простоюватимуть?» — питає бухгалтерка Марта. Олена згадує, що великі магазини давно живуть у хмарі. Але що це насправді: «чужий комп'ютер» чи щось більше?

 **Образ.** Сто років тому великі фабрики будували власні електростанції. Сьогодні ніхто не будує електростанцію для свого будинку: вмикаєш прилад у розетку і платиш за кіловат-години. **Хмара — це розетка для ІТ:** потужність серверів, сховищ і баз даних «тече з розетки» за хвилини, а платиш за те, що спожив.

Як жили до хмари: власний дата-центр

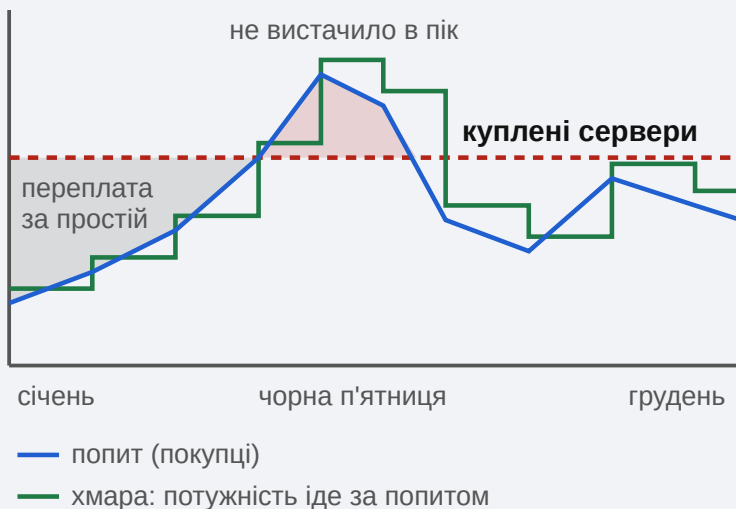
Модель **on-premises** («на території», скорочено on-prem) — усе своє: купуєш сервери, ставиш у серверну, наймаєш адміністраторів, платиш за електрику й охолодження, стежиш за заміною дисків і оновленнями. Так працювала Хмаринка — тільки в мініатюрі.

Проблеми цієї моделі:

- **Великі гроші наперед.** Обладнання купують одразу на кілька років — це капітальні витрати (**CapEx**).

- **Довго.** Від рішення до працюючого сервера — тижні або місяці: закупівля, доставка, монтаж.
- **Вгадування потужності.** Купуєш під пік — більшу частину року обладнання простоює. Купуєш «під середнє» — у пік сайт падає.
- **Рутину.** Заміна дисків, оновлення, кондиціонери, безпека приміщення — усе на тобі.
- **Географія.** Щоб бути ближче до клієнтів у Німеччині, треба будувати дата-центр у Німеччині.

потужність



Червона пунктирна лінія — потужність, куплену заздалегідь: більшу частину року вона простоює (сіра зона), а в пік її забракло (червона зона). У хмарі потужність (зелені сходишки) додається і прибирається слідом за попитом.

Що таке хмарні обчислення

Хмарні обчислення (cloud computing) — оренда ІТ-ресурсів (серверів, сховищ, баз даних, мереж, програм) через інтернет **на вимогу**, з оплатою

за фактичне використання. Класичне визначення (NIST) називає п'ять ознак справжньої хмари:

1. **Самообслуговування на вимогу** — ресурс створюєш сам, за хвилини, без дзвінків і договорів.
2. **Широкий мережевий доступ** — усім керуєш через інтернет: консоль, API, застосунки.
3. **Спільний пул ресурсів** — провайдер обслуговує мільйони клієнтів на спільному обладнанні, надійно ізолюючи їх одне від одного.
4. **Швидка еластичність** — ресурси додаються й прибираються швидко, часто автоматично.
5. **Вимірюваний сервіс** — усе вимірюється (години, гігабайти, запити), і платиш саме за це.

Якщо в «хмарі» сервер треба замовляти листом і чекати тиждень — це не хмара, а просто чужий дата-центр.

Шість переваг хмари за версією AWS

AWS формулює переваги хмари шістьма тезами — їх варто знати дослівно, бо на них спираються питання про вартість і гнучкість:

Перевага (англ.)	Що означає	Для Хмаринки
Trade fixed expense for variable expense	Замість великих вкладень наперед (CapEx) — оплата за використання (OpEx)	Не треба 16 000 \$ на сервери — платимо щомісяця за спожите
Benefit from massive economies of scale	AWS купує обладнання мільйонами — і ділиться знижкою	Ціна за сервер-годину нижча, ніж могла б бути своя
Stop guessing capacity	Не треба вгадувати потужність — вона масштабується	Розпродаж? Сервери додадуться самі
Increase speed and agility	Нові ресурси — за хвилини	Тарас тестує ідею сьогодні, а не через шість тижнів

Перевага (англ.)	Що означає	Для Хмаринки
Stop spending money running and maintaining data centers	Не витрачай гроші на «залізну» рутину	Замість кондиціонерів — робота над магазином
Go global in minutes	Розгорни систему в іншій частині світу за хвилини	Копія сайту для Німеччини — у регіоні Франкфурт

Моделі сервісів: IaaS, PaaS, SaaS

Хмарні сервіси відрізняються тим, **скільки роботи бере на себе провайдер**. Класичний образ — піца:

Модель	Образ	Що робиш ти	Що робить провайдер	Приклади
On-premises	Готуєш піцу вдома з нуля	Усе: від заліза до застосунку	Нічого	Сервер у підсобці
IaaS — інфраструктура як сервіс	Купуєш готове тісто, печеш у своїй духовці	ОС, патчі, програми, дані	Залізо, мережа, віртуалізація	Amazon EC2, EBS, VPC
PaaS — платформа як сервіс	Замовляєш піцу з доставкою, стіл і напої — свої	Код застосунку, дані, налаштування	Також ОС, середовище виконання, масштабування	AWS Elastic Beanstalk, Amazon RDS, AWS Lambda
SaaS — програма як сервіс	Ідеш у піцерію	Лише користуєшся	Усе	Gmail, Microsoft 365, Salesforce

Для архітектора найважливіше слово тут — **керований сервіс** (managed service). Це сервіс, де AWS бере на себе рутину: встановлення, оновлення, резервне копіювання, відмовостійкість, масштабування. Приклад: базу MySQL можна поставити самому на EC2 (IaaS — усе на тобі) або взяти Amazon RDS for MySQL (керований — AWS латає, робить бекапи, перемикає на резервну копію при збої). На іспиті вимога **«LEAST operational overhead»** майже завжди означає: обери керований або serverless-сервіс.

Serverless («безсерверний») — крайній ступінь керованості: серверів не бачиш зовсім, платиш за виконання, масштабування — автоматичне, до нуля, коли немає навантаження. Lambda, S3, DynamoDB, SQS, API Gateway, Fargate — усе це serverless.

Моделі розгортання

- **Хмара (public cloud)** — усе в AWS.
- **Гібрид (hybrid)** — частина в AWS, частина у власному дата-центрі, з'єднані мережею. Дуже часто — під час переїзду або через закони, що вимагають тримати певні дані локально. Хмаринка якийсь час буде саме гібридною: склад лишиться в офісі.
- **On-premises / приватна хмара** — власний дата-центр, іноді з хмарними технологіями всередині (віртуалізація, самообслуговування). AWS може навіть привезти свої стійки до тебе — **AWS Outposts** (розділ 6).
- **Multi-cloud** — кілька провайдерів одночасно (AWS + Azure). Буває, але ускладнює все.

Модель спільної відповідальності

Найважливіша тема цього розділу для іспиту. Безпека в хмарі — **спільна** справа AWS і клієнта:

- **AWS відповідає за безпеку CAMOЇ хмари (security OF the cloud):** будівлі дата-центрів, охорона, живлення, обладнання, мережа між дата-центрами, гіпервізор і програмне забезпечення керованих сервісів.
- **Клієнт відповідає за безпеку В ХМАPI (security IN the cloud):** свої дані, хто має доступ (IAM), налаштування файрволів, шифрування, ОС і програми на своїх серверах.

Межа зсувається залежно від сервісу: що «керованіший» сервіс, то більше бере AWS.



EC2 (IaaS)

Ти: дані, доступи IAM, Security Groups, **ОС** і її патчі, програми, шифрування
AWS: залізо, мережа, гіпервізор, дата-центр



RDS (керована БД)

Ти: дані, користувачі БД і права, Security Groups, чи ввімкнене шифрування, вікно обслуговування
AWS: **ОС, патчі рушія БД,** бекапи за твоїми налаштуваннями, залізо



Lambda, S3, DynamoDB

Ти: дані, код, політики доступу, налаштування (наприклад, щоб bucket не був публічним)
AWS: усе інше — сервери, ОС, середовище виконання, масштабування

Незмінне в усіх випадках: твої дані й доступи до них — завжди твоя відповідальність. Якщо bucket S3 став публічним через налаштування клієнта — це не «злам AWS».

Є і **спільні** контролі, де кожен робить свою частину:

- **Patch management:** AWS латає інфраструктуру й керовані сервіси, клієнт — гостьову ОС і програми на EC2.
- **Configuration management:** AWS налаштовує свою інфраструктуру, клієнт — свої сервери, бази, застосунки.
- **Навчання персоналу:** AWS навчає своїх працівників, клієнт — своїх.

А фізичний захист і середовище (охорона, пожежогасіння) клієнт **успадковує** від AWS повністю — і може показати аудиторам звіти AWS про це (сервіс AWS Artifact, розділ 12).



Хто винен?

1. На EC2 Хмаринки зловмисник використав вразливість у старій версії Linux, яку ніхто не оновив. → **Клієнт** (патчі гостьової ОС на EC2 — його відповідальність).
2. Вразливість у рушії MySQL на RDS. → Патч випускає і встановлює **AWS** у вікно обслуговування, яке обрав клієнт.
3. Через помилку в політиці bucket фото стали доступні всім. → **Клієнт** (налаштування доступу).
4. У дата-центрі зламався диск фізичного сервера. → **AWS**.

5. Співробітник Хмаринки поділився паролем від акаунта в чаті. →
Клієнт (керування доступами).

Скільки коштує хмара

Модель оплати AWS проста за принципом і складна в деталях. Три головні «лічильники»:

1. **Обчислення** — за час роботи серверів (для Linux EC2 — посекундно, з мінімумом 60 секунд), за кількість викликів і тривалість функцій Lambda.
2. **Зберігання** — за гігабайти на місяць (і клас зберігання).
3. **Передача даних** — **вхідний** трафік в AWS з інтернету **безкоштовний**; **вихідний** в інтернет — платний; між зонами доступності й регіонами — теж платний.

Плюс окремі сервіси мають свої одиниці: запити до API, кількість повідомлень, години роботи балансувальника тощо. Є способи платити менше: знижки за зобов'язання (Savings Plans), «вільні» потужності зі знижкою до 90% (Spot), автоматичне переміщення даних у дешевші класи. Цьому присвячений увесь розділ 43.



Чесно: коли хмара — не найкращий вибір

- Величезне **стабільне** навантаження роками без змін — власне залізо може вийти дешевше (хоча з урахуванням персоналу — не завжди).
- **Суворі вимоги** тримати дані фізично в своєму приміщенні — тоді гібрид або AWS Outposts.
- Застосунок має працювати з **мінімальною затримкою** до локального обладнання (заводські лінії) — тоді edge-рішення.

Архітектор обирає не «хмару заради хмари», а те, що відповідає вимогам. На іспиті, щоправда, відповідь майже завжди всередині AWS



Спробуй: порахуй сервер у AWS Pricing Calculator

Вартість: безкоштовно, акаунт не потрібен. **Час:** 10 хвилин.

1. Відкрий <https://calculator.aws> → **Create estimate**.
2. Знайди сервіс **Amazon EC2** → **Configure**.
3. Регіон: **Europe (Frankfurt)**. Операційна система: Linux.
4. Тип інстансу: введи `t4g.small` (2 vCPU, 2 GiB пам'яті). Кількість: 1.
Використання: 100% місяця.
5. Модель оплати: спершу **On-Demand** — запиши ціну за місяць.
Потім обери **Compute Savings Plans** на 1 рік — порівняй.
6. Додай сховище EBS: 20 GB gp3. Подивись, як змінилася сума.
7. Для порівняння: прикинь, скільки таких «серверів-місяців» можна оплатити за 8 000 \$, які Тарас хотів витратити на залізо.

Прибирати нічого не треба — калькулятор нічого не створює в AWS.



Запам'ятай

Хмара = ресурси **на вимогу**, через мережу, **еластично**, з оплатою **за використання**.

Шість переваг: змінні витрати замість фіксованих, економія масштабу, не вгадувати потужність, швидкість, не утримувати дата-центри, глобально за хвилини.

IaaS (EC2) — ти відповідаєш за ОС і вище; **PaaS** (Beanstalk, RDS) — лише за застосунок і дані; **SaaS** — лише користуєшся.

Керований сервіс = менше операційної роботи → улюблена відповідь на «LEAST operational overhead».

Спільна відповідальність: AWS — **OF** the cloud, клієнт — **IN** the cloud.
Дані й доступи — завжди клієнта.

Вхідний трафік безкоштовний, вихідний в інтернет і між зонами — платний.

🎯 Як питають на іспиті

Хто встановлює патчі гостьової ОС на Amazon EC2 → **клієнт**

Хто встановлює патчі ОС і рушія бази в Amazon RDS → **AWS (у вікно обслуговування, яке задає клієнт)**

Хто відповідає за фізичну безпеку дата-центрів і обладнання → **AWS**

Хто налаштовує Security Groups, політики IAM і доступ до bucket → **клієнт**

Компанія не хоче купувати обладнання під пікове навантаження → **хмара: еластичність, «stop guessing capacity»**

Потрібно мінімізувати адміністрування бази даних → **керований сервіс (RDS / Aurora) замість бази на EC2**

⚠ Пастки

«Серверів не видно» (serverless) не означає «за безпеку відповідає AWS»: права доступу, код і дані — твої.

RDS — керований, але **не** повністю «без клієнта»: користувачі БД, мережевий доступ і ввімкнення шифрування — на тобі.

Вхідний трафік в AWS безкоштовний, а от **між зонами доступності** — платний, навіть усередині однієї мережі.

✓ Перевір себе

ПИТАННЯ 1

Компанія запускає вебсервер на Amazon EC2. Хто відповідає за встановлення оновлень безпеки операційної системи на цьому інстансі?

- A** Ніхто: ОС на EC2 оновлюється автоматично
- B** AWS, бо сервер працює в дата-центрі AWS
- C** Клієнт, бо гостьова ОС на EC2 — його відповідальність
- D** AWS і клієнт по черзі

Правильна відповідь: C. EC2 — це IaaS: AWS відповідає за залізо, мережу й гіпервізор, а все, що вище — гостьова ОС, програми, дані й налаштування — відповідальність клієнта.

ПИТАННЯ 2

Яка перевага хмари описує ситуацію «більше не треба купувати сервери під пікове навантаження, яке буває кілька днів на рік»?

- A** Go global in minutes
- B** Benefit from massive economies of scale
- C** Trade variable expense for fixed expense
- D** Stop guessing capacity

Правильна відповідь: D. «Stop guessing capacity»: у хмарі потужність масштабується слідом за попитом, тож не треба вгадувати і тримати обладнання під пік. Зверни увагу: перевага звучить «trade FIXED expense for VARIABLE expense», а не навпаки.

ПИТАННЯ 3

Компанія хоче базу даних MySQL з мінімальними зусиллями на адміністрування: без встановлення патчів і ручних бекапів. Що обрати?

- A** MySQL на Amazon EC2
- B** Amazon RDS for MySQL
- C** MySQL у контейнері на ноутбучі розробника
- D** MySQL на власному сервері в офісі

Правильна відповідь: B. Amazon RDS — керований сервіс: AWS встановлює ОС і рушій, латає їх, робить автоматичні бекапи й може перемикає на резервну копію. На EC2 усе це довелося б робити самому.

ПИТАННЯ 4

Що з переліку є відповідальністю клієнта навіть для serverless-сервісів на кшталт AWS Lambda і Amazon S3?

- A** Заміна несправних дисків
- B** Оновлення операційної системи серверів
- C** Налаштування прав доступу до даних і захист самих даних
- D** Фізична охорона дата-центру

Правильна відповідь: C. У serverless-сервісах AWS бере на себе сервери, ОС і масштабування. Але дані, код і права доступу (IAM-політики, політики bucket) завжди лишаються відповідальністю клієнта.

ПИТАННЯ 5

Хмаринка переносить у хмару сайт, але складська програма поки залишається в офісі, і вони обмінюються даними через захищене з'єднання. Як називається така модель розгортання?

- A** Hybrid (гібридна)
- B** SaaS
- C** Public cloud
- D** Multi-cloud

Правильна відповідь: A. Гібридна модель поєднує власну інфраструктуру (on-premises) і хмару, з'єднані мережею. Multi-cloud — це кілька хмарних провайдерів одночасно, SaaS — модель сервісу, а не розгортання.

ПИТАННЯ 6

До якої моделі сервісів належить Amazon EC2?

- A** IaaS — інфраструктура як сервіс
- B** FaaS — функція як сервіс
- C** SaaS — програма як сервіс
- D** PaaS — платформа як сервіс

Правильна відповідь: A. EC2 дає віртуальні сервери: ти обираєш ОС і відповідаєш за все, що на ній. Це класичний IaaS. Lambda — ближче до FaaS/serverless, Elastic Beanstalk — PaaS.



Глобальна інфраструктура AWS — регіони, зони доступності, edge



Відмовостійкість · 26%



Продуктивність · 24%



Історія Хмаринки. Олена відкриває карту: покупці в Україні, незабаром — у Польщі й Німеччині. «Де фізично житиме наш магазин? — питає вона. — І що буде, якщо в тому місці вимкнуть світло?» Юрист нагадує: персональні дані клієнтів з ЄС мають оброблятися за правилами GDPR. Перше справжнє рішення архітектора — **де** розмістити систему. Для цього треба знати, як AWS фізично влаштований по всьому світу.



Образ. AWS — мережа **міст** по всьому світу. Кожне місто — **регіон** (Region). У кожному місті кілька окремих **районів** — **зон доступності** (Availability Zones): зі своїми електростанціями, водогонами й дорогами, щоб пожежа чи блекаут в одному районі не зачепили інші. Між районами — швидкісні тунелі. А біля людей по всьому світу стоять сотні **кіосків** — **edge locations**, де швидко видають популярний товар (контент), не везучи його щоразу з міського складу.

Регіони

Регіон AWS — географічна область (місто чи агломерація), де AWS має кілька ізольованих дата-центрів. Регіонів понад три десятки, на всіх континентах, крім Антарктиди. Кожен має код, який ти бачитимеш скрізь:

Код	Місто	Коментар
eu-central-1	Франкфурт	Найближчий великий регіон до України, повний набір сервісів
eu-west-1	Ірландія	Один з найстаріших і найбільших регіонів Європи
eu-north-1	Стокгольм	Часто дешевший, «зелена» енергія
eu-west-2 / eu-west-3	Лондон / Париж	
eu-central-2	Цюрих	Вимоги Швейцарії щодо даних
eu-south-1 / eu-south-2	Мілан / Іспанія	
us-east-1	Північна Вірджинія	Найбільший і найстаріший регіон; тут з'являються нові сервіси; деякі глобальні налаштування живуть саме тут

Ключові властивості регіонів:

- **Ізоляція.** Регіони незалежні один від одного. Аварія в одному не зачіпає інші.
- **Дані не переїжджають самі.** AWS не копіює твої дані в інший регіон, поки ти сам не налаштуєш реплікацію чи копіювання. Це основа для вимог **data residency** («дані мають лишатися в ЄС»).
- **Більшість сервісів — регіональні.** Сервер, база, черга створюються в конкретному регіоні й живуть там.

Як обрати регіон: чотири питання

1. **Закони і вимоги (compliance).** Де дозволено зберігати дані? Для клієнтів з ЄС — регіон у ЄС. Це питання **завжди перше**: інші переваги не важливі, якщо регіон заборонений.
2. **Близькість до користувачів (latency).** Менша відстань — швидший сайт.
3. **Доступність сервісів.** Не кожен сервіс і не кожен тип сервера є в кожному регіоні. Нові сервіси часто з'являються спершу в **us-east-1**.
4. **Ціна.** Ціни в регіонах різні (часто на 5–20%).



Рішення для Хмаринки

- Закон: клієнти з ЄС → регіон у ЄС.  Франкфурт, Ірландія, Стокгольм — усі підходять.
- Затримка: Львів, Варшава, Берлін → Франкфурт найближчий (десятки мілісекунд).
- Сервіси: у Франкфурті є все потрібне.
- Ціна: трохи дорожче за Стокгольм, але затримка важливіша для магазину.

Висновок: основний регіон — `eu-central-1` (Франкфурт). Резервний на випадок аварії цілого регіону — `eu-west-1` (Ірландія): інше місто, інша енергосистема, теж ЄС. Важкі нічні задачі без вимог до затримки колись можна буде перенести в дешевший регіон.

Зони доступності

Availability Zone (AZ) — один або кілька дата-центрів у межах регіону з **незалежним** живленням, охолодженням і мережею. У кожному регіоні щонайменше **три** зони (у великих — більше). Зони:

- **фізично рознесені** — на відстані, достатній, щоб пожежа, повінь чи блекаут не зачепили дві зони одночасно (але в межах приблизно 100 км одна від одної);
- **з'єднані** швидкими приватними оптоволоконними каналами із затримкою в **одинарці мілісекунд** — достатньо для **синхронної** реплікації баз даних між ними.

Назви зон — код регіону плюс літера: `eu-central-1a`, `eu-central-1b`, `eu-central-1c`.



РЕГІОН EU-CENTRAL-1 (ФРАНКФУРТ)

AZ EU-CENTRAL-1A



дата-центр



дата-центр

своє живлення, охолодження,
мережа

AZ EU-CENTRAL-1B



дата-центр

кілька км або десятки км від
інших зон

AZ EU-CENTRAL-1C



дата-центр



дата-центр

з'єднана з іншими швидким
оптоволоконном

Регіон $\supset$ зони доступності $\supset$ дата-центри. Зона — найменша «одиниця аварії», яку враховує архітектор: розклади систему щонайменше у дві зони.

Головне правило архітектора: усе важливе розгортай **щонайменше у двох зонах доступності**. Якщо зона **a** зникне через аварію, копії в зоні **b** продовжать обслуговувати клієнтів. Це і є **висока доступність** (high availability, HA) в AWS. Сервіси, у назві яких є «Multi-AZ», роблять це за тебе.



Глибше: чому «eu-central-1a» у тебе і в мене — різні зони

—

Щоб клієнти рівномірно розподілялися по зонах (а не всі вибирали «a»), AWS для кожного акаунта **випадково зіставляє** літери з фізичними зонами. Твоя eu-central-1a може бути моєю eu-central-1c. Якщо двом акаунтам треба домовитися про ту саму фізичну зону (наприклад, при спільному використанні підмереж), використовують **AZ ID** — стабільний ідентифікатор на кшталт euc1-az1, однаковий для всіх акаунтів. На іспиті це іноді з'являється як «забезпечити, що ресурси двох акаунтів в одній фізичній зоні → використовуй AZ ID».

Edge-локації: кіоски біля людей

Окрім регіонів, AWS має **сотні точок присутності** (понад 600) у великих містах світу — **edge locations**. Там не запускають звичайні сервери чи бази

даних. Там працюють сервіси, яким важливо бути **ближче до користувача**:

- **Amazon CloudFront** — мережа доставки контенту (CDN): кешує фото, відео, сторінки біля людей (розділ 18).
- **Amazon Route 53** — DNS, що відповідає з найближчої точки (розділ 17).
- **AWS Global Accelerator** — вхід у швидку приватну мережу AWS найближче до користувача.
- **AWS WAF і AWS Shield** — захист від атак ще до того, як трафік дійде до регіону.

Покупниця у Варшаві отримає фото товарів не з Франкфурта, а з найближчої edge-локації — за кілька мілісекунд.

Особливі «філії» AWS

Коли потрібна ще менша затримка або дані мусять лишатися в конкретному місці:

Варіант	Де фізично	Для чого	Образ
AWS Local Zones	У великому місті, як розширення найближчого регіону	Одноцифрові мілісекунди для мешканців міста: ігри, стрімінг, рендеринг, відеомонтаж	Філія AWS у твоєму місті
AWS Wavelength	Усередині 5G-мережі мобільного оператора	Ультранизька затримка для мобільних пристроїв: AR/VR, ігри, IoT, автономні машини	AWS у вежі 5G біля телефона
AWS Outposts	У твоєму дата-центрі — стійки або сервери AWS	Дані мають фізично лишатися в тебе, або потрібна мінімальна затримка до локальних систем; ті самі API AWS	Шафа-філія AWS у твоїй серверній

Мережа AWS між регіонами

Регіони й edge-локації з'єднує **глобальна приватна мережа AWS** — власні оптоволоконні канали, зокрема підводні кабелі. Трафік між регіонами й сервісами AWS іде нею, а не загальним інтернетом: стабільніше, швидше,

безпечніше (трафік між дата-центрами AWS шифрується). Сервіси на кшталт Global Accelerator і CloudFront дозволяють і твоїм користувачам якомога швидше «зайти» в цю мережу.

Регіональні й глобальні сервіси

Більшість сервісів — **регіональні**. Але кілька — **глобальні**: налаштовуються один раз для всього акаунта.

Глобальні	Регіональні (приклади)
IAM (користувачі, ролі, політики)	EC2, EBS, VPC
Route 53 (DNS)	RDS, Aurora, DynamoDB
CloudFront (CDN)	Lambda, SQS, SNS
AWS Organizations	S3 bucket (ім'я — глобально унікальне, але сам bucket живе в одному регіоні)
Global Accelerator	KMS-ключі, секрети, сертифікати ACM*

* Сертифікат для CloudFront треба створювати саме в `us-east-1` — одна з улюблених пасток (розділ 11).

Від чого захищатися: рівні відмов

Архітектор завжди питає: **відмову чого має пережити система?**

Що може зламатися	Як захиститися в AWS	Ціна
Один сервер	Кілька серверів + балансувальник + автозаміна (Auto Scaling)	Низька
Ціла зона доступності	Розгортання в 2+ зонах (Multi-AZ)	Помірна — стандарт для prod
Цілий регіон	Копія системи або даних в іншому регіоні (multi-region, DR)	Висока — лише коли бізнес цього вимагає

Що може зламатися	Як захиститися в AWS	Ціна
Помилка людини, атака	Бекапи, версії, заборона видалення, розмежування доступів	Помірна — потрібна завжди

Відомий вислів СТО Amazon Вернера Фогельса: *«Everything fails, all the time»* — «Усе ламається, постійно». Добра архітектура не намагається зробити так, щоб нічого не ламалося, — вона робить так, щоб поломки **не помічали клієнти**.

Квоти сервісів (Service Quotas)

У кожному регіоні в акаунта є **ліміти**: скільки серверів певного типу можна запустити, скільки VPC створити тощо. Більшість лімітів («м'які») можна підняти запитом. Важливий нюанс для аварійного відновлення: якщо резервний регіон доведеться раптово навантажити, квоти там треба підняти **заздалегідь** — під час аварії чекати схвалення запиту ніколи.

Спробуй: подивись на інфраструктуру AWS

Вартість: безкоштовно. **Час:** 10 хвилин.

1. Відкрий у браузері сторінку **AWS Global Infrastructure** (пошук: «AWS global infrastructure map») — знайди європейські регіони, кількість зон у кожному, Local Zones і edge-локації.
2. Якщо акаунт уже є (як його створити — наступний розділ): у консолі AWS праворуч угорі відкрий **перемикач регіонів**. Обери Frankfurt — адреса сторінки зміниться на `eu-central-1`.
3. Відкрий сервіс **EC2** → на головній сторінці знайди блок зі статусом зон (Zones) — побачиш назви зон і їхні **AZ ID** (`eu-central-1` тощо).
4. Перемикни регіон на інший і переконайся: список серверів порожній — ресурси живуть у своєму регіоні.

Прибирати нічого не треба — ми нічого не створювали.

Запам'ятай

Регіон — географічна область з кількома ізольованими зонами; дані між регіонами самі не переїжджають.

Зона доступності — один чи кілька дата-центрів з незалежним живленням і мережею; у регіоні їх щонайменше три; між ними — швидкі канали.

Висока доступність = 2+ зони. Захист від втрати регіону = інший регіон.

Вибір регіону: **закон** → **затримка** → **сервіси** → **ціна**.

Edge-локації — сотні точок для CloudFront, Route 53, Global Accelerator, WAF, Shield.

Local Zones — місто, **Wavelength** — 5G, **Outposts** — твій дата-центр.

Глобальні сервіси: **IAM, Route 53, CloudFront, Organizations**. Більшість решти — регіональні.

Як питають на іспиті

Дані клієнтів мають фізично лишатися в ЄС → **region у ЄС (і заборона інших регіонів політиками організації)**

Застосунок має пережити збій одного дата-центру → **розгортання в кількох Availability Zones**

Застосунок має пережити збій цілого регіону → **multi-region: копія даних і інфраструктури в іншому регіоні**

Одноцифрові мілісекунди затримки для користувачів конкретного мегаполіса → **AWS Local Zones**

Ультранизька затримка для мобільних пристроїв у 5G-мережі → **AWS Wavelength**

Дані мають лишатися у власному дата-центрі, але потрібні сервіси й API AWS → **AWS Outposts**

Два акаунти мають розмістити ресурси в одній фізичній зоні → **орієнтуватися на AZ ID, а не на назву зони**

Під час аварії резервний регіон не зміг запустити потрібну кількість серверів → **заздалегідь підняти Service Quotas у DR-регіоні**

Пастки

Edge-локація — **не** регіон і не зона: там не запускають EC2 чи RDS.

Multi-AZ захищає від втрати зони, але **не** від втрати регіону.

Ім'я bucket S3 глобально унікальне, але сам bucket — **регіональний**.

Назва зони (`eu-central-1a`) у різних акаунтах може вказувати на різні фізичні зони.

Перевір себе

ПИТАННЯ 1

Компанія з Польщі зобов'язана за законом зберігати персональні дані клієнтів лише в ЄС. Який критерій вибору регіону AWS має бути першим?

- A** Відповідність законам і вимогам щодо розміщення даних
- B** Найнижча ціна
- C** Найменша затримка до офісу компанії
- D** Найбільша кількість доступних сервісів

Правильна відповідь: A. Вимоги регуляторів (compliance, data residency) — жорсткі обмеження: якщо регіон не відповідає закону, його не можна обрати незалежно від ціни чи швидкості. Серед дозволених регіонів далі дивляться на затримку, сервіси й ціну.

ПИТАННЯ 2

Що таке Availability Zone?

- A** Окрема країна, де AWS має дата-центри
- B** Один або кілька дата-центрів у регіоні з незалежним живленням, охолодженням і мережею
- C** Стійка AWS у дата-центрі клієнта
- D** Точка кешування контенту біля користувачів

Правильна відповідь: B. AZ — ізольована частина регіону з власною інфраструктурою. Регіон складається щонайменше з трьох AZ, з'єднаних швидкими каналами. Точка кешування — це edge location, стійка в дата-центрі клієнта — Outposts.

ПИТАННЯ 3

Вебзастосунок має продовжувати роботу навіть при повній відмові одного дата-центру AWS. Яке рішення мінімально достатнє?

- A** Розгорнути сервери й базу даних щонайменше у двох Availability Zones одного регіону
- B** Розгорнути застосунок у двох регіонах в режимі active-active
- C** Зробити щоденний бекап у S3
- D** Запустити більший сервер в одній зоні

Правильна відповідь: A. Збій дата-центру чи зони закриває розгортання в кількох AZ (Multi-AZ). Два регіони теж захистили б, але це значно дорожче й складніше, ніж вимагає задача. Бекап не дає безперервної роботи.

ПИТАННЯ 4

Ігровій компанії потрібна одноцифрова мілісекундна затримка для гравців у конкретному великому місті, де немає регіону AWS. Що обрати?

A Amazon CloudFront

B AWS Outposts

C AWS Local Zones

D Додатковий регіон

Правильна відповідь: C. Local Zones — розширення регіону у великих містах з мінімальною затримкою для мешканців. Outposts ставлять у власний дата-центр клієнта, CloudFront кешує контент, але не запускає ігрові сервери.

ПИТАННЯ 5

**Які з цих сервісів AWS є глобальними, а не регіональними?
(Оберіть 2)**

Оберіть 2 відповіді

A Amazon Route 53

B Amazon SQS

C AWS IAM

D Amazon RDS

E Amazon EC2

Правильна відповідь: A, C. IAM і Route 53 налаштовуються для всього акаунта одразу. EC2, RDS і SQS створюються в конкретному регіоні.

ПИТАННЯ 6

Два акаунти компанії мають розмістити ресурси в одній фізичній зоні доступності, щоб мінімізувати затримку між ними. На що орієнтуватися?

- A** На назву зони, наприклад eu-central-1a — вона однакова для всіх акаунтів
- B** На IP-адреси серверів
- C** На AZ ID, наприклад euc1-az1 — він однаковий для всіх акаунтів
- D** На назву регіону

Правильна відповідь: C. AWS для кожного акаунта випадково зіставляє літери зон із фізичними зонами, тож «1a» в різних акаунтах може бути різними зонами. AZ ID — стабільний ідентифікатор фізичної зони.



Твій акаунт AWS — безкоштовний план, бюджет, root і Well-Architected



Безпека · 30%



Вартість · 20%



Історія Хмаринки. Тарас реєструє акаунт AWS для Хмаринки на свою пошту і пропонує: «Пароль скину всім у робочий чат, щоб кожен міг зайти». Марта хапається за голову: «А хто стежитиме, щоб ми не отримали рахунок на тисячі доларів за забутий сервер?» Обидва питання — про перші кроки в хмарі. Помилки тут найдорожчі: зламаний головний акаунт або «забутий» ресурс, що місяцями тихо рахує гроші. У цьому розділі — безпечний старт, безкоштовний план і головна рамка якості AWS.



Образ. Акаунт AWS — це будівля бізнес-центру, яку ти орендуєш. **Root user** — оригінал ключа від усіх дверей плюс право власності на договір оренди: його не носять із собою щодня, а тримають у сейфі. Для щоденної роботи кожен співробітник отримує **перепустку** (користувача або роль) з доступом лише до потрібних поверхів. А **бюджет з алертами** — лічильник електроенергії з сигналізацією, що дзвонить, коли споживання росте.

Що таке акаунт AWS

Акаунт AWS — це «контейнер» для всього, що ти створюєш в AWS. Він має три ролі одночасно:

- **Межа ресурсів** — сервери, бази, мережі належать конкретному акаунту.

- **Межа безпеки** — за замовчуванням ніхто з іншого акаунта не має доступу до твоїх ресурсів.
- **Межа оплати** — усе, що створено в акаунті, потрапляє в його рахунок.

Кожен акаунт має 12-значний номер (Account ID), наприклад **111122223333**. Великі компанії мають десятки й сотні акаунтів (окремо prod, dev, безпека, логи) і об'єднують їх у **AWS Organizations** — про це розділ 10.

Для реєстрації потрібні: email, пароль, номер телефону для підтвердження і банківська картка (для перевірки особи й оплати понад безкоштовні ліміти).

Безкоштовний план AWS (з липня 2025 року)

AWS змінив умови для нових клієнтів 15 липня 2025 року. Тепер при реєстрації обираєш один з двох планів:

	Free plan (безкоштовний)	Paid plan (платний)
Кредити	\$100 одразу + до \$100 за виконання завдань	Ті самі кредити
Чи можуть зняти гроші з картки	Ні — поки не перейдеш на платний план	Так — за використання понад кредити
Скільки діє	6 місяців або доки не закінчатся кредити (що настане раніше)	Без обмежень
Доступ до сервісів	Більшість, але не ті, що миттєво «з'їли» б кредити (наприклад, резервування Savings Plans, Reserved Instances, платна підтримка)	Усі
Що після завершення	Акаунт закривається; 90 днів можна перейти на Paid plan і відновити доступ до ресурсів	—

Як заробити ще \$100: по \$20 за кожне з п'яти стартових завдань — запустити сервер **EC2**, створити базу **RDS**, зробити функцію **Lambda**, спробувати **Amazon Bedrock** і налаштувати бюджет у **AWS Budgets**. Практики цього курсу проходять саме через ці сервіси, тож кредити ти отримаєш природно.

Крім кредитів, є **Always Free** — понад 30 сервісів із безкоштовними місячними лімітами, які діють завжди. Наприклад: **AWS Lambda** — 1 мільйон викликів і 400 000 GB-секунд на місяць; **Amazon DynamoDB** — 25 GB сховища; **Amazon SQS** — 1 мільйон запитів; **Amazon SNS** — 1 мільйон публікацій; базові метрики й аларми **CloudWatch**.

Якщо акаунт створено до 15 липня 2025

Для старих акаунтів діють попередні умови: 12 місяців безкоштовних лімітів (наприклад, 750 годин на місяць найменших серверів EC2). Точні актуальні умови свого акаунта дивись у консолі: **Billing and Cost Management** → **Free Tier** та **Credits**.

Root user: головний ключ, який тримають у сейфі

При реєстрації створюється **root user** — користувач з email-адресою акаунта. Він має **необмежені права**, і їх не можна обмежити звичайними політиками. Якщо root зламують — зловмисник може все: видалити дані, запустити сотні серверів для майнінгу, закрити акаунт.

Правила для root:

1. **Увімкни MFA** (двофакторну автентифікацію) одразу після реєстрації. Підходить застосунок-автентифікатор на телефоні, ключ безпеки або passkey. AWS тепер і сам вимагає MFA для root.
2. **Не створюй access keys** (ключі для програмного доступу) для root. Ніколи.
3. **Не використовуй root для щоденної роботи**. Створи окремого адміністратора (про це — у наступному розділі) і працюй під ним.
4. **Не діли пароль root** ні з ким. Кожна людина отримує власний вхід.
5. Використовуй для root **спільну захищену поштову скриньку** компанії (наприклад, aws-root@khhmarynka.ua), а не особисту — щоб доступ не зник разом зі співробітником.

Лише root може виконувати кілька особливих дій: змінити email, ім'я чи пароль акаунта, **закрити акаунт**, відновити доступ, якщо адміністратори IAM випадково заблокували себе, увімкнути **MFA Delete** для bucket S3, виправити політику bucket, що забороняє доступ усім. Решта — під звичайним адміністратором.

Щоб рахунок не став сюрпризом: бюджети й алерти

Найчастіша біда новачків — «забутий» ресурс. Кілька речей тарифікуються **погодинно, навіть коли ними не користуються**:

- запущені сервери **EC2** і бази **RDS** (зупинена база RDS через 7 днів запускається сама!);
- **NAT Gateway** — за кожну годину і кожен гігабайт;
- балансувальники навантаження;
- **публічні IPv4-адреси** — з 2024 року кожна коштує приблизно \$3,6 на місяць;
- диски **EBS** і їхні знімки, що лишилися після видалення сервера;
- ресурси в **іншому регіоні**, про які забули, бо дивилися лише на свій.

Захист — **AWS Budgets**: задаєш суму і отримуєш лист (або повідомлення), коли фактичні чи **прогнозовані** витрати наближаються до неї. Є готові шаблони:

- **Zero spend budget** — сигнал, щойно витрати перевищать \$0,01. Ідеально для навчального акаунта.
- **Monthly cost budget** — місячна сума з алертами, наприклад, на 85% і 100%.

Бюджет може не лише попереджати, а й **діяти** (Budget Actions): наприклад, автоматично застосувати політику, що забороняє запускати нові сервери. Інші інструменти витрат — Cost Explorer, Cost Anomaly Detection, звіти про використання — розберемо в розділі 43.



Budgets чи Cost Explorer?

Budgets дивиться **вперед** і **попереджає**: «ти от-от перевищиш». **Cost Explorer** дивиться **назад** і **аналізує**: «куди пішли гроші за три місяці і який прогноз». На іспиті «надіслати сповіщення, коли витрати перевищать суму» → Budgets.

Як працювати з AWS: консоль, CLI, SDK

Усе в AWS — це **API-виклики**. Способів їх робити кілька:

- **AWS Management Console** — вебінтерфейс. Зручно вчитися й дивитися.
- **AWS CLI** — командний рядок: `aws s3 ls`, `aws ec2 describe-instances`. Зручно автоматизувати.
- **AWS CloudShell** — командний рядок прямо в браузері, всередині консолі, з уже встановленим CLI і твоїми правами. Безкоштовний.
- **SDK** — бібліотеки для Python, JavaScript, Java, .NET, Go тощо: застосунок Хмаринки викликатиме AWS з коду.
- **Інфраструктура як код** (Infrastructure as Code) — шаблони CloudFormation, AWS CDK, Terraform: уся інфраструктура описана у файлах і розгортається однаково щоразу (розділ 41).

Кожен API-виклик **підписується** обліковими даними (хто ти) і перевіряється на права (що тобі можна). І кожен записується в журнал **AWS CloudTrail**: за замовчуванням історія керувальних дій за останні **90 днів** доступна в консолі безкоштовно. Так завжди можна з'ясувати, хто і коли видалив сервер.



Плани підтримки AWS

Basic — безкоштовно для всіх: документація, форуми, AWS Health Dashboard, базові перевірки Trusted Advisor. Платні плани (**Developer, Business, Enterprise On-Ramp, Enterprise**) дають технічну підтримку

інженерів з різним часом відповіді. Для іспиту корисно пам'ятати: **повний набір перевірок AWS Trusted Advisor** — з планом **Business** і вищим, а **персональний технічний менеджер (TAM)** — на Enterprise-рівнях.

AWS Well-Architected Framework: шість стовпів

За роки роботи з тисячами клієнтів AWS зібрав кращі практики в **Well-Architected Framework** — шість «стовпів» (pillars), на яких стоїть якісна архітектура. Увесь іспит SAA-C03 побудований на них, тож повертатимемося до них у кожному розділі.



Operational Excellence

Операційна досконалість

Інфраструктура як код, часті малі зміни, спостережуваність, вчитися на кожному інциденті



Security

Безпека

Найменші права, без довгострокових ключів, журнали всього, захист на всіх рівнях, шифрування



Reliability

Надійність

Автоматичне відновлення, горизонтальне масштабування, тестування відновлення, не вгадувати потужність



Performance Efficiency

Ефективність продуктивності

Керовані сервіси, serverless, глобально за хвилини, обирати технологію під задачу



Cost Optimization

Оптимізація витрат

Платити за використане, вимірювати, розподіляти витрати, не витрачатися на «важку рутину»



Sustainability

Сталий розвиток

Максимально завантажувати ресурси, ефективніші процесори (Graviton), менше зайвих даних

Мнемоніка «CROPS + S»: Cost, Reliability, Operational excellence, Performance, Security + Sustainability.

Кілька принципів, які найчастіше «ховуються» в правильних відповідях іспиту:

- **Security:** принцип найменших привілеїв; тимчасові облікові дані замість постійних ключів; журнали й аудит усіх дій; шифрування даних **at rest** і **in transit**; «тримай людей подалі від даних» (автоматизація замість ручного доступу).
- **Reliability:** автоматично відновлюватися після збоїв; масштабуватися горизонтально, а не одним великим ресурсом; перевіряти процедури відновлення; змінювати інфраструктуру через автоматизацію.
- **Performance Efficiency:** використовувати керовані сервіси й serverless, щоб не витрачати зусилля на «важку рутину»; розміщуватися ближче до користувачів; експериментувати.
- **Cost Optimization:** модель «плати за використане»; правильний розмір ресурсів; вимикати непотрібне; вимірювати і розподіляти витрати тегами й акаунтами.
- **Operational Excellence:** «операції як код»; малі, часті, зворотні зміни; передбачати збої; вчитися на інцидентах.
- **Sustainability:** максимізувати завантаження, обирати ефективніше обладнання й керовані сервіси, не зберігати й не пересилати зайве.

Для самоперевірки архітектури є безкоштовний **AWS Well-Architected Tool** у консолі: відповідаєш на питання по кожному стовпу — отримуєш список ризиків і рекомендацій. Існують і спеціалізовані «лінзи» (lenses) — для serverless, аналітики, SaaS, машинного навчання.

Спробуй у справжньому AWS: безпечний старт акаунта

Вартість: безкоштовно. Бонус — **\$20 кредитів** за налаштування бюджету. **Час:** 20–30 хвилин.

1. **Реєстрація.** Відкрий <https://aws.amazon.com> → **Create an AWS Account**. Вкажи email, назву акаунта (наприклад, `khmarynka-learning`), підтверди телефон і картку. Обери **Free plan**.

2. **MFA для root.** Увійди як root → меню з назвою акаунта праворуч угорі → **Security credentials** → **Assign MFA device**. Обери застосунок-автентифікатор (Google Authenticator, Microsoft Authenticator, 1Password) або passkey і підтверди двома кодами.
3. **Регіон.** Праворуч угорі обери **Europe (Frankfurt) eu-central-1**. Надалі всі практики — в одному регіоні, щоб нічого не загубити.
4. **Бюджет.** Пошук у консолі → **Budgets** → **Create budget** → шаблон **Zero spend budget** → вкажи свій email → Create. Потім створи другий: шаблон **Monthly cost budget**, сума **\$5**. Тепер ти дізнаєшся про будь-які витрати.
5. **CloudShell.** Натисни значок терміналу `>_` у верхній панелі консолі. Коли відкриється, введи по черзі:

```
aws sts get-caller-identity
aws ec2 describe-regions --query "Regions[].RegionName" --output ta
aws ec2 describe-availability-zones --output table
```

Перша команда покаже, **хто** ти (номер акаунта і ARN). Друга — список регіонів. Третя — зони доступності Франкфурта з їхніми AZ ID.

1. **Кредити.** Billing and Cost Management → **Credits**: переконайся, що кредити нараховані, і подивись строк їхньої дії.

Прибери за собою: нічого не треба — бюджети безкоштовні й корисні, залиш їх. Наступний крок — окремий адміністратор замість root (розділ 8).

Запам'ятай

Акаунт AWS — межа ресурсів, безпеки й оплати.

Free plan (з 15.07.2025): до \$200 кредитів (\$100 + до \$100 за завдання), 6 місяців, без списань з картки; **Paid plan** — без обмежень. **Always Free** ліміти діють завжди.

Root: MFA, без access keys, не для щоденної роботи, не ділитися паролем.

AWS Budgets попереджає про витрати (фактичні й прогнозовані) і вміє діяти; **Cost Explorer** аналізує минуле.

Усе в AWS — API-виклики; консоль, CLI, CloudShell, SDK, IaC — різні способи їх робити; CloudTrail записує їх (90 днів історії за замовчуванням).

Well-Architected — 6 стовпів: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.

Як питають на іспиті

Захистити root user акаунта → **увімкнути MFA, не створювати access keys, не використовувати для щоденних задач**

Сповіщення, коли прогноз місячних витрат перевищить суму → **AWS Budgets (forecasted alert)**

Автоматично заборонити запуск нових ресурсів при перевищенні бюджету → **AWS Budgets Actions**

Проаналізувати, на що пішли гроші за останні місяці → **AWS Cost Explorer**

Перевірити архітектуру на відповідність кращим практикам AWS → **AWS Well-Architected Tool**

Повний набір перевірок Trusted Advisor → **план підтримки Business або вищий**

Хто і коли видалив ресурс в акаунті → **AWS CloudTrail**

Пастки

Root не можна обмежити звичайними IAM-політиками — тому його захищають MFA і не використовують.

Зупинена база RDS **сама запуститься через 7 днів** — видаляй навчальні бази після практики.

Погодинні ресурси (NAT Gateway, балансувальник, публічна IPv4) коштують гроші, навіть коли трафіку немає.

Budgets не «зупиняє AWS» сам по собі — лише попереджає, поки не налаштовані Budget Actions.

✓ Перевір себе

ПИТАННЯ 1

Новий співробітник Хмаринки має щодня працювати з сервісами AWS. Як правильно надати йому доступ?

- A** Створити для root user ключі доступу й передати співробітнику
- B** Створити для нього окремий обліковий запис з потрібними правами і MFA
- C** Дозволити йому працювати в CloudShell під root
- D** Дати йому пароль root user і попросити не ділитися ним

Правильна відповідь: B. Root має необмежені права і використовується лише для особливих задач. Кожна людина отримує власний вхід з мінімально потрібними правами і MFA — так дії відстежуються, а доступ легко відкликати.

ПИТАННЯ 2

Фінансовий директор хоче отримувати лист, щойно прогнозовані витрати за місяць перевищать \$1 000. Що налаштувати?

- A** AWS CloudTrail з підпискою на білінг
- B** AWS Budgets з алертом на прогнозовані витрати
- C** Amazon CloudWatch Logs з фільтром «cost»
- D** AWS Cost Explorer з фільтром за місяць

Правильна відповідь: B. AWS Budgets відстежує фактичні й прогнозовані витрати і надсилає сповіщення при досягненні порогів. Cost Explorer — інструмент аналізу, а не сповіщень.

ПИТАННЯ 3

Яке твердження про безкоштовний план AWS (Free plan, з липня 2025 року) правильне?

- A** Він дає необмежене використання всіх сервісів протягом року
- B** Він діє 12 місяців і автоматично переходить у платний
- C** Він не дозволяє запускати сервери EC2
- D** Він діє до 6 місяців або до вичерпання кредитів, і з картки не списують гроші, поки клієнт сам не перейде на платний план

Правильна відповідь: D. Free plan триває 6 місяців або до вичерпання кредитів (що раніше). Гроші не списуються, доки клієнт не оновиться до Paid plan. Після завершення акаунт закривається, але 90 днів можна перейти на платний план і відновити ресурси.

ПИТАННЯ 4

Які дві дії варто зробити з root user одразу після створення акаунта? (Оберіть 2)

Оберіть 2 відповіді

- ☐ **A** Видалити root user
- ☐ **B** Поділитися паролем з командою в захищеному чаті
- ☐ **C** Увімкнути MFA
- ☐ **D** Створити access keys для скриптів
- ☐ **E** Створити окремого адміністратора і далі працювати під ним

Правильна відповідь: C, E. Root захищають MFA і не використовують щодня — для роботи створюють окремих адміністраторів. Access keys для root створювати не можна (з погляду кращих практик), а видалити root user в окремому акаунті неможливо.

ПИТАННЯ 5

Який стовп Well-Architected Framework найбільше стосується автоматичного відновлення після збоїв і горизонтального масштабування?

- A** Cost Optimization
- B** Operational Excellence
- C** Reliability
- D** Sustainability

Правильна відповідь: C. Reliability — здатність системи відновлюватися після збоїв і витримувати зміни навантаження: автоматичне відновлення, горизонтальне масштабування, перевірка процедур відновлення.

ПИТАННЯ 6

Розробник хоче швидко виконати кілька команд AWS CLI, не встановлюючи нічого на свій комп'ютер. Що найпростіше?

- A** Завантажити SDK для Python
- B** Створити шаблон CloudFormation
- C** Відкрити AWS CloudShell у консолі
- D** Запустити сервер EC2 і встановити на нього CLI

Правильна відповідь: C. CloudShell — безкоштовний термінал у браузері з уже встановленим AWS CLI і правами поточного користувача. Не потрібно ні сервера, ні локального встановлення.



IAM — користувачі, групи, політики і принцип найменших привілеїв

🔒 Безпека · 30%



Історія Хмаринки. Перший тиждень у хмарі. Тарас налаштовує сервери, Марта хоче бачити рахунки, Олена — «просто подивитися», а на склад прийшов стажер, якому треба лише завантажувати фото товарів. Раніше всі входили на сервер під одним паролем адміністратора. Одного ранку зникла папка з фото — і ніхто не зміг сказати, хто її видалив. Більше так не буде: у кожного — свій вхід, свої права і повний журнал дій. За це в AWS відповідає **IAM**.



Образ. Бізнес-центр з електронними перепустками. **Користувач** — перепустка конкретної людини. **Група** — відділ: усім бухгалтерам видають однаковий набір дверей. **Політика** — список дверей, які відчиняє перепустка («поверх 3 — так, серверна — ні»). За замовчуванням **нова перепустка не відчиняє нічого** — доступ додають явно. А якщо десь написано «сюди — заборонено», ця заборона сильніша за будь-який дозвіл.

Що таке IAM

AWS Identity and Access Management (IAM) — сервіс, що відповідає на два питання для кожного запиту до AWS:

1. **Хто ти?** — автентифікація: пароль і MFA для людей у консолі, підписані ключами запиту для програм.

2. **Що тобі можна?** — авторизація: політики, що дозволяють або забороняють дії з ресурсами.

IAM — **глобальний і безкоштовний** сервіс. Будь-який запит в AWS можна розкласти на чотири частини — запам'ятай їх, бо з них складається кожна політика:

- **Principal** — хто просить (користувач, роль, сервіс AWS);
- **Action** — що саме хоче зробити (`s3:GetObject` , `ec2:StartInstances`);
- **Resource** — з яким ресурсом (конкретний bucket, сервер, таблиця);
- **Condition** — за яких умов (з якої IP-адреси, з MFA чи без, в якому регіоні).

Користувачі, групи і ролі

IAM user — постійна «особа» в акаунті з довгостроковими обліковими даними:

- **пароль** для входу в консоль (плюс MFA);
- **access keys** — пара ключів (Access Key ID + Secret Access Key) для CLI, SDK і програм.

IAM group — набір користувачів. Політики прикріплюють до групи — і всі її учасники отримують ці права. Прийшов новий бухгалтер — додав у групу Finance, звільнився — прибрав. Особливості груп:

- групи **не вкладаються** одна в одну;
- користувач може бути в кількох групах (права додаються);
- група — **не principal**: на неї не можна послатися в політиці ресурсу («дозволити групі Finance доступ до bucket» напругу не вийде).

IAM role — «особа без людини»: набір прав, який тимчасово **беруть на себе** (assume) люди, сервіси чи програми, отримуючи **тимчасові** облікові дані. Ролі — найважливіший механізм безпеки AWS, їм присвячений увесь наступний розділ.



Сучасна рекомендація AWS для людей

Для співробітників AWS радить не створювати IAM users у кожному акаунті, а використовувати **єдиний вхід** через **IAM Identity Center** (з тимчасовими обліковими даними). IAM users лишаються для особливих випадків: аварійний доступ, старі інструменти, що не вміють працювати з ролями. На іспиті вимога «керувати доступом співробітників до багатьох акаунтів» майже завжди веде до Identity Center (розділ 9). Але механіка політик однакова для всіх — тож розбираємо її тут.

Політики: мова дозволів

Політика — JSON-документ з одним чи кількома **твердженнями** (statements). Кожне твердження каже: «**Дозволити** (або **заборонити**) **такі дії** над **такими ресурсами** за **таких умов**».

Ось політика, що дозволяє стажерові завантажувати й переглядати фото лише в одній «папці» bucket Хмаринки:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListPhotosFolder",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::khmarynka-photos",
      "Condition": { "StringLike": { "s3:prefix": "products/*" } }
    },
    {
      "Sid": "ReadWritePhotos",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::khmarynka-photos/products/*"
    }
  ]
}
```

Розберімо поля:

- **Version** — версія мови політик. Завжди "2012-10-17" (це не дата створення твоєї політики!).
- **Sid** — необов'язкова назва твердження для зручності.
- **Effect** — Allow або Deny .
- **Action** — дії у форматі сервіс:Дія . Можна зірочки: s3:* (усі дії S3), s3:Get* (усі, що починаються на Get).
- **Resource** — на які ресурси діє, у форматі **ARN**.
- **Condition** — додаткові умови (про них нижче).
- **Principal** — «кому» — з'являється лише в політиках, прикріплених до **ресурсів** (resource-based), бо в політиці користувача «кому» і так очевидно.

ARN — повна адреса ресурсу

ARN (Amazon Resource Name) однозначно називає будь-який ресурс в AWS:

```
arn:aws:<сервіс>:<region>:<номер-акаунта>:<ресурс>
```

```
arn:aws:iam::111122223333:user/taras (IAM глобальн  
arn:aws:s3:::khmarynka-photos (bucket — ні  
arn:aws:s3:::khmarynka-photos/products/mug.jpg (об'єкт у бу  
arn:aws:ec2:eu-central-1:111122223333:instance/i-0abc123 (сервер у Фра
```

Зверни увагу на пастку з S3, яку обоюноє іспит: дія `s3:ListBucket` (переглянути список файлів) стосується **самого bucket** — ARN без `/*`. А `s3:GetObject` і `s3:PutObject` стосуються **об'єктів** — ARN з `/*`.
Переплутаєш — отримаєш `AccessDenied`.

Види політик

Вид	До чого прикріплюється	Хто керує	Коли використовувати
AWS managed	До користувачів, груп, ролей	AWS (оновлює сам)	Швидкий старт: <code>ReadOnlyAccess</code> , <code>AdministratorAccess</code> , <code>AmazonS3ReadOnlyAccess</code> , політики за посадами
Customer managed	До користувачів, груп, ролей	Ти	Точні права під свої ресурси; одна політика — для багатьох; є версії
Inline	Вбудована в одного користувача/групу/роль	Ти	Рідко: коли права мають жити і видалятися разом з цією «особою»
Resource-based	До ресурсу (bucket S3, черга SQS, ключ KMS, функція Lambda...)	Власник ресурсу	Дати доступ до ресурсу, зокрема іншим акаунтам ; має поле <code>Principal</code>

Ще кілька типів — **permission boundaries**, **SCP** і **RCP** організацій, **session policies** — не дають прав, а **обмежують** їх зверху. Про них — у розділах 9 і 10.

Як AWS вирішує: дозволити чи заборонити

Це серце IAM. Запам'ятай три правила — вони пояснюють 90% питань іспиту про доступ:

1. **За замовчуванням усе заборонено** (implicit deny). Новий користувач без політик не може нічого.
2. **Явний дозвіл** (Allow) у будь-якій доречній політиці відкриває дію.
3. **Явна заборона** (Deny) у будь-якій політиці **перемагає все**. Жоден Allow її не скасує.

1

Є явний Deny у будь-якій політиці, що стосується запиту? (SCP/RCP, політика ресурсу, межа прав, політика особи)

Так → ❌ Заборонено

↓ ні

2

Обмеження зверху дозволяють дію? SCP/RCP організації, permission boundary, session policy — якщо вони задані

Ні → ❌ Заборонено

↓ так

3

Є явний Allow у політиці особи (identity-based) або в політиці ресурсу (resource-based)?

Так → ✅ Дозволено

↓ ні

4

Неявна заборона — дозволу ніхто не давав

❌ Заборонено

У межах одного акаунта достатньо Allow або в політиці особи, або в політиці ресурсу. Для доступу з **іншого** акаунта потрібні обидва: дозвіл «на виході» (IAM-політика в акаунті А) і «на вході» (політика ресурсу або довіра ролі в акаунті В).

✚ Потренуймося

Тарас у двох групах. Група **Developers** має політику Allow s3:* на всі bucket. Група **Safety** має Deny s3:DeleteBucket на khmarynka-photos .

- Чи може Тарас завантажити файл у khmarynka-photos ? **Так** — є Allow, немає Deny на цю дію.
- Чи може Тарас видалити bucket khmarynka-photos ? **Ні** — явний Deny перемагає Allow.

- Чи може Тарас запустити сервер EC2? Ні — жодного Allow для EC2, спрацьовує неявна заборона.

Умови: коли дозвіл діє лише за певних обставин

Блок `Condition` робить політики «розумними». Найвідоміші ключі умов:

Ключ	Що перевіряє	Приклад використання
<code>aws:MultiFactorAuthPresent</code>	Чи увійшла людина з MFA	Дозволити видаляти лише з MFA
<code>aws:SourceIp</code>	IP-адреса запиту	Доступ лише з офісної мережі
<code>aws:RequestedRegion</code>	Регіон, у якому виконується дія	Працювати лише в регіонах ЄС
<code>aws:SecureTransport</code>	Чи запит по HTTPS	Заборонити незашифровані запити до S3
<code>aws:PrincipalOrgID</code>	З якої AWS Organization той, хто просить	Пускати в bucket лише акаунти своєї організації
<code>aws:PrincipalTag</code> / <code>aws:ResourceTag</code>	Теги особи й ресурсу	Доступ за тегами (ABAC)
<code>aws:CurrentTime</code>	Час запиту	Тимчасовий доступ до дати

Приклад: заборонити будь-кому видаляти об'єкти, якщо вхід виконано без MFA:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": ["s3:DeleteObject", "s3:DeleteObjectVersion"],
    "Resource": "arn:aws:s3:::khmarynka-photos/*",
    "Condition": { "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" } }
  }]
}
```



Глибше: чому часто пишуть Deny з умовою, а не Allow з умовою

Уяви, що потрібно «дозволяти видалення лише з MFA». Якщо написати Allow з умовою MFA, то будь-яка **інша** політика з простим Allow s3:* відкриє видалення і без MFA — Allow-и додаються. А Deny з умовою «без MFA» перекриває **всі** можливі Allow. Тому захисні правила («ніколи без HTTPS», «ніколи поза ЄС», «ніколи без MFA») пишуть як **Deny + умова**. Суфікс **IfExists** (`BoolIfExists`) означає: «якщо ключа в запиті немає — теж вважати умову виконаною» — інакше запити з ключами доступу (де ключа MFA немає взагалі) проскочили б повз заборону.

Принцип найменших привілеїв

Least privilege — кожна особа має рівно ті права, що потрібні для її роботи, і не більше. Стажер завантажує фото — йому не потрібен доступ до бази замовлень. Якщо його ключі вкрадуть, шкода буде мінімальною.

Як досягти цього на практиці, не витрачаючи тижні:

1. **Почни з готових політик за посадами** (AWS managed job-function policies: `ReadOnlyAccess` , `PowerUserAccess` , `Billing` , `DatabaseAdministrator` ...) — це безпечніше за `AdministratorAccess` для всіх.
2. **Звужуй за даними.** **IAM Access Analyzer** може згенерувати політику з реальної активності за журналом CloudTrail: «ось що цей користувач справді робив за 90 днів». Він же знаходить **невикористані** ролі, ключі й дозволи.
3. **Перевірй останнє використання.** Вкладка **Last accessed** (Access Advisor) показує, коли особа востаннє користувалася кожним сервісом: права на сервіси, яких не торкалися місяцями, можна прибрати.

4. **Шукай зайвий зовнішній доступ.** Access Analyzer також знаходить ресурси (bucket, ключі KMS, ролі, черги), до яких мають доступ **інші акаунти чи весь світ**.
5. **Перевір'яй політики до застосування:** валідатор політик Access Analyzer і **IAM Policy Simulator** показують, чи дозволить політика конкретну дію.
6. **Звітуй: Credential report** — таблиця всіх користувачів акаунта: чи є MFA, коли змінювали пароль, коли востаннє використовували ключі.

Облікові дані: паролі, MFA, ключі доступу

- **Політика паролів** акаунта — мінімальна довжина, складність, строк дії, заборона повторів.
- **MFA** — для кожної людини з доступом до консолі. Типи: застосунок-автентифікатор на телефоні, апаратний ключ безпеки (FIDO2/passkey), апаратний генератор кодів.
- **Access keys** — для програмного доступу. Правила:
 - ніколи не вписувати в код і не класти в git-репозиторій (боти шукають ключі в публічних репозиторіях за хвилини);
 - для програм у AWS (на EC2, Lambda, ECS) ключі **не потрібні взагалі** — використовуються **ролі**;
 - регулярно ротувати й видаляти невикористані;
 - якщо ключ «витік»: одразу **деактивувати**, створити новий, перевірити в CloudTrail, що встигли зробити зловмисники.

Admins

Тарас

AdministratorAccess + обов'язкова MFA.
Аварійний доступ, мінімум людей.

Developers

Розробники

PowerUserAccess або вужче; без керування IAM і білінгом; лише dev-ресурси.

Finance

Марта

Billing, Cost Explorer, Budgets — лише читання витрат, жодних серверів.

Warehouse

Стажер складу

Лише завантаження фото в `khmarynka-photos/products/\*`.

Права видаються групам, а не людям. Олена отримує ReadOnlyAccess: бачить усе, нічого не зламає. Пізніше все це переїде в IAM Identity Center (розділ 9).

Спробуй у справжньому AWS: адміністратор замість root і перевірка прав

Вартість: безкоштовно (IAM не тарифікується). **Час:** 25 хвилин.

1. Увійди як root → сервіс **IAM** → **User groups** → **Create group** Admins → прикріпи політику AdministratorAccess .
2. **Users** → **Create user** admin-твоєім'я → увімкни **Provide user access to the AWS Management Console** → добав у групу Admins . Збережи посилання для входу (Sign-in URL) — воно містить номер акаунта.
3. **Dashboard** → праворуч **Account Alias** → **Create**: наприклад, khmarynka-learning . Тепер вхід виглядатиме як <https://khmarynka-learning.signin.aws.amazon.com/console> .
4. Вийди з root і **увійди як новий адміністратор**. Одразу: IAM → Users → свій користувач → **Security credentials** → **Assign MFA device**.
5. Створи групу ReadOnly з політикою ReadOnlyAccess і користувача test-reader у ній (без MFA, лише для тесту).
6. У вікні інкогніто увійди як test-reader і спробуй створити bucket S3 — отримаєш помилку доступу. Це **неявна заборона** в дії.

7. Повернись під адміністратора: IAM → **Policy simulator** (посилання на сторінці політик) → обери `test-reader` → сервіс S3 → дії `CreateBucket` і `ListAllMyBuckets` → **Run simulation**: перше — `denied`, друге — `allowed`.

8. IAM → **Credential report** → **Download**: подивись таблицю користувачів і стан MFA.

Прибери за собою: видали користувача `test-reader` і групу `ReadOnly`. Адміністратора з MFA залиш — надалі працюй під ним, а `root` поклади «в сейф».

Запам'ятай

IAM — глобальний і безкоштовний; відповідає на «хто ти» і «що тобі можна».

Users — постійні облікові дані; **groups** — набори користувачів (не вкладаються, не `principal`); **roles** — тимчасові права.

Політика: **Effect, Action, Resource, Condition** (+ **Principal** у `resource-based`). `Version` — завжди `2012-10-17`.

Оцінка: **за замовчуванням заборонено** → **явний Allow дозволяє** → **явний Deny перемагає все**.

Крос-акаунтний доступ потребує дозволу з **обох** боків.

`s3:ListBucket` — на ARN bucket; `s3:GetObject / PutObject` — на `bucket/*`.

Найменші привілеї: готові політики за посадами → звуження за даними (**Access Analyzer, Last accessed**) → валідація (Policy Simulator).

Ключі доступу не зберігають у коді; програмам в AWS потрібні ролі, а не ключі.

Як питають на іспиті

Користувач має Allow у одній політиці і Deny на ту ж дію в іншій → **дію заборонено (явний Deny перемагає)**

Новий IAM-користувач без політик намагається читати S3 → **AccessDenied (неявна заборона)**

Однакові права для 20 бухгалтерів з простим додаванням нових → **IAM group з політикою**

Знайти невикористані права і згенерувати мінімальну політику з реальної активності → **IAM Access Analyzer (policy generation, unused access)**

Знайти bucket і ключі, доступні іншим акаунтам або публічно → **IAM Access Analyzer (external access)**

Дозволити небезпечну дію лише з MFA → **умова aws:MultiFactorAuthPresent (краще — Deny, якщо MFA немає)**

Доступ до API лише з офісної мережі → **умова aws:SourceIp**

Звіт про всіх користувачів, їхні MFA і вік ключів → **IAM credential report**

Застосунок на EC2 має звертатися до S3, ключі на сервері зберігати заборонено → **IAM role (instance profile), не access keys**

Пастки

Групу не можна вказати як Principal у політиці ресурсу і не можна вкласти в іншу групу.

"Version": "2012-10-17" — це версія мови, а не дата. Не змінюй її.

Allow з умовою MFA не захистить, якщо інша політика дає простий Allow — захисні правила пишуть як Deny.

Розшарені облікові записи («один логін на відділ») руйнують аудит: у CloudTrail не видно, хто саме діяв.

✓ Перевір себе

ПИТАННЯ 1

Користувача додали у дві групи: перша дає `Allow ec2:*`, друга — `Deny ec2:TerminateInstances`. Що він зможе робити з серверами EC2?

- A** Усе, включно з видаленням серверів, бо `Allow ec2:*` ширший
- B** Лише те, що дозволено в першій групі за часом додавання
- C** Нічого, бо конфлікт політик блокує весь сервіс
- D** Усе, крім видалення (термінації) серверів

Правильна відповідь: D. Права з усіх політик об'єднуються, а явний Deny завжди перемагає Allow. Тому все з `ec2:*` дозволено, крім `TerminateInstances`.

ПИТАННЯ 2

Політика дає `Allow s3:ListBucket` і `s3:GetObject` з ресурсом `arn:aws:s3:::khmarynka-photos/*`. Користувач може завантажувати файли, але не бачить список об'єктів. Чому?

- A** ListBucket діє на ARN самого bucket (без /*), а в політиці вказано лише ARN об'єктів
- B** ListBucket не підтримується в IAM-політиках
- C** Потрібна окрема політика на рівні регіону
- D** ListBucket вимагає MFA

Правильна відповідь: A. Дія `s3:ListBucket` стосується bucket, тому ресурс має бути `arn:aws:s3:::khmarynka-photos`. Дії над об'єктами (`GetObject`, `PutObject`) — на `arn:aws:s3:::khmarynka-photos/*`. Часто в політиці вказують обидва ARN.

ПИТАННЯ 3

Служба безпеки хоче знайти в акаунті IAM-ролі й дозволи, якими ніхто не користувався останні 90 днів, і прибрати зайве з найменшими зусиллями. Що використати?

- A** Amazon Inspector
- B** Вручну переглянути журнали CloudTrail за 90 днів
- C** AWS Config з власними правилами
- D** IAM Access Analyzer (unused access) і дані last accessed

Правильна відповідь: D. IAM Access Analyzer виявляє невикористані ролі, ключі й дозволи, а Last accessed показує, коли сервіси використовувались востаннє. Inspector шукає вразливості ПЗ, Config — відповідність конфігурацій правилам.

ПИТАННЯ 4

Як найнадійніше гарантувати, що об'єкти в bucket не можна видалити без MFA, навіть якщо користувачеві дали ширші дозволи іншими політиками?

- A** Прибрати всі політики з користувачів
- B** Увімкнути шифрування bucket
- C** Додати Deny s3:DeleteObject з умовою BoolIfExists
aws:MultiFactorAuthPresent = false
- D** Додати Allow s3:DeleteObject з умовою MFA

Правильна відповідь: C. Allow з умовою не допоможе, якщо інша політика дає простий Allow — дозволи додаються. Deny з умовою «немає MFA» перекриває всі Allow. IfExists закриває і запити без ключа MFA (наприклад, з access keys).

ПИТАННЯ 5

Застосунку на сервері EC2 потрібен доступ до таблиці DynamoDB. Яке рішення найбезпечніше?

- A** Зробити таблицю публічною
- B** Використати access keys root user
- C** Прикріпити до сервера IAM-роль з потрібними правами (instance profile)
- D** Створити IAM-користувача і записати його access keys у конфігурацію застосунку

Правильна відповідь: C. Роль, прикріплена до EC2 через instance profile, дає застосунку тимчасові облікові дані, що автоматично оновлюються. Жодних довгострокових ключів на сервері — нічого вкрасти.

ПИТАННЯ 6

Команда з 15 аналітиків має отримати однакові права лише на читання, а нових аналітиків треба додавати швидко. Що зробити?

- A** Створити групу з потрібною політикою і додавати туди користувачів
- B** Прикріпити inline-політику кожному користувачу окремо
- C** Дати всім AdministratorAccess, щоб не витратити час
- D** Створити одного спільного користувача для всіх аналітиків

Правильна відповідь: A. Групи — стандартний спосіб видати однакові права багатьом людям. Спільний обліковий запис руйнує аудит, а AdministratorAccess порушує принцип найменших привілеїв.

ПИТАННЯ 7

Що з цього можна вказати як Principal у resource-based політиці bucket S3? (Оберіть 2)

Оберіть 2 відповіді

- ☐ A Увесь AWS-акаунт за номером
- ☐ B Регіон AWS
- ☐ C Security Group
- ☐ D IAM-групу
- ☐ E IAM-роль в іншому акаунті

Правильна відповідь: A, E. Principal — це той, хто робить запит: акаунт, IAM user, роль, сервіс AWS, федеративний користувач. IAM-група не є principal, а регіон і Security Group — не особи.



Ролі, тимчасові ключі і вхід через корпоративні акаунти

🎯 Безпека · 30%



Історія Хмаринки. Тарас ледь не зробив класичну помилку: щоб сайт міг читати фото з S3, він збирався вписати ключі доступу прямо в конфігурацію сервера. Потім з'явилися ще три задачі. Бухгалтерська фірма-партнер хоче щомісяця забирати звіти з акаунта Хмаринки. Співробітників стає більше, і всі хочуть входити своїм робочим акаунтом Microsoft, а не окремим паролем AWS. А мобільний застосунок має пускати **покупців**, яких тисячі. Усі чотири задачі розв'язує одна ідея: **тимчасові права замість постійних ключів**.



Образ. Роль — це форма з бейджем, яку видають на рецепції на один день. Одягнув форму «Склад» — проходиш на склад, але поки ти в ній, доступу до бухгалтерії немає. Форма діє кілька годин, потім сама «розчиняється» — вкрасти її безглуздо. Хто має право отримати форму, записано в журналі рецепції (**trust policy**), а куди вона пускає — на бейджі (**permissions policy**). А видає форми вахтер **STS**.

IAM-роль: права без постійних ключів

IAM role — особа в акаунті з набором прав, але **без пароля і без постійних ключів**. Роль не належить комусь конкретному: її **беруть на себе** (assume) ті, кому це дозволено, і отримують **тимчасові облікові дані**.

Кожна роль має дві політики:

- 1. **Trust policy (політика довіри)** — хто може взяти роль: сервіс AWS (EC2, Lambda), користувач чи роль іншого акаунта, федеративний користувач.
- 2. **Permissions policy (політика дозволів)** — що можна робити в ролі.

Приклад trust policy, що дозволяє серверам EC2 брати роль:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

STS — вахтер, що видає тимчасові ключі

AWS STS (Security Token Service) видає тимчасові облікові дані: **Access Key ID**, **Secret Access Key** і **Session Token** з часом дії — від 15 хвилин до кількох годин (для ролі типово 1 година, максимум — до 12 годин за налаштуванням ролі). Коли строк мине, ключі стають непотрібними, а SDK сам отримує нові.

Головні виклики STS:

Виклик	Хто використовує	Приклад
AssumeRole	Користувачі й ролі (свого чи іншого акаунта), сервіси	Перемкнутися в роль адміністратора прод-акаунта
AssumeRoleWithSAML	Співробітники через корпоративний каталог (SAML 2.0)	Вхід через Active Directory / Entra ID
AssumeRoleWithWebIdentity	Застосунки з токеном OIDC (Google, Facebook, Kubernetes)	Pod у EKS отримує права AWS; для мобільних застосунків AWS радить Cognito
GetSessionToken	IAM user, що підтверджує MFA	Тимчасові ключі «з MFA» для CLI

1. Хто просить

Сервер EC2, функція Lambda, користувач іншого акаунта, співробітник через SSO



2. STS перевіряє

Чи дозволяє

trust policy

ролі цьому principal брати роль (і чи дозволено це самому principal)



3. Тимчасові ключі

Access Key + Secret + Session Token, діють 15 хв – 12 год



4. Виклики AWS

З правами

permissions policy

ролі. Усе пишеться в CloudTrail з назвою ролі і сесії

Взявши роль, ти тимчасово «відкладаєш» свої власні права і працюєш лише з правами ролі.

Ролі для сервісів AWS

Найчастіше роль потрібна **програмі**, що працює в AWS:

- **EC2 — instance profile.** Роль «прикріплюють» до сервера через instance profile. Застосунок на сервері бере тимчасові ключі з **сервісу метаданих інстансу** (IMDS) — SDK робить це автоматично. Ключі ротуються самі. Використовуй **IMDSv2** (з токенами сесії) — він захищає від атак, коли зловмисник змушує застосунок звернутися до адреси метаданих.
- **Lambda — execution role.** Права функції: читати чергу, писати в базу, писати логи.

- **ECS — task role.** Права конкретного контейнера (окремо від **task execution role**, якою ECS тягне образ і пише логи).
- **Service-linked roles** — ролі, які сервіси AWS створюють для себе самі (наприклад, для Auto Scaling). Їх не редагують.

Приклад з Хмаринки: сайт на EC2 має роль `web-server-role` з дозволом `s3:GetObject` на `khmarynka-photos/*`. Жодних ключів у конфігурації — нічого красти.

Доступ між акаунтами

Партнерська бухгалтерська фірма (акаунт `444455556666`) має забирати звіти з bucket Хмаринки (`111122223333`). Два способи:

Спосіб 1: роль в акаунті Хмаринки. Створюємо роль `partner-reports-reader`, у trust policy довіряємо акаунту партнера, у permissions policy — читання звітів. Партнер у своєму акаунті дозволяє своїм людям `sts:AssumeRole` на ARN цієї ролі. Потрібні дозволи з **обох боків**.

Спосіб 2: resource-based політика на самому bucket — дозвіл `principal`-у з акаунта партнера. Партнер звертається до bucket зі своїми правами, не беручи роль.

	Роль (AssumeRole)	Політика ресурсу
Для яких сервісів	Будь-яких	Лише тих, що мають resource-based політики: S3, SQS, SNS, KMS, Lambda, Secrets Manager, ECR...
Права того, хто звертається	Тимчасово замінюються правами ролі	Лишаються його власними (зручно, коли треба одночасно працювати з ресурсами в обох акаунтах)

Проблема «заплутаного заступника» і ExternalId

Уяви SaaS-сервіс аналітики, що працює з акаунтами сотень клієнтів через ролі. Зловмисник, теж клієнт цього сервісу, вказує в налаштуваннях **ARN ролі Хмаринки** — і сервіс «з добрих намірів» отримує доступ до чужих

даних. Це **confused deputy**. Захист — **ExternalId**: секретний ідентифікатор, який сервіс видає кожному клієнту і який перевіряється в trust policy ролі:

```
"Condition": { "StringEquals": { "sts:ExternalId": "khmarynka-7f3a91" } }
```

Для сервісів AWS, що звертаються до твоїх ресурсів (наприклад, SNS пише в чергу), та сама проблема розв'язується умовами `aws:SourceArn` і `aws:SourceAccount`.

Межі прав (permission boundaries)

Тарас хоче, щоб розробники самі створювали ролі для своїх функцій Lambda — без нього. Але якщо розробник може створювати ролі, він може створити роль з `AdministratorAccess` і взяти її — ескалація привілеїв.

Permission boundary — політика-«стеля», прикріплена до користувача чи ролі: **ефективні права = перетин** звичайної політики і межі. Межа сама нічого не дозволяє — лише обмежує.

Рецепт делегування: розробникам дозволено `iam:CreateRole` **лише за умови**, що до нової ролі прикріплено визначену permission boundary (умова `iam:PermissionsBoundary`). Тепер будь-яка роль, створена розробником, не перевищить стелі — навіть якщо він прикріпить їй `AdministratorAccess`.



Identity policy

Що дозволено: S3, DynamoDB, IAM (повністю)



Permission boundary

Стеля: S3, DynamoDB, Lambda



Ефективні права

Лише S3 і DynamoDB
IAM відрізано межею, Lambda не дозволено політикою

Межі, SCP і session policies працюють однаково: вони звужують, але ніколи не розширюють права.

АВАС: доступ за тегами

Класичний підхід — **RBAC** (за ролями): для кожного проєкту — своя політика. Коли проєктів сотні, політик стає забагато. **АВАС** (attribute-based access control) — одна політика на всіх, що порівнює **теги** особи і ресурсу: «дозволити керувати серверами, якщо тег `Team` сервера збігається з тегом `Team` користувача».

```
"Condition": { "StringEquals": { "aws:ResourceTag/Team": "${aws:Principa
```

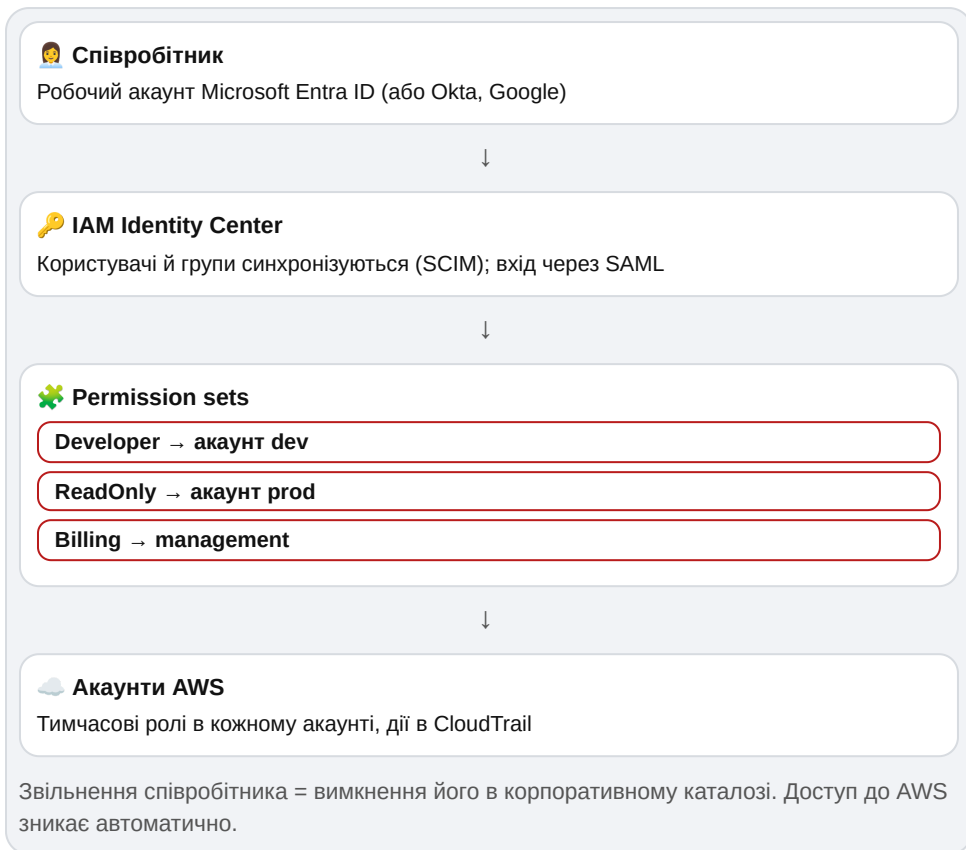
Новий проєкт — нові теги, а політика та сама. На іспиті: «масштабоване керування доступом для багатьох команд без постійного редагування політик» → АВАС.

Співробітники: єдиний вхід і федерація

IAM Identity Center

AWS IAM Identity Center (колишня назва — AWS Single Sign-On) — рекомендований спосіб впускати **співробітників** в AWS:

- **один вхід** (портал доступу) у **всі акаунти** організації і в бізнес-застосунки (Salesforce, Microsoft 365 тощо);
- **джерело користувачів** — на вибір: вбудований каталог Identity Center, **Active Directory** (через AWS Managed Microsoft AD або AD Connector) або **зовнішній провайдер** (Microsoft Entra ID, Okta, Google Workspace) через **SAML 2.0**, з автоматичною синхронізацією користувачів і груп через **SCIM**;
- **permission sets** — шаблони прав («Developer», «ReadOnly», «Billing»), які Identity Center перетворює на ролі в потрібних акаунтах;
- у консоль і CLI люди отримують **тимчасові** облікові дані — жодних постійних ключів.



SAML-федерація напряму з IAM

Старіший спосіб: у кожному акаунті створюють **SAML identity provider** і ролі; корпоративний каталог (наприклад, AD FS) видає SAML-підтвердження, а STS міняє його на тимчасові ключі (`AssumeRoleWithSAML`). Працює, але для багатьох акаунтів Identity Center значно простіший.

AWS Directory Service: Active Directory в AWS

Багато компаній живуть на **Microsoft Active Directory** (AD). AWS пропонує три варіанти:

Варіант	Що це	Коли обирати
AWS Managed Microsoft AD	Справжній Microsoft AD, керований AWS, у двох зонах	Потрібен повноцінний AD в AWS; довіра (trust) з локальним AD; застосунки, що потребують AD (FSx for Windows, RDS for SQL Server з Windows-автентифікацією, WorkSpaces)
AD Connector	Проксі, що пересилає запити до твого локального AD; нічого не зберігає в AWS	Використати наявний on-prem AD без копіювання даних у хмару
Simple AD	Недорогий AD-сумісний каталог (на базі Samba)	Малі середовища з базовими функціями, без довіри з іншими AD

IAM Roles Anywhere

Сервер у **власному** дата-центрі Хмаринки (складська програма) має писати в S3. Замість постійних ключів — **IAM Roles Anywhere**: сервер доводить свою особу **сертифікатом X.509** (від корпоративного центру сертифікації чи AWS Private CA) і отримує тимчасові облікові дані ролі.

Клієнти застосунку: Amazon Cognito

Співробітників — десятки, а покупців мобільного застосунку — тисячі. Для **клієнтів** застосунків є **Amazon Cognito**. У нього дві окремі частини, і їх постійно плутають:

	User pool — «хто ти»	Identity pool — «ключі до AWS»
Що робить	Реєстрація і вхід користувачів: email і пароль, MFA, відновлення пароля, вхід через Google/Facebook/Apple або корпоративний SAML/OIDC	Міняє токен входу (з user pool, соцмережі, SAML) на тимчасові AWS-credentials через STS
Що видає	JWT-токени (ID, access, refresh)	Тимчасові ключі ролі IAM
З чим інтегрується	Application Load Balancer і API Gateway перевіряють токени	Прямі виклики S3, DynamoDB тощо з мобільного застосунку
Особливості	Готові сторінки входу, тригери Lambda (наприклад, перевірка при реєстрації)	Гостьовий (неавтентифікований) доступ, різні ролі для різних груп

Покупець

Входить через email або Google



Cognito User Pool

Перевіряє пароль / соцмережу, видає JWT-токен



Cognito Identity Pool

Міняє токен на тимчасові ключі ролі IAM



S3

Покупець вантажить фото відгуку лише у свій префікс

У політиці ролі змінна `${cognito-identity.amazonaws.com:sub}` обмежує кожного користувача власною «папкою» в bucket.

Не плутай

User pool = паспортний стіл (посвідчує особу). **Identity pool** = бюро перепусток (видає ключі до будівлі AWS). **Identity Center** = для співробітників у акаунтах AWS; **Cognito** = для клієнтів твого застосування.

Спробуй у справжньому AWS: візьми роль і подивись на тимчасові ключі

Вартість: безкоштовно. **Час:** 15 хвилин.

1. Увійди як адміністратор (не root) → **IAM** → **Roles** → **Create role**.
2. Trusted entity: **AWS account** → **This account**. Увімкни **Require MFA**.
Next.

3. Дозволи: `ReadOnlyAccess` . Назва полі: `auditor-readonly` . Create.
4. Відкрий роль і скопіюй посилання **Link to switch roles in console** — відкрий його. Ти в консолі від імені ролі (назва ролі видно вгорі праворуч). Спробуй створити bucket S3 — отримаєш відмову: права ролі лише на читання, а свої права адміністратора ти тимчасово «відклав».
5. Натисни **Switch back**, відкрий **CloudShell** і виконай (підстав свій номер акаунта):

```
aws sts get-caller-identity
aws iam get-role --role-name auditor-readonly --query "Role.AssumeRolePolicyDocument"
```

Друга команда покаже trust policy ролі.

1. Подивись у **CloudTrail** → **Event history** події `SwitchRole` і `AssumeRole` — видно, хто і коли брав роль.

Прибери за собою: IAM → Roles → видали `auditor-readonly` .

Запам'ятай

Роль = trust policy (хто може взяти) + permissions policy (що можна).
Облікові дані — **тимчасові**, від STS.

EC2 → **instance profile**, **Lambda** → **execution role**, **ECS** → **task role**.
Ключі на серверах і в коді — ніколи.

Взявши роль, principal працює лише з її правами; resource-based політика дозволяє зберегти власні.

Крос-акаунтний доступ: роль з trust policy для іншого акаунта **або** resource-based політика; дозволи потрібні з **обох** боків.

ExternalId — захист від confused deputy для сторонніх сервісів.

Permission boundary — стеля прав; делегування створення ролей без ескалації.

ABAC — доступ за тегами, одна політика на багато команд.

Identity Center — співробітники, багато акаунтів, SSO, permission sets, зовнішній IdP через SAML + SCIM.

Cognito User Pool — вхід клієнтів (JWT); **Identity Pool** — тимчасові AWS-ключі клієнтам.

Managed Microsoft AD — справжній AD з trust; **AD Connector** — проксі до on-prem AD; **Simple AD** — дешевий базовий.

Як питають на іспиті

Застосунок на EC2 має читати S3 без збереження ключів → **IAM role через instance profile**

Надати іншому акаунту доступ до ресурсів → **IAM role з trust policy для того акаунта (sts:AssumeRole) або resource-based policy**

Сторонній SaaS-вендор отримує доступ через роль, треба захист від confused deputy → **умова sts:ExternalId у trust policy**

Розробники мають створювати ролі, але не ширші за певні права → **permission boundary**

Сотні команд, доступ до ресурсів своєї команди без окремої політики на кожну → **ABAC (теги + умови)**

Єдиний вхід співробітників у багато акаунтів через Okta / Entra ID → **IAM Identity Center + зовнішній IdP (SAML, SCIM)**

Реєстрація і вхід користувачів мобільного застосунку, вхід через соцмережі → **Cognito User Pool**

Користувачі застосунку мають напряму вантажити файли в S3, кожен у свою папку → **Cognito Identity Pool + політика з cognito-identity sub**

Використати наявний on-prem Active Directory без копіювання в AWS → **AD Connector**

Повноцінний AD в AWS з довірою до локального AD → [AWS Managed Microsoft AD](#)

Локальні сервери мають отримувати тимчасові AWS-ключі без збережених секретів → [IAM Roles Anywhere](#)

ALB має пускати до застосунку лише автентифікованих користувачів → [ALB authentication з Cognito User Pool](#)

Пастки

Cognito **User Pool** не видає AWS-ключів — для прямого доступу до S3/DynamoDB потрібен **Identity Pool**.

Identity Center — для співробітників і акаунтів AWS; **Cognito** — для клієнтів застосунку. Не навпаки.

Permission boundary і SCP **не дають** прав — лише обмежують.

Для крос-акаунтного доступу мало дозволу лише на одному боці.

Instance profile ≠ access keys на сервері: навіть «зашифровані» ключі на EC2 — гірше за роль.

✓ Перевір себе

ПИТАННЯ 1

Функція Lambda має записувати дані в таблицю DynamoDB. Як правильно надати їй права?

- A** Зберегти access keys IAM-користувача в змінних середовища функції
- B** Прикріпити до функції Security Group
- C** Зробити таблицю DynamoDB публічною
- D** Надати execution role функції політику з дозволом на запис у таблицю

Правильна відповідь: D. Execution role — роль, яку Lambda бере під час виконання; права задаються її політикою, облікові дані тимчасові й керуються AWS. Ключі в змінних середовища — довгострокові секрети, яких треба уникати.

ПИТАННЯ 2

Аудиторська компанія (окремий акаунт AWS) має регулярно читати журнали з bucket компанії. Потрібне рішення без передачі довгострокових ключів. Що обрати?

- A** Скопіювати журнали в акаунт аудиторів вручну щомісяця
- B** Зробити bucket публічним на час аудиту
- C** Створити в акаунті компанії роль з trust policy для акаунта аудиторів і правами на читання bucket
- D** Створити IAM-користувача для аудиторів і передати їм ключі

Правильна відповідь: C. Крос-акаунтна роль дає тимчасові облікові дані через STS AssumeRole. Акаунт аудиторів також має дозволити своїм людям sts:AssumeRole на ARN ролі. Жодних постійних ключів і публічного доступу.

ПИТАННЯ 3

SaaS-компанія працює з акаунтами багатьох клієнтів через IAM-ролі. Як захистити клієнтів від атаки confused deputy?

- A** Дозволити AssumeRole лише з MFA
- B** Використовувати лише resource-based політики
- C** Вимагати в trust policy ролі умову sts:ExternalId з унікальним значенням для кожного клієнта
- D** Обмежити тривалість сесії 15 хвилинами

Правильна відповідь: C. ExternalId — секретний ідентифікатор, який вендор видає кожному клієнту. Навіть знаючи ARN чужої ролі, інший клієнт не зможе змусити вендора взяти її без правильного ExternalId.

ПИТАННЯ 4

Керівник розробки хоче, щоб розробники самі створювали IAM-ролі для своїх застосунків, але щоб ці ролі ніколи не мали прав ширших за визначений набір. Що використати?

- A** Окрему IAM-групу для кожної ролі
- B** Permission boundary, обов'язкову для ролей, які створюють розробники
- C** Inline-політику з Deny на всі дії IAM
- D** Service control policy на рівні ролі

Правильна відповідь: B. Дозвіл iam:CreateRole з умовою, що до ролі прикріплена певна permission boundary, дає делегувати створення ролей без ризику ескалації: ефективні права ролі не перевищать меж.

ПИТАННЯ 5

Мобільний застосунок має дозволяти користувачам входити через Google і потім напряму завантажувати фото в S3, кожен — лише у свій префікс. Які компоненти потрібні? (Оберіть 2)

Оберіть 2 відповіді

- A** Amazon Cognito Identity Pool для видачі тимчасових AWS-credentials з обмеженням префіксу
- B** IAM-користувач для кожного покупця
- C** Amazon Cognito User Pool для входу через Google
- D** AWS Managed Microsoft AD
- E** IAM Identity Center для кожного покупця

Правильна відповідь: A, C. User Pool автентифікує користувачів (зокрема через Google), Identity Pool міняє токен на тимчасові ключі ролі; політика з `{cognito-identity.amazonaws.com:sub}` обмежує префікс. Identity Center і IAM-користувачі — не для тисяч клієнтів.

ПИТАННЯ 6

Компанія хоче, щоб співробітники входили в 30 акаунтів AWS через наявний Okta, з правами за групами і без IAM-користувачів у кожному акаунті. Що обрати з найменшими зусиллями?

- A** SAML-провайдер і ролі вручну в кожному з 30 акаунтів
- B** AD Connector до Okta
- C** Cognito User Pool з федерацією Okta
- D** IAM Identity Center з Okta як зовнішнім провайдером і permission sets

Правильна відповідь: D. Identity Center централізовано керує доступом співробітників до багатьох акаунтів: Okta як джерело (SAML + SCIM), permission sets перетворюються на ролі в акаунтах. Ручна SAML-федерація в кожному акаунті — багато роботи.

ПИТАННЯ 7

Компанія має локальний Active Directory і хоче, щоб співробітники входили в AWS зі своїми обліковими даними AD, не копіюючи каталог у хмару. Що обрати?

- A** AD Connector
- B** AWS Managed Microsoft AD з окремою копією всіх користувачів
- C** Simple AD
- D** Amazon Cognito Identity Pool

Правильна відповідь: A. AD Connector — проксі, що перенаправляє запити автентифікації до локального AD і нічого не зберігає в AWS. Simple AD — окремий незалежний каталог, Managed AD — повноцінний AD у хмарі.



Багато акаунтів — AWS Organizations, SCP, Control Tower

🎯 Безпека · 30%

🎯 Вартість · 20%



Історія Хмаринки. Хмаринка росте. Стажер у тестовому середовищі випадково видалив базу — добре, що тестову, а не бойову: обидві жили в одному акаунті. Тепер Тарас хоче розділити: **prod** окремо від **dev**, журнали безпеки — там, де їх ніхто не зможе стерти, а для експериментів — «пісочниця» з лімітом витрат. І ще правило від юриста: жодних ресурсів поза ЄС, хоч би хто що клацнув. Як керувати п'ятьма (а згодом — п'ятдесятьма) акаунтами як одним цілим? Для цього є **AWS Organizations**.



Образ. Корпорація з окремими філіями. Кожна філія (акаунт) має свій бюджет, персонал і сейф, і пожежа в одній не перекидається на інші. Над ними — головний офіс (management account), який оплачує всі рахунки разом (оптом дешевше) і видає **внутрішній статут (SCP)**: «жодна філія не відкриває представництво поза ЄС». Статут нікого не наймає і нічого не дозволяє — він лише визначає, чого не можна робити навіть директору філії.

Навіщо багато акаунтів

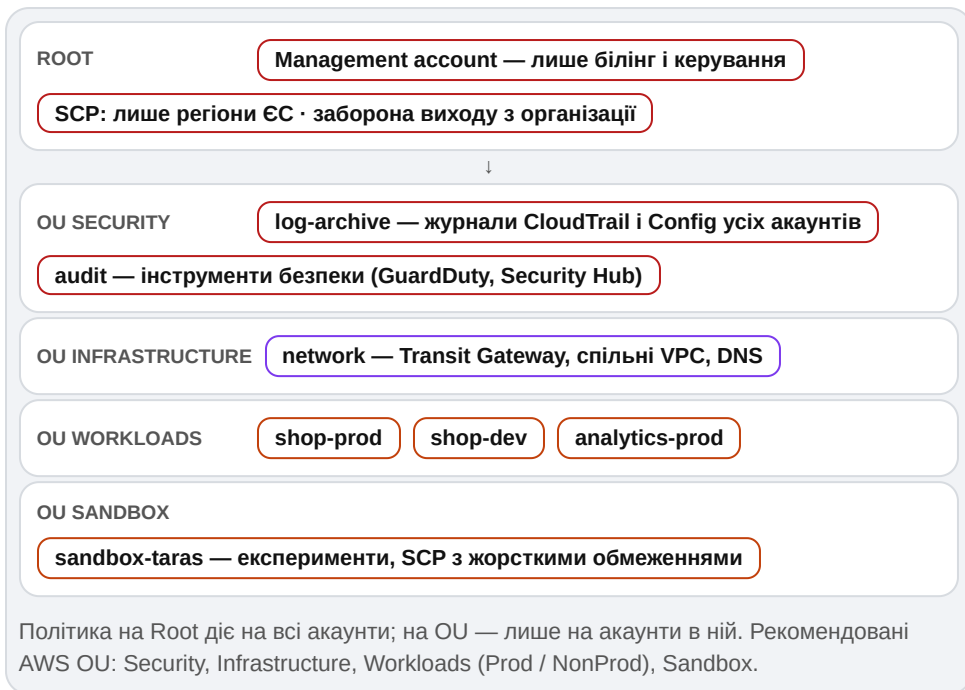
Один акаунт на все — як один спільний сейф для всієї компанії. AWS рекомендує **стратегію багатьох акаунтів**, бо акаунт — найсильніша межа ізоляції в AWS:

- **Безпека і «радіус ураження».** Помилка чи злам у dev не зачепить prod.
- **Різні правила.** У пісочниці можна експериментувати, у prod — лише через перевірені конвеєри.
- **Прозорі витрати.** Рахунок кожного акаунта — окремий рядок: видно, скільки коштує кожен проєкт.
- **Квоти.** Ліміти сервісів — на акаунт: навантаження одного проєкту не «з'їсть» квоти іншого.
- **Аудит.** Журнали безпеки — в окремому акаунті, куди мають доступ лише кілька людей.

AWS Organizations

AWS Organizations об'єднує акаунти в одну структуру з централізованим керуванням. Сервіс безкоштовний.

- **Management account** (раніше — master, payer) — акаунт, що створив організацію. Він платить за всіх, керує структурою і політиками. Ресурсів робочих навантажень у ньому **не тримають**.
- **Member accounts** — усі інші. Їх можна створювати прямо з Organizations (програмно, за хвилини) або запрошувати наявні.
- **Organizational Units (OU)** — «папки» для акаунтів, можуть бути вкладеними. Політики, прикріплені до OU, успадковують усі акаунти всередині.
- **Root** — вершина дерева (не плутай з root user акаунта!).



Consolidated billing: один рахунок і знижки

Усі акаунти організації отримують **один спільний рахунок**:

- витрати видно окремо по кожному акаунту;
- **обсяги сумуються** — ступінчасті знижки (наприклад, у S3 за великі обсяги) досягаються швидше;
- знижки від **Reserved Instances і Savings Plans** за замовчуванням **діляться** між акаунтами: якщо в prod зарезервований сервер простоює, знижку «підхопить» такий самий сервер у dev. (Спільне використання можна вимкнути для окремих акаунтів.)

SCP — «статут» організації

Service Control Policy (SCP) — політика, що визначає **максимум дозволів** для всіх IAM-користувачів і ролей (і навіть root user) у member-акаунтах. Ключові правила — кожне на іспиті:

1. SCP **не дає** жодних прав. Щоб дія стала можливою, потрібні і дозвіл SCP, і дозвіл IAM-політики.
2. SCP **діє на всіх** у member-акаунті, включно з адміністраторами й root user акаунта. Обійти зсередини неможливо.
3. SCP **не діє на management account** — ще одна причина не тримати там робочих ресурсів.
4. SCP не обмежують **service-linked полі** (якими сервіси AWS керують собою).
5. SCP успадковуються вниз по дереву: ефективні — **перетин** SCP усіх рівнів від Root до акаунта.

За замовчуванням до кожного рівня прикріплена SCP `FullAWSAccess` (дозволити все) — тому організація «працює як раніше», поки ти не додаси заборони. Дві стратегії:

- **Deny list** (типова): лишаємо `FullAWSAccess` і додаємо явні заборони. Простіше в підтримці.
- **Allow list**: прибираємо `FullAWSAccess` і дозволяємо лише перелічені сервіси. Суворіше, але кожен новий сервіс треба дозволяти явно.

Приклад SCP для Хмаринки — заборонити роботу поза ЄС (глобальні сервіси виключені, бо вони працюють через `us-east-1`):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyOutsideEU",
    "Effect": "Deny",
    "NotAction": ["iam:*", "organizations:*", "route53:*", "cloudfront:*"],
    "Resource": "*",
    "Condition": { "StringNotEquals": { "aws:RequestedRegion": ["eu-cent
  ]}
}
```

Інші класичні SCP:

- заборонити `organizations:LeaveOrganization` (акаунт не «втече» з-під контролю);
- заборонити вимикати CloudTrail, GuardDuty, AWS Config (`cloudtrail:StopLogging` , `cloudtrail:DeleteTrail` ...);
- заборонити створювати IAM users і access keys (yci — через Identity Center);
- заборонити дії root user у member-акаунтах (умова `aws:PrincipalArn` на `arn:aws:iam::*:root`);
- дозволити лише певні типи серверів у пісочниці (щоб ніхто не запустив дорогий GPU-сервер).

RCP — захист з боку ресурсів

Resource Control Policy (RCP) — новіший тип політик Organizations (з кінця 2024 року). Якщо SCP обмежує, що можуть робити **твої** люди й ролі, то RCP обмежує, хто може отримати доступ до **твоїх ресурсів** — навіть якщо якийсь розробник помилково відкрив bucket іншому акаунту. Класичний приклад: «до об'єктів S3, ключів KMS, секретів і черг організації не може звертатися ніхто поза організацією» (умова `aws:PrincipalOrgID`).

Разом вони будують **периметр даних** (data perimeter):

Периметр	Питання	Інструменти
Ідентичності	Мої люди звертаються лише до довірених ресурсів?	SCP з умовами на ресурси
Ресурсів	До моїх ресурсів звертаються лише довірені особи?	RCP, resource-based політики з <code>aws:PrincipalOrgID</code>
Мережі	Доступ лише з моїх мереж?	VPC endpoints, умови <code>aws:SourceVpce</code> , <code>aws:SourceIp</code>



Інші політики Organizations

- **Tag policies** — стандартизують теги (наприклад, обов'язковий тег `CostCenter` з дозволеними значеннями).

- **Backup policies** — централізовані плани резервного копіювання AWS Backup для всіх акаунтів.
- **AI services opt-out policies** — заборона AWS використовувати дані для покращення AI-сервісів.
- **Declarative policies** — «бажаний стан» для сервісів (наприклад, заборонити публічні образи серверів), що діє навіть при появі нових API.

Централізоване керування безпекою

У багатьох сервісах безпеки є режим **delegated administrator**: management account призначає окремий акаунт (наприклад, `audit`) адміністратором сервісу для всієї організації. Так вмикають **GuardDuty**, **Security Hub**, **Macie**, **Inspector**, **AWS Config**, **IAM Access Analyzer**, **Firewall Manager** — і нові акаунти підключаються автоматично.

Інші «організаційні» можливості:

- **Organization trail** у CloudTrail — один журнал API-викликів усіх акаунтів у bucket акаунта log-archive.
- **CloudFormation StackSets** — розгорнути однакові ресурси в багатьох акаунтах і регіонах.
- **Централізоване керування root-доступом** — прибрати паролі root у member-акаунтах; рідкісні привілейовані дії виконуються короткими сесіями з management account.
- Умова `aws:PrincipalOrgID` у політиках ресурсів — «пускати всі акаунти моєї організації» одним рядком, замість переліку сотні номерів.

AWS Control Tower: готова «зона приземлення»

Налаштувати все вищеописане вручну — тижні роботи. **AWS Control Tower** робить це за години за кращими практиками AWS і створює **landing zone** —

готове багатоакаунтне середовище:

- створює Organizations і базові OU (Security з акаунтами **Log Archive** і **Audit**);
- вмикає централізовані журнали CloudTrail і Config;
- налаштовує IAM Identity Center для входу;
- **Account Factory** — нові акаунти «за шаблоном» за кілька кліків (або з коду, наприклад через Terraform);
- **Controls (guardrails)** — готові правила з трьома типами:
 - **preventive** — не дають зробити заборонене (реалізовані через SCP/RCP): «не можна вимкнути CloudTrail»;
 - **detective** — виявляють порушення (через правила AWS Config): «знайти bucket без шифрування»;
 - **proactive** — перевіряють ресурси **до** створення (через хуки CloudFormation);
- **дашборд** відповідності і виявлення дрейфу (коли хтось змінив налаштування вручну).



Organizations чи Control Tower?

Organizations — конструктор: акаунти, OU, політики, білінг. **Control Tower** — готовий будинок з цього конструктора за проектом AWS: landing zone, журнали, guardrails, фабрика акаунтів. На іспиті «швидко налаштувати багатоакаунтне середовище за кращими практиками з guardrails» → **Control Tower**.

AWS RAM: ділитися ресурсами між акаунтами

Не все треба дублювати в кожному акаунті. **AWS Resource Access Manager (RAM)** дозволяє **ділитися** ресурсами з іншими акаунтами (або з усією організацією чи OU):

- **підмережі VPC — VPC sharing**: мережевий акаунт володіє VPC, а акаунти застосунків запускають у спільних підмережах свої сервери (кожен керує своїми ресурсами, але мережа одна);
- **Transit Gateway** — один хаб для VPC усіх акаунтів;
- правила **Route 53 Resolver** (гібридний DNS), prefix lists, політики Network Firewall;
- **Capacity Reservations**, License Manager, Aurora-кластери для клонування та інше.

Спробуй у справжньому AWS: перевір SCP до застосування

Вартість: безкоштовно. **Час:** 10 хвилин. Організацію створювати не треба: використаємо валідатор політик IAM Access Analyzer, який розуміє і SCP.

1. Відкрий **CloudShell** (під адміністратором).
2. Створи файл політики:

```
cat > scp.json <<'JSON'
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "DenyLeaveOrg", "Effect": "Deny", "Action": "organizat
    { "Sid": "ProtectTrail", "Effect": "Deny",
      "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail"
    ]
  }
}
JSON
```

1. Перевір її:

```
aws accessanalyzer validate-policy --policy-type SERVICE_CONTROL_PO
```

Порожній список `findings` — політика коректна.

1. Навмисно зіпсуй: заміни "Action":
"organizations:LeaveOrganization" на "Action":
"organizations:LeaveOrganisation" (британське написання) і
повтори перевірку — валідатор підкаже, що такої дії не існує.
2. Подумай: чому SCP з цими двома правилами треба прикріплювати
до **Root** або OU, а не до management account?

Прибери за собою: `rm scp.json`.

Запам'ятай

Акаунт — найсильніша межа ізоляції; AWS рекомендує багато акаунтів (prod, dev, security, log archive, sandbox).

Organizations: management account + member accounts + OU;
consolidated billing — один рахунок, сумарні обсяги, спільні RI/Savings Plans.

SCP — стеля прав для member-акаунтів; **не дає** прав; діє на всіх, включно з root акаунта; **не діє на management account**; не обмежує service-linked полі.

RCP — стеля для доступу до ресурсів (периметр даних): «ніхто поза організацією».

aws:PrincipalOrgID — пустити всю організацію одним рядком.

Delegated administrator — централізовано вмикає GuardDuty, Security Hub, Config тощо для всіх акаунтів.

Control Tower — landing zone за кращими практиками: Log Archive + Audit, Account Factory, guardrails (preventive / detective / proactive).

RAM — ділитися підмережами (VPC sharing), Transit Gateway, правилами Resolver між акаунтами.

Як питають на іспиті

Заборонити всім акаунтам в OU вимикати CloudTrail, навіть адміністраторам → **SCP з Deny**

Дозволити роботу лише в регіонах ЄС для всіх акаунтів → **SCP з умовою aws:RequestedRegion (глобальні сервіси виключити)**

SCP прикріплена до Root, але адміністратор management account усе ще може все → **SCP не діє на management account**

Один рахунок і спільні знижки на всі акаунти компанії → **Organizations consolidated billing**

Швидко створити багатоакаунтне середовище за кращими практиками, з guardrails і централізованими журналами → **AWS Control Tower**

Виявляти (а не забороняти) порушення правил у всіх акаунтах → **detective controls (AWS Config) у Control Tower**

Кілька акаунтів мають запускати ресурси в одній спільній VPC → **VPC sharing через AWS RAM**

Дозволити доступ до bucket усім акаунтам організації, включно з майбутніми → **bucket policy з умовою aws:PrincipalOrgID**

Гарантувати, що ресурси організації недоступні зовнішнім особам, навіть при помилці в політиці ресурсу → **RCP**

Увімкнути GuardDuty в усіх акаунтах і автоматично — в нових → **delegated administrator в Organizations**

Обов'язковий tag CostCenter з дозволеними значеннями в усіх акаунтах → **tag policies**

Пастки

SCP — не IAM-політика: вона нічого не дозволяє, і ефективні права = перетин SCP і IAM.

Root **організації** (вершина дерева) ≠ root **user** акаунта.

SCP з allow-list без `FullAWSAccess` заблокує все, що явно не дозволено, — включно з сервісами, про які ти забув.

Control Tower **використовує** Organizations, SCP і Config — це не заміна, а надбудова.

✓ Перевір себе

ПИТАННЯ 1

У компанії 25 акаунтів в AWS Organizations. Жоден member-акаунт не повинен мати змоги вимкнути AWS CloudTrail, навіть його адміністратор. Яке рішення найефективніше?

- A** Сповіщення CloudWatch при вимкненні CloudTrail
- B** Правило AWS Config, що виявляє вимкнений CloudTrail
- C** SCP з Deny на `cloudtrail:StopLogging` і `cloudtrail>DeleteTrail`, прикріплена до Root або OU
- D** IAM-політика з Deny в кожному акаунті

Правильна відповідь: C. SCP обмежує всіх principal-ів у member-акаунтах, включно з адміністраторами і root user, і її не можна змінити зсередини акаунта. IAM-політики адміністратор акаунта може змінити, а Config і CloudWatch лише виявляють порушення.

ПИТАННЯ 2

До Root організації прикріплено SCP, що забороняє запуск серверів EC2 поза eu-central-1. Адміністратор management account запускає сервер у us-east-1. Що станеться?

- A** Запит буде відхилено SCP
- B** Сервер запуститься, бо SCP не діє на management account
- C** Запит буде відхилено, якщо в організації більше 10 акаунтів
- D** Сервер запуститься лише з MFA

Правильна відповідь: B. SCP не обмежують management account. Саме тому в ньому не тримають робочих ресурсів і суворо обмежують доступ до нього.

ПИТАННЯ 3

Користувачу в member-акаунті IAM-політика дозволяє s3:, а SCP на його OU дозволяє лише ec2:. Що він може робити?

- A** Лише S3
- B** Нічого з S3 і EC2 — потрібен дозвіл і в SCP, і в IAM
- C** Усе в S3 і EC2
- D** Лише EC2

Правильна відповідь: B. Ефективні права — перетин. S3 не дозволено SCP, EC2 не дозволено IAM-політикою. Жодна дія не має дозволу з обох боків.

ПИТАННЯ 4

Компанія хоче за кілька годин налаштувати багатоакаунтне середовище за кращими практиками AWS: централізовані журнали, акаунти для аудиту, готові guardrails і фабрику нових акаунтів. Що обрати?

- A** AWS Config conformance packs
- B** AWS Control Tower
- C** AWS Service Catalog
- D** Вручну створити акаунти й SCP в AWS Organizations

Правильна відповідь: B. Control Tower автоматизує створення landing zone: Organizations, акаунти Log Archive і Audit, CloudTrail і Config, Identity Center, Account Factory і набір preventive/detective/proactive controls.

ПИТАННЯ 5

Мережева команда хоче керувати однією VPC, а команди застосунків в інших акаунтах — запускати в її підмережах свої сервери. Що використати?

- A** Окрему копію VPC в кожному акаунті
- B** VPC sharing через AWS Resource Access Manager
- C** VPC peering між усіма акаунтами
- D** Direct Connect Gateway

Правильна відповідь: B. AWS RAM дозволяє поділитися підмережами VPC з іншими акаунтами організації. Кожен акаунт керує своїми серверами, а мережею — власник VPC.

ПИТАННЯ 6

Потрібно, щоб bucket S3 був доступний усім акаунтам організації, включно з тими, що з'являться в майбутньому, без редагування політики щоразу. Як це зробити?

- A** Створити IAM-користувача для кожного акаунта
- B** Використати в bucket policy умову `aws:PrincipalOrgID` з ідентифікатором організації
- C** Перелічити в bucket policy номери всіх акаунтів
- D** Зробити bucket публічним

Правильна відповідь: B. Умова `aws:PrincipalOrgID` дозволяє доступ будь-якому principal з вказаної організації. Нові акаунти отримують доступ автоматично.

ПИТАННЯ 7

Які твердження про SCP правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** SCP можна прикріпити до окремого IAM-користувача
- B** SCP надають користувачам дозволи, яких немає в IAM-політиках
- C** SCP застосовуються до користувачів Cognito
- D** SCP не впливають на management account
- E** SCP обмежують дозволи навіть root user member-акаунта

Правильна відповідь: D, E. SCP діють на всіх IAM-principal-ів у member-акаунтах, включно з root user, не діють на management account і прикріплюються до Root, OU або акаунтів — не до окремих користувачів. Прав вони не надають.



Шифрування, ключі, секрети й сертифікати — KMS, CloudHSM, ACM, Secrets Manager

🎯 Безпека · 30%



Історія Хмаринки. Три неприємні знахідки першого аудиту.

Перша: пароль до бази замовлень лежить відкритим текстом у файлі налаштувань на сервері — й у git-репозиторії. Друга: торік сайт «ліг» на пів дня, бо сертифікат HTTPS прострочився, а поновити забули.

Третя: юрист питає, чи зашифровані адреси й телефони клієнтів і **хто** має доступ до ключів. Цей розділ — про те, як AWS шифрує все «за замовчуванням», і про те, яку частину роботи все ж треба зробити тобі.



Образ. KMS — банківське сховище ключів з охороною і журналом відвідувань: ключі ніколи не залишають сховища, ти лише просиш «зашифруй» чи «розшифруй», і кожен візит записується. **CloudHSM** — власний сейф у банку, код від якого знаєш тільки ти (навіть банк не відчинить). **Secrets Manager** — сейф для паролів, який сам міняє шифр кожні 30 днів. **ACM** — служба, що безкоштовно видає і вчасно поновлює «паспорти» сайтів (сертифікати).

Два стани даних — два види шифрування

- **In transit** (у дорозі): дані, що летять мережею, — захищає **TLS** (HTTPS). Між браузером і балансувальником, між сервером і базою, між твоїм офісом і AWS (VPN).
- **At rest** (у спокої): дані на дисках, у базах, у S3, у бекапах — захищає шифрування ключами **KMS**.

Well-Architected вимагає **обидва**. На іспиті вимога «encrypt data at rest and in transit» — це два окремі кроки: увімкнути шифрування сховища і примусити HTTPS/TLS.

AWS KMS: керування ключами

AWS Key Management Service (KMS) — керований сервіс створення і використання ключів шифрування. Головне:

- ключі живуть у **сертифікованих апаратних модулях безпеки (HSM)** і **ніколи не покидають KMS у відкритому вигляді**;
- KMS **інтегрований** з понад сотнею сервісів AWS: увімкнув шифрування в S3, EBS, RDS, DynamoDB — вони самі звертаються до KMS;
- **кожне використання ключа записується в CloudTrail** — видно, хто, коли і яким ключем шифрував чи розшифровував;
- ключі **регіональні**: ключ у Франкфурті не працює в Ірландії (крім multi-region keys, про них нижче).

Типи ключів

Тип	Хто керує	Політика ключа	Ротація	Ціна
AWS owned keys	AWS, невидимі для тебе	—	AWS	Безкоштовно

Тип	Хто керує	Політика ключа	Ротація	Ціна
AWS managed keys (aws/s3 , aws/ebs , aws/rds ...)	AWS від твого імені, видно в акаунті	Не можна змінити	Автоматично щороку	Без щомісячної плати
Customer managed keys	Ти	Повний контроль: хто використовує, хто адмініструє	Автоматично (період від 90 до 2 560 днів, типово рік) або вручну на вимогу	~\$1 на місяць за ключ + запити

Customer managed key потрібен, коли треба: власна політика доступу до ключа; поділитися зашифрованими даними з **іншим акаунтом**; власний період ротації; можливість вимкнути чи видалити ключ; окремий ключ на кожен проєкт чи клас даних.

Ключі бувають **симетричні** (AES-256 — один ключ шифрує і розшифровує, стандарт для даних в AWS) і **асиметричні** (пари RSA/ECC — для цифрових підписів і шифрування за межами AWS публічним ключем), а також HMAC-ключі для перевірки цілісності.

Політика ключа (key policy)

Кожен ключ KMS має **key policy** — resource-based політику. Це **головний** механізм доступу до ключа: якщо key policy не дозволяє акаунту делегувати доступ через IAM, жодна IAM-політика не допоможе. Типова key policy довіряє акаунту (його root), і тоді доступ далі керується IAM-політиками. Для **доступу з іншого акаунта** потрібні обидва боки: дозвіл у key policy і IAM-політика в іншому акаунті.

Окремо в key policy розділяють **адміністраторів ключа** (можуть керувати, але не шифрувати) і **користувачів ключа** (можуть шифрувати, але не змінювати політику) — принцип розділення обов'язків.

Envelope encryption: чому KMS не шифрує великі файли напряму

KMS-операція **Encrypt** приймає не більше **4 KB** даних — достатньо для пароля, але не для фото чи бекапу бази. І ганяти гігабайти в KMS по мережі

було б повільно. Тому використовують **envelope encryption** («шифрування в конверті»):

1. GenerateDataKey

Застосунок просить KMS згенерувати

ключ даних

. KMS повертає його у двох видах: відкритим і зашифрованим ключем KMS



2. Шифрування локально

Відкритим ключем даних застосунок шифрує файл будь-якого розміру (швидко, без мережі) і

одразу стирає

відкритий ключ з пам'яті



3. Зберігання

Поруч із зашифрованим файлом зберігається

зашифрований

ключ даних — «конверт»



4. Розшифрування

KMS Decrypt розшифровує маленький ключ даних (перевіряючи права!), ним — файл

Головний ключ KMS не покидає KMS, а кожен файл має свій ключ даних. Саме так під капотом працюють S3, EBS, RDS та інші сервіси.

Корисні можливості KMS

- **Multi-Region keys** — «той самий» ключ (однаковий матеріал і ID) у кількох регіонах: дані, зашифровані у Франкфурті, розшифровуються в Ірландії без перешифрування. Корисно для DR і глобальних таблиць. Без них при копіюванні знімка в інший регіон дані перешифровуються ключем цільового регіону.

- **Видалення ключа** — лише з **періодом очікування від 7 до 30 днів** (типово 30). Поки строк не минув, видалення можна **скасувати**. Після видалення всі дані, зашифровані цим ключем, **назавжди недоступні**. Безпечніший крок — спершу **вимкнути** (disable) ключ і подивитися, чи щось не зламалося.
- **Квоти запитів**. KMS має ліміти запитів за секунду. Якщо мільйони об'єктів S3 шифруються з SSE-KMS, можна отримати `ThrottlingException`. Рішення — **S3 Bucket Keys**: S3 бере в KMS ключ рівня bucket і сам генерує з нього ключі об'єктів — запитів до KMS у сотні разів менше (і дешевше).
- **Імпорт власного ключового матеріалу** (BYOK) і **custom key stores** — ключі в твоєму кластері CloudHSM або навіть у зовнішньому менеджері ключів поза AWS.

CloudHSM: власний апаратний модуль

AWS CloudHSM — виділений (single-tenant) апаратний модуль безпеки у твоїй VPC. Відмінність від KMS — у контролі:

	AWS KMS	AWS CloudHSM
Хто володіє модулем	Спільні модулі AWS (multi-tenant)	Виділений модуль лише для тебе
Хто керує ключами й користувачами HSM	AWS керує модулями, ти — політиками ключів	Лише ти ; AWS не має доступу до ключів
Інтеграція з сервісами AWS	Нативна з сотнею сервісів	Через KMS custom key store або напямую з застосунку
Стандартні API	API KMS	PKCS#11, JCE, Microsoft CNG — для наявного ПЗ
Висока доступність	Вбудована	Сам будуєш кластер з 2+ HSM у різних AZ

	AWS KMS	AWS CloudHSM
Коли обирати	Майже завжди	Вимоги регулятора до виділеного HSM під повним контролем клієнта (FIPS 140 Level 3), SSL/TLS offload, Oracle TDE, власний центр сертифікації

Сертифікати: AWS Certificate Manager

ACM видає **безкоштовні публічні TLS-сертифікати** і **автоматично поновлює** їх (для доменів, підтверджених через DNS) — жодних прострочених сертифікатів, як торік у Хмаринці. Сертифікат ACM ставиться на **інтегровані сервіси: Elastic Load Balancing, CloudFront, API Gateway** (а також App Runner, Amplify тощо).

Правила, які перевіряє іспит:

- для **CloudFront** сертифікат має бути створений у регіоні **us-east-1** (Північна Вірджинія), бо CloudFront — глобальний сервіс; для балансувальника — у регіоні самого балансувальника;
- класичне правило: публічний сертифікат ACM **не встановлюють прямо на EC2** — TLS завершують на балансувальнику чи CloudFront (з 2025 року ACM вміє видавати платні експортовані сертифікати, але на іспиті діє класика);
- **імпортовані** в ACM сторонні сертифікати ACM **не поновлює** — стеж за строком (події ACM в EventBridge, правило AWS Config про строк дії).

Для **внутрішніх** сервісів (мікросервіси всередині VPC, взаємна TLS-автентифікація) є **AWS Private CA** — власний приватний центр сертифікації.

Секрети: Secrets Manager чи Parameter Store

Паролі до баз, ключі API партнерів, токени — **секрети**. Їх не можна тримати в коді, конфігураційних файлах чи змінних середовища відкритим текстом.

Два сервіси:

	AWS Secrets Manager	SSM Parameter Store
Для чого	Секрети, які треба регулярно міняти	Конфігурація і секрети без вимоги ротації
Автоматична ротація	Так: вбудована для RDS, Aurora, Redshift, DocumentDB; для інших — через функцію Lambda	Вбудованої немає
Шифрування	KMS	SecureString — KMS
Реплікація в інші регіони	Так	Ні
Політики доступу на самому секреті	Так (resource-based)	Лише IAM
Ціна	Плата за кожен секрет на місяць + запити	Standard-параметри безкоштовні; Advanced — платні (більший розмір, політики строку дії)
Зручності	Інтеграція з RDS (пароль майстер-користувача керується автоматично)	Ієрархія /shop/prod/db-url , версії, простий доступ з EC2, Lambda, ECS

Правило вибору на іспиті: «**rotate automatically**» → **Secrets Manager**.
«Cheapest way to store configuration / connection strings», ротація не згадується → **Parameter Store**.

Як шифрують дані основні сервіси

Amazon S3

Варіант	Хто керує ключем	Коли
SSE-S3	S3 (AES-256)	Типово для всіх нових об'єктів з 2023 року — нічого налаштовувати не треба
SSE-KMS	KMS (AWS managed або customer managed key)	Потрібен аудит використання ключа в CloudTrail, окремий контроль доступу до ключа, власна ротація
DSSE-KMS	KMS, два шари шифрування	Вимоги до двошарового шифрування

Варіант	Хто керує ключем	Коли
SSE-C	Клієнт надсилає ключ у кожному запиті, AWS його не зберігає	Ключі мають бути лише в клієнта; тільки HTTPS
Client-side	Клієнт шифрує до завантаження	S3 взагалі не бачить відкритих даних

Примусити HTTPS для bucket — bucket policy з **Deny**, якщо `aws:SecureTransport = false`. Примусити конкретний ключ KMS — Deny на `s3:PutObject`, якщо в запиті інший тип шифрування або інший ключ.

EBS, RDS, знімки

- **EBS**: увімкни **encryption by default** для регіону — усі нові диски шифруватимуться автоматично. Шифруються і диск, і знімки, і трафік між сервером і диском.
- **RDS / Aurora**: шифрування вмикається **лише при створенні бази**. Для наявної незашифрованої — **знімок** → **копія знімка з шифруванням** → **відновлення** з копії. Read replica має той самий стан шифрування, що й основна база.
- **Наявний незашифрований EBS-диск** — так само: знімок → копія з шифруванням → новий диск.
- **Поділитися зашифрованим знімком з іншим акаунтом** можна, лише якщо він зашифрований **customer managed key** (і той акаунт отримав доступ до ключа). Знімок з AWS managed key спершу копіюють, перешифрувавши своїм ключем.
- **DynamoDB** шифрує всі таблиці за замовчуванням (вибір — ключ AWS, AWS managed або customer managed).

Рішення для Хмаринки

- База замовлень (RDS) — створюється **зашифрованою** customer managed key `khmarynka-data`; стару незашифровану переносимо через копію знімка.

- Пароль до бази — у **Secrets Manager** з ротацією кожні 30 днів; застосунок читає його через роль, а не з файлу. Старий пароль, що «засвітився» в git, — одразу змінюємо.
- Фото — **SSE-S3** (достатньо); рахунки й договори — **SSE-KMS з Bucket Keys** (аудит доступу до ключа).
- HTTPS — сертифікат **ACM** на балансувальнику (region eu-central-1) і на CloudFront (**us-east-1**), поновлення автоматичне.
- Юристу — звіт із CloudTrail про використання ключа `khmarynka-data`.

Спробуй у справжньому AWS: секрет у Parameter Store і конверт KMS

Вартість: Parameter Store (Standard) — безкоштовно; ключ KMS — кілька центів за день існування. **Час:** 20 хвилин.

1. **CloudShell** → збережи зашифрований параметр (використає безкоштовний AWS managed key `aws/ssm`):

```
aws ssm put-parameter --name /khmarynka/dev/db-password --type SecureString
aws ssm get-parameter --name /khmarynka/dev/db-password
aws ssm get-parameter --name /khmarynka/dev/db-password --with-decryption
```

Без `--with-decryption` бачиш шифротекст, з ним — пароль (бо маєш права на ключ).

1. **KMS** → **Customer managed keys** → **Create key** → Symmetric, Encrypt and decrypt → alias `khmarynka-lab` → адміністратор і користувач ключа — твій адміністратор → Finish.
2. Envelope encryption вручну:

```
aws kms generate-data-key --key-id alias/khmarynka-lab --key-spec AI
wc -c datakey.enc
aws kms encrypt --key-id alias/khmarynka-lab --plaintext file://<(
```

Перша команда отримала зашифрований ключ даних («конверт»), друга — зашифрувала короткий текст прямо в KMS (до 4 KB).

1. **CloudTrail** → **Event history**: знайди події `GenerateDataKey` і `Encrypt` — хто і коли використовував ключ.

Прибери за собою: `aws ssm delete-parameter --name /khmarynka/dev/db-password`; KMS → ключ `khmarynka-lab` → **Key actions** → **Schedule key deletion** → 7 днів; `rm datakey.enc`.

Запам'ятай

Шифруємо **at rest** (KMS) і **in transit** (TLS) — це дві окремі вимоги.

KMS: регіональні ключі в HSM, інтеграція з сервісами, аудит у CloudTrail. Типи: AWS owned, AWS managed (щорічна ротація), **customer managed** (свій контроль, ротація 90–2560 днів або вручну).

Key policy — головний контроль доступу до ключа; для іншого акаунта — дозвіл у key policy + IAM.

`Encrypt` — до **4 KB**; більше — **envelope encryption** (`GenerateDataKey`).

Multi-Region keys — один ключ у кількох регіонах. **Видалення** — через 7–30 днів очікування.

Масові SSE-KMS і `ThrottlingException` → **S3 Bucket Keys**.

CloudHSM — виділений HSM, повний контроль клієнта, PKCS#11/JCE/CNG, кластер у 2+ AZ.

ACM — безкоштовні сертифікати з автопоновленням; для CloudFront — **us-east-1**; імпортовані не поновлюються.

Secrets Manager — ротація; **Parameter Store** — дешева конфігурація без ротації.

Наявні незашифровані RDS/EBS → **знімок** → **копія з шифруванням** → **відновлення**.

Як питають на іспити

Аудит, хто і коли використовував ключ шифрування даних S3 → **SSE-KMS (викилики ключа в CloudTrail)**

Ключі під виключним контролем клієнта в апаратному модулі, AWS не має доступу → **AWS CloudHSM**

Автоматично змінювати пароль до бази кожні 30 днів → **AWS Secrets Manager з ротацією**

Найдешевше зберігати рядки підключення і конфігурацію без ротації → **SSM Parameter Store**

HTTPS для ALB чи CloudFront без ручного поновлення сертифікатів → **AWS Certificate Manager**

Сертифікат ACM для CloudFront «не видно» при налаштуванні → **сертифікат треба створити в us-east-1**

Шифрувати в одному регіоні й розшифровувати в іншому тим самим ключем → **KMS Multi-Region keys**

ThrottlingException від KMS при масовому завантаженні в S3 з SSE-KMS → **S3 Bucket Keys**

Зашифрувати наявну незашифровану базу RDS → **знімок** → **копія з шифруванням** → **відновлення**

Зашифрувати файл 50 МБ ключем KMS → **envelope encryption (data key)**

Поділитися зашифрованим знімком RDS з іншим акаунтом → **customer managed key + дозвіл на ключ для того акаунта**

Ключ KMS випадково запланували на видалення → **скасувати видалення до кінця періоду очікування**

Усі нові диски EBS в регіоні мають бути зашифровані → **EBS encryption by default**

Пастки

На наявних RDS чи EBS не можна просто «увімкнути шифрування» — лише через копію знімка.

AWS managed key не можна використати для обміну знімками з іншим акаунтом і не можна змінити його політику.

Parameter Store не має вбудованої ротації — якщо в питанні «rotate», це Secrets Manager.

SSE-C працює лише через HTTPS, і якщо загубиш ключ — даних не повернеш.

Видалений ключ KMS = назавжди втрачені дані, зашифровані ним.
Спершу — disable.

Перевір себе

ПИТАННЯ 1

Регулятор вимагає журнал кожного використання ключа, яким шифруються документи клієнтів у S3, і окремий контроль, хто може цим ключем користуватися. Яке шифрування обрати?

- A** SSE-S3
- B** Шифрування не потрібне, бо S3 і так надійний
- C** SSE-KMS з customer managed key
- D** SSE-C

Правильна відповідь: C. SSE-KMS записує кожне використання ключа в CloudTrail, а customer managed key дає власну key policy — хто може шифрувати й розшифровувати. SSE-S3 не дає окремого аудиту й контролю ключа, SSE-C перекладає керування ключами на клієнта.

ПИТАННЯ 2

Компанія має незашифровану базу Amazon RDS for MySQL у продакшні. Як її зашифрувати?

- A** Увімкнути опцію шифрування в налаштуваннях бази без зупинки
- B** Зробити знімок, скопіювати його з увімкненим шифруванням і відновити нову базу з копії
- C** Створити зашифровану read replica і більше нічого не робити
- D** Увімкнути шифрування EBS за замовчуванням у регіоні

Правильна відповідь: B. Шифрування RDS задається лише при створенні. Шлях для наявної бази — знімок → копія з шифруванням → відновлення нової зашифрованої бази й перемикання застосунку. Read replica незашифрованої бази теж буде незашифрованою.

ПИТАННЯ 3

Застосунок використовує пароль до Amazon Aurora, який має автоматично змінюватися кожні 30 днів без зміни коду й без простою. Що обрати?

- A** Customer managed key у KMS
- B** SSM Parameter Store SecureString
- C** Змінну середовища на сервері
- D** AWS Secrets Manager з увімкненою ротацією

Правильна відповідь: D. Secrets Manager має вбудовану автоматичну ротацію для Aurora і RDS. Parameter Store не вміє ротувати сам, а KMS керує ключами шифрування, а не паролями баз.

ПИТАННЯ 4

Архітектор налаштовує HTTPS для CloudFront з власним доменом, але не бачить свого сертифіката ACM у списку. Сертифікат створено в eu-central-1. У чому причина?

- A** Потрібно імпортувати сертифікат у CloudFront вручну через CLI
- B** Сертифікат ще не прострочився
- C** ACM не підтримує CloudFront
- D** Для CloudFront сертифікат ACM має бути в регіоні us-east-1

Правильна відповідь: D. CloudFront — глобальний сервіс і використовує сертифікати ACM лише з us-east-1. Для балансувальника сертифікат має бути в регіоні балансувальника.

ПИТАННЯ 5

Потрібно шифрувати ключем KMS файли розміром від 10 до 500 МБ. Як це робиться?

- A** Отримати ключ даних через GenerateDataKey, зашифрувати ним файл локально і зберегти поруч зашифрований ключ даних
- B** Розбити файл на частини по 4 KB і шифрувати кожную через KMS
- C** Викликати KMS Encrypt для кожного файлу цілком
- D** Використати хешування SHA-256 замість шифрування

Правильна відповідь: A. KMS Encrypt приймає до 4 KB. Великі дані шифрують через envelope encryption: ключ даних від KMS шифрує файл локально, а сам ключ даних зберігається зашифрованим ключем KMS.

ПИТАННЯ 6

Фінансова компанія за вимогою регулятора має використовувати виділений апаратний модуль безпеки, де лише вона керує ключами, а також інтегрувати його з наявним ПЗ через PKCS#11. Що обрати?

- A** AWS KMS з AWS managed keys
- B** AWS Secrets Manager
- C** AWS CloudHSM
- D** AWS Certificate Manager

Правильна відповідь: C. CloudHSM — виділений (single-tenant) HSM у VPC клієнта: ключі й користувачі під повним контролем клієнта, стандартні інтерфейси PKCS#11, JCE, CNG. Для HA — кластер з кількох HSM у різних AZ.

ПИТАННЯ 7

Масове завантаження мільйонів об'єктів у S3 з SSE-KMS завершується помилками ThrottlingException. Як найпростіше це виправити?

- A** Створити окремий ключ KMS для кожного об'єкта
- B** Вимкнути шифрування на час завантаження
- C** Перейти на SSE-C
- D** Увімкнути S3 Bucket Keys

Правильна відповідь: D. S3 Bucket Keys зменшують кількість запитів до KMS у разі: S3 отримує ключ рівня bucket і сам генерує з нього ключі для об'єктів. Це прибирає тротлінг і знижує вартість.



Захист від атак і виявлення загроз — WAF, Shield, GuardDuty та компанія

🎯 Безпека · 30%



Історія Хмаринки. Перший розпродаж у хмарі пройшов ідеально — сайт витримав. А наступного ранку дві тривоги. Боти з тисяч IP-адрес засипали сторінку входу спробами підібрати паролі покупців, а хтось підставляв у пошук товарів дивні рядки на кшталт ' OR 1=1 - - . І ще: хтось із розробників випадково залив у публічний репозиторій ключ доступу — і вночі в акаунті з'явилися 40 потужних серверів, що майнили криптовалюту. Цей розділ — про те, як AWS захищає від атак **ззовні** і як **помічає** зловмисника, що вже всередині.



Образ. Хмаринка — торговий центр. **WAF** — охоронець на вході, що перевіряє кожного відвідувача за списком прикмет («у масці — не пускати», «з того району — не пускати», «хто заходить уже сотий раз за хвилину — зупинити»). **Shield** — дамба від повені: коли натовп у мільйон людей намагається штурмувати двері, щоб ніхто інший не пройшов. **GuardDuty** — камери відеоспостереження з розумним аналітиком, що помічає підозрілу поведінку. **Inspector** — інженер, що обходить будівлю й шукає несправні замки. **Macie** — перевіряльник, що знаходить забуті на видноті документи з персональними даними. **Security Hub** — пульт охорони, куди стікаються всі сигнали.

Від чого захищаємось: карта атак

Атака	Що відбувається	Рівень	Захист в AWS
DDoS (об'ємна)	Мільйони пакетів «забивають» канал чи сервер	L3/L4	Shield Standard (автоматично), Shield Advanced, CloudFront, Global Accelerator
HTTP flood	Шквал «нормальних» HTTP-запитів на важкі сторінки	L7	WAF (rate-based rules), Shield Advanced
SQL injection, XSS	Шкідливий код у полях форм і URL	L7	WAF (managed rules)
Боти, підбір паролів	Автоматизовані спроби входу, скрапінг цін	L7	WAF Bot Control , захист від захоплення акаунтів, CAPTCHA
Скомпрометовані ключі	Хтось діє в акаунті від твого імені	API	GuardDuty , CloudTrail, найменші привілеї
Вразливості ПЗ	Стара бібліотека з відомою дірою (CVE)	ОС/код	Inspector + патчі (Patch Manager)
Витік даних	Персональні дані лежать там, де не повинні	Дані	Macie , шифрування, Block Public Access

AWS WAF: веб-файрвол рівня 7

AWS WAF (Web Application Firewall) перевіряє **HTTP/HTTPS-запити** за правилами і вирішує: пропустити, заблокувати, порахувати або показати CAPTCHA / перевірку «ти не робот».

Де працює WAF: Amazon **CloudFront**, **Application Load Balancer**, **API Gateway** (REST API), **AWS AppSync**, **Cognito User Pool**, **App Runner**, **Amplify**. **Не** на Network Load Balancer і **не** прямо на EC2 — це улюблена пастка іспиту.

Правила збирають у **Web ACL**:

- **AWS Managed Rules** — готові набори від AWS: базовий захист від поширених атак (OWASP Top 10), SQL injection, відомі шкідливі вхідні дані, IP з поганою репутацією, анонімні проксі й VPN, правила для Linux/Windows/PHP. Є й набори від партнерів з AWS Marketplace.
- **Власні правила:** за IP-адресами (IP sets), **країною** (geo match), заголовками, розміром тіла, регулярними виразами.
- **Rate-based rules** — обмеження частоти: «не більше 500 запитів за 5 хвилин з однієї IP-адреси на /login». Головна зброя проти HTTP flood і підбору паролів.
- **Bot Control, Account Takeover Prevention, Account Creation Fraud Prevention** — платні керовані набори проти ботів і шахрайства з акаунтами.

Web ACL для Хмаринки (на CloudFront)

1. AWS Managed Rules — Core rule set і SQL database → блокують `OR 1=1 -- .`
2. IP reputation list → блокуємо відомі ботнети.
3. Rate-based rule на `/login` : понад 100 запитів за 5 хвилин з однієї IP → CAPTCHA.
4. Geo match: панель адміністратора `/admin/*` — лише з України (решта — блок).
5. Спершу всі нові правила — у режимі **Count**, щоб подивитися, чи не заблокують вони справжніх покупців, і лише потім **Block**.

AWS Shield: захист від DDoS

- **Shield Standard** — **безкоштовно й автоматично** для всіх клієнтів AWS. Захищає від найпоширеніших атак рівнів 3–4 (SYN flood, UDP reflection) на інфраструктуру AWS, зокрема CloudFront і Route 53.

- **Shield Advanced** — платна підписка (близько **\$3 000 на місяць на організацію**, з річним зобов'язанням). Дає:
 - **Shield Response Team (SRT)** — експерти AWS 24/7, які допомагають під час атаки;
 - **захист від рахунку**: компенсація витрат на масштабування, спричинене DDoS (кредити);
 - розширене виявлення і **автоматичне пом'якшення атак рівня 7** разом з WAF (WAF для захищених ресурсів без додаткової плати);
 - захист ресурсів: **CloudFront, Route 53, Global Accelerator, Elastic Load Balancing, EC2 з Elastic IP**;
 - детальні звіти й метрики атак.

Архітектурний захист від DDoS: ставити перед застосунком **CloudFront** і/або **Global Accelerator** (величезна пропускна здатність edge-мережі поглинає атаку), використовувати **Auto Scaling**, тримати сервери в **приватних підмережах** за балансувальником.

Firewall Manager, Network Firewall, DNS Firewall

- **AWS Firewall Manager** — **централізоване** керування правилами безпеки в **усіх акаунтах AWS Organizations**: web ACL WAF, Shield Advanced, **Security Groups**, Network Firewall, DNS Firewall, сторонні файрволи. Нові акаунти й ресурси отримують політики **автоматично**. Потребує Organizations і AWS Config.
- **AWS Network Firewall** — керований **stateful**-файрвол і система запобігання вторгненням (IPS) **на рівні VPC**: фільтрує весь трафік, а не лише HTTP; правила сумісні з **Suricata**; фільтрація вихідного трафіку за **доменними іменами** («сервери можуть ходити лише на *.amazonaws.com і сайт платіжного партнера»). Часто розгортається в окремій «inspection VPC» за Transit Gateway.
- **Route 53 Resolver DNS Firewall** — блокує DNS-запити з VPC до **шкідливих доменів** (готові списки AWS: малварь, ботнети) — зупиняє «дзвінок додому» зараженого сервера.



Хто на якому рівні

WAF — L7, лише веб (HTTP). **Shield** — DDoS (L3/L4, з Advanced — і L7). **Network Firewall** — увесь трафік VPC, IPS, домени. **DNS Firewall** — DNS-запити. **Security Groups / NACL** — базові порти й адреси (розділ 14). **Firewall Manager** — пульт, що розставляє все це по всіх акаунтах.

Виявлення загроз: GuardDuty

Amazon GuardDuty — «розумні камери» акаунта. Вмикається одним кліком, **без агентів** і без налаштування журналів: GuardDuty сам аналізує **CloudTrail** (API-виклики), **VPC Flow Logs** (мережевий трафік) і **DNS-журнали**, застосовуючи машинне навчання, виявлення аномалій і розвідку загроз AWS.

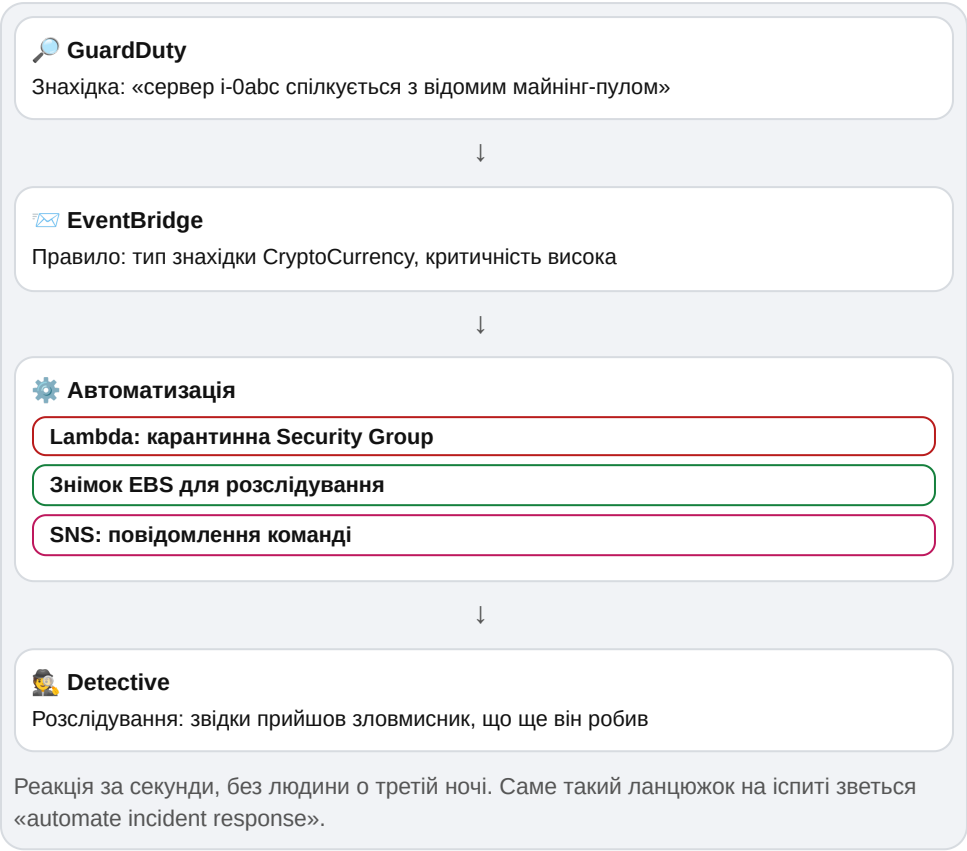
Що він знаходить:

- **скомпрометовані облікові дані**: виклики API з незвичних країн, із відомих шкідливих IP, запуск незвичних сервісів о третій ночі;
- **криптомайнінг** на серверах;
- сервер, що спілкується з **командним центром ботнету** (C&C) або сканує порти;
- спроби вимкнути журнали, змінити налаштування безпеки.

Додаткові плани захисту розширюють огляд: **S3 Protection** (підозрілий доступ до даних), **EKS і Runtime Monitoring** (контейнери й сервери зсередини), **Malware Protection** (сканування дисків EC2 і нових об'єктів у S3), **RDS Protection** (підозрілі входи в бази), **Lambda Protection**. Нові можливості поєднують окремі сигнали в **послідовності атаки** (extended threat detection).

Автоматична реакція: знахідка GuardDuty → правило **EventBridge** → функція **Lambda** або **SSM Automation**: ізолювати сервер (замінити Security

Group на «карантинну»), зробити знімок диска для розслідування, відкликати ключі, надіслати сповіщення.



Вразливості, чутливі дані, розслідування

- **Amazon Inspector** — постійне сканування вразливостей: серверів **EC2** (пакети ОС, відомі CVE; з агентом SSM або без агента), **образів контейнерів у ECR**, функцій **Lambda** (залежності й код), плюс аналіз **мережевої досяжності** («цей порт відкритий в інтернет»). Кожна знахідка отримує оцінку ризику з урахуванням середовища. Inspector **знаходить** — латає **SSM Patch Manager**.

- **Amazon Macie** — знаходить **чутливі дані в S3** за допомогою машинного навчання і шаблонів: персональні дані (імена, адреси, номери паспортів, телефони), номери банківських карток, облікові дані. А ще показує bucket, які **публічні, незашифровані чи відкриті іншим акаунтам**.
- **Amazon Detective** — **розслідування**: будує граф зв'язків з CloudTrail, VPC Flow Logs і знахідок GuardDuty, щоб знайти **першопричину** інциденту і все, до чого торкався зловмисник.



Три сервіси, які постійно плутають

GuardDuty = загрози («хтось поводить себе підозріло»). **Inspector** = вразливості («у тебе незачинене вікно»). **Macie** = чутливі дані в S3 («паспорти клієнтів лежать на видноті»). А **Detective** = «що сталося і хто винен».

Security Hub: пулт охорони

AWS Security Hub збирає знахідки з **GuardDuty, Inspector, Macie, IAM Access Analyzer, Firewall Manager, AWS Config** і партнерських рішень в **одному місці** й автоматично перевіряє акаунти на відповідність **стандартам безпеки**: AWS Foundational Security Best Practices, **CIS AWS Foundations Benchmark, PCI DSS, NIST**. Результат — оцінка безпеки і список конкретних проблем («bucket без блокування публічного доступу», «root без MFA»). Для кількох акаунтів і регіонів — агрегація в одному адміністративному акаунті.



Зміни 2025 року

У червні 2025 року AWS перейменував класичний Security Hub на **Security Hub CSPM** (перевірка налаштувань за стандартами), а назву **Security Hub** отримав новий, ширший сервіс, що корелює сигнали всіх сервісів безпеки й допомагає пріоритезувати ризики. Для іспиту суть та

сама: «одна панель стану безпеки всіх акаунтів + перевірки стандартів» → Security Hub.

Відповідність і аудит

- **AWS Artifact** — портал **звітів відповідності AWS** (SOC 1/2/3, ISO 27001, PCI DSS) і **угод** (наприклад, БАА для медичних даних за HIPAA). Аудитор просить «доказ, що дата-центри AWS сертифіковані» → Artifact.
- **AWS Audit Manager** — **автоматично збирає докази** для твоїх аудитів (GDPR, HIPAA, PCI, SOC 2): знімки налаштувань, журнали, результати перевірок — у готовий звіт.
- **AWS Config** — історія конфігурацій ресурсів і правила відповідності з автовиправленням (розділ 41).
- **CloudTrail** — хто і коли викликав API.



CloudTrail, Config, CloudWatch

CloudTrail — «хто і коли» (журнал дій). **Config** — «яким був і чи правильний» (стан ресурсів). **CloudWatch** — «як працює» (метрики, логи, аларми). Питання «хто видалив bucket?» → CloudTrail; «чи всі диски зашифровані і що змінилось учора?» → Config; «CPU сервера понад 90%» → CloudWatch.



Спробуй у справжньому AWS: увімкни «камери» акаунта

Вартість: GuardDuty і Inspector мають **30-денний безкоштовний пробний період**; після нього для навчального акаунта — вимкни. **Час:** 20 хвилин.

1. **GuardDuty** → **Get started** → **Enable GuardDuty**. Нічого більше налаштовувати не треба.

2. **Settings** → **Generate sample findings** — GuardDuty створить приклади знахідок усіх типів (позначені [SAMPLE]). Відкрий кілька: `CryptoCurrency:EC2/BitcoinTool` , `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration` — прочитай опис, рівень критичності й рекомендації.
3. **EventBridge** → **Rules** → **Create rule** → Event pattern: AWS services → **GuardDuty** → **GuardDuty Finding** → ціль: тема **SNS** зі своїм email (створи тему прямо тут або в SNS і підтверди підписку). Згенеруй зразки знахідок ще раз — лист має прийти.
4. **Inspector** → **Activate Inspector** — подивись дашборд покриття (поки що ресурсів для сканування немає; повернешся сюди після розділу про EC2).
5. **Security Hub** (або Security Hub CSPM) → увімкни стандарт **AWS Foundational Security Best Practices** — через кілька годин побачиш оцінку свого навчального акаунта.

Прибери за собою: після завершення курсу (або пробного періоду) — GuardDuty → Settings → **Suspend/Disable**; Inspector → **Deactivate**; Security Hub → **Disable**; видали правило EventBridge і тему SNS.

Запам'ятай

WAF — L7: SQL injection, XSS, боти, **rate-based rules**, geo match. Працює на **CloudFront, ALB, API Gateway, AppSync, Cognito** — не на **NLB/EC2**.

Shield Standard — безкоштовно, L3/L4. **Shield Advanced** — ~\$3 000/міс: **SRT 24/7, компенсація витрат, L7 з WAF**.

Firewall Manager — централізовані правила WAF/Shield/SG/Network Firewall в усіх акаунтах організації.

Network Firewall — stateful-файрвол/IPS для VPC, фільтрація за доменами. **DNS Firewall** — блок шкідливих доменів.

GuardDuty — загрози (ML по CloudTrail, VPC Flow Logs, DNS), без агентів. **Inspector** — вразливості (EC2, ECR, Lambda). **Macie** — чутливі дані в S3. **Detective** — розслідування.

Security Hub — єдина панель + стандарти (CIS, PCI, AWS FSBP).

Artifact — звіти AWS. **Audit Manager** — докази для твого аудиту.

Реакція на інциденти: **знахідка** → **EventBridge** → **Lambda / SSM Automation**.

Як питають на іспиті

Захистити вебзастосунок на ALB від SQL injection і XSS → **AWS WAF з managed rules на ALB**

Обмежити кількість запитів з однієї IP-адреси (HTTP flood, підбір паролів) → **WAF rate-based rule**

Заблокувати доступ до сайту з певних країн → **WAF geo match (або CloudFront geo restriction)**

DDoS-захист з експертами 24/7 і компенсацією витрат на масштабування → **AWS Shield Advanced**

Керувати правилами WAF і Security Groups у всіх акаунтах, включно з новими → **AWS Firewall Manager**

Фільтрувати вихідний трафік VPC за доменними іменами, IPS → **AWS Network Firewall**

Незвичні API-виклики, криптомайнінг, скомпрометовані ключі → **Amazon GuardDuty**

Сканувати сервери EC2 і образи ECR на відомі CVE → **Amazon Inspector**

Знайти персональні дані й номери карток у bucket S3 → **Amazon Macie**

Знайти першопричину інциденту безпеки → **Amazon Detective**

Одна панель стану безпеки всіх акаунтів і перевірки CIS/PCI → [AWS Security Hub](#)

Аудитор просить звіт SOC 2 чи PCI про інфраструктуру AWS → [AWS Artifact](#)

Автоматично ізолювати сервер при знахідці GuardDuty → [EventBridge rule](#) → [Lambda \(карантинна Security Group\)](#)

Вебзастосунок за Network Load Balancer, потрібен захист L7 → [поставити CloudFront або ALB з WAF \(WAF не підтримує NLB\)](#)

Пастки

WAF **не** ставиться на NLB і прямо на EC2.

Shield Standard уже ввімкнений у всіх — «увімкнути Shield Standard» не є окремою дією.

GuardDuty не блокує атаки сам — він **виявляє**; блокування — через автоматизацію чи інші сервіси.

Inspector **знаходить** вразливості, але не латає — латає Patch Manager.

Масіє працює з **S3** (не з базами даних і не з дисками EC2).

✓ Перевір себе

ПИТАННЯ 1

Вебзастосунок за Application Load Balancer отримує запити зі спробами SQL injection. Що найпростіше застосувати?

- A** Amazon GuardDuty
- B** Network ACL з правилами Deny для підозрілих запитів
- C** AWS Shield Standard
- D** AWS WAF web ACL з керованими правилами проти SQL injection, прикріплену до ALB

Правильна відповідь: D. SQL injection — атака рівня 7 у вмісті HTTP-запиту. Її розпізнає WAF (AWS Managed Rules). Shield — від DDoS, NACL бачить лише IP і порти, GuardDuty виявляє загрози в акаунті, а не фільтрує веб-запити.

ПИТАННЯ 2

Сторінку входу атакує бот, надсилаючи тисячі запитів на хвилину з кожної з кількох сотень IP-адрес. Як обмежити це з найменшими зусиллями?

- A** Увімкнути Shield Standard
- B** Додати кожну IP-адресу в Security Group з Deny
- C** Налаштувати WAF rate-based rule для шляху /login
- D** Збільшити кількість серверів в Auto Scaling

Правильна відповідь: C. Rate-based rule автоматично блокує (або вимагає CAPTCHA) IP-адреси, що перевищують ліміт запитів за період. У Security Group взагалі немає Deny, а масштабування лише збільшить рахунок.

ПИТАННЯ 3

Інтернет-магазин хоче захист від великих DDoS-атак з допомогою експертів AWS під час атаки і компенсацією додаткових витрат на масштабування. Що обрати?

- A** AWS WAF
- B** Amazon Inspector
- C** AWS Shield Advanced
- D** AWS Shield Standard

Правильна відповідь: C. Shield Advanced дає доступ до Shield Response Team 24/7 і захист від зростання рахунку через DDoS, а також розширене виявлення. Shield Standard — лише базовий безкоштовний захист L3/L4.

ПИТАННЯ 4

Служба безпеки хоче виявляти незвичну активність в акаунтах: виклики API з підозрілих IP, криптомайнінг на серверах, використання вкрадених ключів — без встановлення агентів. Що обрати?

- A** Amazon Macie
- B** AWS Config
- C** Amazon Inspector
- D** Amazon GuardDuty

Правильна відповідь: D. GuardDuty аналізує CloudTrail, VPC Flow Logs і DNS-журнали та виявляє загрози за допомогою ML і розвідки загроз, без агентів. Inspector шукає вразливості, Macie — чутливі дані в S3, Config — відповідність налаштувань.

ПИТАННЯ 5

Компанія повинна знайти всі об'єкти S3 з номерами банківських карток і паспортними даними клієнтів. Який сервіс це робить?

- A** AWS Security Hub
- B** Amazon GuardDuty
- C** Amazon Macie
- D** Amazon Detective

Правильна відповідь: C. Macie використовує машинне навчання і шаблони, щоб знаходити чутливі дані (PII, фінансові дані) в S3 і показувати ризикові налаштування bucket.

ПИТАННЯ 6

У компанії 60 акаунтів в AWS Organizations. Потрібно, щоб усі ALB і CloudFront у всіх акаунтах, включно з новими, мали однаковий набір правил WAF. Що використати?

- A** Налаштувати WAF вручну в кожному акаунті
- B** AWS Shield Advanced в management account
- C** SCP, що вимагає WAF
- D** AWS Firewall Manager з політикою WAF для організації

Правильна відповідь: D. Firewall Manager централізовано застосовує політики WAF, Shield Advanced, Security Groups і Network Firewall до всіх акаунтів організації і автоматично — до нових ресурсів. SCP може заборонити дії, але не налаштує правила WAF.

ПИТАННЯ 7

Потрібно автоматично ізолювати сервер EC2, щойно GuardDuty виявить на ньому криптомайнінг. Як це побудувати? (Оберіть 2)

Оберіть 2 відповіді

- A** Правило Amazon EventBridge, що реагує на знахідки GuardDuty потрібного типу
- B** Правило AWS WAF на сервері EC2
- C** Amazon Macie для сканування сервера
- D** Функція Lambda, що замінює Security Group сервера на карантинну без правил
- E** Налаштувати GuardDuty блокувати сервер самостійно

Правильна відповідь: A, D. GuardDuty лише виявляє. Знахідки надходять в EventBridge, правило запускає Lambda (або SSM Automation), яка ізолює сервер. WAF не ставиться на EC2, Macie працює з S3.



VPC з нуля — адреси, підмережі, маршрути, Internet Gateway і NAT



Безпека · 30%



Відмовостійкість · 26%



Вартість · 20%



Історія Хмаринки. Коли Тарас перевіряв старий сервер з підсобки, волосся стало дибки: у нього була публічна IP-адреса, і з інтернету були відкриті не лише сайт, а й **база даних** на порту 3306 і віддалений доступ. Будь-хто у світі міг стукати в базу замовлень — і, судячи з журналів, стукали щодня. У хмарі все буде інакше: база житиме в мережі, **куди з інтернету немає дороги взагалі**. Будуємо власну приватну мережу Хмаринки в AWS — **VPC**.



Образ. VPC — огорожений житловий комплекс, твоя приватна територія в місті-регіоні. **Підмережі** — будинки комплексу. Будинки на **першій лінії** мають вихід на вулицю (публічні підмережі), будинки у **внутрішньому дворі** виходу на вулицю не мають (приватні). **Таблиця маршрутів** — вказівники у дворі: «до вулиці — туди». **Internet Gateway** — головна брама на вулицю. **NAT Gateway** — поштове відділення на першій лінії: мешканці внутрішніх будинків відправляють листи через нього й отримують відповіді, але сторонній з вулиці через пошту у двір не пройде.

Що таке VPC

Amazon VPC (Virtual Private Cloud) — логічно ізольована мережа в AWS, яку ти повністю контролюєш: діапазон адрес, підмережі, маршрути, шлюзи,

правила файрволів. Властивості:

- VPC живе в **одному регіоні**, але охоплює **всі його зони доступності**.
- Діапазон IPv4 задається в CIDR — від **/16** (65 536 адрес) до **/28** (16 адрес). Згодом можна додати додаткові діапазони. За бажанням — ще й IPv6.
- Різні VPC ізольовані одна від одної, навіть в одному акаунті. З'єднати їх можна лише явно (розділ 15).

У кожному регіоні акаунта вже є **default VPC** (`172.31.0.0/16`) з публічною підмережею в кожній зоні — щоб новачок міг одразу запустити сервер з доступом з інтернету. Для реальних систем створюють **власні VPC** з продуманою структурою.

Підмережі: будинки комплексу

Підмережа (subnet) — частина діапазону VPC, прив'язана до **однієї зони доступності**. Ресурси (сервери, бази, балансувальники) запускаються в підмережах. Щоб система пережила втрату зони, ставлять однакові підмережі в кількох зонах.

Нагадування з розділу 3: у кожній підмережі AWS резервує **5 адрес** — першу (адреса мережі), другу (маршрутизатор VPC), третю (DNS), четверту (запас) і останню. У `/24` доступно 251 адреса.

Публічна чи приватна? Назва підмережі нічого не означає. Підмережа **публічна**, якщо в її таблиці маршрутів є маршрут `0.0.0.0/0 → Internet Gateway`. Немає такого маршруту — **приватна**. Це визначення треба знати напам'ять.

Таблиці маршрутів

Таблиця маршрутів (route table) каже, куди відправляти трафік з підмережі. Кожна підмережа прив'язана рівно до однієї таблиці (якщо не прив'язати явно — до головної, main route table).

Призначення (Destination)	Ціль (Target)	Значення
10.0.0.0/16	local	Трафік усередині VPC — є в кожній таблиці, прибрати не можна
0.0.0.0/0	igw-...	Усе інше — в інтернет через Internet Gateway (публічна підмережа)
0.0.0.0/0	nat-...	Усе інше — через NAT Gateway (приватна підмережа з виходом в інтернет)
pl-... (S3)	vpce-...	Трафік до S3 — через gateway endpoint
192.168.0.0/16	vgw-... / tgw-...	Трафік в офіс — через VPN або Transit Gateway

Правило вибору маршруту — **найспецифічніший** (longest prefix match): для адреси 10.0.5.7 маршрут 10.0.0.0/16 (local) переможе 0.0.0.0/0 , бо /16 точніший за /0 .

Internet Gateway: брама на вулицю

Internet Gateway (IGW) — компонент, що з'єднує VPC з інтернетом. Він керований AWS, горизонтально масштабується і високодоступний — за нього не треба хвилюватися, і він безкоштовний. До VPC прикріплюється **один** IGW.

Щоб сервер був доступний з інтернету, потрібні **всі чотири** умови:

1. До VPC прикріплений **Internet Gateway**.
2. У таблиці маршрутів підмережі є маршрут **0.0.0.0/0 → IGW** .
3. Сервер має **публічну IPv4-адресу** (автоматичну або Elastic IP) чи IPv6.
4. **Security Group** і **Network ACL** дозволяють потрібний трафік.

Забудеш хоча б одне — «сервер не відповідає». На іспиті це улюблена задача на діагностику.

Публічні адреси: автоматичні й Elastic IP

- **Автоматична публічна IPv4** — сервер у публічній підмережі може отримати її при запуску. Але вона **змінюється** після зупинки й запуску сервера.
- **Elastic IP (EIP)** — статична публічна IPv4, закріплена за акаунтом. Її можна перечепити на інший сервер (швидка заміна при збої). З лютого 2024 року **кожна публічна IPv4-адреса платна** (близько \$0,005 на годину, ~\$3,6 на місяць) — і використовується, і «забута» в акаунті.
- **IPv6** — безкоштовні глобально унікальні адреси, NAT для них не потрібен.

NAT Gateway: вихід в інтернет без входу

Серверам застосунку в приватних підмережах іноді потрібно **виходити** в інтернет: завантажити оновлення ОС, викликати API платіжного партнера. Але **вхідних** з'єднань з інтернету бути не повинно. Для цього — **NAT Gateway**:

- **Класичний (zonal) NAT Gateway** створюється в **публічній** підмережі конкретної зони з Elastic IP. У таблицях маршрутів **приватних** підмереж: `0.0.0.0/0 → nat-...`.
- Трафік назовні виходить з адреси NAT, відповіді повертаються, а **нові з'єднання ззовні неможливі**.
- Керований AWS, масштабується до десятків Gbps, **без Security Groups** (їх налаштовують на серверах).
- **Платний**: за кожну годину роботи і за кожен оброблений гігабайт.
- Один NAT Gateway працює в **одній зоні**. Для високої доступності — **по NAT Gateway у кожній зоні**, і приватна підмережа кожної зони маршрутизується на «свій» NAT. Інакше при втраті зони з NAT інші зони втратять вихід в інтернет, а трафік між зонами ще й коштує грошей.



Regional NAT Gateway (з листопада 2025)

AWS додав **регіональний** режим NAT Gateway: один NAT на всю VPC, що сам розширюється на потрібні зони і не потребує публічної підмережі. Це спрощує мережу. Але на іспиті частіше зустрічається класична схема «**NAT Gateway у публічній підмережі кожної AZ**» — відповідай за нею, якщо в питанні не сказано інше.

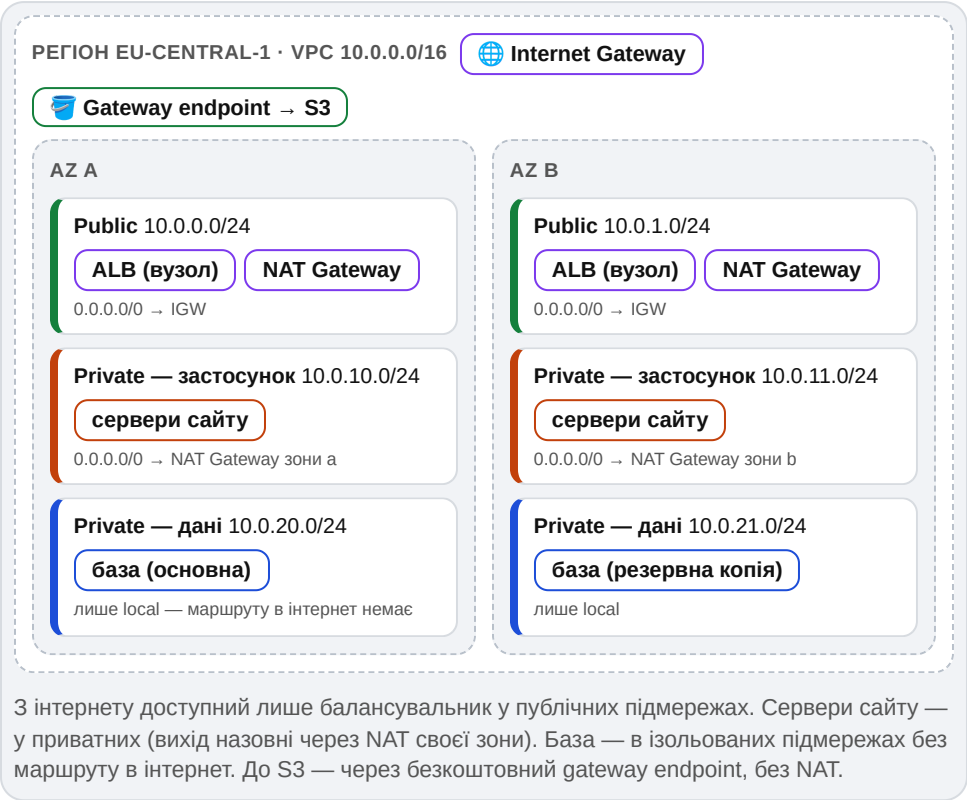
NAT instance — застарілий варіант: звичайний сервер EC2, що виконує роль NAT (треба вимкнути перевірку *Source/Destination check*). Дешевший, але ти сам відповідаєш за його доступність, масштабування й оновлення. На іспиті — майже завжди програє NAT Gateway, крім питань «найдешевше для тестового середовища, доступність не важлива».

Egress-only Internet Gateway — «NAT для IPv6»: сервери з IPv6 можуть виходити в інтернет, але ззовні з'єднання до них неможливі.

DNS усередині VPC

Кожна VPC має вбудований DNS-резолвер AWS (**Route 53 Resolver**, адреса — база VPC плюс два, наприклад `10.0.0.2`). Два налаштування VPC: **enableDnsSupport** (резолвер увімкнено) і **enableDnsHostnames** (сервери отримують DNS-імена). Обидва потрібні для приватних DNS-зон і для interface endpoints.

Мережа Хмаринки



Логіка рішень:

- **Три рівні** (публічний, застосунок, дані): кожен наступний рівень закритіший за попередній — це **глибокий захист** (defense in depth).
- **Дві зони** (для prod краще три): втрата зони не зупиняє магазин.
- **Діапазон з запасом**: `/16` на VPC і `/24` на підмережі — вистачить на роки. Між рівнями залишені «дірки» (`10.0.2-9` , `10.0.12-19`), щоб додати третю зону.
- **Без перетинів**: офіс Хмаринки має `192.168.1.0/24` , майбутня dev-VPC отримає `10.1.0.0/16` . Тоді їх можна буде з'єднати.



VPC — повністю програмна мережа: немає фізичних комутаторів, які ти налаштовуєш. Кожен сервер має **мережевий інтерфейс (ENI)** з приватною IP-адресою в підмережі; AWS на рівні своєї інфраструктури (Nitro) доставляє пакети лише туди, куди дозволяють маршрути і правила. Тому в VPC немає broadcast-трафіку, а «підслухати» чужий трафік неможливо. «Маршрутизатор VPC» (друга адреса підмережі) — теж віртуальний, і саме він застосовує таблиці маршрутів. Internet Gateway виконує трансляцію адрес 1:1: сервер «думає», що має лише приватну адресу, а IGW підміняє її на публічну при виході в інтернет.



Спробуй у справжньому AWS: побудуй VPC Хмаринки

Вартість: безкоштовно — якщо не створювати NAT Gateway (він платний погодинно). **Час:** 20 хвилин.

1. Консоль → **VPC** → **Create VPC** → **VPC and more**.
2. Назва: khmarynka . IPv4 CIDR: 10.0.0.0/16 . Зон: **2**. Публічних підмереж: **2**, приватних: **2**.
3. **NAT gateways: None** (заради економії). **VPC endpoints: S3 Gateway** (безкоштовний). Create VPC.
4. Відкрий вкладку **Resource map** своєї VPC: побачиш підмережі, таблиці маршрутів, IGW і зв'язки між ними — живу версію схеми вище.
5. **Route tables** → відкрий таблицю публічних підмереж: є 0.0.0.0/0 → igw-... . Відкрий таблицю приватної: маршруту в інтернет немає, зате є маршрут до S3 через vpcse-... .
6. **Subnets** → обери публічну підмережу → Actions → **Edit subnet settings**: подивись опцію «Enable auto-assign public IPv4 address».

7. Подумай: що треба додати, щоб сервери в приватних підмережах могли завантажувати оновлення з інтернету? (Відповідь: NAT Gateway у кожній публічній підмережі і маршрути $0.0.0.0/0 \rightarrow nat$ у приватних таблицях.)

Прибери за собою: Your VPCs → обери `khmarynka-vpc` → Actions → **Delete VPC** (разом з підмережами, IGW, таблицями й endpoint). Або залиш VPC — вона безкоштовна і знадобиться в наступних практиках.

Запам'ятай

VPC — ізольована мережа в регіоні, охоплює всі AZ; CIDR від **/16 до /28**.

Підмережа — в **одній AZ**; AWS резервує **5 адрес**.

Публічна підмережа = маршрут $0.0.0.0/0 \rightarrow$ Internet Gateway .
Приватна — без нього.

Доступ з інтернету потребує: **IGW + маршрут + публічна IP + дозволи SG/ACL**.

NAT Gateway — вихід в інтернет для приватних підмереж без вхідних з'єднань; стоїть у **публічній** підмережі; для HA — **по одному в кожній AZ**; платний за годину і гігабайт; без Security Groups.

Elastic IP — статична публічна IPv4; усі публічні IPv4 платні.

Egress-only IGW — вихідний доступ для IPv6.

Найспецифічніший маршрут перемагає; маршрут `local` є завжди.

Як питають на іспиті

Сервери в приватній підмережі мають завантажувати оновлення з інтернету, але бути недоступними ззовні → **NAT Gateway у публічній підмережі + маршрут $0.0.0.0/0 \rightarrow$ NAT у приватній**

Архітектура має пережити втрату AZ, а приватні сервери — не втратити інтернет → **NAT Gateway у кожній AZ з окремими таблицями маршрутів**

Сервер у публічній підмережі не доступний з інтернету, хоча має публічну IP → [перевір маршрут до IGW, Security Group і NACL](#)

Постійна публічна адреса, яку можна швидко перенести на інший сервер → [Elastic IP](#)

IPv6-сервери мають виходити в інтернет, але бути недоступними ззовні → [Egress-only Internet Gateway](#)

Найдешевший вихід в інтернет для тестового середовища, доступність не важлива → [одна NAT Gateway на всю VPC \(або NAT instance\)](#)

Великий рахунок за NAT Gateway, бо сервери качають багато даних з S3 → [gateway VPC endpoint для S3](#)

Скільки адрес доступно в підмережі /24 → [251](#)

Пастки

NAT Gateway ставлять у **публічну** підмережу, а маршрут на нього прописують у таблиці **приватної**.

Підмережа — в **одній** AZ, а не в кількох.

Назва «public-subnet» нічого не гарантує — вирішує таблиця маршрутів.

Автоматична публічна IP змінюється після stop/start — для постійної адреси потрібен Elastic IP.

NAT Gateway не має Security Group і не пропускає вхідні з'єднання з інтернету.

Перевір себе

ПИТАННЯ 1

Що робить підмережу в VPC публічною?

- A** Її таблиця маршрутів має маршрут 0.0.0.0/0 на Internet Gateway
- B** Вона має діапазон адрес /24
- C** Назва підмережі містить слово public
- D** Вона розташована в першій зоні доступності регіону

Правильна відповідь: A. Публічність визначається лише маршрутизацією: маршрут за замовчуванням (0.0.0.0/0) на Internet Gateway. Без нього підмережа приватна, хоч би як називалась.

ПИТАННЯ 2

Сервери застосунку в приватних підмережах двох AZ мають звертатися до API платіжного партнера в інтернеті. Потрібна висока доступність. Яке рішення правильне?

- A** NAT Gateway у публічній підмережі кожної AZ і маршрут 0.0.0.0/0 з приватної підмережі на NAT своєї зони
- B** Internet Gateway у кожній приватній підмережі
- C** Одна NAT Gateway у приватній підмережі AZ а
- D** Elastic IP на кожному сервері застосунку

Правильна відповідь: A. NAT Gateway розміщують у публічних підмережах, а приватні підмережі маршрутизують на них. По NAT у кожній AZ — щоб втрата однієї зони не відрізала інтернет іншим. Elastic IP зробили б сервери доступними ззовні.

ПИТАННЯ 3

Новий сервер EC2 у публічній підмережі має публічну IP-адресу, Security Group дозволяє HTTP з 0.0.0.0/0, але сайт не відкривається з інтернету. VPC створена вручну. Що найімовірніше забули?

- A** Створити NAT Gateway
- B** Увімкнути DNS hostnames
- C** Прикріпити Internet Gateway до VPC або додати маршрут 0.0.0.0/0 на нього
- D** Додати другу зону доступності

Правильна відповідь: C. Для доступу з інтернету потрібні IGW, маршрут на нього, публічна адреса і дозволи SG/NACL. NAT Gateway — для вихідного трафіку приватних підмереж, а не для вхідного.

ПИТАННЯ 4

Скільки IP-адрес можна призначити ресурсам у підмережі 10.0.1.0/24?

A 254

B 250

C 251

D 256

Правильна відповідь: C. У /24 — 256 адрес, AWS резервує 5 (перші чотири й останню). Доступно 251.

ПИТАННЯ 5

Компанія хоче, щоб IPv6-сервери могли звертатися до інтернету, але ніхто з інтернету не міг ініціювати з'єднання до них. Що використати?

A NAT Gateway

B Elastic IP

C Virtual Private Gateway

D Egress-only Internet Gateway

Правильна відповідь: D. NAT Gateway працює для IPv4. Для IPv6 (де всі адреси публічні) роль «лише вихід» виконує Egress-only Internet Gateway.

ПИТАННЯ 6

Яке твердження про NAT Gateway правильне?

- A** Він дозволяє серверам з інтернету ініціювати з'єднання до приватних серверів
- B** Він безкоштовний, платити треба лише за Elastic IP
- C** Класичний NAT Gateway працює в одній AZ, тому для HA потрібен NAT у кожній AZ
- D** До нього прикріплюють Security Group для фільтрації трафіку

Правильна відповідь: C. Zonal NAT Gateway живе в одній зоні і не має Security Groups; він пропускає лише вихідні з'єднання та відповіді на них. Тарифікується погодинно й за оброблені гігабайти.

ПИТАННЯ 7

Компанія планує з'єднати VPC з офісною мережею 10.0.0.0/16 через VPN. Який діапазон обрати для нової VPC?

- A** 10.0.0.0/16
- B** 10.1.0.0/16
- C** 10.0.0.0/8
- D** 10.0.128.0/17

Правильна відповідь: B. Діапазони мереж, які з'єднуються, не повинні перетинатися. 10.0.128.0/17 і 10.0.0.0/8 перетинаються з 10.0.0.0/16, а 10.1.0.0/16 — ні.



Охорона мережі — Security Groups, Network ACL і безпечний доступ до серверів

🎯 Безпека · 30%



Історія Хмаринки. Минув тиждень після переїзду. Тарас, налагоджуючи сервер, «тимчасово» відкрив порт SSH для всього інтернету — і забув. Через годину в журналах з'явилися тисячі спроб підібрати пароль з усього світу. Ще одна знахідка: база даних приймала з'єднання від будь-якого сервера у VPC, включно з тестовими. Правило, яке ми впровадимо: **кожен компонент приймає трафік лише від того, кому це справді потрібно**, а адміністратори заходять на сервери взагалі без відкритих портів.



Образ. Security Group — особистий охоронець біля дверей кожної квартири, що має список гостей і **пам'ятає**, кого випустив: вийшов ти в магазин — назад пропустить без питань (**stateful**). **Network ACL** — пропускний пункт на вході в будинок: перевіряє **кожного** і на вхід, і на вихід за нумерованим списком правил, нікого не пам'ятає (**stateless**), зате вміє **заборонити** конкретним людям.

Security Groups: файрвол біля кожного ресурсу

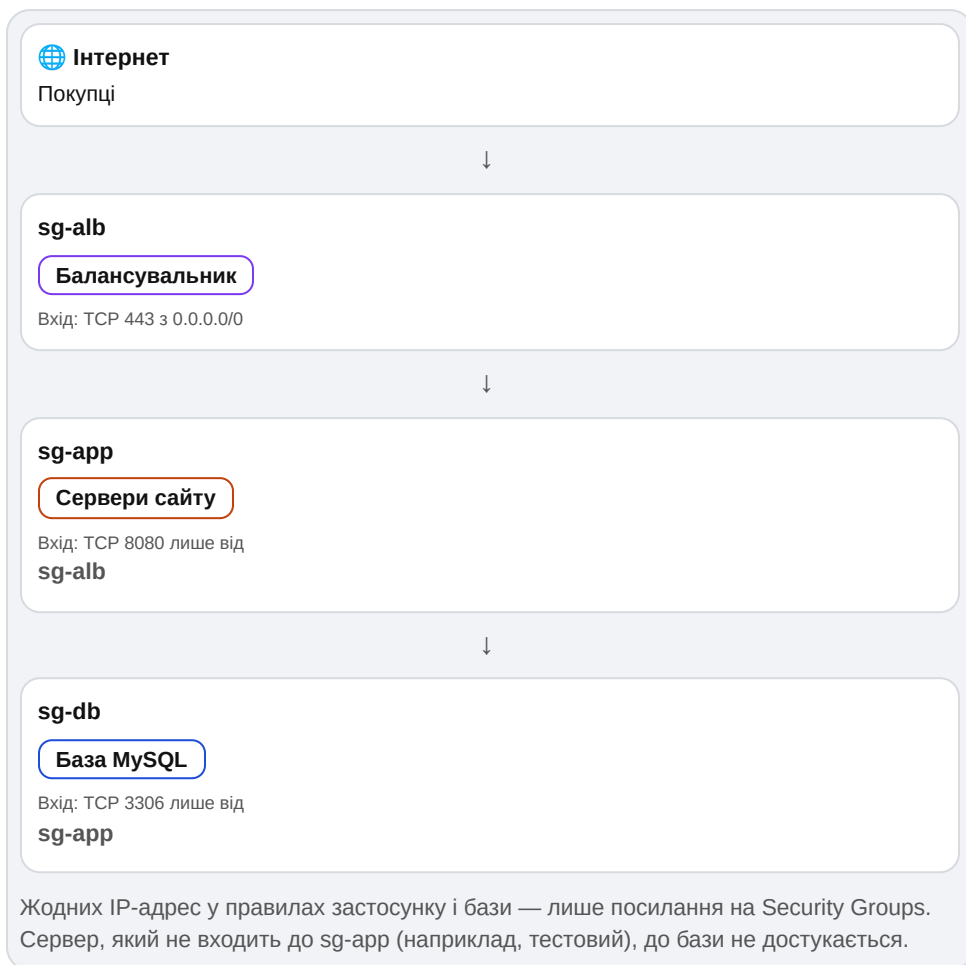
Security Group (SG) — віртуальний файрвол на рівні **мережевого інтерфейсу (ENI)**: серверів EC2, балансувальників, баз RDS, функцій Lambda у VPC, VPC endpoints тощо.

Ключові властивості:

- **Лише дозволи (Allow).** Заборонних правил немає: усе, що не дозволено, — заборонено.
- **Stateful.** Якщо вхідне з'єднання дозволене, відповідь на нього автоматично виходить — окреме вихідне правило не потрібне. І навпаки: відповіді на вихідні запити сервера пропускаються всередину.
- **Окремі вхідні (inbound) і вихідні (outbound) правила.** Нова SG за замовчуванням: вхідних правил **немає** (усе заблоковано), вихідні — **все дозволено**.
- **Джерело правила** — діапазон IP (CIDR), **інша Security Group** або список префіксів (prefix list).
- Правила оцінюються **всі разом** (порядку немає); до ресурсу можна прикріпити кілька SG — дозволи додаються.
- Зміни діють **миттєво**.

Посилання на іншу Security Group — суперсила

Замість «дозволити порт 3306 з діапазону 10.0.10.0/24» пишемо «дозволити порт 3306 **від Security Group серверів застосунку**». Тепер доступ до бази має **будь-який** сервер з цією SG — хоч два, хоч двісті, хоч з новими адресами після Auto Scaling — і **тільки** вони.



Network ACL: файрвол на рівні підмережі

Network ACL (NACL) — файрвол на межі підмережі. Діє на весь трафік, що входить у підмережу чи виходить з неї.

- **Allow і Deny.** Можна явно заборонити, наприклад, конкретну IP-адресу зловмисника.

- **Stateless.** Кожен пакет перевіряється окремо, пам'яті про з'єднання немає. Якщо дозволив вхідний запит — треба окремо дозволити **вихідну відповідь**.
- **Нумеровані правила** оцінюються від меншого номера до більшого; спрацьовує **перше**, що збіглося. Решта ігнорується. Останнє правило — заборонити все. *
- **Default NACL** (створюється з VPC) — дозволяє весь трафік в обидва боки. **Власна (custom) NACL** — за замовчуванням **забороняє все**, поки не додаси правила.
- Кожна підмережа має рівно одну NACL; одна NACL може обслуговувати кілька підмереж.

	Security Group	Network ACL
Рівень	Мережевий інтерфейс (ресурс)	Підмережа
Правила	Лише Allow	Allow і Deny
Стан	Stateful — відповіді автоматично	Stateless — відповіді дозволяти окремо
Порядок	Усі правила разом	За номером, перше збіжне
Джерело	CIDR, інша SG , prefix list	Лише CIDR
За замовчуванням	Вхід заборонено, вихід дозволено	Default — усе дозволено; custom — усе заборонено
Типове застосування	Основний захист кожного ресурсу	Додатковий шар, блокування конкретних IP

Ephemeral ports: пастка stateless-правил

Нагадування з розділу 3: клієнт відкриває з'єднання зі **свого тимчасового порту** (ephemeral port, приблизно 1024–65535) на порт сервера (наприклад, 443). Відповідь сервера йде **на цей тимчасовий порт**.

Браузер покупця

Порт відправника:

51734

(тимчасовий)



Вхід у підмережу

NACL inbound: дозволити TCP

443

з 0.0.0.0/0 



Сервер

Відповідає на 51734



Вихід з підмережі

NACL outbound: дозволити TCP

1024–65535

до 0.0.0.0/0 — інакше відповідь загубиться 

Security Group цього правила не потребує (stateful). У NACL без діапазону ephemeral ports сайт «зависне»: запит проходить, відповідь — ні.

Те саме у зворотний бік: якщо сервер у підмережі сам звертається в інтернет (оновлення з порту 443), то **вхідне** правило NACL має дозволити відповіді на ephemeral ports.

Як адміністраторам заходити на сервери

Відкритий SSH (22) чи RDP (3389) в інтернет — одна з найчастіших причин зламів. Варіанти від старого до найкращого:

1. Публічна IP + відкритий порт 22 —  ніколи для prod.

2. **Bastion host** (jump box) — окремий захищений сервер у публічній підмережі; SSH на нього дозволено **лише з офісної IP**, а з нього — на приватні сервери. Ще один сервер, який треба латати й захищати.
3. **EC2 Instance Connect** — тимчасовий SSH-ключ «проштовхується» через API на 60 секунд. **EC2 Instance Connect Endpoint** дозволяє так підключатися навіть до серверів **без публічної адреси**, без бастіону.
4. **AWS Systems Manager Session Manager** —  рекомендовано: командний рядок у браузері чи CLI **без відкритих вхідних портів** взагалі, без SSH-ключів, без бастіону. Доступ керується **IAM**, кожна сесія може **записуватися** в S3 чи CloudWatch Logs. На сервері — агент SSM і роль з потрібними правами; для повністю приватних підмереж — interface endpoints для SSM.

На іспиті

«Адміністративний доступ до серверів **без відкриття порту 22**, з **аудитом сесій**, з найменшими операційними зусиллями» → **SSM Session Manager**. «Доступ через SSH лише з офісу» → Security Group з CIDR офісу (або бастіон).

Спостереження за мережею

- **VPC Flow Logs** — журнал **метаданих** IP-трафіку: хто (IP, порт) до кого, протокол, скільки байтів, і головне — **АЦЕПТ чи REJECT**. Вмикається для VPC, підмережі чи інтерфейсу; пише в **CloudWatch Logs**, **S3** або **Data Firehose**; аналізується CloudWatch Logs Insights чи Athena. Вміст пакетів **не** записує.
- **Traffic Mirroring** — копія **самих пакетів** з інтерфейсу на систему аналізу (IDS, форензика). Коли потрібен вміст.
- **Reachability Analyzer** — перевіряє шлях між двома ресурсами («чому сервер А не бачить базу В?») і показує, **який компонент блокує**: SG, NACL, маршрут.

- **Network Access Analyzer** — шукає **небажані** шляхи доступу («чи є шлях з інтернету до підмереж з базами?»).

Діагностика з Flow Logs

Сайт не бачить базу. У Flow Logs інтерфейсу бази видно запити з 10.0.10.25 на порт 3306 зі статусом **REJECT**. Отже, трафік доходить, але його блокує SG бази або NACL підмережі даних. Якщо вхідний запис АСЕРТ, а вихідна відповідь REJECT — це майже напевно **NACL** без ephemeral ports (бо SG stateful і відповіді не блокує).

Глибокий захист: шари охорони Хмаринки

Жоден окремий механізм не ідеальний, тому захист будують **шарами** (defense in depth):

КРАЙ	Shield (DDoS) WAF на CloudFront/ALB
ПІДМЕРЕЖІ	Лише балансувальник у публічних підмережах NACL: блок відомих поганих IP
РЕСУРСИ	Security Groups з посиланнями одна на одну
СЕРВЕРИ	Патчі, Inspector, без SSH — Session Manager
ДАНІ	Шифрування KMS, TLS, IAM-ролі з найменшими правами
НАГЛЯД	Flow Logs, CloudTrail, GuardDuty

Якщо зловмисник обійде один шар, його зупинить наступний, а журнали покажуть, що сталося.

Спробуй у справжньому AWS: ланцюжок Security Groups

Вартість: безкоштовно. **Час:** 15 хвилин. Потрібна VPC з попередньої практики (або default VPC).

1. **VPC** → **Security groups** → **Create security group** `sg-alb` : вхідне правило **HTTPS (443)**, джерело `0.0.0.0/0`.
2. Створи `sg-app` : вхідне правило **Custom TCP 8080**, джерело — почни вводити `sg-` і обери **sg-alb**.
3. Створи `sg-db` : вхідне правило **MYSQL/Aurora (3306)**, джерело — **sg-app**.
4. Відкрий `sg-db` → вкладка **Outbound rules**: за замовчуванням дозволено все. Подумай, чи потрібен базі вихід «куди завгодно» (найменші права кажуть — ні).
5. **Network ACLs** → **Create network ACL** у своїй VPC → відкрий її **Inbound rules**: одне правило * **DENY**. Це і є «власна NACL забороняє все за замовчуванням». Не прив'яжуй її до підмереж!
6. Відкрий **default NACL** твоєї VPC: правило 100 **ALLOW** усе + * **DENY**. Порівняй.

Прибери за собою: видали створену NACL і три Security Groups (спершу `sg-db`, потім `sg-app`, потім `sg-alb` — через посилання між ними).

Запам'ятай

Security Group: рівень ресурсу (ENI), **лише Allow, stateful**, джерелом може бути **інша SG**; нова SG — вхід закритий, вихід відкритий.

NACL: рівень підмережі, **Allow і Deny, stateless**, нумеровані правила (перше збіжне); default — усе дозволено, custom — усе заборонено.

NACL мусить дозволяти **ephemeral ports (1024–65535)** для відповідей.

Заблокувати конкретну IP-адресу → **NACL Deny** (у SG заборон немає) або WAF для HTTP.

Доступ до серверів без відкритих портів, з аудитом → **SSM Session Manager**; для приватних серверів без бастіону — також **EC2 Instance Connect Endpoint**.

Flow Logs — метадані трафіку (ACCEPT/REJECT), не вміст; вміст — **Traffic Mirroring**.

Reachability Analyzer — «чому не з'єднується»; **Network Access Analyzer** — «чи немає зайвих шляхів».

Як питають на іспиті

База має приймати трафік лише від серверів застосунку, кількість яких змінюється → **SG бази з джерелом = SG серверів застосунку**

Заблокувати конкретну IP-адресу, що атакує підмережу → **NACL з правилом Deny (з меншим номером, ніж Allow)**

Після налаштування власної NACL запити доходять до сервера, але відповіді клієнти не отримують → **дозволити вихідний трафік на ephemeral ports 1024–65535**

Адміністративний доступ до приватних серверів без відкриття порту 22 і з журналом сесій → **AWS Systems Manager Session Manager**

Проаналізувати відхилені з'єднання і хто стукає до сервера → **VPC Flow Logs**

Потрібна копія пакетів для системи виявлення вторгнень → **Traffic Mirroring**

З'ясувати, який компонент блокує з'єднання між двома ресурсами → **Reachability Analyzer**

SSH на бастіон лише з офісу → **Security Group бастіону з CIDR офісної IP**

Пастки

У Security Group **немає Deny** — варіант «додати Deny-правило в SG» завжди неправильний.

NACL **stateless**: без ephemeral ports ламаються відповіді.

NACL оцінюється за номерами: Deny з номером 200 не спрацює, якщо Allow з номером 100 уже пропустив трафік.

Flow Logs не показують вміст пакетів і не блокують трафік.

Власна (custom) NACL за замовчуванням **забороняє все** — нова підмережа з нею «мертва», поки не додаси правила.

Перевір себе

ПИТАННЯ 1

Три вебсервери за балансувальником масштабуються Auto Scaling, їхні IP-адреси постійно змінюються. Як дозволити доступ до бази RDS лише цим серверам?

- A** У SG бази дозволити порт бази з джерелом — Security Group вебсерверів
- B** Дозволити порт бази з усієї VPC
- C** Додати в SG бази IP-адреси серверів і оновлювати їх скриптом
- D** Налаштувати NACL з дозволом на IP вебсерверів

Правильна відповідь: A. Посилання на Security Group автоматично охоплює всі ресурси з нею, незалежно від IP. Це і найменші права, і нуль ручної роботи при масштабуванні.

ПИТАННЯ 2

Служба безпеки виявила атаку з IP 198.51.100.7 на сервери в підмережі. Як заблокувати саме цю адресу?

- A** Увімкнути VPC Flow Logs
- B** Додати правило Deny в Security Group серверів
- C** Видалити Internet Gateway
- D** Додати в Network ACL підмережі правило Deny для 198.51.100.7/32 з номером меншим за правила Allow

Правильна відповідь: D. Security Groups не мають Deny-правил. NACL має, але правила оцінюються за номерами: Deny має стояти раніше (менший номер), ніж правило, що дозволяє трафік.

ПИТАННЯ 3

Після прив'язки власної NACL до підмережі вебсервера сайт перестав відповідати. Вхідне правило дозволяє TCP 443 з 0.0.0.0/0, вихідне — TCP 443 до 0.0.0.0/0. Що виправити?

- A** Змінити NACL на stateful
- B** Додати вхідне правило на порт 80
- C** Дозволити у вихідних правилах TCP 1024–65535 до 0.0.0.0/0
- D** Прикріпити до сервера ще одну Security Group

Правильна відповідь: C. NACL stateless: відповіді сервера йдуть на тимчасові порти клієнтів (ephemeral ports), тож вихідне правило має дозволяти 1024–65535. Зробити NACL stateful неможливо.

ПИТАННЯ 4

Адміністраторам потрібен доступ до командного рядка приватних серверів Linux без відкритих вхідних портів, без бастіону, із записом усіх сесій. Що обрати?

- A** Bastion host у публічній підмережі
- B** VPN до кожного сервера
- C** Відкрити порт 22 лише для офісу
- D** AWS Systems Manager Session Manager

Правильна відповідь: D. Session Manager працює через агента SSM без вхідних портів, доступ керується IAM, сесії записуються в S3 або CloudWatch Logs. Бастіон і відкритий порт вимагають відкритого SSH.

ПИТАННЯ 5

Яке твердження про Security Groups правильне?

- A** Правила оцінюються за номерами, перше збіжне перемагає
- B** Нова Security Group за замовчуванням дозволяє весь вхідний трафік
- C** Вони діють на рівні підмережі
- D** Вони stateful: відповідь на дозволений вхідний запит пропускається автоматично

Правильна відповідь: D. SG stateful, діють на рівні мережевого інтерфейсу, правила оцінюються разом. Нова SG не має вхідних правил (усе заблоковано), але дозволяє весь вихідний трафік.

ПИТАННЯ 6

Команда хоче бачити, які з'єднання до серверів відхиляються, щоб знайти помилки в правилах. Вміст пакетів не потрібен. Що увімкнути?

- A** VPC Flow Logs
- B** AWS WAF logs
- C** CloudTrail data events
- D** Traffic Mirroring

Правильна відповідь: А. Flow Logs записують метадані з'єднань, зокрема статус ACCEPT/REJECT. Traffic Mirroring копіює вміст пакетів (дорожче і зайве тут), WAF бачить лише HTTP на підтримуваних ресурсах, CloudTrail — API-виклики.

ПИТАННЯ 7

Які два твердження про Network ACL правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** До одного інстансу можна прикріпити кілька NACL
- B** NACL може посилатися на Security Group як на джерело
- C** NACL підтримують правила Allow і Deny
- D** NACL автоматично пропускає відповіді на дозволені запити
- E** Власна (custom) NACL за замовчуванням забороняє весь трафік

Правильна відповідь: С, Е. NACL мають Allow і Deny, працюють на рівні підмережі (одна NACL на підмережу), джерелом є лише CIDR, вони stateless. Нова custom NACL містить лише правило «заборонити все».



Як VPC говорить із сервісами й іншими VPC — endpoints, PrivateLink, Peering, Transit Gateway

🎯 Безпека · 30%

🎯 Продуктивність · 24%

🎯 Вартість · 20%



Історія Хмаринки. Марта принесла рахунок за перший місяць: чималий рядок «NAT Gateway — Data Processing». З'ясувалося, що сервери в приватних підмережах щодня качають гігабайти фото з S3 — **через NAT і інтернет**, хоча і сервери, і S3 знаходяться в тому самому регіоні AWS. А ще платіжний партнер хоче, щоб його сервіс перевірки платежів був доступний Хмаринці **приватно**, без інтернету. І з'явилася друга VPC — для аналітики, яку треба з'єднати з першою. Сьогодні — про всі способи з'єднати мережі й сервіси, **і скільки вони коштують**.



Образ. Gateway endpoint — безкоштовний службовий тунель з двору твого комплексу прямо на склад S3, минаючи вулицю.

Interface endpoint — окрема «віконце-каса» сервісу, встановлена прямо в твоєму будинку: платиш оренду, зате обслуговують усередині. **VPC Peering** — пряма хвіртка між двома сусідніми комплексами (але через сусіда в третій комплекс не пройдеш).

Transit Gateway — центральний вокзал, від якого йдуть потяги до всіх комплексів і в офіс.

Навіщо VPC endpoints

Сервіси AWS на кшталт S3, SQS, DynamoDB мають **публічні** адреси. Без спеціальних налаштувань сервер у приватній підмережі може звернутися до них лише через **NAT Gateway** (тобто через інтернет-вихід), а це:

- **гроші** — NAT бере плату за кожен гігабайт;
- **ризик** — підмережі потрібен вихід в інтернет, навіть якщо іншого інтернету їй не треба;
- **вимоги регуляторів** — «трафік до сховища не повинен виходити в інтернет».

VPC endpoint — приватний шлях від VPC до сервісу AWS по мережі AWS, без інтернету, NAT і публічних IP. Два типи:



Gateway endpoint

Запис у **таблиці маршрутів**: «трафік до S3 → vpcse-...»

✓ Лише S3 і DynamoDB

✓ **Безкоштовний**

⚠ Працює лише для ресурсів у цій VPC — не з офісу через VPN/DX і не з іншої VPC



Interface endpoint (PrivateLink)

Мережевий інтерфейс (ENI) з

приватною IP у твоїй підмережі; до нього прикріплюється Security Group

✓ Майже всі сервіси: SQS, SNS, KMS, Secrets Manager, SSM, ECR, CloudWatch, STS, API Gateway, S3...

\$ Платний: за годину в кожній AZ + за гігабайт

✓ Доступний з офісу (VPN/DX) і з інших VPC

Для S3 існують обидва. Усередині VPC — безкоштовний gateway. Для доступу до S3 приватно з офісу через Direct Connect — interface endpoint.

Private DNS для interface endpoint: звичайне ім'я сервісу (наприклад, `sqs.eu-central-1.amazonaws.com`) усередині VPC починає вказувати на **приватні** адреси endpoint — код застосунку не змінюється взагалі.

Політики endpoint і bucket

- **Endpoint policy** — політика на самому endpoint: «через цей тунель можна звертатися лише до bucket `khmaynka-*`». Захищає від витоку даних у чужі bucket (ексфільтрації).

- **Bucket policy** з умовою `aws:SourceVpce` (конкретний endpoint) або `aws:SourceVpc` — «цей bucket доступний лише з нашої VPC». Разом вони будують мережевий периметр даних (розділ 10).

Рахунок Марти

До: 3 ТБ на місяць з S3 через NAT Gateway — плата за обробку кожного гігабайта в NAT плюс години NAT. Після: **gateway endpoint для S3** — трафік до S3 йде в обхід NAT, а сам endpoint безкоштовний. Рядок «NAT — Data Processing» зменшився в рази. Один рядок у таблиці маршрутів — одна з найвигідніших оптимізацій в AWS.

PrivateLink: свій сервіс — приватно для інших

AWS PrivateLink — технологія, на якій працюють interface endpoints. І нею можна користуватися для **власних** сервісів:

1. **Провайдер** (платіжний партнер) ставить свій сервіс за **Network Load Balancer** (або Gateway Load Balancer) і створює **endpoint service**.
2. **Споживач** (Хмаринка) створює у своїй VPC **interface endpoint** до цього сервісу. Провайдер схвалює підключення.
3. Трафік іде приватно мережею AWS: Хмаринка бачить **лише цей сервіс**, а не всю мережу партнера.

Переваги: **односторонній** доступ (провайдер не бачить мережу споживача), **діапазони адрес можуть перетинатися** (адреси маршрутизувати не треба), масштабується на **тисячі** споживачів. Так працюють SaaS-сервіси в AWS Marketplace.

VPC Peering: пряме з'єднання двох VPC

VPC Peering — приватне з'єднання двох VPC, ніби це одна мережа. Працює між акаунтами і між регіонами.

- Маршрути треба додати **з обох боків** (у таблиці VPC A — діапазон B через `rsx-...`, і навпаки).
- **Діапазони не повинні перетинатися.**
- **Нетранзитивне:** якщо A з'єднана з B, а B — з C, то A **не** бачить C. Потрібне окреме з'єднання A–C.
- Не можна «користуватися» шлюзами сусіда: VPC A не вийде в інтернет через IGW чи NAT VPC B і не дійде до офісу через VPN VPC B.
- Немає окремої плати за з'єднання — лише за передачу даних (у межах однієї AZ безкоштовно, між AZ і регіонами — платно).

Для 2–3 VPC peering — найпростіше і найдешевше. Але для N VPC повна сітка потребує $N \times (N-1) / 2$ з'єднань: 10 VPC — 45 peering, 50 VPC — 1225. Тут на сцену виходить Transit Gateway.

Transit Gateway: мережевий вокзал

AWS Transit Gateway (TGW) — регіональний **хаб**, до якого підключаються (attachments) VPC, **Site-to-Site VPN**, **Direct Connect** (через Direct Connect Gateway) і інші Transit Gateway в інших регіонах (peering).

- **Транзитивна маршрутизація:** усе, що підключено до TGW, може спілкуватися через нього (якщо дозволяють таблиці маршрутів).
- **Власні таблиці маршрутів TGW** — сегментація: prod не бачить dev, але обидва бачать спільні сервіси.
- Шариться між акаунтами через **AWS RAM**.
- **ECMP** для VPN — кілька тунелів VPN сумують пропускну здатність.
- **Inter-region peering** між TGW різних регіонів.
- Плата — за кожне підключення на годину і за кожен оброблений гігабайт.



VPC Peering

$A \nleftrightarrow B, B \nleftrightarrow C, \text{ але } A \times C$

Нетранзитивно. Кожна пара — окреме з'єднання і маршрути. Дешево: лише трафік.



Transit Gateway

A, B, C, D, офіс (VPN/DX) → **один хаб**

Транзитивно, сегментація таблицями, сотні VPC. Плата за підключення і гігабайти.

Кілька VPC і великий трафік між ними → peering (дешевше). Десятки VPC плюс офіс, централізований контроль → Transit Gateway.



Ще два сервіси, які варто впізнати

- **AWS Cloud WAN** — глобальна мережа з централізованою політикою, що об'єднує VPC і офіси в багатьох регіонах (надбудова над ідеєю Transit Gateway у світовому масштабі).
- **Amazon VPC Lattice** — з'єднання **сервісів** (застосунків) між VPC і акаунтами на рівні застосунку, з політиками автентифікації, без керування маршрутами й перетинами діапазонів.

Що обрати: таблиця рішень

Задача	Рішення
Приватний доступ до S3 або DynamoDB з VPC, найдешевше	Gateway endpoint
Приватний доступ до SQS, KMS, Secrets Manager, SSM, ECR...	Interface endpoint
Доступ до S3 приватно з офісу через VPN/DX	Interface endpoint для S3
Дати свій сервіс сотням клієнтів приватно, діапазони можуть перетинатися	PrivateLink (NLB + endpoint service)
З'єднати 2–3 VPC просто й дешево	VPC Peering
З'єднати десятки VPC і офіс, транзитивно, з сегментацією	Transit Gateway
Кілька акаунтів працюють в одній спільній мережі	VPC sharing (AWS RAM)

Спробуй у справжньому AWS: peering двох VPC

Вартість: безкоштовно (створення peering і маршрутів не тарифікується; ми не передаватимемо даних). **Час:** 20 хвилин.

1. Якщо VPC `khmarynka` (10.0.0.0/16) з попередньої практики видалена — створи знову (**VPC and more**, без NAT).
2. Створи другу VPC: **VPC only**, назва `analytics`, CIDR 10.1.0.0/16.
3. **Peering connections** → **Create peering connection**: Requester — `khmarynka`, Acceptor — `analytics` (цей же акаунт і регіон). Create.
4. Обери peering → Actions → **Accept request**. Статус — Active.
5. **Route tables**: у таблицях підмереж `khmarynka` додай маршрут 10.1.0.0/16 → `psx-...`; у головній таблиці `analytics` — 10.0.0.0/16 → `psx-...`. Без маршрутів з обох боків peering не працює.
6. Подумай: якби була третя VPC 10.2.0.0/16, з'єднана peering лише з `analytics`, чи могла б `khmarynka` дістатися до неї? (Ні — peering нетранзитивний.)
7. Подивись **Endpoints** своєї VPC: gateway endpoint S3 і його **Policy** (за замовчуванням — повний доступ). Спробуй **Edit policy** і подивись приклад з обмеженням на конкретні bucket.

Прибери за собою: видали peering connection, потім VPC `analytics` (і `khmarynka`, якщо більше не потрібна).

Запам'ятай

Gateway endpoint — лише S3 і DynamoDB, безкоштовно, через таблицю маршрутів, лише зсередини VPC.

Interface endpoint (PrivateLink) — майже всі сервіси, ENI з приватною IP і Security Group, платний, доступний з офісу й інших VPC.

Endpoint policy + `aws:SourceVpce` / `aws:SourceVpc` у `bucket policy` — приватний периметр.

PrivateLink для власних сервісів: провайдер — **NLB + endpoint service**, споживач — `interface endpoint`; однобічно, CIDR можуть перетинатися.

VPC Peering: 1:1, **нетранзитивно**, **без перетину CIDR**, маршрути з обох боків, без використання шлюзів сусіда.

Transit Gateway: хаб для сотень VPC, VPN і DX; **транзитивно**; таблиці маршрутів для сегментації; шариться через RAM; платний за підключення і гігабайти.

Як питають на іспиті

Приватні сервери звертаються до S3/DynamoDB без інтернету і безкоштовно → **gateway VPC endpoint**

Великий рахунок за NAT Gateway через трафік до S3 → **gateway endpoint для S3**

Приватний доступ до SQS, KMS чи Secrets Manager без NAT → **interface VPC endpoint**

On-prem сервери через Direct Connect мають звертатися до S3 приватно → **interface endpoint для S3 (gateway не працює з on-prem)**

Bucket має бути доступний лише через конкретний VPC endpoint → **bucket policy з умовою `aws:SourceVpce`**

Надати свій сервіс сотням клієнтських VPC, діапазони адрес перетинаються → **PrivateLink (NLB + endpoint service)**

З'єднати дві VPC у різних акаунтах просто і дешево → **VPC Peering**

VPC A з'єднана з B, B — з C, але A не бачить C → **peering нетранзитивний: додати A–C або Transit Gateway**

З'єднати 40 VPC і офіс через VPN з транзитивною маршрутизацією → **Transit Gateway**

Ізолювати prod від dev, але дати обом доступ до спільних сервісів через хаб → **окремі таблиці маршрутів Transit Gateway**

Дві VPC обмінюються великим обсягом даних, треба найдешевше → **VPC Peering (а не Transit Gateway)**

Пастки

Gateway endpoint для SQS, SNS чи KMS **не існує** — лише S3 і DynamoDB.

Gateway endpoint не працює для трафіку з офісу (VPN/DX) і з peered VPC.

Peering не можна створити між VPC з перетинними діапазонами.

Peering не дає доступу до IGW, NAT, VPN чи DX сусідньої VPC.

Transit Gateway — не безкоштовний: для великого трафіку між двома VPC він дорожчий за peering.

Перевір себе

ПИТАННЯ 1

Сервери в приватних підмережах завантажують 5 ТБ на місяць з Amazon S3 у тому ж регіоні через NAT Gateway. Як найпростіше зменшити витрати?

- A** Створити gateway VPC endpoint для S3 і додати його до таблиць маршрутів приватних підмереж
- B** Перенести сервери в публічні підмережі
- C** Створити interface endpoint для S3 у кожній AZ
- D** Замінити NAT Gateway на NAT instance

Правильна відповідь: A. Gateway endpoint для S3 безкоштовний і відправляє трафік до S3 в обхід NAT, тож зникає плата за обробку даних у NAT. Interface endpoint теж працює, але він платний.

ПИТАННЯ 2

Застосунок у приватній підмережі має читати повідомлення з Amazon SQS без доступу в інтернет. Що створити?

- A** Gateway endpoint для SQS
- B** Internet Gateway з обмежувальною Security Group
- C** Interface endpoint для SQS
- D** VPC peering з SQS

Правильна відповідь: C. Gateway endpoints існують лише для S3 і DynamoDB. Для SQS — interface endpoint (PrivateLink) з приватною IP у підмережі.

ПИТАННЯ 3

Компанія підключена до AWS через Direct Connect. Локальні сервери мають приватно звертатися до S3, не через інтернет. Що використати?

- A** Gateway endpoint для S3
- B** Interface endpoint для S3
- C** NAT Gateway
- D** VPC peering

Правильна відповідь: B. Gateway endpoint працює лише для ресурсів усередині VPC. Трафік з одпису через VPN чи Direct Connect може використати interface endpoint для S3 — він має приватні IP, досяжні з on-prem.

ПИТАННЯ 4

SaaS-компанія хоче надати свій API сотням клієнтів у їхніх власних VPC, приватно, без peering. У багатьох клієнтів діапазони адрес перетинаються з мережею компанії. Що обрати?

- A** Transit Gateway, спільний з усіма клієнтами
- B** Site-to-Site VPN до кожного клієнта
- C** AWS PrivateLink: Network Load Balancer + endpoint service, клієнти створюють interface endpoints
- D** VPC peering з кожним клієнтом

Правильна відповідь: C. PrivateLink дає односторонній приватний доступ до конкретного сервісу, масштабується на тисячі споживачів і не залежить від перетину діапазонів. Peering і TGW вимагають непересічних CIDR.

ПИТАННЯ 5

VPC А з'єднана peering з VPC В, а VPC В — з VPC С. Сервер у VPC А не може звернутися до сервера у VPC С. Чому?

- A** Peering не працює між трьома VPC одного регіону
- B** Бракує маршруту до Internet Gateway
- C** VPC Peering нетранзитивний: для А–С потрібне окреме з'єднання або Transit Gateway
- D** Потрібно увімкнути DNS resolution у peering

Правильна відповідь: C. Peering з'єднує лише дві VPC. Трафік не проходить «транзитом» через VPC В. Рішення — прямий peering А–С або хаб Transit Gateway.

ПИТАННЯ 6

Компанія має 60 VPC у кількох акаунтах і два офіси з VPN. Потрібна централізована транзитивна маршрутизація і розділення prod і dev. Що обрати?

- A** Повна сітка VPC peering
- B** AWS Transit Gateway з окремими таблицями маршрутів, шарений через RAM
- C** Окремий VPN з кожної VPC до кожного офісу
- D** PrivateLink між усіма VPC

Правильна відповідь: B. Transit Gateway — хаб для сотень VPC, VPN і Direct Connect; таблиці маршрутів TGW дають сегментацію, а RAM — спільний доступ для інших акаунтів. Повна сітка peering на 60 VPC — 1770 з'єднань.

ПИТАННЯ 7

Які твердження про gateway VPC endpoint правильні? (Оберіть 2)

Оберіть 2 відповіді

- ☐ А Він створює мережевий інтерфейс з приватною IP-адресою у підмережі
- ☐ В До нього прикріплюється Security Group
- ☐ С Він доступний з офісу через Site-to-Site VPN
- ☐ D Він підтримує лише Amazon S3 і Amazon DynamoDB
- ☐ Е За нього немає окремої плати

Правильна відповідь: D, E. Gateway endpoint — це запис у таблиці маршрутів, лише для S3 і DynamoDB, безкоштовний. ENI, Security Group і доступ з on-prem — властивості interface endpoint.



Офіс ↔ AWS — Site-to-Site VPN, Direct Connect, Client VPN і гібридний DNS

🎯 Безпека · 30%

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Вартість · 20%



Історія Хмаринки. Сайт переїхав в AWS, а склад — ні: складська програма й принтери етикеток лишаються в офісі у Львові. Але тепер склад має бачити нові замовлення з бази в AWS, а сайт — залишки товарів зі складської програми. Відкривати складську базу в інтернет? Ніколи. Потрібен **приватний захищений канал** між офісом і VPC. Швидкий і дешевий — сьогодні, стабільний і потужний — коли Хмаринка виросте. І ще дрібниця: сервери в AWS мають знати адресу `sklad.office.lan`, а комп'ютери в офісі — `db.internal.khmarynka`.



Образ. Site-to-Site VPN — броньований автомобіль, що возить посилки звичайною громадською дорогою (інтернетом): швидко організувати і недорого, але на дорозі бувають затори. **Direct Connect** — власна приватна залізнична колія від твого складу до складу AWS: будувати тижнями, зате швидко, стабільно й без заторів. **Client VPN** — особиста захищена перепустка співробітника, щоб заходити в мережу з дому.

Два способи з'єднати офіс з AWS

	Site-to-Site VPN	AWS Direct Connect
Як іде трафік	Шифрованими тунелями IPsec через інтернет	Приватним фізичним каналом до мережі AWS
Скільки налаштовувати	Хвилини	Тижні або місяці (фізичне підключення)
Швидкість	До 1,25 Gbps на тунель (NEW до 5 Gbps для великих тунелів на Transit Gateway)	50 Mbps – 400 Gbps
Стабільність	Залежить від інтернету: затримка й швидкість «гуляють»	Стабільна затримка й пропускна здатність
Шифрування	Так, IPsec вбудований	Ні за замовчуванням → MACsec або VPN поверх DX
Ціна	Недорого: погодинно за з'єднання + трафік	Дорожче за порт, зате дешевший вихідний трафік на великих обсягах
Типова роль	Старт, невеликі обсяги, резерв для DX	Великі обсяги, критичні системи, передбачуваність

Site-to-Site VPN

Складові:

- **Customer Gateway (CGW)** — опис твого роутера чи файрвола в офісі (його публічна IP-адреса, тип маршрутизації).
- **Virtual Private Gateway (VGW)** — VPN-шлюз з боку AWS, прикріплений до однієї VPC. Або замість нього — **Transit Gateway**, якщо VPC багато.
- **VPN connection** — з'єднання між ними з двома тунелями IPsec, які закінчуються в різних зонах доступності AWS. Якщо один тунель впаде (обслуговування чи збій), трафік піде другим. Обидва тунелі треба налаштувати на офісному пристрої!

- **Маршрутизація:** статична (вписуєш діапазони вручну) або **динамічна через BGP** (роутери самі обмінюються маршрутами — рекомендовано). **Route propagation** автоматично додає маршрути до офісу в таблиці маршрутів VPC.

Корисні варіації:

- **VPN CloudHub** — кілька офісів підключено до одного VGW, і вони можуть спілкуватися **між собою** через AWS (модель «зірка»).
- **ECMP на Transit Gateway** — кілька VPN-тунелів з однаковою вартістю **сумують** пропускну здатність (наприклад, 4 тунелі × 1,25 Gbps).
- **Accelerated Site-to-Site VPN** — тунелі заходять у мережу AWS через найближчу edge-точку Global Accelerator: менше «гуляння» інтернету. Лише з Transit Gateway.

AWS Direct Connect

Direct Connect (DX) — виділене фізичне підключення твоєї мережі до AWS у спеціальній **DX-локації** (дата-центрі-партнері, де стоїть обладнання AWS). Від офісу до DX-локації канал прокладає твій провайдер.

- **Dedicated connection** — власний фізичний порт **1, 10, 100 або 400 Gbps**, замовляється в AWS.
- **Hosted connection** — через **партнера AWS**, від **50 Mbps до 25 Gbps**; часто швидше отримати і дешевше на старті.
- Від замовлення до роботи — **тижні або місяці**. Якщо даних треба перенести «вже завтра» — Direct Connect не встигне.

Віртуальні інтерфейси (VIF)

Один фізичний канал ділиться на логічні інтерфейси:

VIF	Куди веде	Для чого
Private VIF	До VPC (через VGW або Direct Connect Gateway)	Доступ до серверів і баз у VPC за приватними адресами

VIF	Куди веде	Для чого
Public VIF	До публічних сервісів AWS (S3, DynamoDB тощо) у всіх регіонах	Великі обсяги в S3 без інтернету
Transit VIF	До Transit Gateway через Direct Connect Gateway	Багато VPC через один канал

Direct Connect Gateway — глобальний об'єкт: **один DX-канал** до VPC (через VGW чи Transit Gateway) у **різних регіонах**. Наприклад, DX у Франкфурті дає доступ і до VPC в Ірландії.

Шифрування і стійкість DX

Direct Connect — приватний канал, але **не зашифрований**. Якщо вимоги кажуть «encrypt in transit»:

- **MACsec** — шифрування на рівні каналу (для dedicated 10/100/400 Gbps);
- **IPsec VPN поверх Direct Connect** — зашифровані тунелі всередині приватного каналу.

Одне підключення — **єдина точка відмови**. Моделі стійкості від AWS:

\$ DX + VPN резерв

Один DX-канал, а при збої — Site-to-Site VPN через інтернет

Найдешевший резерв; швидкість резерву нижча

\$ \$ High resiliency

По одному каналу в **двох різних DX-локаціях**

Переживає втрату цілої локації

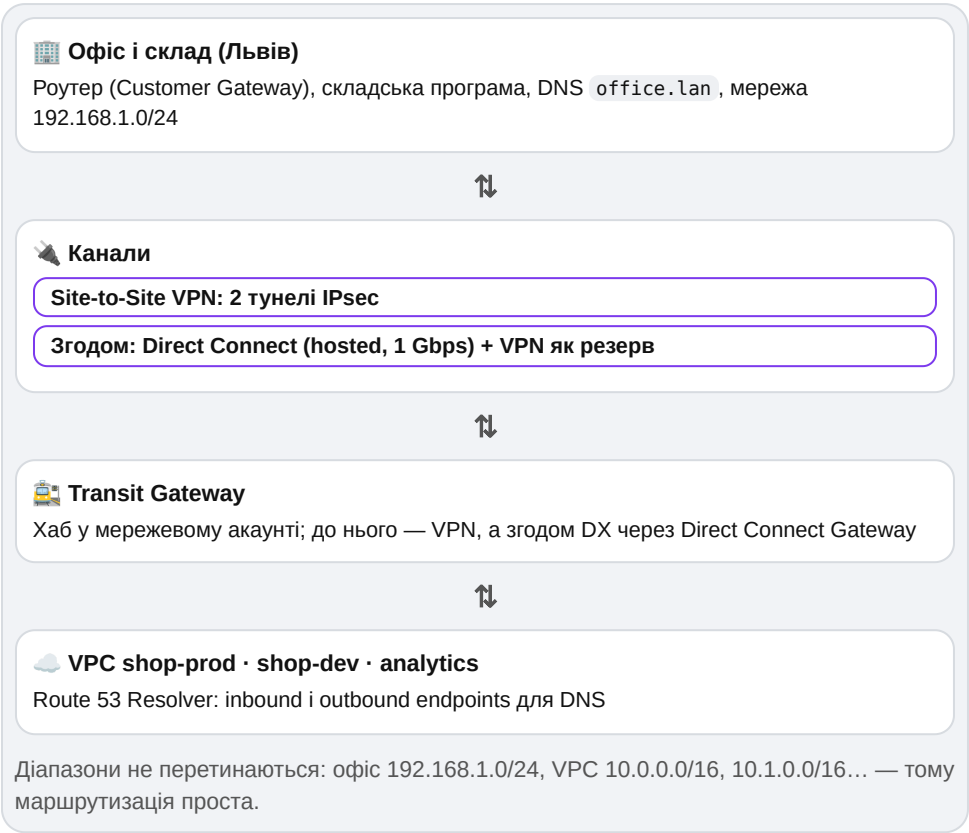
\$ \$ \$ Maximum resiliency

Два канали на різних пристроях у **кожній з двох DX-локацій**

Переживає і втрату локації, і збій пристрою — для критичних систем

На іспиті: «максимальна стійкість гібридного з'єднання» → кілька DX у різних локаціях; «найдешевший резерв для DX» → Site-to-Site VPN.

Гібридна схема Хмаринки



Client VPN: співробітники з дому

AWS Client VPN — керований VPN-сервіс для **окремих користувачів**: співробітник запускає VPN-клієнт на ноутбучі (протокол OpenVPN) і потрапляє в VPC (а через неї — і в офісну мережу, якщо налаштовано). Автентифікація — через **Active Directory**, **SAML** (наприклад, Entra ID, Okta) або сертифікати. Масштабується автоматично.



Site-to-Site чи Client VPN?

Site-to-Site — мережа ↔ мережа (офіс ↔ VPC), налаштовується на роутері. **Client VPN** — людина ↔ мережа (ноутбук ↔ VPC). На іспиті «віддалені співробітники підключаються до VPC» → Client VPN.

Гібридний DNS: Route 53 Resolver endpoints

Усередині VPC імена резолвить **Route 53 Resolver**. В офісі — свій DNS-сервер. Щоб вони розуміли одне одного:

- **Inbound endpoint** — «вхід» у DNS AWS для офісу: офісний DNS пересилає запити про `internal.khmarynka` на IP-адреси inbound endpoint у VPC, і офісні комп'ютери бачать приватні імена AWS.
- **Outbound endpoint + forwarding rules** — «вихід» з VPC: правило «запити про `office.lan` пересилай на офісний DNS 192.168.1.10» — і сервери в AWS знаходять `sklad.office.lan`.
- Правила можна поширити на інші VPC і акаунти через **AWS RAM**.



→ Inbound endpoint

Офісний DNS: «про `internal.khmarynka` питай 10.0.0.53» → Resolver у VPC відповідає з `private hosted zone`
Офіс бачить імена AWS



→ Outbound endpoint

Правило Resolver: «`office.lan` → 192.168.1.10» → запит іде через VPN в офісний DNS
AWS бачить імена офісу

Обидва endpoints — мережеві інтерфейси в підмережах VPC (краще у двох AZ); трафік іде приватним каналом VPN або DX.



Спробуй у справжньому AWS: подивись, як виглядає VPN-з'єднання

Вартість: VPN-з'єднання тарифікується погодинно — якщо видалити його за 15–20 хвилин, це кілька центів. Customer Gateway і Virtual

Private Gateway самі по собі не тарифікуються. **Час:** 20 хвилин.

1. **VPC → Customer gateways → Create:** назва `office-lviv`, IP-адреса — будь-яка публічна для тесту (наприклад, `203.0.113.10` з документаційного діапазону), BGP ASN — залиш за замовчуванням.
2. **Virtual private gateways → Create** `vgw-khmarynka` → Actions → **Attach to VPC** (твоя VPC).
3. **Site-to-Site VPN connections → Create:** Target — Virtual private gateway, Customer gateway — `office-lviv`, Routing — **Static**, Static IP prefixes — `192.168.1.0/24`. Create.
4. Коли з'єднання створиться (кілька хвилин), відкрий вкладку **Tunnel details:** **два** тунелі з різними зовнішніми IP-адресами AWS. Статус Down — нормально, бо справжнього роутера немає.
5. **Download configuration** → Vendor: Generic → подивись файл: параметри IPsec для обох тунелів, які адміністратор вписав би в офісний роутер.
6. Відкрий таблицю маршрутів приватної підмережі → **Route propagation** → Edit → увімкни для VGW: маршрути до офісу з'являтимуться автоматично.

Прибери за собою (обов'язково!): видали **VPN connection** (зупиняє оплату), потім Detach і Delete VGW, потім Delete Customer gateway.

Запам'ятай

Site-to-Site VPN: IPsec через інтернет, **хвилини** на налаштування, **2 тунелі** в різних AZ, VGW (одна VPC) або Transit Gateway (багато VPC), до 1,25 Gbps на тунель; ECMP на TGW сумує тунелі.

Direct Connect: приватний фізичний канал, **тижні-місяці**, dedicated 1/10/100/400 Gbps, hosted 50 Mbps–25 Gbps; стабільно і дешевший вихідний трафік; **не шифрується** сам → **MACsec** або **VPN поверх DX**.

VIF: **private** (VPC), **public** (публічні сервіси AWS), **transit** (Transit Gateway). **Direct Connect Gateway** — один DX до VPC у різних регіонах.

Стійкість: **DX + VPN** — дешевий резерв; **DX у двох локаціях** — висока/максимальна стійкість.

Client VPN — окремі користувачі з ноутбуків; **CloudHub** — кілька офісів між собою через AWS.

Resolver inbound — офіс резолвить імена AWS; **outbound + правила** — AWS резолвить імена офісу.

Як питають на іспиті

Швидко і недорого створити зашифрований канал до офісу → **Site-to-Site VPN**

Стабільна пропускна здатність, приватний канал, великі щоденні обсяги → **AWS Direct Connect**

Direct Connect + вимога шифрувати трафік → **IPsec VPN поверх DX або MACsec**

Найдешевший резерв для Direct Connect → **Site-to-Site VPN як backup**

Максимальна стійкість гібридного з'єднання для критичної системи → **кілька DX-з'єднань у різних DX-локаціях**

Один Direct Connect має вести до VPC у кількох регіонах → **Direct Connect Gateway**

Доступ до S3 через Direct Connect без інтернету → **public VIF (або private VIF + interface endpoint для S3)**

Дані треба перенести через тиждень, а Direct Connect ще не проведено → **DX не встигне: VPN, DataSync або фізичний пристрій**

Віддалені співробітники підключаються до ресурсів у VPC → **AWS Client VPN**

Кілька філій мають спілкуватися між собою через AWS → **VPN CloudHub (або Transit Gateway)**

Збільшити пропускну здатність VPN понад один тунель → [Transit Gateway з ECMP \(кілька тунелів\)](#)

Сервери в AWS мають резолвити внутрішні імена офісу → [Route 53 Resolver outbound endpoint + forwarding rule](#)

Офісні сервери мають резолвити приватні імена в AWS → [Route 53 Resolver inbound endpoint](#)

Пастки

Direct Connect **не шифрує** трафік сам по собі.

Direct Connect не підключається «за день» — у терміновій задачі він неправильна відповідь.

Один DX-канал — єдина точка відмови; «високодоступний» DX = щонайменше два канали (або DX + VPN).

Virtual Private Gateway прикріплюється до **однієї** VPC; для багатьох VPC — Transit Gateway чи Direct Connect Gateway.

Client VPN ≠ Site-to-Site VPN: перший для людей, другий для мереж.

Перевір себе

ПИТАННЯ 1

Компанії потрібно за один день налаштувати зашифроване з'єднання між офісом і VPC для невеликих обсягів даних. Що обрати?

- A** AWS Direct Connect dedicated connection
- B** VPC peering з офісом
- C** AWS Site-to-Site VPN
- D** AWS Direct Connect hosted connection

Правильна відповідь: C. Site-to-Site VPN налаштовується за хвилини і шифрує трафік IPsec. Direct Connect потребує тижнів на фізичне підключення і не шифрує сам. Peering з офісом неможливий — це з'єднання між VPC.

ПИТАННЯ 2

Компанія використовує Direct Connect, і служба безпеки вимагає шифрувати весь трафік між офісом і VPC. Яке рішення правильне?

- A** Замінити Direct Connect на VPC endpoint
- B** Увімкнути шифрування EBS
- C** Налаштувати IPsec VPN поверх з'єднання Direct Connect (або MACsec на підтримуваному з'єднанні)
- D** Direct Connect шифрує трафік автоматично, нічого робити не треба

Правильна відповідь: C. Direct Connect — приватний, але незашифрований канал. Шифрування in transit досягається VPN-тунелями IPsec поверх DX або MACsec на рівні каналу.

ПИТАННЯ 3

Критичний застосунок залежить від Direct Connect. Потрібна максимальна стійкість до збоїв з'єднання і втрати цілої DX-локації. Що рекомендувати?

- A** Один Direct Connect з резервним Site-to-Site VPN
- B** Один Direct Connect з більшою швидкістю порту
- C** Кілька Direct Connect-з'єднань на окремих пристроях у двох різних DX-локаціях
- D** Два Direct Connect-з'єднання в одній DX-локації

Правильна відповідь: C. Максимальна стійкість — з'єднання в кількох DX-локаціях на різних пристроях: переживає і збій пристрою, і втрату цілої локації. DX + VPN — дешевший резерв, але з меншою пропускнуою здатністю.

ПИТАННЯ 4

Компанія має Direct Connect у Франкфурті й хоче через нього отримувати доступ до VPC у eu-central-1 та eu-west-1. Що використати?

- A** Public VIF
- B** Direct Connect Gateway
- C** Два окремі фізичні з'єднання DX
- D** VPC peering між регіонами

Правильна відповідь: B. Direct Connect Gateway — глобальний ресурс, що з'єднує один DX-канал з VPC (через VGW або Transit Gateway) у різних регіонах.

ПИТАННЯ 5

150 співробітників працюють віддалено і мають безпечно підключатися до внутрішніх застосунків у VPC зі своїх ноутбуків, входячи корпоративним обліковим записом. Що обрати?

- A** Direct Connect hosted connection
- B** Публічні IP-адреси для внутрішніх застосунків
- C** Site-to-Site VPN з кожним ноутбуком
- D** AWS Client VPN з автентифікацією через SAML або Active Directory

Правильна відповідь: D. Client VPN — керований VPN для окремих користувачів, з автентифікацією через AD, SAML чи сертифікати. Site-to-Site з'єднує мережі, а не окремі пристрої.

ПИТАННЯ 6

Сервери в VPC мають звертатися до внутрішнього сервісу за іменем `erp.corp.local`, яке відоме лише офісному DNS-серверу. Що налаштувати?

- A** Route 53 Resolver outbound endpoint і forwarding rule для `corp.local` на офісний DNS
- B** Public hosted zone `corp.local` у Route 53
- C** Запис у файлі `hosts` кожного сервера
- D** Route 53 Resolver inbound endpoint

Правильна відповідь: A. Outbound endpoint пересилає запити з VPC до зовнішніх DNS-серверів за правилами. Inbound endpoint потрібен для протилежного напрямку — коли офіс резолвить імена AWS.

ПИТАННЯ 7

Компанія використовує Site-to-Site VPN на Transit Gateway, але одного тунелю (1,25 Gbps) не вистачає. Як збільшити пропускну здатність без Direct Connect?

- A** Перейти на Client VPN
- B** Збільшити розмір Virtual Private Gateway
- C** Увімкнути VPC Flow Logs
- D** Налаштувати кілька VPN-тунелів з ECMP на Transit Gateway

Правильна відповідь: D. Transit Gateway підтримує ECMP (equal-cost multi-path) для VPN: кілька тунелів з однаковими маршрутами сумують пропускну здатність. Virtual Private Gateway цього не вміє.



Route 53 — DNS, який керує трафіком



Відмовостійкість · 26%



Продуктивність · 24%



Історія Хмаринки. Домен `khmarynka.ua` зареєстрований в українського реєстратора, а його DNS досі обслуговує старий хостинг, де запис змінюється «протягом доби». Тарас хоче більшого: щоб `khmarynka.ua` вказував на CloudFront, `api.khmarynka.ua` — на балансувальник, щоб при аварії сайт сам перемикався на сторінку «Технічні роботи», щоб нову версію бачили спершу 10% покупців, а поляки й німці потрапляли на свою мовну версію. Усе це — не код, а **DNS-правила** в Amazon Route 53.



Образ. Звичайний DNS — довідкова служба, що на питання «де живе `khmarynka.ua`?» завжди дає одну й ту саму адресу. **Route 53** — розумний диспетчер таксі: зважає, звідки ти телефонуєш (країна, відстань, затримка), хто з водіїв зараз вільний і здоровий (health checks), і навіть розподіляє замовлення у пропорції «90% старим водіям, 10% новому».

Що вміє Route 53

Amazon Route 53 — керований DNS-сервіс AWS з гарантією доступності **100%** (SLA). Три функції:

1. **Реєстрація доменів** (багато доменних зон: `.com`, `.net`, `.org`, `.de`... — хоча не всі; `.ua`, наприклад, реєструють місцеві реєстратори).

2. **DNS-хостинг**: відповідає на запити про твій домен з сотень edge-локацій по світу.
3. **Health checks** і **політики маршрутизації**: відповідь залежить від здоров'я серверів, розташування користувача, ваг.

Hosted zone — контейнер записів одного домену:

- **Public hosted zone** — відповідає всьому інтернету.
- **Private hosted zone** — відповідає лише всередині прив'язаних **VPC** (наприклад, `internal.khmarynka` → адреси баз). Потребує увімкнених у VPC `enableDnsSupport` і `enableDnsHostnames`. Можна мати публічну і приватну зони з **однаковою** назвою — різні відповіді зсередини і ззовні (split-horizon).

Щоб Route 53 обслуговував домен, куплений в іншого реєстратора, створюєш hosted zone і вписуєш у реєстратора її чотири **NS-сервери** Route 53 (делегування).

Записи й особливий Alias

Типи записів ті самі, що в будь-якому DNS (розділ 3): A, AAAA, CNAME, MX, TXT, NS, CAA... Плюс «суперсила» Route 53 — **Alias record**:

	CNAME	Alias (Route 53)
На що вказує	Будь-яке інше ім'я	Ресурси AWS : балансувальник ELB, CloudFront, S3 website , API Gateway, Global Accelerator, VPC endpoint, Elastic Beanstalk, інший запис у зоні
Zone apex (<code>khmarynka.ua</code> без <code>www</code>)	 Заборонено стандартом DNS	 Можна
Плата за запити	Платні	Безкоштовні запити до ресурсів AWS
Зміна IP ресурсу	Відповідь через ім'я	Route 53 відстежує сам
TTL	Задаєш ти	Береться з ресурсу

Тому класичне питання «направити `example.com` (apex) на Application Load Balancer» має одну відповідь — **Alias record**. Alias **не може** вказувати на ім'я сервера EC2 — лише на перелічені ресурси.

Вісім політик маршрутизації

1. Simple

Одна відповідь (можна кілька IP — клієнт обирає випадково). Без health checks.

Простий сайт на одному ресурсі

2. Weighted

Відповіді за вагами: 90 / 10.

A/B-тести, поступовий реліз, міграція трафіку

3. Latency-based

Регіон з найменшою затримкою для цього користувача.

Глобальний застосунок у кількох регіонах

4. Failover

Primary, а якщо health check провалено — secondary.

Active-passive DR, сторінка «технічні роботи»

5. Geolocation

За країною / континентом користувача (+ запис за замовчуванням).

Мовні версії, ліцензії, закони, заборони

6. Geoproximity

За відстанню до ресурсу, з **bias** — «розширити» чи «стиснути» зону ресурсу.

Змістити частку трафіку до одного регіону

7. Multivalued answer

До 8 **здорових** записів у відповіді.

Простий розподіл на боці клієнта з перевіркою здоров'я (не заміна балансувальника)

8. IP-based

За діапазоном IP клієнта (CIDR-колекції).

Трафік певного провайдера — на певний endpoint

Політики можна комбінувати деревом: наприклад, latency між регіонами, а всередині кожного регіону — weighted між версіями.



Geolocation, Geoproximity чи Latency?

Geolocation — «де ти живеш» (країна за законом чи мовою): поляки — на польську версію, навіть якщо Ірландія ближча. **Geoproximity** — «як далеко» + ручка **bias**, щоб перекинути частку трафіку. **Latency** — «куди швидше» (виміряна затримка до регіонів). Питання про закони, ліцензії, мову → Geolocation. Про швидкість → Latency. Про «зсунути більше трафіку» → Geoproximity.

Health checks і аварійне перемикання

Health check Route 53 регулярно перевіряє endpoint з кількох точок світу (HTTP, HTTPS або TCP; можна шукати рядок у відповіді). Три типи:

- **Endpoint** — перевірка конкретної адреси чи імені.
- **Calculated** — комбінація інших перевірок («здоровий, якщо щонайменше 2 з 3»).
- **CloudWatch alarm** — стан аларму. Це спосіб перевіряти **приватні** ресурси: перевіряльники Route 53 працюють з інтернету й не бачать приватних підмереж, а аларм CloudWatch — бачить метрики будь-чого.

Failover-маршрутизація Хмаринки:

Покупець

Питає DNS: де khmarynka.ua?



Route 53

Health check основного endpoint: здоровий?



Відповідь

 Так → PRIMARY: CloudFront → ALB у Франкфурті

 Ні → SECONDARY: статична сторінка на S3 (або регіон Ірландія)

Час перемикання = час виявлення збою (інтервал перевірок × кількість невдач) + **TTL**, протягом якого клієнти ще пам'ятають стару відповідь. Для швидкого failover — малий TTL.

Для Alias-записів є опція **Evaluate target health** — Route 53 сам враховує здоров'я цілі (наприклад, чи має балансувальник здорові сервери) без окремого health check.

Active-active — усі записи здорові й отримують трафік (weighted, latency, multivalue з health checks); збійний просто випадає. **Active-passive** — резерв отримує трафік лише при збої основного (failover).



Глибше: чому DNS-перемикання не миттєве

—

Клієнти й резолвери кешують відповідь на час **TTL**. Якщо TTL = 300 секунд, частина користувачів ще до 5 хвилин ходитиме на «мертву» адресу. Деякі резолвери й застосунки ігнорують TTL і тримають відповідь довше. Тому для задач «перемикання між регіонами за секунди, незалежно від DNS-кешу» на іспиті правильна відповідь — **Global Accelerator** зі статичними anycast-IP (розділ 18). А для складних сценаріїв аварійного відновлення з контрольованим

перемиканням є **Route 53 Application Recovery Controller** — перевірки готовності резервного регіону і «рубильники» маршрутизації.

DNS Хмаринки

Запис	Тип / політика	Ціль
khmarynka.ua	Alias , failover primary	CloudFront distribution
khmarynka.ua	Alias, failover secondary	S3 static website «Технічні роботи»
www.khmarynka.ua	CNAME (або Alias)	khmarynka.ua
api.khmarynka.ua	Alias , weighted 90 / 10	ALB поточної версії / ALB нової версії
shop.khmarynka.ua	Geolocation : Польща / Німеччина / за замовчуванням	Мовні версії
db.internal.khmarynka	Private hosted zone, CNAME	Endpoint бази RDS
khmarynka.ua	MX , TXT	Пошта, підтвердження домену для сервісів

Спробуй у справжньому AWS: зона, записи й тестова відповідь

Вартість: hosted zone — \$0,50 на місяць, але якщо видалити її протягом 12 годин після створення, плати немає. **Час:** 15 хвилин.

1. **Route 53** → **Hosted zones** → **Create hosted zone**: ім'я — будь-яке, навіть не твоє (наприклад, khmarynka-lab.com), тип **Public**. Домен купувати не треба — просто не буде делегування з інтернету.
2. Подивись створені записи **NS** і **SOA**: чотири name servers — їх вписують у реєстратора при делегуванні.
3. **Create record**: www, тип **A**, значення 192.0.2.10, routing **Simple**.

4. Створи два записи `api` з routing **Weighted**: перший — `192.0.2.21`, weight 90, Record ID `v1`; другий — `192.0.2.22`, weight 10, Record ID `v2`.
5. Відкрий запис `api` → **Test record** (або в CloudShell: `aws route53 test-dns-answer --hosted-zone-id <ID> --record-name api.khmarynka-lab.com --record-type A`). Повтори кілька разів — здебільшого відповідь `v1`, іноді `v2`.
6. Подумай: як зробити, щоб при падінні `v2` трафік на нього припинився? (Відповідь: прив'язати health check до запису.)

Прибери за собою: видали створені записи (крім NS і SOA), потім **Delete hosted zone** — у межах 12 годин.

Запам'ятай

Route 53: реєстрація доменів, DNS з SLA 100%, health checks, політики маршрутизації.

Public hosted zone — для інтернету; **private** — лише для прив'язаних VPC.

Alias — на ресурси AWS (ELB, CloudFront, S3 website, API Gateway...), працює на **zone apex**, безкоштовні запити. **CNAME на apex не можна**. Alias на EC2 — не можна.

Політики: **Simple, Weighted, Latency, Failover, Geolocation, Geoproximity (bias), Multivalue (до 8 здорових), IP-based**.

Health checks: endpoint, calculated, **CloudWatch alarm** (для приватних ресурсів).

Швидкість DNS-перемикання обмежена **TTL**.

Як питають на іспиті

Направити `example.com` (zone apex) на Application Load Balancer → **Route 53 Alias record**

Перевести 10% трафіку на нову версію → **Weighted routing**

Користувачі мають потрапляти в регіон з найменшою затримкою → **Latency-based routing**

Active-passive DR між регіонами через DNS → **Failover routing + health checks**

Показувати контент залежно від країни (мова, ліцензії, закон) → **Geolocation routing (+ default record)**

Змістити більше трафіку до одного регіону → **Geoproximity routing з bias**

Повертати кілька здорових IP-адрес для простого розподілу навантаження → **Multivalued answer routing**

Health check для ресурсу в приватній підмережі → **CloudWatch alarm + Route 53 health check**

Імена ресурсів мають розв'язатися лише всередині VPC → **Private hosted zone**

При збої основного сайту показувати статичну сторінку-заглушку → **Failover-запис на S3 static website**

Пастки

CNAME не можна створити для кореневого домену — лише Alias.

Route 53 health checks не бачать приватні IP напряму — потрібен CloudWatch alarm.

Geolocation ≠ latency: Geolocation дивиться на країну, а не на швидкість.

Multivalued answer — не балансувальник: він лише повертає кілька здорових адрес.

Великий TTL сповільнює аварійне перемикання.

✓ Перевір себе

ПИТАННЯ 1

Компанія хоче, щоб домен example.com (без www) вказував на Application Load Balancer. Який запис створити в Route 53?

- A** Alias-запис A для example.com на ALB
- B** A-запис з IP-адресою ALB
- C** CNAME-запис example.com → DNS-ім'я ALB
- D** MX-запис на ALB

Правильна відповідь: A. CNAME заборонено на zone apex, а IP-адреси ALB змінюються. Alias-запис Route 53 працює на apex і сам відстежує адреси ALB.

ПИТАННЯ 2

Команда випускає нову версію API і хоче спершу спрямувати на неї 5% користувачів, поступово збільшуючи частку. Яка політика маршрутизації підходить?

- A** Geolocation
- B** Latency-based
- C** Weighted
- D** Failover

Правильна відповідь: C. Weighted routing розподіляє відповіді за вагами (95/5), і ваги можна змінювати поступово. Це класичний спосіб канаркових релізів і міграцій через DNS.

ПИТАННЯ 3

Контент стримінгового сервісу ліцензований лише для Польщі. Користувачі з інших країн мають бачити іншу сторінку. Що використати?

- A** Geoproximity routing з bias
- B** Multivalued answer routing
- C** Latency-based routing
- D** Geolocation routing з записом для Польщі і записом за замовчуванням

Правильна відповідь: D. Geolocation визначає країну користувача і повертає відповідний запис. Запис за замовчуванням обслуговує всіх, для кого немає окремого правила.

ПИТАННЯ 4

Застосунок працює в регіонах eu-central-1 і us-east-1. Кожен користувач має потрапляти в регіон, де відповідь буде найшвидшою. Яка політика?

- A** Simple
- B** Geolocation
- C** Weighted 50/50
- D** Latency-based

Правильна відповідь: D. Latency-based routing обирає регіон з найменшою виміряною затримкою для мережі користувача — саме «найшвидший», а не «найближчий за кордоном».

ПИТАННЯ 5

Потрібно автоматично перемикає трафік на резервний регіон, коли основний перестає відповідати. База основного регіону — у приватній підмережі, її стан відображає аларм CloudWatch. Що налаштувати? (Оберіть 2)

Оберіть 2 відповіді

- ☐ A Geolocation routing
- ☐ B Health check Route 53 напряму на приватну IP бази
- ☐ C Weighted routing з вагою 0 для резерву
- ☐ D Failover routing policy з primary і secondary записами
- ☐ E Health check Route 53, прив'язаний до аларму CloudWatch

Правильна відповідь: D, E. Failover-політика перемикає на secondary при провалі health check. Перевіряльники Route 53 не бачать приватних адрес, тому health check будується на аларму CloudWatch.

ПИТАННЯ 6

Сервери в VPC мають звертатися до бази за ім'ям db.internal.shop, і це ім'я не повинно резолвитися з інтернету. Що створити?

- A** Route 53 Resolver inbound endpoint
- B** Private hosted zone internal.shop, прив'язану до VPC
- C** Alias-запис на базу в публічній зоні
- D** Public hosted zone internal.shop

Правильна відповідь: B. Private hosted zone відповідає лише для прив'язаних VPC. Для неї у VPC мають бути увімкнені DNS support і DNS hostnames.

ПИТАННЯ 7

Після аварії основного регіону DNS-запис failover перемкнувся, але частина користувачів ще 10 хвилин потрапляла на старий endpoint. Що найімовірніше?

- A** Великий TTL запису — клієнти й резолвери кешували стару відповідь
- B** Alias-записи не враховують здоров'я цілі
- C** Route 53 не підтримує failover між регіонами
- D** Health check не працює з HTTPS

Правильна відповідь: A. Відповідь кешується на час TTL (а деякі клієнти тримають довше). Для швидшого failover знижують TTL; для перемикання за секунди без залежності від кешу — Global Accelerator.



Швидко для всього світу — Amazon CloudFront і AWS Global Accelerator



Безпека · 30%



Продуктивність · 24%



Вартість · 20%



Історія Хмаринки. Пам'ятаєш скаргу покупниці з Варшави з розділу 3? Кожне фото товару їхало з Франкфурта окремо, а сторінка каталогу має 60 фото. А ще кур'єрська служба, з якою працює Хмаринка, вимагає: «Дайте нам **постійні IP-адреси** вашого API, ми внесемо їх у свій фаїрвол». Але в балансувальника адреси змінюються. Дві різні задачі — «ближче до людей» і «стабільний вхід у мережу AWS» — і два сервіси на краю мережі: **CloudFront** і **Global Accelerator**.



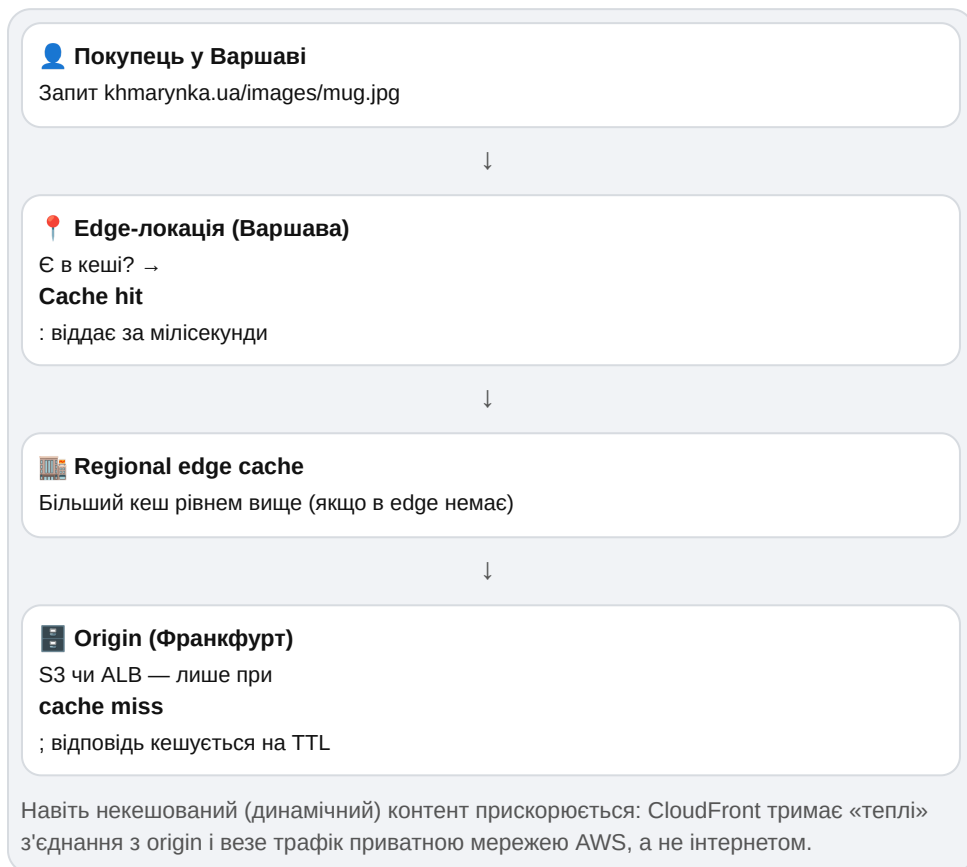
Образ. CloudFront — мережа кіосків з копіями найпопулярніших товарів у кожному районі міста: покупець бере товар за рогом, а не їде на центральний склад. **Global Accelerator** — швидкісна платна автомагістраль із двома постійними в'їздами біля кожного міста світу: товарів на в'їзді немає (нічого не кешує), зате з будь-якого міста ти одразу потрапляєш на рівну приватну трасу без світлофорів аж до потрібного складу — а якщо склад закрито, тебе за секунди розвертають на інший.

CloudFront: мережа доставки контенту

Amazon CloudFront — CDN (content delivery network): копії контенту кешуються в **сотнях edge-локацій** по світу. Покупець отримує файл з

найближчої точки за мілісекунди, а сервери Хмаринки (**origin** — джерело) отримують у рази менше запитів.

Як працює запит:



Origins: звідки CloudFront бере контент

- **S3 bucket** — статика: фото, відео, CSS, JavaScript, файли для завантаження.
- **Application Load Balancer, EC2, API Gateway**, функції Lambda (URL) — динамічний контент і API.
- **Будь-який HTTP-сервер**, навіть у власному дата-центрі (custom origin).

- **VPC origins** (з 2024 року) — приватні ALB, NLB чи EC2 **без публічних адрес**: CloudFront звертається до них прямо в приватні підмережі.

Закрити S3 від усіх, крім CloudFront: OAC

Якщо bucket з фото публічний, будь-хто може качати файли в обхід CloudFront (і WAF, і кешу). Правильна схема: bucket **приватний**, а доступ має лише CloudFront через **Origin Access Control (OAC)** — bucket policy дозволяє читання лише цьому конкретному distribution. OAC — сучасна заміна старого **OAI** (Origin Access Identity) і, на відміну від нього, підтримує шифрування **SSE-KMS**, усі регіони й запис (PUT).

Cache behaviors: різні правила для різних шляхів

Один distribution може мати кілька **behaviors** за шаблоном шляху — кожен зі своїм origin і своїм кешуванням:

Шлях	Origin	Кешування	Інше
/images/* , /static/*	S3 (через OAC)	Довго (дні), файли з версією в імені	Стиснення
/api/*	ALB	Не кешувати ; передавати заголовки, cookies, query string	Дозволені POST/PUT/DELETE
* (default)	ALB	Коротко (хвилини)	Переадресація HTTP → HTTPS

TTL і інвалідація. Скільки файл живе в кеші, визначає **cache policy** (мінімальний, типовий і максимальний TTL) разом із заголовками **Cache-Control** від origin. Щоб примусово оновити файл — **invalidation** (/images/mug.jpg або /images/*), але краще **версіонувати імена** файлів (style.v42.css): нова версія — нове ім'я, старий кеш не заважає, інвалідації не потрібні.

Приватний контент: signed URL і signed cookies

Хмаринка продає електронні подарункові сертифікати в PDF — не всім, а лише покупцям:

- **Signed URL** — підписане посилання на **один** файл з терміном дії (і, за бажанням, дозволеною IP).

- **Signed cookies** — доступ до **багатьох** файлів (увесь розділ «для підписників», відеокурс) без зміни URL-адрес.
- Підписує застосунок ключем з **trusted key group** CloudFront.

Не плутай з **S3 presigned URL** — тимчасовим посиланням напряду на об'єкт S3 (з правами того, хто підписав), без CloudFront.

Безпека й надійність на краю

- **HTTPS**: сертифікат **ACM у us-east-1** (розділ 11); примусова переадресація HTTP → HTTPS; власні домени (alternate domain names).
- **AWS WAF** на distribution і автоматичний **Shield Standard**; CloudFront поглинає DDoS своєю гігантською пропускнуою здатністю.
- **Geo restriction** — дозволити або заборонити цілі країни.
- **Origin failover** — **origin group** з основним і резервним origin: якщо основний повертає помилки (наприклад, 500, 502, 503, 504), запит іде на резервний (скажімо, копію bucket в іншому регіоні).
- **Field-level encryption** — шифрує окремі поля форми (номер картки) вже на edge, щоб лише потрібний мікросервіс міг їх розшифрувати.

Код на краю: CloudFront Functions і Lambda@Edge

	CloudFront Functions	Lambda@Edge
Мова	JavaScript (легкий)	Node.js, Python
Де і коли виконується	На кожній edge-локації, лише запит глядача і відповідь глядачу	У regional edge caches, на запити й відповіді глядача і origin
Швидкість і масштаб	Субмілісекунди, мільйони запитів за секунду	Мілісекунди–секунди
Мережа і файли	 Без мережевих викликів	 Можна звертатися до інших сервісів
Типові задачі	Заголовки безпеки, редиректи, переписування URL, проста перевірка токена, A/B за cookie	Зміна розміру зображень, складна автентифікація, вибір origin за даними з бази

Гроші

- Передача даних з origins AWS (S3, ALB, EC2) **до CloudFront** — **безкоштовна**, а віддача з CloudFront користувачам зазвичай дешевша за прямий вихід з регіону. Тож CloudFront часто **зменшує** рахунок за трафік.
- **Price classes** — обмежити distribution дешевшими регіонами edge-мережі (наприклад, лише Європа й Північна Америка).
- Є постійний безкоштовний рівень (терабайт трафіку і мільйони запитів на місяць) — для навчальних проєктів цього вистачає з головою.

AWS Global Accelerator

AWS Global Accelerator дає застосунок **дві статичні anycast IP-адреси**.

«Anycast» означає: ті самі дві адреси анонсуються з усіх edge-локацій AWS, і користувач потрапляє в **найближчу**. Далі трафік іде **приватною мережею AWS** до найкращого здорового endpoint у потрібному регіоні.

- **Endpoints:** Application Load Balancer, Network Load Balancer, EC2, Elastic IP — в одному чи кількох регіонах (endpoint groups).
- **Health checks** і **перемикання між регіонами за секунди** — без залежності від DNS-кешу, бо адреси не змінюються.
- **Traffic dials** (відсоток трафіку на регіон) і **ваги endpoint** — для поступових релізів і техобслуговування.
- **TCP і UDP**, будь-які протоколи зверху. **Нічого не кешує**.
- **Статичні IP** — ідеально, коли клієнти вносять адреси в білі списки файрволів.

Для кур'єрської служби Хмаринки: Global Accelerator перед ALB з API → дві постійні адреси для їхнього файрвола, а бонусом — стабільніша затримка з будь-якої країни.

CloudFront чи Global Accelerator?

	CloudFront	Global Accelerator
Кешування	✔ Так — головна ідея	✗ Ні
Протоколи	HTTP/HTTPS (і WebSocket)	TCP / UDP — будь-які застосунки
Вхідні адреси	DNS-ім'я, IP змінюються	2 статичні anycast IP
Перемикання регіонів	Origin failover у межах запиту	Секунди, на рівні мережі, без DNS-кешу
Типові сценарії	Сайти, фото, відео, API з кешем, статичні сайти з S3	Ігри, IoT, VoIP, API зі статичними IP, мультирегіональний failover
Захист	WAF, Shield	Shield (з Advanced)

🧠 Одним реченням

Є що кешувати і це веб → CloudFront. Потрібні статичні IP, UDP чи швидке перемикання регіонів → Global Accelerator. І ще сусід: S3 Transfer Acceleration — прискорює завантаження в S3 з далеких країн через edge-мережу.

🔧 Спробуй у справжньому AWS: приватний S3 за CloudFront з ОАС

Вартість: у межах безкоштовного рівня CloudFront і кількох центів S3.
Час: 25 хвилин (створення distribution займає кілька хвилин).

1. S3 → **Create bucket:** ім'я `khmarynka-edge-<випадкові-цифри>`, регіон `eu-central-1`, **Block all public access** — увімкнено (залиш як є).
2. Створи на комп'ютері файл `index.html` з текстом `<h1>Привіт з Хмаринки!</h1>` і завантаж у bucket.

3. **CloudFront** → **Create distribution**: Origin domain — твій bucket; **Origin access** — **Origin access control settings (recommended)** → Create new OAC. Default root object: `index.html`. WAF — можна не вмикати для тесту. Create.
4. Натисни **Copy policy** у жовтому банері і встав її в **Bucket policy** свого bucket (S3 → Permissions → Bucket policy) — або погодься, щоб консоль оновила її сама. Подивись: дозволено лише `cloudfront.amazonaws.com` з умовою на ARN твого distribution.
5. Коли статус distribution — Enabled, відкрий його адресу `https://dxxxx.cloudfront.net` — сторінка працює (з HTTPS!).
6. Відкрий прямий URL об'єкта S3 (Object URL) — **AccessDenied**: bucket приватний, доступ лише через CloudFront.
7. Зміни `index.html`, завантаж знову — CloudFront ще показує стару версію (кеш). Зроби **Invalidations** → **Create** з шляхом `/*` — і отримаєш нову.

Прибери за собою: CloudFront → distribution → **Disable**, дочекайся і **Delete**; потім очисти й видали bucket.

Запам'ятай

CloudFront — CDN: кеш на сотнях edge-локацій, менша затримка, менше навантаження на origin, часто дешевший трафік.

Origins: **S3, ALB, EC2, API Gateway**, будь-який HTTP-сервер, **VPC origins** для приватних ресурсів.

OAC — bucket лише для CloudFront (заміна OAI, підтримує SSE-KMS).

Cache behaviors за шляхом: `/static/*` → S3 з довгим кешем, `/api/*` → ALB без кешу.

Signed URL — один файл; **signed cookies** — багато файлів; **S3 presigned URL** — напяму в S3.

Geo restriction, origin failover (origin group), **field-level encryption, WAF**, сертифікат **ACM у us-east-1**.

CloudFront Functions — легкі й миттєві (заголовки, редиректи);
Lambda@Edge — важча логіка з мережею.

Global Accelerator — 2 статичні anycast IP, TCP/UDP, без кешу, перемикання регіонів за секунди.

🎯 Як питають на іспиті

Прискорити статичний і динамічний контент для користувачів по всьому світу → **Amazon CloudFront**

Заборонити прямий доступ до S3, лише через CloudFront → **Origin Access Control (OAC)**

Доступ підписників до багатьох платних відео → **CloudFront signed cookies**

Тимчасове посилання на один приватний файл → **CloudFront signed URL (або S3 presigned URL)**

Заблокувати доступ до контенту з певних країн → **CloudFront geo restriction (або WAF geo match)**

Статика й API на одному домені з різним кешуванням → **cache behaviors за шляхом**

Легкі зміни заголовків і редиректи на edge з мінімальною затримкою → **CloudFront Functions**

Змінювати розмір зображень на льоту з викликами інших сервісів → **Lambda@Edge**

Автоматично віддавати контент з резервного bucket, якщо основний повертає 5xx → **CloudFront origin failover (origin group)**

Статичні IP для глобального застосунку, клієнти вносять їх у білий список → **AWS Global Accelerator (або NLB з Elastic IP в одному регіоні)**

Онлайн-гра на UDP, гравці по всьому світу, швидке перемикання регіонів → **Global Accelerator**

Зменшити витрати на вихідний трафік для популярного статичного контенту → **CloudFront перед S3**

Користувачі з інших континентів повільно завантажують великі файли в S3 → **S3 Transfer Acceleration (+ multipart upload)**

Пастки

Global Accelerator **не кешує** — для кешу потрібен CloudFront.

CloudFront — про HTTP(S); для «UDP-гри» це неправильна відповідь.

Сертифікат ACM для CloudFront — лише з **us-east-1**.

OAI — застарілий; якщо в питанні є SSE-KMS — лише **OAC**.

Signed URL ≠ presigned URL: перший — CloudFront, другий — S3.

Перевір себе

ПИТАННЯ 1

Інтернет-магазин зберігає фото товарів у S3 і хоче, щоб їх отримували лише через CloudFront, а прямий доступ до bucket був заборонений. Що налаштувати?

- A** Зробити bucket публічним і обмежити доступ Security Group
- B** S3 presigned URL для кожного фото
- C** Origin Access Control і bucket policy, що дозволяє читання лише цьому distribution
- D** Signed cookies для всіх відвідувачів

Правильна відповідь: C. OAC дозволяє CloudFront підписувати запити до приватного bucket, а bucket policy пускає лише сервіс CloudFront з умовою на конкретний distribution. У S3 немає Security Groups.

ПИТАННЯ 2

Відеоплатформа хоче, щоб платні підписники мали доступ до сотень відео без зміни посилань, а інші — ні. Що обрати?

- A** CloudFront geo restriction
- B** CloudFront signed URL для кожного відео
- C** S3 presigned URL
- D** CloudFront signed cookies

Правильна відповідь: D. Signed cookies дають доступ до багатьох файлів одночасно без зміни URL. Signed URL зручний для одного файлу, geo restriction обмежує за країнами, а не за підпискою.

ПИТАННЯ 3

Партнер вимагає фіксованих IP-адрес API, щоб додати їх у свій фаїрвол. API працює за Application Load Balancer у двох регіонах, і потрібне швидке перемикавання при збої регіону. Що обрати?

- A** Elastic IP на ALB
- B** AWS Global Accelerator з endpoints — ALB в обох регіонах
- C** Route 53 latency-based routing
- D** CloudFront distribution перед ALB

Правильна відповідь: B. Global Accelerator дає дві статичні anycast IP і перемикає між регіонами за секунди за health checks. ALB не підтримує Elastic IP, у CloudFront адреси змінюються, а DNS-перемикавання залежить від TTL.

ПИТАННЯ 4

Компанія запускає багатокористувацьку гру на UDP з гравцями на всіх континентах. Потрібна низька затримка і стабільне з'єднання. Що підходить найкраще?

- A** Route 53 geolocation routing
- B** S3 Transfer Acceleration
- C** AWS Global Accelerator
- D** Amazon CloudFront

Правильна відповідь: C. Global Accelerator працює з TCP і UDP, заводить трафік у приватну мережу AWS у найближчій точці й має статичні адреси. CloudFront призначений для HTTP(S) і кешування.

ПИТАННЯ 5

Один domain має обслуговувати статичні файли з S3 з довгим кешем і API з ALB без кешування. Як це зробити в одному distribution CloudFront?

- A** Два окремі distribution з однаковим доменом
- B** Lambda@Edge, що копіює файли з ALB у S3
- C** Два cache behaviors: /static/* → S3 з довгим TTL, /api/* → ALB з вимкненим кешуванням
- D** Route 53 weighted routing між S3 і ALB

Правильна відповідь: C. Cache behaviors за шаблоном шляху дозволяють різні origins і різні правила кешування в межах одного distribution.

ПИТАННЯ 6

Потрібно додавати заголовки безпеки (HSTS, CSP) до всіх відповідей сайту з мінімальною затримкою і вартістю. Що обрати?

- A** AWS WAF
- B** EC2 з nginx перед CloudFront
- C** Lambda@Edge на подію origin request
- D** CloudFront Functions на подію viewer response

Правильна відповідь: D. CloudFront Functions — легкий JavaScript, що виконується на кожній edge-локації за субмілісекунди і дешево. Для простих змін заголовків це найкращий вибір; Lambda@Edge — для важчої логіки.

ПИТАННЯ 7

Сайт зберігає фото в bucket у eu-central-1 і має копію bucket в eu-west-1. Як зробити, щоб CloudFront автоматично віддавав фото з копії, якщо основний bucket повертає помилки 5xx?

- A** S3 Transfer Acceleration
- B** Origin group у CloudFront з основним і резервним origin та критеріями failover
- C** Два distribution з перемиканням вручну
- D** Route 53 failover для bucket

Правильна відповідь: B. Origin group задає основний і резервний origin; при вказаних кодах помилок CloudFront повторює запит до резервного.



Amazon EC2 — віртуальні сервери зсередини



Відмовостійкість · 26%



Продуктивність · 24%



Історія Хмаринки. Мережа готова — час переселяти сам сайт. Тарас відкриває консоль EC2 і бачить понад вісімсот типів серверів: `t4g.micro`, `m7i.large`, `c8g.4xlarge`, `r7iz`, `i4i`, `p5` ... «Який з них наш?» Олена ставить інше питання: «Як зробити, щоб новий сервер за п'ять хвилин ставав таким самим, як старий, з усіма програмами?» Відповіді — у будові EC2: типи, образи, скрипти запуску, життєвий цикл і розміщення.



Образ. EC2 — автопрокат. **Сімейство інстансу** — клас авто: малолітражка на щодень (Т), універсал (М), спорткар (С), вантажівка з величезним кузовом (R — багато пам'яті), фургон-рефрижератор з власним складом (I — швидкі локальні диски), болід з реактивним двигуном (P/G — графічні прискорювачі). **AMI** — авто, в яке вже поклали твої речі й налаштували сидіння. **User data** — записка на сидінні: «Перед першою поїздкою заправ і ввімкни навігатор».

Що таке EC2 instance

Amazon EC2 (Elastic Compute Cloud) — віртуальні сервери в оренду. Один сервер — **інстанс** (instance). Щоб запустити інстанс, тиобираєш:

**AMI**
Образ: ОС і встановлене ПЗ

**Тип інстансу**
vCPU, пам'ять, мережа, локальні диски

**Мережа**
VPC, підмережа (= AZ), публічна IP чи ні

**Security Groups**
Які порти й від кого відкриті

**Сховище**
Кореневий диск EBS (+ додаткові) і/або instance store

**IAM-роль**
Instance profile — права без ключів

**User data**
Скрипт першого запуску

**Key pair**
Ключ для SSH (якщо не Session Manager)

**Теги**
Name, Environment, Team — для пошуку й рахунків

Усе це можна зберегти в **launch template** — шаблоні запуску, який потім використовує Auto Scaling (розділ 23).

Типи інстансів: як читати назву

Назва типу кодує все важливе. Розберемо `m7g.2xlarge` :

- **m** — сімейство (для чого сервер);
- **7** — покоління (що більше, то новіше і зазвичай вигідніше);
- **g** — атрибути: g — Graviton (ARM), a — AMD, i — Intel, d — локальні NVMe-диски, n — посилена мережа, e — більше пам'яті чи сховища, flex — «гнучкі» дешевші варіанти;
- **2xlarge** — розмір: nano, micro, small, medium, large, xlarge, 2xlarge ... 48xlarge, metal. Кожен наступний розмір приблизно подвоює ресурси (і ціну).

Сімейство	Призначення	Типові задачі
T (t3, t4g)	Загальне, burstable	Невеликі сайти, dev/test, мікросервіси з низьким середнім навантаженням

Сімейство	Призначення	Типові задачі
M (m7g, m7i)	Загальне, збалансоване	Сервери застосунків, бекенди, середні бази
C (c7g, c8g)	Обчислення (compute optimized)	Кодування відео, пакетна обробка, HPC, ігрові сервери, ML-інференс на CPU, високонавантажені вебсервери
R, X, z	Пам'ять (memory optimized)	Бази й кеші в пам'яті, SAP HANA, аналітика великих наборів у RAM
I, D, H	Сховище (storage optimized): швидкі локальні NVMe чи великі HDD	NoSQL, OLTP з великими IOPS, сховища даних, розподілені файлові системи
P, G, Inf, Trn, F	Прискорювачі (accelerated): GPU, чипи Inferentia/Trainium, FPGA	Навчання й запуск ML-моделей, графіка, рендеринг



Глибше: burstable-інстанси (Т) і CPU-кредити

—

Сервери сімейства Т мають **базовий рівень** CPU (наприклад, 20% ядра). Поки навантаження нижче базового — сервер **накопичує кредити**; коли потрібно більше — **витрачає** їх і працює на повну. Для сайту, що більшу частину доби відпочиває, це дуже вигідно. Але якщо кредити закінчатся, продуктивність падає до базової. У режимі **unlimited** (типовий для t3/t4g) сервер може працювати понад базу й без кредитів — з доплатою. Якщо навантаження постійно високе — Т не підходить, бери М чи С.

Як обрати на практиці: почни з типу за профілем навантаження (сайт — М або С, кеш — R), запусти, подивись метрики і скоригуй. **AWS Compute Optimizer** проаналізує реальну роботу і порадить правильний розмір (right-sizing). На іспиті назви конкретних розмірів не потрібні — потрібне сімейство за задачею.

AMI: образ сервера

AMI (Amazon Machine Image) — шаблон, з якого запускається сервер: ОС, налаштування, встановлені програми, опис дисків.

- **Джерела:** AMI від AWS (Amazon Linux 2023, Windows Server), від спільноти, з **AWS Marketplace** (готове ПЗ з ліцензією — файрволи, бази, CMS), **власні**.
- **Власний «золотий» образ** (golden AMI): встановив і налаштував застосунок → створив AMI → нові сервери стартують **за хвилини вже готовими**. Це прискорює масштабування і гарантує однаковість.
- **AMI — регіональний**. Щоб запустити такий самий сервер в іншому регіоні (наприклад, для DR), AMI **копіюють** у той регіон. AMI можна **ділити** з іншими акаунтами.
- **EC2 Image Builder** автоматизує створення, тестування і регулярне оновлення (патчі) золотих образів.

User data і метадані

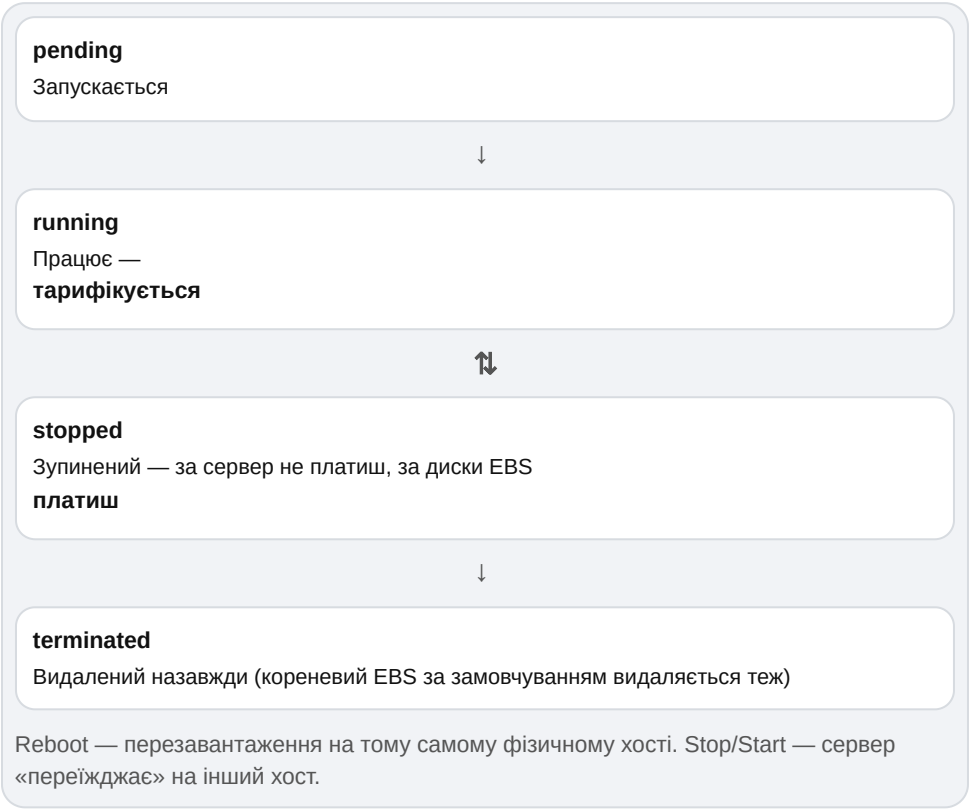
User data — скрипт, який сервер виконує **при першому запуску** (від імені root). Приклад для сервера Хмаринки:

```
#!/bin/bash
dnf install -y nginx
DB_HOST=$(aws ssm get-parameter --name /khmarynka/prod/db-host --query P
echo "DB_HOST=$DB_HOST" >> /etc/khmarynka.env
systemctl enable --now nginx
```

Типова архітектурна дилема: **ставити все в user data** (гнучко, але сервер довго стартує) чи **запікати в AMI** (швидкий старт, але образ треба оновлювати). Для швидкого масштабування — більше в AMI, у user data — лише налаштування середовища.

Instance Metadata Service (IMDS) — внутрішня адреса `169.254.169.254`, з якої сервер дізнається про себе: свій ID, тип, зону, IP-адреси і — головне — **тимчасові облікові дані своєї IAM-ролі**. Використовуй **IMDSv2**: він вимагає токен сесії і захищає від атак, коли злоумисник змушує застосунок звернутися до метаданих (SSRF). У launch template можна **вимагати** IMDSv2.

Життєвий цикл інстансу



Що відбувається при **stop/start**:

- автоматична **публічна IP змінюється** (Elastic IP лишається);
- приватна IP **зберігається**;

- дані на дисках **EBS зберігаються**, а на **instance store** — **зникають**;
- сервер опиняється на іншому фізичному хості — так лікують проблеми обладнання.

Hibernate — «сплячий режим»: вміст оперативної пам'яті зберігається на (обов'язково **зашифрований**) кореневий диск EBS. Після пробудження застосунок продовжує з того самого місця з «теплим» кешем, без довгої ініціалізації.

Termination protection — захист від випадкового видалення.

Перевірки стану (status checks)

- **System status check** — проблема на боці **AWS**: фізичний хост, живлення, мережа. Лікування: **stop/start** (переїзд на інший хост) або автоматичне відновлення — більшість сучасних інстансів відновлюються самі, а для гарантії є аларм CloudWatch з дією **recover** (сервер переїжджає на новий хост, зберігаючи ID, IP-адреси й метадані).
- **Instance status check** — проблема **всередині** сервера: ОС зависла, закінчилась пам'ять, помилка мережі в ОС. Лікування — перезавантаження, виправлення конфігурації (це твоя зона відповідальності).
- **Attached EBS status check** — проблема з підключеним диском.

Мережа й розміщення інстансів

- **ENI** (Elastic Network Interface) — мережева карта сервера. Додаткову ENI можна **перечепити** на інший сервер разом з її приватною IP і MAC — старий прийом для швидкого failover і ліцензій, прив'язаних до MAC.
- **ENA** (Enhanced Networking) — висока пропускна здатність і низька затримка, на сучасних типах увімкнена за замовчуванням.
- **EFA** (Elastic Fabric Adapter) — для **HPC і розподіленого ML**: застосунки (MPI, NCCL) спілкуються в обхід ОС з наднизькою затримкою.

Placement groups — як розставити сервери фізично:

Стратегія	Як розміщує	Навіщо	Обмеження
Cluster	Поруч, в одній AZ, в одній частині мережі	Найменша затримка й найвища пропускна між серверами: HPC, розподілене навчання ML	Одна AZ — усі разом ризикують однаково
Spread	Кожен сервер на окремому обладнанні (окремі стійки)	Кілька критичних серверів не повинні відмовити разом	Максимум 7 запущених інстансів на AZ у групі
Partition	Групи-«партиції» на окремих стійках, до 7 партицій на AZ	Великі розподілені системи з реплікацією: Hadoop, HDFS, Cassandra, Kafka	Сотні інстансів; застосунок знає, в якій він партиції

Сервери сайту Хмаринки

- Тип **m7g.large** (Graviton, 2 vCPU, 8 GiB) — збалансовано й вигідно; застосунок на Java/Node.js/Python працює на ARM без змін.
- Власний **golden AMI** на базі Amazon Linux 2023 з уже встановленим застосунком; оновлюється щотижня через Image Builder.
- **User data** лише підтягує параметри середовища з Parameter Store.
- **IAM-роль** з правами на читання фото в S3 і секрету бази; **IMDSv2 обов'язковий**.
- **Приватні підмережі** у двох AZ, доступ адміністраторів — лише Session Manager.
- Кількість серверів змінюватиме Auto Scaling (розділ 23), а як за них платити — наступний розділ.



Спробуй у справжньому AWS: вебсервер за п'ять хвилин

Вартість: тип з позначкою *Free tier eligible* (наприклад, `t3.micro` чи `t4g.micro`) — у межах кредитів безкоштовного плану; якщо видалити сервер одразу, це центи. Бонус: запуск EC2 — одне з п'яти завдань, за які нараховують **\$20 кредитів**. **Час:** 20 хвилин.

1. **EC2** → **Launch instance**. Назва: khmarynka-web-lab .
2. AMI: **Amazon Linux 2023** (для t4g — версія arm64). Тип: t3.micro або t4g.micro .
3. Key pair: **Proceed without a key pair** (зайдемо через браузер).
4. Network: твоя VPC або default VPC, **публічна** підмережа, Auto-assign public IP — **Enable**. Security group: створи нову з правилом **HTTP (80)**, Source — **My IP**.
5. **Advanced details** → **Metadata version: V2 only (token required)**. У **User data** встав:

```
#!/bin/bash
dnf install -y nginx
echo "<h1>Хмаринка на EC2: $(hostname -f)</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

1. **Launch**. Через хвилину відкрий у браузері публічну IP сервера (http://...) — побачиш сторінку.
2. **Connect** → **EC2 Instance Connect** (якщо не підключається — додай у SG правило SSH з My IP) і виконай запит до метаданих з токеном IMDSv2:

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/api/token)
```

1. **Instance state** → **Stop**, потім **Start** — порівняй публічну IP до і після.

Прибери за собою: Instance state → **Terminate** і перевір, що кореневий диск EBS зник (Elastic Block Store → Volumes). Видали Security Group.

Запам'ятай

Запуск EC2: **AMI + тип + мережа + SG + сховище + IAM-роль + user data + key pair** → зручно зберегти в **launch template**.

Назва типу: **сімейство + покоління + атрибути + розмір**
(m7g.2xlarge); **g** = Graviton.

T — burstable (кредити CPU); **M** — загальне; **C** — обчислення; **R/X** — пам'ять; **I/D** — локальне сховище; **P/G/Inf/Trn** — прискорювачі.

AMI регіональний — копією в інший регіон; **golden AMI** = швидкий старт.

User data — скрипт першого запуску; **IMDSv2** — безпечні метадані й облікові дані ролі.

Stop/start: нова публічна IP (крім EIP), EBS зберігається, instance store — зникає. **Hibernate** — RAM на зашифрований EBS.

System check — проблема AWS (stop/start, recover); **instance check** — проблема в ОС (твоя).

Placement: **cluster** — затримка; **spread** — ізоляція (до 7 на AZ); **partition** — Hadoop/Cassandra/Kafka.

Як питають на іспиті

In-memory база чи кеш потребує багато RAM → **memory optimized (R, X)**

Кодування відео, наукові розрахунки, пакетна обробка з високим CPU → **compute optimized (C)**

Навчання ML-моделей на GPU → **accelerated (P, G, Trn)**

Дуже високі IOPS на локальних дисках для NoSQL → **storage optimized (I)**

Найкраще співвідношення ціни й продуктивності для сумісного з ARM застосунку → **Graviton**

Нові сервери мають стартувати швидко вже з усім ПЗ → **golden AMI**

Запустити той самий сервер в іншому регіоні → **copу AMI в інший регіон**

НПС, мінімальна затримка між вузлами → **cluster placement group + EFA**

Кілька критичних серверів не повинні відмовити одночасно через обладнання → **spread placement group**

Hadoop / Cassandra / Kafka на сотнях серверів з ізоляцією стійок → **partition placement group**

Сервер не проходить system status check → **stop/start або CloudWatch alarm з дією recover**

Швидко продовжити роботу з «теплою» пам'яттю після зупинки → **EC2 Hibernate**

Захистити облікові дані ролі від SSRF-атак через метадані → **вимагати IMDSv2**

Пастки

Instance store **зникає** при stop, terminate чи збої хоста — не для важливих даних.

Зупинений сервер не коштує, але його **диски EBS коштують**.

Spread placement group — максимум **7 запущених інстансів на AZ**.

Cluster placement group — в **одній AZ**, тобто не про високу доступність.

AMI не «глобальний»: без копіювання в іншому регіоні його не видно.

Перевір себе

ПИТАННЯ 1

Компанія запускає базу даних в оперативній пам'яті, якій потрібні сотні гігабайт RAM при помірному навантаженні на CPU. Яке сімейство інстансів обрати?

A Memory optimized (R або X)

B Burstable (T)

C Compute optimized (C)

D Storage optimized (I)

Правильна відповідь: A. Memory optimized інстанси мають найбільше пам'яті на vCPU — для in-memory баз, кешів і аналітики в пам'яті. C — для інтенсивних обчислень, I — для локальних дисків з високими IOPS.

ПИТАННЯ 2

Застосунок на EC2 під час масштабування стартує 15 хвилин, бо user data встановлює все ПЗ з нуля. Як найкраще прискорити запуск нових серверів?

- A** Перенести скрипт з user data в cron
- B** Збільшити тип інстансу
- C** Використати instance store замість EBS
- D** Створити golden AMI з уже встановленим ПЗ і запускати сервери з нього

Правильна відповідь: D. Золотий образ містить усе потрібне, тож сервер готовий за хвилину-дві. User data лишають лише для налаштувань середовища.

ПИТАННЯ 3

Сервер EC2 не проходить system status check. Що найімовірніше допоможе?

- A** Перезавантажити ОС командою reboot
- B** Зупинити і запустити інстанс (або налаштувати alarm з дією recover), щоб він переїхав на інший фізичний хост
- C** Змінити Security Group
- D** Перевстановити застосунок

Правильна відповідь: B. System status check сигналізує про проблему інфраструктури AWS (хост, живлення, мережа). Stop/start переносить сервер на інший хост; дія recover робить це автоматично, зберігаючи ID і IP. Reboot лишає сервер на тому самому хості.

ПИТАННЯ 4

Науковий застосунок на 16 серверах обмінюється величезними обсягами даних між вузлами і потребує мінімальної затримки мережі. Що обрати?

- A** Partition placement group
- B** Spread placement group у трьох AZ
- C** Cluster placement group і інстанси з Elastic Fabric Adapter
- D** Інстанси в різних регіонах

Правильна відповідь: C. Cluster placement group розміщує сервери поруч в одній AZ для найменшої затримки й найбільшої пропускної здатності; EFA дає HPC-застосункам обхід ОС. Spread і partition — про ізоляцію від збоїв, а не про швидкість.

ПИТАННЯ 5

Три інстанси контролера кластера критичні: вони не повинні відмовити разом через збій спільного обладнання. Що обрати?

- A** Один великий інстанс замість трьох
- B** Dedicated Host
- C** Cluster placement group
- D** Spread placement group

Правильна відповідь: D. Spread розміщує кожен сервер на окремому обладнанні (окремих стійках). Обмеження — 7 запущених інстансів на AZ у групі, для трьох серверів цього достатньо.

ПИТАННЯ 6

Застосунок після зупинки й запуску сервера втратив тимчасові файли, які зберігав на локальному диску. Що це був за диск?

- A** Кореневий том EBS gp3
- B** Snapshot EBS
- C** Amazon EFS
- D** Instance store

Правильна відповідь: D. Instance store — фізичні диски хоста, дані на них зникають при stop, terminate чи збої. EBS і EFS зберігають дані незалежно від стану сервера.

ПИТАННЯ 7

Служба безпеки хоче захистити облікові дані IAM-ролі на серверах від атак типу SSRF через сервіс метаданих. Що зробити?

- A** Вимкнути IAM-ролі і використовувати access keys
- B** Використовувати лише публічні підмережі
- C** Вимагати IMDSv2 (token required) для всіх інстансів
- D** Відкрити доступ до 169.254.169.254 у Security Group

Правильна відповідь: C. IMDSv2 вимагає токен сесії, отриманий PUT-запитом, що блокує більшість SSRF-атак. Access keys — гірше за ролі, а Security Groups не керують доступом до метаданих.



Як платити за сервери — On-Demand, Savings Plans, Reserved, Spot і Dedicated

🎯 Вартість · 20%



Історія Хмаринки. Марта принесла першу аналітику: «Чотири сервери сайту працюють цілодобово весь рік — і ми платимо за них повну ціну щогодини. А нічна обробка фото й звітів запускає ще двадцять серверів на дві години». Тарас додає: «А на старому складському сервері стоїть програма з ліцензією, прив'язаною до кількості фізичних процесорних ядер». Один і той самий сервер в AWS можна купити шістьма різними способами — і різниця в ціні сягає **90%**. Обрати правильний спосіб для кожного навантаження — пряма відповідальність архітектора.



Образ. Житло. On-Demand — готель: заселився будь-коли, виїхав будь-коли, але ніч дорожча. **Savings Plans / Reserved** — довгострокова оренда квартири на рік чи три: значно дешевше, але платиш, навіть коли їдеш у відпустку. **Spot** — «гаряча путівка» на вільний номер за копійки, але готель може попросити звільнити його за дві хвилини, якщо прийде гість за повну ціну. **Dedicated Host** — орендуєш увесь будинок: ніяких сусідів і видно, скільки в ньому кімнат (для ліцензій «за кімнату»).

On-Demand: плати за годину без зобов'язань

- Посекундна оплата (для Linux і Windows, мінімум 60 секунд), без передоплат і договорів.
- Найвища ціна за годину, але **повна свобода**: запустив, вимкнув, змінив тип.
- Для **короткочасних, непередбачуваних, нових** навантажень, тестів, піків, яких не можна переривати.

Savings Plans: знижка за обіцянку витратити

Savings Plans — ти зобов'язуєшся витратити певну суму **на годину** (наприклад, \$2/год) протягом **1 або 3 років**. Усе використання в межах цієї суми отримує знижку; понад неї — звичайна ціна On-Demand. Оплата: повністю наперед, частково чи без передоплати (що більше наперед — то більша знижка).

План	Знижка (до)	На що поширюється	Гнучкість
Compute Savings Plan	~66%	EC2 будь-якого сімейства, розміру, ОС, регіону + Fargate + Lambda	Максимальна: можна перейти з EC2 на контейнери чи змінити регіон
EC2 Instance Savings Plan	~72%	Конкретне сімейство в конкретному регіоні (наприклад, m7g у Франкфурті)	Можна змінювати розмір, ОС, орендність
 Database Savings Plans (з грудня 2025)	~35%	RDS, Aurora, DynamoDB, ElastiCache та інші бази (1 рік)	Між сервісами баз даних

Reserved Instances: класична знижка

Reserved Instances (RI) — старіша модель: знижка на **конкретний тип** інстансу на 1 або 3 роки.

- **Standard RI** — до ~72%; змінити сімейство не можна, зате непотрібні RI можна **продати** на RI Marketplace.
- **Convertible RI** — до ~66%; можна **обміняти** на інше сімейство, ОС чи орендність.
- **Регіональний RI** — знижка на будь-яку AZ регіону (і гнучкість розміру в межах сімейства для Linux), але **без резервування ємності**.
- **Зональний RI** — прив'язаний до AZ і **резервує ємність** у ній.

Для EC2 AWS сьогодні рекомендує **Savings Plans** (простіше й гнучкіше). Але резервування лишаються основним способом знижки для **RDS**, **ElastiCache**, **OpenSearch**, **Redshift** (reserved instances / nodes) і DynamoDB (reserved capacity) — тож термін «Reserved» на іспиті зустрінатиметься.



Як обрати знижку

Хочеш максимум свободи (можливо, перейдеш на Fargate чи інший регіон) → **Compute Savings Plan**. **Точно знаєш сімейство і регіон на роки** → **EC2 Instance Savings Plan** (або Standard RI). **Треба ще й гарантована ємність у конкретній AZ** → зональний RI або **Capacity Reservation + Savings Plan**.

Spot: до 90% знижки за готовність до переривань

Spot Instances — вільні потужності AWS зі знижкою **до 90%**. Умова: AWS може **забрати** інстанс, коли потужність знадобиться іншим, попередивши за **2 хвилини** (сигнал у метаданих інстансу й подія в EventBridge; ще

раніше може прийти *rebalance recommendation* — «ризик переривання зростає»).

При перериванні інстанс за налаштуванням **видаляється** (типово), **зупиняється** або **присипляється** (hibernate).

Для чого Spot ідеальний: задачі, що можна перервати й продовжити: пакетна обробка, рендеринг, кодування відео, CI/CD, аналітика й big data (task-вузли EMR), контейнери, stateless-вебсервери за балансувальником (частина парку). **Не для:** основної бази даних, довгих задач без збереження проміжного стану (checkpoints), будь-чого, що не можна перервати.

Як використовувати Spot надійно:

- **Диверсифікація** — дозволь багато типів інстансів і всі AZ: що більше варіантів, то менше переривань.
- Стратегія розподілу **price-capacity-optimized** (рекомендована) — обирає пули з найменшим ризиком переривання і хорошою ціною.
- **EC2 Fleet / Spot Fleet** або **Auto Scaling group з mixed instances policy**: база з On-Demand + решта Spot.
- Застосунок має вміти **зберігати прогрес** і повторювати задачі (черги SQS, AWS Batch).

Dedicated: виділене обладнання

	Dedicated Host	Dedicated Instance
Що отримуєш	Цілий фізичний сервер під твої інстанси	Інстанси на обладнанні, що не ділиться з іншими клієнтами
Видимість заліза	Бачиш сокети, ядра, ID хоста; контролюєш розміщення	Ні
Для чого	BYOL — ліцензії, прив'язані до фізичних сокетів/ядер (Windows Server, SQL Server, Oracle...); вимоги комплаєнсу	Вимога «не ділити обладнання з іншими», без ліцензійних нюансів
Ціна	За хост (On-Demand або резервування)	За інстанс + доплата

Орендність (tenancy) інстансу: *default* (спільне обладнання), *dedicated* (Dedicated Instance), *host* (Dedicated Host).

Резервування ємності

Знижка ≠ гарантія, що потрібні сервери будуть доступні. Для важливої події (великий розпродаж, запуск продукту):

- **On-Demand Capacity Reservation** — резервує ємність конкретного типу в **конкретній AZ** на будь-який строк, без довгих зобов'язань. Платиш за зарезервоване навіть без використання, за ціною On-Demand. Знижку дає поєднання з **Savings Plans** чи регіональними RI.
- **Capacity Blocks for ML** — бронювання GPU-серверів на майбутні дати для навчання моделей.

Стратегія закупівлі Хмаринки

ПЕРЕРИВНЕ

 **Spot (до -90%)**

нічна обробка фото, звіти, тести — через AWS Batch з повторами

ПІКИ

 **On-Demand**

додаткові сервери сайту в розпродажі — Auto Scaling додає і прибирає

ПОСТІЙНА БАЗА

 **Compute Savings Plan (1 рік)**

4 сервери сайту цілодобово; план гнучкий, якщо сайт переїде в контейнери Fargate

ОСОБЛИВЕ

 **Dedicated Host**

стара складська програма з ліцензією на фізичні ядра — до її заміни

Правило: спершу **правильний розмір** (Compute Optimizer), потім **зобов'язання** на стабільну базу (не на пік!), піки — On-Demand, усе переривне — Spot.



Порахуймо (умовні ціни)

Нехай сервер коштує \$0,10/год On-Demand. Місяць $\approx$ 730 годин.

- 4 сервери цілодобово On-Demand: $4 \times 0,10 \times 730 = \mathbf{\$292/\text{міс}}$.
- Ті самі 4 під Compute Savings Plan зі знижкою $\sim 30\%$ (1 рік без передоплати): $\approx \mathbf{\$204/\text{міс}}$ — економія $\sim \$88$ щомісяця, $\sim \$1\,050$ на рік.
- Нічна обробка: 20 серверів $\times$ 2 год $\times$ 30 днів = 1 200 годин. On-Demand: $\mathbf{\$120/\text{міс}}$. Spot зі знижкою $\sim 70\%$: $\approx \mathbf{\$36/\text{міс}}$.
- Якби ми купили Savings Plan під пік (24 сервери), план простоював би більшу частину доби — ми платили б за невикористане зобов'язання. Тому зобов'язання — лише під **мінімальне постійне** навантаження.



Спробуй у справжньому AWS: ціни Spot і рекомендації знижок

Вартість: безкоштовно (лише перегляд). **Час:** 10 хвилин.

1. **EC2 $\rightarrow$ Spot Requests $\rightarrow$ Pricing history.** Обери тип `m7g.large`, ОС Linux, період — 3 місяці. Подивись ціни в різних AZ Франкфурта і порівняй з ціною On-Demand (калькулятор або сторінка цін EC2). Зверни увагу: ціни в різних AZ відрізняються — ще одна причина дозволяти Spot усі зони.
2. **Launch instance $\rightarrow$ розгорни Advanced details $\rightarrow$ Purchasing option** — побачиш, що Spot — просто прапорець при запуску (не запускай!).
3. **Billing and Cost Management $\rightarrow$ Savings Plans $\rightarrow$ Recommendations** — на новому акаунті тут порожньо, але саме тут AWS рахує рекомендований погодинний обсяг зобов'язання за історією використання (за 7, 30 або 60 днів). На Free plan купувати Savings Plans не можна — лише дивитися.

4. **calculator.aws** → EC2 → m7g.large , 4 шт., 100% часу → порівняй **On-Demand, Compute Savings Plans 1 рік / 3 роки** та **EC2 Instance Savings Plans**.

Прибирати нічого не треба.

Запам'ятай

On-Demand — без зобов'язань, найдорожче; короткі й непередбачувані навантаження.

Savings Plans — зобов'язання \$/год на 1–3 роки: **Compute** (~66%, будь-яке EC2 + Fargate + Lambda), **EC2 Instance** (~72%, сімейство в регіоні),  **Database** (~35%).

Reserved Instances — Standard (~72%, можна продати) і Convertible (~66%, можна обміняти); зональні резервують ємність. Для RDS, ElastiCache, Redshift, OpenSearch — основний спосіб знижки.

Spot — до **90%**, переривання з попередженням за **2 хвилини**; для переривних задач; диверсифікація + price-capacity-optimized.

Dedicated Host — видно сокети/ядра → **BYOL**; **Dedicated Instance** — просто без сусідів.

Capacity Reservation — гарантія ємності в AZ без знижки (знижку дають SP/RI).

Зобов'язання купують під **постійну базу**, а не під пік; спершу — правильний розмір.

Як питають на іспиті

Стабільне навантаження 24/7 на найближчі 1–3 роки, мінімальна ціна → **Savings Plans** або **Reserved Instances**

Знижка з можливістю змінювати сімейство, регіон або перейти на Fargate/Lambda → **Compute Savings Plan**

Пакетні задачі, які можна переривати й повторювати, найдешевше → **Spot Instances**

Критична задача, яку не можна переривати, на кілька годин → **On-Demand (не Spot)**

Ліцензії, прив'язані до фізичних ядер і сокетів (BYOL) → **Dedicated Hosts**

Гарантувати наявність серверів у конкретній AZ на важливу подію без довгого зобов'язання → **On-Demand Capacity Reservation**

Вебсервери за ALB: база гарантована, решта — якомога дешевше → **ASG з mixed instances: On-Demand база + Spot**

Непотрібні зарезервовані інстанси після переїзду → **продати Standard RI на RI Marketplace**

Знизити ризик переривань Spot → **диверсифікувати типи й AZ, стратегія price-capacity-optimized**

Пастки

Spot **не** для баз даних і задач без збереження прогресу.

Savings Plans і RI — це **знижка на оплату**, а не окремі сервери; вони не резервують ємність (крім зональних RI).

Зобов'язання «під пік» — переплата: план простоює, а платиш за нього.

Dedicated Instance не дає видимості сокетів — для BYOL «на ядро» потрібен **Dedicated Host**.

Перевір себе

ПИТАННЯ 1

Компанія має 10 серверів, що працюватимуть цілодобово щонайменше три роки. Протягом цього часу вона планує частину навантаження перевести на AWS Fargate. Який спосіб оплати дасть знижку і збереже гнучкість?

- A** EC2 Instance Savings Plan на 3 роки
- B** Compute Savings Plan на 3 роки
- C** Spot Instances
- D** Standard Reserved Instances на 3 роки

Правильна відповідь: B. Compute Savings Plan поширюється на EC2 будь-якого сімейства й регіону, а також на Fargate і Lambda. Standard RI і EC2 Instance SP прив'язані до типу чи сімейства, а Spot не підходить для постійного некритичного до переривань навантаження.

ПИТАННЯ 2

Щоночі 200 задач рендерингу відео мають виконатися до ранку. Кожну задачу можна перезапустити, якщо сервер зникне. Що найдешевше?

- A** On-Demand Instances
- B** Dedicated Hosts
- C** Spot Instances з диверсифікацією типів і AZ
- D** 3-річні Reserved Instances

Правильна відповідь: C. Переривні задачі з можливістю повтору — ідеальний сценарій для Spot зі знижкою до 90%. Диверсифікація зменшує ризик переривань.

ПИТАННЯ 3

Застосунок використовує ліцензоване ПЗ, вартість ліцензії якого залежить від кількості фізичних процесорних сокетів. Компанія хоче використати свої наявні ліцензії в AWS. Що обрати?

- A** Capacity Reservations
- B** Dedicated Instances
- C** Dedicated Hosts
- D** Spot Instances

Правильна відповідь: C. Dedicated Host дає цілий фізичний сервер з видимістю сокетів і ядер — це потрібно для BYOL-ліцензій «на сокет/ядро». Dedicated Instances такої видимості не дають.

ПИТАННЯ 4

Інтернет-магазин проводить розпродаж через два тижні і має гарантувати, що в eu-central-1a буде доступно 50 інстансів певного типу, без трирічного зобов'язання. Що використати?

- A** Spot Instances
- B** Compute Savings Plan
- C** On-Demand Capacity Reservation у цій AZ
- D** Регіональні Reserved Instances

Правильна відповідь: C. Capacity Reservation гарантує ємність у конкретній AZ на будь-який строк і скасовується, коли не потрібна. Savings Plans і регіональні RI дають знижку, але не гарантують ємності.

ПИТАННЯ 5

Яке твердження про Spot Instances правильне?

- A** AWS попереджає про переривання за 2 години
- B** Spot найкраще підходить для основної реляційної бази
- C** AWS попереджає про переривання приблизно за 2 хвилини
- D** Spot-інстанси не можна переривати, якщо вони вже запущені

Правильна відповідь: C. Попередження про переривання Spot надходить за дві хвилини (через метадані інстансу й EventBridge). Тому Spot підходить лише для задач, що витримують переривання.

ПИТАННЯ 6

Команда хоче знизити вартість вебтиру за ALB: мінімум 4 сервери мають працювати завжди без ризику переривань, а додаткові під час піків — якомога дешевше. Що налаштувати?

- A** Усі сервери — Spot
- B** Dedicated Hosts для всіх серверів
- C** Auto Scaling group з mixed instances policy: On-Demand база з 4 серверів, решта — Spot
- D** Усі сервери — On-Demand

Правильна відповідь: C. Mixed instances policy дає задати On-Demand base capacity і частку Spot понад неї. База надійна, піки — дешеві. Для бази можна ще й купити Savings Plan.

ПИТАННЯ 7

Компанія купила Standard Reserved Instances на 3 роки, але через рік перейшла на інше сімейство інстансів. Що можна зробити з непотрібними RI?

- A** Перенести на Spot Instances
- B** Скасувати і отримати повне повернення коштів
- C** Продати їх на Reserved Instance Marketplace
- D** Обміняти на Convertible RI безкоштовно

Правильна відповідь: C. Standard RI можна продати іншим клієнтам на RI Marketplace. Обмінювати на інші сімейства можна лише Convertible RI.



Диски серверів — Amazon EBS, instance store і снапшоти



Відмовостійкість · 26%



Продуктивність · 24%



Вартість · 20%

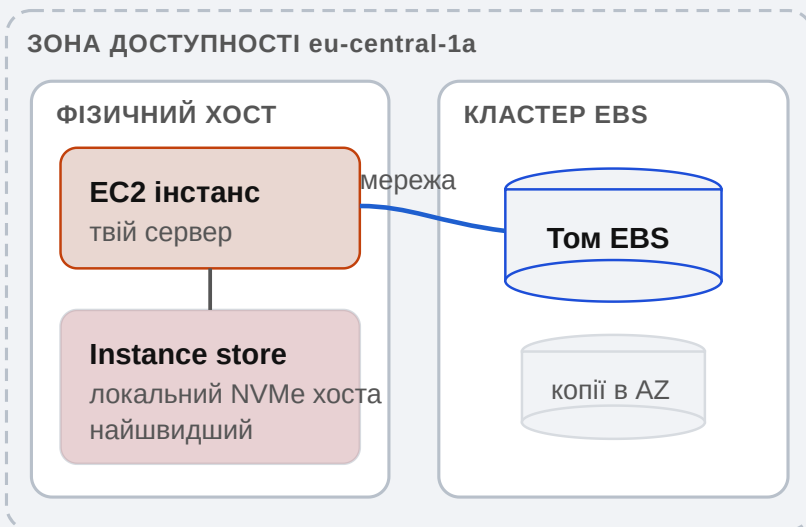


Історія Хмаринки. Три пригоди одного тижня. Перша: сервер пошуку товарів «літав», поки Тарас не зупинив його на ніч для економії — вранці індекс пошуку зник, бо лежав на **instance store**. Друга: у розпродаж база на EC2 гальмувала — диск упирався в ліміт **IOPS**, хоча процесор відпочивав. Третя: розробник видалив том з тестовими даними, і відновити їх не було звідки — **снапшотів** ніхто не робив. Сьогодні розберемося з дисками серверів раз і назавжди — з трьома картинками, які варто запам'ятати.



Образ. EBS — зовнішній SSD, підключений до ноутбука мережевим кабелем: вимкнув ноутбук — диск цілий; можна від'єднати й підключити до іншого ноутбука в **тій самій кімнаті (AZ)**. **Instance store** — вбудований диск орендованого ноутбука: найшвидший, але повертаєш ноутбук (stop) — і дані йдуть разом з ним. **Snapshot** — фотокопія диска, покладена в банківський сейф (S3): перша копія повна, наступні — лише змінені сторінки.

Два види дисків для EC2



Instance store:

зникає при stop, terminate, hibernate, збої хоста;
не від'єднується. Для кешу, буферів, тимчасового.

EBS: живе окремо, переживає stop; лише в одній AZ.

Головне: EBS не всередині сервера, а поруч у мережі тієї самої зони. Тому його можна від'єднати, перепідключити, збільшити, зробити знімок. А instance store — частина фізичного хоста і йде разом з ним.

Amazon EBS

Amazon Elastic Block Store (EBS) — мережеві блочні диски (томи, volumes) для EC2.

- **Живе в одній AZ** і підключається до серверів у **тій самій AZ**. Усередині AZ дані автоматично **реплікуються** — окремий зламаний носій нічого не зіпсує. (Від втрати всієї AZ рятують снапшоти.)
- Зазвичай один том підключено до **одного** сервера (але сервер може мати багато томів). Виняток — **Multi-Attach** (лише io1/io2): один том до **16 серверів на Nitro** в одній AZ; застосунок має вміти працювати з таким спільним диском (кластерна файлова система).
- **Переживає** зупинку сервера. При видаленні сервера кореневий том за замовчуванням видаляється (прапорець *Delete on termination*), а додаткові — ні.
- Розмір — до **64 TiB** на том. Платиш за **виділений** обсяг (і за додаткові IOPS/throughput), а не за записані дані.
- **Elastic Volumes** — змінити тип, розмір, IOPS і пропускну здатність **на льоту**, без зупинки сервера (після збільшення розміру розшир файлову систему в ОС).
- Шифрування — KMS; увімкни **encryption by default** для регіону (розділ 11).

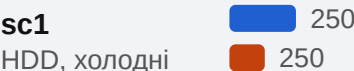
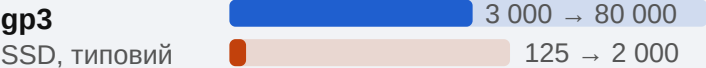
Типи томів EBS

Тип	Носій	Макс. IOPS	Макс. пропусчна	Для чого
gp3	SSD	3 000 базово, до 80 000	125 MiB/s базово, до 2 000 MiB/s	Типовий вибір: кореневі диски, більшість застосунків і баз. IOPS і пропусчна налаштовуються окремо від розміру
gp2	SSD	3 IOPS на кожен GiB (малі томи — сплески до 3 000), до 16 000	250 MiB/s	Старше покоління: щоб отримати IOPS, треба купувати гігабайти
io2 Block Express	SSD	до 256 000	до 4 000 MiB/s	Критичні бази з вимогою стабільних IOPS і затримки < 1 мс, довговічність 99,999%, Multi-Attach
io1	SSD	до 64 000	до 1 000 MiB/s	Старший Provisioned IOPS

Тип	Носій	Макс. IOPS	Макс. пропускна	Для чого
st1	HDD	500	500 MiB/s	Великі послідовні дані: логи, big data, сховища даних, ETL. Не може бути кореневим
sc1	HDD	250	250 MiB/s	Холодні дані, найдешевший. Не може бути кореневим

МАКСИМУМ НА ОДИН ТОМ

- IOPS (логарифмічна шкала)
- MiB/s (лінійна шкала)



У gp3 світла частина смуги — те, що можна докупити понад базові 3 000 IOPS і 125 MiB/s.

HDD-томи (st1, sc1) програвать SSD у тисячі разів за IOPS, але дешеві й тягнуть великі послідовні потоки. Тому для баз даних і корневих дисків — лише SSD.

Як обрати тип — чотири запитання:

1

Кореневий диск, база даних чи багато дрібних операцій?

Так → SSD (крок 2) · ні → HDD (крок 3)

2

SSD: потрібно понад 80 000 IOPS, затримка < 1 мс, Multi-Attach чи довговічність 99,999%?

Так → io2 BE · ні → gp3

3

HDD: великі послідовні дані читають часто (логи, big data, DWH)?

Так → st1 · рідко → sc1

На практиці 90% томів — gp3. Перехід gp2 → gp3 зазвичай дешевший на ~20% і робиться без простою (Elastic Volumes).



Глибше: чому диск «гальмує», хоча IOPS ще є

—

Продуктивність обмежує **найвужче місце**. Окрім ліміту тому, є **ліміт інстансу** на пропускну здатність до EBS (EBS-optimized bandwidth): маленький сервер не «прокачає» 80 000 IOPS, навіть якщо том це вміє. Тому під швидкий диск обирай відповідний тип сервера. Старий прийом «склеїти кілька томів у RAID 0» дає сумарні IOPS понад ліміт одного тому — але з io2 Block Express це рідко потрібно. RAID 1 (дзеркало) для EBS не має сенсу: том і так реплікується всередині AZ.

Instance store

Instance store — диски, **фізично вставлені в хост**, на якому працює сервер (у типах з літерою **d** — m7gd, c7gd — і в сімействі I).

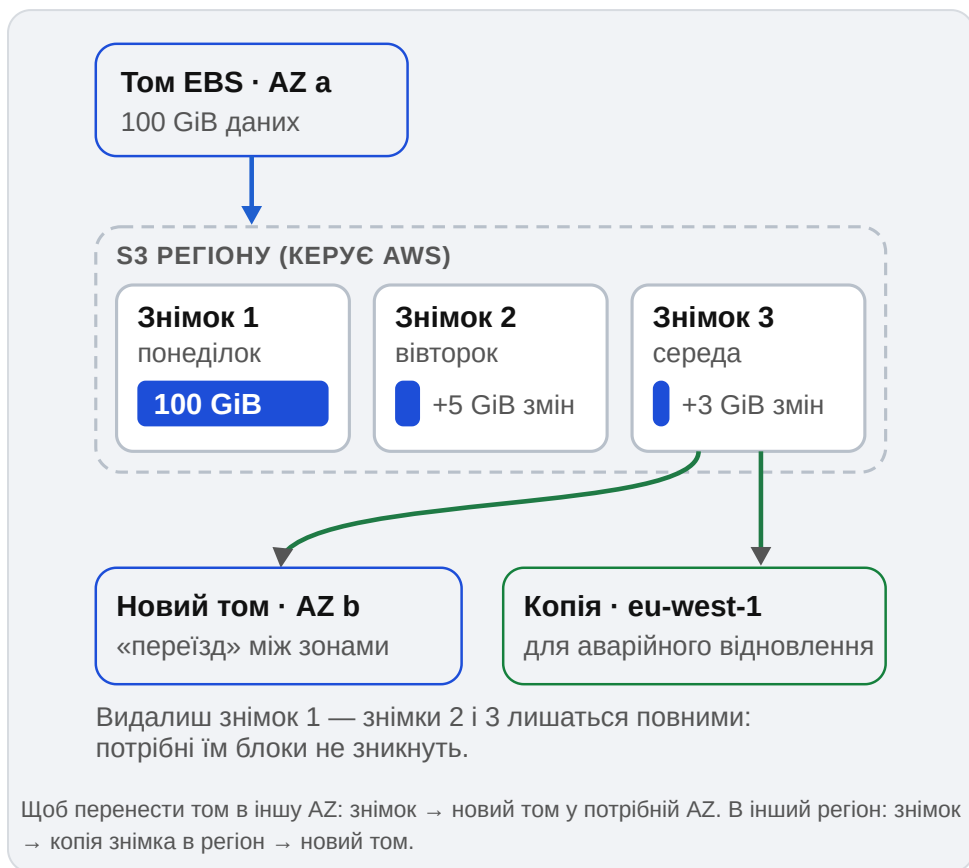
- **Найвища** продуктивність (NVMe, сотні тисяч і мільйони IOPS) і входить у ціну сервера.
- Дані **зберігаються при reboot**, але **зникають при stop, hibernate, terminate** і **збої хоста**.
- Не від'єднується і не переноситься; розмір фіксований типом сервера.

Для чого: **кеші, буфери, тимчасові й проміжні файли**, «scratch» для обчислень, а також дані, що **вже реплікуються** між серверами (вузли Cassandra, Kafka, Elasticsearch з репліками). Пошуковий індекс Хмаринки можна тримати на instance store — але лише якщо при старті сервер уміє перебудувати його або скопіювати з S3.

Снапшоти EBS

EBS snapshot — знімок тому на певний момент, що зберігається в **S3** (керованому AWS, окремого bucket ти не бачиш).

- **Інкрементальні**: перший знімок — усі використані блоки, кожен наступний — лише **змінені** з попереднього. Економно й швидко.
- **Регіональні**: зі знімка можна створити новий том **у будь-якій AZ** регіону. Так том «переїжджає» між зонами.
- **Копіювання** в інший регіон (DR) та іншому акаунту (шифрований знімок — лише з customer managed key).
- Видалення знімка прибирає лише ті блоки, які не потрібні іншим знімкам — **будь-який лишений знімок залишається повним**.



Корисні можливості знімків:

- **Fast Snapshot Restore (FSR)** — новий том зі знімка одразу видає **повну** продуктивність. Без FSR блоки довантажуються з S3 при першому читанні («ледаче завантаження»), і перші хвилини том повільніший.
- **Snapshot Archive** — архівний рівень, до ~75% дешевший для знімків, що рідко потрібні (зберігання від 90 днів, відновлення — години).
- **Recycle Bin** — «кошик»: видалені знімки й AMI можна відновити протягом заданого строку. Захист від випадкового чи зловмисного видалення.

- **Консистентність:** звичайний знімок — «як після раптового вимкнення живлення» (crash-consistent). Для баз даних краще робити знімки узгоджено з застосунком (зупинити запис на мить, VSS для Windows) або **multi-volume snapshots** для кількох томів одночасно.

Автоматизація:

- **Amazon Data Lifecycle Manager (DLM)** — політики «знімок щодня о 3:00, зберігати 7 штук, щотижня копіювати в інший регіон» для томів і AMI за тегами.
- **AWS Backup** — централізовані плани бекапів для **багатьох** сервісів (EBS, EC2, RDS, DynamoDB, EFS, S3...), з копіями між регіонами й акаунтами і захистом від видалення (Vault Lock) — розділ 42.

Рішення для Хмаринки

- Кореневі диски серверів сайту — **gp3** 20 GiB (зашифровані за замовчуванням).
- База на EC2 до переїзду в RDS — **io2 Block Express**: стабільні IOPS для розпродажу; сервер з достатньою пропускну здатністю EBS.
- Логи для нічної аналітики — **st1** (дешево, великий послідовний потік).
- Пошуковий індекс — **instance store** на сервері сімейства I з відновленням індексу з S3 при старті.
- **DLM**: щоденні знімки о 3:00 з зберіганням 7 днів, щотижнева копія в eu-west-1; **Recycle Bin** на 14 днів.

 **Спробуй у справжньому AWS: том, файл, знімок, переїзд в іншу AZ**

Вартість: том 1 GiB і знімок на годину-дві — частки цента; плюс сервер з розділу 19 (запусти заново, якщо видалив). **Час:** 25 хвилин.

1. **EC2** → **Volumes** → **Create volume**: тип **gp3**, розмір **1 GiB**, та сама **AZ**, що й сервер. Create.

2. Обери том → **Actions** → **Attach volume** → свій сервер, ім'я пристрою `/dev/sdf`.
3. Підключись до сервера (EC2 Instance Connect або Session Manager) і виконай:

```
lsblk                                # новий диск, напр. nvme1n1,
sudo mkfs -t xfs /dev/nvme1n1        # файлова система
sudo mkdir /data && sudo mount /dev/nvme1n1 /data
echo "Замовлення №1042" | sudo tee /data/order.txt
sudo umount /data
```

1. У консолі: том → **Actions** → **Detach volume**. Потім **Actions** → **Create snapshot**.
2. **Snapshots** → обери знімок → **Actions** → **Create volume from snapshot** → обери **іншу AZ**. Ось так дані «переїхали» між зонами.
3. На першому томі: **Actions** → **Modify volume** → зміни IOPS з 3000 на 4000 — це Elastic Volumes без зупинки (зверни увагу: наступна зміна можлива через 6 годин).

Прибери за собою: видали обидва томи й знімок; якщо сервер більше не потрібен — **Terminate**.

Запам'ятай

EBS — мережевий блочний диск: **одна AZ**, реплікація всередині AZ, переживає stop, до **64 TiB**, **Elastic Volumes** без простою.

gp3 — типовий (IOPS окремо від розміру: 3 000 → 80 000; 125 → 2 000 MiB/s). **io2 Block Express** — до **256 000 IOPS**, < 1 мс, 99,999%, **Multi-Attach**. **st1** — потоки великих даних; **sc1** — холодні; **HDD не бувають кореневими**.

Multi-Attach — лише io1/io2, до 16 Nitro-серверів в одній AZ.

Instance store — найшвидший, **ефемерний** (зникає при stop/terminate/збої) — для кешу й тимчасових даних.

Снапшоти — інкрементальні, у S3, регіональні; переїзд тому між AZ і регіонами — через знімок.

FSR — повна швидкість одразу; **Archive** — дешеве довге зберігання; **Recycle Bin** — захист від видалення.

Автоматизація знімків: **DLM** (EBS і AMI) або **AWS Backup** (багато сервісів).

Як питають на іспиті

База на EC2 потребує понад 100 000 стабільних IOPS і затримки менше 1 мс → **io2 Block Express**

Знизити витрати на EBS без простою → **перейти з gp2 на gp3 (Elastic Volumes)**

Невеликий том має давати 10 000 IOPS недорого → **gp3 (IOPS налаштовуються окремо від розміру)**

Дешевий диск для великих послідовних логів і big data → **st1**

Найдешевший блочний диск для рідко потрібних даних → **sc1**

Перенести том EBS в іншу AZ → **snapshot** → **новий том в іншій AZ**

Перенести том в інший регіон → **snapshot** → **copy snapshot** → **новий том**

Кластерний застосунок на кількох серверах з одним спільним блочним томом → **EBS Multi-Attach (io1/io2)**

Найвищі IOPS для тимчасових даних або кешу → **instance store**

Щоденні знімки з автоматичним видаленням старих → **Data Lifecycle Manager (або AWS Backup)**

Новий том зі знімка має одразу працювати на повній швидкості → **Fast Snapshot Restore**

Захистити знімки від випадкового видалення → **Recycle Bin**

Дешево зберігати річні знімки для аудиту → [EBS Snapshot Archive](#)

Пастки

EBS не підключиться до сервера в **іншій** AZ — лише через знімок.

st1 і sc1 **не** можуть бути кореневими томами.

Instance store зникає при **stop** — навіть без видалення сервера.

Дані в instance store не можна «зберегти знімком EBS» — снапшоти лише для EBS.

Продуктивність обмежують і том, і тип сервера — маленький сервер не видасть максимуму io2.

Перевір себе

ПИТАННЯ 1

Компанія запускає базу даних на EC2 і потребує стабільних 150 000 IOPS із затримкою менше мілісекунди. Який тип EBS обрати?

A st1

B io2 Block Express

C gp3

D gp2

Правильна відповідь: B. gp3 дає до 80 000 IOPS, gp2 — до 16 000. io2 Block Express — до 256 000 IOPS із затримкою менше 1 мс і найвищою довговічністю. st1 — HDD для послідовних потоків.

ПИТАННЯ 2

Сервер обробляє великі файли логів послідовно: потрібна висока пропускна здатність за мінімальну ціну, диск не буде кореневим. Що обрати?

- A** st1
- B** sc1
- C** Instance store
- D** io2 Block Express

Правильна відповідь: A. st1 — throughput optimized HDD: дешевий і розрахований на великі послідовні потоки (до 500 MiB/s). sc1 дешевший, але повільніший і для холодних даних, io2 — дорогий SSD для IOPS.

ПИТАННЯ 3

Том EBS з даними знаходиться в eu-central-1a, а новий сервер — в eu-central-1b. Як зробити дані доступними новому серверу?

- A** Від'єднати том і підключити до сервера в іншій AZ
- B** Створити знімок тому і з нього — новий том в eu-central-1b
- C** Увімкнути Multi-Attach
- D** Скопіювати том в інший регіон

Правильна відповідь: B. Том EBS прив'язаний до своєї AZ. Знімки регіональні: зі знімка створюється новий том у будь-якій AZ регіону. Multi-Attach працює лише в межах однієї AZ.

ПИТАННЯ 4

Після зупинки й повторного запуску сервера з його локального диска зникли тимчасові файли кешу. Яке пояснення правильне?

- A** Снапшот автоматично перезаписав диск
- B** Кеш був на EFS, а EFS очищується при stop
- C** Кеш зберігався на instance store, дані якого не переживають stop
- D** Том EBS видаляється при зупинці

Правильна відповідь: C. Instance store — ефемерний: дані зберігаються при reboot, але зникають при stop, hibernate, terminate і збої хоста. EBS зберігає дані при зупинці.

ПИТАННЯ 5

Команда хоче щодня о 3:00 робити знімки всіх томів з тегом Backup=daily і зберігати їх 14 днів, з мінімальними операційними зусиллями. Що використати?

- A** EBS Multi-Attach
- B** Amazon Data Lifecycle Manager (або план AWS Backup)
- C** Вручну створювати знімки щодня
- D** Cron-скрипт на окремому сервері, що викликає CLI

Правильна відповідь: B. DLM автоматично створює і видаляє знімки за розкладом і тегоми. AWS Backup робить те саме централізовано для багатьох сервісів. Скрипт на сервері — зайва операційна робота.

ПИТАННЯ 6

Що правильно щодо знімків EBS? (Оберіть 2)

Оберіть 2 відповіді

- A** Знімки можна робити лише зупиненого тому
- B** Знімок можна скопіювати в інший регіон
- C** Видалення першого знімка робить наступні непридатними
- D** Знімки зберігаються в тій самій AZ, що й том
- E** Кожен наступний знімок зберігає лише змінені з попереднього блоки

Правильна відповідь: B, E. Знімки інкрементальні, регіональні (зберігаються в S3 регіону) і копіюються між регіонами. Видалення знімка прибирає лише непотрібні іншим блоки — решта знімків лишаються повними. Знімок можна робити на працюючому томі.

ПИТАННЯ 7

Том 200 GiB gp2 дає замало IOPS для бази, а збільшувати розмір лише заради IOPS не хочеться. Яке найекономніше рішення без простою?

- A** Замінити на io2 Block Express з 64 000 IOPS
- B** Перейти на st1
- C** Змінити тип на gp3 через Elastic Volumes і задати потрібні IOPS окремо від розміру
- D** Додати instance store

Правильна відповідь: C. У gp2 IOPS прив'язані до розміру (3 IOPS на GiB). gp3 дозволяє купити IOPS і пропускну здатність окремо, зазвичай дешевше, і зміна типу відбувається на льоту.



Балансувальники навантаження — ALB, NLB і Gateway Load Balancer

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Безпека · 30%



Історія Хмаринки. Тепер у Хмаринки кілька серверів сайту в двох зонах. Але покупці знають лише одну адресу — `khmarynka.ua`. Хто вирішить, на який сервер відправити кожен запит? Хто помітить, що сервер у зоні b «завис», і перестане слати туди людей? Хто візьме на себе HTTPS, щоб серверам не возитися з сертифікатами? І як відправити `api.khmarynka.ua` на одні сервери, а `/admin` — на інші, з окремим входом? Усе це — робота **балансувальника навантаження**.



Образ. Балансувальник — адміністратор на рецепції великого ресторану. Гості (запити) приходять до одних дверей, а він розсаджує їх по вільних столиках (серверах), обходячи ті, де офіціант захворів (health checks). **ALB** — адміністратор, що читає записку гостя («я на бізнес-ланч», «я з дітьми») і веде у відповідний зал. **NLB** — швейцар на вході величезного стадіону: не читає записок, але пропускає мільйони людей за секунду і завжди стоїть на тому самому місці (статична адреса). **GWLB** — пункт огляду: кожен гість спершу проходить через рамку металошукача (файрвол), а лише потім — у зал.

Навіщо балансувальник

Elastic Load Balancing (ELB) — керовані балансувальники AWS. Вони:

- **розподіляють** запити між серверами (targets) у **кількох AZ**;
- постійно роблять **health checks** і відправляють трафік лише на здорові цілі;
- дають **одну точку входу** (DNS-ім'я), за якою сервери можна додавати й прибирати (Auto Scaling);
- **завершують TLS** (сертифікати ACM), знімаючи шифрування з серверів;
- самі **масштабуються** й високодоступні — достатньо обрати підмережі щонайменше у двох AZ.

Будова:

- **Listener** — «на якому протоколі й порту слухати» (HTTPS:443).
- **Rules** — правила: «якщо шлях `/api/*` → відправ у target group api».
- **Target group** — група цілей (сервери EC2, IP-адреси, функції Lambda, контейнери ECS) з налаштуваннями health check.
- Балансувальник буває **internet-facing** (публічний, у публічних підмережах) або **internal** (лише всередині VPC — між шарами застосунку).

Application Load Balancer (L7)

ALB працює з **HTTP/HTTPS** (а також **HTTP/2**, **gRPC**, **WebSocket**) і бачить **вміст запиту**. Тому вміє розумну маршрутизацію:

Умова правила	Приклад
Host (ім'я домену)	<code>api.khmarynka.ua</code> → target group api; <code>khmarynka.ua</code> → web
Path (шлях)	<code>/images/*</code> → сервери зображень; <code>/api/*</code> → API
HTTP-заголовок, метод, query string	<code>?version=beta</code> → нова версія

Умова правила	Приклад
IP джерела	/admin/* лише з офісної мережі

Дії правил:

- **forward** — у target group, зокрема **зважаєно між кількома** (90% — стара версія, 10% — нова: blue/green і канаркові релізи);
- **redirect** — наприклад, **HTTP** → **HTTPS**;
- **fixed-response** — повернути готову відповідь (сторінка «технічні роботи», 404);
- **authenticate** — вимагати вхід через **Cognito** або будь-який **OIDC**-провайдер ще до того, як запит дійде до застосунку.

Інші можливості ALB:

- **Кілька сертифікатів** на одному listener завдяки **SNI**: `khmarynka.ua`, `khmarynka.pl`, `api.khmarynka.ua` — кожен зі своїм сертифікатом.
- **Цілі**: EC2, IP-адреси, **функції Lambda**, контейнери ECS з **динамічними портами** (кілька контейнерів на одному сервері).
- **Sticky sessions** — прив'язати користувача до одного сервера cookie (якщо застосунок тримає сесію на сервері; краще — винести сесії в кеш).
- **Security Group** на самому ALB; **WAF**; журнали доступу в S3.
- Застосунок бачить IP клієнта в заголовку **X-Forwarded-For** (бо TCP-з'єднання проходить від ALB).
- **Статичної IP-адреси немає** — лише DNS-ім'я.



Покупці

HTTPS :443 (HTTP :80 → redirect на HTTPS)



ALB

Сертифікати ACM (SNI), WAF, 2 публічні підмережі



Правила

Host `api.khmarynka.ua` → TG «api» (90% v1 / 10% v2)

Path `/admin/*` + IP офісу → вхід Cognito → TG «admin»

Default → TG «web»



Target groups

Сервери в приватних підмережах 2 AZ, health check `/health`

Один балансувальник обслуговує кілька доменів і сервісів. Сервери не мають публічних адрес і не знають про сертифікати.

Network Load Balancer (L4)

NLB працює з **TCP, UDP і TLS** на транспортному рівні. Не бачить URL і заголовків, зате:

- витримує **мільйони запитів за секунду з наднизькою затримкою**;
- має **статичну IP-адресу в кожній AZ** (можна призначити свої **Elastic IP**) — для клієнтів, що вносять адреси в білі списки;
- **зберігає IP клієнта** (для цілей-інстансів);
- лежить в основі **PrivateLink**: власний сервіс для інших VPC публікують саме через NLB (розділ 15);

- цілі: EC2, IP-адреси (зокрема сервери в офісі через VPN/DX) і навіть **ALB** — так можна поєднати статичні IP NLB з розумною маршрутизацією ALB;
- підтримує Security Groups і завершення TLS.

Для чого: ігрові сервери, IoT, VoIP, фінансові системи з мінімальною затримкою, будь-які не-HTTP протоколи, статичні IP.

Gateway Load Balancer (L3)

GWLB — для віртуальних мережевих апаратів сторонніх виробників: файрволів нового покоління, систем виявлення й запобігання вторгненням (IDS/IPS), глибокої інспекції пакетів. Він **прозора** пропускає весь трафік через парк таких апаратів (протокол **GENEVE**, порт **6081**), розподіляє навантаження між ними й прибирає нездорові. У VPC-споживачах трафік спрямовують на **GWLB endpoint** через таблиці маршрутів.

На іспиті: «пропустити весь трафік VPC через файрвол-апарат стороннього вендора, масштабовано» → **Gateway Load Balancer**. (Керований файрвол AWS без сторонніх апаратів — **AWS Network Firewall**, розділ 12.)

Classic Load Balancer (CLB) — застарілий, у нових рішеннях не обирають.

Порівняння

	ALB	NLB	GWLB
Рівень	7 (HTTP/HTTPS, gRPC, WebSocket)	4 (TCP, UDP, TLS)	3 (IP-пакети, GENEVE)
Маршрутизація	Хост, шлях, заголовки, метод, IP	Порт і протокол	Весь трафік через апарати
Статична IP	❌ (лише DNS)	✅ по одній на AZ, Elastic IP	—
Продуктивність	Висока	Екстремальна, мінімальна затримка	Висока
Цілі	EC2, IP, Lambda , ECS	EC2, IP, ALB	Апарати (EC2)
WAF	✅	❌	❌

	ALB	NLB	GWLB
Автентифікація користувачів	✓ Cognito / OIDC	✗	✗
Типові задачі	Веб, API, мікросервіси, контейнери	Ігри, IoT, статичні IP, PrivateLink	Сторонні файрволи, IDS/IPS

Важливі налаштування

- **Health checks:** протокол, шлях (/health), інтервал, кількість успіхів/невдач. Добра перевірка дивиться, чи застосунок справді працює (може звернутися до бази), а не лише чи відкритий порт.
- **Cross-zone load balancing:** розподіляти трафік рівномірно між **усіма** цілями в усіх AZ, а не між зонами порівну. **ALB** — увімкнено завжди й безкоштовно. **NLB і GWLB** — **вимкнено** за замовчуванням; якщо ввімкнути — трафік між AZ платний.
- **Deregistration delay** (connection draining) — перш ніж прибрати ціль (scale-in, оновлення), балансувальник чекає завершення поточних запитів (за замовчуванням **300 секунд**, можна 0–3600).
- **TLS:** завершувати на балансувальнику (сертифікат ACM), а для вимоги «шифрувати на всьому шляху» — **знову шифрувати** до цілей (HTTPS до серверів). NLB може пропустити TLS наскрізь без розшифрування (TCP listener) — тоді сертифікат на серверах.
- **Коди помилок балансувальника:** **502** — ціль дала некоректну відповідь або обірвала з'єднання; **503** — **немає здорових цілей**; **504** — ціль **не відповіла вчасно** (таймаут).



Одним рядком

HTTP і розумна маршрутизація → ALB. Швидкість, TCP/UDP, статичні IP → NLB. Сторонні файрвол-апарати → GWLB. WAF — лише на ALB (з трьох).

Спробуй у справжньому AWS: ALB над двома серверами

Вартість: ALB і два маленькі сервери на 30–40 хвилин — кілька центів з кредитів. **Час:** 35 хвилин.

1. Запусти **два** сервери як у розділі 19 (з user data, що показує hostname), у **різних** AZ, у публічних підмережах. Security Group серверів: HTTP 80 **лише від Security Group балансувальника** (створи sg-lab-alb з HTTP 80 з 0.0.0.0/0 задалегідь).
2. **EC2** → **Target groups** → **Create**: тип Instances, HTTP:80, health check path / . Зареєструй обидва сервери.
3. **Load balancers** → **Create** → **Application Load Balancer**: internet-facing, дві публічні підмережі в різних AZ, SG sg-lab-alb , listener HTTP:80 → твоя target group.
4. Коли стан Active, відкрий DNS-ім'я ALB і кілька разів онови сторінку — відповідатимуть різні сервери.
5. Зайди на один сервер і виконай `sudo systemctl stop nginx` . Через хвилину в target group він стане **unhealthy**, а сайт працюватиме далі — лише з другим сервером.
6. Listener → **Add rule**: умова Path /sale* → дія **Return fixed response** 200, текст «Розпродаж скоро!». Відкрий /sale на DNS-імені ALB.

Прибери за собою: видали ALB, target group, обидва сервери і Security Groups.

Запам'ятай

ELB: listener → rules → target groups з health checks; щонайменше **2 AZ**; internet-facing або internal.

ALB (L7): маршрутизація за хостом, шляхом, заголовками, методом, IP; зважені target groups (blue/green); redirect HTTP → HTTPS; fixed response; автентифікація **Cognito/OIDC**; **SNI**; цілі — EC2, IP, **Lambda**, ECS; **WAF**; без статичної IP.

NLB (L4): TCP/UDP/TLS, мільйони запитів/с, **статична IP на AZ (Elastic IP)**, зберігає IP клієнта, основа **PrivateLink**, може мати ціллю ALB.

GWLB (L3): GENEVE 6081, прозорий прогін трафіку через **сторонні апарати безпеки**.

Cross-zone: ALB — завжди; NLB/GWLB — вимкнено за замовчуванням (платно між AZ).

Deregistration delay — 300 с; помилки: **502** — погана відповідь цілі, **503** — немає здорових цілей, **504** — таймаут.

Як питають на іспиті

Маршрутизувати `/api` і `/images` на різні групи серверів → **ALB path-based routing**

Кілька доменів з різними сертифікатами на одному балансувальнику → **ALB з SNI**

Поступово перевести 10% трафіку на нову версію без DNS → **ALB weighted target groups**

Автентифікувати користувачів до того, як запит дійде до застосунку → **ALB + Cognito / OIDC**

Клієнти вносять IP-адресу балансувальника в білий список → **NLB з Elastic IP (або Global Accelerator)**

TCP/UDP, мільйони запитів за секунду, мінімальна затримка → **NLB**

Пропустити трафік через фایрвол-апарат стороннього вендора з масштабуванням → **Gateway Load Balancer**

Функція Lambda має відповідати на HTTP-запити за балансувальником → **ALB з ціллю Lambda**

Захистити застосунок за балансувальником від SQL injection → **WAF на ALB (WAF не ставиться на NLB)**

Потрібні і статичні IP, і маршрутизація за шляхом → **NLB з ціллю ALB (або Global Accelerator перед ALB)**

Користувачі отримують 503 від ALB → **немає здорових цілей: перевір health checks і Security Groups**

Сервери видаляються при scale-in, а поточні запити обриваються → **налаштувати deregistration delay**

Пастки

ALB **не має** статичної IP — будь-який варіант «призначити Elastic IP на ALB» неправильний.

WAF **не** працює з NLB і GWLB.

У NLB cross-zone вимкнений за замовчуванням: при нерівній кількості серверів у зонах навантаження розподіляється нерівно.

CLB — застарілий, не обирай для нових рішень.

Security Group серверів має дозволяти трафік **від балансувальника** (краще — з посиланням на його SG), інакше health checks провалюються.

✓ Перевір себе

ПИТАННЯ 1

Вебзастосунок має відправляти запити до `/api/*` на одну групу серверів, а решту — на іншу, і використовувати сертифікати для трьох доменів. Який балансувальник обрати?

- A** Gateway Load Balancer
- B** Network Load Balancer з TLS listener
- C** Classic Load Balancer
- D** Application Load Balancer з path-based правилами і SNI

Правильна відповідь: D. ALB працює на рівні 7: бачить шлях і хост, підтримує кілька сертифікатів на одному listener через SNI. NLB не бачить шляхів, GWLB — для апаратів безпеки, CLB застарілий.

ПИТАННЯ 2

Банк-партнер дозволяє з'єднання лише з двох-трьох фіксованих IP-адрес, які він внесе у свій фаїрвол. Застосунок працює в одному регіоні на TCP. Що обрати?

- A** CloudFront distribution
- B** Network Load Balancer з Elastic IP у кожній AZ
- C** Application Load Balancer з Elastic IP
- D** Route 53 weighted routing

Правильна відповідь: B. NLB має статичну IP в кожній AZ, і їх можна замінити своїми Elastic IP. ALB не підтримує статичних IP, а в CloudFront адреси змінюються.

ПИТАННЯ 3

Компанія має закон, що вимагає пропускати весь вхідний і вихідний трафік VPC через фаїрвол-апарати сертифікованого стороннього виробника з автоматичним масштабуванням. Що використати?

- A** Application Load Balancer з правилами
- B** Gateway Load Balancer з парком апаратів вендора
- C** Network ACL
- D** AWS WAF

Правильна відповідь: B. GWLB прозоро розподіляє весь трафік між віртуальними апаратами сторонніх вендорів (GENEVE, порт 6081) і стежить за їхнім здоров'ям. WAF працює лише з HTTP, NACL — прості правила без апаратів.

ПИТАННЯ 4

Користувачі отримують від ALB помилку 503 Service Unavailable. Що найімовірніше?

- A** Сертифікат прострочено
- B** Сервер повернув некоректну відповідь
- C** У target group немає здорових цілей
- D** Застосунок відповідає занадто повільно

Правильна відповідь: C. 503 від балансувальника зазвичай означає, що немає зареєстрованих або здорових цілей. Некоректна відповідь — 502, таймаут — 504.

ПИТАННЯ 5

Сервери за NLB розподілені нерівно: 2 в AZ a і 6 в AZ b. Сервери в AZ a перевантажені. Що допоможе?

- A** Увімкнути cross-zone load balancing на NLB
- B** Змінити health check на HTTP
- C** Перейти на Classic Load Balancer
- D** Додати Security Group на NLB

Правильна відповідь: A. Без cross-zone кожен вузол NLB ділить трафік лише між цілями своєї зони, тож зона з меншою кількістю серверів перевантажується. У NLB cross-zone вимкнений за замовчуванням (увімкнення додає плату за трафік між AZ); в ALB — увімкнений завжди.

ПИТАННЯ 6

Команда хоче, щоб запити до вебзастосунку проходили автентифікацію через Amazon Cognito ще до того, як потраплять на сервери, без змін у коді застосунку. Що налаштувати?

- A** CloudFront geo restriction
- B** Правило ALB з дією authenticate-cognito
- C** NLB з TLS listener
- D** Security Group з обмеженням по IP

Правильна відповідь: B. ALB вміє автентифікувати користувачів через Cognito User Pool або OIDC і лише потім пересилати запит цілям, передаючи дані користувача в заголовках.

ПИТАННЯ 7

Які твердження про Application Load Balancer правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** Працює лише з протоколом UDP
- B** Може мати функцію Lambda як ціль
- C** Потребує cross-zone load balancing, який вимкнено за замовчуванням
- D** Підтримує AWS WAF
- E** Має статичну IP-адресу, яку можна замінити на Elastic IP

Правильна відповідь: B, D. ALB підтримує цілі Lambda і інтеграцію з WAF. Статичних IP у нього немає, UDP він не обробляє, а cross-zone у ALB увімкнений завжди.



Auto Scaling — сервери, що додаються й замінюються самі



Відмовостійкість · 26%



Продуктивність · 24%



Вартість · 20%



Історія Хмаринки. Друга «чорна п'ятниця» в хмарі. О 19:55 на сайті 300 покупців, о 20:05 — 6 000. Минулого року Тарас сидів би до ранку й вручну запускав сервери, а о третій ночі — забув би їх вимкнути і приніс би Марті рахунок-сюрприз. Цього року він спить: кількість серверів сама виросла з 4 до 18, а до ранку — сама повернулася до 4. Посеред ночі один сервер «завис» — його теж замінили без людей. Це **EC2 Auto Scaling**.



Образ. Менеджер супермаркету: черги ростуть — відкриває нові каси, зал порожніє — закриває; касир захворів — ставить заміну; знає, що в понеділок о 9:00 завжди наплив, і відкриває каси заздалегідь; а з досвіду кількох місяців уже передбачає щоденні піки. Каси — сервери, черги — навантаження, менеджер — Auto Scaling.

Auto Scaling group

Auto Scaling group (ASG) — група однакових серверів EC2, кількість яких автоматично підтримується і змінюється. Налаштування:

- **Launch template** — що запускати: AMI, тип, Security Groups, IAM-роль, user data, диски (розділ 19). Шаблони мають версії. (Старі *launch configurations* застаріли.)

- **Min / Desired / Max** — мінімум серверів, поточна бажана кількість і стеля. ASG ніколи не опуститься нижче min і не перевищить max.
- **Підмережі в кількох AZ** — ASG рівномірно розподіляє сервери по зонах і при втраті зони запускає заміну в інших.
- **Прив'язка до target group** балансувальника — нові сервери автоматично реєструються в ALB/NLB, видалені — знімаються.

Самолікування: health checks

ASG постійно перевіряє сервери і **замінює нездорові** (видаляє і запускає новий):

- **EC2 health check** (за замовчуванням) — чи працює сам інстанс (status checks).
- **ELB health check** — чи **відповідає застосунок** на перевірку балансувальника. Обов'язково **увімкни** його: інакше сервер із «завислим» застосунком, але живою ОС, ASG вважатиме здоровим і не замінить.
- **Health check grace period** — скільки секунд після запуску не перевіряти здоров'я, поки застосунок стартує. Закороткий — ASG вбиватиме сервери, які просто не встигли запуститися.



ASG навіть для одного сервера

Старий застосунок, що не вміє працювати на кількох серверах, усе одно варто запускати в ASG з **min = max = desired = 1** у кількох підмережах. Якщо сервер чи ціла AZ впаде — ASG сам підніме новий в іншій зоні. Дешева висока доступність без переписування коду.

Політики масштабування

Політика	Як працює	Коли обирати
Target tracking	«Тримай метрику на рівні»: середній CPU 50% або 800 запитів на сервер. ASG сам рахує, скільки серверів додати чи прибрати	Типовий вибір — найпростіше і найкраще для більшості
Step scaling	Аларм CloudWatch + «сходинки»: CPU 60–75% → +2, понад 75% → +5	Потрібна різна реакція на різну силу навантаження
Simple scaling	Одна дія на аларм + пауза (cooldown)	Застаріла; на іспиті — рідко правильна
Scheduled	Дія за розкладом: «у п'ятницю о 19:30 — min 12»	Відомий заздалегідь пік
Predictive	Машинне навчання прогнозує циклічне навантаження за історією і масштабує заздалегідь	Щоденні чи щотижневі повторювані піки, довгий старт серверів

Метрики для target tracking: середній **CPU**, **ALBRequestCountPerTarget** (запити на сервер — часто точніше за CPU для веб), мережевий трафік або **власна метрика**.

Воркери з черги SQS

Для серверів, що обробляють чергу (розділ 31), CPU — погана метрика. Правильна: **backlog per instance** — кількість повідомлень у черзі (`ApproximateNumberOfMessagesVisible`), поділена на кількість серверів. Target tracking тримає її на рівні «скільки один сервер встигає обробити за прийнятний час». Черга росте — серверів більше, черга порожня — лишається мінімум.

Паузи: cooldown і warmup

Щойно запущений сервер ще не працює на повну: якщо відразу рахувати його метрики, ASG може запанікувати й додати зайвого. **Default instance warmup** (для target tracking і step) каже, скільки секунд не враховувати метрики нового сервера. **Cooldown** (300 с за замовчуванням) — пауза між діями простої політики.

🌙 03:00

Навантаження мінімальне

ASG тримає **min = 4** (по 2 у кожній AZ)

🌐 18:00

Predictive scaling

Прогноз вечірнього піку — сервери додаються заздалегідь

📅 19:30

Scheduled action

Старт розпродажу: **min = 12**

🔥 20:05

Target tracking

800 запитів на сервер → ASG росте до **18**

Політики працюють разом: розклад і прогноз готують ємність наперед, а target tracking реагує на реальність. Вранці scheduled action повертає **min = 4**, і зайве прибирається.

Життєвий цикл і прискорення

- **Lifecycle hooks** — «пауза» при запуску чи видаленні сервера, щоб виконати свої дії: встановити ПЗ, зареєструвати сервер дець, а перед видаленням — **забрати логи**, завершити задачі. Сповіщення йдуть в EventBridge, SNS чи SQS; пауза може тривати годинами.
- **Warm pools** — пул заздалегідь підготовлених серверів (зупинених, «приспаних» або запущених). Для застосунків з довгим стартом масштабування стає швидким: сервер береться з пулу, а не створюється з нуля.
- **Instance refresh** — поступово замінити **всі** сервери групи на нову версію launch template чи AMI: хвилями, зберігаючи мінімальний відсоток здорових, з можливістю відкату. Так застосовують «незмінну інфраструктуру» — не латають живі сервери, а замінюють їх новими.
- **Scale-in protection** — захистити конкретні сервери від видалення при зменшенні групи (наприклад, той, що виконує довгу задачу).

Кого видаляти при зменшенні

Політика видалення за замовчуванням (спрощено): спершу обирається **AZ з найбільшою кількістю серверів** (щоб зберегти баланс зон), у ній — сервер з **найстарішим launch template**, далі — той, що найближчий до

наступної години оплати. Можна задати інші політики (найстаріший/найновіший інстанс) або власну через Lambda.

Змішаний парк: On-Demand + Spot

Mixed instances policy дозволяє одній ASG:

- використовувати **кілька типів** інстансів (m7g, m6g, c7g...) або задати їх **атрибутами** («2–4 vCPU, 8–16 GiB пам'яті» — ASG сам підбере всі підходящі типи);
- поєднувати способи оплати: **On-Demand base capacity** (гарантована база) + відсоток **Spot** понад базу;
- для Spot — стратегія **price-capacity-optimized** і **Capacity Rebalancing** (завчасно замінювати Spot-сервери з високим ризиком переривання).

Масштабування інших ресурсів

Автомасштабування є не лише для EC2. **Application Auto Scaling** масштабує: сервіси **ECS**, ємність таблиць **DynamoDB**, репліки **Aurora**, Spot Fleet, функції Lambda (provisioned concurrency), EMR, SageMaker та інше. Сервіс **AWS Auto Scaling** дає «план масштабування» для кількох ресурсів застосунку разом.

Щоб масштабування працювало: stateless

Сервер у ASG може з'явитися і зникнути будь-якої миті. Тож на ньому не можна тримати нічого унікального:

- **сесії й кошики** — в **ElastiCache** або **DynamoDB**;
- **завантажені файли** — в **S3** (або спільна **EFS**);
- **налаштування** — в **Parameter Store** / **Secrets Manager**;
- **логи** — одразу в **CloudWatch Logs**;
- **ПЗ і версія застосунку** — в **AMI** чи контейнерному образі.

Тоді всі сервери взаємозамінні — «худоба, а не домашні улюбленці» (cattle, not pets).

Спробуй у справжньому AWS: група, що лікує сама себе

Вартість: 2–3 маленькі сервери на 30 хвилин — центи з кредитів. **Час:** 30 хвилин.

1. **EC2** → **Launch templates** → **Create**: AMI Amazon Linux 2023, тип `t3.micro`, Security Group з HTTP 80, user data з розділу 19, IMDSv2 required.
2. **Auto Scaling groups** → **Create**: шаблон з кроку 1, VPC і **дві публічні підмережі** в різних AZ. (Якщо лишився ALB з розділу 22 — прикріпи його target group і увімкни **ELB health checks**.)
3. Group size: **Desired 2, Min 2, Max 4**. Scaling policy: **Target tracking**, Average CPU utilization **50%**.
4. Коли два сервери запускаються (**Activity** група), зайди на вкладку **Instances** і вручну **Terminate** один з них. Подивись **Activity**: ASG помітить і запустить заміну.
5. Зміни **Desired** на 3 — з'явиться третій сервер. Поверни на 2 — один видалиться (подумай, який саме і чому).
6. Створи **Scheduled action**: на завтра о 09:00 — Min 3; і ще одну о 18:00 — Min 2. Подивись, як вони виглядають у консолі (і видали їх після перегляду).

Прибери за собою: видали ASG (разом з серверами), launch template, Security Group; ALB — якщо залишався.

Запам'ятай

ASG: launch template + **min/desired/max** + підмережі в **кількох AZ** + target group балансувальника.

Самолікування: нездорові сервери замінюються; **увімкни ELB health checks**; задай **grace period** під час старту застосунку.

Навіть **min = max = 1** у кількох AZ — дешева НА для старого застосунку.

Політики: **target tracking** (типова), **step, scheduled** (відомий пік), **predictive** (циклічні піки, ML), **simple** (застаріла).

Воркери черги → масштабування за **backlog per instance**.

Lifecycle hooks — дії при запуску/видаленні; **warm pools** — швидкий старт; **instance refresh** — заміна всіх серверів на новий AMI; **scale-in protection**.

Видалення за замовчуванням: спершу баланс AZ, потім найстаріший шаблон.

Mixed instances: кілька типів + On-Demand база + Spot.

Сервери в ASG мають бути **stateless**.

Як питають на іспиті

Тримати середній CPU групи близько 50% → **target tracking scaling policy**

Відомий пік щопонеділка о 9:00 → **scheduled scaling**

Щоденні циклічні піки, сервери мають бути готові заздалегідь → **predictive scaling**

Різна реакція на помірне і сильне перевищення порогу → **step scaling**

Масштабувати воркерів за довжиною черги SQS → **target tracking за backlog per instance**

Балансувальник бачить сервер нездоровим, але ASG його не замінює → **увімкнути ELB health checks у ASG**

Нові сервери вбиваються як нездорові, бо застосунок стартує 5 хвилин → **збільшити health check grace period**

Застосунок стартує 20 хвилин, масштабування запізнюється → **warm pools (або golden AMI)**

Перед видаленням сервера треба вивантажити логи → **lifecycle hook (terminating)**

Оновити всі сервери групи на новий AMI без простою → **instance refresh**

Старий застосунок на одному сервері має автоматично відновлюватися навіть при втраті AZ → **ASG з min = max = 1 у кількох AZ**

Сесії користувачів губляться при масштабуванні → **винести сесії в ElastiCache або DynamoDB**

Пастки

Без ELB health checks ASG не помітить «завислий» застосунок.

Scheduled scaling — для **відомих** подій; для повторюваних патернів без ручного розкладу — predictive.

Launch configurations застаріли — у нових рішеннях launch templates.

Масштабування не допоможе, якщо вузьке місце — одна база даних: масштабуй і шар даних (репліки, кеш).

Scale-in видаляє сервер разом з усім, що на ньому було: локальні файли — втрачено.

Перевір себе

ПИТАННЯ 1

Вебзастосунок за ALB працює в Auto Scaling group. Один сервер перестав відповідати на запити (застосунок завис), але ОС працює, і ASG його не замінює. Що виправити?

- A** Замінити target tracking на step scaling
- B** Збільшити max у групі
- C** Увімкнути ELB health checks для ASG
- D** Збільшити cooldown

Правильна відповідь: C. За замовчуванням ASG дивиться лише на EC2 status checks, які бачать ОС і обладнання. ELB health check перевіряє застосунок: ASG замінить сервер, якого балансувальник вважає нездоровим.

ПИТАННЯ 2

Інтернет-магазин щоп'ятниці о 18:00 запускає розпродаж і заздалегідь знає, що потрібно щонайменше 20 серверів. Як це забезпечити найпростіше?

- A** Predictive scaling
- B** Вручну запускати сервери щоп'ятниці
- C** Scheduled action, що о 17:45 у п'ятницю встановлює min = 20
- D** Simple scaling з великим cooldown

Правильна відповідь: C. Відомий заздалегідь час і обсяг — випадок для scheduled scaling. Predictive навчається на історії й підходить для повторюваних патернів без точних знань.

ПИТАННЯ 3

Воркери на EC2 обробляють повідомлення з черги SQS. Навантаження на CPU мало змінюється, а черга інколи росте до сотень тисяч повідомлень. Як масштабувати воркерів?

- A** Target tracking за середнім CPU 50%
- B** Збільшити тип інстансу
- C** Target tracking за метрикою backlog per instance (кількість повідомлень у черзі ÷ кількість інстансів)
- D** Scheduled scaling щогодини

Правильна відповідь: C. Для воркерів черги правильна метрика — скільки повідомлень припадає на один сервер. CPU тут не відображає реальної потреби.

ПИТАННЯ 4

Застосунок стартує на новому сервері 15 хвилин, і під час раптових піків нові сервери з'являються запізно. Що допоможе найбільше?

- A** Зменшити health check grace period
- B** Warm pool з попередньо ініціалізованими серверами
- C** Перейти на simple scaling
- D** Додати lifecycle hook при видаленні

Правильна відповідь: B. Warm pool тримає готові (зупинені, приспані або запущені) сервери, тож масштабування бере їх замість створення з нуля. Золотий AMI теж скорочує час старту.

ПИТАННЯ 5

Старий застосунок не підтримує кілька серверів одночасно, але має автоматично відновлюватися навіть при втраті AZ. Що зробити з мінімальними змінами?

- A** Використати spread placement group
- B** Увімкнути EC2 Hibernate
- C** Запустити сервер в Auto Scaling group з $\text{min} = \text{max} = \text{desired} = 1$ і підмережами у двох AZ
- D** Запустити два сервери в різних AZ за ALB

Правильна відповідь: C. ASG з одним сервером автоматично замінить його при збої, зокрема в іншій AZ. Два сервери одночасно застосунок не підтримує.

ПИТАННЯ 6

Компанія хоче, щоб при кожному оновленні AMI всі сервери групи поступово замінювалися новими без простою і з можливістю відкату. Що використати?

- A** Lifecycle hooks при запуску
- B** Scheduled scaling
- C** Instance refresh в Auto Scaling group
- D** Лаштувати сервери вручну через SSH

Правильна відповідь: C. Instance refresh хвилями замінює сервери на нову версію launch template, зберігаючи мінімальний відсоток здорових, і підтримує відкат. Це і є незмінна інфраструктура.

ПИТАННЯ 7

Які дві дії допоможуть зробити сервери в ASG взаємозамінними (stateless)? (Оберіть 2)

Оберіть 2 відповіді

- A** Зберігати завантажені файли в Amazon S3
- B** Зберігати сесії користувачів в Amazon ElastiCache або DynamoDB
- C** Увімкнути sticky sessions назавжди
- D** Зберігати завантажені користувачами файли на instance store
- E** Зберігати конфігурацію в локальному файлі кожного сервера з ручним оновленням

Правильна відповідь: A, B. Стан виносять у спільні керовані сховища: сесії — у кеш чи DynamoDB, файли — в S3 (або EFS). Instance store зникає разом із сервером, а sticky sessions лише маскують проблему.



Контейнери — Docker, Amazon ECS, Amazon EKS, AWS Fargate і ECR



Відмовостійкість · 26%



Продуктивність · 24%



Вартість · 20%



Історія Хмаринки. У Тараса з'явилися двоє колег, і сайт почав ламатися по-новому: «у мене на ноутбучі працює, а на сервері — ні». Різні версії бібліотек, різні налаштування, золотий AMI перебудовується годину. А ще сайт поступово ділиться на частини: каталог, кошик, оплата, пошук — кожну хочеться оновлювати окремо, не зупиняючи решту. Тарас пропонує **контейнери**: упакувати кожну частину разом з усіма залежностями в образ, що працює однаково всюди, і доручити AWS запускати їх — бажано взагалі без серверів.



Образ. Контейнер — стандартний морський контейнер. Що б у ньому не везли — іграшки чи холодильники, — кран, судно і вантажівка працюють з ним однаково. **Образ** — креслення і вміст контейнера. **ECR** — склад контейнерів у порту. **ECS** і **EKS** — диспетчери порту, що вирішують, на яке судно і скільки контейнерів вантажити (оркестрація). **Fargate** — ти навіть не бачиш суден: здаєш контейнер, а перевезення — повністю турбота порту.

Контейнер за 3 хвилини

Нагадування з розділу 2: контейнер пакує **застосунок і його залежності**, але використовує спільне ядро ОС хоста — легкий, стартує за секунди, працює **однаково** всюди.

- **Dockerfile** — рецепт: «візьми базовий образ з Node.js, скопіюй код, встанови бібліотеки, запускай командою...».
- **Образ (image)** — результат збирання: незмінний, складається з шарів, має версію-тег (`shop-api:1.4.2`).
- **Реєстр (registry)** — сховище образів.
- **Контейнер** — запущений екземпляр образу.

```
FROM public.ecr.aws/docker/library/node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 8080
CMD ["node", "server.js"]
```

Коли контейнерів десятки й сотні, потрібен **оркестратор**: запускати потрібну кількість копій, розкладати їх по серверах, перезапустити впаді, оновлювати версії без простою, підключати до балансувальника. В AWS таких два: **ECS** і **EKS**.

Amazon ECR: реєстр образів

Amazon Elastic Container Registry (ECR) — приватний керований реєстр Docker/OCI-образів:

- доступ через **IAM** (і resource-based політики репозиторію);
- **сканування вразливостей** образів (розширене — через **Amazon Inspector**, постійне);
- **lifecycle policies** — автоматично видаляти старі образи («тримати останні 30»);
- **реплікація** між регіонами й акаунтами (для DR і мультирегіональних розгортань);
- **immutable tags** — заборонити перезапис тегу (`1.4.2` завжди означає той самий образ);

- **pull through cache** — кешувати образи з публічних реєстрів.

Публічні образи є в **ECR Public Gallery**.

Amazon ECS

Amazon Elastic Container Service (ECS) — «рідний» оркестратор AWS: простіший за Kubernetes і глибоко інтегрований з сервісами AWS. Поняття:

- **Cluster** — логічна група, де працюють задачі.
- **Task definition** — опис застосунку (як docker-compose): образи, CPU і пам'ять, порти, змінні середовища, **секрети** (з Secrets Manager чи Parameter Store), журнали (у CloudWatch Logs), **IAM-полі**.
- **Task** — запущений екземпляр task definition (один чи кілька контейнерів разом).
- **Service** — тримає **потрібну кількість** задач (desired count), перезапускає впалі, реєструє їх у **target group ALB/NLB**, робить **поступові оновлення** (rolling) або blue/green, масштабується (**Service Auto Scaling** за CPU, пам'яттю чи запитами на ціль).

Дві IAM-ролі, які плутають на іспиті:

- **Task role** — права **самого застосунку** в контейнері: читати S3, писати в DynamoDB.
- **Task execution role** — права **агента ECS**: завантажити образ з ECR, прочитати секрети для змінних середовища, писати логи в CloudWatch.

Мережа: у режимі **awsvpc** кожна задача отримує **власний мережевий інтерфейс** з приватною IP і **власну Security Group** — як окремий маленький сервер.



Де фізично працюють контейнери ECS: EC2 чи Fargate

	ECS на EC2	ECS на Fargate
Сервери	Твої інстанси EC2 з агентом ECS (зазвичай в Auto Scaling group)	Серверів не видно — AWS виділяє ресурси під кожну задачу
Що ти керуєш	Тип серверів, їх кількість, патчі ОС, щільність розміщення	Лише CPU і пам'ять задачі
Оплата	За сервери EC2 (можна Savings Plans, Spot)	За vCPU і пам'ять задачі посекундно (Savings Plans, Fargate Spot до –70%)
Коли обирати	Велике стабільне навантаження, щільне «пакування» заради економії, GPU, особливі вимоги до хоста	Мінімум операційної роботи , змінне навантаження, швидкий старт, ізоляція задач

Capacity providers дозволяють кластеру поєднувати джерела: наприклад, частина задач на Fargate, частина — на Fargate Spot чи EC2.

ECS Anywhere — запуск задач ECS на **власних серверах** у дата-центрі, з керуванням з AWS.

Amazon EKS

Amazon Elastic Kubernetes Service (EKS) — керований **Kubernetes**: AWS запускає і підтримує **control plane** (API-сервер, etcd) у кількох AZ, оновлює і латає його. Ти працюєш стандартними інструментами Kubernetes (`kubectl` , Helm).

Де працюють поди (data plane):

- **Managed node groups** — EC2-вузли, якими AWS допомагає керувати (оновлення, заміна);
- **Self-managed nodes** — повністю свої EC2;
- **Fargate** — поди без вузлів;
- **EKS Auto Mode** — AWS сам обирає, запускає, масштабує й оновлює вузли; популярний автоскейлер вузлів **Karpenter**.

Права AWS для окремих подів — через **EKS Pod Identity** або IAM-ролі для service accounts. Для власних дата-центрів — **EKS Anywhere** і **EKS Distro**.

Коли EKS, а не ECS? Компанія **вже** використовує Kubernetes, потрібна **переносимість** між хмарами й on-prem, потрібна екосистема Kubernetes (оператори, Helm-чарти, open-source інструменти). Якщо цього немає — ECS простіший і дешевший в обслуговуванні.

AWS Fargate

Fargate — serverless-двигун для контейнерів, працює і з **ECS**, і з **EKS**:

- ніяких серверів: не треба обирати тип, латати ОС, масштабувати кластер;
- кожна задача ізольована в окремому мікро-VM;
- платиш за **запитані vCPU і пам'ять** посекундно, поки задача працює;
- **Fargate Spot** — для переривних задач зі знижкою.

Що обрати для обчислень

Вимога	Вибір
Повний контроль над ОС, особливе ПЗ, ліцензії, GPU	EC2
Контейнери з мінімумом операційних зусиль	ECS на Fargate
Уже є Kubernetes, потрібна переносимість	EKS (з Fargate або Auto Mode для мінімуму зусиль)
Велике стабільне контейнерне навантаження, максимальна економія	ECS/EKS на EC2 з Savings Plans і Spot
Короткі події задачі до 15 хвилин	Lambda (розділ 25)
Довгі пакетні задачі з чергами	AWS Batch (розділ 26)
Перенести застосунок у контейнери без переписування	Образ у ECR + ECS/EKS на Fargate

Рішення для Хмаринки

- Сайт розділено на сервіси `web` , `api` , `image-worker` ; образи — в **ECR** з постійним скануванням і `lifecycle policy`.
- ECS на Fargate** за ALB — Тарасу більше не треба латати сервери; ASG з розділу 23 замінює **Service Auto Scaling**.
- Секрети — з **Secrets Manager** через `task definition`; права — через **task role**.
- `image-worker` — на **Fargate Spot**, масштабується за довжиною черги.
- Розгортання нових версій — поступове (`rolling`), з автоматичним відкатом при провалі `health checks`.
- Kubernetes не потрібен: немає ні досвіду, ні вимог до переносимості.



Спробуй у справжньому AWS: контейнер на Fargate за 10

Вартість: задача 0,25 vCPU / 0,5 GB на 15 хвилин — менше цента.

Час: 20 хвилин.

1. **ECS** → **Clusters** → **Create cluster**: назва `khmarynka-lab`, інфраструктура — **AWS Fargate (serverless)**. Create.
2. **Task definitions** → **Create new**: назва `hello-web`, launch type **Fargate**, CPU `.25` vCPU, Memory `.5` GB. Контейнер: ім'я `web`, image URI `public.ecr.aws/nginx/nginx:latest`, port mapping **80**. Create.
3. Кластер → **Tasks** → **Run new task**: launch type Fargate, task definition `hello-web`. Networking: твоя VPC, **публічна** підмережа, **Public IP — Turned on**, Security Group з HTTP 80 з My IP. Create.
4. Коли задача в статусі **Running**, відкрий її → Configuration → **Public IP** — відкрий у браузері: вітальна сторінка nginx. Жодного сервера EC2 ти не створював.
5. Вкладка **Logs** задачі — журнали контейнера (якщо ввімкнено CloudWatch Logs у task definition).
6. Подумай: що додати, щоб таких задач завжди було 3 за балансувальником? (Відповідь: **ECS service** з desired count 3 і ALB.)

Прибери за собою: зупини задачу (**Stop**), видали кластер, деактивуй і видали task definition, видали Security Group і журнали в CloudWatch Logs.

Запам'ятай

Контейнер = застосунок + залежності в образі; однаково всюди, старт за секунди.

ECR — приватний реєстр: IAM, сканування (Inspector), lifecycle, реплікація, immutable tags.

ECS: cluster → **service** (кількість, ALB, оновлення, автомасштабування) → **task** → контейнери; **task definition** — опис.

Task role — права застосунку; **task execution role** — права агента (ECR, секрети, логи).

ECS на EC2 — контроль і економія на великому стабільному навантаженні; **ECS на Fargate** — без серверів, мінімум роботи.

EKS — керований Kubernetes: коли вже є K8s чи потрібна переносимість; вузли — **managed node groups**, **Fargate**, **Auto Mode**.

Fargate — serverless-двигун для ECS і EKS; **Fargate Spot** — дешево для переривних задач.

On-prem: **ECS Anywhere**, **EKS Anywhere**.

Як питають на іспиті

Запускати контейнери без керування серверами → **AWS Fargate (ECS або EKS)**

Компанія вже використовує Kubernetes і хоче керований сервіс → **Amazon EKS**

Контейнерам потрібен доступ до S3 з найменшими правами → **ECS task role**

ECS не може завантажити образ з ECR або прочитати секрет для змінної середовища → **перевір task execution role**

Перенести застосунок у контейнери без переписування коду → **образ у ECR + ECS/EKS на Fargate**

Сканувати образи контейнерів на вразливості → **ECR scanning (Amazon Inspector)**

Автоматично видаляти старі образи в реєстрі → **ECR lifecycle policy**

Кілька контейнерів на одному сервері за ALB з різними портами → **ECS з динамічним мапінгом портів**

Переривні контейнерні задачі найдешевше → **Fargate Spot (або ECS на EC2 Spot)**

Запускати контейнери ECS на власних серверах у дата-центрі → **ECS Anywhere**

⚠ Пастки

Task role ≠ task execution role: права застосунок проти прав агента ECS.

Fargate — не окремий оркестратор, а двигун **для** ECS та EKS.

EKS не «простіший ECS»: без потреби в Kubernetes ECS зазвичай дає менше операційної роботи.

Контейнер теж має бути **stateless**: дані — у S3, базах, EFS, а не у файлової системі контейнера.

✓ Перевір себе

ПИТАННЯ 1

Команда пакує застосунок у Docker-образи і хоче запускати їх з мінімальними операційними зусиллями: без керування серверами, патчів ОС і масштабування кластера. Досвіду Kubernetes немає. Що обрати?

- A** Amazon ECS на EC2 з власною Auto Scaling group
- B** Amazon EKS з self-managed вузлами
- C** Docker на окремому інстансі EC2
- D** Amazon ECS на AWS Fargate

Правильна відповідь: D. ECS на Fargate не потребує керування серверами: AWS виділяє ресурси під кожну задачу. EKS має сенс, коли вже є Kubernetes, а варіанти з EC2 додають керування серверами.

ПИТАННЯ 2

Застосунок у контейнері ECS має записувати дані в таблицю DynamoDB. Як правильно надати права?

- A** Відкрити таблицю через Security Group
- B** Покласти access keys у змінні середовища контейнера
- C** Надати права IAM task role в task definition
- D** Надати права task execution role

Правильна відповідь: C. Task role — права застосунку всередині контейнера. Task execution role використовує агент ECS для завантаження образів, секретів і запису журналів.

ПИТАННЯ 3

Компанія має 40 мікросервісів у Kubernetes у власному дата-центрі і хоче перенести їх в AWS, зберігши наявні маніфести й Helm-чарти. Що обрати?

- A** AWS Lambda
- B** Amazon ECS на Fargate
- C** AWS Elastic Beanstalk
- D** Amazon EKS

Правильна відповідь: D. EKS — керований Kubernetes, сумісний з наявними інструментами. Перехід на ECS вимагав би переписати опис розгортань.

ПИТАННЯ 4

Щоночі тисячі короткочасних контейнерних задач обробляють зображення; задачі можна переривати й перезапустити. Як знизити вартість при запуску на ECS без серверів?

- A** Використовувати Fargate Spot
- B** Перейти на EKS
- C** Збільшити CPU задач
- D** Використовувати Dedicated Hosts

Правильна відповідь: A. Fargate Spot дає значну знижку для переривних задач. Dedicated Hosts — навпаки, дорожчі, а зміна оркестратора не впливає на ціну обчислень.

ПИТАННЯ 5

Служба безпеки вимагає постійно перевіряти образи контейнерів у реєстрі на відомі вразливості. Що використати?

- A** Amazon Macie
- B** Amazon ECR з розширеним скануванням (Amazon Inspector)
- C** Amazon GuardDuty для S3
- D** AWS WAF

Правильна відповідь: B. ECR інтегрований з Inspector для постійного сканування образів на CVE. Macie шукає чутливі дані в S3, WAF фільтрує HTTP, GuardDuty виявляє загрози в акаунті.

ПИТАННЯ 6

Задачі ECS не запускаються з помилкою «CannotPullContainerError: access denied» при завантаженні образу з ECR. Що перевірити насамперед?

- A** Права task execution role на читання з ECR
- B** Налаштування cross-zone load balancing
- C** Security Group бази даних
- D** Права task role на DynamoDB

Правильна відповідь: A. Образ завантажує агент ECS від імені task execution role (для Fargate). Якщо в неї немає прав ECR (або немає мережевого шляху до ECR), образ не завантажиться.

ПИТАННЯ 7

Які твердження про AWS Fargate правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** Працює як двигун і для Amazon ECS, і для Amazon EKS
- B** Вимагає керувати вузлами кластера Kubernetes
- C** Підходить лише для задач тривалістю до 15 хвилин
- D** Дозволяє встановлювати патчі на хост-ОС задачі
- E** Тарифікується за запитані vCPU і пам'ять задачі

Правильна відповідь: A, E. Fargate — serverless-двигун для ECS і EKS, оплата за ресурси задачі. Хостами керує AWS, а обмеження 15 хвилин стосується Lambda, а не Fargate.



AWS Lambda — код без серверів



Відмовостійкість · 26%



Продуктивність · 24%



Вартість · 20%



Історія Хмаринки. Продавці-партнери щодня завантажують тисячі фото товарів, і для кожного треба зробити три мініатюри. Тарас тримає для цього окремий сервер, що 90% часу нічого не робить, а в пікові дні не встигає. Ще є задачі «раз на годину перевірити залишки» і «після кожного замовлення надіслати лист». «Мені не потрібні сервери, — каже він, — мені потрібно, щоб шматочок коду запускався **у відповідь на подію**, а я платив лише за секунди його роботи». Це і є **AWS Lambda**.



Образ. Lambda — кухар на виклик. Не треба тримати кухаря в штаті цілодобово: подзвонив (подія) — він прийшов, приготував страву (виконав код) за кілька секунд чи хвилин (але не довше 15), пішов. Платиш лише за час готування. Прийшло сто замовлень одночасно — прийде сто кухарів. Перший кухар іноді запізнюється, бо ще переодягається (**cold start**), — але можна тримати кількох уже вдягнених наготові (**provisioned concurrency**).

Що таке Lambda

AWS Lambda — сервіс, що виконує твій **код (функцію)** у відповідь на **подію**, без жодних серверів з твого боку:

- не треба обирати сервер, латати ОС, налаштовувати масштабування — AWS запускає стільки копій функції, скільки прийшло подій;

- **оплата:** за кількість викликів і за **тривалість × пам'ять** (GB-секунди); коли функція не працює — не коштує нічого; великий постійний безкоштовний ліміт (мільйон викликів на місяць);
- мови: Python, Node.js, Java, .NET, Ruby, а також будь-яка інша через власне середовище виконання чи **container image**;
- процесори **x86 або Arm (Graviton)** — Graviton зазвичай дешевший.

Ліміти, які треба знати напам'ять

Параметр	Значення
Максимальна тривалість виклику	15 хвилин
Пам'ять	128 MB – 10 240 MB; CPU виділяється пропорційно пам'яті (окремо CPU не задається)
Тимчасовий диск /tmp	512 MB – 10 GB
Розмір запиту/відповіді	6 MB для синхронного виклику (поточкова відповідь — більше); 1 MB для асинхронного
Пакет коду	50 MB zip (250 MB розпакований, разом із шарами); container image — до 10 GB
Шари (layers)	до 5 на функцію — спільні бібліотеки
Concurrency (одночасні виконання)	1 000 на регіон за замовчуванням (можна підняти)

Головний наслідок: **задача довша за 15 хвилин — не для Lambda**. Її розбивають на кроки (Step Functions), або запускають у контейнері (ECS/Fargate), або в AWS Batch.

Як Lambda викликають



Синхронно

Хтось чекає відповіді
API Gateway, ALB, function URL, виклик з коду/CLI, Cognito
Помилку отримує той, хто викликав



Асинхронно

Подію кладуть у внутрішню чергу Lambda і не чекають
S3 events, SNS, EventBridge
При помилці — 2 повтори, потім DLQ або destination



Опитування (event source mapping)

Lambda сама читає пакети записів з джерела
SQS, Kinesis, DynamoDB Streams, MSK, Amazon MQ
Пакети, повтори за правилами джерела

Від способу виклику залежить, що буде з помилками: хто повторить і куди подінуться події, що не вдалося обробити.

Помилки і невдалі події

- **Асинхронні виклики:** Lambda сама повторює двічі. Невдалі події не повинні губитися — налаштуй **destination** для помилок (SQS, SNS, EventBridge, інша Lambda) або **DLQ** (SQS/SNS). Destinations сучасніші: можуть надсилати й **успішні** результати, з повною інформацією про виклик.
- **SQS як джерело:** невдалі повідомлення повертаються в чергу; після N спроб — у **DLQ черги**. Корисно вмикати **partial batch response** — щоб одне погане повідомлення не змушувало повторювати весь пакет. І головне правило: **visibility timeout черги $\geq 6 \times$ timeout функції** — інакше повідомлення стане видимим і його оброблять двічі (розділ 31).
- **Kinesis / DynamoDB Streams:** записи обробляються по порядку в межах shard; невдалий пакет блокує shard, доки не оброблено → налаштування **bisect** (розділити пакет), максимальний вік запису, destination для невдалих.

Concurrency: скільки копій одночасно

Кожен одночасний виклик — окреме середовище виконання. Ліміт акаунта за замовчуванням — **1 000** одночасних у регіоні на всі функції разом.

- **Reserved concurrency** — **гарантує** функції частку ліміту і **одночас обмежує** її цим числом. Приклад: функція пише в стару базу, яка витримує 50 з'єднань → reserved concurrency 50. А ще — захист: одна «навіжена» функція не з'їсть весь ліміт акаунта. Reserved concurrency = 0 — фактично «вимкнути» функцію.
- **Provisioned concurrency** — заздалегідь **ініціалізовані** («прогріті») середовища: **немає cold start**. Платно, навіть без викликів. Можна масштабувати за розкладом чи метриками.

Cold start

Перший виклик нового середовища: AWS створює мікро-VM, завантажує код, запускає ініціалізацію (підключення до бази, завантаження бібліотек) — від сотень мілісекунд до кількох секунд. Наступні виклики в тому самому середовищі — «теплі» і швидкі. Як зменшити:

- **provisioned concurrency** — для критичних до затримки API;
- **SnapStart** (Java, Python, .NET) — знімок уже ініціалізованого середовища, відновлення за частки секунди;
- менший пакет, лінива ініціалізація, ініціалізація поза обробником (щоб повторно використовувати з'єднання між викликами).

Lambda і мережа

За замовчуванням функція працює в мережі AWS і має доступ до інтернету та публічних сервісів AWS, але **не бачить** твоїх приватних ресурсів. Щоб звертатися до RDS, ElastiCache чи серверів у приватних підмережах — **підключи функцію до VPC** (підмережі + Security Group).

Нюанси:

- Функція у VPC **втрачає доступ в інтернет**, якщо немає **NAT Gateway** (підмережі функції мають бути приватними з маршрутом на NAT) — публічна IP функції не дається. До сервісів AWS — через **VPC endpoints**.
- Сотні одночасних копій функції відкривають сотні з'єднань до реляційної бази і можуть її «покласти». Рішення — **RDS Proxy**: пул з'єднань між функціями і базою (розділ 34).

Версії, псевдоніми і розгортання

- **Version** — незмінний знімок коду й налаштувань (\$LATEST — поточна чернетка).
- **Alias** — іменований покажчик на версію: `prod → v12`. Клієнти викликають `alias`, а ти перемикаєш його на нову версію.
- **Weighted alias** — `alias` ділить трафік між двома версіями (90% `v12`, 10% `v13`) — канарковий реліз з миттєвим відкатом. **AWS CodeDeploy** автоматизує поступове перемикання з відкатом за алармами.

Безпека функції

- **Execution role** — права функції (писати в DynamoDB, читати S3, писати логи) — розділ 9.
- **Resource-based policy** — **хто може викликати** функцію (S3 bucket, SNS topic, інший акаунт). Консоль додає її сама, коли ти налаштовуєш тригер.
- Секрети — у Secrets Manager чи Parameter Store, не у відкритих змінних середовища (змінні шифруються KMS, але доступ до них має кожен, хто бачить налаштування функції).

Lambda в архітектурі Хмаринки



Продавець

Завантажує фото в S3 `uploads/`



S3 event

ObjectCreated → асинхронний виклик



Lambda «thumbnails»

1 769 MB пам'яті (≈ 1 vCPU), до 60 с; destination для помилок → SQS



Результат

3 мініатюри → S3 `thumbs/`

Метадані → DynamoDB

Тисяча фото одночасно — тисяча копій функції паралельно. Жодного фото — жодних витрат. Важливо: результат пишемо в **інший** префікс чи bucket, інакше нова мініатюра знову запустить функцію (нескінченна рекурсія).

Інші функції Хмаринки: `order-email` читає чергу SQS і надсилає листи; `stock-check` запускається щогодини за розкладом **EventBridge Scheduler**; API мобільного застосунку — **API Gateway + Lambda** (наступний розділ).



Скільки коштує Lambda (порядок чисел)

Функція `thumbnails`: 300 000 фото на місяць, кожне — 2 секунди з пам'яттю 1,75 GB. GB-секунди: $300\,000 \times 2 \times 1,75 = \mathbf{1\,050\,000\,GB\text{-}с.}$

Постійний безкоштовний ліміт — 400 000 GB-с і мільйон викликів на місяць, тож платимо лише за ~650 000 GB-с — за типовими цінами це **близько десятка доларів на місяць**, замість сервера, що працює цілодобово. Але якщо функція завантажена 24/7 з тисячами викликів щосекунди, контейнери чи EC2 з Savings Plans можуть вийти дешевшими — рахуй.

Спробуй у справжньому AWS: вебфункція і реакція на завантаження в S3

Вартість: у межах постійного безкоштовного ліміту Lambda. Бонус: створення вебзастосунку на Lambda — одне з завдань, за які нараховують **\$20 кредитів**. **Час:** 25 хвилин.

1. **Lambda** → **Create function** → Author from scratch → назва `khmarynka-hello`, runtime **Python 3.13** (або новіший), архітектура **arm64**. Create.
2. Заміни код на:

```
import json

def lambda_handler(event, context):
    records = event.get("Records")
    if records: # подія від S3
        for r in records:
            print("Новий файл:", r["s3"]["object"]["key"], r["s3"]
            return {"ok": True}
    return {"statusCode": 200, "headers": {"Content-Type": "text/html"},
            "body": "<h1>Привіт з Хмаринки! Це Lambda.</h1>"}
```

Deploy.

1. **Configuration** → **Function URL** → **Create**: Auth type **NONE** (лише для навчання!). Відкрий URL — побачиш сторінку.

2. Створи bucket `khmarynka-lambda-<цифри>`. У функції: **Add trigger** → **S3** → твій bucket, подія **All object create events**, префікс `uploads/`. Підтверди попередження про рекурсію.
3. Завантаж у bucket будь-який файл у «папку» `uploads/`. Відкрий **Monitor** → **View CloudWatch logs** — побачиш рядок «Новий файл: ...».
4. **Configuration** → **Concurrency**: подивись, як виглядає `reserved concurrency` (не змінюй або встанови 5 і поверни).

Прибери за собою: видали function URL і тригер, функцію, bucket (спершу очисти) і групу журналів `/aws/lambda/khmarynka-hello` у CloudWatch Logs.

Запам'ятай

Lambda — код у відповідь на подію, без серверів; оплата за виклики і GB-секунди.

15 хвилин максимум; пам'ять 128 MB–10 GB (**CPU ∝ пам'яті**); `/tmp` до 10 GB; container image до 10 GB.

Виклики: **синхронні** (API Gateway, ALB), **асинхронні** (S3, SNS, EventBridge — 2 повтори, DLQ/destinations), **опитування** (SQS, Kinesis, DynamoDB Streams).

Concurrency **1 000** на регіон; **reserved** — гарантує й обмежує; **provisioned** — без cold start; **SnapStart** — швидкий старт Java/Python/.NET.

Lambda у VPC — доступ до приватних ресурсів; інтернет — лише через **NAT**; багато з'єднань до БД → **RDS Proxy**.

Версії й alias, weighted alias — поступові релізи.

Lambda + SQS: **visibility timeout** $\geq 6 \times$ **timeout функції**.

Як питають на іспиті

Обробити файл одразу після завантаження в S3 без серверів → **S3 event notification** → **Lambda**

Задача виконується 40 хвилин → **не Lambda: ECS/Fargate, AWS Batch або розбити через Step Functions**

API на Lambda має непередбачувані затримки через холодний старт → **provisioned concurrency (для Java — SnapStart)**

Lambda вичерпує з'єднання до RDS під навантаженням → **RDS Proxy**

Обмежити кількість одночасних виконань функції, щоб не перевантажити базу → **reserved concurrency**

Невдалі асинхронні події не повинні губитися → **Lambda destinations (on failure) або DLQ**

Функція у VPC не може звернутися до зовнішнього API → **NAT Gateway у публічній підмережі і маршрут з приватних підмереж функції**

Поступово випустити нову версію функції на 10% трафіку → **weighted alias (або CodeDeploy)**

Повідомлення SQS обробляються функцією повторно → **visibility timeout $\geq 6 \times$ timeout функції**

Функція має реагувати на кожну зміну в таблиці DynamoDB → **DynamoDB Streams** → **Lambda**

Задача за розкладом щоночі о 2:00, кілька хвилин → **EventBridge Scheduler** → **Lambda**

Пастки

«Серверів немає» не означає «ліміти немає»: 15 хвилин, розмір запиту, concurrency.

Окремо збільшити CPU не можна — лише пам'ять (CPU зростає разом з нею).

Функція у VPC без NAT не має інтернету — навіть якщо підмережа «публічна».

Тригер S3 і запис результату в той самий bucket і префікс → нескінченна рекурсія.

Для постійного високого навантаження 24/7 Lambda може бути дорожчою за контейнери чи EC2.

✓ Перевір себе

ПИТАННЯ 1

Щоразу, коли в bucket S3 завантажується PDF-рахунок, потрібно витягнути з нього дані й записати в базу. Обробка займає до 30 секунд. Яке рішення з найменшими операційними зусиллями?

- A** Сервер EC2 щохвилини перевіряє bucket скриптом cron
- B** Кластер EMR, що обробляє файли раз на день
- C** S3 event notification запускає функцію Lambda
- D** Контейнер ECS, що працює цілодобово і опитує bucket

Правильна відповідь: C. Подієва модель: S3 сам викликає Lambda при створенні об'єкта. Жодних серверів і опитування, оплата лише за обробку.

ПИТАННЯ 2

Функція Lambda під навантаженням відкриває сотні одночасних з'єднань до Amazon RDS for MySQL, і база відмовляє з помилкою «too many connections». Що зробити з мінімальними змінами коду?

- A** Перенести функцію з VPC
- B** Замінити RDS на DynamoDB
- C** Використати Amazon RDS Proxy і підключатися через його endpoint
- D** Збільшити пам'ять функції

Правильна відповідь: C. RDS Proxy тримає пул з'єднань до бази і ділить його між копіями функції. Змінюється лише рядок підключення. Заміна бази — великі зміни, а пам'ять не впливає на кількість з'єднань.

ПИТАННЯ 3

Синхронний API на Lambda (Java) іноді відповідає за 3–4 секунди через холодний старт. Бізнес вимагає стабільно швидких відповідей. Що обрати? (Оберіть 2)

Оберіть 2 відповіді

- ☐ **A** Збільшити timeout функції
- ☐ **B** Reserved concurrency = 0
- ☐ **C** Provisioned concurrency для функції
- ☐ **D** Перевести виклики в асинхронний режим
- ☐ **E** Lambda SnapStart

Правильна відповідь: C, E. Provisioned concurrency тримає прогріті середовища, SnapStart відновлює вже ініціалізоване середовище зі знімка. Reserved concurrency = 0 вимкне функцію, а timeout і асинхронність не зменшують затримку ініціалізації.

ПИТАННЯ 4

Щомісячне формування звіту триває приблизно 2 години. Команда хоче serverless-рішення. Що обрати?

- A** Lambda@Edge
- B** Одна функція Lambda з timeout 2 години
- C** Функція Lambda з provisioned concurrency
- D** Задача на AWS Fargate (ECS), запущена за розкладом EventBridge

Правильна відповідь: D. Lambda обмежена 15 хвилинами на виклик. Для довгих задач без серверів — контейнер на Fargate (або AWS Batch), запущений за розкладом.

ПИТАННЯ 5

Функція пише в стару базу, яка витримує не більше 20 одночасних з'єднань. Як гарантувати, що функція не перевищить цей ліміт?

- A** Встановити provisioned concurrency = 20
- B** Розмістити функцію у двох AZ
- C** Встановити reserved concurrency = 20
- D** Зменшити пам'ять функції до 128 MB

Правильна відповідь: C. Reserved concurrency обмежує максимальну кількість одночасних виконань функції (і гарантує їй цю частку ліміту акаунта). Provisioned concurrency лише прогріває середовища, але не обмежує масштаб зверху.

ПИТАННЯ 6

Функція Lambda у приватній підмережі VPC має звертатися до публічного API платіжного партнера. Виклики завершуються таймаутом. Що додати?

- A** NAT Gateway у публічній підмережі і маршрут 0.0.0.0/0 на нього з підмереж функції
- B** Gateway VPC endpoint для зовнішнього API
- C** Публічну IP-адресу для функції
- D** Security Group з дозволом усього вхідного трафіку

Правильна відповідь: A. Функції у VPC не отримують публічних IP; вихід в інтернет можливий лише через NAT Gateway. Gateway endpoints існують тільки для S3 і DynamoDB.

ПИТАННЯ 7

Асинхронні виклики функції (від SNS) інколи завершуються помилкою, і події губляться. Як зберегти невдалі події для аналізу і повторної обробки?

- A** Перейти на синхронні виклики
- B** Налаштувати on-failure destination (наприклад, чергу SQS) або DLQ для функції
- C** Увімкнути provisioned concurrency
- D** Збільшити пам'ять функції

Правильна відповідь: B. Для асинхронних викликів Lambda робить повтори, а потім може надіслати подію в destination при помилці (SQS, SNS, EventBridge, Lambda) або в DLQ. Так нічого не губиться.



API й платформи — Amazon API Gateway, Elastic Beanstalk, AWS Batch, Amplify

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Безпека · 30%



Історія Хмаринки. Хмаринка запускає мобільний застосунок, а кур'єрські служби й продавці-партнери хочуть працювати з магазином через **API**. Потрібні: вхідні двері для тисяч запитів, перевірка токенів, ліміти «не більше 100 запитів за секунду для безкоштовних партнерів», кеш, захист від зловживань — і все це без серверів. Паралельно маркетолог хоче «просто задеплоїти» маленький сайт акцій, не вникаючи в інфраструктуру, а бухгалтерія — щомісяця переганяти мільйони записів у звіти. Сьогодні — про **API Gateway** і кілька «платформ», що знімають з тебе рутину.



Образ. API Gateway — рецепція бізнес-центру: перевіряє перепустки (авторизація), рахує відвідувачів і не пускає зайвих (throttling), видає типові довідки з полицки без походу в кабінети (кеш), відправляє кожного у потрібний кабінет (інтеграції). **Elastic Beanstalk** — ресторан «під ключ»: приносиш рецепти (код), а зал, кухню і персонал (сервери, балансувальник, масштабування) облаштовують за тебе. **AWS Batch** — цех з чергою великих замовлень, що сам наймає робітників, коли замовлень багато.

Amazon API Gateway

Amazon API Gateway — керовані «вхідні двері» для API: приймає HTTP-запити, перевіряє їх і передає далі — у **Lambda**, у будь-який **HTTP-сервіс**, прямо в **сервіси AWS** (наприклад, покласти повідомлення в SQS без жодного коду) або в **приватні** ресурси VPC через **VPC Link** (NLB/ALB). Serverless, масштабується автоматично, оплата за запити.

Три типи API

Тип	Для чого	Можливості
REST API	Повнофункціональні API	Кешування відповідей, API keys i usage plans , WAF , валідація й трансформація запитів, приватні API, канаркові релізи стадій
HTTP API	Прості й дешеві API, проксі до Lambda чи HTTP	Дешевше і швидше , вбудована перевірка JWT (Cognito, будь-який OIDC); менше функцій
WebSocket API	Двосторонній зв'язок у реальному часі	Чати, сповіщення, онлайн-ігри, дашборди, що оновлюються самі

Типи endpoint

- **Edge-optimized** — запити заходять через мережу CloudFront (для клієнтів по всьому світу).
- **Regional** — для клієнтів у тому ж регіоні (або якщо ставиш свій CloudFront попереду).
- **Private** — API доступний **лише з VPC** через interface endpoint, з resource policy (внутрішні API компанії).

Безпека і контроль

- **Авторизація: IAM** (підписані запити — для сервісів і внутрішніх клієнтів), **Cognito User Pools** (токени користувачів застосунку), **Lambda authorizer** (власна логіка: перевірити токен стороннього провайдера, заголовки, ключі), для HTTP API — **JWT authorizer**.

- **Throttling:** за замовчуванням близько **10 000 запитів за секунду** на акаунт у регіоні (з піками до 5 000 «сплеску»). Понад ліміт — відповідь **429 Too Many Requests**. Можна задати ліміти для стадій і методів.
- **API keys + usage plans (REST)** — **окремі ліміти й квоти для кожного клієнта**: «безкоштовний тариф — 100 запитів/с і 1 млн на місяць, платний — 1 000 запитів/с». Важливо: API key — інструмент обліку й лімітів, **а не** надійна автентифікація.
- **WAF** на REST API, власні домени з сертифікатами **ACM, mTLS**.

Продуктивність і релізи

- **Кешування (REST):** відповіді зберігаються на заданий TTL — менше навантаження на бекенд і швидші відповіді.
- **Таймаут інтеграції** — за замовчуванням **29 секунд** (для regional і private REST API його можна збільшити); розмір запиту — до **10 MB**. Довгі операції роблять асинхронно: прийняти запит, покласти в чергу, повернути «прийнято».
- **Стадії** (`dev` , `prod`) зі змінними стадій і **canary release** — частина трафіку стадії йде на нову версію.



Застосунок і партнери

Токен Cognito або API key партнера



API Gateway (REST)

WAF · Cognito authorizer · usage plans · кеш каталогу 5 хв



Інтеграції

GET /products → Lambda → DynamoDB

POST /orders → **напрям у SQS (без коду)**

/legacy/* → VPC Link → приватний ALB

Замовлення не губляться в пік: API одразу кладе їх у чергу SQS і відповідає «прийнято», а обробка йде у своєму темпі.



GraphQL: AWS AppSync

Якщо клієнтам потрібна мова запитів **GraphQL** (клієнт сам обирає, які поля отримати, кілька джерел даних в одному запиті) і **підписи в реальному часі** — це **AWS AppSync**, а не API Gateway.

AWS Elastic Beanstalk

Elastic Beanstalk — PaaS: завантажуєш код (Java, .NET, Node.js, Python, PHP, Ruby, Go або Docker-образ), а Beanstalk сам створює й налаштовує **EC2, Auto Scaling, балансувальник, моніторинг**, оновлює платформу. Ти керуєш застосунком, а не інфраструктурою — але всі ресурси видно в акаунті, і їх можна тонко налаштувати.

- **Середовища:** *web server* (за балансувальником) і *worker* (обробляє чергу SQS — фонові задачі).
- **Політики розгортання:**

Політика	Як	Простий	Ризик / вартість
All at once	Усі сервери оновлюються разом	Так, короткий	Найшвидше, найризикованіше
Rolling	Партіями	Ні, але менша потужність під час оновлення	Відкат повільний
Rolling with additional batch	Спершу додає нову партію, потім оновлює	Ні, потужність збережена	Трохи дорожче
Immutable	Нові сервери в новій ASG, потім заміна	Ні	Найбезпечніший відкат, дорожче на час оновлення
Traffic splitting	Частина трафіку на нові сервери (канарка)	Ні	Перевірка на реальних користувачах
Blue/green	Окреме нове середовище + swap URL	Ні	Миттєвий відкат; повна копія середовища

На іспиті: «розробники хочуть просто розгорнути вебзастосунок, не займаючись інфраструктурою» → **Elastic Beanstalk**. Але пам'ятай: Beanstalk **не serverless** — він керує EC2 за тебе.



AWS App Runner

Сервіс «запусти контейнер чи код як вебзастосунок одним кліком» — **AWS App Runner** — з **30 квітня 2026 року закритий для нових клієнтів**. AWS радить для нових проєктів **Amazon ECS Express Mode** (спрощене розгортання контейнерних вебзастосунків на ECS з балансувальником і масштабуванням) або Elastic Beanstalk.

AWS Batch

AWS Batch — керована черга й планувальник **пакетних задач** будь-якої тривалості:

- описуєш **job definition** (контейнерний образ, CPU, пам'ять), кладеш задачі в **job queue**;
- Batch сам підбирає і запускає ресурси в **compute environment** — **EC2** (зокрема **Spot**) або **Fargate** — і прибирає їх після роботи;
- **array jobs** (тисячі схожих задач одним запитом), **залежності** між задачами, пріоритети черг, повтори.

Batch чи Lambda? Lambda — коротко (до 15 хв), на подію. Batch — довго, великі обсяги, черги й залежності, дешево на Spot. **Batch чи Step Functions?** Step Functions оркеструють кроки процесу; Batch виконує важкі обчислення (і Step Functions можуть запускати задачі Batch).

AWS Amplify

AWS Amplify — інструменти й хостинг для **фронтенду і мобільних застосунків**: підключаєш git-репозиторій — Amplify збирає і розгортає сайт (React, Vue, Next.js) з CDN і HTTPS, створює preview-середовища для гілок; бекенд — автентифікація (Cognito), дані, сховище, функції. Для команд фронтенду, яким потрібен **full-stack** без глибокого занурення в AWS. Поруч — **AWS Device Farm**: тестування застосунків на сотнях **реальних** телефонів і браузерів.

Що обрати

Задача	Сервіс
Serverless REST API з кешем, API keys, лімітами для клієнтів	API Gateway REST API + Lambda

Задача	Сервіс
Простий дешевий HTTP-проксі до Lambda з JWT	API Gateway HTTP API
Двосторонній зв'язок у реальному часі	API Gateway WebSocket API (або AppSync)
GraphQL, підписки в реальному часі	AWS AppSync
«Просто задеплоїти» вебзастосунок з коду без роботи з інфраструктурою	Elastic Beanstalk
Тисячі пакетних задач, довгих, з чергами, на Spot	AWS Batch
Хостинг фронтенду з CI/CD з git + бекенд для мобільного застосунку	AWS Amplify

Спробуй у справжньому AWS: HTTP API перед Lambda

Вартість: API Gateway і Lambda — у межах безкоштовних лімітів для кількох запитів. **Час:** 15 хвилин. Потрібна функція `khmarynka-hello` з розділу 25 (або створи нову з тим самим кодом).

1. **API Gateway** → **Create API** → **HTTP API** → **Build**. Integration: **Lambda** → `khmarynka-hello`. Назва: `khmarynka-api`.
2. Route: метод **GET**, шлях `/hello` → інтеграція з функцією. Stage: `$default`, auto-deploy. Create.
3. Скопіюй **Invoke URL** і відкрий у браузері `<Invoke URL>/hello` — побачиш сторінку від Lambda.
4. **Protect** → **Throttling** → задай для маршруту rate **2**, burst **2** запити/с. Швидко онови сторінку багато разів — отримаєш **429** (`Too Many Requests`).
5. Подивись у **Monitor**: кількість запитів, помилки 4xx.
6. Для порівняння відкрий **Create API** → **REST API** і подивись опції, яких немає в HTTP API: API keys, usage plans, кешування стадії (не створюй).

Прибери за собою: видали HTTP API; функцію — якщо більше не потрібна.

Запам'ятай

API Gateway: REST (кеш, API keys + **usage plans**, WAF, валідація, приватні API), **HTTP** (дешевше, JWT), **WebSocket** (двосторонній).

Endpoints: **edge-optimized, regional, private** (лише з VPC).

Авторизація: **IAM, Cognito, Lambda authorizer**, JWT (HTTP API).

Throttling ~**10 000 запитів/с** на акаунт у регіоні → **429**; таймаут інтеграції **29 с** за замовчуванням; payload до 10 MB.

Інтеграції: Lambda, HTTP, **сервіси AWS напряду** (SQS, DynamoDB...), **VPC Link** до приватних ALB/NLB.

AppSync — GraphQL і підписки.

Elastic Beanstalk — PaaS над EC2; політики: all at once, rolling, rolling with additional batch, **immutable**, traffic splitting, **blue/green (swap URL)**.

AWS Batch — пакетні задачі будь-якої тривалості на EC2/Spot/Fargate.
Amplify — фронтенд і мобільні з CI/CD.

 App Runner закритий для нових клієнтів (з 30.04.2026).

Як питають на іспиті

Serverless REST API → **API Gateway + Lambda (+ DynamoDB)**

Різні ліміти запитів і квоти для різних клієнтів API → **API Gateway usage plans + API keys**

Кешувати відповіді API, щоб зменшити навантаження на бекенд → **API Gateway caching (REST API)**

Власна перевірка токенів стороннього провайдера → **Lambda authorizer**

Користувачі застосунку входять через Cognito і викликають API → **Cognito User Pool authorizer**

Двосторонній чат у реальному часі → **API Gateway WebSocket API**

API має бути доступний лише зсередини VPC → **private REST API + interface endpoint + resource policy**

Приймати замовлення через API і не губити їх у пік → **API Gateway → SQS (напрямку) → обробка асинхронно**

Клієнти отримують 429 → **спрацював throttling: підняти ліміти або налаштувати usage plans**

Розробники хочуть лише завантажити код, а сервери, балансувальник і масштабування створюються самі → **Elastic Beanstalk**

Нова версія з мінімальним ризиком і миттєвим відкатом у Beanstalk → **blue/green (swap environment URL) або immutable**

Тисячі пакетних задач з чергами і Spot → **AWS Batch**

Хостинг SPA-фронтенду з автоматичним розгортанням з git → **AWS Amplify Hosting**

Пастки

API key у API Gateway — не автентифікація, а облік і ліміти.

Кешування, usage plans і WAF — у **REST** API, не в HTTP API.

Таймаут інтеграції API Gateway обмежений (29 с за замовчуванням) — довгі задачі робимо асинхронно.

Elastic Beanstalk — не serverless: під ним EC2.

Batch — не для відповідей користувачу в реальному часі.

✓ Перевір себе

ПИТАННЯ 1

Компанія продає доступ до свого API трьома тарифами: Free (10 запитів/с), Pro (100 запитів/с), Enterprise (1 000 запитів/с), з місячними квотами. Що використати?

- A** API Gateway REST API з API keys і usage plans
- B** AWS WAF rate-based rules для кожного клієнта
- C** Lambda reserved concurrency для кожного тарифу
- D** Три окремі Application Load Balancers

Правильна відповідь: A. Usage plans задають ліміти швидкості й квоти для груп API keys — саме тарифи для клієнтів. Reserved concurrency обмежує функцію, а не клієнтів, WAF — захист, а не тарифікація.

ПИТАННЯ 2

API каталогу товарів отримує тисячі однакових GET-запитів на хвилину, і бекенд перевантажений. Дані змінюються раз на годину. Як найпростіше зменшити навантаження?

- A** Додати Lambda authorizer
- B** Збільшити таймаут інтеграції
- C** Перейти з REST API на WebSocket API
- D** Увімкнути кешування на стадії REST API в API Gateway

Правильна відповідь: D. Кеш API Gateway віддає збережені відповіді протягом TTL без звернення до бекенду. Інші варіанти не зменшують кількість звернень до бекенду.

ПИТАННЯ 3

Мобільний застосунок використовує Amazon Cognito для входу користувачів. Як дозволити виклики API лише автентифікованим користувачам з мінімальними зусиллями?

- A** IAM-користувач для кожного покупця
- B** Lambda authorizer, що вручну перевіряє паролі
- C** Cognito User Pool authorizer в API Gateway
- D** API key для кожного користувача

Правильна відповідь: C. API Gateway вміє перевіряти токени Cognito User Pool без жодного коду. Lambda authorizer потрібен для нестандартних перевірок, API keys не автентифікують, а IAM-користувачі — не для тисяч клієнтів.

ПИТАННЯ 4

Під час розпродажу бекенд обробки замовлень не встигає, і клієнти отримують помилки. Замовлення не можна втрачати, а відповідь «прийнято» має приходити швидко. Яка архітектура найкраща?

- A** API Gateway інтегрується напряму з Amazon SQS; воркери (Lambda чи контейнери) обробляють чергу
- B** API Gateway викликає Lambda синхронно з provisioned concurrency
- C** Збільшити таймаут API Gateway до 10 хвилин
- D** Замінити API Gateway на Classic Load Balancer

Правильна відповідь: A. Черга розв'язує прийом і обробку: API миттєво кладе замовлення в SQS, а обробники беруть їх у своєму темпі. Жодне замовлення не губиться, піки згладжуються.

ПИТАННЯ 5

Розробники хочуть завантажити Java-застосунок і отримати готове середовище з балансувальником, Auto Scaling і моніторингом, не налаштовуючи інфраструктуру вручну. Що обрати?

- A** Amazon EC2 з вручну налаштованою ASG
- B** AWS Batch
- C** AWS CloudFormation з ручними шаблонами
- D** AWS Elastic Beanstalk

Правильна відповідь: D. Elastic Beanstalk — PaaS, що сам створює й обслуговує EC2, ASG, балансувальник і моніторинг під код. CloudFormation — інструмент IaC, але шаблони треба писати самому.

ПИТАННЯ 6

Команда оновлює застосунок у Elastic Beanstalk і хоче миттєвого відкату без простою, навіть ціною повної копії середовища на час релізу. Яку стратегію обрати?

- A** Blue/green з перемиканням URL середовищ (swap URL)
- B** All at once
- C** Rolling with additional batch
- D** Rolling

Правильна відповідь: A. Blue/green створює окреме нове середовище; після перевірки URL міняються місцями, а відкат — це повторна заміна URL. All at once має простий, rolling — повільний відкат.

ПИТАННЯ 7

Щовечора потрібно виконати 10 000 обчислювальних задач тривалістю від 20 хвилин до 3 годин, з залежностями між деякими з них, якомога дешевше. Що обрати?

- A** AWS Lambda
- B** API Gateway WebSocket API
- C** AWS Amplify
- D** AWS Batch з compute environment на Spot

Правильна відповідь: D. Batch керує чергами, залежностями й ресурсами для довгих пакетних задач і дозволяє використовувати Spot. Lambda обмежена 15 хвилинами.



Amazon S3 — основи безрозмірного складу об'єктів

🎯 Безпека · 30%

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Вартість · 20%



Історія Хмаринки. Папка «Фото_НОВІ_остаточні_2» на старому сервері займала 400 ГБ і щомісяця росла. Диск закінчувався, бекап на флешку робили раз на тиждень, а коли сервер «ліг», зникли й фото — сайт показував порожні рамки. Тарас переносить усі файли — фото товарів, рахунки, експорти для бухгалтерії, бекапи — у **Amazon S3**: сховище, де місце не закінчується, файли зберігаються в кількох дата-центрах одночасно, а платиш лише за те, що лежить.



Образ. S3 — безрозмірний автоматизований склад. **Bucket** — окремий склад зі своєю адресою і правилами входу. **Об'єкт** — коробка: всередині будь-що (фото, відео, архів), на ній — ярлик з описом (**метадані**) і унікальний номер (**ключ**). Коробку не розпаковують на місці й не міняють одну річ усередині — її **замінюють цілком**. Склад автоматично тримає копії кожної коробки в кількох будівлях, тож втратити її практично неможливо.

Bucket, об'єкт, ключ

Amazon S3 (Simple Storage Service) — **об'єктне** сховище (розділ 4):

- **Bucket** — контейнер для об'єктів. Створюється в **конкретному регіоні**, але **ім'я глобально унікальне** серед усіх акаунтів AWS світу (3–63 символи, малі літери, цифри, дефіси). Кількість об'єктів у bucket не обмежена.
- **Об'єкт** — дані + метадані. Розмір — від 0 байт до **50 TB** ( з грудня 2025; раніше максимум був 5 TB — на іспиті ще може трапитися стара цифра).
- **Ключ** — повне ім'я об'єкта: `products/2026/mug-blue.jpg`. «Папок» у S3 насправді немає: `products/2026/` — лише **префікс** ключа, який консоль показує як папку.

Доступ — через **HTTP API** (консоль, CLI, SDK): `PUT` — завантажити, `GET` — отримати, `DELETE` — видалити, `LIST` — перелічити. S3 не «монтує» як диск, і не можна змінити шматок об'єкта — лише записати новий цілком.

Надійність і доступність

- **Durability** — **99,999999999% («11 дев'яток»)**: для класів з кількома AZ S3 зберігає дані щонайменше в **трьох** зонах доступності. Якщо зберігати 10 мільйонів об'єктів, у середньому можна втратити один раз на 10 000 років.
- **Availability** залежить від класу: для S3 Standard — 99,99% на рік.
- **Консистентність**: S3 дає **strong read-after-write consistency** для всіх операцій: щойно записав чи видалив — одразу читаєш актуальний стан. (На старих курсах можна зустріти «eventual consistency» — це давно не так.)

Як завантажувати: від кілобайтів до терабайтів

- **Один PUT** — до 5 GB.

- **Multipart upload** — об'єкт ділиться на частини, які завантажуються **паралельно** й незалежно; збійна частина перезавантажується окремо. **Рекомендовано від 100 MB, обов'язково понад 5 GB**. Незавершені multipart-завантаження займають місце (і коштують) — прибирай їх правилом lifecycle.
- **S3 Transfer Acceleration** — завантаження з далеких країн через найближчу edge-локацію CloudFront і далі мережею AWS. Для користувачів, що вантажать великі файли з іншого континенту.
- **Byte-range fetches** — читати лише потрібну частину об'єкта, зокрема паралельно кількома запитами (швидке скачування великих файлів, читання заголовка файлу).

Продуктивність

S3 масштабується автоматично: щонайменше **3 500** запитів на запис (PUT/COPY/POST/DELETE) і **5 500** на читання (GET/HEAD) **за секунду на кожен префікс**. Більше префіксів — більше сумарної швидкості: 10 префіксів = 55 000 читань за секунду. Для найнижчої затримки (одноцифрові мілісекунди) в одній AZ є клас **S3 Express One Zone** з особливими *directory buckets* (розділ 28).

Класи зберігання — коротко

Кожен об'єкт має **клас зберігання** — компроміс між ціною зберігання, ціною і швидкістю доступу:

Клас	Коли	Ключова особливість
Standard	Часто читають	Типовий, без мінімального строку і плати за отримання
Intelligent-Tiering	Патерн доступу невідомий або змінюється	Сам переносить об'єкти між рівнями, без плати за отримання
Standard-IA / One Zone-IA	Читають рідко, але потрібні миттєво	Дешевше зберігання, плата за отримання, мінімум 30 днів; One Zone — одна AZ

Клас	Коли	Ключова особливість
Glacier Instant / Flexible / Deep Archive	Архіви	Від мілісекунд до годин на отримання, мінімум 90–180 днів
Express One Zone	Найшвидший доступ в одній AZ	Одноцифрові мілісекунди

Детально — з цифрами, правилами життєвого циклу й пастками — у наступному розділі.

Хто має доступ: безпека за замовчуванням

Новий bucket **приватний**:

- **Block Public Access** увімкнено за замовчуванням — чотири «запобіжники», що блокують будь-який публічний доступ, навіть якщо хтось помилково напише публічну політику. Можна ввімкнути на рівні **всього акаунта**.
- **ACL вимкнені** (Object Ownership = *Bucket owner enforced*): усі об'єкти належать власнику bucket, а доступ керується **політиками**.
- **Шифрування за замовчуванням** — усі нові об'єкти шифруються **SSE-S3** (розділ 11).

Як дати доступ:

- **IAM-політики** — користувачам і ролям **свого** акаунта (наприклад, роль серверів сайту читає `products/*`).
- **Bucket policy** — resource-based політика на bucket: доступ **іншим акаунтам**, сервісам (CloudFront через OAC), умови (лише HTTPS, лише з певного VPC endpoint, лише з організації).
- **Presigned URL** — тимчасове посилання для **завантаження чи скачування** приватного об'єкта: підписане обліковими даними того, хто його створив, з терміном дії. Користувач мобільного застосунку отримує від бекенду presigned URL і вантажить фото **напрямую в S3**, не перевантажуючи сервери.

Приклад bucket policy: дозволити читання всього bucket лише акаунтам своєї організації і лише по HTTPS:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "OrgRead", "Effect": "Allow", "Principal": "*",
      "Action": "s3:GetObject", "Resource": "arn:aws:s3:::khmarynka-repo",
      "Condition": { "StringEquals": { "aws:PrincipalOrgID": "o-a1b2c3d4" } } },
    { "Sid": "HttpsOnly", "Effect": "Deny", "Principal": "*", "Action":
      "s3:GetObject", "Resource": ["arn:aws:s3:::khmarynka-reports", "arn:aws:s3:::khmarynka-repo"],
      "Condition": { "Bool": { "aws:SecureTransport": "false" } } }
  ]
}
```

Події S3: сховище, що реагує

Event notifications — при створенні, видаленні, відновленні об'єктів S3 може повідомити:

- **Amazon SQS** — покласти подію в чергу (надійна обробка з повторами);
- **Amazon SNS** — розіслати кільком підписникам;
- **AWS Lambda** — одразу запустити функцію;
- **Amazon EventBridge** — надіслати всі події в шину з гнучкими фільтрами за ключем, розміром, метаданими і десятками можливих цілей.

Так Хмаринка генерує мініатюри одразу після завантаження фото (розділ 25). Щоб S3 міг писати в чергу чи тему, **політика черги/теми** має це дозволяти.

Статичний сайт у S3

S3 вміє віддавати **статичний сайт** (HTML, CSS, JavaScript, картинки) без жодних серверів — режим **static website hosting** з індексною сторінкою і

сторінкою помилки. Але вебсайт-endpoint S3 працює **лише по HTTP**. Для HTTPS, власного домену й швидкості — ставлять **CloudFront** перед приватним bucket з **OAC**, сертифікатом **ACM** і записом **Route 53 Alias** (розділ 18).

Корисні дрібниці

- **Requester Pays** — за завантаження даних з bucket платить той, хто їх качає (для великих публічних датасетів).
- **Теги об'єктів** — для правил життєвого циклу, доступу й обліку.
- **Метадані** — системні (тип вмісту, дата) і власні (`x-amz-meta-*`).
- **CORS** — дозволити JavaScript з іншого домену звертатися до bucket (наприклад, сайт `khmarynka.ua` вантажить файли в bucket напряму з браузера).

Спробуй у справжньому AWS: bucket, об'єкт, presigned URL

Вартість: частки цента за кілька файлів. **Час:** 15 хвилин.

1. **S3** → **Create bucket**: ім'я `khmarynka-<твое-ім'я>-<цифри>`, регіон `eu-central-1`. Залиш **Block all public access** увімкненим. Create.
2. Завантаж будь-яке фото в «папку» `products/` (кнопка **Create folder**, потім **Upload**).
3. Відкрий об'єкт і скопіюй **Object URL** — відкрий у браузері: **AccessDenied**. Bucket приватний — так і має бути.
4. **Object actions** → **Share with a presigned URL** → строк 5 хвилин → скопіюй і відкрий: фото відкривається. Через 5 хвилин посилання перестане працювати.
5. У **CloudShell** зроби те саме з командного рядка:

```
aws s3 ls s3://<твій-bucket>/products/  
aws s3 presign s3://<твій-bucket>/products/<фото>.jpg --expires-in  
aws s3api head-object --bucket <твій-bucket> --key products/<фото>.
```

Остання команда покаже метадані, клас зберігання і тип шифрування (ServerSideEncryption: AES256 — це SSE-S3 за замовчуванням).

1. Вкладка **Properties** → **Event notifications** — подивись, які цілі можна обрати (SQS, SNS, Lambda, EventBridge).

Прибери за собою: очисти (**Empty**) і видали bucket — або залиш для наступного розділу.

Запам'ятай

S3 — **об'єктне** сховище: bucket (періональний, ім'я глобально унікальне)
→ об'єкти (до **50 TB**) з ключами; «папки» — префікси.

11 дев'яток durability, дані щонайменше в 3 AZ (крім One Zone-класів);
strong consistency.

Один PUT — до **5 GB**; **multipart** — від 100 MB рекомендовано, понад 5 GB обов'язково; **Transfer Acceleration** — з далеких країн.

3 500 записів / 5 500 читань за секунду на префікс — більше префіксів, більше швидкості.

Безпека за замовчуванням: **Block Public Access**, ACL вимкнені, **SSE-S3**.
Доступ — IAM і **bucket policy**; тимчасово — **presigned URL**.

Event notifications → SQS, SNS, Lambda, **EventBridge**.

Static website hosting — лише HTTP; для HTTPS і домену — **CloudFront + OAC + ACM + Route 53**.

Як питають на іспиті

Зберігати мільйони фото чи відео дешево, надійно, з доступом через інтернет → **Amazon S3**

Завантаження файлів понад 100 MB через нестабільну мережу → **multipart upload**

Користувачі з інших континентів повільно вантажать великі файли → **S3 Transfer Acceleration**

Дати клієнту тимчасово завантажити або скачати приватний файл → **presigned URL**

Мобільний застосунок має вантажити фото напряму в S3, не через сервери → **presigned URL від бекенду (або Cognito Identity Pool)**

Запустити обробку, щойно файл з'явився в bucket → **S3 Event Notification** → **Lambda / SQS / EventBridge**

Дуже високий темп запитів до S3 → **розподілити об'єкти по кількох префіксах**

Статичний сайт з HTTPS і власним доменом → **S3 + CloudFront (OAC) + ACM + Route 53**

Доступ до bucket усім акаунтам своєї організації → **bucket policy з aws:PrincipalOrgID**

Заборонити будь-який публічний доступ до всіх bucket акаунта → **S3 Block Public Access на рівні акаунта**

Партнери качають великі датасети — хай самі платять за трафік → **Requester Pays**

Пастки

S3 — не файлова система і не диск: не монтується під базу, не змінює частину об'єкта.

Ім'я bucket глобально унікальне, але сам bucket живе в одному регіоні.

Static website endpoint S3 — лише HTTP.

Presigned URL працює з правами того, хто підписав, і не довше, ніж діють його облікові дані.

«Eventual consistency» у S3 — застаріла інформація: запоз strong consistency.

✓ Перевір себе

ПИТАННЯ 1

Мобільний застосунок має дозволити користувачам завантажувати відео до 2 GB у приватний bucket без передачі файлів через сервери компанії. Яке рішення найпростіше і безпечне?

- A** Надсилати відео через API Gateway в Lambda, а звідти в S3
- B** Бекенд генерує presigned URL для завантаження конкретного об'єкта з коротким терміном дії
- C** Зробити bucket публічним на запис
- D** Вбудувати access keys IAM-користувача в застосунок

Правильна відповідь: B. Presigned URL дає тимчасовий дозвіл на одну операцію з конкретним об'єктом, підписаний правами бекенду. Файл іде напряму в S3. API Gateway обмежений розміром запиту (10 MB), ключі в застосунку і публічний запис — небезпечні.

ПИТАННЯ 2

Компанія завантажує в S3 файли резервних копій по 80 GB через мережу, що інколи обривається. Що використати?

- A** S3 Select
- B** Multipart upload
- C** Presigned URL на читання
- D** Один PUT-запит

Правильна відповідь: B. Один PUT обмежений 5 GB. Multipart upload ділить об'єкт на частини, які вантажаться паралельно, а збірні частини перезавантажуються окремо.

ПИТАННЯ 3

Застосунок читає об'єкти з S3 з темпом 40 000 запитів за секунду і отримує помилки 503 Slow Down. Усі об'єкти мають один префікс. Що зробити?

- A** Використати S3 Transfer Acceleration
- B** Розподілити об'єкти між кількома префіксами
- C** Увімкнути versioning
- D** Перейти на клас Standard-IA

Правильна відповідь: B. S3 дає щонайменше 5 500 GET/HEAD за секунду на префікс. Розподіл ключів по кількох префіксах пропорційно збільшує сумарну пропускну здатність.

ПИТАННЯ 4

Що означає durability 99,99999999% для S3 Standard?

- A** Імовірність втрати об'єкта надзвичайно мала, бо дані зберігаються з надлишковістю в кількох AZ
- B** S3 доступний 99,99999999% часу
- C** Дані автоматично копіюються в інший регіон
- D** Об'єкт неможливо видалити

Правильна відповідь: A. Durability — про збереження даних, а не про доступність сервісу (у S3 Standard доступність — 99,99%). Копіювання в інший регіон потребує окремо налаштованої реплікації.

ПИТАННЯ 5

Після кожного завантаження зображення в bucket потрібно запускати кілька незалежних обробників з гнучкими фільтрами за ключем і розміром об'єкта. Що найзручніше?

- A** Налаштувати cron на EC2, що перевіряє bucket
- B** Увімкнути надсилання подій S3 в Amazon EventBridge і створити правила для кожного обробника
- C** S3 Transfer Acceleration
- D** Requester Pays

Правильна відповідь: B. EventBridge отримує всі події S3 і дозволяє кілька правил з детальними фільтрами та різними цілями. Опитування bucket скриптом — зайва робота й затримки.

ПИТАННЯ 6

Статичний сайт розміщено в S3 з увімкненим static website hosting. Потрібен HTTPS і власний домен, а bucket має лишитися приватним. Яке рішення правильне?

- A** Увімкнути HTTPS у налаштуваннях static website hosting
- B** CloudFront з origin на bucket через ОАС, сертифікат ACM у us-east-1 і Route 53 Alias на distribution
- C** Зробити bucket публічним і додати сертифікат у властивості bucket
- D** Поставити Network Load Balancer перед bucket

Правильна відповідь: B. Website endpoint S3 підтримує лише HTTP. HTTPS, власний домен і приватний bucket дає CloudFront з ОАС і сертифікатом ACM; DNS — Alias-запис Route 53.

ПИТАННЯ 7

Які твердження про нові bucket S3 за замовчуванням правильні?
(Оберіть 2)

Оберіть 2 відповіді

- ☐ A Bucket автоматично реплікується в інший регіон
- ☐ B Об'єкти доступні всім для читання
- ☐ C Block Public Access увімкнено
- ☐ D ACL є основним способом керування доступом
- ☐ E Нові об'єкти шифруються SSE-S3

Правильна відповідь: C, E. Нові bucket приватні (Block Public Access), ACL вимкнені на користь політик, а нові об'єкти шифруються SSE-S3. Реплікацію треба налаштовувати окремо.



S3 глибше — класи, lifecycle, версії, реплікація, Object Lock і керування даними

🎯 Безпека · 30%

🎯 Відмовостійкість · 26%

🎯 Вартість · 20%



Історія Хмаринки. Через рік у S3 Хмаринки — 30 ТБ: фото, рахунки, екпорти, журнали, бекапи. Марта питає, чому рахунок за S3 росте щомісяця, якщо рахунки за 2021 рік ніхто не відкриває. Податкова вимагає зберігати бухгалтерські документи роками — і так, щоб їх **неможливо** було змінити заднім числом. Стажер випадково перезаписав фото головного банера — повернути його не було звідки. А Тарас хоче копію найважливіших даних в іншому регіоні на випадок аварії. Усе це — тонкі налаштування S3, яких найбільше на іспиті.



Образ. S3 — склад з кількома зонами зберігання. **Standard** — полиці біля входу: брати швидко, але оренда дорога. **Infrequent Access** — задні полиці: оренда дешевша, але за кожен винос коробки — доплата. **Glacier** — холодний склад за містом: зберігати копійки, а привезти коробку — від мілісекунд до двох діб. **Lifecycle** — завскаду, що за графіком переносить старі коробки все далі. **Versioning** — копії кожної попередньої версії коробки. **Object Lock** — сейф із таймером: до вказаної дати не відкриє навіть директор.

Класи зберігання з цифрами

Клас	Зони	Доступ	Мінімальний строк оплати	Плата за отримання	Для чого
Standard	3+ AZ	мс	—	—	Часті звернення: сайт, активні дані
Intelligent-Tiering	3+ AZ	мс (архівні рівні — години)	—	Немає; є плата за моніторинг об'єктів	Невідомий або змінний патерн доступу
Standard-IA	3+ AZ	мс	30 днів	За ГБ	Рідко, але потрібні миттєво (бекапи, старі звіти)
One Zone-IA	1 AZ	мс	30 днів	За ГБ	Рідко й можна відтворити (копії, мініатюри, вторинні бекапи)
Glacier Instant Retrieval	3+ AZ	мс	90 днів	За ГБ (вища)	Архів, до якого звертаються ~раз на квартал, але миттєво
Glacier Flexible Retrieval	3+ AZ	Expedited 1–5 хв, Standard 3–5 год, Bulk 5–12 год (безкоштовно)	90 днів	Залежить від швидкості	Архіви й бекапи, доступ рідко
Glacier Deep Archive	3+ AZ	Standard до 12 год, Bulk до 48 год	180 днів	Залежить від швидкості	Найдешевше: зберігання 7–10+ років для регулятора
Express One Zone	1 AZ	одноцифрові мс	—	—	Найшвидший S3 для ML, аналітики, обчислень в одній AZ

Ще два дрібні, але важливі правила:

- **Мінімальний розмір об'єкта для оплати — 128 KB** у Standard-IA, One Zone-IA і Glacier Instant Retrieval: мільйон дрібних файлів по 10 KB в IA коштуватиме як мільйон файлів по 128 KB.
- **Видалення раніше мінімального строку** все одно оплачується за весь мінімум (поклав у Deep Archive і видалив за місяць — заплатиш за 180 днів).

Provisioned capacity для Glacier Flexible Retrieval гарантує, що Expedited-відновлення за 1–5 хвилин спрацює навіть у пікові періоди.

Intelligent-Tiering: коли не знаєш, як читатимуть

S3 Intelligent-Tiering сам стежить за доступом до кожного об'єкта і переносить його між рівнями: немає звернень **30 днів** → рівень Infrequent Access; **90 днів** → Archive Instant Access; за бажанням — ще й рівні Archive Access і Deep Archive Access (години на отримання). Звернулися до об'єкта — він повертається в частий рівень. **Плати за отримання немає**, є невелика плата за моніторинг (об'єкти менші за 128 KB не моніторяться і лишаються в частому рівні). Правило для іспиту: **«unknown or changing access patterns»** → **Intelligent-Tiering**.

1 Патерн доступу невідомий чи змінюється?

Так → **Intelligent-Tiering**

2 Читають часто?

Так → **Standard** (найнижча затримка в одній AZ — **Express One Zone**)

3 Рідко, але потрібні миттєво?

Standard-IA · можна відтворити → **One Zone-IA** · раз на квартал → **Glacier Instant**

4 Архів: скільки чекати можна?

Хвилини–години → **Glacier Flexible** · до **12–48 год** → **Deep Archive**

Пам'ятай про мінімальні строки (30 / 90 / 180 днів) і плату за отримання — «най дешевше зберігання» не завжди найдешевше в сумі.

Lifecycle: дані дешевшають з віком

Lifecycle rules автоматично виконують дві дії для об'єктів (усього bucket, префікса або за тегами):

- **Transition** — перенести в дешевший клас через N днів після створення;
- **Expiration** — видалити через N днів.

Окремі правила можна задати для **попередніх версій** (noncurrent versions), для **незавершених multipart-завантажень** (abort через N днів) і для «осиротілих» delete markers.

Обмеження: у Standard-IA чи One Zone-IA можна перенести не раніше ніж через **30 днів** після створення; переходи йдуть лише «вниз» по сходах класів.

Standard

0–30 днів: документи в роботі



Standard-IA

30–365 днів: рідко, але миттєво для перевірок



Glacier Flexible

1–3 роки: архів, години на отримання



Deep Archive

3–7 років: для регулятора



Expiration

видалити через 7 років

Одне правило — і щомісячний рахунок за старі документи падає в десятки разів. Чи підходить твоїм даним такий графік, підкаже **Storage Class Analysis**.

Versioning: захист від перезапису і видалення

Versioning зберігає **усі версії** кожного об'єкта:

- перезапис створює **нову версію**, стара лишається;
- «видалення» лише додає **delete marker** — об'єкт виглядає видаленим, але всі версії на місці; прибери маркер — і об'єкт «повернувся»;
- остаточно видалити можна лише конкретну версію за її ID;
- стани bucket: не ввімкнено → **enabled** → **suspended** (вимкнути повністю вже не можна);

- кожна версія — окремий об'єкт у рахунку → обмежуй lifecycle-правилами для noncurrent versions.

MFA Delete — додатковий захист: для **остаточного видалення** версій і зміни стану versioning потрібен код MFA. Вмикає лише **root user** (через CLI/API).

Versioning — обов'язкова умова для **реплікації** і **Object Lock**.

Реплікація: копія в іншому регіоні чи акаунті

- **CRR** (Cross-Region Replication) — в інший регіон: аварійне відновлення, менша затримка для інших регіонів, вимоги регуляторів «копія в іншому місці».
- **SRR** (Same-Region Replication) — у тому ж регіоні: зібрати журнали з багатьох bucket в один, копія з prod у тестовий акаунт, суверенність даних.

Правила реплікації:

- **Versioning увімкнено** в обох bucket; S3 потрібна IAM-роль з правами.
- Реплікуються **нові** об'єкти після ввімкнення правила. **Наявні** — через **S3 Batch Replication**.
- Можна реплікувати в **інший клас** зберігання (наприклад, копія одразу в Glacier) і в **інший акаунт** зі зміною власника на акаунт призначення (захист від компрометації основного акаунта).
- Видалення **конкретних версій** не реплікуються (захист від зловмисного видалення); реплікація delete markers — опційна.
- **Replication Time Control (RTC)** — гарантія за SLA: **99,99% об'єктів за 15 хвилин**, з метриками і сповіщеннями.
- Ланцюжки не транзитивні: $A \rightarrow B$ і $B \rightarrow C$ не означає, що об'єкти з A потраплять у C.

Object Lock: WORM-сейф

S3 Object Lock — модель **WORM** (write once, read many): версію об'єкта неможливо видалити чи перезаписати до вказаної дати.

Режим	Хто може обійти	Для чого
Compliance	Ніхто — навіть root user; строк не можна скоротити	Вимоги регуляторів (фінанси, SEC 17a-4, медицина), захист від ransomware
Governance	Користувачі з окремим дозволом <code>s3:BypassGovernanceRetention</code>	Внутрішні правила з можливістю адміністративного винятку; тестування налаштувань
Legal hold	Знімається користувачем з дозволом <code>s3:PutObjectLegalHold</code>	Утримання без строку — на час суду чи розслідування

Object Lock вмикається на bucket з увімкненим versioning; можна задати строк за замовчуванням для всіх нових об'єктів.

Доступ і керування на масштабі

- **S3 Access Points** — іменовані «входи» в bucket, кожен зі **своєю політикою**: окремий access point для аналітиків (лише `reports/`), для застосунку сайту, для партнера. Access point можна обмежити **лише доступом з VPC**. Замість однієї гігантської bucket policy — кілька простих.
- **Multi-Region Access Points** — **один глобальний endpoint** для bucket у кількох регіонах: запити йдуть до найближчої копії мережею AWS, з керованим failover між регіонами (разом з двосторонньою реплікацією).
- **S3 Batch Operations** — одна дія над **мільярдами** об'єктів за списком: скопіювати, змінити теги чи клас, відновити з Glacier, змінити налаштування Object Lock, **викликати Lambda** для кожного об'єкта.

- **S3 Inventory** — щоденний чи щотижневий звіт-список об'єктів з метаданими (клас, шифрування, стан реплікації) у CSV/Parquet — для аудиту й як вхід для Batch Operations.
- **S3 Storage Lens** — аналітика використання і активності сховища по всій організації з рекомендаціями економії.
- **Storage Class Analysis** — аналізує, як часто читають дані, і підказує, коли вигідно переводити їх у Standard-IA.
- **Журнали:** server access logs (хто що запитував, в інший bucket) і **CloudTrail data events** (API-виклики на рівні об'єктів).



Що змінилось

S3 Select закрито для нових клієнтів з 2024 року (для SQL-запитів по файлах — **Athena**), **S3 Object Lambda** — з листопада 2025. Окремий старий сервіс **Amazon Glacier** зі «сховищами» (vaults) і Vault Lock теж закритий для нових клієнтів — але **класи S3 Glacier** працюють як завжди. На іспиті назви можуть зустрічатися в старому вигляді.

Схема даних Хмаринки в S3

Дані	Клас і правила
Фото товарів (khmarynka - photos)	Intelligent-Tiering (популярність товарів непередбачувана); versioning + видалення старих версій через 30 днів; CRR в eu-west-1 для DR
Бухгалтерські документи (khmarynka - accounting)	Lifecycle: Standard → IA (30 д) → Glacier Flexible (1 p) → Deep Archive (3 p) → видалення (7 p); Object Lock Compliance 7 років
Звіти для партнерів (генеруються щоночі)	One Zone-IA — можна відтворити; видалення через 90 днів
Журнали (ALB, CloudTrail, Flow Logs)	Standard 30 днів → Glacier Instant → видалення через 1 рік

Дані	Клас і правила
Усі bucket	Block Public Access на рівні акаунта; правило «abort incomplete multipart uploads — 7 днів»; Storage Lens для контролю

Спробуй у справжньому AWS: версії, delete marker і lifecycle

Вартість: частки цента. **Час:** 20 хвилин. Потрібен bucket з розділу 27 (або новий).

1. Bucket → **Properties** → **Bucket Versioning** → **Enable**.
2. Створи на комп'ютері `banner.txt` з текстом «Знижка 10%» і завантаж. Зміни текст на «Знижка 50%» і завантаж знову з тим самим ім'ям.
3. Увімкни перемикач **Show versions** — побачиш дві версії. Відкрий стару — «Знижка 10%» ціла.
4. Вимкни Show versions і **видали** `banner.txt`. Увімкни Show versions знову — з'явився **Delete marker**, а обидві версії на місці. Видали сам delete marker — файл «повернувся».
5. **Management** → **Create lifecycle rule** `archive-old`: префікс `products/`; дії: *Transition current versions* → **Standard-IA** **через 30 днів**, **Glacier Flexible Retrieval** **через 365 днів**; *Permanently delete noncurrent versions* **через 30 днів**; *Delete incomplete multipart uploads* **через 7 днів**. Подивись у підсумку «timeline» правила.
6. **S3** → **Storage Lens** → **Dashboards** → **default-account-dashboard** — безкоштовні метрики твого сховища (дані з'являються з затримкою до 48 годин).

Прибери за собою: видали lifecycle-правило; щоб видалити bucket з версіями — **Empty** (видаляє всі версії), потім **Delete**.

Запам'ятай

Класи: **Standard**; **Intelligent-Tiering** (невідомий патерн, без плати за отримання); **Standard-IA** (30 днів, мс); **One Zone-IA** (1 AZ, відтворювані дані); **Glacier Instant** (90 днів, мс); **Glacier Flexible** (90 днів; 1–5 хв / 3–5 год / 5–12 год); **Deep Archive** (180 днів; до 12 / 48 год); **Express One Zone** (найшвидший, 1 AZ).

Мінімальний розмір для оплати **128 KB** в IA і Glacier Instant; раннє видалення оплачується за весь мінімум.

Lifecycle: transition + expiration, окремо для noncurrent versions і multipart; в IA — не раніше ніж через 30 днів.

Versioning: delete marker замість видалення; **MFA Delete** — лише root.

CRR/SRR: versioning з обох боків; лише нові об'єкти (наявні — **Batch Replication**); **RTC** — 99,99% за 15 хв.

Object Lock: Compliance — ніхто не обійде; **Governance** — з дозволом; **Legal hold** — без строку.

Access Points (свої політики, VPC-only), **Multi-Region Access Points**, **Batch Operations**, **Inventory**, **Storage Lens**, **Storage Class Analysis**.

Як питають на іспиті

Невідомий або змінний патерн доступу, без ручного керування → **S3 Intelligent-Tiering**

Дані читають рідко, але потрібні миттєво → **S3 Standard-IA (раз на квартал — Glacier Instant Retrieval)**

Відтворювані дані, рідко потрібні, найдешевше з мілісекундним доступом → **S3 One Zone-IA**

Архів, іноді потрібен доступ за кілька хвилин → **Glacier Flexible Retrieval (Expedited)**

Зберігати 7–10 років, доступ до 48 годин прийнятний, найдешевше → **Glacier Deep Archive**

Гарантувати, що Expedited-відновлення завжди спрацює → **provisioned capacity**

Автоматично перенести в Glacier через 90 днів і видалити через 5 років → **lifecycle rule**

Захист від випадкового видалення або перезапису → **versioning (+ MFA Delete)**

Ніхто, навіть root, не може видалити дані N років → **Object Lock, Compliance mode**

Копія bucket в іншому регіоні для DR → **Cross-Region Replication (наявні об'єкти — Batch Replication)**

Копія має з'явитися в іншому регіоні з гарантованим часом → **CRR + Replication Time Control**

Зібрати журнали з кількох bucket в один у тому ж регіоні → **Same-Region Replication**

Багато команд з різними правами до одного великого bucket → **S3 Access Points**

Змінити клас чи теги мільйонів наявних об'єктів → **S3 Batch Operations**

Вирішити на основі даних, коли переводити об'єкти в IA → **Storage Class Analysis**

Пастки

One Zone-IA і Express One Zone живуть в **одній AZ** — втрата зони означає втрату даних.

IA і Glacier мають **мінімальні строки** і плату за отримання — для часто читаних даних вони дорогі.

Реплікація не копіює наявні об'єкти сама по собі і не працює без versioning.

Versioning не можна повністю вимкнути — лише призупинити.

Governance mode можна обійти; якщо в питанні «ніхто, навіть адміністратор» — **Compliance**.

✓ Перевір себе

ПИТАННЯ 1

Компанія зберігає медичні знімки: перші 30 днів їх часто переглядають, потім — кілька разів на рік, але лікар має отримати знімок миттєво. Через 7 років знімки треба видаляти. Що налаштувати найекономніше?

- A** One Zone-IA з першого дня
- B** Lifecycle: Standard → Glacier Deep Archive через 30 днів
- C** Lifecycle: Standard → Glacier Instant Retrieval через 30 днів → видалення через 7 років
- D** Зберігати все в S3 Standard і видаляти вручну

Правильна відповідь: C. Glacier Instant Retrieval дає мілісекундний доступ і дешеве зберігання для даних, які читають кілька разів на рік. Deep Archive вимагає годин на отримання, а One Zone-IA ризикує даними при втраті AZ.

ПИТАННЯ 2

Регулятор вимагає, щоб фінансові записи не можна було змінити чи видалити 10 років — ні користувачам, ні адміністраторам, ні root. Що обрати?

- A** S3 Object Lock у режимі Compliance з retention 10 років
- B** Bucket policy з Deny на s3:DeleteObject
- C** S3 Object Lock у режимі Governance
- D** Versioning з MFA Delete

Правильна відповідь: A. Compliance mode не дозволяє нікому (навіть root) видалити версію чи скоротити строк. Governance можна обійти з окремим дозволом, а bucket policy адміністратор може змінити.

ПИТАННЯ 3

У bucket 500 мільйонів об'єктів з різною і непередбачуваною частотою звернень. Команда не хоче аналізувати доступ і налаштовувати правила. Що обрати для зниження витрат?

- A** Lifecycle у Standard-IA через 30 днів
- B** One Zone-IA для всіх об'єктів
- C** Glacier Deep Archive з Bulk-відновленням
- D** S3 Intelligent-Tiering

Правильна відповідь: D. Intelligent-Tiering сам переносить об'єкти між рівнями за реальним доступом, без плати за отримання. Lifecycle «наосліп» у IA може здорожчити часто читані об'єкти.

ПИТАННЯ 4

Компанія увімкнула Cross-Region Replication для bucket, у якому вже 20 ТБ даних. Через тиждень у bucket призначення з'явилися лише нові об'єкти. Як скопіювати наявні?

- A** Вимкнути і знову увімкнути versioning
- B** Використати S3 Batch Replication
- C** Змінити клас зберігання
- D** Увімкнути Replication Time Control

Правильна відповідь: B. Правила реплікації застосовуються лише до об'єктів, створених після їх увімкнення. Наявні об'єкти реплікує S3 Batch Replication.

ПИТАННЯ 5

Співробітник випадково перезаписав важливий файл у bucket, а за день — видалив інший. Як захиститися від таких ситуацій надалі?

- A** Перейти на клас One Zone-IA
- B** Увімкнути static website hosting
- C** Увімкнути S3 Transfer Acceleration
- D** Увімкнути versioning (і за потреби MFA Delete)

Правильна відповідь: D. З versioning перезапис створює нову версію, а видалення — delete marker; попередні версії можна відновити. MFA Delete додатково захищає від остаточного видалення версій.

ПИТАННЯ 6

Дані з архіву Glacier Flexible Retrieval іноді потрібні терміново, і Expedited-відновлення за 1–5 хвилин має спрацьовувати завжди, навіть у пікові періоди. Що додати?

- A** Перевести дані в Glacier Deep Archive
- B** Використати Bulk retrieval
- C** Provisioned capacity для Expedited-відновлень
- D** Увімкнути Cross-Region Replication

Правильна відповідь: C. Provisioned capacity гарантує пропускну здатність для Expedited-запитів. Bulk — найповільніший (5–12 год), Deep Archive — ще повільніший.

ПИТАННЯ 7

П'ять команд працюють з одним bucket з різними правами: аналітики лише читають reports/, застосунок пише в uploads/, партнер читає exports/. Bucket policy стала величезною і заплутаною. Що зробити?

- A** Створити окремі S3 Access Points для кожної команди з власними політиками
- B** Розділити дані на п'ять регіонів
- C** Використати S3 Select
- D** Увімкнути Requester Pays

Правильна відповідь: A. Access Points — окремі точки доступу з власними політиками (і, за бажанням, лише з VPC). Керувати кількома простими політиками легше, ніж однією гігантською.

ПИТАННЯ 8

Які твердження про мінімальні строки й оплату класів S3 правильні? (Оберіть 2)

Оберіть 2 відповіді

- ☐ А У Standard-IA об'єкти менші за 128 KB оплачуються як 128 KB
- ☐ В Intelligent-Tiering бере плату за кожне отримання даних
- ☐ С One Zone-IA зберігає дані в трьох AZ
- ☐ D Об'єкт, видалений з Glacier Deep Archive через місяць, оплачується за 180 днів
- ☐ Е Standard-IA не має мінімального строку зберігання

Правильна відповідь: А, D. Deep Archive має мінімум 180 днів, Standard-IA — 30 днів і мінімальний розмір 128 KB. Intelligent-Tiering не бере плати за отримання, а One Zone-IA живе в одній AZ.



Спільні файли — Amazon EFS і сімейство Amazon FSx



Відмовостійкість · 26%



Продуктивність · 24%



Вартість · 20%



Історія Хмаринки. Три «файлові» болі. Перша: старий модуль сайту зберігає завантажені продавцями файли в звичайну папку на диску — а тепер серверів шість, і кожен бачить лише свою папку. Друга: у бухгалтерії Windows-комп'ютери працюють зі спільним мережевим диском `\\buh\docs` з правами за обліковими записами Active Directory — і його треба перенести в хмару. Третя: аналітики хочуть навчати модель прогнозу попиту на терабайтах даних з S3, і їм потрібна дуже швидка файлова система. Три задачі — три різні файлові сервіси.



Образ. EFS — спільна шафа з документами в коридорі офісу на кількох поверхах (зонах): кожен Linux-сервер відкриває її і бачить те саме, а шафа сама росте, коли документів більшає. **FSx** — набір спеціалізованих архівів: **Windows** — архів за правилами Windows з перепустками Active Directory; **Lustre** — гоночний конвеєр для суперкомп'ютерів; **NetApp ONTAP** — універсальний архів, що говорить усіма мовами; **OpenZFS** — знайомий архів для тих, хто вже звик до ZFS.

Навіщо файлове сховище

Нагадування з розділу 4: **блочне** сховище (EBS) — диск одного сервера в одній AZ; **об'єктне** (S3) — необмежений склад через HTTP, без «монтування». **Файлове** — спільна файлова система, яку **одночасно** монтують багато серверів і працюють з нею як зі звичайною папкою: відкрити файл, дописати рядок, заблокувати файл від одночасного запису. Мова доступу — **протокол**: **NFS** для Linux, **SMB** для Windows.

Amazon EFS

Amazon Elastic File System — керована **NFS**-файлова система для **Linux**:

- **спільна**: тисячі клієнтів одночасно — сервери EC2, контейнери **ECS** і **EKS**, функції **Lambda**, навіть сервери в офісі через VPN/Direct Connect;
- **еластична**: росте й зменшується автоматично до петабайтів; платиш за **фактично збережені** гігабайти (без попереднього виділення);
- **регіональна** (Regional): дані зберігаються в **кількох AZ**, а в кожній AZ створюється **mount target** — мережевий інтерфейс у підмережі, через який клієнти монтують систему. Або **One Zone** — дешевше, дані в одній AZ.



Продуктивність і вартість EFS

- **Throughput modes: Elastic** (рекомендовано: пропускна здатність автоматично під навантаження, оплата за використане), **Provisioned** (фіксована пропускна здатність незалежно від розміру), **Bursting** (залежить від обсягу даних, накопичує кредити).
- **Performance mode:** General Purpose — найнижча затримка, стандарт для більшості задач.
- **Класи зберігання: Standard, Infrequent Access (IA) і Archive + lifecycle management:** файли, яких не відкривали N днів, автоматично переходять у дешевші класи (і за бажанням повертаються в Standard при зверненні). EFS за гігабайт дорожча за EBS — lifecycle суттєво знижує рахунок.
- **Безпека:** шифрування at rest (KMS) і в дорозі (TLS через EFS mount helper), POSIX-права, **IAM-авторизація** клієнтів, **access points** — окремі «входи» для застосунків з фіксованим користувачем POSIX і кореневою папкою.
- **Надійність: EFS Replication** — копія файлової системи в іншому регіоні (RPO — хвилини); бекапи — AWS Backup.

Amazon FSx: чотири спеціалізовані файлові системи

FSx for Windows File Server

Повноцінний **Windows-файловий сервер**, керований AWS:

- протокол **SMB**, файлова система **NTFS**, права **ACL Windows**;
- інтеграція з **Active Directory** — AWS Managed Microsoft AD або **власним** AD в офісі;
- **DFS Namespaces**, тінюві копії (Previous Versions), дедуплікація, квоти;
- **Multi-AZ** розгортання для високої доступності;
- доступ з офісу через VPN/Direct Connect.

Для чого: спільні диски Windows-користувачів, домашні папки, застосунки на .NET і SharePoint, спільне сховище для кластерів Microsoft SQL Server.

EFS для Windows не підходить — це NFS і Linux.

FSx for Lustre

Файлова система для **високопродуктивних обчислень**: сотні ГБ/с, мільйони IOPS, затримка менше мілісекунди.

- **Інтеграція з S3**: файлова система «підключається» до bucket — файли підтягуються з S3 при першому зверненні, а результати записуються назад у S3.
- **Scratch** — тимчасова, найдешевша, без реплікації (для короточасних обчислень); **Persistent** — довготривала, з реплікацією в межах AZ.
- Лише **Linux**.

Для чого: **НРС, навчання ML-моделей, рендеринг, моделювання, геноміка, фінансові розрахунки.**

FSx for NetApp ONTAP

Керований **NetApp ONTAP** — «швейцарський ніж»:

- **NFS, SMB і iSCSI одночасно** — Linux, Windows і macOS до тих самих даних;
- знімки, миттєві клони (FlexClone), реплікація SnapMirror, дедуплікація й стиснення, автоматичне перенесення холодних даних у дешевий рівень;
- **Multi-AZ**.

Для чого: **міграція з NetApp** в офісі, змішані Linux/Windows середовища, бази даних на iSCSI.

FSx for OpenZFS

Керована **OpenZFS** з доступом по **NFS**: знімки, клони, стиснення, низька затримка. Для чого: **міграція з ZFS** і Linux-файлових серверів без змін у застосунках.

Що обрати

Вимога	Вибір
Спільна файлова система для Linux у кількох AZ, еластична, без керування	Amazon EFS
Спільна папка для Windows з Active Directory , SMB, NTFS	FSx for Windows File Server
HPC / ML , сотні ГБ/с, дані лежать у S3	FSx for Lustre
NFS + SMB + iSCSI одночасно, міграція з NetApp	FSx for NetApp ONTAP
Міграція з ZFS , NFS з низькою затримкою	FSx for OpenZFS
Диск для одного сервера (база, ОС)	EBS (розділ 21)
Необмежений склад файлів через HTTP	S3 (розділ 27)



Протокол підказує сервіс

NFS + Linux + кілька AZ → EFS. SMB + Windows + AD → FSx for Windows. «HPC» чи «Lustre» чи «сотні ГБ/с» → FSx for Lustre. «NetApp» або «NFS і SMB одночасно» → FSx for ONTAP. «ZFS» → FSx for OpenZFS.

Рішення для Хмаринки

- Файли, які завантажують продавці через старий модуль сайту, → **EFS Regional** з **access point** для застосунку; усі шість серверів (і згодом контейнери) бачать одну папку; lifecycle → IA через 30 днів.
- Спільний диск бухгалтерії → **FSx for Windows File Server Multi-AZ**, інтегрований з корпоративним AD через AWS Managed Microsoft AD (з довірою до офісного AD); офіс підключається через VPN.
- Навчання моделі прогнозу попиту → **FSx for Lustre (Scratch)**, прив'язана до bucket з історією продажів, на час тренування.

Спробуй у справжньому AWS: одна EFS для двох серверів

Вартість: кілька МБ в EFS і два маленькі сервери на 30 хвилин — центи. **Час:** 30 хвилин.

1. Запусти два сервери Amazon Linux 2023 у **різних AZ** твоєї VPC (як у розділі 19). Security Group серверів — `sg-efs-clients`.
2. Створи Security Group `sg-efs` з вхідним правилом **NFS (2049)**, джерело — `sg-efs-clients`.
3. **EFS** → **Create file system** → **Customize**: тип **Regional**, throughput **Elastic**, lifecycle — IA через 30 днів. На кроці Network для mount targets в обох AZ обери Security Group `sg-efs`.
4. На **кожному** сервері виконай (підстав ID своєї файлової системи `fs-...`):

```
sudo dnf install -y amazon-efs-utils
sudo mkdir -p /mnt/shared
sudo mount -t efs -o tls fs-0123456789abcdef0:/ /mnt/shared
df -h /mnt/shared
```

1. На першому сервері: `echo "Замовлення з AZ a" | sudo tee /mnt/shared/hello.txt`. На другому: `cat /mnt/shared/hello.txt` — файл той самий.
2. Навмисно прибери правило 2049 з `sg-efs` і спробуй змонтувати ще раз на новій папці — отримаєш таймаут. Поверни правило.

Прибери за собою: `sudo umount /mnt/shared` на обох серверах; видали файлову систему EFS, сервери, обидві Security Groups.

Запам'ятай

EFS — керована **NFS** для **Linux**; тисячі клієнтів (EC2, ECS, EKS, Lambda, on-prem); **Regional** (кілька AZ, mount target у кожній) або **One Zone**; росте сама; оплата за збережене.

EFS: throughput **Elastic** (рекомендовано) / Provisioned / Bursting; класи **Standard, IA, Archive** + lifecycle; access points; replication в інший регіон; SG з **TCP 2049**.

FSx for Windows — SMB, NTFS, **Active Directory**, DFS, Multi-AZ.

FSx for Lustre — HPC/ML, сотні ГБ/с, **інтеграція з S3**; Scratch чи Persistent; Linux.

FSx for NetApp ONTAP — **NFS + SMB + iSCSI**, міграція з NetApp, Multi-AZ.

FSx for OpenZFS — міграція з ZFS, NFS.

Як питають на іспиті

Спільна файлова система для багатьох Linux-серверів у різних AZ → [Amazon EFS](#)

Контейнерам ECS/EKS або функціям Lambda потрібне спільне постійне файлове сховище → [Amazon EFS](#)

Спільна файлова система для Windows з інтеграцією з Active Directory → [FSx for Windows File Server](#)

HPC або навчання ML з дуже високою продуктивністю, дані в S3 → [FSx for Lustre](#)

Потрібні NFS і SMB одночасно або міграція з NetApp → [FSx for NetApp ONTAP](#)

Файли в EFS майже не відкривають після 30 днів, рахунок росте → [EFS lifecycle management](#) → [IA / Archive](#)

Сервери не можуть змонтувати EFS (таймаут) → [дозволити TCP 2049 у Security Group mount target](#)

Найдешевша EFS для dev-середовища, висока доступність не потрібна → [EFS One Zone](#)

Тимчасова швидка файлова система на час обчислень, найдешевше → [FSx for Lustre Scratch](#)

Копія файлової системи EFS в іншому регіоні для DR → [EFS Replication](#)

Пастки

EFS — **не** для Windows (NFS, Linux); для Windows — FSx for Windows File Server.

EBS не можна одночасно підключити до багатьох серверів у різних AZ — для спільних файлів EFS.

FSx for Lustre Scratch не реплікує дані — лише для тимчасових обчислень.

EFS за гігабайт дорожча за EBS і S3 — без lifecycle рахунок швидко росте.

✓ Перевір себе

ПИТАННЯ 1

Вебзастосунок на десяти Linux-серверах у трьох AZ має зберігати завантажені користувачами файли так, щоб усі сервери бачили їх одночасно. Обсяг непередбачуваний. Що обрати?

- A** FSx for Windows File Server
- B** Amazon EFS (Regional)
- C** Instance store на кожному сервері
- D** Том EBS gp3, підключений до всіх серверів

Правильна відповідь: B. EFS — спільна NFS-файлова система для Linux, доступна з кількох AZ, що росте автоматично. EBS живе в одній AZ (Multi-Attach — лише io1/io2 і в межах AZ), instance store — локальний і тимчасовий.

ПИТАННЯ 2

Компанія переносить Windows-застосунки, що використовують спільні папки SMB з правами користувачів Active Directory. Що обрати?

- A** Amazon FSx for Windows File Server
- B** Amazon EFS
- C** Amazon FSx for Lustre
- D** Amazon S3 з bucket policy

Правильна відповідь: A. FSx for Windows File Server дає SMB, NTFS і інтеграцію з Active Directory. EFS працює з NFS і Linux, Lustre — для HPC.

ПИТАННЯ 3

Дослідницька команда навчає модель на 200 ТБ даних, що лежать у S3. Потрібна файлова система з сотнями ГБ/с пропускної здатності лише на час навчання, якомога дешевше. Що обрати?

- A** FSx for NetApp ONTAP
- B** EFS з Provisioned throughput
- C** FSx for Lustre (Scratch), пов'язана з bucket S3
- D** EBS io2 Block Express

Правильна відповідь: C. FSx for Lustre інтегрується з S3 і дає продуктивність HPC. Scratch-розгортання — найдешевше для тимчасових задач (без реплікації).

ПИТАННЯ 4

Компанія переносить в AWS сховище NetApp, до якого одночасно звертаються Linux-сервери по NFS і Windows-сервери по SMB, а бази даних — по iSCSI. Що обрати?

- A** FSx for OpenZFS
- B** FSx for NetApp ONTAP
- C** EFS і FSx for Windows окремо з синхронізацією
- D** S3 з кількома Access Points

Правильна відповідь: B. FSx for NetApp ONTAP підтримує NFS, SMB і iSCSI одночасно і має звичні можливості NetApp — ідеально для міграції.

ПИТАННЯ 5

Сервери EC2 у приватній підмережі не можуть змонтувати EFS — команда mount завершується таймаутом. Що найімовірніше?

- A** Security Group mount target не дозволяє вхідний TCP 2049 від серверів
- B** Потрібна публічна IP-адреса для кожного сервера
- C** EFS треба спершу реплікувати в інший регіон
- D** EFS не підтримує приватні підмережі

Правильна відповідь: A. Клієнти звертаються до mount target по NFS (TCP 2049). Якщо Security Group mount target не дозволяє цей порт від серверів, з'єднання не встановлюється.

ПИТАННЯ 6

Файли в EFS активно використовують перший тиждень, а потім майже не відкривають, але мають лишатися доступними. Як знизити вартість з мінімальними зусиллями?

- A** Щотижня копіювати файли в Glacier скриптом
- B** Перейти на FSx for Lustre
- C** Увімкнути EFS lifecycle management з переходом у Infrequent Access (і Archive)
- D** Перейти на EFS One Zone з перенесенням даних вручну

Правильна відповідь: C. Lifecycle management автоматично переносить файли, яких давно не відкривали, у дешевші класи EFS; файли лишаються доступними в тій самій файлової системі.

ПИТАННЯ 7

Які твердження про Amazon EFS правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** Регіональна EFS зберігає дані лише в одній AZ
- B** EFS розрахована на Windows-клієнтів через SMB
- C** EFS може бути кореневим диском сервера EC2
- D** Файлову систему можуть одночасно монтувати тисячі клієнтів, зокрема ECS, EKS і Lambda
- E** Оплата йде за фактично збережені дані, розмір задавати наперед не потрібно

Правильна відповідь: D, E. EFS — спільна еластична NFS для Linux-клієнтів: платиш за збережене, розмір росте сам. Для Windows — FSx, кореневий диск — EBS, а Regional EFS зберігає дані в кількох AZ.



Гібридні сховища і перенесення даних — Storage Gateway, DataSync, Transfer Family, Snow

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Вартість · 20%



Історія Хмаринки. Залишки «старого світу» в офісі: мережеве сховище (NAS) на 8 ТБ з фото й документами за десять років; бухгалтерія, яка звикла до мережевої папки зі сканами і не хоче нічого міняти; стара програма резервного копіювання, що досі пише на **магнітні стрічки (!)**, які кур'єр щотижня везе в банківську комірку; і постачальники, що надсилають прайс-листи через **SFTP**. Як перенести дані в AWS швидко й без втрат, а тим, хто не готовий мінятися, дати хмару «під виглядом» звичних інструментів?



Образ. Storage Gateway — перекладач-посередник в офісі: офісні програми говорять звичною мовою (мережева папка, диск, стрічка), а посередник тихо зберігає все в хмарі, тримаючи найпотрібніше під рукою (локальний кеш). **DataSync** — вантажна компанія з власними швидкими фурами, що перевозить твої архіви мережею і звіряє кожну коробку. **Transfer Family** — поштове відділення в хмарі, що приймає посилки у звичному для партнерів форматі SFTP. **Snowball** — фура-сейф, яку привозять до дверей, коли мережею везти занадто довго.

Карта рішень

Задача	Сервіс
Офісні програми працюють з хмарним сховищем через звичні протоколи (NFS, SMB, iSCSI, стрічки) з локальним кешем — постійно	AWS Storage Gateway
Перенести або синхронізувати великі обсяги даних мережею	AWS DataSync
Партнери надсилають файли по SFTP / FTPS / FTP / AS2, зберігати в S3 чи EFS	AWS Transfer Family
Мережею занадто довго: сотні ТБ, повільний канал, жорсткий дедлайн	Фізичне перенесення (Snowball, Data Transfer Terminal, партнери)
Прискорити завантаження в S3 з далеких країн	S3 Transfer Acceleration (розділ 27)
Постійний приватний канал з офісу	Direct Connect / VPN (розділ 16)
Бази даних і сервери	DMS, MGN (розділ 33)

AWS Storage Gateway

Storage Gateway — віртуальна машина (VMware, Hyper-V, KVM) в офісі або сервер EC2, що з'єднує локальні застосунки з хмарним сховищем. Чотири типи:

S3 File Gateway

Офіс бачить мережеву папку **NFS** або **SMB** (з Active Directory). Кожен файл стає **об'єктом у S3**, «гарячі» файли — у локальному кеші.

Далі працюють lifecycle, Glacier, аналітика по S3

Volume Gateway

Офіс бачить **блочні диски iSCSI**.

Cached: основні дані в S3, у кеші — гарячі.

Stored: усі дані локально, асинхронні бекапи в S3 як знімки EBS

Tape Gateway

Віртуальна **стрічкова бібліотека (VTL)** для наявного бекап-ПЗ (Veeam, NetBackup, Commvault...).

Стрічки — в S3, архів — у Glacier / Deep Archive. Фізичні стрічки більше не потрібні

FSx File Gateway

Локальний кеш для **FSx for Windows File Server**.

З жовтня 2024 недоступний новим клієнтам

Спільне для всіх: застосунки в офісі не змінюються, а дані живуть у надійному й масштабованому сховищі AWS.

Cached чи Stored?

Cached — «основне в хмарі, під рукою лише гаряче»: економить місце в офісі. **Stored** — «усе в офісі, хмара — страховка»: мінімальна затримка до всіх даних локально, бекап в AWS. У питанні «основні дані в S3, локально лише часто використовувані» → **Cached**. «Усі дані мають бути доступні локально з мінімальною затримкою» → **Stored**.

AWS DataSync

DataSync — керований сервіс **онлайн-перенесення і синхронізації** даних:

- **Джерела**: NFS, SMB, HDFS, самокеровані об'єктні сховища, інші хмари — через **агента** (віртуальна машина в офісі); а також сервіси AWS між собою (без агента).
- **Призначення**: **S3** (будь-який клас, зокрема Glacier), **EFS**, усі **FSx**.

- **Швидко:** оптимізований протокол, паралельність — у рази швидше за самописні скрипти; до 10 Gbps на агента.
- **Надійно:** шифрування TLS, **перевірка цілісності** (контрольні суми), збереження метаданих і прав доступу.
- **Керовано:** розклад (щодоночі синхронізувати зміни), фільтри файлів, **обмеження пропускної здатності** (щоб не «забити» канал офісу вдень), журнали й звіти.
- Працює через інтернет, VPN або **Direct Connect**.

Типові сценарії: разова міграція NAS в S3 чи EFS; регулярна синхронізація на час переходу; перенесення даних для аналітики; копіювання між регіонами (EFS → EFS, S3 → S3).

AWS Transfer Family

Transfer Family — керовані сервери **SFTP, FTPS, FTP** і **AS2** з даними в **S3** або **EFS**:

- партнери працюють **як раніше** — тими самими клієнтами й скриптами, змінюється лише адреса сервера;
- користувачі — у самому сервісі, в Active Directory або в **власному провайдері** (через Lambda/API Gateway);
- endpoint публічний або **у VPC** (зі своїми Elastic IP для білих списків партнерів);
- файли потрапляють у S3 → **подія** запускає обробку (Lambda, Step Functions).

Фізичне перенесення: коли мережа не встигає

Швидко порахувати, скільки йтимуть дані мережею:



Скільки днів мережею?

Час $\approx$ обсяг (біти) $\div$ реальна швидкість. 1 ТБ $\approx 8 \times 10^{12}$ біт.

- **100 ТБ через 1 Gbps** при $\sim 80\%$ завантаженні каналу: $800\,000\,000\,000\,000\,000\text{ біт} \div 800\,000\,000\text{ біт/с} \approx 1\,000\,000\text{ с} \approx$ **11–12 днів**.
- Ті самі **100 ТБ через 100 Mbps**: $\approx$ **115 днів** — майже чотири місяці.
- **8 ТБ Хмаринки через 200 Mbps** (офісний канал, $\sim 70\%$ вільно): $\approx 8 \times 8 \times 10^{12} \div 1,4 \times 10^8 \approx 460\,000\text{ с} \approx$ **5–6 днів**. Прийнятно для DataSync з обмеженням швидкості вдень.

Орієнтир: якщо мережею це займе **понад тиждень-два**, а дедлайн жорсткий, розглядай фізичне перенесення.

- **AWS Snowball Edge** — захищений фізичний пристрій (десятки–сотні ТБ), який привозять, ти копіюєш дані локально, повертаєш — AWS завантажує їх у S3. Також — обчислення на краю (edge computing) там, де немає мережі.
-  **З 7 листопада 2025 року Snowball Edge недоступний новим клієнтам.** AWS радить **DataSync** (онлайн), **AWS Data Transfer Terminal** (фізичні локації AWS, куди привозиш свої носії і швидко завантажуєш дані в хмару) або партнерів. На іспиті Snowball усе ще може бути правильною відповіддю на питання «сотні ТБ, повільний канал, тиждень на все».

Рішення для Хмаринки

Офіс

NAS 8 ТБ

Папка сканів бухгалтерії (SMB)

Бекап-ПЗ на стрічки



Сервіси

DataSync: разово + нічна синхронізація

S3 File Gateway: та сама папка SMB з AD

Tape Gateway: віртуальні стрічки

Transfer Family SFTP: постачальники



AWS

S3 (Intelligent-Tiering)

S3 → lifecycle → Glacier

Glacier Deep Archive для стрічок

Подія S3 → Lambda: обробка прайсів

Бухгалтерія і бекап-ПЗ нічого не помітили — змінилося лише те, де фізично лежать дані. Кур'єр зі стрічками більше не потрібен.

Спробуй у справжньому AWS: DataSync між двома bucket

Вартість: DataSync бере невелику плату за кожен перенесений ГБ — для кількох МБ це частки цента. **Час:** 20 хвилин.

1. Створи два bucket: `khmarynka-src-<цифри>` і `khmarynka-dst-<цифри>`. Завантаж у перший кілька файлів.
2. **DataSync** → **Tasks** → **Create task**. Source location: **Amazon S3** → перший bucket (дозволь консолі створити IAM-роль). Destination: **Amazon S3** → другий bucket, клас **Standard**.
3. Settings: Verify data — **Verify only the data transferred**; Transfer mode — **Transfer only data that has changed**. Schedule — **Not scheduled**. Create.
4. **Start** → дочекайся Success. Подивись звіт: скільки файлів, байтів, час, результат перевірки.
5. Додай ще один файл у перший bucket і запусти задачу знову — перенесеться лише новий файл (інкрементально).

Прибери за собою: видали задачу і локації DataSync, очисти й видали обидва bucket, видали створену IAM-роль DataSync.

Запам'ятай

Storage Gateway — постійний гібридний доступ з локальним кешем: **S3 File Gateway** (NFS/SMB → об'єкти S3), **Volume Gateway** (iSCSI: **Cached** — основне в S3; **Stored** — усе локально, бекап в S3), **Tape Gateway** (VTL → S3/Glacier), FSx File Gateway (закритий для нових).

DataSync — онлайн-перенесення і синхронізація: NFS/SMB/HDFS/об'єкти/інші хмари ↔ S3, EFS, FSx; агент; перевірка цілісності, метадані, розклад, обмеження швидкості.

Transfer Family — керовані SFTP/FTPS/FTP/AS2 поверх S3 чи EFS; партнери нічого не змінюють.

Фізичне перенесення — коли мережею понад тиждень-два: Snowball Edge ( закритий для нових клієнтів з 07.11.2025), **Data Transfer Terminal**, партнери.

Рахуй: 100 ТБ через 1 Gbps ≈ 11–12 днів.

Як питають на іспиті

On-prem застосунки зберігають файли в S3 через NFS/SMB з локальним кешем → **S3 File Gateway**

Замінити фізичні стрічки, залишивши наявне бекап-ПЗ → **Tape Gateway**

iSCSI-томи: основні дані в хмарі, локально лише кеш → **Volume Gateway (Cached)**

Усі дані мають бути локально з мінімальною затримкою, бекап в AWS → **Volume Gateway (Stored)**

Перенести 50 ТБ з NAS в S3 або EFS мережею, за розкладом, з перевіркою цілісності → **AWS DataSync**

Регулярно синхронізувати файлову систему офісу з EFS під час міграції → **DataSync за розкладом**

Партнери надсилають файли по SFTP, зберігати треба в S3 → **AWS Transfer Family**

Сотні терабайтів, повільний канал, дедлайн — тиждень → **фізичне перенесення (Snowball Edge; для нових клієнтів — Data Transfer Terminal або партнери)**

Перенести дані з on-prem HDFS у S3 для аналітики → **DataSync**

Скопіювати дані між EFS у двох регіонах → **DataSync (або EFS Replication)**

Пастки

DataSync — перенесення і синхронізація; **Storage Gateway** — постійний гібридний доступ; **Transfer Family** — протоколи для партнерів. Не міняй їх місцями.

Snowball — **офлайн**, DataSync — **онлайн**.

Direct Connect не допоможе в терміновій задачі: його підключення займає тижні.

Volume Gateway Stored $\neq$ Cached: питання завжди підказує, де мають жити **основні** дані.

✓ Перевір себе

ПИТАННЯ 1

Компанія хоче, щоб офісні сервери зберігали файли в Amazon S3 через звичну мережеву папку SMB з правами Active Directory, а часто використовувані файли були доступні локально без затримок. Що обрати?

- A** Volume Gateway (Stored)
- B** AWS Transfer Family
- C** AWS DataSync
- D** AWS Storage Gateway — S3 File Gateway

Правильна відповідь: D. S3 File Gateway надає NFS/SMB-папки, зберігає файли як об'єкти S3 і тримає локальний кеш. DataSync переносить дані, а не дає постійний доступ; Transfer Family — для SFTP/FTP; Volume Gateway — блочні iSCSI-диски.

ПИТАННЯ 2

Резервне копіювання в компанії робиться наявним ПЗ на фізичні стрічки, які щотижня вивозять. Потрібно позбутися стрічок, не змінюючи бекап-ПЗ і процеси. Що обрати?

- A** AWS Backup
- B** DataSync з розкладом
- C** Tape Gateway з архівуванням у Glacier Deep Archive
- D** S3 File Gateway

Правильна відповідь: C. Tape Gateway емулює стрічкову бібліотеку для наявного ПЗ, а віртуальні стрічки зберігає в S3 і архівує в Glacier/Deep Archive. ПЗ «думає», що пише на стрічки.

ПИТАННЯ 3

Потрібно перенести 60 ТБ з локального NAS в Amazon EFS. Канал — 1 Gbps, строк — місяць, потрібна перевірка цілісності та збереження прав доступу. Що обрати?

- A** Скрипт rsync на EC2
- B** AWS Transfer Family
- C** AWS Snowball Edge
- D** AWS DataSync з агентом в офісі

Правильна відповідь: D. 60 ТБ через 1 Gbps — близько тижня, що вкладається в строк. DataSync переносить мережею з перевіркою цілісності й збереженням метаданих. Скрипт — більше роботи і менше гарантій.

ПИТАННЯ 4

Компанія має 400 ТБ даних, інтернет-канал 100 Mbps і лише тиждень на перенесення в S3. Як перенести дані (для клієнта, що вже користувався Snow)?

- A** Site-to-Site VPN
- B** S3 Transfer Acceleration
- C** DataSync через наявний канал
- D** Фізичні пристрої AWS Snowball Edge

Правильна відповідь: D. 400 ТБ через 100 Mbps — понад рік. Лише фізичне перенесення вкладається в строк. Для нових клієнтів AWS пропонує Data Transfer Terminal або партнерів, бо Snowball Edge з листопада 2025 року їм недоступний.

ПИТАННЯ 5

Постачальники роками надсилають файли на FTP-сервер компанії по SFTP і не хочуть нічого змінювати. Компанія переносить сервер в AWS і хоче зберігати файли в S3 без керування серверами. Що обрати?

- A** AWS Transfer Family з SFTP-endpoint і даними в S3
- B** Amazon S3 static website hosting
- C** EC2 з самостійно налаштованим SFTP-сервером
- D** S3 File Gateway

Правильна відповідь: A. Transfer Family — керований SFTP/FTPS/FTP/AS2-сервер, що пише прямо в S3 чи EFS. Партнери змінюють лише адресу; сервери підтримувати не треба.

ПИТАННЯ 6

Компанія використовує iSCSI-диски в офісі. Усі дані мають бути доступні локально з мінімальною затримкою, а в AWS потрібна асинхронна резервна копія, яку можна відновити як том EBS. Що обрати?

- A** Volume Gateway (Stored)
- B** Volume Gateway (Cached)
- C** S3 File Gateway
- D** Tape Gateway

Правильна відповідь: A. Stored-режим тримає всі дані локально і асинхронно зберігає знімки в AWS (їх можна відновити як EBS). Cached тримає основні дані в S3, а локально — лише кеш.

ПИТАННЯ 7

Які твердження про AWS DataSync правильні? (Оберіть 2)

Оберіть 2 відповіді

- ☐ А Може переносити дані за розкладом і з обмеженням пропускної здатності
- ☐ В Потребує фізичного пристрою для перенесення
- ☐ С Працює лише між сервісами AWS
- ☐ D Надає постійний NFS-доступ до S3 для офісних застосунків
- ☐ Е Перевіряє цілісність перенесених даних

Правильна відповідь: А, Е. DataSync — онлайн-перенесення з перевіркою цілісності, розкладом і обмеженням швидкості; з агентом працює з локальними і сторонніми сховищами. Постійний доступ з кешем — це Storage Gateway.



Черги й розсилки — Amazon SQS і Amazon SNS



Відмовостійкість · 26%



Продуктивність · 24%



Історія Хмаринки. Під час розпродажу стався ланцюговий збій: сайт після кожного замовлення **синхронно** викликав склад, бухгалтерію, сервіс листів і аналітику. Сервіс листів «подумав» 20 секунд — і покупці бачили вічне коло завантаження, а частина замовлень загубилась, бо склад якраз перезапускався. Тарас формулює нове правило: **сайт приймає замовлення і відразу відповідає «дякуємо», а решту системи дізнається про замовлення через черги і розсилки** — кожна у своєму темпі, нічого не губиться. Це розв'язування (decoupling), і на ньому тримається чверть питань про відмовостійкість.



Образ. SQS — поштова скринька з квитками замовлень на кухні ресторану: офіціант кидає квиток і повертається до гостей, кухарі беруть квитки, коли звільняться; якщо кухар захворів — квитки чекають, а не зникають. **SNS** — гучномовець: оголошення «нове замовлення №1042!» одночасно чують склад, бухгалтерія і відділ листів. Найнадійніша схема — **гучномовець, під яким у кожного відділу своя скринька (fan-out)**: оголошення не пропаде, навіть якщо відділ саме на обіді.

Навіщо розв'язувати компоненти

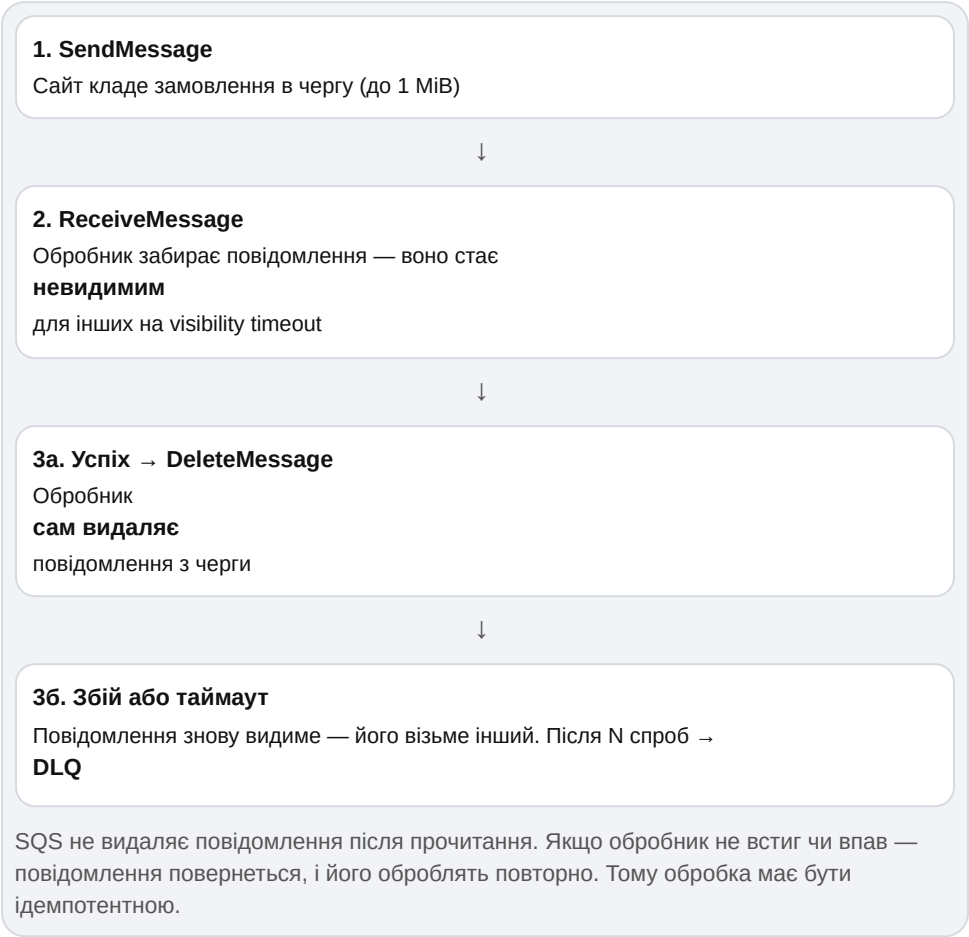
Коли компоненти викликають одне одного **синхронно**, вони зчеплені: повільний або «впалий» учасник тягне за собою всіх. **Асинхронна** взаємодія через посередника дає:

- **стійкість** — якщо обробник недоступний, повідомлення чекають у черзі;
- **буфер для піків** — сайт приймає 5 000 замовлень за хвилину, а склад обробляє 500 за хвилину, без втрат;
- **незалежне масштабування** — більше обробників, коли черга росте;
- **незалежні зміни** — можна переписати склад, не чіпаючи сайт.

Amazon SQS: черга

Amazon Simple Queue Service — керована черга повідомлень, **pull-модель**: обробники (consumers) **самі забирають** повідомлення. Serverless, масштабується майже безмежно.

Життя одного повідомлення



Ключові параметри:

Параметр	Значення	Навіщо
Розмір повідомлення	до 1 MiB (з серпня 2025; раніше 256 KB)	Більше — Extended Client Library : тіло в S3, у черзі — посилання (до 2 GB)
Зберігання	від 1 хвилини до 14 днів (за замовчуванням 4 дні)	Скільки повідомлення чекатиме обробника

Параметр	Значення	Навіщо
Visibility timeout	за замовчуванням 30 с , до 12 годин	Скільки повідомлення невидиме після отримання. Має бути довшим за час обробки
Long polling	<code>WaitTimeSeconds</code> до 20 с	Чекати повідомлень замість порожніх відповідей — менше запитів, дешевше
Delay queue	до 15 хвилин	Відкласти доставку нових повідомлень
DLQ + <code>maxReceiveCount</code>	напр., 5 спроб	«Отруйні» повідомлення, що постійно падають, переносяться в окрему чергу для аналізу

Visibility timeout — найчастіша пастка. Якщо обробка триває 2 хвилини, а visibility timeout — 30 секунд, через 30 секунд повідомлення знову стане видимим, його візьме інший обробник — і замовлення оброблять **двічі**.
Рішення: **збільшити visibility timeout** (або обробник продовжує його викликом `ChangeMessageVisibility`). Для Lambda-обробника правило: **visibility timeout черги $\geq 6 \times \text{timeout функції}$** .

Standard чи FIFO

	Standard	FIFO
Пропускна здатність	Майже необмежена	300 викликів/с (до 3 000 повідомлень/с з пакетами по 10); high-throughput mode — у рази більше
Доставка	At-least-once : можливі дублікати	Exactly-once processing : дедуплікація за 5 хв
Порядок	Не гарантовано (best-effort)	Суворий порядок у межах MessageGroupId
Назва	будь-яка	закінчується на .fifo
Коли	Більшість задач, де порядок і рідкісні дублікати не критичні (обробка фото, листи)	Фінансові операції, зміни стану замовлення, де важливі порядок і без дублікатів

MessageGroupId у FIFO — «канал порядку»: повідомлення однієї групи (наприклад, одного замовлення чи клієнта) обробляються строго по черзі, а різні групи — паралельно. Так і порядок збережено, і масштабування є.

Масштабування обробників

Обробники черги — функції **Lambda** (event source mapping, розділ 25), контейнери чи сервери в **Auto Scaling** за метрикою **backlog per instance** (розділ 23). Черга росте — обробників більшає.

Безпека SQS

Шифрування SSE-SQS або SSE-KMS; доступ — IAM-політики і **access policy черги** (resource-based). Щоб **SNS** чи **S3** могли класти повідомлення в чергу, **політика черги** має дозволити їм `sqs:SendMessage` (з умовою на ARN теми чи bucket). Приватний доступ з VPC — interface endpoint.

Amazon SNS: розсилка

Amazon Simple Notification Service — **pub/sub, push-модель**: видавець публікує повідомлення в **тему (topic)**, а SNS **сам доставляє** копію кожному підписнику:

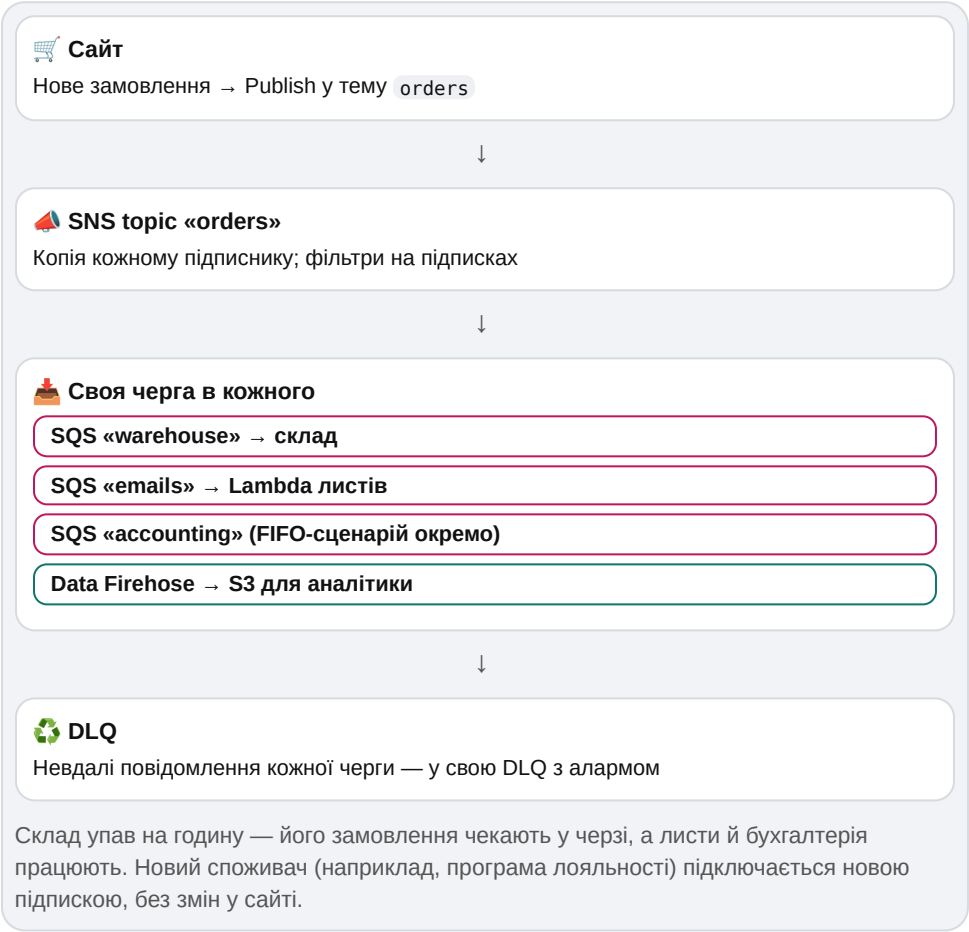
- **Amazon SQS** (черги), **AWS Lambda**, **HTTP/HTTPS-endpoints**, **Amazon Data Firehose**;
- **email, SMS, мобільні push**-сповіщення (для людей).

Властивості:

- SNS **не зберігає** повідомлення: якщо підписник недоступний, SNS повторює доставку за політикою повторів, а невдалі можна скинути в DLQ підписки. Для надійної обробки — **SQS за SNS**.
- **Message filtering** — у кожної підписки своя **filter policy**: підписник отримує лише потрібне (наприклад, лише замовлення з `country = PL`).
- **FIFO topics** → лише **SQS FIFO** підписники: порядок і дедуплікація.
- Розмір повідомлення — 256 KB (NEW з вересня 2026 — до 1 MiB за налаштуванням теми).

Fan-out: SNS + SQS

Класичний патерн: **одна подія — кілька незалежних обробників**, кожен зі своєю чергою.



Що обрати для обміну повідомленнями

Потреба	Сервіс
Черга між компонентами, обробники забирають задачі у своєму темпі	SQS Standard
Суворий порядок і без дублікатів	SQS FIFO (+ SNS FIFO для fan-out)
Одне повідомлення — багатьом одержувачам (push)	SNS
Надійний fan-out до кількох систем	SNS → кілька SQS
Маршрутизація подій за вмістом, події від SaaS, розклад	EventBridge (наступний розділ)
Потік подій у реальному часі з повторним читанням (replay)	Kinesis Data Streams (розділ 38)
Наявні застосунки на JMS, AMQP, MQTT	Amazon MQ (наступний розділ)



Спробуй у справжньому AWS: fan-out SNS → SQS з фільтром

Вартість: у межах постійного безкоштовного ліміту SQS і SNS. **Час:** 20 хвилин.

1. **SQS → Create queue:** тип **Standard**, назва `orders-warehouse` .
Створи ще одну: `orders-poland` .
2. **SNS → Topics → Create topic:** тип **Standard**, назва `orders` .
3. У черзі `orders-warehouse` натисни **Subscribe to Amazon SNS topic** → обери `orders` (консоль сама оновить політику черги, щоб SNS міг у неї писати). Те саме для `orders-poland` .
4. SNS → тема → **Subscriptions** → підписка черги `orders-poland` → **Edit** → **Subscription filter policy:**

```
{ "country": ["PL"] }
```

Зверни увагу: фільтр за замовчуванням дивиться на **атрибути повідомлення**.

1. **Publish message** у тему: тіло `{"order":1042}`, атрибут `country = UA` (String). Потім ще одне — з `country = PL`.
2. `SQS → orders-warehouse → Send and receive messages → Poll for messages`: обидва повідомлення. У `orders-poland` — лише польське.
3. Відкрий повідомлення, **не видаляючи** його, і повтори Poll через 5 секунд — його не видно (`visibility timeout 30 c`). Через 30 с воно з'явиться знову. Видали його кнопкою **Delete**.

Прибери за собою: видали тему SNS (разом з підписками) і обидві черги.

Запам'ятай

Розв'язування через черги: стійкість до збоїв, **буфер для піків**, незалежне масштабування.

SQS — pull-черга: повідомлення до **1 MiB**, зберігання **1 хв – 14 днів** (типово 4), **visibility timeout** 30 с – 12 год, **long polling** до 20 с, delay до 15 хв, **DLQ** після N спроб. Обробник **сам видаляє** повідомлення.

Standard — майже необмежено, at-least-once, порядок не гарантовано.
FIFO — порядок у межах **MessageGroupId**, exactly-once, 300/3 000 за секунду (є high-throughput).

Повторна обробка → **збільшити visibility timeout**; $\text{Lambda: } \geq 6 \times \text{timeout функції}$.

SNS — push pub/sub: SQS, Lambda, HTTP(S), Firehose, email, SMS, push; **message filtering**; не зберігає повідомлення.

Fan-out: SNS → **кілька SQS** — надійна доставка кільком незалежним системам.

Щоб SNS/S3 писали в чергу — **access policy черги**.

Як питають на іспиті

Розв'язати компоненти, замовлення не повинні губитися під час піків → **Amazon SQS**

Обробка суворо по порядку і без дублікатів → **SQS FIFO (MessageGroupId)**

Одне повідомлення мають обробити кілька незалежних систем → **SNS fan-out** → **кілька SQS**

Повідомлення обробляються двічі, бо обробка довша за 30 секунд → **збільшити visibility timeout**

Повідомлення, що постійно завершуються помилкою, блокують обробку → **dead-letter queue (maxReceiveCount)**

Забгато порожніх ReceiveMessage і зайві витрати → **long polling (WaitTimeSeconds до 20 с)**

Повідомлення більші за 1 MiB → **SQS Extended Client Library + S3**

Різні підписники мають отримувати лише свої типи повідомлень → **SNS message filtering**

Масштабувати воркерів на EC2 за довжиною черги → **Auto Scaling за backlog per instance**

SNS не може доставити повідомлення в чергу SQS → **дозволити sqs:SendMessage для теми в access policy черги**

Відкласти обробку нових повідомлень на 10 хвилин → **delay queue**

Пастки

SQS **не** видаляє повідомлення автоматично після читання.

SQS Standard може доставити **двічі і не по порядку** — обробка має бути ідемпотентною.

SNS **не** зберігає повідомлення — для надійності ставлять SQS за SNS.

FIFO має обмеження пропускнуої здатності; «сотні тисяч повідомлень на секунду без вимог до порядку» — Standard.

Зберігання в SQS — максимум **14 днів**, а не «назавжди».

✓ Перевір себе

ПИТАННЯ 1

Під час розпродажів сервіс обробки замовлень не встигає, і частина замовлень губиться. Потрібно, щоб усі замовлення зберігалися, а обробка йшла у своєму темпі. Що обрати?

- A** Amazon SNS без підписників-черг
- B** Збільшити таймаут HTTP на сайті
- C** Amazon SQS між сайтом і сервісом обробки
- D** Синхронний виклик з повторами на боці сайту

Правильна відповідь: C. Черга SQS приймає повідомлення незалежно від стану обробника і зберігає їх до 14 днів. Обробник забирає їх у своєму темпі. SNS не зберігає повідомлення.

ПИТАННЯ 2

Обробник черги SQS витрачає до 3 хвилин на кожне повідомлення. Частина замовлень обробляється двічі. Що змінити?

- A** Збільшити visibility timeout черги до значення, більшого за час обробки
- B** Перейти на delay queue
- C** Зменшити період зберігання повідомлень
- D** Увімкнути long polling

Правильна відповідь: A. Поки повідомлення обробляється, воно має лишатися невидимим. Якщо visibility timeout (30 с за замовчуванням) коротший за обробку, повідомлення знову стає видимим, і його бере інший обробник.

ПИТАННЯ 3

Банківські транзакції для кожного рахунку мають оброблятися строго в порядку надходження і без дублікатів, але різні рахунки можна обробляти паралельно. Що обрати?

- A** SQS Standard з delay queue
- B** SNS Standard topic
- C** SQS Standard з кількома обробниками
- D** SQS FIFO з MessageGroupId = номер рахунку

Правильна відповідь: D. FIFO гарантує порядок у межах групи повідомлень і обробку без дублікатів. Групування за рахунком дає паралельність між рахунками.

ПИТАННЯ 4

Кожне нове замовлення має незалежно обробити склад, сервіс листів і аналітика. Якщо один з них тимчасово недоступний, його повідомлення не повинні губитися. Яка архітектура?

- A** SNS topic з трьома підписками — по черзі SQS для кожного споживача
- B** SNS topic з підписками email
- C** Одна черга SQS, яку читають усі три споживачі
- D** Три окремі виклики API з сайту

Правильна відповідь: A. Fan-out SNS → SQS дає кожному споживачу власну копію повідомлення в надійній черзі. Одна спільна черга не підходить: повідомлення отримає лише один споживач.

ПИТАННЯ 5

Повідомлення з некоректними даними постійно завершуються помилкою і повертаються в чергу, заважаючи обробці. Як це розв'язати?

- A** Збільшити період зберігання до 14 днів
- B** Перейти на SNS
- C** Налаштувати dead-letter queue з maxReceiveCount, наприклад 5
- D** Увімкнути long polling

Правильна відповідь: C. Після вказаної кількості невдалих отримань SQS переносить повідомлення в DLQ — його можна проаналізувати окремо, а основна обробка не блокується.

ПИТАННЯ 6

Споживачі черги постійно виконують ReceiveMessage і отримують порожні відповіді, що збільшує кількість запитів і витрати. Що зробити?

- A** Зменшити visibility timeout
- B** Перейти на FIFO-чергу
- C** Збільшити розмір повідомлень
- D** Увімкнути long polling (WaitTimeSeconds до 20 секунд)

Правильна відповідь: D. Long polling чекає, доки з'являться повідомлення (до 20 секунд), замість миттєвих порожніх відповідей — менше запитів і нижча вартість.

ПИТАННЯ 7

Які твердження про SQS правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** Standard-черга може доставити повідомлення більше одного разу
- B** FIFO-черга підтримує необмежену пропускну здатність без будь-яких лімітів
- C** SQS автоматично видаляє повідомлення одразу після його отримання
- D** SNS може писати в чергу без дозволу в політиці черги
- E** Повідомлення зберігаються до 14 днів

Правильна відповідь: A, E. Зберігання — до 14 днів; Standard — at-least-once (можливі дублікати). Видаляє повідомлення обробник, у FIFO є ліміти пропускну здатності, а SNS потребує дозволу в access policy черги.



Події й оркестрація — EventBridge, Step Functions, Amazon MQ, AppFlow



Відмовостійкість · 26%



Продуктивність · 24%



Історія Хмаринки. Процес замовлення розрісся: зарезервувати товар на складі → списати оплату → якщо сума понад 10 000 грн — дочекатися підтвердження менеджера → створити відправлення в кур'єрській службі → повідомити покупця. А якщо кур'єрка недоступна — повторити тричі; якщо оплата не пройшла — зняти резерв. Тарас написав це в одній функції з сотнею `if` і `try/catch`, і ніхто, крім нього, не розуміє, що там відбувається. Ще задачі: платіжний провайдер шле **події**, склад живе на старому брокері повідомлень **ActiveMQ**, а маркетинг хоче щоночі вивантажувати дані з CRM. Сьогодні — інструменти, що перетворюють хаос викликів на зрозумілі **події й робочі процеси**.



Образ. EventBridge — диспетчерська таксі: події (виклики) надходять від будь-кого, а диспетчер за правилами («аеропорт — бізнес-клас», «з дитиною — з кріслом») направляє кожну потрібній машині. **Step Functions** — диригент оркестру з партитурою: кожен музикант (сервіс) грає свою партію, диригент знає порядок, паузи, повторі і що робити, якщо скрипаль збився. **Amazon MQ** — старий знайомий поштар, що говорить мовою давніх програм (JMS), тож їх не треба переписувати. **AppFlow** — кур'єр між SaaS-системами і сховищем даних.

Amazon EventBridge: шина подій

Подія — факт, що щось сталося: «замовлення оплачено», «файл завантажено», «сервер зупинено». **Amazon EventBridge** (колишня назва — CloudWatch Events) — serverless **шина подій**, що приймає події і **маршрутизує** їх за правилами:

- **Джерела: сервіси AWS** (зміна стану EC2, знахідка GuardDuty, API-виклик з CloudTrail — у **default** шину), **твої застосунки** (PutEvents у **custom** шину), **SaaS-партнери** (Zendesk, Datadog, Shopify, Auth0 та інші — у **partner** шину).
- **Правила з event pattern** — фільтр за вмістом події (джерело, тип, будь-які поля, префікси, числові діапазони, «існує/не існує»).
- **Цілі правил**: Lambda, SQS, SNS, Step Functions, Kinesis, Data Firehose, задача ECS, SSM Automation, CloudWatch Logs, інша шина (зокрема в **іншому акаунті чи регіоні**), **API destinations** — HTTP-endpoint будь-якого зовнішнього сервісу з автентифікацією.

Приклад правила «успішна оплата понад 10 000 грн»:

```
{
  "source": ["khmarynka.payments"],
  "detail-type": ["PaymentSucceeded"],
  "detail": { "amount": [{ "numeric": [ ">", 10000 ] } ] }
}
```

Додаткові можливості:

- **EventBridge Scheduler** — serverless-планувальник: разові й повторювані запуски (cron, rate) з часовими поясами, повторами і DLQ; мільйони розкладів (наприклад, «нагадати про кинутий кошик через 24 години» — окремий розклад для кожного покупця).
- **Archive & replay** — зберігати події й «програти» їх знову (після виправлення помилки в обробнику).
- **Schema registry** — опис структури подій, генерація коду.

- **EventBridge Pipes** — з'єднання «точка-точка»: джерело (SQS, Kinesis, DynamoDB Streams, MSK, MQ) → **фільтр** → **збагачення** (Lambda, Step Functions, API) → **ціль**. Без «клеювого» коду.



EventBridge чи SNS?

SNS — гучномовець: швидка масова розсилка копій підписникам (зокрема SMS, email, push). **EventBridge** — розумний диспетчер: **фільтри за вмістом**, події **AWS і SaaS**, **розклад**, архів і повтор, цілі в інших акаунтах. На іспиті «реагувати на подію від SaaS-партнера», «маршрутизувати за полями події», «запуск за розкладом» → EventBridge.

AWS Step Functions: оркестрація

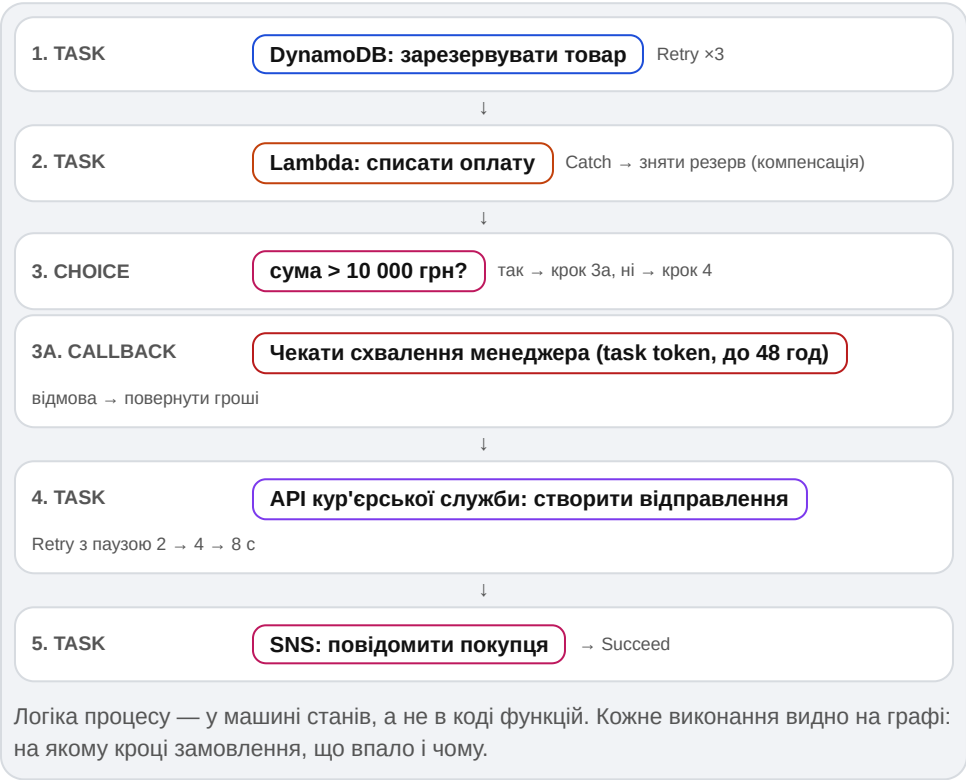
AWS Step Functions — serverless-**оркестратор**: процес описується як **машина станів** (state machine) — у JSON (Amazon States Language) або у візуальному редакторі Workflow Studio. Кожен крок видно на графі, історія кожного виконання зберігається.

Типи станів:

Стан	Що робить
Task	Виконати роботу: Lambda, пряма інтеграція з понад 200 сервісами (DynamoDB, SQS, SNS, ECS, Glue, Batch, SageMaker, Bedrock...) без жодного коду
Choice	Розгалуження за умовою («сума > 10 000?»)
Parallel	Кілька гілок одночасно
Map	Той самий крок для кожного елемента списку; Distributed Map — масова паралельна обробка мільйонів об'єктів S3
Wait	Пауза: секунди або до певної дати
Pass / Succeed / Fail	Службові: передати дані, успіх, помилка

Обробка помилок вбудована: **Retry** (повторити N разів з експоненційною паузою) і **Catch** (перейти до «запасного» кроку). На цьому будується патерн **Saga**: якщо крок 4 провалився, виконуються **компенсувальні** дії для кроків 1–3 (повернути гроші, зняти резерв).

Людське схвалення та зовнішні системи — патерн **callback з task token**: Step Functions надсилає токен (наприклад, у лист менеджеру з посиланням) і **чекає** — хвилини чи дні, — доки хтось не поверне токен з рішенням. Патерн **.sync** — запустити довгу задачу (Batch, ECS, Glue) і дочекатися її завершення.



Standard чи Express

	Standard	Express
Тривалість	до 1 року	до 5 хвилин

	Standard	Express
Гарантія виконання	Exactly-once	At-least-once (асинхронні) / at-most-once (синхронні)
Історія	Повна історія кожного виконання в консолі	Журнали в CloudWatch Logs
Обсяги	Помірні	Сотні тисяч виконань за секунду
Оплата	За переходи між станами	За кількість і тривалість (дешевше для великих обсягів)
Для чого	Довгі бізнес-процеси, людське схвалення, оркестрація ETL і ML	Масова обробка подій, IoT, короткі мікросервісні ланцюжки

Amazon MQ: для наявних брокерів

Amazon MQ — керовані брокери **Apache ActiveMQ** і **RabbitMQ**. Вони говорять стандартними протоколами **JMS, AMQP, MQTT, STOMP, OpenWire, WebSocket**. Навіщо, якщо є SQS і SNS? Щоб **перенести наявні застосунки без переписування**: складська програма Хмаринки роками працює з ActiveMQ — її достатньо перенаправити на Amazon MQ.

Особливості: працює на інстансах брокера у **VPC** (не serverless), висока доступність — **active/standby у двох AZ** (ActiveMQ) або кластер з трьох вузлів (RabbitMQ). Для **нових** застосунків AWS радить SQS/SNS/EventBridge — вони масштабуються без меж і не потребують керування.

Amazon AppFlow: дані з SaaS

Amazon AppFlow — керовані потоки даних між **SaaS-системами** (Salesforce, SAP, ServiceNow, Zendesk, Slack, Google Analytics та інші) і сервісами AWS (S3, Redshift) — **без коду**: за розкладом, за подією чи вручну, з фільтрацією, маскуванням полів і шифруванням; для деяких

джерел — приватно через PrivateLink. Маркетинг Хмаринки щоночі отримує дані CRM у S3 для аналітики.

Що обрати для інтеграції

Потреба	Сервіс
Черга між компонентами	SQS (розділ 31)
Розсилка кільком підписникам, SMS, email, push	SNS
Маршрутизація подій за вмістом, події AWS і SaaS, цілі в інших акаунтах	EventBridge
Запуск за розкладом (cron) без серверів	EventBridge Scheduler
Багатокроковий процес з розгалуженнями, повторами, компенсаціями, людським схваленням	Step Functions Standard
Масові короткі робочі процеси	Step Functions Express
Джерело → фільтр → збагачення → ціль без коду	EventBridge Pipes
Перенести застосунок на JMS/AMQP/MQTT без переписування	Amazon MQ
Регулярно вивантажувати дані з Salesforce чи іншої SaaS у S3 без коду	Amazon AppFlow



Спробуй у справжньому AWS: машина станів з розгалуженням і очікуванням

Вартість: у межах безкоштовного ліміту Step Functions (тисячі переходів на місяць). **Час:** 20 хвилин.

1. **Step Functions** → **Create state machine** → **Blank** → переключись у режим **Code** і встав:

```
{
  "Comment": "Замовлення Хмаринки (навчальне)",
  "StartAt": "Перевірка суми",
  "States": {
    "Перевірка суми": { "Type": "Choice",
      "Choices": [{ "Variable": "$.amount", "NumericGreaterThan": 1000000 },
      "Default": "Оформлено" },
    "Очікування менеджера": { "Type": "Wait", "Seconds": 10, "Next": "Оформлено" },
    "Оформлено": { "Type": "Pass", "Result": { "status": "confirmed" } }
  }
}
```

1. Тип — **Standard**. Назва `khmarynka-order-lab`. Create (дозволь консолі створити роль).
2. **Start execution** з вводом `{ "amount": 500 }` — подивись на графі: одразу «Оформлено».
3. Ще раз з `{ "amount": 150000 }` — виконання пройде через **Wait** 10 секунд. Відкрий вкладку подій (Events) — кожен перехід з часом і даними.
4. Для порівняння відкрий **EventBridge** → **Scheduler** → **Create schedule** і подивись варіанти: одноразовий, cron, часовий пояс, ціль — наприклад, твоя state machine (не створюй або видали після).

Прибери за собою: видали state machine і її IAM-роль.

Запам'ятай

EventBridge: шини **default** (AWS), **custom** (свої), **partner** (SaaS); **правила** з фільтрами за вмістом; цілі — Lambda, SQS, SNS, Step Functions, ECS, **API destinations**, інші шини й акаунти. **Scheduler**, **archive & replay**, **schema registry**, **Pipes**.

Step Functions: машина станів — Task, Choice, Parallel, **Map / Distributed Map**, Wait; **Retry/Catch**; **Saga** з компенсаціями; **callback з task token** для людського схвалення; інтеграції з 200+ сервісами без Lambda.

Standard — до **1 року**, exactly-once, повна історія; **Express** — до **5 хвилин**, величезні обсяги, дешевше.

Amazon MQ — ActiveMQ/RabbitMQ для наявних застосунків на **JMS, AMQP, MQTT**; не serverless.

AppFlow — SaaS ↔ S3/Redshift без коду.

Як питають на іспиті

Реагувати на подію від SaaS-партнера (Zendesk, Datadog, Shopify) → **EventBridge partner event bus**

Маршрутизувати події до різних цілей за полями події → **EventBridge rules з event patterns**

Запускати функцію щоночі о 2:00 без серверів → **EventBridge Scheduler (cron)**

Реагувати на конкретний API-виклик (зміну Security Group) → **EventBridge rule на подію CloudTrail** → **Lambda / SNS**

Надіслати події з акаунта в центральний акаунт безпеки → **EventBridge cross-account event bus**

Багатокроковий процес з повторами, обробкою помилок і ручним схваленням → **Step Functions Standard (callback з task token)**

Мільйони коротких робочих процесів за секунду → **Step Functions Express**

Обробити паралельно мільйони файлів у S3 → **Step Functions Distributed Map**

Скасувати попередні кроки, якщо наступний провалився (розподілена транзакція) → **Saga-патерн у Step Functions (Catch + компенсації)**

Мігрувати застосунок на ActiveMQ чи RabbitMQ без переписування → [Amazon MQ](#)

Щоночі вивантажувати дані з Salesforce в S3 без коду → [Amazon AppFlow](#)

Після виправлення помилки в обробнику повторно обробити події за минулий тиждень → [EventBridge archive & replay](#)

Пастки

Step Functions Express — максимум **5 хвилин**; довгі процеси й очікування схвалення — лише **Standard**.

Amazon MQ — **не** для нових serverless-систем і не «замінник SQS» у питаннях про масштаб без керування.

EventBridge — не черга: для надійної обробки з буфером і повторами став **SQS** ціллю правила.

Писати оркестрацію в одній великій Lambda — антипатерн: 15 хвилин, немає історії і візуалізації.

✓ Перевір себе

ПИТАННЯ 1

Процес оформлення замовлення має 6 кроків з повторами при збоях, розгалуженням за сумою і очікуванням ручного схвалення менеджера до 2 днів. Яке рішення з найменшими операційними зусиллями?

- A** Черга SQS з delay queue на 2 дні
- B** AWS Step Functions Standard workflow з callback-патерном (task token) для схвалення
- C** Step Functions Express workflow
- D** Одна функція Lambda з циклом очікування

Правильна відповідь: B. Standard workflow триває до року, має вбудовані Retry/Catch і підтримує очікування зовнішньої відповіді через task token. Lambda обмежена 15 хвилинами, Express — 5 хвилинами, delay queue — максимум 15 хвилин.

ПИТАННЯ 2

Компанія використовує SaaS для підтримки клієнтів (партнер EventBridge). Коли створюється білет з пріоритетом «high», потрібно запускати функцію Lambda. Що налаштувати?

- A** Опитувати API SaaS щохвилини з сервера EC2
- B** Amazon MQ з чергою для SaaS
- C** SNS topic з підпискою SaaS
- D** Partner event bus в EventBridge і правило з фільтром за пріоритетом, ціль — Lambda

Правильна відповідь: D. EventBridge приймає події SaaS-партнерів у partner event bus, а правило з event pattern фільтрує потрібні й передає цілі. Жодного опитування.

ПИТАННЯ 3

Щоночі о 03:00 за київським часом потрібно запускати задачу очищення, що триває 3 хвилини. Яке рішення без серверів?

- A** EventBridge Scheduler з cron-виразом і часовим поясом Europe/Kyiv, ціль — функція Lambda
- B** Step Functions Wait на 24 години в нескінченному циклі
- C** SQS delay queue
- D** Cron на окремому сервері EC2

Правильна відповідь: A. EventBridge Scheduler запускає цілі за розкладом з урахуванням часового поясу, без серверів. Сервер з cron — зайва інфраструктура.

ПИТАННЯ 4

Застосунок складу працює з брокером Apache ActiveMQ через JMS. Компанія хоче перенести брокер в AWS з мінімальними змінами коду. Що обрати?

- A** Amazon Kinesis Data Streams
- B** Amazon SNS
- C** Amazon SQS
- D** Amazon MQ для ActiveMQ

Правильна відповідь: D. Amazon MQ — керований ActiveMQ/RabbitMQ з підтримкою JMS, AMQP, MQTT, STOMP, OpenWire. Перехід на SQS вимагав би переписати застосунок.

ПИТАННЯ 5

Потрібно обробити мільйони IoT-подій за секунду короткими робочими процесами з трьох кроків (кожен — мілісекунди). Що найекономніше?

- A** Amazon MQ
- B** Step Functions Express
- C** Step Functions Standard
- D** AWS Batch

Правильна відповідь: B. Express workflows розраховані на дуже великі обсяги коротких виконань (до 5 хвилин) і тарифікуються за кількістю і тривалістю — дешевше за Standard для масових подій.

ПИТАННЯ 6

Служба безпеки хоче отримувати сповіщення, щойно хтось змінює правила Security Group у будь-якому акаунті. Що налаштувати?

- A** VPC Flow Logs
- B** AWS Config з щотижневим звітом
- C** Правило EventBridge на подію AuthorizeSecurityGroupIngress з CloudTrail, ціль — SNS
- D** Amazon Inspector

Правильна відповідь: C. CloudTrail записує API-виклики, а EventBridge отримує ці події і за правилом одразу надсилає сповіщення. Config фіксує стан, але не миттєво сповіщає без додаткових налаштувань.

ПИТАННЯ 7

Які твердження про EventBridge правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** Правило може фільтрувати події за значеннями полів у тілі події
- B** EventBridge гарантує зберігання подій 14 днів, як SQS
- C** EventBridge підтримує лише події сервісів AWS
- D** Подію можна надіслати в event bus іншого акаунта
- E** EventBridge замінює потребу в DLQ для обробників

Правильна відповідь: A, D. Event patterns фільтрують за вмістом, а шини можуть обмінюватися подіями між акаунтами й регіонами. Для власних і SaaS-подій є custom і partner шини; зберігання — через archive, а надійну обробку часто будують зі SQS і DLQ.



Переїзд у хмару — стратегії 7R, MGN, DMS, SCT і план міграції



Відмовостійкість · 26%



Вартість · 20%



Історія Хмаринки. Олена ставить дедлайн: «До наступного сезону розпродажів у підсобці не має лишитися жодного сервера». А там — сайт з базою MySQL, NAS з фото, складська програма на Windows з ліцензією «на ядро», файлова шафа бухгалтерії, стара CRM, якою ніхто не користується, і сервер звітів, про який ніхто не пам'ятає, навіщо він. Переносити все «як є»? Переписати все з нуля? Правильна відповідь — **для кожної системи свою стратегію**, план хвиль і перевірений інструмент. Це і є міграція — одна з найпрактичніших тем для архітектора.



Образ. Переїзд у новий будинок. Щось **викидаєш** (retire), щось поки **лишаєш** у старій квартирі (retain), щось **перевозиш як є** (rehost), щось перевозиш **разом з шафою** (relocate), щось перевозиш і **трохи ремонтуєш** (replatform), щось **купуєш нове** замість старого (repurchase), а щось **перебудовуєш** під новий будинок (refactor). Переїзд ніхто не робить за один вечір: спершу оцінка, потім хвили, в кожній — пробне перевезення і лише потім остаточне.

Три фази міграції

- 1. **Assess (оцінка)** — що є, скільки коштує зараз, скільки коштуватиме в AWS, навіщо переїжджати (бізнес-обґрунтування). Інструменти: інвентаризація серверів і залежностей, **Migration Evaluator** (оцінка вартості і правильних розмірів у AWS).
- 2. **Mobilize (підготовка)** — **landing zone** (акаунти, мережа, безпека — Control Tower, розділ 10), навчання команди, детальний план: яка система, за якою стратегією, в якій хвилі.
- 3. **Migrate & Modernize** — перенесення хвилями, тестування, перемикання (cutover), оптимізація після переїзду.

Сім стратегій: 7R

Стратегія	Суть	Приклад	Інструменти AWS
Retire	Вимкнути непотрібне	Сервер звітів, яким не користуються	— (економія одразу)
Retain	Поки залишити як є	Система, яку міняють через рік; жорсткі вимоги	Гібрид: VPN/DX, Storage Gateway
Rehost («lift and shift»)	Перенести сервер як є на EC2	Складська програма без змін коду	AWS Application Migration Service (MGN)
Relocate	Перенести гіпервізор цілими віртуальними машинами	Сотні VM на VMware без змін інструментів	VMware Cloud on AWS, Amazon Elastic VMware Service
Replatform («lift, tinker and shift»)	Невеликі оптимізації при переносі	MySQL → Amazon RDS / Aurora ; застосунок → контейнери чи Beanstalk	DMS, ECS, Beanstalk
Repurchase	Перейти на готовий SaaS	Стара CRM → хмарна CRM	AppFlow (для даних)

Стратегія	Суть	Приклад	Інструменти AWS
Refactor / Re-architect	Переписати під cloud-native	Моноліт → мікросервіси, serverless, події	Lambda, Step Functions, DynamoDB...

Як обирати: **rehost** — найшвидший і найдешевший шлях переїзду (часто перший крок, а оптимізація — потім); **replatform** — розумний компроміс (керовані бази без переписування); **refactor** — найбільша вигода в довгостроковій перспективі, але найдовше і найдорожче; **retire** — найдешевша міграція з усіх.

AWS Application Migration Service (MGN)

MGN — основний інструмент **rehost**:

1. На вихідний сервер (фізичний, VMware, Hyper-V, інша хмара) ставиться **агент реплікації**.
2. Агент **безперервно реплікує диски на рівні блоків** у недорогу проміжну зону (staging area) в AWS.
3. **Тестовий запуск** — копія сервера стартує в AWS для перевірки, не зупиняючи оригінал.
4. **Cutover** — у погоджене вікно: остання синхронізація, запуск робочого сервера в AWS, перемикання DNS. Простій — хвилини.
5. MGN автоматично конвертує сервер для роботи на EC2 (драйвери, завантажувач) і може виконати дії після запуску (встановити агентів, налаштувати).

AWS DMS: міграція баз даних

AWS Database Migration Service переносить бази з **мінімальним простоєм**:

- **Full load** — копія всіх наявних даних;

- **CDC** (change data capture) — **безперервна реплікація змін**, поки стара база працює і приймає замовлення;
- у зручний момент — перемикання застосунку на нову базу (простій — хвилини).

Робоча частина — **replication instance** (або **DMS Serverless**), між ними — **source** і **target endpoints**. Цілі: RDS, Aurora, Redshift, DynamoDB, S3, Kinesis, OpenSearch, DocumentDB та інші. Є перевірка (validation) збіжності даних.



MySQL в офісі

Продовжує приймати замовлення



AWS DMS

- 1) full load — копія всього
- 2) CDC — кожна нова зміна
- 3) validation



Aurora MySQL

Відстає на секунди



Cutover (03:00)

Сайт у режим «лише читання» на кілька хвилин → останні зміни → рядок підключення на Aurora

MySQL → Aurora MySQL — гомогенна міграція: той самий рушій, конвертація схеми не потрібна.

Гомогенна чи гетерогенна

- **Гомогенна** — той самий рушій: MySQL → RDS/Aurora MySQL, PostgreSQL → Aurora PostgreSQL, Oracle → RDS for Oracle. Достатньо DMS (або нативних інструментів бази).
- **Гетерогенна** — різні рушії: **Oracle** → **Aurora PostgreSQL**, SQL Server → MySQL. Спершу треба **конвертувати схему і код** (таблиці, збережені процедури, функції) — **DMS Schema Conversion** (у консолі) або **AWS Schema Conversion Tool (SCT)** (настільний застосунок). Потім дані переносить DMS. SCT конвертує й схеми **сховищ даних** (Oracle, Teradata, Netezza → Redshift).
- Для SQL Server → PostgreSQL без переписування T-SQL є ще **Babelfish for Aurora PostgreSQL** (розділ 35).

Інші інструменти переїзду

- **Дані й файли:** **DataSync** (мережею), **Snowball / Data Transfer Terminal** (фізично), **Transfer Family**, **Storage Gateway** для гібридного періоду — розділ 30.
- **VMware:** **VMware Cloud on AWS** і **Amazon Elastic VMware Service** — ті самі інструменти vSphere, але в AWS (relocate).
- **AWS Elastic Disaster Recovery (DRS)** — та сама технологія безперервної реплікації, але для **аварійного відновлення**: сервери з офісу чи іншого регіону реплікуються в AWS з RPO в секунди і RTO в хвилини (розділ 42).
- **Мейнфрейми** — AWS Mainframe Modernization.
-  **AWS Transform** — сервіс з AI-агентами для масової модернізації (VMware, .NET, мейнфрейми). А старі **Migration Hub** і **Application Discovery Service** з листопада 2025 недоступні новим клієнтам — на іспиті їх назви ще можуть зустрітися.

План переїзду Хмаринки

Система	Стратегія	Як
Сервер звітів (ніхто не користується)	Retire	Вимкнути, архів даних — у S3 Glacier
Стара CRM	Repurchase	Хмарна CRM; історію даних — експортом
Фото на NAS (8 ТБ)	Перенесення даних	DataSync у S3 з нічною синхронізацією до cutover
База MySQL замовлень	Replatform	DMS (full load + CDC) → Aurora MySQL
Сайт (моноліт)	Replatform	Контейнерний образ → ECS на Fargate за ALB
Складська програма (Windows, ліцензія на ядра)	Rehost	MGN → EC2 на Dedicated Host (BYOL); пізніше — заміна
Файлова шафа бухгалтерії	Replatform	FSx for Windows + DataSync; AD — з довірою
Обробка замовлень (після переїзду)	Refactor	Step Functions + Lambda + SQS (розділи 31–32)

Хвилі: 1) landing zone, мережа, VPN; 2) retire і repurchase — швидкі перемоги; 3) дані (DataSync, DMS у режимі CDC); 4) сайт і база — cutover уночі з планом відкату (стара база ще тиждень отримує зміни у зворотному напрямку); 5) склад; 6) оптимізація витрат і рефакторинг.



Правила успішного cutover

- Кожна хвиля — з **тестовим запуском** і **планом відкату**.
- Понизь **TTL** DNS-записів за кілька днів до перемикання — щоб зміна поширилась за хвилини (розділ 17).
- Перемикай трафік поступово, якщо можна (**Route 53 weighted** 10% → 50% → 100%).

- Після переїзду — **правильний розмір** (Compute Optimizer), Savings Plans, вимкнення старого обладнання.

Спробуй: розплануй міграцію (вправа) і подивись на MGN і DMS

Вартість: безкоштовно — нічого не реплікуємо. **Час:** 20 хвилин.

1. **Вправа на папері.** Для кожної системи обери стратегію 7R: (а) сайт-візитка на WordPress, (б) Oracle-база ERP з тисячами збережених процедур, яку треба перевести на PostgreSQL, (в) 300 віртуальних машин VMware, які треба перенести за місяць без змін, (г) поштовий сервер Exchange, (д) застосунок, що має лишатися в офісі через закон. Відповіді: (а) replatform (Lightsail/контейнери) або repurchase; (б) replatform з **SCT/DMS Schema Conversion + DMS**; (в) **relocate** (VMware Cloud on AWS / Elastic VMware Service) або rehost через MGN; (г) repurchase (SaaS-пошта); (д) retain (+ гібрид чи Outposts).
2. Консоль → **Application Migration Service** → подивись розділи **Source servers, Replication template, Launch template** (не встановлюй агентів).
3. Консоль → **Database Migration Service** → подивись **Replication instances / Serverless replications, Endpoints, Database migration tasks** (тип задачі: Full load, Full load + CDC, CDC only) і **Schema conversion**.

Прибирати нічого не треба, якщо нічого не створював. (Якщо MGN попросить ініціалізацію — це безкоштовно, але можна пропустити.)

Запам'ятай

Фази: **Assess** → **Mobilize** → **Migrate & Modernize**; спершу — **landing zone**.

7R: Retire, Retain, **Rehost** (MGN), **Relocate** (VMware), **Replatform** (RDS, Aurora, контейнери), **Repurchase** (SaaS), **Refactor** (cloud-native).

MGN — безперервна блокова реплікація серверів, тестовий запуск, cutover за хвилини.

DMS — **full load + CDC**, мінімальний простій; **гомогенна** міграція — лише DMS; **гетерогенна** — спершу **SCT / DMS Schema Conversion**, потім DMS.

Дані — DataSync/Snowball/Transfer Family; DR серверів — **Elastic Disaster Recovery**.

Cutover: тест, план відкату, низький TTL, поступове перемикання.

Як питають на іспиті

Перенести сотні серверів «як є» в EC2 швидко і з мінімальним простоєм → **AWS Application Migration Service (MGN)**

Oracle → Aurora PostgreSQL з мінімальним простоєм → **SCT / DMS Schema Conversion (схема) + DMS (дані з CDC)**

MySQL on-prem → RDS MySQL з мінімальним простоєм → **DMS з full load + CDC (SCT не потрібен)**

Перенести дані сховища Teradata в Redshift → **SCT (схема) + DMS/агенти SCT (дані)**

Перенести віртуальні машини VMware без зміни інструментів → **VMware Cloud on AWS (relocate)**

Невелика оптимізація при переїзді: база переходить у керований сервіс → **Replatform (RDS / Aurora)**

Замінити власну CRM на хмарну → **Repurchase**

Вимкнути сервери, якими ніхто не користується → **Retire**

DR для серверів в офісі з RPO в секунди → **AWS Elastic Disaster Recovery**

Переписати моноліт на мікросервіси й serverless → [Refactor / Re-architect](#)

⚠ Пастки

SCT не потрібен для гомогенних міграцій (MySQL → MySQL).

DMS мігрує **дані**, а не схему для різних рушіїв — схему конвертує SCT/DMS Schema Conversion.

Rehost переносить і всі неефективності — плануй оптимізацію після переїзду.

Без плану відкату і тестового запуску cutover — лотерея.

✓ Перевір себе

ПИТАННЯ 1

Компанія має 200 серверів Linux і Windows у власному дата-центрі і хоче перенести їх в AWS за два місяці без змін у застосунках і з мінімальним простоем. Що обрати?

- ☐ А AWS Application Migration Service (MGN)
- ☐ В Переписати застосунки на Lambda
- ☐ С AWS DMS
- ☐ D AWS DataSync

Правильна відповідь: А. MGN безперервно реплікує диски серверів у AWS і дозволяє виконати cutover за хвилини — класичний rehost. DMS мігрує бази, DataSync — файли.

ПИТАННЯ 2

Потрібно перенести базу Oracle у Amazon Aurora PostgreSQL з мінімальним простоем. База містить багато збережених процедур. Які інструменти використати? (Оберіть 2)

Оберіть 2 відповіді

- A** AWS DMS з full load і CDC для перенесення даних
- B** AWS SCT або DMS Schema Conversion для конвертації схеми і коду
- C** AWS DataSync для файлів бази
- D** AWS MGN для сервера бази
- E** Amazon S3 Transfer Acceleration

Правильна відповідь: А, В. Гетерогенна міграція: спершу конвертуються схема і процедури (SCT/DMS Schema Conversion), потім DMS переносить дані з безперервною реплікацією змін. MGN перенесе сервер «як є», але не змінить рушій.

ПИТАННЯ 3

Компанія переносить MySQL з власного сервера в Amazon RDS for MySQL. Простій має бути мінімальним. Що потрібно?

- A** AWS DMS з full load і CDC; SCT не потрібен
- B** Експорт у CSV і завантаження вручну на вихідних
- C** AWS SCT для конвертації схеми, потім DMS
- D** AWS MGN

Правильна відповідь: A. Гомогенна міграція (той самий рушій) не потребує конвертації схеми. DMS копіює дані й реплікує зміни, поки стара база працює, тож перемикання займає хвилини.

ПИТАННЯ 4

Під час міграції компанія переносить базу з власного сервера в Amazon RDS, а застосунок — у контейнери, не переписуючи код. Як називається ця стратегія?

- A** Rehost
- B** Refactor
- C** Repurchase
- D** Replatform

Правильна відповідь: D. Replatform («lift, tinker and shift») — невеликі оптимізації при переносі, зокрема перехід на керовані сервіси без зміни архітектури застосунку.

ПИТАННЯ 5

Компанія має 500 віртуальних машин VMware vSphere і хоче перенести їх в AWS, зберігши наявні інструменти й процеси адміністраторів. Що обрати?

- A** VMware Cloud on AWS (або Amazon Elastic VMware Service)
- B** AWS Lambda
- C** AWS MGN для кожної VM з конвертацією в EC2
- D** Amazon EKS

Правильна відповідь: A. Relocate: гіпервізор VMware працює в AWS, віртуальні машини переносяться без конвертації, адміністратори працюють звичними інструментами vSphere.

ПИТАННЯ 6

Під час оцінки виявилось, що 15% серверів ніхто не використовує вже рік. Яка стратегія для них найвигідніша?

- A** Retain
- B** Relocate
- C** Rehost
- D** Retire

Правильна відповідь: D. Retire — вимкнути непотрібне. Це найдешевша «міграція»: жодних витрат на перенесення і зменшення рахунку одразу.

ПИТАННЯ 7

Під час DMS-міграції стара база продовжує приймати замовлення. Що гарантує, що нові замовлення потраплять у нову базу до перемикання?

- A** Multi-AZ у новій базі
- B** Validation у DMS
- C** Change data capture (CDC) — безперервна реплікація змін
- D** Full load, запущений повторно щогодини

Правильна відповідь: C. CDC відстежує зміни в джерелі й безперервно застосовує їх у цільовій базі, тож на момент cutover різниця — секунди.



Amazon RDS — реляційні бази без рутинного адміністрування



Відмовостійкість · 26%



Продуктивність · 24%



Безпека · 30%



Вартість · 20%



Історія Хмаринки. Стара база MySQL у підсобці — це Тарас о другій ночі: оновити MySQL, перевірити, чи відпрацював бекап, почистити місце на диску, перезапустити після зависання. Одного разу бекап не відпрацював тиждень — і ніхто не помітив. «Я розробник, а не нічний сторож бази», — каже Тарас. У хмарі базу даних можна **орендувати разом із сторожем**: Amazon RDS сам ставить патчі, робить бекапи, перемикається на резервну копію при аварії. Тобі лишаються дані, запити і правильні налаштування.



Образ. RDS — банківська комірka з охороною замість сейфа вдома. Ти кладеш і забираєш документи (дані), а банк стежить за замками, сигналізацією, копіями ключів і відповідає за будівлю.

Multi-AZ — двійник сейфа в іншому відділенні, що синхронно отримує кожен документ і за хвилину стає основним, якщо з першим щось сталося. **Read replica** — копія архіву в читальній залі: читати можуть усі, але оновлюється вона із затримкою.

Що таке Amazon RDS

Amazon Relational Database Service — керовані реляційні бази даних.

Рушії: **MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server, IBM**

Db2 (і окремо — Amazon Aurora, наступний розділ).

Що робить AWS:

- встановлення і налаштування рушія, **патчі ОС і бази** (у вікно обслуговування, яке обираєш ти);
- **автоматичні бекапи** і відновлення на будь-яку секунду;
- **Multi-AZ** з автоматичним перемиканням;
- моніторинг, масштабування сховища, заміна несправного обладнання.

Що робиш ти: схема і запити, користувачі й права в базі, мережевий доступ (Security Groups), чи ввімкнене шифрування, розмір інстансу, параметри.

Доступу до ОС бази немає (не підключишся по SSH). Якщо він потрібен (агенти, специфічні налаштування Oracle чи SQL Server) — **RDS Custom** (для Oracle і SQL Server) або база на EC2 (тоді вся адміністративна робота — твоя).

Будова: інстанс, сховище, мережа

- **Клас інстансу:** db.t4g (burstable, для dev), db.m7g (загальний), db.r7g / db.x2g (багато пам'яті для великих баз).
- **Сховище:** EBS — **gp3** або **io2** для високих IOPS. **Storage autoscaling** — диск бази росте сам, коли закінчується місце (до заданої межі).
- **DB subnet group** — перелік **приватних** підмереж щонайменше у **двох AZ**, де може жити база. База отримує **endpoint** — DNS-ім'я, за яким застосунок підключається.
- **Parameter group** (параметри рушія) і **option group** (додаткові можливості Oracle і SQL Server).

Висока доступність: Multi-AZ

Multi-AZ DB instance — поруч з основною базою в **іншій AZ** працює **standby**, і кожен запис **синхронно** копіюється туди. Якщо основна база падає (збій обладнання, втрата AZ, оновлення ОС, зміна класу інстансу),

RDS **автоматично** перемикає endpoint на standby — зазвичай за **1–2 хвилини**. Застосунок перепідключається за тим самим DNS-ім'ям — міняти нічого не треба.

Головне: **standby не обслуговує читання** — він лише для доступності.

(Бонус: бекапи беруться зі standby, не навантажуючи основну базу.)

Multi-AZ DB cluster (MySQL і PostgreSQL) — новіший варіант: **один writer і два reader-standby** у трьох AZ. Standby **можна читати** (через reader endpoint), а перемикання — зазвичай до **35 секунд**.

Масштабування читання: read replicas

Read replica — копія бази, що отримує зміни **асинхронно**:

- до **15** реплік для MySQL, MariaDB і PostgreSQL (для Oracle і SQL Server — менше);
- у тій самій AZ, іншій AZ або **іншому регіоні**;
- кожна має **свій endpoint** — застосунок сам спрямовує туди читання (звіти, каталог, аналітику);
- можлива **затримка реплікації** (replica lag) — свіжі дані з'являються з запізненням;
- репліку можна **promote** — зробити самостійною базою (так будують недорогий DR в іншому регіоні);
- репліка сама може бути Multi-AZ.



Multi-AZ DB instance

Primary + **синхронний standby** в іншій AZ

Мета — **доступність**.

Автоматичний failover ~1–2 хв, той самий endpoint.

Standby не читається.



Multi-AZ DB cluster

Writer + 2 **читабельні standby** у 3 AZ

Доступність + трохи читання.

Failover зазвичай до 35 с.

MySQL і PostgreSQL.



Read replicas

До 15 **асинхронних** копій, зокрема в інших регіонах

Мета — **масштабування читання** і DR (promote).

Можлива затримка даних, окремі endpoints.

На іспиті: «висока доступність, автоматичний failover» → Multi-AZ. «Звіти гальмують базу, багато читань» → read replica. «DR в іншому регіоні недорого» → cross-region read replica.

Автоскейлінгу реплік у RDS немає — їх додають вручну (в Aurora — є, розділ 35). Для масштабування **запису** RDS може лише вертикально — більший клас інстансу (з коротким простоем, в Multi-AZ — через failover).

Бекапи і відновлення

- **Автоматичні бекапи:** щоденний знімок у вікно бекапу + журнали транзакцій кожні кілька хвилин. Зберігання — **від 1 до 35 днів** (0 — вимкнено). Дозволяють **point-in-time restore (PITR)** — відновлення на будь-яку секунду в межах строку (найпізніший момент — зазвичай кілька хвилин тому).
- **Ручні знімки (snapshots)** — живуть, **доки не видалиш**, навіть після видалення бази. Можна копіювати в інші регіони й акаунти (зашифровані — з customer managed key, розділ 11).
- **Відновлення завжди створює НОВУ базу з НОВИМ endpoint** — застосунок треба перенаправити.
- Автоматичні бекапи можна **реплікувати в інший регіон**; централізовано — через **AWS Backup**.

Безпека

- База — у **приватних підмережах**; Security Group пускає лише сервери застосунку (посилання на їхню SG, розділ 14).
- **Шифрування at rest (KMS)** — **лише при створенні**; для наявної незашифрованої бази — знімок → копія з шифруванням → відновлення.
- **TLS** до бази; примусити — параметром (наприклад, `rds.force_ssl` для PostgreSQL).
- **IAM database authentication** (MySQL, PostgreSQL, MariaDB) — замість пароля короткоживучий токен, виданий за IAM-правами.
- Пароль майстер-користувача — у **Secrets Manager** з автоматичною ротацією (RDS вміє керувати ним сам).
- Автентифікація через **Active Directory** (Kerberos) для SQL Server, Oracle, MySQL, PostgreSQL.

RDS Proxy: пул з'єднань

Кожне з'єднання з реляційною базою коштує пам'яті й часу. Сотні функцій Lambda чи контейнерів, що масово відкривають і закривають з'єднання, «кладуть» базу помилкою *too many connections*. **Amazon RDS Proxy** — керований проксі між застосунком і базою:

- **пул і мультиплексування** з'єднань — тисячі клієнтів ділять невелику кількість з'єднань до бази;
- **швидший failover** (до ~66% швидше) — проксі тримає з'єднання клієнтів і сам перемикається на нову основну базу;
- **IAM-автентифікація** клієнтів і паролі з **Secrets Manager**;
- **без змін у коді** — змінюється лише endpoint.

Оновлення без простою: Blue/Green Deployments

RDS Blue/Green Deployments створює «зелену» копію бази, синхронізовану з робочою («синьою»). На зеленій — нова мажорна версія рушія, нові параметри чи зміни схеми; ти тестуєш, а потім **switchover** — перемикання, зазвичай **менш ніж за хвилину**, без змін endpoint у застосунку. Відкат — просто не перемикаєти.

Моніторинг

- **CloudWatch** — CPU, пам'ять, вільне місце, IOPS, кількість з'єднань, **ReplicaLag**.
- **Enhanced Monitoring** — метрики **ОС** бази (процеси, пам'ять, файлова система) з інтервалом до **1 секунди**.
- **Performance Insights** (тепер — частина **CloudWatch Database Insights**) — **які SQL-запити** і очікування навантажують базу.

Гроші

- Інстанси — On-Demand або **Reserved Instances** (для баз — основний спосіб знижки) і  **Database Savings Plans**.
- Для dev/test базу можна **зупинити** — але **не довше ніж на 7 днів**, потім RDS запустить її сам.
- Multi-AZ приблизно подвоює вартість інстансу — для prod це ціна доступності, для dev — зайве.

База замовлень Хмаринки на RDS

Перший крок після переїзду (до переходу на Aurora):

- **RDS for MySQL**, клас db.r7g.large, **Multi-AZ DB instance**, gp3 з autoscaling, **шифрування KMS** з першого дня.
- **Read replica** в тій самій зоні регіону — для звітів бухгалтерії; ще одна в **eu-west-1** — для DR (promote при аварії регіону).
- **RDS Proxy** — для функцій Lambda, що пишуть замовлення.
- Автоматичні бекапи **14 днів** + щотижневий знімок, скопійований в eu-west-1.
- Пароль — у **Secrets Manager** з ротацією; мажорне оновлення MySQL — через **Blue/Green**.



Спробуй у справжньому AWS: база RDS, знімок і PITR

Вартість: клас з позначкою Free tier (db.t4g.micro чи db.t3.micro, 20 GB) — у межах кредитів; видали базу одразу після практики. Бонус: створення бази RDS — одне з завдань, за які нараховують **\$20 кредитів**. **Час:** 30 хвилин.

1. **RDS → Create database → Standard create:** рушій **MySQL**, шаблон **Free tier** (або Dev/Test з класом db.t4g.micro). Ідентифікатор khmarynka-lab . Credentials — **Managed in AWS Secrets Manager**.
2. Connectivity: твоя VPC, **Public access: No**, нова Security Group. Зверни увагу на **DB subnet group** і вибір AZ.
3. Розгорни **Additional configuration:** автоматичні бекапи — **7 днів**, **encryption увімкнено**. Create.
4. Поки база створюється (~10 хвилин), відкрий вкладку **Maintenance & backups**. Коли база готова — **Actions → Take snapshot** khmarynka-lab-manual .
5. **Actions → Restore to point in time** — подивись вибір «Latest restorable time» чи конкретної секунди і зверни увагу: відновлення створить **нову** базу з новим ідентифікатором. **Не запускай** — просто закрій.
6. Подивись вкладку **Monitoring** (метрики CloudWatch) і секрет у **Secrets Manager**, створений для пароля.

Прибери за собою (обов'язково!): база → **Actions** → **Delete** → зніми «Create final snapshot», підтверди; видали ручний знімок; видали Security Group. Секрет, керований RDS, видалиться разом з базою.

Запам'ятай

RDS: **MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2**; AWS патчить, бекапить, перемикає. Доступу до ОС немає (крім **RDS Custom**).

Multi-AZ DB instance — синхронний standby, автоматичний failover ~1–2 хв, **standby не читається**.

Multi-AZ DB cluster — writer + 2 читабельні standby, failover до ~35 с.

Read replicas — асинхронні, до 15, в інших регіонах, **promote** для DR; автоскейлінгу реплік немає.

Бекапи: **1–35 днів, PITR** до секунди; ручні знімки — до видалення; **відновлення = нова база, новий endpoint**.

Шифрування — **лише при створенні**; IAM DB auth; Secrets Manager; TLS.

RDS Proxy — пул з'єднань, швидший failover, без змін коду.

Blue/Green — оновлення з перемиканням менш ніж за хвилину.

Enhanced Monitoring — метрики ОС; **Performance Insights / Database Insights** — важкі запити.

Зупинка — максимум **7 днів**.

Як питають на іспиті

Висока доступність бази з автоматичним перемиканням при втраті AZ → **RDS Multi-AZ**

Звіти й аналітика гальмують основну базу → **read replica (або кеш ElastiCache)**

Недороге відновлення бази в іншому регіоні → **cross-region read replica (promote при аварії) або копії знімків**

Потрібні і HA, і читання зі standby → **Multi-AZ DB cluster**

Lambda вичерпує з'єднання з RDS → **RDS Proxy**

Відновити базу на стан 10 хвилин тому → **point-in-time restore (створить нову базу)**

Потрібен доступ до ОС бази для агентів і власних налаштувань → **RDS Custom (або база на EC2)**

Оновити мажорну версію бази з мінімальним простоем і можливістю відкату → **RDS Blue/Green Deployments**

Не зберігати паролі до бази в кодї → **IAM DB authentication або Secrets Manager**

Зашифрувати наявну незашифровану базу → **знімок → копія з шифруванням → відновлення**

Метрики ОС бази (пам'ять процесів) щосекунди → **Enhanced Monitoring**

Знайти SQL-запити, що найбільше навантажують базу → **Performance Insights / Database Insights**

Диск бази раптово закінчується → **storage autoscaling**

Пастки

Multi-AZ = доступність (standby **не** читається); read replica = масштабування читання (**асинхронна**).

Відновлення зі знімка чи PITR — **новий endpoint**, застосунок треба перенаправити.

Шифрування не вмикається на наявній базі «галочкою».

Зупинена база **сама стартує** через 7 днів.

RDS не додає репліки автоматично — це вміє Aurora.

Перевір себе

ПИТАННЯ 1

Критична база RDS for PostgreSQL має автоматично продовжувати роботу при відмові AZ з мінімальними змінами в застосунку. Що налаштувати?

- A** Щоденні ручні знімки
- B** Multi-AZ deployment
- C** Збільшити клас інстансу
- D** Read replica в тій самій AZ

Правильна відповідь: B. Multi-AZ тримає синхронний standby в іншій AZ і автоматично перемикає на нього той самий endpoint. Read replica — асинхронна і вимагає ручного promote.

ПИТАННЯ 2

Аналітичні звіти виконуються на основній базі RDS for MySQL і сповільнюють оформлення замовлень. Звітам не потрібні дані «до секунди». Що зробити?

- A** Перейти на Multi-AZ з синхронним standby в третій AZ
- B** Збільшити період зберігання бекапів
- C** Створити read replica і спрямувати на неї звіти
- D** Увімкнути Multi-AZ і читати зі standby

Правильна відповідь: C. Read replica приймає читання і знімає навантаження з основної бази; невелика затримка реплікації для звітів прийнятна. Standby Multi-AZ DB instance не обслуговує читання.

ПИТАННЯ 3

Розробник випадково видалив таблицю о 14:32. Автоматичні бекапи увімкнено на 7 днів. Як відновити дані?

- A** Увімкнути Multi-AZ
- B** Promote read replica
- C** Point-in-time restore на 14:31 у нову базу, потім перенести дані або перемкнути застосунок
- D** Відновити поверх наявної бази в тому самому endpoint

Правильна відповідь: C. PITR відновлює стан на будь-яку секунду в межах строку, але завжди в нову базу з новим endpoint. Read replica і Multi-AZ уже отримали видалення — реплікація копіює і помилки.

ПИТАННЯ 4

Сотні одночасних функцій Lambda підключаються до Amazon RDS for MySQL, і база відмовляє через забагато з'єднань. Як розв'язати з мінімальними змінами коду?

- A** Перенести базу на EC2
- B** Додати Amazon RDS Proxy і підключати функції через його endpoint
- C** Збільшити таймаут функцій
- D** Створити 15 read replicas

Правильна відповідь: B. RDS Proxy об'єднує з'єднання в пул і ділить їх між клієнтами; код змінює лише endpoint. Репліки не допоможуть із записом і з'єднаннями до основної бази.

ПИТАННЯ 5

Компанія має незашифровану базу RDS і нову вимогу шифрувати дані at rest. Як це зробити?

- A** Створити знімок, скопіювати його з шифруванням KMS, відновити нову базу з копії і перемкнути застосунок
- B** Увімкнути шифрування EBS за замовчуванням
- C** Змінити налаштування бази: увімкнути encryption
- D** Створити зашифровану read replica і вважати її основною без додаткових кроків

Правильна відповідь: A. Шифрування RDS задається лише при створенні. Шлях — зашифрована копія знімка і відновлення з неї. Репліка незашифрованої бази теж незашифрована.

ПИТАННЯ 6

Потрібно оновити RDS for MySQL з версії 8.0 на 8.4 з мінімальним простоєм, перевірити роботу на новій версії і мати простий відкат. Що використати?

- A** RDS Blue/Green Deployments
- B** Read replica в іншому регіоні
- C** Оновлення «на місці» у вікно обслуговування
- D** Multi-AZ DB cluster

Правильна відповідь: A. Blue/Green створює синхронізовану копію, на якій можна оновити версію й перевірити; switchover займає зазвичай менше хвилини, а відкат — просто не перемикається.

ПИТАННЯ 7

Команді потрібні метрики пам'яті й процесів ОС сервера бази з інтервалом у секунду. Що увімкнути?

- A** Enhanced Monitoring
- B** VPC Flow Logs
- C** Базові метрики CloudWatch
- D** CloudTrail data events

Правильна відповідь: A. Enhanced Monitoring збирає метрики ОС агентом з інтервалом до 1 секунди. Базові метрики CloudWatch — рівня гіпервізора і рідше.

ПИТАННЯ 8

Які твердження про read replicas в Amazon RDS правильні? (Оберіть 2)

Оберіть 2 відповіді

- ☐ A RDS автоматично додає і прибирає репліки за навантаженням
- ☐ B Реплікація асинхронна, тому можлива затримка даних
- ☐ C Read replica автоматично стає основною при збої, як Multi-AZ standby
- ☐ D Read replica можна створити в іншому регіоні і зробити самостійною базою (promote)
- ☐ E Кожна read replica використовує той самий endpoint, що й основна база

Правильна відповідь: B, D. Репліки асинхронні, можуть бути в інших регіонах і promote-итися вручну. Автоматичного failover на репліку і автоскейлінгу реплік у RDS немає (це в Aurora), а кожна репліка має свій endpoint.



Amazon Aurora — реляційна база, переосмислена для хмари



Відмовостійкість · 26%



Продуктивність · 24%



Вартість · 20%



Історія Хмаринки. RDS for MySQL добре служить Хмаринці, але зростання ставить нові питання. На розпродажах п'ять read replicas не встигають, і додавати шосту доводиться вручну. Перемикання Multi-AZ триває хвилину-дві — для магазину це сотні втрачених кошків. Тестувальники щотижня чекають годину на копію бойової бази. А Олена вже думає про Німеччину і про вимогу «переживемо аварію цілого регіону з втратою не більше секунди даних». Усе це — сильні сторони **Amazon Aurora**.

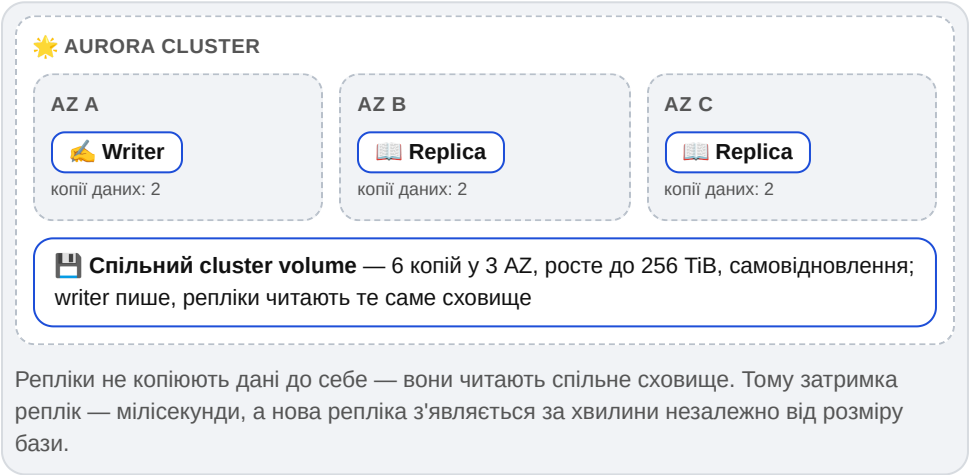


Образ. Звичайна база — кухар, який сам записує кожне замовлення в єдиний зошит на столі. **Aurora** — кухар, який лише диктує, а записує **розподілений архів**: шість писарів у трьох окремих будівлях одночасно заносять кожен рядок, і запис вважається збереженням, щойно встигли четверо. Помічники-кухарі (репліки) читають з того самого архіву — їм не треба копіювати зошит, тому вони завжди майже в курсі, а стати головним можуть за секунди.

Чим Aurora відрізняється від RDS

Amazon Aurora — реляційна база, **сумісна з MySQL і PostgreSQL** (ті самі драйвери й запити), але з власною хмарною архітектурою. За даними AWS

- до **5× швидша** за стандартний MySQL і до **3×** за PostgreSQL на тому самому залізі.
- Головне — **розподілене сховище кластера** (cluster volume):
- дані зберігаються в **6 копіях у 3 AZ** (по дві в кожній);
 - запис підтверджується, коли його зберегли **4 з 6** копій; читання — з **3 з 6**: система переживає втрату цілої AZ і ще однієї копії без втрати даних;
 - сховище **росте автоматично** до **256 TiB** (з 2025 року; раніше 128 TiB) — ніякого виділення місця наперед;
 - самовідновлюється**: пошкоджені блоки автоматично відновлюються з інших копій;
 - бекапи — **безперервні** в S3, без впливу на продуктивність.



	RDS (MySQL/PostgreSQL)	Aurora
Сховище	EBS на інстанс, розмір задаєш (autoscaling — до межі)	Розподілене, 6 копій у 3 AZ, росте саме до 256 TiB
Репліки для читання	До 15, асинхронні копії, секунди затримки	До 15 Aurora Replicas на спільному сховищі, мілісекунди затримки
Автомасштабування реплік	✗	✔ Aurora Auto Scaling
Failover	Multi-AZ: ~1–2 хв на standby	Зазвичай до ~30 с на репліку

	RDS (MySQL/PostgreSQL)	Aurora
Між регіонами	Cross-region read replica (асинхронна логічна)	Global Database: реплікація сховища, < 1 с
Serverless	✗	✓ Aurora Serverless v2
Ціна	Нижча для малих і простих баз	Трохи дорожча за інстанс, але часто вигідніша на навантаженні

Endpoints: куди підключатися

- **Cluster (writer) endpoint** — завжди на поточний **writer**: INSERT, UPDATE, DELETE. Після failover сам перемикається на новий writer.
- **Reader endpoint** — балансує читання між усіма репліками.
- **Custom endpoints** — власні групи реплік: наприклад, дві великі репліки для звітів бухгалтерії окремо від реплік для сайту.
- **Instance endpoints** — до конкретного інстансу (для діагностики).

Правило: застосунок пише **через writer endpoint**, читає **через reader** (або custom) endpoint.

Висока доступність і масштабування

- **Failover:** якщо writer падає, одна з реплік стає writer'ом — зазвичай **до ~30 секунд**. Порядок обирається **failover tiers** (пріоритети 0–15). Без реплік Aurora створить новий writer, але це довше.
- **Aurora Auto Scaling** — автоматично додає і прибирає репліки за навантаженням (CPU чи кількість з'єднань) — те, чого бракувало в RDS.
- Масштабування **запису** — вертикальне (більший клас writer'а) або **Aurora Serverless v2**; для горизонтального шардування запису PostgreSQL є **Aurora Limitless Database**.

Aurora Serverless v2

Aurora Serverless v2 автоматично й миттєво змінює потужність бази в одиницях **ACU** (Aurora Capacity Unit; 1 ACU $\approx$ 2 GiB пам'яті плюс відповідні CPU і мережа):

- масштабується дрібними кроками за секунди, без переривання з'єднань;
- з кінця 2024 року може опускатися до **0 ACU**: база «засинає» (auto-pause), коли немає з'єднань, — платиш лише за сховище; пробудження — до ~15 секунд;
- serverless і звичайні (provisioned) інстанси можна **змішувати** в одному кластері (наприклад, provisioned writer і serverless-репліки для піків).

Для чого: **непередбачуване** або **переривчасте** навантаження, нові застосунки без історії навантаження, dev/test, бази багатьох клієнтів SaaS.

Aurora Global Database

Global Database — один основний регіон і до **10 вторинних** (з травня 2025; раніше 5):

- реплікація **на рівні сховища**, типова затримка **менше 1 секунди**;
- у вторинних регіонах — репліки для **швидкого локального читання** (до 16 у кожному);
- **керований failover/switchover**: вторинний регіон стає основним зазвичай **менш ніж за хвилину** (RTO), втрата даних — близько секунди (RPO);
- **write forwarding** — застосунок у вторинному регіоні може надсилати записи, які пересилаються в основний.

Не плутай з **cross-region read replica** Aurora MySQL (логічна реплікація через binlog): вона повільніша (секунди–хвилини) і менш керована. Для мінімальних RPO/RTO між регіонами — **Global Database**.

eu-central-1 (основний)

Writer

2 репліки

Запис і читання для всіх покупців



Реплікація сховища

Затримка < 1 с, без навантаження на writer



eu-west-1 (вторинний)

Репліки для читання

При аварії Франкфурта — стає основним менш ніж за хвилину

Для Хмаринки це варіант DR «warm standby» для бази: RPO близько секунди, RTO — хвилина (розділ 42).

Особливі можливості

- **Backtrack** — «перемотати» кластер назад у часі (до **72 годин**) без відновлення з бекапу — хвилини замість години. **Лише Aurora MySQL.**
- **Cloning** — миттєва копія кластера за принципом copy-on-write: нові дані пишуться окремо, спільні — не дублюються. Тестове середовище на свіжих бойових даних за хвилини і майже без додаткових витрат на сховище.
- **Aurora I/O-Optimized** — конфігурація без плати за операції вводу-виводу (дорожчі інстанси й сховище): вигідно, коли I/O становить понад ~25% витрат на Aurora.
- **Babelfish for Aurora PostgreSQL** — розуміє **T-SQL** і протокол SQL Server: застосунки з Microsoft SQL Server переходять на Aurora PostgreSQL майже без змін коду.

- **Aurora Machine Learning** — ML-прогнози прямо в SQL (виклики SageMaker AI, Comprehend, Bedrock) без знань ML.
- **Zero-ETL integration з Amazon Redshift** — дані Aurora майже в реальному часі з'являються в Redshift для аналітики без ETL-конвеєрів.
- **Database Activity Streams** — потік дій з базою для аудиту безпеки.
- **Aurora DSQL** (з 2025) — розподілена serverless SQL-база, сумісна з PostgreSQL, **active-active у кількох регіонах** з сильною консистентністю.

Aurora для Хмаринки

- **Aurora MySQL**, writer + 2 репліки в трьох AZ; **reader endpoint** — для каталогу, **custom endpoint** «reports» на двох великих репліках — для бухгалтерії.
- **Aurora Auto Scaling**: від 2 до 8 реплік за CPU — розпродажі без ручних дій.
- **RDS Proxy** перед кластером для функцій Lambda.
- **Global Database** з вторинним регіоном eu-west-1 — коли бізнес вирішить, що регіональна аварія критична.
- **Cloning** — щотижнева копія для тестувальників; **Backtrack** на 24 години — від «випадкового DELETE без WHERE».
- **Serverless v2 з мінімумом 0 ACU** — для dev і тестових баз, що простоюють уночі.
- **Zero-ETL** в Redshift — для аналітики (розділ 39).



Спробуй у справжньому AWS: Aurora Serverless v2, що засинає

Вартість: Aurora не входить у безкоштовні ліміти, але з мінімумом 0 ACU і видаленням через 30–40 хвилин — десятки центів з кредитів.

Час: 30 хвилин.

1. **RDS → Create database → Standard create: Aurora (MySQL Compatible).** Шаблон **Dev/Test**.
2. Ідентифікатор кластера `khmarynka-aurora-lab`, credentials — **Managed in AWS Secrets Manager**.
3. Instance configuration: **Serverless v2**, мінімум **0 ACU** (або найменше доступне значення), максимум **2 ACU**. Увімкни паузу через кілька хвилин неактивності, якщо консоль пропонує.
4. Availability: **Don't create an Aurora Replica**. Connectivity: твоя VPC, Public access **No**. Create.
5. Коли кластер готовий, відкрий вкладку **Connectivity & security**: знайди **writer endpoint** і **reader endpoint** — два різні DNS-імені.
6. **Actions** → подивись пункти **Add reader**, **Add AWS Region** (так створюється Global Database), **Create clone**, **Backtrack** (якщо ввімкнено) — не виконуй їх.
7. Через 10–15 хвилин без з'єднань відкрий **Monitoring** → метрику **ServerlessDatabaseCapacity** (ACU): вона впаде до 0 — база «заснула».

Прибери за собою (обов'язково!): видали інстанс, потім кластер (без фінального знімка), і автоматичні бекапи, якщо консоль запропонує їх зберегти.

Запам'ятай

Aurora — сумісна з **MySQL і PostgreSQL**, власне **сховище: 6 копій у 3 AZ**, росте до **256 TiB**, самовідновлюється.

До **15 Aurora Replicas** на спільному сховищі (мілісекунди затримки), **Aurora Auto Scaling**, failover **до ~30 с**.

Endpoints: **writer, reader, custom** (групи реплік), instance.

Serverless v2 — масштаб в ACU, до **0 ACU** з auto-pause, для непередбачуваного й переривчастого навантаження.

Global Database — до **10** вторинних регіонів, реплікація сховища < 1 с, RTO < 1 хв, write forwarding.

Backtrack (лише MySQL, до 72 год), **Cloning** (copy-on-write), **I/O-Optimized** (без плати за I/O), **Babelfish** (T-SQL), **Aurora ML**, **zero-ETL** з **Redshift**, **DSQL** (active-active).

Як питають на іспиті

Максимальна доступність і продуктивність реляційної бази з мінімальним адмініструванням → [Amazon Aurora](#)

Репліки мають додаватися автоматично під час піків читання → [Aurora Auto Scaling](#)

Розділити аналітичні звіти і вебтрафік на різні групи реплік → [Aurora custom endpoints](#)

Найшвидша реплікація між регіонами, RPO ~1 с, RTO < 1 хв → [Aurora Global Database](#)

Швидке локальне читання для користувачів у кількох регіонах → [Aurora Global Database \(secondary regions\)](#)

Непередбачувані сплески або база, що простоє вночі → [Aurora Serverless v2](#)

Швидко повернути кластер на стан години тому без відновлення з бекапу → [Aurora Backtrack \(лише MySQL\)](#)

Швидко створити копію бойової бази для тестів → [Aurora Cloning](#)

Великі витрати на операції вводу-виводу → [Aurora I/O-Optimized](#)

Міграція з SQL Server без переписування T-SQL → [Babelfish for Aurora PostgreSQL](#)

Аналітика в Redshift на даних Aurora без ETL → [zero-ETL integration](#)

ML-прогнози звичайними SQL-запитами без знань ML → [Aurora Machine Learning](#)

Пастки

Backtrack — лише **Aurora MySQL**; для PostgreSQL — PITR або клон.

Global Database (реплікація сховища) $\neq$ cross-region read replica (логічна, повільніша).

Застосунок має **писати** через writer endpoint і **читати** через reader — інакше репліки не працюють на тебе.

Serverless v2 з мінімумом 0 ACU прокидається до ~15 секунд — для API з жорсткими вимогами затримки мінімум варто тримати вище нуля.

Перевір себе

ПИТАННЯ 1

Компанії потрібна реляційна база, сумісна з PostgreSQL, з автоматичним додаванням реплік для читання під час піків і перемиканням при збої за секунди. Що обрати?

- ☐ A PostgreSQL на EC2 з Auto Scaling group
- ☐ B Amazon RDS for PostgreSQL з read replicas
- ☐ C Amazon DynamoDB
- ☐ D Amazon Aurora PostgreSQL з Aurora Auto Scaling

Правильна відповідь: D. Aurora Auto Scaling додає і прибирає Aurora Replicas за навантаженням, а failover на репліку зазвичай займає до ~30 секунд. У RDS репліки додаються вручну.

ПИТАННЯ 2

Глобальний застосунок має головний регіон у Європі і користувачів у США та Азії. Потрібне швидке локальне читання в кожному регіоні і відновлення після аварії регіону з RPO близько секунди. Що обрати?

- A** Aurora Global Database з вторинними регіонами
- B** Multi-AZ DB cluster
- C** Щогодинне копіювання знімків в інші регіони
- D** Cross-region read replicas RDS

Правильна відповідь: A. Global Database реплікує сховище з затримкою менше секунди, дає локальні репліки для читання у вторинних регіонах і керований failover менш ніж за хвилину. Знімки дають RPO в години, Multi-AZ — лише в межах регіону.

ПИТАННЯ 3

Застосунок пише в Aurora, а важкі звіти на тих самих репліках сповільнюють сайт. Як розділити навантаження?

- A** Спрямувати звіти на writer endpoint
- B** Увімкнути Backtrack
- C** Створити custom endpoint на окремих великих репліках для звітів
- D** Перейти на Aurora I/O-Optimized

Правильна відповідь: C. Custom endpoints групують вибрані репліки: звіти підключаються до своєї групи, сайт — до reader endpoint з іншими репліками.

ПИТАННЯ 4

Розробник виконав DELETE без WHERE у кластері Aurora MySQL 20 хвилин тому. Потрібно повернути стан кластера якнайшвидше. Що використати?

- A** Aurora Backtrack
- B** Aurora Cloning
- C** Promote Aurora Replica
- D** Point-in-time restore в новий кластер

Правильна відповідь: А. Backtrack перемотує той самий кластер назад у часі без відновлення з бекапу — хвилини. PITR теж допоможе, але створює новий кластер і триває довше. Репліки вже отримали видалення.

ПИТАННЯ 5

Навантаження на внутрішню систему бронювання непередбачуване: вдень сплески, уночі — тиша годинами. Команда хоче реляційну базу, яка масштабується автоматично і майже нічого не коштує в простій. Що обрати?

- A** Aurora Serverless v2 з мінімальною потужністю 0 ACU
- B** Aurora provisioned з однією великою реплікою
- C** Amazon Redshift Serverless
- D** Amazon RDS for MySQL db.r7g.4xlarge з Reserved Instances

Правильна відповідь: А. Aurora Serverless v2 змінює потужність в ACU за секунди і може засинати до 0 ACU, коли з'єднань немає. Redshift — сховище даних для аналітики, а не OLTP.

ПИТАННЯ 6

Тестувальникам щотижня потрібна повна копія бойової бази Aurora (5 ТБ) якнайшвидше і з мінімальними витратами на сховище. Що використати?

- A** Read replica в іншому регіоні
- B** DMS з full load
- C** Aurora Cloning (copy-on-write)
- D** Знімок і відновлення в новий кластер

Правильна відповідь: C. Клон створюється за хвилини і не дублює сховище: спільні сторінки лишаються спільними, нові дані пишуться окремо. Відновлення зі знімка значно довше.

ПИТАННЯ 7

Компанія переносить застосунок з Microsoft SQL Server і не хоче переписувати тисячі рядків T-SQL. Цільова база — Aurora. Що допоможе?

- A** Aurora Backtrack
- B** Aurora MySQL з binlog-реплікацією
- C** Babelfish for Aurora PostgreSQL
- D** Aurora Global Database

Правильна відповідь: C. Babelfish дає Aurora PostgreSQL розуміти T-SQL і протокол SQL Server (TDS), тож застосунок працює майже без змін.

ПИТАННЯ 8

Які твердження про сховище Aurora правильні? (Оберіть 2)

Оберіть 2 відповіді

- ☐ А Сховище Aurora живе в одній AZ разом з writer
- ☐ В Розмір сховища потрібно задати при створенні і збільшувати вручну
- ☐ С Дані зберігаються в шести копіях у трьох AZ
- ☐ D Кожна репліка зберігає власну повну копію даних на своєму диску EBS
- ☐ Е Aurora Replicas читають те саме спільне сховище, що й writer

Правильна відповідь: С, Е. Cluster volume розподілений: 6 копій у 3 AZ, росте автоматично, і всі інстанси кластера працюють з одним сховищем — тому репліки мають мінімальну затримку.



Amazon DynamoDB — NoSQL для будь-якого масштабу



Відмовостійкість · 26%



Продуктивність · 24%



Вартість · 20%



Історія Хмаринки. Кошики покупців Хмаринка тримала в реляційній базі — і в «чорну п'ятницю» саме таблиця кошиків стала найгарячішою: тисячі оновлень щосекунди, блокування, повільні відповіді. Тарас помічає закономірність: кошик завжди читають і змінюють **за одним ключем** — ідентифікатором покупця. Жодних складних JOIN, жодних звітів по кошиках. Така задача просить іншого інструмента — бази, що дає **однакові мілісекунди** і на сотні, і на мільйони запитів за секунду, без серверів і без адміністрування. Це **Amazon DynamoDB**.



Образ. DynamoDB — величезна камера схову на вокзалі з мільйонами комірок. Кожна комірка має **номер** (ключ), за яким працівник миттєво знаходить її, хоч комірок тисяча, хоч мільярд. Що лежить у комірці — справа твоя (гнучка структура). Але попросити «знайди всі сумки червоного кольору» — довго й дорого: доведеться перебрати всі комірки (Scan). Тому камеру схову проєктують від питання «**як я шукатиму**».

Що таке DynamoDB

Amazon DynamoDB — повністю керована **serverless NoSQL**-база типу **key-value** і документна:

- **одноцифрові мілісекунди** на будь-якому масштабі — від кількох запитів до мільйонів за секунду;
- **без серверів**: немає інстансів, патчів, вікон обслуговування; масштабування — автоматичне;
- дані автоматично реплікуються в **кількох AZ** регіону;
- **гнучка схема**: у кожного елемента можуть бути різні атрибути (обов'язковий лише ключ);
- елемент (item) — до **400 KB**. Великі файли — в S3, а в таблиці — посилання.

Модель даних: таблиця, елемент, ключ

- **Таблиця** — набір **елементів** (items), елемент — набір **атрибутів** (як JSON-документ).
- **Первинний ключ** задається при створенні таблиці:
 - **partition key** (ключ розділу) — простий ключ: `userId` ;
 - або **partition key + sort key** — складений: `customerId` + `orderId` → усі замовлення клієнта, відсортовані за датою.
- DynamoDB хешує partition key і за ним розподіляє дані між **розділами** (partitions). Тому partition key має мати **високу кардинальність** (багато різних значень, рівномірне навантаження): `userId` , `orderId` . Погано — `status` чи сьогоднішня дата: усі запити летять в один розділ («гаряча партиція») і отримують **throttling**.

customerId (partition key)	orderDate (sort key)	status	total	items
c-1042	2026-09-01T10:12	shipped	780.50	[MUG-01 ×2]
c-1042	2026-09-28T19:40	paid	1 250	[LAMP-7]
c-2077	2026-09-29T08:03	new	320	[CARD-3 ×4]

«Усі замовлення клієнта c-1042 за вересень» — один швидкий Query (partition key = c-1042, sort key між датами). «Усі замовлення зі статусом paid» — потрібен індекс (GSI), інакше лише повільний Scan.

Як читати дані

- **GetItem** — один елемент за повним ключем. Найшвидше.
- **Query** — елементи з одним partition key і умовою на sort key («між датами», «починається з»). Швидко й ефективно.
- **Scan** — перебір **усієї таблиці** з фільтром. Повільно й дорого: для великих таблиць — лише для рідкісних задач (або експорт у S3 і аналітика в Athena).
- **PartiQL** — SQL-подібний синтаксис поверх тих самих операцій.

Запис: PutItem, UpdateItem (зокрема **атомарні лічильники** «+1 до переглядів»), DeleteItem, пакетні операції, **умовні записи** («записати, лише якщо версія не змінилась» — оптимістичне блокування).

Вторинні індекси

	GSI (Global Secondary Index)	LSI (Local Secondary Index)
Ключ	Інший partition key (і sort key)	Той самий partition key, інший sort key
Коли створювати	Будь-коли	Лише разом з таблицею
Консистентність читання	Лише eventual	Можна strong
Ємність	Своя, окремо від таблиці	Спільна з таблицею
Приклад	Замовлення за status чи за productId	Замовлення клієнта, відсортовані за сумою замість дати

Головний принцип проєктування: **спершу випиши всі патерни доступу** («знайти кошик за `userId`», «замовлення клієнта за датою», «нові замовлення для складу»), а потім підбери ключі та індекси. У реляційній базі починають зі структури даних, у DynamoDB — з **запитів**.

Ємність: on-demand чи provisioned

Режим	Як платиш	Коли
On-demand	За кожен запит читання/запису; таблиця миттєво підлаштовується	Непередбачуваний чи новий трафік, піки, мінімум керування
Provisioned	За заплановану ємність RCU/WCU щогодини (+ auto scaling у межах)	Передбачуваний трафік — дешевше; ще дешевше з reserved capacity

Рахуємо RCU і WCU

- **1 RCU** = **1 strongly consistent** читання за секунду елемента до **4 KB** (або **2 eventually consistent**). Транзакційне читання — **2 RCU**.
- **1 WCU** = 1 запис за секунду елемента до **1 KB**. Транзакційний запис — **2 WCU**.
- Розмір **округлюється вгору** до 4 KB (читання) чи 1 KB (запис).



Приклад з іспиту

- Застосунок читає 100 елементів за секунду по **10 KB** і записує 50 елементів за секунду по **2,5 KB**.
- Читання: 10 KB → 3 блоки по 4 KB. Strongly consistent: $100 \times 3 = \mathbf{300 \text{ RCU}}$. Eventually consistent — вдвічі менше: **150 RCU**.
 - Запис: 2,5 KB → 3 блоки по 1 KB. $50 \times 3 = \mathbf{150 \text{ WCU}}$.
 - Транзакційний запис тих самих елементів — **300 WCU**.

Консистентність і транзакції

- **Eventually consistent** читання (за замовчуванням) — може на мить не бачити щойно записане; вдвічі дешевше.
- **Strongly consistent** читання — завжди найсвіжіші дані (у межах регіону).
- **Транзакції** (TransactWriteItems, TransactGetItems) — **ACID** для кількох елементів і таблиць: «списати товар зі складу і створити замовлення — разом або ніяк».

Кешування: DAX

DynamoDB Accelerator (DAX) — кластер кешу в пам'яті **перед** DynamoDB: читання за **мікросекунди** замість мілісекунд. **API сумісний** з DynamoDB — у коді змінюється лише клієнт і endpoint. Працює у VPC, кешує результати GetItem і Query (eventually consistent). **DAX — лише для DynamoDB**; для RDS та інших баз — ElastiCache (розділ 37).

Streams, TTL, Global Tables

- **DynamoDB Streams** — упорядкований потік змін елементів (що змінилось: було/стало), зберігається **24 години**. Типово: Stream → **Lambda** — надіслати лист при зміні статусу замовлення, оновити пошуковий індекс, агрегувати статистику. **Kinesis Data Streams for DynamoDB** — довше зберігання і більше споживачів.
- **TTL** — атрибут з часом «прострочення»: DynamoDB **безкоштовно** видаляє такі елементи (зазвичай протягом кількох днів після строку). Сесії, кошики, тимчасові коди.
- **Global Tables** — таблиця в **кількох регіонах**, **multi-active**: писати й читати можна в **будь-якому** регіоні, зміни реплікуються (зазвичай менш ніж за секунду; при одночасних змінах перемагає останній запис).  З 2025 року є режим **сильної консистентності між регіонами** (RPO = 0).

- **Бекапи: PITR** — відновлення на будь-яку секунду за останні **35 днів**; on-demand бекапи без впливу на продуктивність; AWS Backup. Відновлення — у **нову** таблицю.
- **Export to S3** (з даних PITR, без витрати ємності таблиці) → аналітика в **Athena**; **zero-ETL** в Redshift і OpenSearch.
- **Table class Standard-IA** — дешевше зберігання для рідко читаних таблиць.

Безпека

- Шифрування at rest — **завжди** (ключ AWS, AWS managed чи customer managed KMS).
- **IAM** з деталізацією до елементів: умова `dynamodb:LeadingKeys = ${cognito-identity.amazonaws.com:sub}` — кожен користувач мобільного застосунку бачить лише свої записи.
- Доступ з VPC — **gateway endpoint** (безкоштовно, розділ 15); resource-based політики для крос-акаунтного доступу.

Коли DynamoDB, а коли реляційна база

DynamoDB 	RDS / Aurora 
Прості відомі запити за ключем	Складні JOIN, довільні запити, звіти
Мільйони запитів за секунду, мілісекунди	Помірний масштаб, складна логіка в SQL
Гнучка схема, serverless, без адміністрування	Суворі схеми, зв'язки, наявний SQL-код
Кошики, сесії, профілі, лічильники, IoT, лідерборди, метадані	Облік, ERP, бухгалтерія, класичні бізнес-системи

ДynamoDB у Хмаринці

- **Carts** (PK `userId`): кошики, **TTL** 7 днів; **on-demand** — розпродажі без планування.
- **Sessions** (PK `sessionId`): сесії stateless-серверів, TTL 24 години.
- **ProductViews** (PK `productId`): атомарні лічильники переглядів.
- **Orders** (PK `customerId`, SK `orderDate`) + **GSI** `status-index` для складу; **Streams** → **Lambda** — лист покупцю при зміні статусу.
- **PITR** на всіх таблицях; щоночі **export to S3** для аналітики.
- Згодом — **Global Tables** з eu-west-1, коли з'являться покупці в Ірландії й Британії.



Спробуй у справжньому AWS: таблиця, Query і TTL

Вартість: безкоштовно (provisioned 5 RCU/5 WCU і кілька КБ даних у межах постійного безкоштовного ліміту). **Час:** 20 хвилин.

1. **DynamoDB** → **Create table:** назва `Orders`, partition key `customerId` (String), sort key `orderDate` (String). **Customize settings** → Capacity mode **Provisioned**, 5 RCU / 5 WCU, auto scaling вимкни. Create.
2. **Explore items** → **Create item** — додай 3 замовлення (як у таблиці вище), для одного — додатковий атрибут `giftWrap = true` (гнучка схема).
3. **Query:** `customerId = c-1042`, sort key **between** `2026-09-01` і `2026-09-30`. Потім зроби **Scan** з фільтром `status = paid` — порівняй «Items scanned» і «Items returned».
4. **CloudShell:**

```
aws dynamodb put-item --table-name Orders --item '{"customerId":{"S
aws dynamodb query --table-name Orders --key-condition-expression "
aws dynamodb update-item --table-name Orders --key '{"customerId":{"
```

Остання команда — атомарний лічильник.

1. Таблиця → **Additional settings** → **Time to Live** → **Turn on**, атрибут `expiresAt`. **Backups** → **Point-in-time recovery** — подивись опції (можна не вмикати).
2. **Indexes** → **Create index**: GSI з `partition key status` — подивись, як його створення не блокує таблицю.

Прибери за собою: видали таблицю `Orders` (разом з індексами).

Запам'ятай

DynamoDB — **serverless NoSQL** key-value/документи: мілісекунди на будь-якому масштабі, кілька AZ, елемент до **400 KB**.

Ключ: **partition key** (+ **sort key**); `partition key` — **висока кардинальність**, інакше гарячі партиції й `throttling`.

GetItem / Query — швидко; **Scan** — дорого. **GSI** — будь-коли, інший ключ; **LSI** — лише при створенні таблиці.

On-demand — непередбачуваний трафік; **provisioned** + `auto scaling` (+ `reserved`) — передбачуваний.

1 RCU = 1 strong (або 2 eventual) читання/с до **4 KB**; **1 WCU** = 1 запис/с до **1 KB**; транзакції $\times 2$; округлення вгору.

DAX — мікросекунди, сумісний API, **лише для DynamoDB**.

Streams (24 год) → `Lambda`; **TTL** — безкоштовне видалення; **Global Tables** — multi-active у кількох регіонах; **PITR** 35 днів; **export to S3**.

Доступ на рівні елементів — **IAM** `dynamodb:LeadingKeys`; з VPC — **gateway endpoint**.

Як питають на іспиті

Serverless-база з мілісекундними відповідями для мільйонів запитів, гнучка схема → **DynamoDB**

Мікросекундне читання з DynamoDB без переписування застосунку → **DAX**

Непередбачуваний трафік, платити лише за запити → **DynamoDB on-demand**

Стабільний передбачуваний трафік, дешевше → **provisioned capacity + auto scaling (+ reserved capacity)**

Глобальний застосунок, запис і читання з низькою затримкою в кількох регіонах → **DynamoDB Global Tables**

Реагувати на кожну зміну в таблиці → **DynamoDB Streams + Lambda**

Автоматично видаляти сесії через 24 години → **TTL**

Відновити таблицю на будь-яку секунду за останні 35 днів → **PITR**

SQL-аналітика на даних DynamoDB без навантаження на таблицю → **export to S3 + Athena**

Throttling, бо всі записи йдуть в один ключ → **partition key з високою кардинальністю (або write sharding)**

Потрібен новий шаблон запиту до наявної таблиці → **GSI**

Кожен користувач застосунку має бачити лише свої записи → **IAM-умова dynamodb:LeadingKeys**

Складні JOIN і реляційна модель → **не DynamoDB, а RDS / Aurora**

Пастки

LSI не додати до наявної таблиці — лише **GSI**.

DAX не прискорює RDS; для RDS — ElastiCache.

Елемент понад **400 KB** не поміститься — великі дані в S3.

Scan великої таблиці — повільно й дорого; проєктуй ключі й індекси під запити.

Strong consistency і транзакції коштують удвічі більше ємності.

Перевір себе

ПИТАННЯ 1

Застосунок щосекунди виконує 80 strongly consistent читань елементів розміром 6 KB. Скільки RCU потрібно в режимі provisioned?

A 480

B 80

C 160

D 120

Правильна відповідь: C. 6 KB округлюється до 8 KB — це 2 блоки по 4 KB. Strongly consistent: $80 \times 2 = 160$ RCU. Eventually consistent було б 80 RCU.

ПИТАННЯ 2

Потрібно щосекунди записувати 200 елементів розміром 1,5 KB. Скільки WCU?

A 300

B 400

C 200

D 600

Правильна відповідь: B. 1,5 KB округлюється до 2 KB — 2 WCU на запис.
 $200 \times 2 = 400$ WCU.

ПИТАННЯ 3

Ігрова компанія зберігає профілі гравців у DynamoDB. Читання профілів має відбуватися за мікросекунди, без суттєвих змін коду. Що додати?

A DynamoDB Accelerator (DAX)

B Amazon ElastiCache for Memcached з новою логікою кешування

C Перейти на provisioned capacity

D Read replicas DynamoDB

Правильна відповідь: A. DAX — кеш у пам'яті, сумісний з API DynamoDB: мікросекундні читання зі зміною лише клієнта. ElastiCache потребує власної логіки кешування.

ПИТАННЯ 4

Таблиця використовує поточну дату як partition key, і всі записи за день потрапляють в один розділ, викликаючи throttling. Що змінити?

- A** Обрати partition key з високою кардинальністю (наприклад, orderId) або додати суфікс-шардинг до ключа
- B** Увімкнути DAX
- C** Створити LSI
- D** Перейти на strongly consistent читання

Правильна відповідь: A. Гаряча партиція — наслідок ключа з малою кількістю значень. Ключ з високою кардинальністю (або write sharding) розподіляє навантаження між розділами. DAX прискорює читання, але не запис.

ПИТАННЯ 5

Застосунок уже працює з таблицею, і з'явилася потреба шукати замовлення за статусом. Як це зробити?

- A** Створити Global Secondary Index з partition key status
- B** Щоразу виконувати Scan з фільтром
- C** Створити Local Secondary Index з partition key status
- D** Перестворити таблицю з новим первинним ключем

Правильна відповідь: A. GSI можна додати до наявної таблиці з будь-яким іншим ключем. LSI створюються лише разом з таблицею і мають той самий partition key.

ПИТАННЯ 6

Мобільний застосунок працює в Європі й Америці. Користувачі мають записувати й читати дані з мінімальною затримкою у своєму регіоні, а дані — бути однаковими всюди. Що обрати?

- A** Одна таблиця в us-east-1 з DAX
- B** DynamoDB Global Tables
- C** DynamoDB Streams з Lambda-копіюванням в інший регіон
- D** Aurora cross-region read replica

Правильна відповідь: B. Global Tables — multi-active реплікація: запис і читання в будь-якому регіоні з автоматичною синхронізацією. Самописне копіювання складніше і гірше.

ПИТАННЯ 7

Сесії користувачів мають автоматично видалятися через 24 години без додаткового коду і витрат. Що використати?

- A** Щогодинний Scan і видалення через Lambda
- B** PITR
- C** DynamoDB TTL
- D** DynamoDB Streams

Правильна відповідь: C. TTL безкоштовно видаляє прострочені елементи за атрибутом часу. Scan з видаленням — зайва ємність і код.

ПИТАННЯ 8

Які твердження про DynamoDB правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** LSI можна додати будь-коли після створення таблиці
- B** Scan — найефективніший спосіб знайти елемент за ключем
- C** Максимальний розмір елемента — 400 KB
- D** DAX можна використати як кеш для Amazon RDS
- E** DynamoDB Streams зберігає зміни 24 години

Правильна відповідь: C, E. Елемент — до 400 KB, Streams — 24 години.
LSI — лише при створенні таблиці, DAX — лише для DynamoDB, а за ключем шукають GetItem/Query.



Кеш і спеціалізовані бази — ElastiCache, MemoryDB, DocumentDB, Neptune, Keyspaces, Timestream, OpenSearch



Продуктивність · 24%



Відмовостійкість · 26%



Вартість · 20%



Історія Хмаринки. Головна сторінка Хмаринки показує «популярні товари» — і щоразу рахує їх заново в базі, тисячі разів на хвилину, хоча список змінюється раз на годину. Пошук за словом «свічка» не знаходить «свічки», «свічок» і «candle». Маркетинг хоче рекомендації «з цим товаром купують», а склад — зберігати показники датчиків температури й вологості. Тарас розуміє: одна універсальна база на все — як один молоток для всіх задач. AWS пропонує **спеціалізовані бази** (purpose-built): кожна найкраща у своїй справі.



Образ. Кеш — стікер на моніторі з номером телефону, який ти набирав двадцять разів на день: не треба щоразу гортати телефонну книгу (базу). **Графова база** — дошка детектива з фотографіями і нитками між ними: головне — зв'язки. **Часові ряди** — щоденник метеостанції. **Пошуковий рушій** — предметний покажчик у кінці книжки, що знає всі форми слова.

Кешування: навіщо і як

Оперативна пам'ять у тисячі разів швидша за диск. **Кеш у пам'яті** зберігає результати, які часто читають, — і відповідає за **мікросекунди–мілісекунди**, знімаючи навантаження з основної бази.

Дві класичні стратегії (і обидві вимагають **змін у коді** застосунку):

Lazy loading (cache-aside)
1) Шукаємо в кеші → **hit**: віддаємо.
2) **Miss**: читаємо з бази, кладемо в кеш з **TTL**, віддаємо.
✅ У кеші лише те, що справді запитують; збій кешу не зупиняє систему.
❌ Перший запит повільний; дані можуть бути застарілими до кінця **TTL**.

Write-through
При кожному **записі** в базу — одночасно оновлюємо кеш.
✅ Кеш завжди свіжий.
❌ Кожен запис довший; у кеші лежить і те, що ніхто не читає (лікується **TTL**).

Часто їх поєднують: **write-through** для важливих даних + **lazy loading** + **TTL** для всього іншого.

Amazon ElastiCache

Amazon ElastiCache — керований кеш у пам'яті з трьома рушіями:

	Valkey / Redis OSS	Memcached
Структури даних	Рядки, хеші, списки, множини, sorted sets , геодані, потоки, pub/sub	Лише прості ключ-значення
Реплікація і Multi-AZ	✅ Репліки й автоматичний failover	❌
Збереження на диск, бекапи	✅ Знімки	❌ Дані в пам'яті — зникнуть при перезапуску
Масштабування	Шардинг (cluster mode), репліки для читання, Global Datastore між регіонами	Шардинг на боці клієнта, багатопотоковість

	Valkey / Redis OSS	Memcached
Коли	Майже завжди: сесії, лідерборди, лічильники, кеш з НА	Найпростіший багатопотоковий кеш, втрата даних некритична

Valkey — відкрита спільнотою гілка Redis; AWS радить її для нових кешів (сумісна з Redis і дешевша). **ElastiCache Serverless** — кеш без вибору вузлів: масштабується автоматично, оплата за використання.

Типові задачі ElastiCache:

- **кеш запитів до бази** (RDS, Aurora, будь-якої) — lazy loading з TTL;
- **сесії stateless-серверів** (розділ 23);
- **лідерборди — sorted sets** (рейтинг гравців, «топ продавців» оновлюється миттєво);
- обмеження частоти запитів (rate limiting), черги, pub/sub.

Безпека: кеш у приватних підмережах VPC, Security Groups, шифрування в дорозі й у спокої, автентифікація (AUTH, RBAC, IAM).



«Без змін коду»

ElastiCache завжди вимагає логіки кешування в застосунку. Якщо в питанні вимога **«without changing the application code»**, дивись на інше: **read replicas** для RDS, **DAX** для DynamoDB (сумісний API), **CloudFront** для HTTP-контенту.

Amazon MemoryDB

Amazon MemoryDB — Valkey/Redis-сумісна **основна база даних** (не кеш!) у пам'яті з **довговічністю**: кожен запис фіксується в розподіленому журналі транзакцій у кількох AZ. Читання — мікросекунди, запис — одноцифрові мілісекунди. Коли потрібна швидкість Redis і гарантія, що дані не зникнуть: профілі, кошики, стан гри, лічильники як джерело правди.

Спеціалізовані бази

- **Amazon DocumentDB** — документна база, **сумісна з MongoDB**: JSON-документи з гнучкою структурою, архітектура сховища подібна до Aurora (копії в трьох AZ, репліки). Для перенесення MongoDB-застосунків у керований сервіс і каталогів з різною структурою документів.
- **Amazon Neptune** — **графова** база: вузли й зв'язки (Gremlin, openCypher, SPARQL). **Соцмережі** («друзі друзів»), **рекомендації** («хто купив це, купив і те»), **виявлення шахрайства** (зв'язані акаунти, картки, пристрої), графи знань.
- **Amazon Keyspaces** — serverless-база, **сумісна з Apache Cassandra** (мова CQL): для наявних Cassandra-застосунків без керування кластером.
- **Amazon Timestream** — **часові ряди**: мільйони вимірів з мітками часу, автоматичне перенесення старих даних у дешевший рівень, функції для аналізу в часі. **IoT-датчики**, метрики серверів і застосунків, телеметрія. (Є і варіант з керованим InfluxDB.)
- **Amazon OpenSearch Service** — **пошук і аналітика журналів** (відкритий форк Elasticsearch): повнотекстовий пошук з морфологією, синонімами, релевантністю, фасетами; дашборди **OpenSearch Dashboards**; **OpenSearch Serverless**; холодні рівні зберігання. Типово: каталог товарів з DynamoDB/Aurora → OpenSearch для пошуку; журнали → Firehose → OpenSearch для аналізу.
- **Amazon Redshift** — сховище даних для аналітики (OLAP) — розділ 39.
- **Amazon QLDB** (журнальна база) **закрита з 31.07.2025** — не обирай її.

Яку базу обрати: головна таблиця

Вимога	Сервіс
Реляційна модель, SQL, JOIN, транзакції (OLTP)	RDS / Aurora

Вимога	Сервіс
Максимальна доступність і продуктивність реляційної бази, глобальні репліки	Aurora
Key-value/документи, мілісекунди на будь-якому масштабі, serverless	DynamoDB
Мікросекунди для DynamoDB без змін коду	DAX
Кеш для будь-якої бази, сесії, лідерборди	ElastiCache (Valkey / Redis OSS)
Найпростіший багатопотоковий кеш без реплікації	ElastiCache Memcached
Основна база в пам'яті з довговічністю	MemoryDB
MongoDB-сумісна документна	DocumentDB
Графи, зв'язки, рекомендації, шахрайство	Neptune
Cassandra без керування	Keyspaces
Часові ряди, IoT, метрики	Timestream
Повнотекстовий пошук, аналіз журналів	OpenSearch Service
Аналітика на петабайтах, BI (OLAP)	Redshift

Хмаринка: правильна база для кожної задачі

**Aurora MySQL**
Замовлення, платежі, облік — транзакції і звіти

**DynamoDB**
Кошки, сесії, лічильники переглядів

**ElastiCache (Valkey Serverless)**
Кеш каталогу і «популярних товарів», лідерборд продавців

**OpenSearch**
Пошук з морфологією: «свічка», «свічки», «candle»

**Neptune**
«З цим товаром купують» — граф покупок

**Timestream**
Температура й вологість на складі свічок

Шість сервісів звучить складно, але кожен керований: жодних серверів, патчів і ручних бекапів. Архітектор обирає за патерном доступу, а не «одну базу на все».

Спробуй у справжньому AWS: кеш Valkey і лідерборд

Вартість: ElastiCache Serverless на 30 хвилин і маленький сервер — кілька десятків центів з кредитів. **Час:** 30 хвилин.

1. **ElastiCache** → **Create cache** → **Valkey** → **Serverless**. Назва `khmrynka-cache`, твоя VPC і дві приватні або публічні підмережі; Security Group, що дозволяє **TCP 6379** від Security Group сервера. Create.
2. Запусти (або використай наявний) сервер Amazon Linux 2023 у тій самій VPC і підключись до нього.
3. Встанови клієнт і підключись (кеш serverless вимагає TLS; endpoint скопіюй з консолі):

```
sudo dnf install -y redis6
redis6-cli --tls -h <endpoint-кешу> -p 6379
```

1. Спробуй кешування з TTL і лідерборд:

```
SET product:42 "{\"name\":\"Свічка Лаванда\",\"price\":320}" EX 60
GET product:42
TTL product:42
ZADD sellers 1500 "Майстерня Оксани" 2300 "Свічки Львова" 900 "Керам
ZREVRANGE sellers 0 2 WITHSCORES
ZINCRBY sellers 1000 "Керамік"
ZREVRANGE sellers 0 2 WITHSCORES
```

Через 60 секунд `GET product:42` поверне порожнечу — TTL спрацював (lazy loading взяв би дані з бази знову). **Прибери за собою:** видали кеш і сервер, Security Groups.

Запам'ятай

Кеш у пам'яті — мікросекунди; стратегії **lazy loading (cache-aside)** і **write-through**, **TTL**; потрібні зміни в коді.

ElastiCache Valkey/Redis: реплікація, **Multi-AZ failover**, бекапи, **sorted sets** (лідерборди), pub/sub, Global Datastore; **Memcached:** простий, багатопотоковий, **без реплікації й збереження**; є **Serverless**.

MemoryDB — основна Redis-сумісна база в пам'яті з довговічністю.

DocumentDB — MongoDB; **Neptune** — графи; **Keyspaces** — Cassandra; **Timestream** — часові ряди; **OpenSearch** — пошук і журнали; **Redshift** — аналітика; **QLDB** — закрита.

«Без змін коду» → не ElastiCache, а **read replicas** (RDS) чи **DAX** (DynamoDB).

Як питають на іспиті

Лідерборд гри в реальному часі → **ElastiCache for Valkey/Redis (sorted sets)**

Зберігати сесії для stateless-вебсерверів → **ElastiCache або DynamoDB**

Зменшити навантаження на RDS для повторюваних запитів → **ElastiCache (lazy loading)**

Простий багатопотоковий кеш, реплікація не потрібна → **ElastiCache Memcached**

Кеш має пережити збій вузла без втрати даних → **Valkey/Redis з Multi-AZ і репліками**

Надшвидка основна база, сумісна з Redis, з довговічністю → **MemoryDB**

Перенести MongoDB-застосунок у керований сервіс → **DocumentDB**

Соцмережа «друзі друзів», рекомендації, виявлення шахрайських зв'язків → **Neptune**

Перенести Cassandra в керований сервіс → **Keyspaces**

Показники IoT-датчиків з мітками часу → **Timestream**

Повнотекстовий пошук по каталогу, аналіз журналів з дашбордами → **OpenSearch Service**

Пастки

ElastiCache не прискорить базу «без змін коду».

Memcached не має реплікації, failover і бекапів — не для даних, які не можна втратити.

Кеш ≠ основна база: для довговічних даних у пам'яті — MemoryDB.

QLDB закрита — не обирай її навіть для «журналу незмінних записів».

Перевір себе

ПИТАННЯ 1

Мобільна гра має показувати глобальний рейтинг гравців, що оновлюється в реальному часі при кожному набранні очок. Що обрати?

- A** Amazon RDS з ORDER BY при кожному запиті
- B** ElastiCache Memcached
- C** Amazon ElastiCache for Valkey або Redis OSS із sorted sets
- D** Amazon S3 з JSON-файлом рейтингу

Правильна відповідь: C. Sorted sets тримають елементи відсортованими за балом: оновлення і вибірка топу — за мікросекунди. Memcached не має таких структур, а сортування в базі при кожному запиті — повільно.

ПИТАННЯ 2

Аплікація на RDS for MySQL щосекунди виконує тисячі однакових запитів до каталогу, що змінюється раз на годину. Команда готова змінити код. Як зменшити навантаження на базу найефективніше?

- A** Додати DAX
- B** Збільшити період зберігання бекапів
- C** Кешувати результати в ElastiCache за стратегією lazy loading з TTL
- D** Перейти на Multi-AZ

Правильна відповідь: C. Кеш у пам'яті віддає результати без звернення до бази, а TTL обмежує застарілість. DAX працює лише з DynamoDB, Multi-AZ не допомагає з читанням.

ПИТАННЯ 3

Кеш сесій має продовжувати роботу при відмові вузла, а сесії не повинні губитися. Що обрати?

- A** Amazon S3 Standard
- B** ElastiCache for Valkey/Redis з репліками і Multi-AZ з автоматичним failover
- C** Instance store на вебсерверах
- D** ElastiCache Memcached з кількома вузлами

Правильна відповідь: B. Valkey/Redis підтримує реплікацію і автоматичний failover у Multi-AZ. Memcached не реплікує дані — при втраті вузла його дані зникають.

ПИТАННЯ 4

Фінтех-компанія хоче виявляти мережі пов'язаних шахрайських акаунтів, що ділять пристрої, адреси й картки. Яка база найпридатніша?

- A** Amazon Timestream
- B** Amazon Neptune
- C** Amazon ElastiCache
- D** Amazon DynamoDB

Правильна відповідь: B. Графова база природно зберігає і швидко обходить зв'язки між сутностями — класичний сценарій виявлення шахрайства. Інші бази не розраховані на глибокі запити по зв'язках.

ПИТАННЯ 5

Компанія переносить застосунок, що використовує MongoDB, і хоче керований сервіс з мінімальними змінами коду. Що обрати?

- A** Amazon Neptune
- B** Amazon RDS for PostgreSQL
- C** Amazon Keyspaces
- D** Amazon DocumentDB

Правильна відповідь: D. DocumentDB сумісний з API MongoDB. Keyspaces — для Cassandra, Neptune — графи, PostgreSQL вимагав би переписування.

ПИТАННЯ 6

Потрібно зберігати й аналізувати мільярди показників датчиків температури з мітками часу, автоматично переносючи старі дані в дешевший рівень. Що обрати?

A ElastiCache Memcached

B Amazon Timestream

C Amazon Neptune

D Amazon DocumentDB

Правильна відповідь: B. Timestream — спеціалізована база часових рядів з рівнями зберігання і функціями аналізу в часі.

ПИТАННЯ 7

Інтернет-магазин хоче повнотекстовий пошук по каталогу з урахуванням форм слів, синонімів і сортуванням за релевантністю. Що обрати?

A Amazon DynamoDB Scan з фільтром

B Amazon Keyspaces

C Amazon RDS LIKE-запити

D Amazon OpenSearch Service

Правильна відповідь: D. OpenSearch — пошуковий рушій з аналізаторами мови, синонімами, релевантністю і фасетами. Scan і LIKE повільні й не розуміють морфології.

ПИТАННЯ 8

Які твердження про ElastiCache Memcached правильні? (Оберіть 2)

Оберіть 2 відповіді

- ☐ А Підтримує багатопотокову роботу
- ☐ В Має вбудовану реплікацію і автоматичний failover
- ☐ С Підтримує sorted sets для лідербордів
- ☐ D Сумісний з API DynamoDB
- ☐ Е Не зберігає дані на диск і не має бекапів

Правильна відповідь: А, Е. Memcached — простий багатопотоковий кеш без реплікації, persistence і складних структур даних. Реплікація, бекапи і sorted sets — у Valkey/Redis.



Потоки даних у реальному часі — Kinesis Data Streams, Data Firehose, Flink, MSK



Продуктивність · 24%



Відмовостійкість · 26%



Вартість · 20%



Історія Хмаринки. Маркетинг хоче знати, **що відбувається на сайті прямо зараз**: які товари розглядають, що кладуть у кошик, звідки прийшли покупці. Під час розпродажу це 20 000 подій за секунду. Ще три бажання: показувати «🔥 зараз купують» у реальному часі, ловити ботів, що скуповують товар, за кілька секунд, і складати всі події в сховище для аналітики. Черга SQS тут не підходить: подію має прочитати **кілька** систем, і **в порядку** надходження, а ще — мати змогу **перечитати** вчорашні події після виправлення помилки. Потрібен **потік даних**.



Образ. Черга SQS — стос квитків на кухні: кухар бере квиток, і для інших його вже немає. **Потік (Kinesis)** — стрічка конвеєра з камерою запису: усе, що проїжджає, записується на плівку (зберігається від доби до року), і кожен відділ дивиться запис зі **своїї** позиції — хтось наживо, хтось перемотує на вчора. Стрічка розділена на доріжки (**shards**): що більше доріжок, то більше вантажу за секунду. **Firehose** — автоматичний навантажувач у кінці конвеєра, що складає вантаж у склад пакетами.

Потік чи черга

	Черга (SQS)	Потік (Kinesis Data Streams, Kafka)
Хто отримує повідомлення	Один обробник (після обробки — видаляється)	Багато споживачів читають ті самі записи незалежно
Порядок	Standard — ні, FIFO — в межах групи	Так, у межах shard (partition)
Зберігання	До 14 днів, до обробки	Від 24 год до 365 днів (Kinesis), незалежно від читання
Повторне читання (replay)	✗	✓ перечитати з будь-якої позиції
Типові дані	Задачі: «обробити замовлення»	Події й телеметрія: кліки, журнали, IoT, транзакції

Amazon Kinesis Data Streams

Kinesis Data Streams (KDS) — потік записів у реальному часі (затримка — сотні мілісекунд) з власними споживачами.

- Записи мають partition key — від нього залежить, у який shard потрапить запис. Порядок гарантовано в межах shard: усі події однієї сесії (однаковий ключ) йдуть по черзі.
- Shard — одиниця пропускної здатності: запис 1 MB/c або 1 000 записів/c, читання 2 MB/c (спільні для всіх споживачів shard).
- Enhanced fan-out — кожен споживач отримує власні 2 MB/c на shard (push), без конкуренції з іншими.
- Зберігання: 24 години за замовчуванням, до 365 днів.
- Розмір запису: до 1 MiB  з жовтня 2025 можна налаштувати до 10 MiB).

- **Режими ємності: on-demand** — потік сам підлаштовується під навантаження (не треба рахувати shards); **provisioned** — задаєш кількість shards (дешевше при стабільному трафіку), збільшуєш **split** (розділити shard), зменшуєш **merge**.
- **Виробники:** SDK (PutRecord/PutRecords), Kinesis Producer Library (пакетування), Kinesis Agent, журнали CloudWatch.
- **Споживачі:** **Lambda**, застосунки на **KCL** (Kinesis Client Library — сама ділить shards між воркерами і зберігає позиції в DynamoDB), **Managed Service for Apache Flink**, **Data Firehose**.



Скільки shards потрібно (provisioned)

Хмаринка пише **6 000 подій/с** по **0,5 KB** = 3 MB/с. І двоє споживачів читають усе.

- Запис за обсягом: $3 \text{ MB/с} \div 1 \text{ MB/с} = \mathbf{3 \text{ shards}}$. За кількістю: $6\,000 \div 1\,000 = \mathbf{6 \text{ shards}}$. Беремо більше — **6**.
- Читання: $2 \text{ споживачі} \times 3 \text{ MB/с} = 6 \text{ MB/с}$; $6 \text{ shards} \times 2 \text{ MB/с} = 12 \text{ MB/с}$ — вистачає. З enhanced fan-out кожен споживач мав би власні 2 MB/с на shard.
- Не хочеш рахувати і трафік стрибає — **on-demand**.

«Гарячий» shard (помилка `ProvisionedThroughputExceededException`, хоча загальне навантаження невелике): partition key з малою кількістю значень гонить усе в один shard. Лікування: **кращий partition key** (з високою кардинальністю), **split** гарячого shard, або **on-demand**.

Amazon Data Firehose

Amazon Data Firehose (колишня назва — Kinesis Data Firehose) — повністю керована **доставка** поточкових даних у сховища:

- **куди:** **S3, Redshift** (через S3 і команду COPY), **OpenSearch, Splunk, HTTP-endpoints** (Datadog, New Relic, власні), Snowflake, **таблиці Apache Iceberg**;
- **звідки:** напряду від застосунків (Direct PUT), з **Kinesis Data Streams, MSK**, журналів CloudWatch, журналів WAF, EventBridge;
- **буферизація** за розміром (МБ) або часом (секунди) → **near real-time** (секунди–хвилини), а не миттєво;
- **без коду і без серверів**, масштабується сама; платиш за обсяг даних;
- **перетворення** на льоту: функція **Lambda** для кожного запису; **конвертація JSON → Parquet/ORC** (за схемою з Glue Data Catalog); стиснення; **динамічне партиціювання** в S3
(year=2026/month=09/day=30/);
- невдалі записи — окремо в S3.

Чого у Firehose **немає**: зберігання для повторного читання (**replay**) і власних споживачів у реальному часі. Для цього — Kinesis Data Streams.



Streams чи Firehose?

Data Streams — «реальний час, свої споживачі, replay» (мілісекунди, код потрібен). **Firehose** — «просто поклади потік у S3 / Redshift / OpenSearch, без коду» (секунди–хвилини). Часто їх поєднують: Streams для реального часу, а Firehose як один зі споживачів складає все в озеро даних.

Обробка потоків: Managed Service for Apache Flink

Amazon Managed Service for Apache Flink (колишня назва — Kinesis Data Analytics) — обробка потоків **у реальному часі** на Apache Flink (Java, Python, Scala, SQL): агрегації **у вікнах часу** («покупки за останні 5 хвилин за категоріями»), з'єднання потоків, **виявлення аномалій**, збагачення.

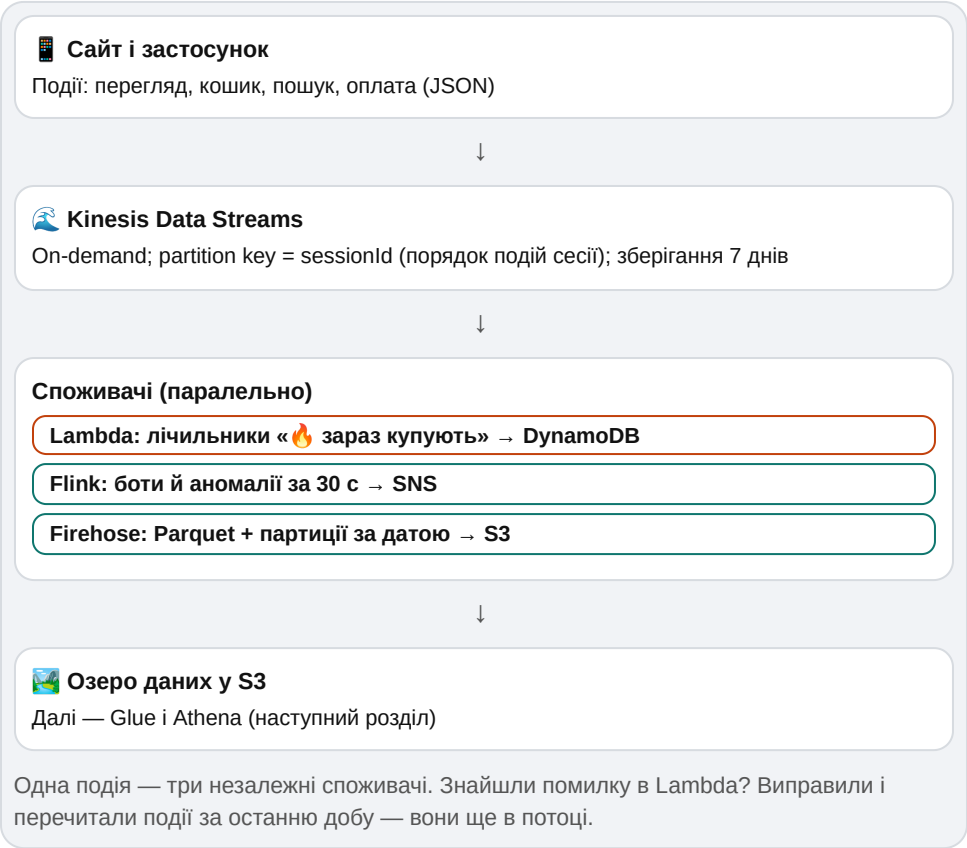
Джерела — Kinesis і MSK; результати — в інший потік, S3, базу, OpenSearch. Серверів немає — AWS запускає і масштабує Flink.

Відео й Kafka

- **Amazon Kinesis Video Streams** — приймає **відео** з камер і пристроїв, зберігає, дає відтворення і аналіз машинним навчанням (наприклад, Rekognition Video — розпізнавання облич, об'єктів). Також відеозв'язок WebRTC.
- **Amazon MSK** (Managed Streaming for Apache Kafka) — керований **Apache Kafka**: для компаній, що вже мають Kafka-застосунки, екосистему Kafka (конектори, стрім-процесори), особливі вимоги до зберігання. Варіанти: кластери з брокерами (зокрема **Express brokers** зі швидким масштабуванням), **MSK Serverless**, **MSK Connect** (керовані конектори Kafka Connect).

Kinesis чи MSK? Kinesis — «рідний» сервіс AWS, простіше почати, serverless on-demand. MSK — коли потрібна **сумісність з Kafka** (наявний код, інструменти, переносимість).

Потокова архітектура Хмаринки



Що обрати

Потреба	Сервіс
Реальний час, кілька власних споживачів, порядок, replay	Kinesis Data Streams
Складати потік у S3 / Redshift / OpenSearch / Splunk без коду, near real-time	Amazon Data Firehose

Потреба	Сервіс
Перетворити JSON у Parquet перед збереженням у S3	Firehose (конвертація формату)
Агрегації у вікнах часу, аномалії в реальному часі	Managed Service for Apache Flink
Наявні застосунки на Apache Kafka	Amazon MSK
Відео з камер для відтворення й ML	Kinesis Video Streams
Задачі, які має обробити один обробник, буфер піків	SQS (розділ 31)

Спробуй у справжньому AWS: потік, записи і доставка в S3

Вартість: 1 shard і Firehose на 30 хвилин з кількома записами — центи. **Час:** 25 хвилин.

1. **Kinesis** → **Data streams** → **Create**: назва `clicks`, режим **Provisioned, 1 shard**. Create.
2. **CloudShell** — запиши кілька подій і прочитай їх:

```
for i in 1 2 3; do
  aws kinesis put-record --stream-name clicks --partition-key "sess
    --cli-binary-format raw-in-base64-out --data "{\"event\":\"view
done
SHARD=$(aws kinesis list-shards --stream-name clicks --query "Shard
IT=$(aws kinesis get-shard-iterator --stream-name clicks --shard-id
aws kinesis get-records --shard-iterator $IT --query "Records[].Data
```

Ти прочитав записи з початку потоку (TRIM_HORIZON) — і можеш повторити це скільки завгодно разів: це і є replay.

1. Створи bucket `khmarynka-lake-<цифри>`. **Amazon Data Firehose** → **Create Firehose stream**: Source **Amazon Kinesis Data Streams** → `clicks`; Destination **Amazon S3** → твій bucket; Buffer interval **60 секунд**. Create.

2. Запиши ще кілька подій командою з кроку 2. Через 1–2 хвилини в bucket з'явиться об'єкт у префіксі з датою — Firehose склав потік у файл.

Прибери за собою: видали Firehose stream, потім Data stream clicks ; очисти й видали bucket; видали IAM-роль Firehose.

Запам'ятай

Потік — багато споживачів, порядок у shard, **replay**; **черга** — один обробник, без replay.

KDS: shard = запис **1 MB/с** або **1 000 записів/с**, читання **2 MB/с**; **enhanced fan-out** — 2 MB/с кожному споживачу; зберігання **24 год – 365 днів**; **on-demand** або provisioned (split/merge); partition key → shard; гарячий shard → кращий ключ.

Firehose: доставка в **S3, Redshift, OpenSearch, Splunk, HTTP, Iceberg**; буферизація → **near real-time**; Lambda-трансформація, **JSON** → **Parquet/ORC**, стиснення, динамічне партиціювання; **без replay**.

Flink — вікна часу, аномалії в реальному часі. **Kinesis Video Streams** — відео. **MSK** — керований Kafka (Serverless, Connect).

Як питають на іспиті

Потік кліків чи IoT у реальному часі, кілька застосунків-споживачів, повторне читання → **Kinesis Data Streams**

Складати потокові дані в S3, Redshift або OpenSearch без коду → **Amazon Data Firehose**

Перетворити потік JSON у Parquet перед збереженням у S3 → **Firehose з конвертацією формату**

Агрегація у вікнах часу, аномалії в реальному часі → **Managed Service for Apache Flink**

Наявні Kafka-застосунки, мінімум змін → **Amazon MSK**

Відео з камер для аналізу ML → **Kinesis Video Streams (+ Rekognition)**

`ProvisionedThroughputExceededException` на одному shard → **кращий partition key, split shard або on-demand**

Кілька споживачів конкурують за пропускну здатність shard → **enhanced fan-out**

Зберігати події потоку рік для перерахування → **KDS з retention до 365 днів**

Потрібно обробити кожну подію рівно одним воркером і згладити піки → **SQS (не Kinesis)**

Пастки

Firehose — **near real-time** (буферизація) і **без replay**.

Порядок у Kinesis — лише **в межах shard** (однаковий partition key).

MSK — не serverless за замовчуванням: є брокери (крім MSK Serverless).

Kinesis Data Analytics тепер зветься **Managed Service for Apache Flink**, Kinesis Data Firehose — **Amazon Data Firehose**.

Перевір себе

ПИТАННЯ 1

Компанія збирає клікові події сайту. Три різні застосунки мають обробляти ті самі події в реальному часі незалежно, а в разі помилки — перерахувати події за останні 24 години. Що обрати?

- A** Amazon Data Firehose
- B** Amazon Kinesis Data Streams
- C** Amazon SQS Standard
- D** Amazon SNS

Правильна відповідь: B. Kinesis Data Streams зберігає записи незалежно від читання, дозволяє кільком споживачам читати ті самі дані і перерахувати їх. SQS видає повідомлення одному обробнику, Firehose не має replay, SNS не зберігає повідомлення.

ПИТАННЯ 2

Потрібно доставляти журнали застосунку в S3 у форматі Parquet з мінімальними операційними зусиллями; затримка в кілька хвилин прийнятна. Що обрати?

- A** Kinesis Data Streams з власним споживачем на EC2
- B** Amazon Data Firehose з конвертацією формату в Parquet
- C** Amazon MSK з Kafka Connect
- D** Amazon SQS з Lambda, що пише файли

Правильна відповідь: B. Firehose — керована доставка в S3 з буферизацією і вбудованою конвертацією JSON у Parquet/ORC, без коду і серверів.

ПИТАННЯ 3

Потік Kinesis з 10 shards отримує помилки `ProvisionedThroughputExceededException`, хоча загальне навантаження вдвічі нижче за ємність. Усі записи мають `partition key` = назва країни, а 90% трафіку — з однієї країни. Що зробити?

- A** Додати ще 10 shards
- B** Увімкнути enhanced fan-out
- C** Використати `partition key` з високою кардинальністю (наприклад, `sessionId`)
- D** Збільшити `retention` до 7 днів

Правильна відповідь: C. Нерівномірний ключ заганяє більшість записів в один shard. Ключ з багатьма значеннями розподіляє навантаження. Додаткові shards не допоможуть, поки ключ той самий.

ПИТАННЯ 4

Банк хоче виявляти підозрілі транзакції за 10 секунд, аналізуючи потік у вікнах часу («понад 5 оплат з однієї картки за хвилину»). Що використати?

- A** Amazon Data Firehose з доставкою в S3
- B** AWS Glue ETL щоночі
- C** Amazon Managed Service for Apache Flink, що читає Kinesis Data Streams
- D** Amazon Athena за розкладом щогодини

Правильна відповідь: C. Flink обробляє потік у реальному часі з агрегаціями у вікнах часу. Athena і Glue — пакетна аналітика, Firehose — доставка.

ПИТАННЯ 5

Компанія має десятки мікросервісів на Apache Kafka і хоче перенести кластер в AWS з мінімальними змінами. Що обрати?

- A** Amazon SQS FIFO
- B** Amazon MSK
- C** Amazon MQ
- D** Amazon Kinesis Data Streams

Правильна відповідь: B. MSK — керований Apache Kafka, сумісний з наявними клієнтами і інструментами. Kinesis вимагав би переписати виробників і споживачів.

ПИТАННЯ 6

Кілька споживачів читають потік Kinesis, і їм не вистачає пропускнуєї здатності читання. Як дати кожному споживачу власні 2 MB/c на shard?

- A** Зменшити retention
- B** Використати SQS замість Kinesis
- C** Перейти на Firehose
- D** Увімкнути enhanced fan-out для споживачів

Правильна відповідь: D. Enhanced fan-out надсилає дані кожному зареєстрованому споживачу окремим каналом 2 MB/c на shard, без конкуренції за спільні 2 MB/c.

ПИТАННЯ 7

Які твердження про Amazon Data Firehose правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** Дозволяє споживачам перерахувати дані за останні 7 днів
- B** Доставляє дані в S3, Redshift, OpenSearch і HTTP-endpoints
- C** Може перетворювати записи через Lambda перед доставкою
- D** Гарантує доставку за мілісекунди без буферизації
- E** Вимагає керувати shards вручну

Правильна відповідь: B, C. Firehose трансформує (Lambda), конвертує формати і доставляє в сховища та сервіси. Він буферизує дані (near real-time), не зберігає їх для replay і не має shards для ручного керування.



Аналітика — озеро даних, Glue, Athena, Redshift, EMR, Lake Formation, Quick Sight

🎯 Продуктивність · 24%

🎯 Вартість · 20%

🎯 Безпека · 30%

🎬 **Історія Хмаринки.** Марта щомісяця будує звіт «продажі за рік по містах і категоріях» — і щоразу оформлення замовлень на сайті помітно гальмує: важкий запит іде просто в основну базу Aurora. Маркетинг хоче поєднати **події сайту** (їх уже складає в S3 Firehose з розділу 38) із **замовленнями** і витратами на рекламу. А Олена просить **дашборд на телефоні**: виручка сьогодні, топ-товари, конверсія. Час будувати **аналітику**, окрему від «касового апарата» — OLTP-бази (про OLTP і OLAP — у розділі 4).



Образ. Озеро даних (S3) — величезне дешеве озеро, куди звозять усе як є. **Glue** — бібліотекар: складає каталог «що де лежить» і перепаковує сире в зручні формати. **Athena** — рибалка з вудкою: закидає SQL-запит прямо в озеро і платить за кожен витягнутий кілограм (прочитані байти). **Redshift** — сучасний склад-холодильник: дані заздалегідь розкладені по колонках на полицях, тож важкі щоденні звіти летять. **EMR** — важкий завод, де Spark перемелює терабайти. **Lake Formation** — охорона з перепустками до окремих ділянок озера. **Quick Sight** — вітрина з графіками для керівників.

Хто що робить в аналітиці

Роль	Сервіс AWS	Одним рядком
Озеро даних	S3	Дешево, безмежно, будь-які формати; зони raw → clean → curated
Каталог і ETL	AWS Glue	Опис таблиць (Data Catalog), crawlers, ETL на Spark без серверів
SQL по файлах у S3	Amazon Athena	Serverless, платиш за прочитані дані
Сховище даних (DWH)	Amazon Redshift	Колонкове, MPP, петабайти, BI і важкі регулярні звіти
Великі дані на фреймворках	Amazon EMR	Spark, Hadoop, Hive, Presto, Flink — кластери або serverless
Доступ до даних в озері	AWS Lake Formation	Права до баз, таблиць, колонок, рядків; обмін між акаунтами
Дашборди	Amazon Quick Sight	BI для бізнесу, serverless
Пошук і аналіз логів	OpenSearch	Повнотекстовий пошук, дашборди — розділ 37
Потоки	Kinesis, Firehose, MSK	Реальний час — розділ 38
Чужі датасети	AWS Data Exchange	Купити чи отримати дані постачальників

AWS Glue: каталог і ETL без серверів

AWS Glue — serverless-сервіс інтеграції даних. Три головні частини:

- **Glue Data Catalog** — центральний **каталог метаданих**: бази, таблиці, колонки, типи, партиції, місце в S3. Сумісний з Hive metastore. Його використовують **Athena, Redshift Spectrum, EMR, Lake Formation** — описав таблицю один раз, і всі її бачать.
- **Crawlers** — «павуки», що проходять по S3 (а також по базах через JDBC і DynamoDB), **самі визначають схему** і створюють чи оновлюють таблиці та **нові партиції** в каталозі. Запускаються за розкладом.

- **ETL-задачі (jobs)** — перетворення даних на **Apache Spark** (PySpark, Scala), Python shell або Ray: очистити, об'єднати, **конвертувати CSV/JSON у Parquet**, розкласти по партиціях. Серверів немає — платиш за **DPU-години** роботи. **Glue Studio** — візуальний редактор задач.

Корисні дрібниці, які люблять на іспиті:

- **Job bookmarks** — задача пам'ятає, що вже обробила, і наступного запуску бере **лише нові файли**.
- **Glue Streaming ETL** — безперервна обробка з Kinesis або Kafka.
- **Glue DataBrew** — **візуальна підготовка даних без коду** (понад 250 готових перетворень) для аналітиків.
- **Glue Data Quality** — правила якості даних («сума не від'ємна», «email не порожній»); **виявлення чутливих даних (PII)**.
- **Triggers і workflows** — запуск задач за розкладом, подіями або ланцюжком.

Amazon Athena: SQL прямо по S3

Amazon Athena — **serverless інтерактивний SQL** (рушій Trino/Presto) по файлах у S3: CSV, JSON, **Parquet**, **ORC**, Avro, а також таблиці **Apache Iceberg** (з UPDATE, DELETE і «подорожжю в часі»). Таблиці описані в Glue Data Catalog. Нічого не завантажувеш і не запускаєш — пишеш запит і отримуєш відповідь.

- **Ціна: \$5 за ТВ прочитаних даних** (мінімум 10 MB на запит), або **provisioned capacity** (оплата за DPU-години) для передбачуваних навантажень.
- **Типові задачі:** разові запити до **журналів у S3** — CloudTrail, VPC Flow Logs, ALB, CloudFront, S3 access logs; дослідження даних в озері.
- **Workgroups** — розділення команд і запитів: окремі налаштування, історія, **ліміти прочитаних даних на запит** для контролю витрат.

- **Federated Query** — SQL до інших джерел через конектори на **Lambda**: DynamoDB, RDS, Redshift, CloudWatch Logs, бази on-premises — і JOIN з даними в S3.
- **Athena for Apache Spark** — інтерактивні Spark-ноутбуки без кластерів.

Оскільки платиш за прочитані байти, головне в Athena — **читати менше**:

ЗАПИТ: СУМА ПРОДАЖІВ ЗА 15.09

CSV — по рядках, без партицій

id	місто	день	сума

Щоб знайти 15.09,
читає кожен рядок
цілком — усі колонки

Прочитано: 100%

Parquet — по колонках + партиції

	id	місто	сума
day=14/			
day=15/			
day=16/			

Інші дні — інші
папки: пропущено.

З потрібної папки —
лише колонка «сума»

Прочитано: ≈ 11%

Колонковий формат читає лише потрібні колонки, партиції (папки на кшталт day=15/) відсікають зайві дні, а стиснення зменшує байти ще в кілька разів. На справжніх обсягах різниця — у сотні й тисячі разів.

Чотири прийоми, що здешевлюють і пришвидшують Athena:

1. **Колонкові формати Parquet або ORC** — читаються лише потрібні колонки.
2. **Стиснення (Snappy, ZSTD, GZIP)** — менше байтів.
3. **Партиції** (`year=2026/month=09/day=15/`) і фільтр у `WHERE` по них — зайві папки не читаються. Нові партиції додає crawler, команда `MSCK REPAIR TABLE` або `partition projection`.
4. **Менше, але більших файлів** (приблизно від 128 MB) — тисячі дрібних файлів гальмують.

Конвертувати дані можна Glue ETL-задачею, запитом **CTAS** в Athena (`CREATE TABLE AS SELECT` у Parquet), а потоки — одразу в Firehose.



Скільки коштує запит до журналів

Хмаринка тримає **2 TB журналів за рік у CSV**. Запит «помилки за вчора»:

- **CSV без партицій:** Athena читає всі 2 TB → $2 \times \$5 = \10 за один запит.
- **Parquet зі стисненням** ($\approx$ у 4 рази менше): 0,5 TB. Партиції за днем: лише 1 день з 365 → $\approx 1,4$ GB. Лише **3 колонки з 20:** $\approx 0,2$ GB → $\approx \$0,001$.

Та сама відповідь — у **тисячі разів дешевше** і в десятки разів швидше.



Новинки, про які варто знати

Amazon S3 Tables (з грудня 2024) — таблиці **Apache Iceberg** прямо в S3 (окремий тип — *table buckets*) з автоматичним ущільненням дрібних файлів; запити з Athena, Redshift, EMR. **Zero-ETL** — готові інтеграції, що без власних конвеєрів копіюють дані з **Aurora, RDS і DynamoDB** у **Redshift** майже в реальному часі.

Amazon Redshift: сховище даних

Amazon Redshift — кероване **сховище даних** (data warehouse) для **OLAP**: складні SQL-запити з JOIN і агрегаціями по мільярдах рядків, BI-дашборди, сотні аналітиків. Основа — PostgreSQL-подібний SQL, але всередині все інакше:

- **Колонкове зберігання + стиснення** — запит читає лише потрібні колонки.
- **MPP** (massively parallel processing) — **leader node** приймає запит і складає план, **compute nodes** виконують частини паралельно, кожен над своїм шматком даних.
- **RA3** — вузли з **Redshift Managed Storage**: дані лежать у S3-подібному сховищі, а гарячі — в локальному SSD-кеші, тож **обчислення і сховище масштабуються окремо**.
- **Redshift Serverless** — без кластерів: потужність у **RPU** підлаштовується сама, платиш лише під час роботи запитів. Ідеально для нерегулярних навантажень.

Як дані потрапляють у Redshift:

- **COPY з S3** — найшвидший спосіб: паралельне завантаження з багатьох файлів (а не тисячі окремих INSERT).
- **Amazon Data Firehose** (через S3 + COPY), **streaming ingestion** напряму з Kinesis Data Streams і MSK.
- **Zero-ETL-інтеграції** з Aurora, RDS і DynamoDB — дані з'являються в Redshift за секунди, без власного ETL.
- **AWS DMS** — з інших баз (розділ 33).

Можливості, які впізнаєш у питаннях:

- **Redshift Spectrum** — запити з Redshift **до даних у S3 без завантаження** (зовнішні таблиці з Glue Data Catalog). Гаряче — у Redshift, архів за 5 років — у S3, а JOIN — в одному запиті.

- **Concurrency Scaling** — під час піку одночасних запитів Redshift на льоту додає тимчасову потужність, щоб запити не стояли в черзі.
- **Data sharing** — живий доступ до даних **іншого кластера чи акаунта без копіювання**.
- **Materialized views, Redshift ML** (моделі SageMaker AI через SQL), **керування навантаженням (WLM)** — черги й пріоритети запитів.

Надійність і безпека:

- **Знімки (snapshots)** в S3: автоматичні (інкрементальні, приблизно кожні 8 годин або кожні 5 GB змін на вузол; зберігаються від 1 дня до 35 днів) і ручні (до видалення). **Копіювання знімків в інший регіон** автоматично — основа DR. Відновлення — у **новий** кластер.
- **Multi-AZ** для RA3 — кластер у двох AZ з автоматичним перемиканням.
- Шифрування **KMS**, розміщення у **VPC**, **Enhanced VPC Routing** — трафік COPY і UNLOAD іде через твою VPC (VPC endpoints, flow logs), а не через інтернет.



Глибше: чому Redshift швидкий і як не зробити його повільним

Дані таблиці розкладаються по вузлах за **стилем розподілу** (distribution style): **KEY** — рядки з однаковим ключем на одному вузлі (JOIN без пересилання даних), **ALL** — маленький довідник копіюється на кожен вузол, **EVEN** — рівномірно, **AUTO** — Redshift обирає сам. **Sort key** впорядковує дані на диску, і запит «за останній тиждень» пропускає блоки з іншими датами (zone maps). Погано обраний ключ розподілу змушує вузли пересилати дані одне одному — запит гальмує. Для іспиту SAA досить знати, що Redshift — колонковий MPP для OLAP, а не для тисяч дрібних транзакцій.

Amazon EMR: Spark і Hadoop

Amazon EMR — керовані кластери фреймворків великих даних: **Apache Spark, Hadoop, Hive, HBase, Presto/Trino, Flink**. Для задач, де потрібен контроль над фреймворком і його налаштуваннями, або для міграції наявного Hadoop з on-premises.

- **Вузли:** **primary** (керує кластером), **core** (виконують задачі і зберігають HDFS), **task** (лише обчислення, без даних).
- **Економія:** **task-вузли на Spot** — якщо AWS забере інстанс, дані не втрачаються. Primary і core — On-Demand або Reserved. **Instance fleets** змішують типи і способи оплати.
- **Сховище:** HDFS на дисках вузлів (зникає з кластером) або **EMRFS** — дані в S3. З S3 можна запускати **тимчасові (transient) кластери**: піднявся, обробив, вимкнувся — платиш лише за години роботи.
- **EMR Serverless** — Spark і Hive без керування кластерами. **EMR on EKS** — Spark у твоєму Kubernetes-кластері.

Glue чи EMR? Glue — serverless ETL, мінімум налаштувань. EMR — повний контроль, багато фреймворків, великі довгі задачі, наявний код Hadoop.

AWS Lake Formation: хто що бачить в озері

Коли в озері лежать і замовлення, і телефони покупців, і фінанси, постає питання: **хто що бачить**. IAM і політики S3 працюють на рівні **файлів і префіксів** — вони не вміють «показати таблицю, але без колонки phone».

AWS Lake Formation вміє:

- **централізовані права** на рівні **бази, таблиці, колонки, рядка і навіть клітинки** — видаються як GRANT у базі даних;
- **LF-Tags** — дозволи за тегами (`confidentiality=pii`), щоб не прописувати права до кожної з сотень таблиць;
- **обмін даними між акаунтами** без копіювання;

- права діють однаково для **Athena, Redshift Spectrum, EMR, Glue і Quick Sight**;
- побудований **поверх Glue Data Catalog**; допомагає швидко наповнити озеро (blueprints для завантаження з баз і журналів).

У Хмаринці маркетинг бачить таблицю `orders` **без** колонок `phone` і `address`, Марта бачить усе, а партнерська агенція в іншому акаунті — лише агреговану вітрину.

Amazon Quick Sight: дашборди

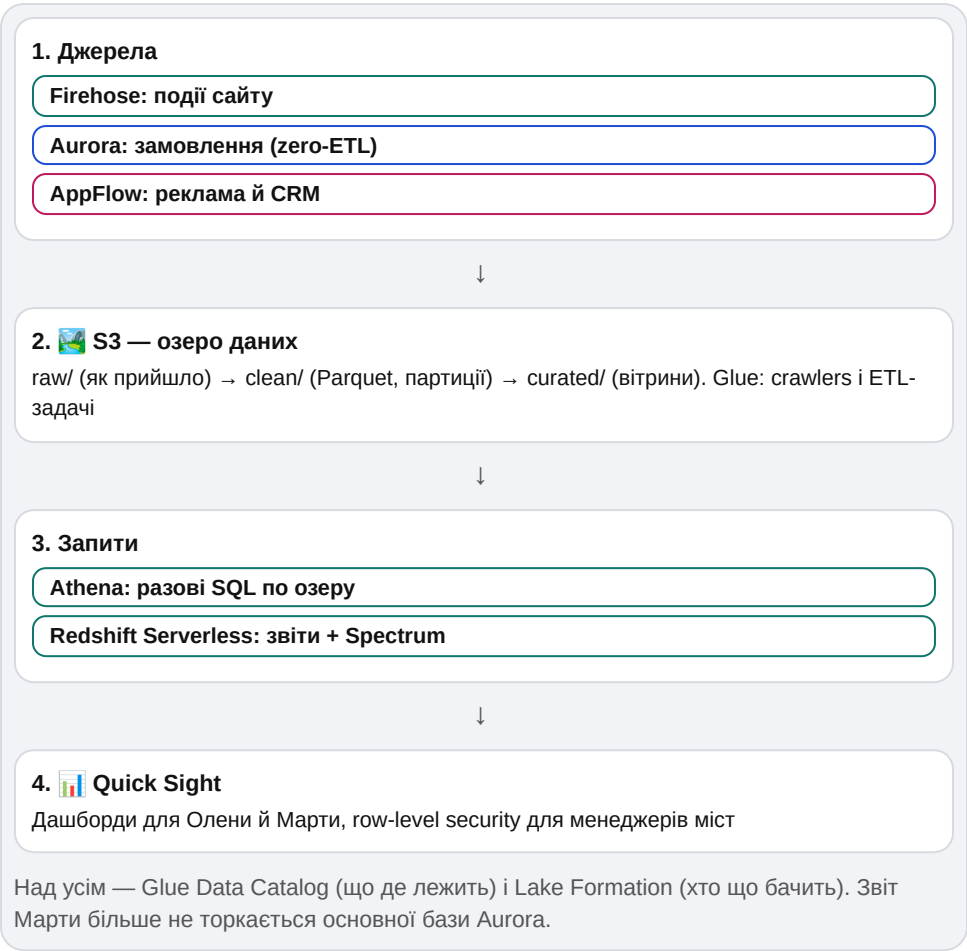
Amazon Quick Sight (раніше — **Amazon QuickSight**; з жовтня 2025 — частина набору Amazon Quick Suite, а з 2026 — просто **Amazon Quick**) — serverless **BI**: інтерактивні дашборди й звіти для бізнесу. На іспиті він може називатися QuickSight.

- **Джерела**: Athena, Redshift, RDS/Aurora, S3, OpenSearch, Salesforce та інші.
- **SPICE** — вбудований рушій у пам'яті: дані імпортуються, і дашборди відкриваються миттєво, не навантажуючи джерело.
- **Row-level security** — менеджер Львова бачить лише Львів; **вбудовування** дашбордів у сайт чи застосунок; **ML-інсайти** (аномалії, прогнози) і запитання природною мовою.
- Користувачі й групи самого сервісу (з інтеграцією IAM Identity Center); оплата — за користувачів.

AWS Data Exchange

AWS Data Exchange — каталог **сторонніх датасетів**: погода, демографія, фінансові ринки, курси валют. Підписуєшся — і файли надходять у твій S3 (або доступні через API чи Redshift data sharing). Хмаринка підписується на прогноз погоди, щоб порівняти продажі свічок із холодними вечорами.

Архітектура аналітики Хмаринки



Що обрати

Потреба	Сервіс
Разові SQL-запити до файлів і журналів у S3, без серверів	Athena

Потреба	Сервіс
Зменшити вартість запитів Athena	Parquet/ORC + стиснення + партиції
Важкі регулярні звіти, BI, петабайти структурованих даних	Redshift (нерегулярно — Redshift Serverless)
Запити з Redshift до даних у S3 без завантаження	Redshift Spectrum
Аналітика даних Aurora/ДynamoDB майже в реальному часі без ETL	Zero-ETL-інтеграція з Redshift
Spark/Hadoop, міграція Hadoop, контроль фреймворку	EMR (task-вузли на Spot)
ETL без серверів, автоматичне визначення схеми	Glue (jobs + crawlers)
Обробляти лише нові файли в ETL	Glue job bookmarks
Аналітики готують дані без коду	Glue DataBrew
Права до колонок і рядків в озері, обмін між акаунтами	Lake Formation
Дашборди для бізнесу	Quick Sight
SQL-запит одночасно до DynamoDB, RDS і S3	Athena Federated Query
Датасети сторонніх постачальників	Data Exchange



Спробуй у справжньому AWS: Athena, CSV → Parquet і різниця в байтах

Вартість: менше цента (Athena бере мінімум 10 MB на запит — це \$0,00005). **Час:** 25 хвилин. Регіон — **eu-central-1**.

1. **CloudShell** — згенеруй 5 000 замовлень і поклади в S3:

```

B=khmarynka-lake-$RANDOM; echo $B
aws s3 mb s3://$B --region eu-central-1
python3 - <<'PY' > orders.csv
import random
print("order_id,city,category,amount,day")
cities = ["Lviv", "Kyiv", "Odesa", "Kharkiv", "Dnipro"]
cats = ["candles", "mugs", "textile", "toys"]
for i in range(1, 5001):
    print(f"{i},{random.choice(cities)},{random.choice(cats)},{rand
PY
aws s3 cp orders.csv s3://$B/raw/orders/orders.csv

```

1. **Athena → Query editor.** Якщо Athena просить місце для результатів — у налаштуваннях вкажи `s3://<твій bucket>/athena-results/` (або обери керовані результати запитів). Виконай по черзі (заміни BUCKET):

```

CREATE DATABASE khmarynka;

CREATE EXTERNAL TABLE khmarynka.orders (
    order_id int, city string, category string, amount double, day st
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LOCATION 's3://BUCKET/raw/orders/'
TBLPROPERTIES ('skip.header.line.count'='1');

SELECT city, round(sum(amount), 2) AS revenue
FROM khmarynka.orders GROUP BY city ORDER BY revenue DESC;

```

1. Перетвори таблицю в Parquet з партиціями за днем (CTAS):

```

CREATE TABLE khmarynka.orders_pq
WITH (format = 'PARQUET', write_compression = 'SNAPPY',
    external_location = 's3://BUCKET/clean/orders/',
    partitioned_by = ARRAY['day'])
AS SELECT order_id, city, category, amount, day FROM khmarynka.orde

```

1. Порівняй **Data scanned** під результатом двох запитів:

```
SELECT sum(amount) FROM khmarynka.orders    WHERE day = '2026-09-15'
SELECT sum(amount) FROM khmarynka.orders_pq WHERE day = '2026-09-15'
```

Відповідь однакова, а прочитано в десятки разів менше. Відкрий bucket: `y clean/orders/` — папки `day=2026-09-01/ ... day=2026-09-30/`. **Прибери за собою:** `DROP TABLE khmarynka.orders_pq; DROP TABLE khmarynka.orders; DROP DATABASE khmarynka;` і в CloudShell `aws s3 rb s3://$B --force`.

Запам'ятай

S3 — озеро даних; **Glue Data Catalog** — спільний каталог для Athena, Redshift Spectrum, EMR, Lake Formation; **crawlers** визначають схему і партиції; **Glue jobs** — serverless ETL на Spark; **job bookmarks** — лише нові дані; **DataBrew** — без коду.

Athena — serverless SQL по S3, **\$5 за TB прочитаного**; дешевше — **Parquet/ORC**, стиснення, партиції, великі файли; **workgroups** — ліміти й розділення команд; **Federated Query** — інші джерела через Lambda.

Redshift — колонкове **MPP**-сховище для **OLAP**; **RA3** (сховище окремо від обчислень), **Serverless**; **COPY з S3**, **zero-ETL**, streaming ingestion; **Spectrum** — S3 без завантаження; **Concurrency Scaling**; **data sharing**; знімки з **копією в інший регіон**; **Enhanced VPC Routing**.

EMR — Spark/Hadoop; **task-вузли на Spot**; **EMRFS (S3)** і тимчасові кластери; EMR Serverless, EMR on EKS.

Lake Formation — права до **колонок, рядків, клітинок, LF-Tags**, обмін між акаунтами. **Quick Sight** — дашборди, **SPICE**, row-level security. **Data Exchange** — сторонні датасети.

Як питають на іспиті

SQL по журналах у S3 без серверів, разово → **Athena**

Зменшити вартість і пришвидшити запити Athena → [Parquet/ORC + стиснення + партиції](#)

Складні аналітичні запити, BI, петабайти, стабільна продуктивність → [Redshift](#)

Запити з Redshift до архіву в S3 без завантаження → [Redshift Spectrum](#)

Аналітика даних Aurora в Redshift майже в реальному часі з мінімумом зусиль → [Aurora zero-ETL integration](#)

Hadoop/Spark з on-premises в AWS, дешевше → [EMR, task-вузли на Spot](#)

Serverless ETL і автоматичне визначення схеми даних у S3 → [Glue ETL + crawler + Data Catalog](#)

ETL щоночі переробляє всі файли, треба лише нові → [Glue job bookmarks](#)

Бізнес-аналітики очищають дані візуально без коду → [Glue DataBrew](#)

Права до окремих колонок і рядків в озері, обмін між акаунтами → [Lake Formation](#)

Дашборди для керівників, serverless, оплата за користувачів → [Quick Sight](#)

SQL-запит, що поєднує DynamoDB, RDS і S3 → [Athena Federated Query](#)

DR для Redshift в іншому регіоні → [автоматичне копіювання знімків у інший регіон](#)

COPY/UNLOAD має йти через VPC, а не інтернет → [Enhanced VPC Routing](#)

Пастки

Redshift — для **OLAP**, не для тисяч дрібних транзакцій (це RDS, Aurora, DynamoDB).

Athena бере гроші за **прочитані** байти, а не за результат: `SELECT *` без фільтра по партиціях — найдорожчий запит.

На **Spot** — лише **task**-вузли EMR; primary і core на Spot ризикують кластером і даними HDFS.

IAM і bucket policies не дають прав на рівні **колонок** таблиці — це Lake Formation.

QuickSight тепер називається **Amazon Quick Sight** (у складі Amazon Quick), але на іспиті можлива стара назва.

✓ Перевір себе

ПИТАННЯ 1

Команді безпеки потрібно час від часу виконувати SQL-запити до журналів CloudTrail, що зберігаються в S3. Потрібне рішення без серверів і з мінімальними витратами. Що обрати?

- A** Amazon Redshift з щоденним завантаженням журналів
- B** Amazon RDS for PostgreSQL з імпортом журналів
- C** Amazon EMR з постійним кластером Hive
- D** Amazon Athena з таблицею в Glue Data Catalog

Правильна відповідь: D. Athena виконує SQL прямо по файлах у S3 без інфраструктури і бере гроші лише за прочитані дані — ідеально для нерегулярних запитів до журналів.

ПИТАННЯ 2

Запити Athena до таблиці з журналами (CSV, 3 TB) майже завжди фільтрують за датою, але дорогі й повільні. Що дасть найбільший ефект?

- A** Створити окремий workgroup для кожного аналітика
- B** Перенести файли в S3 Glacier Flexible Retrieval
- C** Розбити файли на мільйони дрібних файлів
- D** Конвертувати дані в Parquet зі стисненням і партиціювати за датою

Правильна відповідь: D. Колонковий формат читає лише потрібні колонки, стиснення зменшує байти, партиції відсікають непотрібні дати. Glacier не підходить для прямих запитів, а дрібні файли лише гальмують.

ПИТАННЯ 3

Відділ BI щодня виконує складні запити з JOIN по кількох терабайтах даних продажів для сотень дашбордів і потребує стабільної продуктивності. Що обрати?

- A** Amazon Athena
- B** Репліку для читання Amazon RDS
- C** Amazon Redshift
- D** Amazon DynamoDB

Правильна відповідь: C. Redshift — колонкове MPP-сховище даних, створене саме для важких регулярних аналітичних запитів і BI.

ПИТАННЯ 4

Аналітики працюють у Redshift, але хочуть інколи поєднувати поточні дані з архівом за 5 років, що лежить у S3 у форматі Parquet, без завантаження архіву в кластер. Що використати?

- A** Amazon Redshift Spectrum
- B** Redshift Concurrency Scaling
- C** AWS Glue DataBrew
- D** Redshift data sharing

Правильна відповідь: A. Redshift Spectrum виконує запити до даних у S3 через зовнішні таблиці (Glue Data Catalog) і дозволяє поєднувати їх з таблицями Redshift в одному запиті.

ПИТАННЯ 5

Компанія переносить кластер Hadoop/Spark з on-premises і хоче мінімізувати вартість, не ризикуючи даними. Як налаштувати Amazon EMR?

- A** Core-вузли на Spot, щоб HDFS був дешевшим
- B** Усі вузли, включно з primary, — на Spot
- C** Primary і core вузли — On-Demand або Reserved, task-вузли — Spot, дані — в S3 (EMRFS)
- D** Замінити EMR на Glue DataBrew

Правильна відповідь: C. Task-вузли не зберігають дані, тож їх втрата безпечна — ідеальні для Spot. Primary і core на Spot ризикують кластером і даними HDFS; S3 через EMRFS зберігає дані незалежно від кластера.

ПИТАННЯ 6

Нічна задача AWS Glue щоразу обробляє всі файли в bucket, хоча нові з'являються лише за останню добу. Як обробляти тільки нові дані?

- A** Збільшити кількість DPU
- B** Перейти на Athena CTAS
- C** Увімкнути job bookmarks
- D** Запускати crawler частіше

Правильна відповідь: C. Job bookmarks зберігають стан обробки, і наступний запуск бере лише нові дані.

ПИТАННЯ 7

В озері даних на S3 маркетинг має бачити таблицю замовлень без колонок з телефонами й адресами, а фінансовий відділ — усі колонки. Також потрібно надати доступ іншому акаунту. Яке рішення найпростіше?

- A** Окремі workgroups Athena для кожного відділу
- B** Окремі копії даних для кожного відділу в різних buckets
- C** AWS Lake Formation з правами на рівні колонок
- D** Політики S3 bucket з умовами на префікси

Правильна відповідь: C. Lake Formation надає централізовані права до баз, таблиць, колонок і рядків для Athena, Redshift Spectrum, EMR і дозволяє ділитися даними між акаунтами. Політики S3 працюють лише на рівні об'єктів і префіксів.

ПИТАННЯ 8

Компанія хоче аналізувати дані зі своєї бази Aurora MySQL у Redshift майже в реальному часі, не будуючи й не підтримуючи ETL-конвеєри. Що обрати?

- A** AWS Glue job щохвилини
- B** Zero-ETL-інтеграцію Aurora з Amazon Redshift
- C** Реплікацію через Kinesis Data Streams з власним кодом
- D** Щогодинний експорт знімків Aurora в S3 і COPY

Правильна відповідь: B. Zero-ETL-інтеграція автоматично і безперервно реплікує дані з Aurora в Redshift за секунди, без конвеєрів, які треба будувати й обслуговувати.

ПИТАННЯ 9

Які дії зменшать вартість запитів Amazon Athena? (Оберіть 2)

Оберіть 2 відповіді

- A** Партиціювати дані і фільтрувати по партиціях у WHERE
- B** Зберігати дані в тисячах файлів по кілька КБ
- C** Використовувати SELECT * у всіх запитах
- D** Перейти на S3 Glacier Deep Archive
- E** Зберігати дані в колонковому форматі Parquet або ORC

Правильна відповідь: A, E. Athena бере гроші за прочитані байти: колонковий формат і партиції зменшують обсяг читання. SELECT * і дрібні файли погіршують ситуацію, а Glacier Deep Archive не доступний для прямих запитів.



Штучний інтелект — готові AI-сервіси, SageMaker AI і генеративний AI



Продуктивність · 24%



Безпека · 30%



Історія Хмаринки. Покупці завантажують фото своїх інтер'єрів з товарами Хмаринки — і хтось уже виклав неприйнятну картинку. Марта щодня вручну переписує дані з сотень PDF-рахунків постачальників. Служба підтримки тоне в дзвінках і відгуках польською, англійською й українською, а Олена хоче знати, **про що** скаржаться. Тарас не має команди data scientists — і не потребує її: більшість таких задач AWS розв'язує **готовими AI-сервісами**, що викликаються одним API-запитом.



Образ. Готові AI-сервіси — це **органи чуття** застосунку. **Rekognition** — очі. **Textract** — бухгалтер, що читає документи. **Comprehend** — читач, що розуміє настрої і сенс тексту. **Transcribe** — вуха (мова → текст). **Polly** — голос (текст → мова). **Translate** — перекладач. **Lex** — співрозмовник. А **SageMaker AI** — лабораторія, де вирощують **власний мозок**, коли готових органів замало. **Bedrock** — бюро найманих «геніїв»: великі генеративні моделі напрокат через API.

Три рівні AI в AWS

Рівень	Що це	Хто користується	Приклади
Готові AI-сервіси	Навчені моделі за API: надсилаєш дані — отримуєш результат	Будь-який розробник, без знань ML	Rekognition, Textract, Comprehend, Transcribe, Polly, Translate, Lex, Kendra
Генеративний AI	Великі базові моделі (foundation models) за API	Розробники застосунків	Amazon Bedrock
Платформа ML	Будуєш, навчаєш і розгортаєш власні моделі	Data scientists, ML-інженери	Amazon SageMaker AI

Правило іспиту: якщо задачу можна розв'язати готовим сервісом — обирай **готовий сервіс** (найменше операційних зусиль). SageMaker AI — коли потрібна **своя** модель на своїх даних.

Готові AI-сервіси: органи чуття

Rekognition — бачить

Об'єкти й сцени на фото і відео, обличчя (порівняння, пошук у колекції), текст на зображенні, **модерація** неприйняттого контенту, засоби захисту (PPE), Custom Labels — свої об'єкти.

Відео наживо — разом з Kinesis Video Streams

Textract — читає документи

Текст, **форми** (пари «поле — значення») і **таблиці** зі сканів і PDF; рахунки й чеки (AnalyzeExpense), посвідчення (AnalyzeID).

Великі документи — асинхронно, з повідомленням у SNS

Comprehend — розуміє текст

Тональність, сутності (імена, місця, дати), ключові фрази, мова, теми, **персональні дані (PII)**, власні класифікатори.

Comprehend Medical — медичні тексти

Transcribe — чує

Мова → текст: дзвінки, відео, субтитри; розпізнає мовців, **приховує PII**, власний словник термінів; Call Analytics для кол-центрів.

Пакетно і в реальному часі

Polly — говорить

Текст → природне мовлення: нейронні голоси, SSML (паузи, наголоси), лексикони для вимови брендів.

Озвучення статей, голосові меню

Translate — перекладає

Машинний переклад у реальному часі й пакетно, власна термінологія (назви товарів не перекладаються).

Локалізація сайту, переклад звернень

Lex — розмовляє

Чат-боти й голосові боти: розпізнавання мови і наміру (intents, slots), виконання дій через Lambda.

Та сама технологія, що в Alexa; інтеграція з Amazon Connect

Kendra — шукає відповіді

Інтелектуальний пошук по документах компанії запитаннями природною мовою: S3, SharePoint, Confluence, бази знань.

«Яка політика повернення товару?» → точна відповідь з документа

Запам'ятовуй за органом чуття: очі, бухгалтер, читач, вуха, голос, перекладач, співрозмовник, бібліотекар. Два сервіси з пари «навіпаки»: Transcribe (мова → текст) і Polly (текст → мова).

Ще кілька імен, що трапляються у варіантах відповідей:

- **Amazon Personalize** — персональні **рекомендації** в реальному часі («з цим товаром купують...») на основі технологій Amazon.com.
- **Amazon Forecast** — прогноз часових рядів (попит, запаси) і **Amazon Fraud Detector** — виявлення онлайн-шахрайства. ⚠ Обидва **закриті для нових клієнтів** (Forecast — з 2024, Fraud Detector — з листопада 2025); на іспиті ще можуть бути правильною відповіддю, а в нових проєктах замість них — **SageMaker AI** (зокрема no-code **SageMaker Canvas**).
- **Amazon Augmented AI (A2I)** — **людська перевірка** результатів ML з низькою впевненістю: якщо Rekognition не впевнений щодо фото, воно йде модератору.

Як AI-сервіси вбудовуються в архітектуру

Готові сервіси — це регіональні API. Їх викликають з Lambda, контейнерів чи серверів через SDK з правами IAM. Три типові конвеєри Хмаринки:

МОДЕРАЦІЯ ФОТО

S3 (нове фото) → подія → **Lambda** → **Rekognition** (модерація) → впевнено «ок» — публікуємо; сумнівно — **A2I**, людина вирішує



РАХУНКИ

S3 (PDF) → **Step Functions** → **Textract** (асинхронно, SNS) → поля «постачальник, сума, дата» → **DynamoDB** → Марта лише перевіряє



ГОЛОС КЛІЄНТА

Запис дзвінка → **Transcribe** (текст, PII приховано) → **Translate** (одна мова) → **Comprehend** (тональність, теми) → S3 → Athena → Quick Sight

Жодної власної моделі і жодного сервера: події, Lambda або Step Functions і готові сервіси. Такі рішення мають найменше операційних зусиль — улюблена відповідь іспиту.

Безпека і приватність AI-сервісів:

- Доступ — через **IAM**; приватний виклик з VPC без інтернету — через **VPC interface endpoints (PrivateLink)**.
- Дані шифруються **KMS**; для записів і документів з персональними даними — **приховування PII** (Transcribe, Comprehend).
- Деякі сервіси можуть використовувати вміст для покращення моделей AWS — від цього відмовляються для всієї організації **політикою AWS Organizations «AI services opt-out»**.
- Якщо модель не підтримує потрібну мову, типовий прийом — спершу **Translate**, потім аналіз.

Amazon SageMaker AI: власні моделі

Amazon SageMaker AI (до грудня 2024 — просто Amazon SageMaker; тепер «SageMaker» — ширша платформа даних і AI, а ML-частина зветься SageMaker AI) — повний цикл машинного навчання:

1. **Підготовка даних:** Data Wrangler, **Feature Store** (спільні ознаки для навчання й прогнозів), **Ground Truth** — розмітка даних людьми.
2. **Навчання:** Studio і ноутбуки, **training jobs** на керованих інстансах (зокрема GPU), вбудовані алгоритми, **automatic model tuning**; **managed Spot training** — до 90% дешевше. **Canvas** і **Autopilot** — ML без коду. **JumpStart** — готові моделі, які можна донавчити.
3. **Розгортання (inference)** — чотири варіанти, які люблять на іспиті:

Варіант	Коли	Особливості
Real-time endpoint	Постійний трафік, потрібна мала затримка	Сервери працюють завжди, автомасштабування
Serverless inference	Нерегулярний трафік з паузами	Масштабується до нуля, можливий холодний старт
Asynchronous inference	Великі вхідні дані (до 1 GB), довга обробка (до години)	Черга запитів, результат у S3, може зменшуватися до нуля
Batch transform	Прогнози для всього набору даних офлайн	Без постійного endpoint: запустив, обробив, вимкнулося

1. **MLOps: Pipelines** (конвеєри навчання), **Model Registry** (версії моделей), **Model Monitor** (дрейф даних і якості), **Clarify** (упередженість і пояснення прогнозів).

Генеративний AI: Amazon Bedrock

Amazon Bedrock — serverless-доступ до **великих базових моделей** через єдиний API: Anthropic Claude, Amazon Nova, Meta Llama, Mistral та інші. Жодних GPU і серверів — платиш за токени або резервуєш пропускну здатність.

- **Knowledge Bases** — **RAG** (retrieval-augmented generation): модель відповідає на основі **твоїх документів** (S3, бази знань), а не лише загальних знань.
- **Agents** — модель виконує багатокрокові задачі, викликаючи твої API (наприклад, через Lambda).
- **Guardrails** — фільтри шкідливого контенту, заборонені теми, приховування PII.
- **Приватність**: твої запити і дані **не використовуються для навчання** базових моделей; доступ з VPC через PrivateLink; шифрування KMS.

Хмаринка генерує чернетки описів товарів і будує помічника підтримки, що відповідає за правилами доставки й повернення з бази знань.



Що з цього на іспиті SAA-C03

Офіційний перелік сервісів SAA-C03 містить **Comprehend, Forecast, Fraud Detector, Kendra, Lex, Polly, Rekognition, SageMaker, Textract, Transcribe, Translate**. Bedrock у ньому немає, але знати його варто: у реальних архітектурах 2026 року він усюди, а питання іспиту поступово оновлюються. Глибше генеративний AI розбирає окремий іспит AWS Certified AI Practitioner.

Спробуй у справжньому AWS: очі, читач і голос за 15 хвилин

Вартість: кілька викликів — у межах безкоштовного рівня або центи.

Час: 15 хвилин. Регіон — **eu-central-1** (Франкфурт).

1. **Rekognition** → **Label detection** (демо в консолі): завантаж фото кімнати зі свічками — побачиш мітки «Candle», «Furniture» з відсотком впевненості. Відкрий **Content moderation** — так перевіряють фото покупців.
2. **Comprehend** → **Real-time analysis**: встав англійський відгук «Delivery was late, but the candles smell amazing!» — подивись тональність (найімовірніше **MIXED** — змішана) і сутності.
3. **CloudShell** — те саме з командного рядка, а ще переклад і голос:

```
aws translate translate-text --source-language-code uk --target-language-code en --text "Свічки чудові, але доставка затрималася на три дні"
aws comprehend detect-sentiment --language-code en --text "The candles are great, but the delivery was three days late"
aws polly synthesize-speech --output-format mp3 --voice-id Joanna --text "Your order from Khmarynka is on its way!" order.mp3 && ls
```

Файл `order.mp3` можна завантажити з CloudShell (Actions → Download file) і послухати. **Прибери за собою:** нічого не створено, крім файлу в CloudShell — видали `order.mp3`.

Запам'ятай

Готовий сервіс > власна модель, якщо задача стандартна: менше зусиль, немає серверів.

Rekognition — фото/відео, обличчя, **модерація**; **Textract** — текст, **форми, таблиці** з документів; **Comprehend** — тональність, сутності, **PII**; **Transcribe** — мова → текст; **Polly** — текст → мова; **Translate** — переклад; **Lex** — чат-боти; **Kendra** — пошук по документах; **Personalize** — рекомендації.

Forecast і Fraud Detector закриті для нових клієнтів, але можуть бути в питаннях.

A2I — людина перевіряє сумнівні результати ML.

SageMaker AI — власні моделі: Ground Truth (розмітка), training jobs (**Spot**), Canvas (без коду), inference: **real-time / serverless / async / batch transform**, Model Monitor, Clarify.

Bedrock — базові моделі за API, **Knowledge Bases (RAG)**, Agents, Guardrails; дані не йдуть на навчання моделей.

Приватність: **PrivateLink, KMS**, політика Organizations **AI services opt-out**.

Як питають на іспиті

Перевіряти фото користувачів на неприйнятний контент → **Rekognition (moderation)**

Витягнути поля й таблиці з відсканованих рахунків і форм → **Textract**

Визначити тональність відгуків або знайти персональні дані в тексті → **Comprehend**

Розшифрувати записи дзвінків → **Transcribe (+ Comprehend для аналізу)**

Озвучити текст природним голосом → **Polly**

Перекласти контент сайту кількома мовами → **Translate**

Чат-бот для сайту чи кол-центру → **Lex**

Пошук відповідей по внутрішніх документах природною мовою → **Kendra**

Власна модель на своїх даних, навчання і розгортання → **SageMaker AI**

Прогнози для великого набору даних раз на добу, без постійного endpoint → **SageMaker Batch Transform**

Нерегулярні запити до моделі з паузами, мінімум витрат → [SageMaker Serverless Inference](#)

Великі вхідні файли, обробка до години → [SageMaker Asynchronous Inference](#)

Людина має перевіряти сумнівні прогнози → [Amazon A2I](#)

Заборонити AI-сервісам зберігати контент для покращення моделей у всіх акаунтах → [AI services opt-out policy в Organizations](#)

Пастки

Textract читає документи, а **Rekognition** — зображення загалом (текст на фото він бачить, але не форми й таблиці рахунку).

Transcribe — мова → текст, **Polly** — навпаки. Не плутай.

Comprehend не витягує текст зі сканів — спершу Textract.

SageMaker AI — не «готовий сервіс»: якщо питають про мінімум зусиль для стандартної задачі, відповідь — відповідний готовий сервіс.

✓ Перевір себе

ПИТАННЯ 1

Соцмережа хоче автоматично виявляти неприйнятні зображення, які завантажують користувачі, з мінімальними операційними зусиллями. Що обрати?

- A** Amazon Comprehend
- B** Amazon Textract
- C** Amazon Rekognition з модерацією контенту, викликаний Lambda за подією S3
- D** Власну модель у Amazon SageMaker AI

Правильна відповідь: C. Rekognition має готову модерацію зображень і відео. Подія S3 → Lambda → Rekognition не потребує ні серверів, ні власної моделі.

ПИТАННЯ 2

Бухгалтерія отримує тисячі відсканованих рахунків у PDF і хоче автоматично витягати постачальника, суму і таблицю позицій. Що використати?

- A** Amazon Rekognition
- B** Amazon Comprehend
- C** Amazon Textract
- D** Amazon Transcribe

Правильна відповідь: C. Textract витягує текст, форми і таблиці з документів, а для рахунків має спеціальний аналіз (AnalyzeExpense).

ПИТАННЯ 3

Кол-центр хоче аналізувати записи дзвінків: перетворити мову на текст і визначити тональність розмови. Яка комбінація правильна?

- A** Amazon Transcribe, потім Amazon Rekognition
- B** Amazon Transcribe, потім Amazon Comprehend
- C** Amazon Lex, потім Amazon Translate
- D** Amazon Polly, потім Amazon Comprehend

Правильна відповідь: B. Transcribe перетворює мову на текст, Comprehend аналізує тональність і сутності тексту. Polly робить навпаки — текст у мову.

ПИТАННЯ 4

Співробітники мають знаходити відповіді на запитання природною мовою у тисячах внутрішніх документів у S3 і SharePoint. Що обрати?

A Amazon Comprehend

B Amazon Kendra

C Amazon Lex

D Amazon Athena

Правильна відповідь: B. Kendra — інтелектуальний пошук по документах компанії з відповідями на запитання природною мовою.

ПИТАННЯ 5

Модель SageMaker AI щоночі має робити прогнози для 50 мільйонів записів у S3. Постійний endpoint не потрібен. Який варіант розгортання найдешевший?

A Batch Transform

B Serverless Inference

C Real-time endpoint

D Кілька real-time endpoints з автомасштабуванням

Правильна відповідь: A. Batch Transform обробляє весь набір даних офлайн і вимикається після роботи — не треба платити за постійний endpoint.

ПИТАННЯ 6

Застосунок звертається до моделі кілька разів на годину з довгими паузами. Потрібні мінімальні витрати, невеликий холодний старт допустимий. Що обрати?

- A** SageMaker Asynchronous Inference з вхідними файлами по 1 GB
- B** SageMaker Batch Transform щохвилини
- C** Real-time endpoint на великому GPU-інстансі
- D** SageMaker Serverless Inference

Правильна відповідь: D. Serverless Inference масштабується до нуля між запитами, тож платиш лише за обробку. Холодний старт — прийнятна ціна за економію.

ПИТАННЯ 7

Компанія використовує кілька готових AI-сервісів AWS у 40 акаунтах і хоче гарантувати, що контент не зберігається для покращення моделей AWS. Як зробити це найпростіше?

- A** Шифрувати дані власним ключем KMS в кожному акаунті
- B** Налаштувати SCP, що забороняє всі AI-сервіси
- C** Застосувати політику AI services opt-out в AWS Organizations
- D** Використовувати VPC interface endpoints

Правильна відповідь: C. Політика AI services opt-out в Organizations централізовано відмовляється від використання контенту для покращення сервісів у всіх акаунтах. KMS і PrivateLink захищають дані, але не змінюють цю поведінку.

ПИТАННЯ 8

Які пари «задача → сервіс» правильні? (Оберіть 2)

Оберіть 2 відповіді

- A** Перекласти описи товарів польською → Amazon Translate
- B** Розпізнати мову в записах дзвінків → Amazon Lex
- C** Витягнути таблиці зі сканованих договорів → Amazon Comprehend
- D** Озвучити новини сайту природним голосом → Amazon Polly
- E** Навчити власну модель прогнозу попиту → Amazon Rekognition

Правильна відповідь: A, D. Polly озвучує текст, Translate перекладає. Таблиці з документів — Textract, мова в текст — Transcribe, власні моделі — SageMaker AI.



Моніторинг і керування — CloudWatch, CloudTrail, Config, X-Ray, Systems Manager, CloudFormation



Відмовостійкість · 26%



Безпека · 30%



Продуктивність · 24%



Історія Хмаринки. Ніч «чорної п'ятниці». Сайт гальмує вже годину, а Тарас дізнається про це з гнівних коментарів в Instagram. Уранці — нові загадки: хтось **видалив правило** в Security Group — хто і коли? Один bucket раптом став **публічним** — як ловити таке автоматично? Оформлення замовлення триває 6 секунд — **де саме** гальмує: в API, базі чи платіжному сервісі? І як зайти на сервер, не відкриваючи SSH всьому інтернету? Цей розділ — про **очі, пам'ять і руки** архітектора.



Образ. CloudWatch — приладова панель автомобіля: спідометр (метрики), бортовий журнал (логи) і лампочки, що спалахують при небезпеці (аларми). **CloudTrail** — відеореєстратор охорони: записує, **хто** зайшов і **що зробив**. **Config** — фотоархів кожної кімнати з інспектором, що звіряє стан із правилами. **X-Ray** — рентген, що показує шлях запиту крізь сервіси. **Systems Manager** — пульт керування автопарком серверів. **CloudFormation** — креслення будинку: за ним звести такий самий у будь-якому місті.

Три запитання — три сервіси

Найчастіша плутанина на іспиті — CloudWatch, CloudTrail і Config. Розрізняй їх за запитанням:

Запитання	Сервіс	Приклад
Що відбувається з ресурсами зараз, як вони працюють?	CloudWatch	CPU 95%, 500 помилок за хвилину, у логах «ERROR»
Хто, що і коли зробив через API?	CloudTrail	Користувач taras видалив правило SG о 02:14 з IP 93.x.x.x
Яка конфігурація ресурсу була і чи відповідає правилам?	Config	Bucket став публічним о 02:15; правило «S3 без публічного доступу» — NON_COMPLIANT

Watch — Trail — Config

Cloud*Watch* — **дивиться** на роботу (метрики, логи). **Cloud*Trail*** — **слід**, який лишає кожна дія людини чи сервісу. **Config** — **конфігурація** та її історія. «Хто видалив?» → Trail. «Чому повільно?» → Watch. «Що змінилося в налаштуваннях і чи це за правилами?» → Config.

CloudWatch Metrics: цифри роботи

Метрика — ряд чисел у часі: CPUUtilization сервера, Latency балансувальника, ConsumedReadCapacityUnits таблиці DynamoDB. Метрики згруповані в **простори імен** (AWS/EC2 , AWS/ApplicationELB) і мають **виміри** (dimensions) — наприклад, InstanceId .

- **EC2 за замовчуванням** надсилає метрики **кожні 5 хвилин** (basic monitoring, безкоштовно); **detailed monitoring** — **кожну хвилину** (платно). Для швидкого реагування Auto Scaling вмикай detailed.
-  **Пам'яті (RAM) і заповненості диска серед стандартних метрик EC2 немає:** гіпервізор бачить процесор, мережу і дискові операції, але не бачить, що відбувається всередині ОС. Для них встанови **CloudWatch agent** — він надсилає пам'ять, диск, процеси і логи.

- **Власні метрики** (custom) — з коду через `PutMetricData` : «замовлень за хвилину», «товарів у кошиках». **High-resolution** — з кроком до **1 секунди**.
- **Зберігання**: до **15 місяців** зі зменшенням деталізації: хвилинні точки — 15 днів, 5-хвилинні — 63 дні, годинні — 455 днів.
- **Metric Streams** — безперервний потік метрик через Firehose у S3 чи сторонні системи (Datadog, Splunk).

CloudWatch Alarms: лампочки на панелі

Аларм стежить за метрикою і має три стани: **OK**, **ALARM**, **INSUFFICIENT_DATA**. Умова: «середній CPU > 80% протягом **3 з 5** періодів по 1 хвилині» (datapoints to alarm) — так аларм не спрацьовує на короткі сплески.

Що аларм уміє робити сам:

- надіслати сповіщення в **SNS** (пошта, SMS, Lambda, Slack через Chatbot);
- запустити **політику Auto Scaling** (розділ 23);
- **дії EC2**: stop, terminate, reboot і **recover** — перезапустити сервер на **іншому хості** з тим самим ID, приватною IP, Elastic IP і метаданими (при збої `StatusCheckFailed_System`);
- викликати **Lambda** або створити запис в OpsCenter Systems Manager.

Ще два інструменти проти «шуму»:

- **Composite alarms** — поєднують аларми логікою AND/OR: «будити Тараса, лише якщо і CPU високий, і зросли помилки 5xx».
- **Anomaly detection** — ML будує «коридор нормальних значень» з урахуванням добових і тижневих циклів; аларм спрацьовує, коли метрика виходить за коридор (замість фіксованого порогу).

CloudWatch Logs: бортовий журнал

- **Log groups** (зазвичай одна на застосунок) → **log streams** (по одному на сервер чи контейнер) → записи.
- **Джерела**: CloudWatch agent (EC2 і власні сервери), **Lambda**, ECS і EKS, API Gateway, **VPC Flow Logs**, Route 53, CloudTrail, RDS.
- **Зберігання**: за замовчуванням — **назавжди** (платиш за кожен гігабайт роками!). Задавай **retention** — від 1 дня до 10 років. Шифрування KMS. Клас **Infrequent Access** — дешевший для рідко переглянутих логів.
- **Metric filters** — перетворюють рядки логів на метрику: кількість «ERROR» за хвилину → аларм → SNS. Класика іспиту.
- **Logs Insights** — інтерактивні запити до логів мовою запитів: «топ-10 найповільніших запитів за годину».
- **Subscription filters** — потік логів **у реальному часі** в Lambda, Kinesis Data Streams, Firehose або OpenSearch (зокрема централізовано в інший акаунт).
- **Експорт у S3** — пакетно, не в реальному часі; для аналізу в Athena. **Live Tail** — перегляд логів наживо. **Data protection** — маскуванню персональних даних у логах.

Ще інструменти спостереження

- **Dashboards** — власні панелі з графіками, зокрема з кількох регіонів і акаунтів.
- **Synthetics canaries** — скрипти, що кожні кілька хвилин «ходять» сайтом як покупець (відкрити каталог, покласти в кошик) і ловлять поломки **раніше за клієнтів**.
- **RUM** (real user monitoring) — швидкість сайту в браузерх справжніх відвідувачів.

- **Application Signals** — моніторинг застосунків (APM): затримки, помилки, SLO; **Container Insights** і **Lambda Insights** — детальні метрики контейнерів і функцій; **Contributor Insights** — «хто генерує найбільше запитів чи помилок».
- **Amazon Managed Service for Prometheus** і **Amazon Managed Grafana** — керовані Prometheus (метрики контейнерів, зокрема EKS) і Grafana (дашборди) для команд, що звикли до цих open-source-інструментів.

AWS X-Ray: рентген запиту

У монолітному застосунку повільний запит видно в одному лозі. У Хмаринці замовлення проходить **API Gateway** → **Lambda** → **ECS-сервіс оплати** → **DynamoDB** → **SQS**. **AWS X-Ray** збирає **трасу** (trace) кожного запиту: скільки часу він провів у кожному сервісі (**segments**, **subsegments**), де сталася помилка, і малює **карту сервісів** (service map). Частина запитів вибирається для трасування (**sampling**), щоб не платити за все.

- Інтерації: **Lambda** (active tracing — прапорець), **API Gateway**, ECS, EKS, Elastic Beanstalk, SNS, SQS.
-  **SDK і демон X-Ray** перейшли в режим підтримки (maintenance) **25.02.2026**, кінець підтримки — **25.02.2027**. Сьогодні застосунки інструментують через **OpenTelemetry** (AWS Distro for OpenTelemetry або CloudWatch agent), а траси так само потрапляють у X-Ray і CloudWatch (Transaction Search, Application Signals).

AWS CloudTrail: хто, що, коли

AWS CloudTrail записує **виклики API** в акаунті — з консолі, CLI, SDK і від самих сервісів AWS: **хто** (користувач, роль), **що** (`DeleteSecurityGroupRule`), **коли, звідки** (IP, консоль чи CLI), з якими параметрами і чи вдалося.

- **Event history** — **90 днів** management events, безкоштовно й автоматично, без налаштувань.
- **Trail** — постійний запис у **S3** (для років зберігання) і за бажанням у **CloudWatch Logs** (для metric filters і алармів). Новий trail за замовчуванням — **для всіх регіонів**. **Organization trail** — один trail для **всіх акаунтів** організації (зазвичай у окремий акаунт Log Archive).
- **Типи подій:**
 - **management events** — керування ресурсами (створити сервер, змінити SG) — увімкнені за замовчуванням;
 - **data events** — операції з даними: **S3 GetObject/PutObject**, виклики Lambda, операції з елементами DynamoDB — **вимкнені** за замовчуванням, платні, бо їх дуже багато;
 - **Insights events** — незвичайна активність: раптом у 50 разів більше `TerminateInstances`, ніж зазвичай.
- **Захист журналу: log file integrity validation** (файли-дайджести SHA-256 доводять, що журнал не змінювали й не видаляли), шифрування **SSE-KMS**, bucket з **Object Lock** в окремому акаунті.
- **Затримка:** події потрапляють у журнал зазвичай протягом **~5 хвилин**. Щоб реагувати миттєво, використовуй правило **EventBridge** на подію CloudTrail: «хтось викликав `StopLogging` чи `DeleteBucket`» → SNS або Lambda.
-  **CloudTrail Lake** (SQL-запити до подій) **закритий для нових клієнтів з 31.05.2026**. Замість нього аналізують журнали в **CloudWatch** або запитами **Athena** до trail у S3 (розділ 39).

AWS Config: стан і правила

AWS Config постійно записує **конфігурацію ресурсів** (configuration items) і **історію змін**: як виглядала Security Group учора о 14:00, які ресурси з нею пов'язані і хто її змінив (посилання на подію CloudTrail).

- **Config rules** — перевірки відповідності: **сотні готових** (managed) правил — «усі томи EBS зашифровані», «S3 без публічного читання», «SSH не відкритий для 0.0.0.0/0», «CloudTrail увімкнений» — і власні (Lambda або мова Guard). Перевірка — **при кожній зміні** ресурсу або **періодично**.
- **Remediation** — **автоматичне виправлення** через документи **Systems Manager Automation**: bucket став публічним → Config позначає NON_COMPLIANT → автоматично вмикається Block Public Access.
- **Conformance packs** — набори правил і виправлень (наприклад, під CIS чи PCI DSS), що розгортаються на всю організацію.
- **Aggregator** — зведений стан відповідності **всіх акаунтів і регіонів** в одному місці.
-  Config — **детективний** контроль: він **знаходить і виправляє**, але **не забороняє** дію наперед. Заборонити — це **SCP** та IAM (розділ 10).

AWS Systems Manager: пульт автопарку

Щоб сервер став «керованим», на ньому має працювати **SSM Agent** (вже встановлений в Amazon Linux і Windows AMI) і бути **роль IAM** з політикою AmazonSSMManagedInstanceCore . Власні сервери в офісі теж можна підключити (hybrid activations).

Можливість	Що робить	Типове питання
Session Manager	Командний рядок у браузері чи CLI без SSH : без відкритого порту 22, без bastion, без ключів; доступ за IAM; сесії пишуться в S3 і CloudWatch Logs	Безпечний доступ до серверів у приватній підмережі з журналом дій
Run Command	Виконати команду чи скрипт на сотнях серверів одночасно	Оновити конфіг на всіх серверах без входу на кожен
Patch Manager	Оновлення ОС за базовими лініями (patch baselines) у вікна обслуговування	Автоматичні патчі за розкладом із звітом відповідності
Automation	Runbooks — сценарії дій (перезапуск, створення AMI, виправлення для Config)	Автовиправлення порушень

Можливість	Що робить	Типове питання
State Manager, Inventory	Бажаний стан (агент завжди встановлений), перелік ПЗ на серверах	—
Parameter Store	Налаштування й секрети (розділ 11)	—

Інфраструктура як код: CloudFormation

AWS CloudFormation створює інфраструктуру за **шаблоном** (YAML або JSON). Шаблон описує **Resources** (обов'язково) і за бажанням **Parameters**, **Mappings**, **Conditions**, **Outputs**. Розгорнутий шаблон — це **стек** (stack).

```
Resources:
  OrdersQueue:
    Type: AWS::SQS::Queue
    Properties:
      VisibilityTimeout: 60
  PhotosBucket:
    Type: AWS::S3::Bucket
    DeletionPolicy: Retain      # при видаленні стека bucket з фото залишиться
Outputs:
  QueueUrl:
    Value: !Ref OrdersQueue
```

- **Однаково щоразу:** dev, test і prod — з одного шаблону; DR-region піднімається за хвилини тим самим шаблоном.
- **Change sets** — попередній перегляд змін перед застосуванням. **Rollback** — при помилці стек повертається до попереднього стану. **Drift detection** — хтось змінив ресурс руками в обхід шаблону.
- **DeletionPolicy** (Retain , Snapshot) — не втратити дані при видаленні стека. **Stack policy** — заборона змінювати критичні ресурси.
- **StackSets** — розгорнути стек у багатьох акаунтах і регіонах однією дією; з Organizations — **автоматично в кожен новий акаунт**.

- **AWS CDK** — інфраструктура звичайною мовою програмування (TypeScript, Python), що перетворюється на CloudFormation. **AWS SAM** — скорочені шаблони для serverless. **Infrastructure Composer** — візуальний редактор.

Порядок і поради: Service Catalog, Trusted Advisor, Health, License Manager

- **AWS Service Catalog** — «меню дозволених страв»: адміністратори публікують **затверджені продукти** (шаблони CloudFormation) у портфелях, а команди запускають їх **самі**, навіть без широких прав IAM (запуск іде від ролі продукту — launch constraint).
- **AWS Trusted Advisor** — автоматичний аудитор за категоріями: **вартість, продуктивність, безпека, стійкість, ліміти сервісів**, операційна досконалість. Безкоштовно — базові перевірки безпеки і лімітів; повний набір — з платними планами підтримки (у 2026 — **Business Support+** і вищі). Для лімітів є також **Service Quotas**: перегляд квот, запит на збільшення і аларм при наближенні.
- **AWS Health Dashboard** — події AWS, що зачіпають **саме твої** ресурси: планове обслуговування (наприклад, виведення хоста з експлуатації), збої в регіоні. Події йдуть у **EventBridge** → автоматична реакція.
- **AWS License Manager** — облік власних ліцензій (**BYOL**: Windows Server, SQL Server, Oracle) і правила, що **не дадуть** запустити більше, ніж куплено; керування Dedicated Hosts.

Моніторинг Хмаринки

БАЧИТИ

Метрики ALB, ECS, Aurora + **CloudWatch agent** (пам'ять, диск) → дашборд «Чорна п'ятниця»; **Synthetics** кожні 5 хв оформлює тестове замовлення

↓

РЕАГУВАТИ

Composite alarm (p99 затримки ↑ і 5xx ↑) → **SNS** → телефон Тараса; target tracking → Auto Scaling

↓

ШУКАТИ ПРИЧИНУ

Logs Insights по логах; **X-Ray**: 5 з 6 секунд — очікування платіжного API

↓

АУДИТ

Organization trail → S3 в акаунті Log Archive (Object Lock); **EventBridge**: StopLogging, зміни SG → SNS

↓

ВІДПОВІДНІСТЬ

Config rules: EBS зашифровані, S3 не публічні, SSH закритий → автовиправлення через **SSM Automation**

Відтепер про гальмування Тарас дізнається від аларму за хвилину, а не з Instagram за годину. Доступ до серверів — лише через Session Manager, порт 22 закритий.

Що обрати

Потреба	Рішення
Пам'ять і диск EC2	CloudWatch agent
Сповіщення, коли в логах з'являється «ERROR»	Metric filter + alarm + SNS

Потреба	Рішення
Логи в реальному часі в інший сервіс чи акаунт	Subscription filter (Lambda, Kinesis, Firehose)
Хто видалив чи змінив ресурс	CloudTrail
Журнал API всіх акаунтів організації	Organization trail
Довести, що журнал не підрили	Log file integrity validation
Хто читав об'єкти в S3	CloudTrail data events (або S3 server access logs)
Миттєва реакція на небезпечний виклик API	EventBridge rule на подію CloudTrail
Відповідність налаштувань правилам + авто виправлення	Config rules + remediation (SSM Automation)
Історія конфігурації ресурсу	Config
Де гальмує запит між мікросервісами	X-Ray
Доступ до сервера без SSH і bastion	Session Manager
Патчі за розкладом	Patch Manager
Однакова інфраструктура в багатьох акаунтах і регіонах	CloudFormation StackSets
Команди запускають лише затверджені шаблони	Service Catalog
Перевірка лімітів і кращих практик	Trusted Advisor / Service Quotas
Реакція на планове обслуговування AWS	Health Dashboard + EventBridge

 **Спробуй у справжньому AWS: помилки в логах → аларм → лист**

Вартість: у межах безкоштовного рівня (10 алармів, 5 GB логів). **Час:** 20 хвилин.

1. **CloudShell** — створи групу логів з retention, фільтр метрики і тему SNS (заміни email):

```
aws logs create-log-group --log-group-name /khmarynka/app
aws logs put-retention-policy --log-group-name /khmarynka/app --retention-in-days 3
aws logs create-log-stream --log-group-name /khmarynka/app --log-stream-name app
aws logs put-metric-filter --log-group-name /khmarynka/app --filter-name AppErrors \
  --filter-pattern "ERROR" \
  --metric-transformations metricName=AppErrors,metricNamespace=Khmarynka
TOPIC=$(aws sns create-topic --name khmarynka-alerts --query TopicArn --output text)
aws sns subscribe --topic-arn $TOPIC --protocol email --notification-email-address $EMAIL
```

1. Підтверди підписку в листі від AWS. Створи аларм: 3 і більше помилок за хвилину:

```
aws cloudwatch put-metric-alarm --alarm-name khmarynka-errors \
  --namespace Khmarynka --metric-name AppErrors --statistic Sum --period 60 \
  --evaluation-periods 1 --threshold 3 --comparison-operator GreaterThan \
  --treat-missing-data notBreaching --alarm-actions $TOPIC
```

1. «Зламай» застосунок — запиши в лог три помилки:

```
NOW=$(date +%s%3N)
aws logs put-log-events --log-group-name /khmarynka/app --log-stream-name app \
  --timestamp=$NOW,message="ERROR payment timeout" timestamp=$NOW,message="ERROR db connection refused" \
  --timestamp=$NOW,message="ERROR db connection refused"
```

За 1–3 хвилини аларм перейде в **ALARM** (CloudWatch → Alarms), і прийде лист.

1. **CloudTrail** → **Event history** → фільтр Event name = PutMetricAlarm : ти бачиш, **хто** створив аларм, коли і з якої IP.

Прибери за собою:


```
aws cloudwatch delete-alarms --alarm-names khmarynka-errors
aws logs delete-log-group --log-group-name /khmarynka/app
aws sns delete-topic --topic-arn $TOPIC
```

Запам'ятай

CloudWatch — що відбувається (метрики, логи, аларми); **CloudTrail** — хто що зробив (API); **Config** — конфігурація, історія і правила.

EC2: метрики кожні **5 хв** (detailed — 1 хв); **RAM і диск — лише через CloudWatch agent**; власні метрики — до 1 с.

Аларм: SNS, Auto Scaling, **EC2 stop/terminate/reboot/recover**, Lambda; **composite alarms** проти шуму; **anomaly detection**.

Логи: **retention** (за замовчуванням — назавжди), **metric filter** → **alarm**, **Logs Insights**, **subscription filters** у реальному часі, експорт у S3 — пакетно.

X-Ray — траси й карта сервісів; інструментування — через **OpenTelemetry** (SDK X-Ray у maintenance з 2026).

CloudTrail: 90 днів event history; trail у S3 (+ CloudWatch Logs); **organization trail**; data events — окремо й платно; **integrity validation**; ~5 хв затримки → для миттєвої реакції **EventBridge**.

Config: правила, **remediation** через **SSM Automation**, conformance packs, aggregator; **виявляє, а не забороняє**.

Systems Manager: **Session Manager** (без SSH), Run Command, **Patch Manager**, Automation, Parameter Store.

CloudFormation: шаблони, change sets, drift, DeletionPolicy, **StackSets** (багато акаунтів і регіонів); CDK, SAM.

Service Catalog — затверджені продукти; **Trusted Advisor** — поради й ліміти; **Health Dashboard + EventBridge** — події AWS; **License Manager** — BYOL.

Як питають на іспиті

Хто видалив ресурс чи змінив налаштування → **CloudTrail**

Моніторити використання пам'яті EC2 → **CloudWatch agent**

Сповіщення при появі певного тексту в логах → **metric filter + CloudWatch alarm + SNS**

Автоматично перевіряти, що всі томи EBS зашифровані, і виправляти порушення → **Config rule + remediation**

Доступ до інстансів у приватній підмережі без bastion і відкритого порту 22 → **Systems Manager Session Manager**

Знайти вузьке місце в запитах між мікросервісами → **X-Ray**

Однаковий набір ресурсів у кожному акаунті організації → **CloudFormation StackSets**

Швидко відтворити інфраструктуру в DR-регіоні → **CloudFormation-шаблон**

Централізований аудит API всіх акаунтів → **organization trail у S3 окремого акаунта**

Перевірити, чи журнал CloudTrail не змінювали → **log file integrity validation**

Автоматично відновлювати EC2 при збої хоста → **CloudWatch alarm з дією recover**

Зменшити кількість хибних тривог → **composite alarms**

Реагувати на планове обслуговування EC2 від AWS → **AWS Health + EventBridge**

Команди запускають лише затверджені конфігурації → **Service Catalog**

Пастки

CloudTrail **не показує** CPU чи помилки застосунку — це CloudWatch. CloudWatch **не каже**, хто видалив ресурс — це CloudTrail.

Config не забороняє дії — лише виявляє і виправляє. Заборона — SCP та IAM.

Логи CloudWatch за замовчуванням **зберігаються назавжди** — налаштовуй retention.

Data events CloudTrail (читання об'єктів S3) **не записуються** за замовчуванням.

Дія **recover** не зберігає дані **instance store**.

Перевір себе

ПИТАННЯ 1

Хтось видалив правило Security Group у production-акаунті минулої ночі. Як дізнатися, хто це зробив і з якої IP-адреси?

- A** Перевірити VPC Flow Logs
- B** Запустити перевірку AWS Trusted Advisor
- C** Переглянути метрики в Amazon CloudWatch
- D** Переглянути події в AWS CloudTrail

Правильна відповідь: D. CloudTrail записує кожен виклик API: хто, що, коли і звідки. Метрики CloudWatch і Flow Logs не містять інформації про дії користувачів з налаштуваннями.

ПИТАННЯ 2

Архітектору потрібно відстежувати використання оперативної пам'яті на інстансах EC2 і отримувати сповіщення, коли воно перевищить 90%. Що зробити?

- A** Створити аларм на стандартну метрику MemoryUtilization в AWS/EC2
- B** Увімкнути AWS Config для інстансів
- C** Увімкнути detailed monitoring для EC2
- D** Встановити CloudWatch agent, що надсилає метрику пам'яті, і створити аларм

Правильна відповідь: D. Пам'ять — метрика всередині ОС, гіпервізор її не бачить. CloudWatch agent надсилає її як власну метрику. Detailed monitoring лише частішає стандартні метрики, а метрики пам'яті серед них немає.

ПИТАННЯ 3

Компанія має гарантувати, що всі нові томи EBS зашифровані, а про порушення має знати служба безпеки й автоматично їх виправляти. Що обрати?

- A** AWS Config з managed rule і автоматичним remediation
- B** Amazon CloudWatch з metric filter
- C** AWS CloudTrail з organization trail
- D** Amazon Inspector

Правильна відповідь: A. Config rules перевіряють відповідність налаштувань правилам при кожній зміні, а remediation через SSM Automation автоматично виправляє порушення.

ПИТАННЯ 4

Адміністраторам потрібен доступ до командного рядка інстансів у приватних підмережах без bastion host і без відкритого порту 22, з журналом усіх сесій. Що використати?

- A** Bastion host з обмеженою Security Group
- B** EC2 Instance Connect з публічними IP
- C** AWS Systems Manager Session Manager
- D** Site-to-Site VPN

Правильна відповідь: C. Session Manager дає доступ через SSM Agent без вхідних портів і ключів SSH, керується IAM і записує сесії в S3 і CloudWatch Logs.

ПИТАННЯ 5

Застосунок пише логи в CloudWatch Logs. Команда хоче отримувати лист, коли за 5 хвилин з'являється понад 10 рядків з текстом «PaymentFailed». Яке рішення найпростіше?

- A** Щогодинний експорт логів у S3 і запит Athena
- B** Metric filter на групу логів, аларм на отриману метрику і тема SNS з підпискою email
- C** AWS Config rule для групи логів
- D** Lambda, що щохвилини читає всі логи

Правильна відповідь: B. Metric filter перетворює збіги в логах на метрику, аларм стежить за порогом і надсилає сповіщення через SNS — без коду і серверів.

ПИТАННЯ 6

Запит оформлення замовлення проходить через API Gateway, три функції Lambda і сервіс на ECS. Іноді він триває 8 секунд. Як знайти, який компонент гальмує?

- A** Перевірити AWS Config timeline
- B** Увімкнути VPC Flow Logs
- C** Увімкнути трасування AWS X-Ray (через OpenTelemetry) і переглянути service map і траси
- D** Переглянути CloudTrail

Правильна відповідь: C. X-Ray показує трасу запиту крізь усі сервіси з часом у кожному сегменті, тож вузьке місце видно одразу.

ПИТАННЯ 7

Компанія хоче розгорнути однаковий набір ролей IAM і правил Config у кожному з 60 акаунтів організації в трьох регіонах, а також автоматично в кожному новому акаунті. Що обрати?

- A** AWS Systems Manager Run Command
- B** AWS Service Catalog у кожному акаунті
- C** Окремий стек CloudFormation, запущений вручну в кожному акаунті
- D** AWS CloudFormation StackSets з інтеграцією AWS Organizations

Правильна відповідь: D. StackSets розгортають стек у багатьох акаунтах і регіонах однією операцією, а з Organizations — автоматично в нові акаунти.

ПИТАННЯ 8

Аудитор вимагає доказу, що файли журналу CloudTrail не змінювали й не видаляли після доставки в S3. Що увімкнути?

- A** CloudTrail Insights
- B** Перевірку цілісності файлів журналу CloudTrail (log file integrity validation)
- C** Версіонування S3
- D** AWS Config rule для bucket

Правильна відповідь: B. Integrity validation створює дайджест-файли з хешами SHA-256, за якими можна довести, що журнал не змінено і не видалено.

ПИТАННЯ 9

Які дії може виконати CloudWatch alarm напямую? (Оберіть 2)

Оберіть 2 відповіді

- A** Відкотити стек CloudFormation
- B** Надіслати сповіщення в тему SNS
- C** Зробити знімок бази RDS
- D** Відновити (recover) інстанс EC2 на іншому хості
- E** Змінити правило Security Group

Правильна відповідь: B, D. Аларм може надіслати сповіщення в SNS, запустити політику Auto Scaling, виконати дії EC2 (stop, terminate, reboot, recover), викликати Lambda. Інші дії роблять через SNS/Lambda або EventBridge.



Надійність і відновлення після катастроф — RPO, RTO, стратегії DR, AWS Backup



Відмовостійкість · 26%



Вартість · 20%



Історія Хмаринки. Олена ставить три незручні запитання. «Що буде, якщо **весь регіон Франкфурт** ляже на кілька годин?» «А якщо хтось **зашифрує наші дані** і вимагатиме викуп?» «А якщо Тарас **випадково видалить** таблицю замовлень?» Тарас чесно відповідає: «Від першого нас рятує лише копія в іншому регіоні, від другого й третього — бекапи, яких ніхто не може видалити». Разом вони записують вимоги: замовлення — втратити **не більше хвилини** даних і запрацювати **за 30 хвилин**; фото — втратити не більше години; аналітика — може почекаати добу.



Образ. RPO — скільки останніх сторінок щоденника не шкода втратити. **RTO** — за скільки часу ресторан після пожежі знову годує гостей. Чотири стратегії DR — чотири варіанти **запасного будинку**:  лише **креслення і речі на складі** (будуєш заново) ·  у запасному будинку **горить лише котел** — тепло підтримується, решту вмикаєш за потреби ·  будинок **з меблями, де живе черговий** — лише покликати родину ·  **два повноцінні будинки**, і в обох живуть.

Від чого захищаємося

Що ламається	Чим захищаємося	Де в курсі
Сервер або контейнер	Кілька екземплярів, Auto Scaling , health checks	розділи 22–23
Ціла зона доступності (AZ)	Multi-AZ : ресурси у 2–3 AZ	розділ 6, бази — 34–36
Цілий регіон	Multi-region DR — копія в іншому регіоні	цей розділ
Помилка людини, збій даних, ransomware	Бекапи з історією (point-in-time), незмінні копії, версіонування	цей розділ, розділ 28



Глибше: HA, DR і бекап — три різні страховки

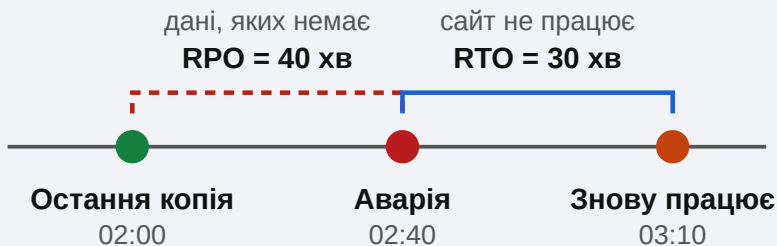
Висока доступність (HA) — система переживає збій **автоматично і майже непомітно** (Multi-AZ, балансувальник, Auto Scaling).

Відновлення після катастрофи (DR) — план, як запрацювати в **іншому регіоні**, коли основний недоступний. **Бекап** — копія в **минулому часі**. Реплікація **не замінює** бекап: якщо хтось видалить таблицю чи шифрувальник зіпсує дані, репліка **слухняно повторить** видалення за мілісекунди. Від помилок людей і ransomware рятують лише копії з історією, які неможливо видалити.

RPO і RTO

- **RPO (Recovery Point Objective)** — **скільки даних** (вимірених у часі) можна **втратити**: відстань від аварії **назад** до останньої доброї копії.
- **RTO (Recovery Time Objective)** — **скільки часу** система може **не працювати**: відстань від аварії **вперед** до відновлення.

ПРИКЛАД: БАЗА ЗАМОВЛЕНЬ



RPO — скільки даних (за часом) можна втратити.

RTO — за скільки часу треба знову запрацювати.

Менші RPO і RTO — дорожча архітектура. Тому їх визначає бізнес: скільки коштує година простою і година втрачених замовлень.

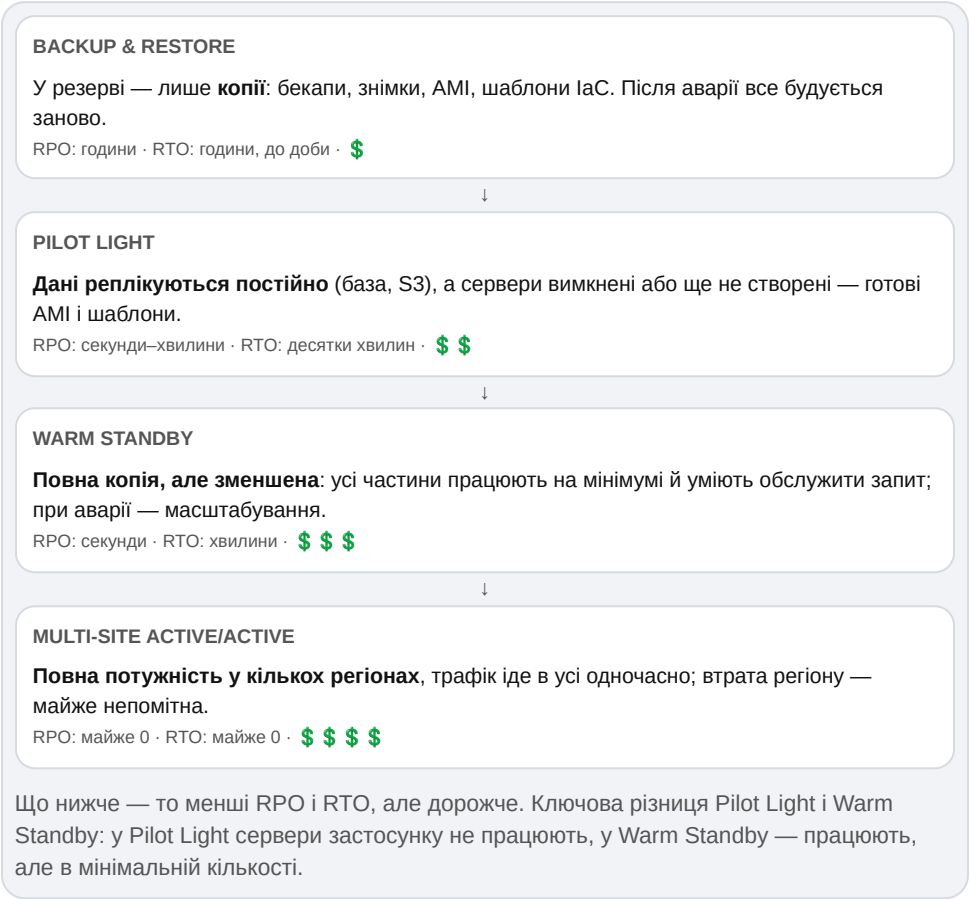


P — Point, T — Time

RPO — Point: до якої **точки** в минулому відкотимося (втрачені дані).

RTO — Time: скільки **часу** на відновлення (простій). «Бекап раз на добу» → RPO до 24 годин. «Відновлення з бекапу триває 4 години» → RTO близько 4 годин.

Чотири стратегії DR



Як вони відновлюються:

- **Backup & Restore:** бекапи й знімки **копіюються в інший регіон** (AWS Backup, копії знімків EBS і RDS, S3 CRR), інфраструктура — **кодом** (CloudFormation). При аварії: розгорнути стек, відновити бази з бекапів, перемкнути DNS. Найдешевше, найдовше.

- **Pilot Light:** «ядро» — дані — живе в резервному регіоні: **Aurora Global Database** чи **cross-region read replica**, **DynamoDB global tables**, **S3 CRR**. При аварії: **promote** бази, запустити сервери (AMI, шаблони, Auto Scaling з 0 до потрібного), **Route 53** перемикає трафік.
- **Warm Standby:** те саме, але застосунок уже працює на мінімумі (наприклад, 1 сервер замість 10). При аварії: збільшити Auto Scaling, promote бази, перемкнути трафік. Можна навіть перевіряти резерв справжнім трафіком.
- **Multi-site active/active:** обидва регіони обслуговують користувачів (**Route 53 latency** чи **geolocation**, **Global Accelerator**), дані — **DynamoDB global tables** (запис у будь-якому регіоні) або **Aurora Global Database** з write forwarding. Найскладніше: потрібно продумати конфлікти записів.



Яку стратегію обрати Хмаринці

Вимоги для замовлень: **RPO ≤ 1 хв**, **RTO ≤ 30 хв**, бюджет обмежений.

- Backup & Restore: RPO — години (між бекапами), RTO — години. ❌ Не проходить.
- **Pilot Light:** Aurora Global Database дає RPO ≈ 1 с; запуск ECS-сервісів з шаблонів і перемикання DNS — 15–25 хв. ✅ Проходить і коштує лише реплікація бази та сховище.
- Warm Standby: RTO — хвилини, але платимо за постійно працюючий мінімальний застосунок. Зайве для цих вимог.
- Active/active: подвійна вартість і складність. Зайве.

Для **аналітики** (RPO 24 год, RTO 1 доба) — **Backup & Restore**.

Будівельні блоки multi-region

Що	Як мати копію в іншому регіоні	Орієнтовний RPO
S3	Cross-Region Replication; з RTC — 99,99% об'єктів за 15 хв	хвилини
DynamoDB	Global tables (active-active); режим multi-Region strong consistency	~1 с (у MRSC — 0)
Aurora	Global Database: репліка до 10 регіонів, switchover і failover	~1 с, RTO — хвилини
RDS	Cross-region read replica → promote; або копії знімків	секунди / години
EBS, AMI	Копіювання знімків і AMI (Data Lifecycle Manager, AWS Backup)	години
EFS	EFS replication	хвилини (цільово 15 хв)
ElastiCache	Global Datastore	< 1 с
Секрети, ключі, образи	Реплікація Secrets Manager, KMS multi-Region keys, реплікація ECR	—
Інфраструктура	CloudFormation / StackSets, однакові шаблони	—
Трафік	Route 53 failover + health checks, Global Accelerator, CloudFront origin failover	—

Amazon Application Recovery Controller (ARC) допомагає перемикатися керовано: **zonal shift** — одним рухом відвести трафік балансувальників від проблемної AZ, **zonal autoshift** — AWS робить це сам, коли бачить збій у зоні, а **Region switch** (з 2025) — готові плани перемикання застосунку між регіонами.



Глибше: статична стабільність

—

Під час великої аварії найненадійніше — **створювати нове**: API запуску серверів (control plane) перевантажене всіма клієнтами одночасно. Тому надійні системи **статично стабільні**: потрібна

потужність **вже працює** до аварії. Приклад: застосунку потрібно 6 серверів; у трьох AZ тримають по 3 (усього 9, тобто 150%). Якщо одна AZ зникне, дві інші вже мають 6 серверів — нічого не потрібно запускати. Так само перемикання трафіку краще робити через **health checks Route 53** (data plane, що працює завжди), а не редагуванням DNS-записів під час аварії.

AWS Backup: один сейф для всіх копій

AWS Backup — централізовані бекапи за **політиками** замість десятка окремих налаштувань у кожному сервісі.

- **Backup plan** — розклад (щогодини, щодня), вікно, **життєвий цикл** (перенесення в холодне сховище, термін зберігання); ресурси додаються **за тегами** (`backup=daily`) — новий сервер з тегом автоматично потрапляє в план.
- **Підтримує:** EC2, EBS, **RDS, Aurora, DynamoDB, EFS, FSx, S3**, DocumentDB, Neptune, Redshift, Timestream, томи Storage Gateway, VMware on-premises та інші.
- **Point-in-time recovery** (безперервні бекапи) для RDS, Aurora і S3 — відновлення на будь-яку секунду до 35 днів назад.
- **Копії в інший регіон і інший акаунт** — основа Backup & Restore DR і захисту від компрометації акаунта. **Backup policies** в AWS Organizations — однакові плани для всіх акаунтів.
- **Backup Vault Lock** — сховище **WORM**: у режимі **compliance** після короткого пільгового періоду **ніхто**, навіть root, не може видалити копії чи скоротити термін їх зберігання. Захист від ransomware і зловмисного адміністратора.
- **Logically air-gapped vault** — ізольоване сховище, заблоковане за замовчуванням; ним можна поділитися (через RAM) з окремим акаунтом, щоб відновитися, навіть якщо основний акаунт скомпрометовано.

- **Restore testing** — автоматичні регулярні тести відновлення; **Backup Audit Manager** — звіти для аудиторів.

AWS Elastic Disaster Recovery

AWS Elastic Disaster Recovery (DRS) захищає **сервери** — фізичні й віртуальні on-premises, в інших хмарах або EC2 в іншому регіоні:

- агент **постійно реплікує диски на рівні блоків** у дешеву **staging-зону** в AWS (маленькі сервери-реплікатори й недорогі диски);
- **RPO — секунди, RTO — хвилини**: при аварії DRS запускає повноцінні сервери з актуальних даних;
- **навчальні запуски (drills)** без впливу на основну систему, а після аварії — **failback** назад.

Це правильна відповідь, коли в питанні **сервери з офісу чи дата-центру**, яким потрібен DR в AWS з RPO в секунди. Для міграції є його «брат» — MGN (розділ 33).

Перевіряй, а не сподівайся

«Бекап, який ніколи не відновлювали, — не бекап». Так само і план DR.

- **AWS Fault Injection Service (FIS)** — керовані **хаос-експерименти**: зупинити сервери, додати мережеву затримку, імітувати **втрату AZ** — і подивитися, чи система справді переживе збій.
- **AWS Resilience Hub** — оцінює застосунок щодо твоїх цілей **RPO/RTO** і радить, що покращити.
- **Game days** — регулярні навчання команди: перемкнутися в DR-region за інструкцією (runbook) і повернутися.

DR Хмаринки

 **eu-central-1 — основний**

ALB + ECS (6 задач у 3 AZ), Aurora (writer + репліки), DynamoDB, S3, ElastiCache

Обслуговує 100% трафіку

 **eu-west-1 — резерв (pilot light)**

Aurora Global Database (вторинний кластер), **DynamoDB global table**, **S3 CRR** для фото, копії AWS Backup; ECS-сервіс з 0 задач, шаблони CloudFormation готові

Платимо переважно за дані

АВАРІЯ

Health check Route 53 фіксує збій → аларм → runbook: **failover** Aurora Global Database, ECS 0 → 6 задач, Route 53 **failover** на ALB в Ірландії. Ціль — 30 хв.

RANSOMWARE

AWS Backup: щоденні копії всього + PITR для Aurora; копії — в окремий акаунт і регіон, **Vault Lock (compliance)** на 35 днів.

ПЕРЕВІРКА

Раз на квартал — **game day** з перемиканням, раз на місяць — **restore testing**, експерименти **FIS** з вимкненням AZ.

Замовлення: RPO ≈ 1 с (Aurora Global Database), RTO ≈ 30 хв. Фото: RPO — хвилини (CRR). Аналітика: Backup & Restore.

Що обрати

Вимога	Рішення
Найдешевший DR, RTO/RPO — години	Backup & Restore (копії в інший регіон + IaC)
RTO — десятки хвилин, мінімум витрат, дані актуальні	Pilot Light
RTO — хвилини, система вже працює в мінімальному розмірі	Warm Standby

3. **Backup vaults** → **Default** → твоя точка відновлення → **Actions** → **Copy**: регіон **eu-west-1**, vault **khm-dr**. Зачекай завершення копіювання. (Якщо копіювання для DynamoDB недоступне — **Settings** → **Advanced features for Amazon DynamoDB backups** → **Enable**.)

4. У **eu-west-1**: **Backup vaults** → **khm-dr** → точка відновлення → **Restore**: нова таблиця **khm-orders-restored**. Коли відновиться, у DynamoDB (eu-west-1) відкрий таблицю — замовлення **A-1001** на місці. Ти щойно провів **Backup & Restore DR** в іншому регіоні.

Прибери за собою: видали таблицю **khm-orders-restored** (eu-west-1) і **khm-orders** (eu-central-1); видали точки відновлення в обох vault (Backup vaults → vault → Delete); видали vault **khm-dr**.

Запам'ятай

HA (Multi-AZ, автоматично) ≠ **DR** (інший регіон, за планом) ≠ **бекап** (копія в минулому часі). Реплікація повторює і помилки — бекап з історією обов'язковий.

RPO — скільки даних втратимо (назад у часі); **RTO** — скільки простоємо (вперед).

Backup & Restore (години, \$) → **Pilot Light** (дані живі, сервери вимкнені; десятки хвилин) → **Warm Standby** (зменшена робоча копія; хвилини) → **Active/active** (майже 0, \$ \$ \$ \$ \$).

Блоки: **S3 CRR (RTC 15 хв)**, **DynamoDB global tables**, **Aurora Global Database (RPO ~1 с)**, cross-region read replica, копії знімків/AMI, EFS replication, **Route 53 failover**, Global Accelerator, CloudFormation.

AWS Backup: плани за тегами, копії між регіонами й акаунтами, **Vault Lock (compliance)**, air-gapped vault, restore testing, політики Organizations.

Elastic Disaster Recovery — DR серверів з блоковою реплікацією, RPO секунди, RTO хвилини.

FIS — хаос-експерименти; **Resilience Hub** — оцінка RPO/RTO; **ARC** — zonal shift, Region switch.

Як питають на іспиті

Мінімальна вартість DR, RTO і RPO — кілька годин → **Backup & Restore з копіями в інший регіон**

RTO десятки хвилин, мінімум витрат, база реплікується → **Pilot Light**

Зменшена повна копія, що працює постійно; RTO хвилини → **Warm Standby**

Майже нульовий простій і втрати даних → **Multi-site active/active**

Реляційна база, RPO ~1 с і RTO ~1 хв між регіонами → **Aurora Global Database**

Централізовані бекапи EC2, RDS, DynamoDB, EFS з копією в інший регіон → **AWS Backup**

Захист бекапів від видалення навіть адміністратором (ransomware) → **Backup Vault Lock у compliance mode**

DR для серверів у власному дата-центрі з RPO в секунди → **AWS Elastic Disaster Recovery**

Автоматичне перемикання DNS на резервний регіон → **Route 53 failover routing + health checks**

Перевірити, як система переживе втрату AZ → **AWS Fault Injection Service**

Об'єкти S3 мають з'явитися в іншому регіоні за 15 хвилин з SLA → **S3 CRR з Replication Time Control**

Пастки

Multi-AZ — не DR: він не рятує від втрати регіону. **Read replica** — не бекап: вона повторює помилки.

Pilot Light: сервери **не працюють**; Warm Standby: **працюють**, але в мінімальному розмірі.

Копії бекапів в **тому самому** акаунті не захищають від компрометації акаунта — копією в окремий акаунт.

Vault Lock **compliance** після пільгового періоду **не можна зняти** — налаштовуй терміни уважно.

✓ Перевір себе

ПИТАННЯ 1

Бізнес погоджується втратити дані максимум за 15 хвилин, а система має запрацювати не пізніше ніж за 2 години після аварії. Які це показники?

- A** RPO = 15 хвилин, RTO = 2 години
- B** RPO = 2 години, RTO = 15 хвилин
- C** RTO = 15 хвилин, SLA = 2 години
- D** RPO і RTO = 15 хвилин

Правильна відповідь: A. RPO — допустима втрата даних (назад у часі від аварії), RTO — допустимий час простою (вперед від аварії).

ПИТАННЯ 2

Внутрішній застосунок може простоювати до 24 годин, а втрата даних за 12 годин прийнятна. Потрібен найдешевший захист від втрати регіону. Що обрати?

- A** Backup & Restore: регулярні бекапи з копіюванням в інший регіон і шаблони CloudFormation
- B** Multi-site active/active
- C** Warm Standby з мінімальною копією застосунку
- D** Pilot Light з Aurora Global Database

Правильна відповідь: A. Великі допустимі RPO і RTO дозволяють найдешевшу стратегію — копії в іншому регіоні і відтворення інфраструктури кодом після аварії.

ПИТАННЯ 3

Компанії потрібне RTO близько 30 хвилин і RPO в секунди для бази даних за мінімальної вартості. Сервери застосунку в резервному регіоні не повинні працювати постійно. Яка стратегія?

- A** Multi-site active/active
- B** Backup & Restore
- C** Pilot Light
- D** Warm Standby

Правильна відповідь: C. Pilot Light тримає в резервному регіоні лише постійно репліковані дані, а сервери запускаються при аварії. Warm Standby вимагав би постійно працюючої зменшеної копії застосунку.

ПИТАННЯ 4

Глобальний застосунок має продовжувати працювати без помітного простою і втрати даних навіть при втраті цілого регіону. Користувачі пишуть дані в різних регіонах. Що обрати?

- A** Warm Standby з щоденними бекапами
- B** Multi-AZ RDS в одному регіоні
- C** Pilot Light з cross-region read replica RDS
- D** Active/active у кількох регіонах з DynamoDB global tables і маршрутизацією Route 53 за затримкою

Правильна відповідь: D. Active/active з global tables дозволяє запис у кожному регіоні і майже нульові RPO/RTO. Multi-AZ не захищає від втрати регіону.

ПИТАННЯ 5

Компанія хоче централізовано бекапити EC2, RDS, DynamoDB і EFS у 30 акаунтах, копіювати бекапи в інший регіон і гарантувати, що їх не може видалити ніхто, навіть адміністратор. Що обрати?

- A** S3 Cross-Region Replication для всіх даних
- B** Data Lifecycle Manager для EBS і автоматичні бекапи RDS
- C** Скрипти Lambda, що створюють знімки в кожному акаунті
- D** AWS Backup з backup policies в Organizations, cross-region copy і Backup Vault Lock у compliance mode

Правильна відповідь: D. AWS Backup централізує бекапи багатьох сервісів, копіює їх між регіонами й акаунтами, а Vault Lock у compliance mode робить їх незмінними.

ПИТАННЯ 6

Компанія має 200 серверів VMware у власному дата-центрі і хоче DR в AWS з RPO в секунди і RTO в хвилини, платячи мінімум, поки аварії немає. Що обрати?

- A** AWS Elastic Disaster Recovery
- B** AWS Application Migration Service
- C** AWS DataSync
- D** AWS Backup з щоденними бекапами VMware

Правильна відповідь: A. Elastic Disaster Recovery постійно реплікує диски в дешеву staging-зону і при аварії запускає повноцінні сервери за хвилини. MGN — для міграції, а щоденні бекапи дають RPO в годинах.

ПИТАННЯ 7

Потрібна реляційна база з реплікацією в інший регіон з RPO близько 1 секунди і відновленням за хвилину. Що обрати?

- A** Щоденні знімки RDS, скопійовані в інший регіон
- B** Amazon DynamoDB global tables
- C** Amazon RDS Multi-AZ
- D** Amazon Aurora Global Database

Правильна відповідь: D. Aurora Global Database реплікує дані в інші регіони із затримкою близько секунди і підтримує швидкий керований failover. Multi-AZ працює в межах одного регіону, а DynamoDB — не реляційна.

ПИТАННЯ 8

Які твердження правильні? (Оберіть 2)

Оберіть 2 відповіді

- ☐ А Multi-AZ захищає від втрати всього регіону
- ☐ В Реплікація не замінює бекап: видалення даних теж реплікується
- ☐ С Backup Vault Lock у compliance mode може зняти root-користувач
- ☐ D У Warm Standby застосунок у резервному регіоні вже працює, але в мінімальному розмірі
- ☐ Е Pilot Light має менший RTO, ніж Warm Standby

Правильна відповідь: В, D. Репліка повторює і помилки, тож бекапи з історією потрібні завжди. Warm Standby — зменшена робоча копія. Multi-AZ діє в межах регіону, Pilot Light повільніший за Warm Standby, а compliance mode не знімає ніхто.



Гроші — архітектура, що коштує менше, і інструменти контролю витрат

🎯 Вартість · 20%



Історія Хмаринки. Марта приходить з рахунком AWS: **\$4 800 за місяць** замість запланованих трьох тисяч. «Куди пішли гроші?»

Розслідування показує типові «діри»: сервери тестового середовища працюють ночами й на вихідних; 30 від'єднаних дисків EBS; **NAT Gateway** перекачує терабайти до S3; бази розраховані на пік «чорної п'ятниці» цілий рік; логи зберігаються назавжди. Жодна з цих проблем не видна без інструментів — і жодна не вимагає жертвувати надійністю. Оптимізація витрат — це четвертий домен іспиту (**20%** питань) і щоденна робота архітектора.



Образ. Рахунок AWS — як рахунок за комуналку великого будинку. **Cost Explorer** — виписка з графіками: куди пішли гроші. **Budgets** — ліміт на картці з SMS. **Cost Anomaly Detection** — банк, що дзвонить: «ви справді купили це о третій ночі?». **CUR** — чек до копійки на кожну лампочку. **Compute Optimizer** — кравець, що каже: «цей костюм вам завеликий на два розміри». А економія — це вимкнене світло в порожніх кімнатах, правильний розмір лампочок і оптовий тариф на те, що горить завжди.

Як AWS рахує гроші

Три головні складові майже будь-якого рахунку:

1. **Обчислення** — секунди чи години роботи серверів, контейнерів, функцій.
2. **Зберігання** — гігабайти на місяць (плюс запити й отримання даних для деяких класів).
3. **Передача даних** — гігабайти, що рухаються **між** зонами, регіонами й **в інтернет**. Найпідступніша складова: її не видно на схемі, поки не прийде рахунок.

Плюс «дрібниці», що накопичуються: **публічні IPv4-адреси** (з 2024 року платні — близько \$3,6 на місяць за кожну, навіть прив'язану), погодинна плата за NAT Gateway, балансувальники, VPC interface endpoints.

Ціна гігабайта: куди тече трафік



З інтернету в AWS

Безкоштовно

Завантажувати дані в хмару можна без плати за трафік



В одній AZ по приватній IP

Безкоштовно

Сервер ↔ сервер, сервер ↔ база в тій самій зоні



Між AZ одного регіону

≈ \$0,01/GB у кожен бік

Ціна надійності Multi-AZ: реплікація, балансувальник, «балакучі» мікросервіси



Між регіонами

≈ \$0,02/GB

Реплікація для DR, CRR, global tables



З AWS в інтернет

≈ \$0,09/GB, перші 100 GB/міс —

безкоштовно

Найдорожчий напрям: сторінки, файли, відео для користувачів



Через NAT Gateway

+ ≈ \$0,045–0,05/GB обробки +

погодинно

Стосується і трафіку до S3 та DynamoDB, якщо йти через NAT



До S3 і DynamoDB через gateway endpoint

Безкоштовно

Заміна NAT для трафіку до цих двох сервісів — розділ 15



З S3 і ALB до CloudFront

Безкоштовно

А з CloudFront до користувачів — дешевше, ніж напряму, і швидше завдяки кешу

Точні ціни залежать від регіону й обсягу, але пропорції варто пам'ятати: вхід — 0, одна AZ — 0, між AZ — копійки, в інтернет — найдорожче.



NAT Gateway чи gateway endpoint

Сервери Хмаринки в приватних підмережах щомісяця читають і пишуть у S3 **10 TB** (фото, бекапи, озеро даних).

- **Через NAT Gateway:** $10\,240\text{ GB} \times \approx \$0,05 = \approx \$500$ на місяць лише за обробку трафіку.
- **Через S3 gateway endpoint:** **\$0**. Налаштування — запис у таблиці маршрутів, 5 хвилин роботи.

Та сама логіка для DynamoDB. Для інших сервісів (SQS, KMS, Secrets Manager) — **interface endpoints**: вони платні ($\approx \$0,01$ на годину в кожній AZ + $\approx \$0,01/\text{GB}$), але при великих обсягах дешевші за NAT і безпечніші.

Інші прийоми економії на трафіку:

- **CloudFront** перед S3 і ALB — менше платного виходу в інтернет, більше кешу.  З листопада 2025 є **фіксовані тарифні плани CloudFront** без переплат (разом із WAF, захистом від DDoS і DNS) — зручно для сайтів з непередбачуваним трафіком.
- **Тримати «балакучі» компоненти в одній AZ**, де це не шкодить надійності: наприклад, обробники пакетних задач і їхній кеш.
- **Стиснення** відповідей і файлів, **менше копій** між регіонами (реплікувати лише те, що потрібно для DR).
- **S3 Requester Pays** — коли партнери завантажують великі датасети, за запити й трафік платять **вони**.
- **Direct Connect** — за великих обсягів трафік з AWS в офіс дешевший, ніж через інтернет.

Інструменти: бачити, передбачати, реагувати

Інструмент	Що робить	Коли на іспиті
AWS Cost Explorer	Графіки витрат за сервісами, акаунтами, тегами, регіонами; прогноз ; рекомендації Savings Plans і RI	«Проаналізувати витрати за останні місяці і спрогнозувати»
AWS Budgets	Сповіщення при перевищенні фактичних або прогнозованих витрат; Budget Actions — автоматичні дії	«Попередити, коли витрати перевищать суму» — розділ 7
Cost Anomaly Detection	ML помічає незвичні стрибки витрат і надсилає сповіщення (email, SNS)	«Автоматично виявити раптове зростання витрат»

Інструмент	Що робить	Коли на іспиті
Cost and Usage Report (CUR 2.0)	Найдетальніші дані: кожен ресурс, кожна година — у S3 (через Data Exports , зокрема у форматі FOCUS) для Athena і Quick Sight	«Найдетальніші дані про витрати для власного аналізу»
AWS Compute Optimizer	ML-рекомендації правильного розміру : EC2, Auto Scaling, EBS, Lambda, ECS на Fargate, RDS	«Сервери завеликі — які типи обрати»
Cost Optimization Hub	Усі рекомендації (розмір, Graviton, простої, Savings Plans) з усіх акаунтів в одному місці з оцінкою економії	«Централізовано знайти найбільші можливості економії»
Trusted Advisor	Перевірки вартості: простої балансувальників, недовантажені сервери, неприв'язані IP	«Регулярні перевірки кращих практик» — розділ 41
AWS Pricing Calculator	Оцінка вартості до побудови архітектури	«Оцінити вартість нового рішення»

Теги розподілу витрат (cost allocation tags). Щоб знати, скільки коштує кожен проєкт чи команда, ресурси позначають тегами (`project=shop` , `team=analytics` , `env=prod`) і **активують** ці теги в розділі Billing — лише після активації вони з'являються в Cost Explorer і CUR. **Cost Categories** групують витрати за власними правилами. В **AWS Organizations** рахунок консолідований: знижки за обсяг сумуються, а Savings Plans і RI за замовчуванням діляться між акаунтами.



Хто дивиться куди

Cost Explorer — дивиться **назад** і трохи вперед (аналіз і прогноз). **Budgets** — **попір** і попередження. **Anomaly Detection** — «**щось дивне**» без порогів. **CUR** — **сірі дані** до копійки. **Compute Optimizer** — «**який розмір**». Запитання «хто має сповістити?» → Budgets або Anomaly Detection; «де взяти найдетальніші дані?» → CUR.

Сім кроків економії

1. ПРИБРАТИ ЗАЙВЕ

Від'єднані томи EBS, старі знімки, неприв'язані IP, забуті тестові середовища, балансувальники без трафіку



2. ВИМИКАТИ НА НІЧ

Dev і test — лише в робочі години: EventBridge Scheduler або Instance Scheduler. 40 годин замість 168 на тиждень — мінус 75%



3. ПРАВИЛЬНИЙ РОЗМІР

Compute Optimizer: зменшити завеликі сервери і бази; перейти на **Graviton** (до 40% кращі ціна/продуктивність); gp2 → gp3



4. ЕЛАСТИЧНІСТЬ

Auto Scaling замість запасу «на пік», serverless (Lambda, Fargate, Aurora Serverless v2, DynamoDB on-demand) для нерівного навантаження



5. МОДЕЛЬ ОПЛАТИ

Стабільна база — **Savings Plans** (Compute, EC2, Database) чи RI; переривне — **Spot** (EC2, Fargate Spot); піки — On-Demand



6. СХОВИЩЕ І ДАНІ

Життєвий цикл S3 і **Intelligent-Tiering**, EFS IA, DynamoDB Standard-IA, retention логів, Parquet замість CSV для аналітики



7. МЕРЕЖА

Gateway endpoints замість NAT, CloudFront, менше міжрегіонального трафіку

Порядок важливий: спершу прибери зайве і підбери розмір, а вже потім купуй зобов'язання (Savings Plans) — інакше закупиш знижку на марнотратство.

Найчастіші прийоми за сервісами:

- **EC2:** правильний розмір, Graviton, Auto Scaling, Savings Plans для бази, Spot для переривних задач (розділ 20).
- **Контейнери й serverless:** Fargate Spot для стійких до переривань задач; Lambda на **arm64** (Graviton) дешевша; правильний обсяг пам'яті Lambda (Compute Optimizer або Power Tuning); Compute Savings Plans діють і на Fargate, і на Lambda.
- **Бази:** Aurora Serverless v2 для нерівного навантаження; **зупинка** RDS для dev (до 7 днів, потім AWS запустить знову); DynamoDB **on-demand** для непередбачуваного трафіку і **provisioned + auto scaling** (плюс reserved capacity) для стабільного; клас таблиці **Standard-IA** для рідко читаних даних;  **Database Savings Plans** (з грудня 2025).
- **S3:** класи зберігання й **lifecycle** (розділ 28), Intelligent-Tiering для непередбачуваного доступу, видалення незавершених multipart-завантажень і старих версій.
- **Логи й моніторинг:** retention для CloudWatch Logs, клас Infrequent Access, не збирати зайвих метрик з високою деталізацією.



Глибше: FinOps — витрати як спільна відповідальність

—

У великих компаніях оптимізацією керує практика **FinOps**: інженери бачать вартість своїх рішень (теги, дашборди за командами), фінансисти — прогнози, керівники — вартість одиниці бізнесу («скільки коштує хмара на одне замовлення»). Остання метрика найкорисніша: якщо рахунок зріс на 30%, а замовлень — на 60%, вартість одного замовлення **впала**, і це успіх, а не проблема. Хмаринка рахує: \$4 800 / 12 000 замовлень = **\$0,40 на замовлення** і ставить ціль \$0,25.

Як Хмаринка зекономила третину

Знахідка	Дія	Економія на місяць
Dev і test працюють 24/7	EventBridge Scheduler: лише будні 8:00–20:00	≈ \$520
10 TB/міс до S3 через NAT	S3 gateway endpoint	≈ \$500
Сервери завеликі (CPU 12%)	Compute Optimizer → менший тип + Graviton	≈ \$310
30 від'єднаних томів, старі знімки	Видалити; політика Data Lifecycle Manager	≈ \$140
Стабільна база обчислень	Compute Savings Plan на 1 рік	≈ \$180
Логи без retention	Retention 30 днів, архів — у S3 Glacier	≈ \$90
Разом		≈ \$1 740 (–36%)

Щоб не повернутися до старого: **Budgets** на кожен акаунт з прогнозним порогом, **Cost Anomaly Detection** для всіх сервісів, теги `project` і `env` обов'язкові (**tag policies** в Organizations), щомісячний перегляд Cost Optimization Hub.

Що обрати

Потреба	Рішення
Аналіз витрат за минулі місяці і прогноз	Cost Explorer
Сповіщення при перевищенні суми (фактичної чи прогнозованої)	AWS Budgets
Автоматично помітити незвичний стрибок витрат	Cost Anomaly Detection
Найдетальніші дані для аналізу в Athena	CUR 2.0 (Data Exports)
Рекомендації щодо розміру ресурсів	Compute Optimizer
Витрати за проектами чи командами	Cost allocation tags (активувати) / Cost Categories

Потреба	Рішення
Оцінити вартість до побудови	Pricing Calculator
Дешевший доступ до S3 з приватних підмереж	S3 gateway endpoint замість NAT
Зменшити витрати на роздачу контенту	CloudFront
Партнери мають платити за завантаження даних	S3 Requester Pays
Dev-середовище лише в робочі години	EventBridge Scheduler / Instance Scheduler

Спробуй у справжньому AWS: навести лад у витратах за 15 хвилин

Вартість: безкоштовно. **Час:** 15 хвилин (частина даних з'явиться через добу).

1. **Billing and Cost Management** → **Cost Explorer**: період — останні 3 місяці, **Group by: Service**, потім **Region** і **Usage type**. Знайди рядки з **DataTransfer** і **NatGateway** — так виглядає «невидимий» трафік. Увімкни **Forecast**.
2. **Cost allocation tags**: знайди тег **project** (якщо ти позначав ним ресурси в попередніх лабораторіях) і натисни **Activate**. Через добу за ним можна групувати витрати.
3. **Cost Anomaly Detection** → **Create monitor**: тип **AWS services**; підписка — email, поріг, наприклад, **\$5** або **40%**. Тепер незвичний стрибок не пройде повз тебе.
4. **Compute Optimizer** → **Get started** → **Opt in** (безкоштовно). Рекомендації з'являться після ~30 годин метрик — повернись завтра.
5. **Cost Optimization Hub** → **Enable**: сюди стікатимуться всі рекомендації з оцінкою економії.

Прибери за собою: нічого платного не створено. Монітор аномалій і opt-in можна залишити — вони безкоштовні й корисні.

Запам'ятай

Рахунок = обчислення + зберігання + передача даних (+ IPv4, NAT, балансувальники, endpoints).

Трафік: вхід — 0, одна AZ по приватній IP — 0, між AZ $\approx$ \$0,01 у кожен бік, між регіонами $\approx$ \$0,02, в інтернет $\approx$ \$0,09; NAT $\approx$ \$0,045/GB; gateway endpoint до S3/DynamoDB — 0; S3/ALB → CloudFront — 0.

Cost Explorer — аналіз і прогноз; **Budgets** — пороги й дії; **Anomaly Detection** — ML-стрибки; **CUR 2.0** — найдетальніші дані в S3 для Athena; **Compute Optimizer** — розмір; **Cost Optimization Hub** — усі рекомендації разом.

Cost allocation tags треба активувати; Organizations — консолідований рахунок, спільні знижки.

Порядок: прибрати зайве → вимикати на ніч → правильний розмір (Graviton, gp3) → еластичність → Savings Plans/Spot → сховище (lifecycle, Intelligent-Tiering) → мережа (endpoints, CloudFront).

Requester Pays — платить той, хто завантажує.

Як питають на іспиті

Проаналізувати витрати за останні пів року і спрогнозувати → **Cost Explorer**

Сповіщення, коли прогноз витрат перевищить бюджет → **AWS Budgets**

Автоматично виявляти незвичне зростання витрат без ручних порогів → **Cost Anomaly Detection**

Найдетальніші погодинні дані про витрати кожного ресурсу для Athena → **Cost and Usage Report (Data Exports)**

Рекомендації щодо розміру EC2 на основі використання → **Compute Optimizer**

Великі витрати на NAT Gateway через трафік до S3 → **S3 gateway VPC endpoint**

Зменшити вартість роздачі статичного контенту глобальним користувачам → **CloudFront**

Розподілити витрати між відділами → **cost allocation tags (активувати) і Cost Categories**

Тестові сервери потрібні лише в робочі години → **вимикати за розкладом (EventBridge Scheduler, Instance Scheduler)**

Партнери завантажують великі датасети і мають платити самі → **S3 Requester Pays**

Трафік між AZ дорожчає через «балакучі» сервіси → **розміщувати тісно пов'язані компоненти в одній AZ, де це прийнятно**

Пастки

Budgets не аналізує історію детально, а **Cost Explorer** не надсилає сповіщень про поріг.

Зупинений EC2 не платить за обчислення, але платить за **томи EBS** і публічні IP.

Трафік до S3 з приватної підмережі **через NAT платний**, навіть у тому самому регіоні.

Теги з'являються у звітах лише **після активації** як cost allocation tags.

Спершу **правильний розмір**, потім **зобов'язання** — інакше Savings Plan закріпить переплату.

Перевір себе

ПИТАННЯ 1

Фінансовий директор хоче побачити, як змінювалися витрати по кожному сервісу за останні 6 місяців, і отримати прогноз на наступні три. Що використати?

- A** AWS Budgets
- B** AWS Trusted Advisor
- C** AWS Cost Explorer
- D** AWS Cost and Usage Report з Athena

Правильна відповідь: C. Cost Explorer показує історію витрат з групуванням за сервісами, акаунтами, тегами і будує прогноз. CUR дає сирі дані, але вимагає власного аналізу.

ПИТАННЯ 2

Команді аналітики потрібні найдетальніші дані про витрати — по кожному ресурсу і кожній годині — щоб будувати власні звіти в Athena. Що налаштувати?

- A** AWS Budgets з деталізацією за ресурсами
- B** Щоденні звіти Cost Explorer у CSV
- C** AWS Cost and Usage Report (CUR 2.0 через Data Exports) у bucket S3
- D** CloudWatch metrics для billing

Правильна відповідь: C. CUR — найдетальніше джерело даних про витрати; Data Exports доставляє його в S3, де його зручно аналізувати в Athena.

ПИТАННЯ 3

Витрати акаунта зазвичай стабільні, але компанія хоче автоматично дізнаватися про незвичні стрибки (наприклад, через помилку в коді) без налаштування фіксованих порогів для кожного сервісу. Що обрати?

- A** AWS Config rules
- B** AWS Cost Anomaly Detection
- C** AWS Budgets з порогом для кожного сервісу
- D** Щотижневий ручний перегляд Cost Explorer

Правильна відповідь: B. Cost Anomaly Detection використовує ML, щоб виявляти незвичні зміни витрат відносно звичної поведінки, і надсилає сповіщення.

ПИТАННЯ 4

Інстанси EC2 у приватних підмережах щомісяця завантажують десятки терабайт з S3 у тому ж регіоні через NAT Gateway. Як найбільше зменшити витрати?

- A** Перенести інстанси в публічні підмережі
- B** Увімкнути S3 Transfer Acceleration
- C** Замінити NAT Gateway на NAT instance
- D** Створити S3 gateway VPC endpoint і додати маршрут у таблиці маршрутів приватних підмереж

Правильна відповідь: D. Gateway endpoint для S3 безкоштовний і прибирає плату NAT Gateway за обробку трафіку. Transfer Acceleration додає витрати, а публічні підмережі погіршують безпеку.

ПИТАННЯ 5

Компанія хоче знати, скільки кожен з п'яти відділів витрачає на AWS в одному акаунті. Що зробити?

- A** Увімкнути CloudTrail data events
- B** Позначити ресурси тегом department і активувати його як cost allocation tag
- C** Створити окремий VPC для кожного відділу
- D** Створити окремий бюджет без фільтрів

Правильна відповідь: B. Активовані cost allocation tags дозволяють групувати витрати в Cost Explorer і CUR за відділами.

ПИТАННЯ 6

Компанія публікує великі набори даних у S3 для партнерів і хоче, щоб плату за запити й передачу даних сплачували самі партнери. Що використати?

- A** S3 Requester Pays
- B** S3 Transfer Acceleration
- C** Presigned URLs
- D** S3 Intelligent-Tiering

Правильна відповідь: A. У bucket з Requester Pays плату за запити й завантаження сплачує той, хто завантажує, а власник платить лише за зберігання.

ПИТАННЯ 7

Команда хоче отримати рекомендації щодо зменшення розміру недовантажених інстансів EC2 і томів EBS на основі фактичного використання. Що використати?

- A** AWS Compute Optimizer
- B** AWS Pricing Calculator
- C** AWS Budgets
- D** AWS Cost Explorer Forecast

Правильна відповідь: A. Compute Optimizer аналізує метрики використання і рекомендує оптимальні типи інстансів, томів EBS, розміри Lambda та інше.

ПИТАННЯ 8

Які ресурси продовжують коштувати гроші, навіть якщо ними ніхто не користується? (Оберіть 2)

Оберіть 2 відповіді

- A** Публічна IPv4-адреса (Elastic IP), не прив'язана до сервера
- B** Порожня VPC з підмережами і таблицями маршрутів
- C** Роль IAM без політик
- D** Том EBS, від'єднаний від сервера
- E** Зупинений інстанс EC2 — за години обчислень

Правильна відповідь: A, D. Томи EBS оплачуються за виділений обсяг, а публічні IPv4 — щогодини. Зупинений інстанс не платить за обчислення (лише за диски), а VPC, підмережі й IAM безкоштовні.



Велика архітектура Хмаринки — усе разом

🎯 Безпека · 30%

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Вартість · 20%

🎬 **Історія Хмаринки.** Минув рік. Від сервера в підсобці не лишилося й сліду. Хмаринка продає в Україні, Польщі й Німеччині, у «чорну п'ятницю» приймає **50 000 відвідувачів за годину**, а платіжний партнер проводить аудит безпеки. Олена просить Тараса: «Покажи інвесторам **усю** нашу архітектуру — на одній сторінці, і поясни, чому кожна деталь саме така». Цей розділ — та сама сторінка. Тут немає нових сервісів: лише **зв'язки** між тими, які ти вже знаєш. Саме так мислить архітектор — і саме так побудовані питання іспиту.



Образ. Хмаринка тепер — не підсобка, а **місто**. Регіон — місто, зони доступності — райони з окремими електростанціями. Акаунти — окремі квартали з власною охороною. VPC — огорожені житлові комплекси, CloudFront — мережа кіосків у кожному місті Європи, черги — пошта між відомствами, бази — архіви, CloudWatch — диспетчерська, а резервний регіон — запасне місто, де вже горить світло в ратуші.

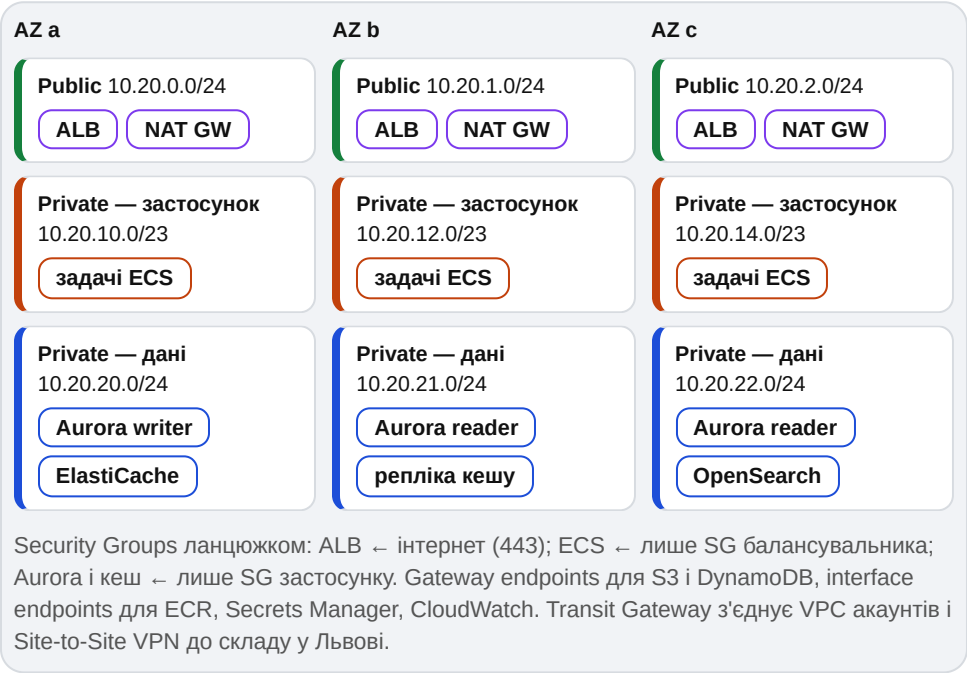
Уся архітектура на одній сторінці



Шари 1–5 — шлях запиту і даних; шари 6–7 пронизують усі інші. Кожен вузол — розділ курсу, і кожен можна пояснити одним реченням «чому саме так».

Мережа й акаунти

Акаунти (Organizations + Control Tower, розділ 10): `log-archive` і `audit` в OU Security, `network` з Transit Gateway в OU Infrastructure, `shop-prod`, `shop-dev`, `analytics-prod` в OU Workloads, пісочниця Тараса в OU Sandbox. **SCP** забороняють: виходити з організації, вимикати CloudTrail, GuardDuty і Config, працювати поза регіонами `eu-central-1` і `eu-west-1`, користуватися root.



Шлях одного замовлення

1. Покупка з Кракова: **Route 53** повертає адресу **CloudFront**; найближча точка присутності віддає з кешу сторінки каталогу й фото (з S3 через **OAC**), а **WAF** відсікає ботів і SQL-ін'єкції.
2. Динамічні запити йдуть на **ALB** → задачі **ECS на Fargate** у трьох AZ. Покупець увійшов через **Cognito** — API перевіряє його токен.
3. Каталог читається з **ElastiCache** (cache-aside); промах — з репліки **Aurora**. Пошук «свічка з корицею» — **OpenSearch**. Кошик — **DynamoDB** (мілісекунди за будь-якого навантаження).
4. «Оформити»: API записує замовлення в **Aurora (writer)** в одній транзакції і публікує подію **OrderPlaced** в **EventBridge**. Покупець одразу бачить «Дякуємо!» — решта відбувається асинхронно.
5. **Step Functions** веде процес: оплата через API партнера (вихід через **NAT**) → резерв на складі (повідомлення в **SQS**, яку читає складська програма через **VPN**) → PDF-рахунок (**Lambda** → S3) → **SNS** надсилає SMS. Помилка на будь-якому кроці — повтори, потім компенсація (скасувати резерв, повернути кошти).
6. Кожна подія сайту летить у **Kinesis Data Streams** → **Firehose** → озеро даних у S3 → **Athena** і **Redshift Serverless** → дашборд **Quick Sight** на телефоні Олени оновлюється.
7. Усе це видно в **CloudWatch** (метрики, логи, аларми) і **X-Ray** (траси), а кожен виклик API записано в **CloudTrail**.

Чому саме так: рішення і альтернативи

Рішення	Чому	Від чого відмовились і чому
ECS на Fargate	Контейнерний моноліт переїхав без переписування, без серверів для патчів	EC2 — більше адміністрування; EKS — зайва складність для однієї команди; Lambda — довгі запити й сталий трафік

Рішення	Чому	Від чого відмовились і чому
Aurora MySQL	Сумісність зі старою базою, 6 копій у 3 AZ, репліки, Global Database для DR	RDS MySQL — повільніший failover і менше реплік; DynamoDB — замовленням потрібні транзакції й JOIN
DynamoDB для кошиків	Мільйони дрібних записів у піки, TTL для старих кошиків	Aurora — саме кошики переважували її в «чорну п'ятницю»
EventBridge + Step Functions + SQS	Сайт не залежить від швидкості складу й оплати; повтори й компенсації без коду	Синхронні виклики — один повільний сервіс валив усе (розділ 31)
CloudFront + WAF	Швидкість у Польщі й Німеччині, захист на краю, дешевший трафік	Лише ALB — повільніше для далеких покупців і дорожчий вихід в інтернет
Pilot light в eu-west-1	RPO ≈ 1 с, RTO ≈ 30 хв за ціною реплікації даних	Active/active — подвійна ціна; Backup & Restore — RTO години
Багато акаунтів	Ізоляція prod і dev, централізовані журнали, SCP	Один акаунт — стажер у dev мір зачепити prod (розділ 10)

Перевірка за Well-Architected

Стовп	Що зроблено в Хмаринці
 Операційна досконалість	Уся інфраструктура — CloudFormation/CDK у git; дашборди й аларми CloudWatch; runbooks для аварій; game days щокварталу
 Безпека	Окремі акаунти, SCP, Identity Center з MFA, найменші привілеї; шифрування KMS скрізь, TLS з ACM; секрети в Secrets Manager з ротацією; GuardDuty, Security Hub, Macie, Config; WAF і Shield
 Надійність	Три AZ, Auto Scaling, Aurora Multi-AZ; черги між компонентами; AWS Backup з Vault Lock; pilot light в іншому регіоні; FIS-експерименти
 Ефективність продуктивності	CloudFront, ElastiCache, DynamoDB для кошиків, OpenSearch для пошуку; Graviton; правильний сервіс під кожен тип даних
 Оптимізація витрат	Savings Plans на базу, Fargate Spot для фонових задач, gateway endpoints, S3 Intelligent-Tiering, вимкнення dev на ніч, бюджети й Anomaly Detection

Стовп	Що зроблено в Хмаринці
 Сталий розвиток	Graviton, serverless і автомасштабування (немає простою заліза), життєвий цикл даних, менше зайвих копій

Двадцять шаблонів, які варто впізнавати

Більшість питань іспиту — варіації кількох десятків архітектурних шаблонів. Ти вже знаєш їх усі:

Шаблон	Коли	Сервіси
Тришаровий вебзастосунок з HA	«Високодоступний сайт з базою»	ALB + ASG у кількох AZ + RDS/Aurora Multi-AZ (+ ElastiCache)
Статичний сайт	«Статичний контент по всьому світу, дешево»	S3 + CloudFront (OAC) + Route 53 + ACM
Serverless API	«Без серверів, оплата за запити»	API Gateway + Lambda + DynamoDB (+ Cognito)
Розв'язування через чергу	«Піки, не втрачати замовлення»	SQS між шарами, Auto Scaling за глибиною черги
Fan-out	«Одна подія — кілька незалежних обробників»	SNS → кілька SQS (або EventBridge)
Обробка за подією	«Щойно файл завантажено — обробити»	Подія S3 → Lambda / EventBridge
Оркестрація процесу	«Кроки, повтори, компенсації, людське схвалення»	Step Functions
Сесії без стану	«Сервери масштабуються, користувачі не вилітають»	Сесії в ElastiCache або DynamoDB
Кеш для читання	«База перевантажена читанням»	ElastiCache, репліки для читання, DAX для DynamoDB
Приватний доступ до сервісів	«Без інтернету, дешевше за NAT»	Gateway / interface endpoints, PrivateLink
Мережа багатьох VPC	«Десятки VPC і офіс»	Transit Gateway (+ VPN / Direct Connect)
Гібридне сховище	«Офіс працює з файлами, дані в AWS»	Storage Gateway, FSx, DataSync

Шаблон	Коли	Сервіси
Міграція	«Сервери й бази в AWS з мінімальним простом»	MGN, DMS (+ SCT), DataSync
Глобальна низька затримка	«Користувачі на всіх континентах»	CloudFront, Global Accelerator, DynamoDB global tables, Aurora Global Database
Аналіз логів	«SQL по журналах без серверів»	CloudTrail / Flow Logs → S3 → Athena
Потоки в реальному часі	«Кліки, IoT, кілька споживачів»	Kinesis Data Streams → Lambda / Flink; Firehose → S3
Секрети з ротацією	«Не зберігати паролі в коді»	Secrets Manager (+ KMS)
Контроль ключів	«Свої ключі, аудит, ротація»	KMS customer managed keys; CloudHSM для окремого HSM
DR за RPO/RTO	«Аварія регіону з певними RPO/RTO»	Backup & Restore → Pilot Light → Warm Standby → Active/active
Оптимізація витрат	«Найдешевше без втрати надійності»	Правильний розмір, Savings Plans, Spot, lifecycle S3, endpoints

Карта курсу для повторення

Шар	Розділи
Основи й глобальна інфраструктура	2–6 · акаунт 7
Безпека й доступ	IAM 8 · ролі 9 · організації 10 · шифрування 11 · захист 12
Мережа	VPC 13 · безпека VPC 14 · зв'язки VPC 15 · гібрид 16 · Route 53 17 · край мережі 18
Обчислення	EC2 19 · ціни 20 · диски 21 · ELB 22 · Auto Scaling 23 · контейнери 24 · Lambda 25 · платформи 26
Сховища	S3 27 · S3 глибше 28 · файли 29 · гібридні сховища 30
Інтеграція й міграція	SQS/SNS 31 · події 32 · міграція 33
Бази даних	RDS 34 · Aurora 35 · DynamoDB 36 · кеш і спеціальні 37
Дані й AI	потоки 38 · аналітика 39 · AI 40
Експлуатація	моніторинг 41 · надійність 42 · гроші 43

Спробуй у справжньому AWS: огляд Well-Architected твоєї архітектури

Вартість: безкоштовно. **Час:** 30 хвилин.

1. Спершу **без підглядання**: на аркуші за 10 хвилин намалюй архітектуру Хмаринки — шари, сервіси, стрілки. Порівняй із схемою на початку розділу. Чого не вистачило — туди й повертайся на повторення.
2. **AWS Well-Architected Tool** → **Define workload**: назва `khmarynka` , регіон eu-central-1, середовище Production.
3. Обери лінзу **AWS Well-Architected Framework** і дай відповіді на перші 3–4 питання стовпа **Reliability** так, ніби це твоя архітектура з цього розділу.
4. Відкрий **Improvement plan**: інструмент покаже ризики (High/Medium risk) і посилання на кращі практики. Саме так проводять огляди архітектури в компаніях.

Прибери за собою: можна видалити workload (Workloads → Delete) — він безкоштовний, тож можна й залишити.

Запам'ятай

Архітектура — це **зв'язки**: вхід (DNS, CDN, WAF) → застосунок (ALB, контейнери, функції) → інтеграція (події, черги, оркестрація) → дані (правильна база під кожен тип даних) → аналітика; безпека, моніторинг, надійність і витрати — наскрізь.

Кожне рішення має **«чому»** і **відкинуту альтернативу** — так і відповідай на іспиті: спершу вимога, потім сервіс.

Мережа: **3 AZ × 3 шари підмереж**, SG ланцюжком, endpoints, Transit Gateway.

Акаунти: окремі prod, dev, log-archive, audit; **SCP** як запобіжники.

20 шаблонів покривають більшість питань — повторюй таблицю, доки не впізнаватимеш шаблон з першого речення питання.

Як питають на іспиті

Високодоступний вебзастосунок з базою → **ALB + Auto Scaling у кількох AZ + Aurora/RDS Multi-AZ**

Користувачі в різних країнах скаржаться на повільне завантаження статичного контенту → **CloudFront**

Сайт не повинен втрачати замовлення під час піків → **SQS між вебшаром і обробкою**

Одне замовлення мають обробити кілька незалежних сервісів → **SNS fan-out або EventBridge**

Багатокроковий процес з повторами й компенсаціями → **Step Functions**

Сервери масштабуються, а сесії користувачів губляться → **сесії в ElastiCache або DynamoDB**

Приватний доступ до S3 з приватних підмереж без NAT → **gateway endpoint**

Централізовані журнали й заборона вимикати аудит у всіх акаунтах → **organization trail + SCP**

Відновлення після аварії регіону з RTO ~30 хв за мінімальну ціну → **Pilot Light**

Пастки

Не тягни в архітектуру «модні» сервіси без вимоги: якщо питання не просить Kubernetes, EKS рідко найпростіша відповідь.

«Високодоступний» ≠ «відновлюється після аварії регіону»: Multi-AZ — HA, інший регіон — DR.

Кожен компонент у **одній** AZ — точка відмови: NAT Gateway, екземпляр бази без Multi-AZ, один сервер.

Перевір себе

ПИТАННЯ 1

Інтернет-магазин працює на одному інстансі EC2 з базою MySQL на тому ж сервері. Потрібно зробити його високодоступним і масштабованим з мінімальними змінами коду. Що обрати?

- A** Два інстанси EC2 в одній AZ з Route 53 simple routing
- B** Переписати застосунок на Lambda і DynamoDB
- C** ALB з Auto Scaling group у кількох AZ і Amazon Aurora MySQL (або RDS MySQL Multi-AZ)
- D** Більший інстанс EC2 з EBS io2

Правильна відповідь: C. Розділити веб і базу, розмістити вебшар у кількох AZ за балансувальником з автомасштабуванням, а базу — в керованому Multi-AZ сервісі. Більший сервер не прибирає єдину точку відмови, а переписування — не «мінімальні зміни».

ПИТАННЯ 2

Після переходу на Auto Scaling користувачів інколи «викидає» з облікового запису, коли балансувальник надсилає запит на інший сервер. Як виправити, зберігши масштабованість?

- A** Зберігати сесії в Amazon ElastiCache або DynamoDB
- B** Перейти на Network Load Balancer
- C** Зберігати сесії на EBS кожного сервера
- D** Увімкнути sticky sessions і вимкнути Auto Scaling

Правильна відповідь: A. Сервери мають бути без стану: сесії в спільному сховищі доступні будь-якому серверу. Sticky sessions лише маскують проблему і ламаються при масштабуванні.

ПИТАННЯ 3

Під час розпродажів сервіс обробки замовлень не встигає і частина замовлень втрачається. Сайт має приймати замовлення навіть тоді, коли обробка відстає. Що зробити?

- A** Додати репліки для читання бази
- B** Додати чергу SQS між сайтом і обробкою, масштабувати обробників за глибиною черги
- C** Використати CloudFront для API замовлень
- D** Збільшити тип інстансу обробки

Правильна відповідь: B. Черга приймає замовлення незалежно від швидкості обробки і зберігає їх до обробки; кількість обробників можна масштабувати за кількістю повідомлень.

ПИТАННЯ 4

Замовлення має незалежно отримати склад, служба листів і аналітика; падіння одного споживача не повинне впливати на інших. Що обрати?

- A** Три записи в DynamoDB
- B** Одну чергу SQS, з якої читають усі три сервіси
- C** Один виклик API, що послідовно викликає три сервіси
- D** Тему SNS, на яку підписані три окремі черги SQS

Правильна відповідь: D. SNS fan-out доставляє копію кожної події в окрему чергу кожного споживача; вони обробляють незалежно, а черги зберігають повідомлення під час збою споживача. З однієї черги кожне повідомлення отримав би лише один споживач.

ПИТАННЯ 5

Задачі ECS у приватних підмережах завантажують образи з ECR і читають секрети з Secrets Manager через NAT Gateway. Служба безпеки вимагає, щоб трафік не виходив в інтернет. Що зробити?

- A** Перемістити задачі в публічні підмережі
- B** Додати правило NACL, що забороняє 0.0.0.0/0
- C** Замінити NAT Gateway на NAT instance
- D** Створити VPC interface endpoints для ECR, Secrets Manager і CloudWatch Logs та gateway endpoint для S3

Правильна відповідь: D. Interface endpoints (PrivateLink) дають приватний доступ до сервісів AWS, а шари образів ECR зберігаються в S3, тому потрібен і gateway endpoint для S3.

ПИТАННЯ 6

Компанія має 12 акаунтів AWS. Потрібно, щоб журнали API всіх акаунтів збиралися централізовано і жоден адміністратор акаунта не міг вимкнути журналювання. Що зробити?

- A** Використати AWS Config aggregator
- B** Налаштувати IAM-політику в management account
- C** Створити organization trail з доставкою в S3 акаунта log-archive і SCP, що забороняє StopLogging і DeleteTrail
- D** Увімкнути CloudTrail окремо в кожному акаунті

Правильна відповідь: C. Organization trail збирає події всіх акаунтів, а SCP обмежує навіть адміністраторів акаунтів-учасників. Config фіксує конфігурації, а не виклики API.

ПИТАННЯ 7

Магазину потрібне відновлення після аварії регіону: втрата даних — не більше кількох секунд, простій — до 30 хвилин, мінімальна вартість. Що обрати?

- A** Aurora Multi-AZ в основному регіоні
- B** Aurora Global Database і DynamoDB global tables в резервному регіоні, застосунок розгортається з шаблонів при аварії, Route 53 failover
- C** Щоденні бекапи в інший регіон
- D** Повноцінна копія всієї системи в другому регіоні з розподілом трафіку 50/50

Правильна відповідь: B. Це Pilot Light: дані реплікуються постійно (RPO — секунди), а обчислення запускаються лише при аварії (RTO — десятки хвилин). Бекапи дають RPO в годинах, active/active дорожчий, Multi-AZ не захищає від втрати регіону.

ПИТАННЯ 8

Які рішення зменшать витрати Хмаринки без шкоди для надійності prod? (Оберіть 2)

Оберіть 2 відповіді

- A** S3 gateway endpoint замість NAT для трафіку до S3
- B** Вимкнути бекапи Aurora
- C** Перенести prod-базу з Multi-AZ в одну AZ
- D** Compute Savings Plan на стабільну базову потужність
- E** Запускати всі prod-сервіси на Spot без On-Demand

Правильна відповідь: A, D. Savings Plans знижують ціну стабільного навантаження, а gateway endpoint прибирає плату NAT за трафік до S3. Решта варіантів економить за рахунок надійності.



Розбір іспиту SAA-C03 — формат, домени, слова-підказки і метод відповіді

🎯 Безпека · 30%

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Вартість · 20%



Історія Хмаринки. Тарас записався на іспит AWS Certified Solutions Architect – Associate. За тиждень до дати він зізнається Олені: «Знаю сервіси, але боюся питань: вони довгі, англійською, і всі варіанти здаються правильними». Олена, яка колись складала бухгалтерські іспити, сміється: «Це не вікторина фактів. Це **65 коротких консультацій**: клієнт описує проблему й обмеження, а тиобираєш найкращу пораду з чотирьох. Навчися читати обмеження — і половина варіантів відпаде сама».



Образ. Кожне питання — **замовлення в ательє**: «костюм на весілля (контекст), до п'ятниці (обмеження часу), недорого (обмеження ціни), щоб не м'явся (вимога)». Чотири кравці пропонують варіанти. Один шиє чудовий костюм, але до наступного місяця. Другий — швидко, але дорого. Третій пропонує джинси. Четвертий виконує **всі** умови. Твоя робота — знайти четвертого.

Формат іспиту

Параметр	Значення
Питань	65: 50 оцінюються + 15 експериментальних (не знаєш, які саме — відповідай на всі)
Типи	Multiple choice — 1 правильна з 4; multiple response — 2 або більше з 5+ (кількість вказана в питанні)
Час	130 хвилин (2 хв на питання)
Бали	100–1000, прохідний — 720
Оцінювання	Компенсаторне: не треба «здати» кожен домен окремо; штрафу за неправильну відповідь немає — ніколи не лишай питання без відповіді
Часткові бали	Немає: у multiple response треба вибрати всі правильні варіанти
Ціна	\$150 (плюс податки)
Де	Тестовий центр Pearson VUE або онлайн з прокторингом (OnVUE)
Мови	Англійська та ще кілька мов (зокрема японська, корейська, китайська); української немає
Дійсний	3 роки



Складаєш англійською, а вона не рідна? Отримай +30 хвилин

AWS дає **ESL +30** — додаткові 30 хвилин для тих, для кого мова іспиту не рідна. Запит подають **до реєстрації на іспит** в акаунті AWS Certification (розділ про пристосування — *exam accommodations*). Це безкоштовно, і 160 хвилин замість 130 — величезна перевага. Обов'язково скористайся.

Чотири домени

Домен	Вага	Про що	Розділи
1. Безпечні архітектури	30%	Доступ до ресурсів (IAM, ролі, федерація, кілька акаунтів, SCP); захист застосунків (VPC, SG і NACL, WAF, Shield, endpoints, Cognito, секрети); захист даних (шифрування, KMS, ACM, бекапи, життєвий цикл)	8–12, 13–15, 27–28
2. Стійкі архітектури	26%	Масштабовані й слабко зв'язані системи (черги, події, Lambda, контейнери, кеш); висока доступність і стійкість до збоїв (Multi-AZ, DR, Route 53, бази)	22–25, 31–32, 34–36, 42
3. Високопродуктивні архітектури	24%	Сховища, обчислення, бази, мережа, отримання й перетворення даних — обрати найпродуктивніше рішення	17–21, 27–30, 34–40
4. Оптимізовані за вартістю	20%	Вартість сховищ, обчислень, баз і мережі	20, 28, 43

Висновок: **безпека** — найбільший домен, і вона присутня майже в кожному питанні інших доменів. А «найдешевше» з'являється набагато частіше, ніж 20% питань: у багатьох питаннях це додаткова умова.

Анатомія питання



Розбираємо питання на частини

*A company runs a web application on Amazon EC2 instances behind an Application Load Balancer. The application stores user-uploaded images on the instances' EBS volumes. **As the company scales**, users report that images are **sometimes missing**. The company needs a solution that is **highly available** and requires the **LEAST operational overhead**. What should a solutions architect do?*

- **Контекст:** EC2 за ALB, картинки на дисках EBS окремих серверів.

- **Симптом:** при масштабуванні картинок «немає» — кожен сервер бачить лише свій диск.
- **Вимоги:** висока доступність + **найменше операційних зусиль**.
- **Кандидати:** спільна файлова система **EFS** (працює, але це все одно файлова система, яку треба монтувати) або **S3** (об'єктне сховище, керований, безмежний, високодоступний).
- **Рішення:** зберігати завантаження в **Amazon S3** — найменше зусиль і найвища доступність. Варіант «синхронізувати диски скриптом через cron» — більше зусиль і ненадійно; «більший EBS» — не розв'язує проблему.

Структура майже завжди однакова: **контекст** → **проблема** → **вимоги й обмеження** → «**What should...?**». Правильну відповідь визначають **обмеження**, а не **контекст**.

Слова-підказки

Якщо в питанні...	Думай про...
LEAST operational overhead , fully managed	Керовані й serverless сервіси, вбудовані можливості замість власних скриптів: Lambda, Fargate, DynamoDB, Aurora Serverless, S3, AWS Backup
MOST cost-effective , lowest cost	Spot (переривне), Savings Plans (стабільне), класи S3 і lifecycle, gateway endpoints, правильний розмір, serverless для нерегулярного
Highly available	Кілька AZ: ALB + ASG, Multi-AZ бази, керовані регіональні сервіси
Fault tolerant , no downtime	Жодної єдиної точки відмови, надлишковість, статична стабільність
Disaster recovery , another Region	Стратегії DR за RPO/RTO, cross-region реплікація
Decouple , spikes, don't lose requests	SQS, SNS, EventBridge
Exactly once , in order	SQS FIFO

Якщо в питанні...	Думай про...
Real-time streaming, multiple consumers, replay	Kinesis Data Streams
Near real-time delivery to S3/Redshift	Amazon Data Firehose
Without code changes , lift and shift	Rehost: MGN, EC2, RDS з тим самим рушієм
Temporary credentials, cross-account	IAM roles, STS
Centrally manage accounts, prevent actions	Organizations, SCP, Control Tower
Rotate database credentials automatically	Secrets Manager
Customer-managed keys, key rotation, audit	KMS customer managed keys; окремий HSM → CloudHSM
SQL injection, XSS, rate limiting	AWS WAF
DDoS, cost protection, 24/7 response team	Shield Advanced
Who made changes	CloudTrail
Configuration compliance	AWS Config
Static IP, non-HTTP, global fast failover	Global Accelerator (+ NLB)
Path/host-based routing, HTTP	Application Load Balancer
Millions of requests, ultra-low latency, TCP/UDP	Network Load Balancer
Shared file system for Linux	EFS
Windows, SMB, Active Directory	FSx for Windows File Server
HPC, sub-millisecond, Lustre	FSx for Lustre
Block storage for one instance, IOPS	EBS (gp3, io2)

Якщо в питанні...	Думай про...
Microsecond reads for DynamoDB	DAX
Read-heavy relational database	Read replicas, ElastiCache
Many short-lived DB connections (Lambda)	RDS Proxy
Private access to AWS services	VPC endpoints (gateway для S3/DynamoDB, interface для решти)
Hundreds of VPCs, transitive routing	Transit Gateway
Dedicated private connection, consistent performance	Direct Connect (+ VPN як резерв або шифрування)
Query data in S3 with SQL, serverless	Athena
Hybrid storage for on-premises apps	Storage Gateway
Migrate databases, minimal downtime	DMS (+ SCT для різних рушіїв)
Transfer files online, schedule, sync	DataSync
SFTP/FTPS for partners	Transfer Family

Метод чотирьох кроків

1. **Прочитай останнє речення першим.** Що саме питають: «Which solution meets these requirements...», «What should... do to...», «Which combination of steps...(Choose two)».
2. **Впиши обмеження.** Найдешевше? Найменше зусиль? Без змін коду? У реальному часі? Висока доступність? Безпечно? Часто їх два-три, і правильна відповідь виконує **всі**.

3. **Відкинь очевидно неправильні.** Сервіс не для цієї задачі (Config для «хто видалив»), порушує обмеження (EC2 при «без серверів»), небезпечний (ключі доступу в коді, root), неіснуюча можливість.
4. **З двох, що лишилися,** обери ту, що виконує **всі** вимоги **найпростіше** і найбільше «по-AWS-івськи»: вбудована можливість сервісу перемагає власний скрипт, керований сервіс — самописний на EC2, автоматика — ручні кроки.



ВОВК

Вимога (що питають) → Обмеження (дешево, швидко, без коду) → Відкинь зайве → Краще з двох. Коли питання здається складним — прожени його через ВОВКа.

Типи хибних варіантів

Тип пастки	Як виглядає	Приклад
Справжній сервіс не на своєму місці	Звучить переконливо, але розв'язує іншу задачу	Config замість CloudTrail для «хто видалив»; Multi-AZ для масштабування читання
Надмірна інженерія	Працює, але складніше і дорожче, ніж треба	Власний кластер Kafka на EC2 замість SQS; EKS для однієї простої задачі
Частково правильний	Виконує одну вимогу, але не другу	Дешево, але в одній AZ; швидко, але з ручними кроками
Ручна робота	«Адміністратор щодня запускає скрипт» при вимозі автоматизації	Cron-скрипт замість lifecycle-правила S3
Антипатерн безпеки	Ключі доступу в коді, публічний bucket, root	«Зберегти access key в конфігураційному файлі на EC2» замість ролі

Тип пастки	Як виглядає	Приклад
Неіснуюча можливість	Вигадана функція сервісу	«Увімкнути шифрування в Security Group»

Розбір трьох питань

Питання 1: «найменше зусиль»

Компанія зберігає журнали застосунку в S3. Журнали мають бути доступні для частих запитів 30 днів, потім рідко — протягом року, потім видалятися. Потрібне рішення з найменшими операційними зусиллями.

- А. Lambda щодня копіює старі файли в інший bucket і видаляє їх через рік.
- В. **Правило S3 Lifecycle: через 30 днів — S3 Standard-IA (або Glacier Instant Retrieval), через 365 днів — видалення.**
- С. Щомісяця адміністратор вручну змінює клас зберігання.
- D. Зберігати журнали на EBS і робити знімки.

Думаємо: «найменше зусиль» відкидає А (код) і С (ручна робота); D — не той тип сховища. **В** — вбудована можливість S3, жодного коду.

Питання 2: дві вимоги одночасно

Застосунок у приватних підмережах має завантажувати великі обсяги даних з S3. Трафік не повинен іти через інтернет, а витрати мають бути мінімальними.

- А. NAT Gateway у кожній AZ.
- В. **Gateway VPC endpoint для S3 і маршрути в таблицях приватних підмереж.**
- С. Interface VPC endpoint для S3.

- D. Перемістити сервери в публічні підмережі.

Думаємо: А і D ведуть трафік через інтернет-шлюз — порушують першу вимогу. С — приватно, але платно (погодинно й за гігабайт). В — приватно і безкоштовно: виконує обидві вимоги.

✚ Питання 3: multiple response

Вебзастосунок на EC2 за ALB має витримувати втрату цілої AZ без простою. Які дві дії потрібні? (Choose two.)

- A. **Розмістити Auto Scaling group у кількох AZ.**
- B. Використати більший тип інстансу.
- C. **Перевести базу RDS у Multi-AZ.**
- D. Увімкнути detailed monitoring у CloudWatch.
- E. Зробити знімки EBS щогодини.

Думаємо: вимога — пережити втрату AZ. Потрібна надлишковість і обчислень (A), і бази (C). В не прибирає єдину точку відмови, D лише спостерігає, E — бекап, а не доступність (відновлення займе час).

Пари-близнюки

Пара	Ключова різниця
SQS vs SNS vs EventBridge	Черга (один обробник, буфер) · розсилка (багато підписників, без зберігання) · шина подій з правилами фільтрації, розкладом і SaaS
ALB vs NLB vs GWLB	HTTP/HTTPS, шляхи, хости · TCP/UDP, статичні IP, мільйони запитів · вставити файрволи сторонніх виробників
CloudFront vs Global Accelerator	Кешування контенту на краю (HTTP) · мережа AWS для будь-якого TCP/UDP, статичні anycast IP, швидкий failover
Security Group vs NACL	На інтерфейсі, stateful, лише allow · на підмережі, stateless, allow і deny, правила по номерах

Пара	Ключова різниця
Gateway vs interface endpoint	Лише S3 і DynamoDB, безкоштовно, маршрут · більшість сервісів, ENI з приватною IP, платно
VPC peering vs Transit Gateway	Пара VPC, нетранзитивно, дешево · хаб для сотень VPC і офісів, транзитивно
VPN vs Direct Connect	Через інтернет, шифровано, швидко налаштувати · приватний канал, стабільно, тижні на підключення, без шифрування за замовчуванням
Multi-AZ vs read replica	Доступність (standby не читається в класичному режимі) · масштабування читання й DR, асинхронно
Aurora Global Database vs cross-region read replica RDS	RPO ~1 с, до 10 регіонів, швидкий failover · простіше, повільніше, promote вручну
DynamoDB global tables vs DAX	Реплікація між регіонами (запис усюди) · кеш у пам'яті перед таблицею (мікросекунди)
KMS vs CloudHSM	Керований сервіс ключів, інтеграція з сервісами · виділений HSM, ключі повністю під твоїм контролем
Secrets Manager vs Parameter Store	Автоматична ротація, платний · налаштування й прості секрети, standard — безкоштовний
CloudTrail vs Config vs CloudWatch	Хто що зробив · яка конфігурація і чи відповідає правилам · як працює (метрики, логи, аларми)
Kinesis Data Streams vs Firehose	Реальний час, свої споживачі, replay · доставка в сховища без коду, near real-time
EFS vs FSx vs S3	Спільна NFS для Linux · Windows/SMB, Lustre, NetApp ONTAP, OpenZFS · об'єкти через API
DataSync vs Transfer Family vs DMS	Копіювання й синхронізація файлів · SFTP/FTPS/FTP для партнерів · міграція й реплікація баз
Savings Plans vs Reserved Instances	Зобов'язання \$/год, гнучкість сімейств і сервісів · конкретний тип інстансу (Convertible — з обміном)
Pilot Light vs Warm Standby	Дані живі, сервери вимкнені · зменшена робоча копія всього застосунку
SCP vs permissions boundary	Межа для всіх у акаунті (Organizations) · межа для конкретного користувача чи ролі
Cognito user pools vs identity pools	Вхід і токени користувачів застосунку · тимчасові AWS-облікові дані для доступу до сервісів

Старі назви і закриті сервіси

Банк питань оновлюється повільніше, ніж AWS. Ти можеш побачити старі назви — не лякайся:

На іспиті може бути	Сьогодні
Kinesis Data Firehose	Amazon Data Firehose
Kinesis Data Analytics	Managed Service for Apache Flink
Amazon QuickSight	Amazon Quick Sight (у складі Amazon Quick)
Amazon SageMaker	Amazon SageMaker AI
AWS Security Hub	Security Hub CSPM + новий Security Hub (з 2025)
Snowball Edge, Snowcone	Закриті для нових клієнтів (Snowcone — з 2024, Snowball Edge — з 11.2025); на іспиті — ще правильна відповідь для офлайн-перенесення терабайтів
Amazon Forecast, Fraud Detector	Закриті для нових клієнтів; на іспиті — ще можливі відповіді
S3 Select, S3 Object Lambda, Glacier vaults	Закриті для нових клієнтів; сучасні альтернативи — Athena, класи S3 Glacier
AWS App Runner	Закритий для нових клієнтів з 30.04.2026 → ECS Express Mode

Правило: **відповідай так, як задумано в питанні**. Якщо питання описує перенесення 500 TB з дата-центру з повільним каналом за тиждень — очікувана відповідь «Snowball Edge», навіть якщо сьогодні новий клієнт обрав би інший шлях.

Керування часом

- **Перший прохід (~90 хв):** відповідай на все; якщо вагаєшся довше 2 хвилин — обери найкращий варіант, **познач (flag)** і рухайся далі.
- **Другий прохід (~30 хв):** лише позначені питання. Часто наступні питання підказують відповідь на попередні.
- **Останні 10 хв:** перевір, що **немає пропущених** питань.

- Змінюй відповідь лише тоді, коли знайшов **конкретну причину**, а не «здається». Перша інтуїція, заснована на знаннях, зазвичай правильна.

День іспиту

- **Напередодні:** легке повторення «Як питають на іспиті» і пар-близнюків, **сон** важливіший за ще 50 питань.
- **Онлайн:** тихе приміщення, чистий стіл, стабільний інтернет, вебкамера; **перевірка системи** Pearson VUE заздалегідь; вхід за 30 хвилин. Не можна виходити з кадру, говорити вголос, мати телефон поруч.
- **У тестовому центрі:** прийди за 30 хвилин, документ з фото, ім'я — як в акаунті AWS Certification.
- **Під час:** дихай, пий воду до початку, не зациклюйся на одному питанні.

Після іспиту

- Результат (pass/fail і бали за доменами) приходить на пошту — зазвичай за кілька годин, максимум за **5 робочих днів**.
- **Цифровий бейдж** (Credly) — для LinkedIn і резюме; **ваучер –50%** на наступний іспит AWS; доступ до спільноти сертифікованих.
- Сертифікат дійсний **3 роки**; продовжується складанням цього ж або вищого іспиту (наприклад, Solutions Architect – Professional).
- Не склав? Повторити можна через **14 днів**. Звіт покаже слабкі домени — повертайся до відповідних розділів.



Спробуй: іспит у повних умовах

Вартість: безкоштовно. **Час:** 130 хвилин.

1. Відкрий пробний іспит цього застосунку: 65 питань, таймер, англійська з українським перекладом.

2. Умови — як на справжньому: без підглядання, без пауз, телефон в іншій кімнаті.
3. Після завершення розбери **кожну** помилку: чому правильна відповідь правильна і **чому твоя — ні**. Запиши обмеження, яке ти пропустив.
4. Додатково: безкоштовний **Official Practice Question Set** (20 питань) і курс **Exam Prep** на AWS Skill Builder.
5. Ціль — **стабільно 80%+** у двох спробах поспіль. Тоді реєструйся.

Запам'ятай

65 питань, 130 хвилин (+30 з ESL), 720 з 1000, штрафу за вгадування немає — відповідай на все.

Домени: **безпека 30%**, стійкість 26%, продуктивність 24%, вартість 20%.

Відповідь визначають **обмеження**: least operational overhead, most cost-effective, highly available, real-time, without code changes.

ВОВК: вимога → обмеження → відкинь зайве → краще з двох.

Вбудована можливість > власний скрипт; керований сервіс > самописний на EC2; автоматика > ручні кроки; роль > ключі доступу.

Старі назви на іспиті — нормально; відповідай так, як задумано в питанні.

Як питають на іспиті

«LEAST operational overhead» → **керований або serverless сервіс, вбудована можливість**

«MOST cost-effective» для переривних задач → **Spot; для стабільних → Savings Plans / RI**

«Highly available» → **кілька AZ; «disaster recovery» → інший region**

«Decouple» / «don't lose requests during spikes» → **SQS**

«Choose two» → **обидві відповіді мають бути необхідними, а не просто правильними фактами**

«Without changing application code» → **rehost / той самий рушій / мережеве рішення, а не переписування**

Пастки

Відповідь, **правильна фактично**, але не виконує **всіх** вимог, — **неправильна**.

Не додавай у питання власних припущень: якщо вимоги мінімальної ціни немає, найдорожчий варіант може бути правильним.

У multiple response перевір кількість: «Choose two» — рівно два.

Не витрачай 8 хвилин на одне питання: познач і повернись.

Перевір себе

ПИТАННЯ 1

Компанія хоче обробляти зображення, які користувачі завантажують у S3, з найменшими операційними зусиллями. Обробка триває до 30 секунд на зображення, навантаження нерівномірне. Що обрати?

- A** Кластер Amazon EMR
- B** Кластер Amazon EKS з автоскейлером вузлів
- C** Постійно працюючий інстанс EC2, що опитує bucket щохвилини
- D** Подія S3 запускає функцію AWS Lambda

Правильна відповідь: D. Короткі задачі за подією з нерівномірним навантаженням — ідеальний випадок для Lambda: без серверів, масштабування і оплата за виклики. Інші варіанти потребують керування інфраструктурою.

ПИТАННЯ 2

Застосунок на EC2 має доступ до DynamoDB. Зараз ключі доступу IAM-користувача записані в конфігураційному файлі. Як зробити найбезпечніше?

- A** Ротувати ключі доступу щомісяця вручну
- B** Зберігати ключі в змінних середовища
- C** Зашифрувати конфігураційний файл за допомогою KMS
- D** Призначити інстансу роль IAM з потрібними правами і прибрати ключі з файлу

Правильна відповідь: D. Роль IAM надає тимчасові облікові дані, що автоматично оновлюються, — жодних довгострокових ключів на сервері.

ПИТАННЯ 3

Компанії потрібне сховище для щоденних бекапів, які майже ніколи не читають, але мають зберігатися 7 років. Отримання даних за 12–48 годин прийнятне. Що найдешевше?

- A** EBS Cold HDD (sc1)
- B** S3 Glacier Deep Archive
- C** S3 Glacier Instant Retrieval
- D** S3 Standard-IA

Правильна відповідь: B. Deep Archive — найдешевший клас S3 для даних, які рідко потрібні і можуть відновлюватися до 48 годин.

ПИТАННЯ 4

Компанія переносить застосунок з власного дата-центру в AWS і хоче зробити це якнайшвидше, без змін коду. Застосунок працює на 40 віртуальних машинах. Що використати?

- A** AWS Elastic Beanstalk
- B** AWS Application Migration Service (rehost)
- C** Переписати застосунок на контейнери ECS
- D** AWS Database Migration Service для всіх серверів

Правильна відповідь: B. MGN реплікує сервери й запускає їх в AWS без змін коду — найшвидший шлях «lift and shift». DMS призначений для баз даних.

ПИТАННЯ 5

Вебсайт має витримувати раптові піки трафіку, а замовлення не повинні губитися, навіть якщо сервіс обробки тимчасово недоступний. Які дві дії допоможуть? (Оберіть 2)

Оберіть 2 відповіді

- A** Використати S3 Transfer Acceleration
- B** Увімкнути detailed monitoring
- C** Розмістити вебшар в Auto Scaling group за балансувальником у кількох AZ
- D** Додати чергу Amazon SQS між вебшаром і сервісом обробки
- E** Збільшити тип інстансу бази даних

Правильна відповідь: C, D. Черга зберігає замовлення, поки обробка недоступна, а Auto Scaling у кількох AZ приймає піки трафіку. Інші дії не розв'язують ні проблему піків, ні втрату замовлень.

ПИТАННЯ 6

Користувачі по всьому світу працюють з ігровим сервером через UDP. Потрібні статичні IP-адреси і швидке перемикання між регіонами при збої. Що обрати?

- A** AWS Global Accelerator
- B** Application Load Balancer
- C** Route 53 з простою маршрутизацією
- D** Amazon CloudFront

Правильна відповідь: A. Global Accelerator підтримує TCP і UDP, дає статичні anycast IP і перемикає трафік між регіонами за секунди. CloudFront і ALB працюють з HTTP(S).

ПИТАННЯ 7

Питання вимагає «LEAST operational overhead». Який варіант зазвичай НЕ підходить?

- A** Вбудована можливість сервісу (наприклад, правило S3 Lifecycle)
- B** Власне рішення на EC2 зі скриптами cron, яке треба патчити й моніторити
- C** Serverless-сервіс (наприклад, AWS Lambda або Fargate)
- D** Керований сервіс (наприклад, Amazon RDS замість бази на EC2)

Правильна відповідь: B. «Найменше операційних зусиль» означає обирати керовані, serverless і вбудовані можливості. Власні сервери й скрипти — це саме те, що створює операційне навантаження.

ПИТАННЯ 8

Компанія повинна зберігати дані в S3 і автоматично видаляти об'єкти через 90 днів. Також потрібно заборонити будь-кому видаляти об'єкти раніше цього терміну. Які дві дії? (Оберіть 2)

Оберіть 2 відповіді

- ☐ А Увімкнути S3 Object Lock у режимі compliance з терміном 90 днів
- ☐ В Використати S3 Transfer Acceleration
- ☐ С Налаштувати правило S3 Lifecycle на видалення через 90 днів
- ☐ D Налаштувати Lambda, що щодня видаляє старі об'єкти
- ☐ Е Увімкнути версіонування без інших налаштувань

Правильна відповідь: А, С. Object Lock у compliance mode забороняє видалення до кінця терміну, а Lifecycle автоматично видаляє об'єкти після нього. Lambda — зайвий код, версіонування саме по собі не забороняє видалення.



Що далі — план підготовки, чек-лист готовності і шлях після сертифіката

🎯 Безпека · 30%

🎯 Відмовостійкість · 26%

🎯 Продуктивність · 24%

🎯 Вартість · 20%



Історія Хмаринки. Тарас склав іспит — **842 бали**. Уранці на його столі стоїть свічка з корицею й записка від Олени: «Вітаю, архітекторе! А тепер — Польща і Німеччина». Тарас розуміє: сертифікат — це не фініш. Це **посвідчення водія**: воно дозволяє їздити самостійно, а справжня майстерність приходить дорогою. Цей розділ — твоя дорожня карта: як дійти до іспиту впевнено і що робити після нього.



Образ. Навчання — це **спортзал**, а не бібліотека. Перечитувати розділи — як дивитися, як тренуються інші. **Згадувати** відповідь без підказки, **пояснювати** вголос і **будувати руками** — це і є тренування: важко, але саме від цього «ростуть м'язи» пам'яті.

План підготовки

Базовий план — **6 тижнів по 1–1,5 години на день**. Поспішаєш — стискай до 3–4 тижнів по 2–3 години; маєш менше часу щодня — розтягуй до 8–10 тижнів. Головне — **регулярність**: 45 хвилин щодня краще за 5 годин у неділю.

Тиждень	Розділи	Практика
1	Старт і основи IT 1–7: хмара, регіони, акаунт; IAM і ролі 8–9	Лабораторії з бюджетом і IAM; «Перевір себе» після кожного розділу
2	Організації, шифрування, захист 10–12; мережа 13–18	Намалюй VPC з трьома шарами підмереж з пам'яті
3	Обчислення 19–26: EC2, ціни, диски, ELB, Auto Scaling, контейнери, Lambda	Щодня 20 хвилин карток
4	Сховища 27–30; інтеграція й міграція 31–33	Перший прохід «Перевір себе» по всіх розділах
5	Бази 34–37; дані й AI 38–40; експлуатація 41–43	Пробний іспит №1 у кінці тижня
6	Фінал 44–46; повторення слабких місць	Картки «Лише слабкі», помилки, пробний іспит №2 ; якщо обидва $\geq 80\%$ — реєструйся

Щоденний ритуал

1. **10–20 хвилин карток** (Практика → Картки): застосунок збирає їх з усіх рядків «Як питають на іспиті». Спершу згадай відповідь сам, потім відкрий і чесно натисни «Знав» чи «Не знав». Картки, які ти не знав, повертаються частіше — це **інтервальне повторення**.
2. **Новий матеріал** — один розділ або його половина. Читай з увімкненими  **Підказками**: торкнись назви сервісу чи терміна — побачиш коротке пояснення.
3. **Одразу після розділу — «Перевір себе»**, не підглядаючи. Помилка — не провал, а найцінніший момент: прочитай пояснення і знайди в розділі, де про це сказано.
4. **Раз на тиждень — лабораторія** в справжньому AWS. Руки пам'ятають довше за очі.
5. **Поясни вголос** одну тему (другу, коту, дзеркалу): «Чим Pilot Light відрізняється від Warm Standby?». Якщо не виходить просто — ти ще не розумієш до кінця (метод Фейнмана).



Чому це працює

Мозок запам'ятовує те, що **витягує**, а не те, що **бачить**. Кожна спроба згадати відповідь — навіть невдала — зміцнює пам'ять сильніше, ніж три перечитування. Тому в курсі стільки тестів, карток і питань «чому».

Як використовувати цей застосунок

- **Курс** — 46 розділів; позиція читання зберігається, а прогрес видно на головній.
- 🔍 **Пошук** — будь-який термін: «NAT», «RPO», «Glacier».
- 🧠 **Підказки** — пояснення сервісів і термінів прямо в тексті; **Словник** — усі сервіси з образом, «коли» і «з чим плутають»; **Схеми** — усі діаграми курсу для швидкого повторення.
- **Практика:** ✅ «Перевір себе» з фільтрами «Ще без відповіді» і «Лише мої помилки», 🇺🇦 **Картки** з режимом слабких місць, 📝 **Пробний іспит** — 65 питань англійською з українським перекладом, таймером і позначками 🚩.
- **Офлайн:** встанови застосунок на телефон (у браузері — «Додати на головний екран» чи «Встановити»), і все працюватиме без інтернету — у метро, літаку, черзі.
- ⬇ **PDF** — увесь підручник для друку чи читання з планшета.

Чек-лист готовності

Перед реєстрацією чесно пройди по списку. Кожен пункт — «так, можу пояснити без підглядання»:



Безпека (30%)

- ☐ Порядок оцінки політик IAM: явна заборона → SCP → дозволи; коли потрібна роль, а коли користувач

- ☐ Доступ між акаунтами через полі; федерація і IAM Identity Center; Cognito user pools і identity pools
- ☐ SCP, RCP і permissions boundary — хто кого обмежує
- ☐ KMS (ключі AWS і власні, політики ключів, envelope encryption), CloudHSM, ACM, Secrets Manager і Parameter Store
- ☐ Security Groups і NACL; WAF, Shield, Firewall Manager, Network Firewall; GuardDuty, Inspector, Macie, Security Hub
- ☐ Захист S3: Block Public Access, bucket policy, ОАС, шифрування, Object Lock, presigned URLs

Стійкість (26%)

- ☐ Multi-AZ проти read replicas; Aurora і її Global Database; DynamoDB global tables
- ☐ SQS (standard, FIFO, visibility timeout, DLQ), SNS fan-out, EventBridge, Step Functions
- ☐ ALB, NLB, GWLB; Auto Scaling (target tracking, step, scheduled, predictive)
- ☐ Route 53: типи маршрутизації і health checks; CloudFront і Global Accelerator
- ☐ RPO/RTO і чотири стратегії DR; AWS Backup, Vault Lock, Elastic Disaster Recovery

Продуктивність (24%)

- ☐ Типи EBS (gp3, io2, st1, sc1) і instance store; EFS, FSx (Windows, Lustre, ONTAP, OpenZFS)
- ☐ Класи S3, multipart upload, Transfer Acceleration, CloudFront
- ☐ Вибір бази: RDS, Aurora, DynamoDB (+ DAX), ElastiCache, OpenSearch, Neptune, Redshift
- ☐ Kinesis, Firehose, Glue, Athena, EMR — хто за що відповідає
- ☐ Lambda (ліміти, способи виклику, provisioned concurrency), ECS/EKS/Fargate

Вартість (20%)

- ☐ On-Demand, Savings Plans, RI, Spot, Dedicated — коли що
- ☐ Життєвий цикл S3, Intelligent-Tiering, Glacier-класи
- ☐ Ціна передачі даних: між AZ, регіонами, в інтернет; NAT проти gateway endpoint
- ☐ Cost Explorer, Budgets, Anomaly Detection, CUR, Compute Optimizer

Усі пункти — «так», два пробні іспити $\geq 80\%$ — **ти готовий**.

Офіційні ресурси

- **Сторінка сертифікації і Exam Guide** — точний перелік тем і сервісів: aws.amazon.com/certification
- **AWS Skill Builder** — безкоштовні **Official Practice Question Set** (20 питань від авторів іспиту) і курс **Exam Prep**; за підпискою — **Official Pretest** і лабораторії: skillbuilder.aws
- **AWS Well-Architected Framework** — шість стовпів з прикладами: docs.aws.amazon.com/wellarchitected
- **FAQ сервісів** (S3, EC2, VPC, RDS, DynamoDB, Lambda) — багато питань іспиту взяті саме з нюансів, описаних там.
- **Що нового в AWS** — щоденні оновлення: aws.amazon.com/new
- **Короткий курс-шпаргалка** цього ж проєкту — для швидкого повторення перед іспитом: nexgen-fullstack.github.io/AWSSAAC03

Наступні кроки після сертифіката

Станом на 2026 рік лінійка сертифікацій AWS виглядає так:

Рівень	Сертифікат	Для кого
Associate	Developer	Розробники: Lambda, DynamoDB, API, CI/CD

Рівень	Сертифікат	Для кого
Associate	CloudOps Engineer (колишній SysOps Administrator, з вересня 2025)	Експлуатація: моніторинг, автоматизація, мережа, контейнери
Associate	Data Engineer	Конвеєри даних: Glue, Kinesis, Redshift, озера даних
Associate	Machine Learning Engineer	ML у продакшені: SageMaker AI, MLOps
Foundational	AI Practitioner	Основи AI і генеративного AI — швидкий і корисний доповнювач
Professional	Solutions Architect – Professional	Логічний наступний крок: багато акаунтів, міграції, складні гібридні архітектури
Professional	DevOps Engineer – Professional	CI/CD, IaC, автоматизація в масштабі
Professional	Generative AI Developer – Professional (з 2026)	Застосунки на Bedrock: RAG, агенти, безпека AI
Specialty	Security – Specialty	Глибока безпека AWS

Сертифікати **Machine Learning – Specialty** (31.03.2026) і **Advanced Networking – Specialty** (25.08.2026) виведено з обігу — не плануй їх.

Кар'єра й досвід

- **Портфоліо.** Збудуй міні-Хмаринку у своєму акаунті **кодом** (CloudFormation, CDK або Terraform) і виклади в GitHub з README, схемою й оцінкою вартості. Роботодавці цінують це більше, ніж ще один сертифікат.
- **Ролі:** Cloud Engineer, DevOps Engineer, Solutions Architect (часто після 2–3 років практики), Cloud Security Engineer, Data Engineer.
- **Спільнота:** AWS User Group Ukraine, конференції, програма **AWS Community Builders**; пиши короткі статті про свої лабораторії — найкращий спосіб закріпити знання.

- **Будь у курсі:** щорічна конференція **re:Invent** (грудень) приносить десятки анонсів; читай «What's New» раз на тиждень і перевіряй, що змінилося в сервісах, з якими працюєш.

Спробуй: твій перший власний проєкт

Вартість: у межах безкоштовного плану AWS, якщо прибрати за собою. **Час:** 1–2 вихідні.

1. **AWS Budgets** — бюджет \$5 з попередженням на 80% (як у розділі 7).
2. **Фронтенд:** статичний сайт-вітрина в S3 + **CloudFront** з ОАС (без публічного bucket).
3. **Бекенд: API Gateway + Lambda + DynamoDB** — «додати товар у кошик» і «показати кошик».
4. **Інфраструктура як код:** опиши все в **AWS SAM** або **CDK**, щоб однією командою створювати й видаляти середовище.
5. **Спостереження:** аларм CloudWatch на помилки Lambda з повідомленням у SNS; трасування.
6. **README** зі схемою архітектури, поясненням «чому саме так» за шістьма стовпами Well-Architected і оцінкою вартості в Pricing Calculator.

Прибери за собою: `sam delete` або `cdk destroy`, перевір, що в S3 і DynamoDB нічого не лишилося, і подивись Cost Explorer через добу.

Останнє слово

Ти пройшов шлях від «що таке сервер» до архітектури, яка витримує розпродажі, переживає аварії регіонів, захищає дані й рахує гроші. Цього не можна вивчити за вечір — і ти не вчив за вечір. Кожен розділ, кожна лабораторія, кожна помилка в тесті додали цеглину.

На іспиті пам'ятай: ти не вгадуєш — ти **консультуєш**. Прочитай вимоги, знайди обмеження, відкинь зайве, обери найпростіше рішення, що виконує все. Ти вже робив це сотні разів у цьому курсі.

Удачі на іспиті — і до зустрічі в хмарі! ☁

Запам'ятай

Регулярність важливіша за тривалість: щодня картки + розділ + «Перевір себе».

Активне пригадування і пояснення вголос працюють краще за перечитування.

Готовність: **чек-лист** без прогалин і **два пробні іспити $\geq 80\%$** .

Використовуй **ESL +30**, якщо складаєш англійською.

Після сертифіката: портфоліо в кодї, спільнота, **Solutions Architect – Professional** як наступна мета.

Перевір себе

ПИТАННЯ 1

Який спосіб підготовки найкраще зміцнює довготривалу пам'ять?

- A** Тричі перечитати розділ підряд
- B** Спершу згадати відповідь без підказки (картки, тести), потім перевірити
- C** Виділити маркером найважливіше в тексті
- D** Переглянути відео на подвійній швидкості

Правильна відповідь: B. Активне пригадування (retrieval practice) змушує мозок витягувати знання, що зміцнює пам'ять значно сильніше, ніж пасивне перечитування.

ПИТАННЯ 2

Ти складаєш іспит англійською, яка не є твоєю рідною мовою. Що варто зробити до реєстрації?

- A** Подати запит на пристосування ESL +30 в акаунті AWS Certification
- B** Попросити проктора під час іспиту продовжити час
- C** Обрати іспит українською
- D** Нічого: додатковий час дають автоматично всім

Правильна відповідь: A. ESL +30 надає додаткові 30 хвилин, але запит потрібно подати заздалегідь, до реєстрації на іспит. Української версії іспиту немає.

ПИТАННЯ 3

Який показник найкраще свідчить, що ти готовий реєструватися на іспит?

- A** Ти знаєш напам'ять ціни всіх сервісів
- B** Ти прочитав усі розділи один раз
- C** Ти пройшов усі лабораторії без помилок
- D** Стабільно 80% і більше у двох пробних іспитах поспіль і чек-лист без прогалин

Правильна відповідь: D. Прохідний бал — 720 з 1000; стабільні 80%+ на пробних іспитах дають запас на хвилювання і незнайомі формулювання.

ПИТАННЯ 4

Скільки дійсний сертифікат AWS Certified Solutions Architect – Associate і як його продовжити?

- A** 1 рік; пройти онлайн-курс
- B** Безстроково
- C** 3 роки; скласти цей самий іспит знову або вищий, наприклад Solutions Architect – Professional
- D** 2 роки; сплатити внесок

Правильна відповідь: C. Сертифікати AWS дійсні три роки. Продовження — складання актуальної версії іспиту того ж або вищого рівня.

ПИТАННЯ 5

Ти не склав іспит з першої спроби. Що найрозумніше зробити?

- A** Розібрати звіт за доменами, повторити слабкі розділи і скласти знову не раніше ніж через 14 днів
- B** Перейти на іншу сертифікацію
- C** Одразу наступного дня записатися знову без підготовки
- D** Вивчити напам'ять відповіді з неофіційних «дампів»

Правильна відповідь: A. Звіт показує слабкі домени — туди й варто спрямувати повторення. Повторна спроба можлива через 14 днів. «Дампи» порушують правила AWS і не дають розуміння.

Додаток. Словник сервісів і термінів

Основи IT

 **API** — Домовленість, як програми звертаються одна до одної: адреса, метод, формат запиту й відповіді (часто JSON через HTTPS).

 **BGP** — Протокол, яким мережі обмінюються маршрутами: «ці адреси — через мене».

 **CapEx і OpEx** — CapEx — великі вкладення наперед (купити сервери); OpEx — поточні витрати за фактичне використання (оплата хмари щомісяця).

 **CI/CD** — Автоматичний конвеєр: кожна зміна коду збирається, тестується й розгортається без ручних кроків.

 **DNS** — «Телефонна книга» інтернету: перетворює імена (khamryuka.ua) на IP-адреси.

 **ETL** — Extract, Transform, Load — витягнути дані з джерел, перетворити (очистити, об'єднати) і завантажити в сховище для аналізу.

 **HTTP і HTTPS** — Протокол вебу: браузер надсилає запит (GET, POST), сервер повертає відповідь з кодом (200 — успіх, 404 — не знайдено, 5xx — помилка сервера). HTTPS — те саме, зашифроване TLS.

 **IaaS, PaaS, SaaS** — Моделі хмари: IaaS — орендуєш сервери й мережу (EC2); PaaS — платформа, де ти лише розгортаєш код (Elastic Beanstalk); SaaS — готова програма (пошта, CRM).

 **IP-адреса** — Числова адреса пристрою в мережі (IPv4 — 10.0.1.25, IPv6 — довша). Буває публічна (в інтернеті) і приватна (у внутрішній мережі).

 **JSON** — Текстовий формат даних з пар «ключ: значення» і списків — мова спілкування API і політик IAM.

 **NoSQL** — Бази без жорсткої табличної схеми: ключ-значення, документи, графи. Масштабуються горизонтально й швидко знаходять дані за ключем.

 **OLTP і OLAP** — OLTP — багато дрібних транзакцій (замовлення, оплати); OLAP — важкі аналітичні запити по мільйонах рядків.

 **On-premises** — Обладнання у власному приміщенні чи дата-центрі компанії, а не в хмарі.

 **SLA** — Угода про рівень сервісу: яку доступність обіцяє постачальник (наприклад, 99,99%) і яку компенсацію дасть, якщо не дотримає.

 **SQL і реляційні бази** — Мова запитів до реляційних баз: таблиці з рядками й колонками, зв'язки (JOIN), транзакції.

-  **SSH і RDP** — Протоколи віддаленого керування серверами: SSH — командний рядок Linux (порт 22), RDP — робочий стіл Windows (порт 3389).
-  **TCP і UDP** — Транспортні протоколи: TCP — з гарантією доставки й порядку (веб, бази), UDP — швидко, без гарантій (відео, ігри, DNS).
-  **TLS (SSL)** — Шифрування з'єднання між клієнтом і сервером; сертифікат доводить, що сервер справжній.
-  **TTL** — Time to live — скільки секунд дані вважаються свіжими або коли їх буде автоматично видалено.
-  **Автентифікація й авторизація** — Автентифікація — хто ти (пароль, MFA, токен); авторизація — що тобі дозволено (політики).
-  **Балансувальник навантаження** — Розподіляє вхідні запити між кількома серверами і не надсилає їх на несправні.
-  **Бекап (резервна копія)** — Копія даних на певний момент часу, з якої можна відновитися після видалення, збою чи атаки.
-  **Висока доступність** — Система працює майже без перерв, переживаючи збої окремих частин — сервера чи цілої зони.
-  **Віртуальна машина** — Програмна «копія» комп'ютера зі своєю ОС, що працює на спільному фізичному сервері поруч з іншими VM.
-  **Гіпервізор (Nitro)** — Програма, що ділить фізичний сервер на віртуальні машини й ізолює їх одна від одної. В AWS — Nitro.
-  **Дата-центр** — Будівля з тисячами серверів, живленням, охолодженням і мережею. Зона доступності AWS — один або кілька дата-центрів.
-  **Затримка (latency)** — Час від запиту до відповіді, зазвичай у мілісекундах. Залежить від відстані, мережі й швидкості сервісу.
-  **Кеш** — Швидке тимчасове сховище (зазвичай у пам'яті) для часто потрібних даних, щоб щоразу не звертатися до повільного джерела.
-  **Контейнер** — Застосунок разом з усіма бібліотеками в одному пакеті (образі), що запускається однаково будь-де і ділить ядро ОС з іншими контейнерами.
-  **Масштабування** — Вертикальне — більший сервер (scale up); горизонтальне — більше серверів (scale out). Хмара любить горизонтальне.
-  **Мікросервіси й моноліт** — Моноліт — увесь застосунок одним блоком; мікросервіси — багато маленьких незалежних сервісів, що спілкуються через API і події.
-  **Озеро даних** — Усі дані компанії в дешевому сховищі (S3) у різних форматах, з якого будують аналітику.

 **Оперативна пам'ять (RAM)** — Швидка тимчасова пам'ять, у якій працюють програми й дані; вміст зникає при вимкненні.

 **Операційна система** — Базова програма комп'ютера (Linux, Windows), що керує залізом і запускає інші програми.

 **Персональні дані (PII)** — Дані, за якими можна визначити людину: ім'я, телефон, адреса, номер документа. Їх захищають закони (наприклад, GDPR).

 **Подієва архітектура** — Компоненти реагують на події («замовлення створено»), а не викликають одне одного напряму.

 **Порт** — Номер «дверей» на сервері, за яким працює конкретна програма: 443 — HTTPS, 22 — SSH, 3306 — MySQL.

 **Пропускна здатність** — Скільки даних проходить за секунду (Mbps, Gbps, MiB/s). Не плутай із затримкою: широка труба може бути довгою.

 **Процесор (CPU, vCPU)** — «Мозок» комп'ютера, що виконує обчислення; має кілька ядер. vCPU — віртуальне ядро інстансу.

 **Реплікація** — Автоматичне копіювання даних в іншу базу, зону чи регіон, щоб мати актуальну копію.

 **Сервер** — Комп'ютер, що цілодобово працює в мережі й виконує запити інших програм: віддає сторінки, рахує, зберігає дані.

 **Синхронно й асинхронно** — Синхронно — чекаєш відповіді, перш ніж йти далі; асинхронно — відправив і працюєш далі, результат прийде пізніше.

 **Стійкість до збоїв** — Система працює без жодної перерви навіть під час збою — за рахунок повної надлишковості.

 **Сховище даних (data warehouse)** — База, оптимізована для аналітики (OLAP): дані зберігаються по колонках, важкі запити йдуть по мільярдах рядків.

 **Токен доступу (JWT)** — Підписаний цифровий «квиток», який сервер видає після входу; застосунок показує його з кожним запитом.

 **Тротлінг** — Обмеження кількості запитів за секунду, щоб захистити сервіс від перевантаження (відповідь 429 Too Many Requests).

 **Узгодженість даних** — Сильна — після запису кожне читання бачить нове значення; «з часом» (eventual) — нове значення з'являється через мить, зате швидше й дешевше.

 **Файрвол** — Фільтр мережевого трафіку за правилами: що пропускати, а що ні.

 **Хешування** — Одностороннє перетворення даних у короткий «відбиток»: з хешу не відновиш дані, але можеш перевірити, що вони не змінилися.

 **Шифрування** — Перетворення даних на нечитабельні без ключа. At rest — на дисках і в сховищах; in transit — під час передачі (TLS).

 **Ідемпотентність** — Повторне виконання операції дає той самий результат, що й одне: двічі оброблене замовлення не списує гроші двічі.

 **Інстанс (instance)** — Один запущений віртуальний сервер (або екземпляр бази чи кешу) в AWS певного типу: скільки vCPU, пам'яті й мережі.

 **Інфраструктура як код** — Опис серверів, мереж і баз у текстових файлах, з яких інфраструктура створюється автоматично й однаково щоразу.

Поняття

 **AMI** — Образ машини: ОС + ПЗ + налаштування для запуску EC2; регіональний, копіюється в інші регіони.

 **Apache Parquet** — Стовпчиковий формат файлів для аналітики: читаються лише потрібні колонки, дані стиснуті, тож Athena чи Redshift Spectrum сканують у рази менше даних.

 **Availability Zone** — Один або кілька ізольованих дата-центрів у регіоні з окремим живленням і мережею.

 **AWS Well-Architected** — Рамка кращих практик AWS із 6 стовпів: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.

 **CIDR** — Запис діапазону IP-адрес (10.0.0.0/16): чим менше число після «/», тим більше адрес.

 **Cold start** — Затримка першого виклику Lambda, поки створюється й ініціалізується середовище.

 **Dead-letter queue** — Окрема черга для повідомлень, які не вдалося обробити після кількох спроб.

 **Envelope encryption** — Дані шифрує ключ даних, а його шифрує ключ KMS; так шифрують дані понад 4 KB.

 **Fan-out** — Шаблон: одне повідомлення SNS розсилається в кілька черг SQS для незалежної обробки.

 **FIFO** — «Першим прийшов — першим пішов»: у SQS і SNS FIFO — строгий порядок у групі і без дублів.

 **IOPS** — Кількість операцій читання-запису за секунду — швидкість диска для дрібних запитів.

 **MFA** — Багатофакторна автентифікація: пароль + одноразовий код; MFA Delete захищає версії в S3 від видалення.

 **Multi-AZ** — Розгортання в кількох AZ; у RDS — синхронна запасна копія з автоматичним перемиканням.

🔊 **NFS, SMB, iSCSI** — Мови, якими сервери говорять із мережевим сховищем. NFS — спільні папки для Linux; SMB — спільні папки для Windows; iSCSI — диск по мережі (блочний доступ).

🔑 **Origin Access Control** — Дозволяє читати приватний bucket S3 лише через CloudFront; OAI — застарілий попередник.

📍 **Placement group** — Як розміщувати EC2: cluster (поруч і швидко), spread (окремо, до 7 на AZ), partition (секціями).

⏮️ **Point-in-time recovery (PITR)** — Відновлення бази на будь-яку секунду в межах періоду зберігання (у RDS і DynamoDB — до 35 днів). Відновлення створює нову базу чи таблицю.

📄 **Presigned URL** — Тимчасове посилання на приватний об'єкт S3 з правами того, хто його підписав.

📖 **Read replica** — Асинхронна копія бази лише для читання: розвантажує основну базу, може бути в іншому регіоні.

🌐 **Region (region)** — Географічний регіон AWS (напр., eu-central-1) з кількома ізольованими зонами доступності.

⏮️ **RPO** — Recovery Point Objective — скільки даних (за часом) можна втратити під час аварії.

🕒 **RTO** — Recovery Time Objective — за скільки часу система має знову запрацювати.

♻️ **S3 Lifecycle** — Правила, що автоматично переводять об'єкти в дешевші класи і видаляють їх з часом.

🔒 **S3 Object Lock** — WORM-захист об'єктів: compliance (не видалить ніхто, навіть root), governance (з особливим дозволом), legal hold.

🔄 **S3 Replication (CRR / SRR)** — Автоматичне копіювання нових об'єктів S3 в інший регіон (CRR) або в той самий (SRR); потрібен versioning.

👤 **Shared Responsibility Model** — AWS відповідає за безпеку хмари (OF), клієнт — за безпеку в хмарі (IN): дані, доступи, налаштування.

🗑️ **Stateless** — Сервер не тримає стан користувача (сесії), тому його можна вільно додавати й прибирати.

🕒 **Versioning** — Зберігає всі версії об'єкта в S3; видалення ставить маркер, стару версію можна повернути.

⌚ **Visibility timeout** — Час, протягом якого взяте з SQS повідомлення невидиме для інших споживачів (за замовчуванням 30 с, максимум 12 год).

🌿 **Підмережа (subnet)** — Частина VPC в одній AZ. Публічна має маршрут на Internet Gateway, приватна — ні.

Обчислення

 **Amazon EC2** — Віртуальні сервери в оренду: обираєш процесор, пам'ять, диск і ОС, платиш посекундно.

 **AWS Batch** — Черги й планувальник для пакетних задач будь-якої тривалості на EC2, Spot або Fargate.

 **AWS Elastic Beanstalk** — PaaS: завантажуєш код, а AWS сам створює EC2, Auto Scaling, балансувальник і моніторинг.

 **AWS Graviton** — Процесори AWS на архітектурі ARM: краще співвідношення ціни й продуктивності та менше енергії, ніж у порівнянних x86-інстансів.

 **AWS Local Zones** — Розширення регіону у великому місті: EC2, EBS та інші сервіси з одноцифровими мілісекундами до мешканців.

 **AWS Outposts** — Стійки з обладнанням AWS у твоєму дата-центрі: ті самі сервіси й API, але локально.

 **AWS Wavelength** — Обчислення AWS усередині 5G-мереж мобільних операторів.

 **Dedicated Hosts і Dedicated Instances** — Обладнання лише для тебе. Dedicated Instances — твої інстанси на серверах, які не ділиш з іншими клієнтами. Dedicated Host — цілий фізичний сервер під твоїм контролем: видно сокети й ядра, тож можна використати власні ліцензії (BYOL).

 **EC2 Auto Scaling** — Автоматично додає й прибирає EC2 за навантаженням і замінює нездорові інстанси; тримає min/desired/max у кількох AZ.

 **VMware Cloud on AWS** — Середовище VMware vSphere на виділеному обладнанні AWS, яким керують звичними інструментами VMware.

Serverless

 **Amazon API Gateway** — Керований «вхід» для API: REST, HTTP і WebSocket, з авторизацією, лімітами, кешем і ключами.

 **AWS Fargate** — Запуск контейнерів ECS або EKS без керування серверами: платиш за процесор і пам'ять задачі.

 **AWS Lambda** — Запускає твій код у відповідь на подію без серверів; до 15 хв на виклик, платиш за час виконання.

Контейнери

 **Amazon ECR** — Приватний реєстр образів контейнерів зі скануванням вразливостей і реплікацією.

 **Amazon ECS** — Оркестрація контейнерів від AWS: запускає й масштабує задачі на EC2 або Fargate.

 **Amazon EKS** — Керований Kubernetes: AWS веде керівну частину кластера, ти запускаєш поди на EC2 або Fargate.

Сховище

 **Amazon Data Lifecycle Manager** — Автоматично створює і видаляє знімки EBS та AMI за розкладом.

 **Amazon EBS** — Блочні диски для EC2 в одній AZ; знімки (snapshots) зберігаються в S3. Типи: gp3 — універсальний SSD, продуктивність налаштовується окремо від розміру (gp2 — попередник, де IOPS залежать від розміру); io2 Block Express — найшвидший SSD для критичних баз; st1 — HDD для великих послідовних даних; sc1 — найдешевший холодний HDD.

 **Amazon EFS** — Керована спільна файлова система NFS для Linux у кількох AZ; об'єм росте автоматично.

 **Amazon FSx** — Сімейство керованих файлових систем: for Windows File Server, for Lustre, for NetApp ONTAP, for OpenZFS.

 **Amazon S3** — Об'єктне сховище будь-якого обсягу: файли як об'єкти в bucket, доступ через API, надійність 11 дев'яток.

 **AWS Backup** — Централізовані плани бекапів для EC2, EBS, RDS, Aurora, DynamoDB, EFS, FSx, S3 тощо, з копіями між регіонами й акаунтами.

 **FSx for Lustre** — Високопродуктивна файлова система для HPC і ML: сотні GB/s, затримка менше мілісекунди, інтеграція з S3.

 **FSx for NetApp ONTAP** — Керований NetApp ONTAP: NFS, SMB і iSCSI одночасно, знімки, клонування, дедуплікація.

 **FSx for Windows File Server** — Керований файловий сервер Windows: SMB, NTFS, інтеграція з Active Directory, Multi-AZ.

 **Instance store** — Локальні диски фізичного сервера EC2: найшвидші, але дані зникають при зупинці чи збої.

 **S3 Glacier** — Архівні класи S3: Instant Retrieval (мілісекунди, мінімум 90 днів), Flexible Retrieval (хвилини–години, 90), Deep Archive (до 12–48 год, 180, найдешевше).

 **S3 Intelligent-Tiering** — Клас S3, що сам переносить об'єкти між рівнями за частотою доступу; без плати за отримання.

 **S3 Standard-IA і One Zone-IA** — Класи S3 для рідкого доступу з миттєвим читанням: зберігання дешевше, але є плата за кожен прочитаний гігабайт і мінімум 30 днів. One Zone-IA зберігає дані лише в одній AZ — ще дешевше, але зникне разом із зоною.

 **S3 Transfer Acceleration** — Прискорене завантаження файлів у S3 з далеких країн: дані заходять у найближчу edge-локацію і далі йдуть мережею AWS.

Бази даних

 **Amazon Aurora** — Сумісна з MySQL і PostgreSQL база від AWS: 6 копій у 3 AZ, до 15 реплік, сховище до 256 TiB.

 **Amazon DocumentDB** — Керована документна база, сумісна з MongoDB.

 **Amazon DynamoDB** — Serverless NoSQL «ключ-значення» з мілісекундною затримкою на будь-якому масштабі; елемент до 400 KB.

 **Amazon ElastiCache** — Керований кеш у пам'яті: Redis OSS / Valkey (реплікація, Multi-AZ, складні структури) або Memcached (простий).

 **Amazon Keyspaces** — Serverless база, сумісна з Apache Cassandra (CQL).

 **Amazon MemoryDB** — Сумісна з Redis/Valkey основна база в пам'яті з надійним збереженням (не просто кеш).

 **Amazon Neptune** — Графова база для даних, де головне — зв'язки.

 **Amazon RDS** — Керовані реляційні бази (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2): патчі, бекапи й Multi-AZ робить AWS.

 **Amazon RDS Proxy** — Пул з'єднань між застосунком і RDS/Aurora: менше навантаження, швидше перемикання, IAM-автентифікація.

 **Amazon Timestream** — База для часових рядів: показники з мітками часу.

 **Aurora Global Database** — Реплікація Aurora на рівні сховища до 10 інших регіонів: затримка менше секунди, перемикання менш ніж за хвилину.

 **Aurora Serverless v2** — Aurora, що сама масштабує потужність в ACU (≈ 2 GiB пам'яті кожна) і може «засинати» до 0 ACU.

 **DynamoDB Accelerator (DAX)** — Кеш у пам'яті перед DynamoDB: мікросекунди, сумісний API, майже без змін коду.

 **DynamoDB Global Tables** — Таблиця DynamoDB у кількох регіонах із записом у будь-якому (multi-active).

 **DynamoDB Streams** — Стрічка змін таблиці DynamoDB (додали, змінили, видалили елемент), яка зберігається 24 години; її можна обробляти функціями Lambda.

Мережа

 **Amazon CloudFront** — CDN: кешує контент на edge-локаціях і прискорює доставку по всьому світу; OAC, signed URL, WAF.

 **Amazon Route 53** — Керований DNS з перевітками здоров'я і політиками маршрутизації (weighted, latency, failover, geolocation...).

 **Amazon VPC** — Приватна мережа в регіоні з твоїм діапазоном адрес, підмережами, маршрутами й охороною.

 **Application Load Balancer** — Балансувальник рівня 7: маршрутизація за шляхом, доменом, заголовками; цілі — EC2, IP, Lambda, контейнери.

 **AWS Client VPN** — VPN-доступ окремих користувачів (ноутбуків) до VPC на базі OpenVPN.

 **AWS Direct Connect** — Виділений приватний канал до AWS: 1–400 Gbps, стабільна швидкість; прокладання — тижні; сам не шифрує.

 **AWS Global Accelerator** — 2 статичні anycast IP; трафік іде мережею AWS до найближчого здорового регіону; TCP і UDP; нічого не кешує.

 **AWS PrivateLink** — Приватний доступ до конкретного сервісу через interface endpoint; власник публікує сервіс через NLB.

 **AWS Site-to-Site VPN** — Зашифровані IPsec-тунелі між офісом і AWS через інтернет; 2 тунелі, налаштування за хвилини.

 **AWS Transit Gateway** — Мережевий хаб для десятків і сотень VPC, VPN і Direct Connect із транзитивною маршрутизацією.

 **Elastic IP** — Статична публічна IPv4-адреса, яку можна швидко переприв'язати до іншого інстансу. Кожна публічна IPv4-адреса в AWS платна, навіть коли не використовується.

 **Elastic Load Balancing** — Керовані балансувальники: ALB (рівень 7), NLB (рівень 4), GWLB (рівень 3); розподіляють трафік і перевіряють здоров'я цілей.

 **Gateway Load Balancer** — Балансувальник рівня 3 (GENEVE, порт 6081) для віртуальних апаратів безпеки сторонніх вендорів.

 **Internet Gateway** — Ворота між VPC та інтернетом; маршрут 0.0.0.0/0 на них робить підмережу публічною. Egress-only — лише вихід для IPv6.

 **NAT Gateway** — Дає серверам у приватних підмережах вихід в інтернет без вхідного доступу; платний за годину і за кожен ГБ.

 **Network Load Balancer** — Балансувальник рівня 4 (TCP/UDP/TLS): мільйони запитів, наднизька затримка, статичний IP у кожній AZ.

 **Route 53 Resolver** — DNS у VPC; inbound і outbound endpoints з'єднують DNS офіси і VPC.

 **VPC endpoints** — Приватний доступ із VPC до сервісів AWS без інтернету: gateway (S3 і DynamoDB, безкоштовно) та interface (PrivateLink, майже всі сервіси, платно).

 **VPC Flow Logs** — Записують метадані IP-трафіку (хто, куди, порт, ACCEPT чи REJECT) у CloudWatch Logs, S3 або Firehose.

 **VPC Peering** — Пряме з'єднання двох VPC (між акаунтами й регіонами); нетранзитивне, адреси не можуть перетинатися.

Безпека

 **Amazon Cognito** — Вхід для клієнтів застосунку: User Pools (реєстрація, вхід, JWT) та Identity Pools (тимчасові AWS-ключі).

 **Amazon Detective** — Розслідування інцидентів: будує граф подій і допомагає знайти першопричину знахідок GuardDuty.

 **Amazon GuardDuty** — Виявляє загрози за CloudTrail, VPC Flow Logs і DNS-логами: криптомайнінг, викрадені ключі, підозрілі IP.

 **Amazon Inspector** — Безперервно шукає вразливості (CVE) в EC2, образах ECR і Lambda.

 **Amazon Macie** — За допомогою ML знаходить чутливі дані (персональні дані, номери карток) в S3.

 **AWS Artifact** — Портал зі звітами відповідності самого AWS (SOC, PCI, ISO) і угодами (напр., BAA).

 **AWS Audit Manager** — Автоматично збирає докази відповідності для аудитів.

 **AWS Certificate Manager** — Безкоштовні публічні TLS-сертифікати для ELB, CloudFront і API Gateway з автоматичним поновленням.

 **AWS CloudHSM** — Виділений апаратний модуль безпеки (FIPS 140 Level 3); ключі під повним контролем клієнта.

 **AWS Control Tower** — Готова landing zone для багатьох акаунтів: Account Factory, guardrails (SCP, Config), централізовані логи.

 **AWS Directory Service** — Active Directory в AWS: Managed Microsoft AD (справжній AD з довірою), AD Connector (проксі до офісного AD), Simple AD.

 **AWS Firewall Manager** — Централізовано керує WAF, Shield Advanced, Security Groups і Network Firewall у всіх акаунтах організації.

 **AWS IAM** — Керує тим, хто (users, groups, roles) і що (policies) може робити в AWS; явний Deny перемагає все.

 **AWS KMS** — Керовані ключі шифрування з аудитом кожного використання; напряму шифрує до 4 KB, далі — envelope encryption.

 **AWS Network Firewall** — Керований stateful/stateless файрвол для VPC з IPS і фільтрацією за доменами.

 **AWS Organizations** — Об'єднує багато акаунтів: OU, спільний рахунок (consolidated billing), SCP і RCP.

 **AWS Resource Access Manager** — Ділиться ресурсами між акаунтами: підмережами (VPC sharing), Transit Gateway, правилами Resolver тощо.

 **AWS Secrets Manager** — Зберігає секрети (паролі, ключі) і автоматично змінює їх (ротація), зокрема для RDS і Aurora.

 **AWS Security Hub** — Єдина панель знахідок безпеки (GuardDuty, Inspector, Macie...) і перевірки стандартів (CIS, PCI DSS).

 **AWS Shield** — Захист від DDoS: Standard — безкоштовно для всіх; Advanced — платно, з командою експертів 24/7 і компенсацією витрат.

 **AWS STS** — Видає тимчасові облікові дані: AssumeRole, AssumeRoleWithSAML, AssumeRoleWithWebIdentity.

 **AWS WAF** — Вебфайрвол рівня 7 для CloudFront, ALB, API Gateway, AppSync, Cognito: SQL injection, XSS, блок країн, ліміти запитів.

 **IAM Access Analyzer** — Знаходить ресурси, відкриті зовнішнім акаунтам, і допомагає прибрати зайві права (Access Advisor показує невикористані сервіси).

 **IAM Identity Center** — Єдиний вхід співробітників у всі акаунти AWS і бізнес-застосунки; з AD або зовнішнім IdP (Okta, Entra ID).

 **IAM role** — Набір прав, який тимчасово «надягають» сервіси, застосунки чи люди; тимчасові ключі видає STS.

 **Network ACL** — Файрвол на рівні підмережі: stateless, є Allow і Deny, правила перевіряються за номерами.

 **Permission boundary** — «Стеля» прав для окремого користувача чи ролі: навіть якщо політика дає більше, діє лише те, що дозволяє і політика, і межа.

 **Resource-based policy (bucket policy)** — Політика, прикріплена до самого ресурсу (S3 bucket, черпа SQS, ключ KMS, функція Lambda): вона каже, хто може до нього звертатися, — зокрема користувачі й сервіси з інших акаунтів.

 **Security Group** — Файрвол на рівні інстансу: stateful, лише Allow-правила, може посилатися на інші Security Groups.

 **Service control policy (SCP)** — Політика Organizations, що задає максимум дозволів для акаунтів і OU; сама прав не дає і не діє на management account. RCP (resource control policy) — така сама «стеля», але для ресурсів акаунтів.

 **Федерація (SAML, OIDC)** — Вхід в AWS зі зовнішнім обліковим записом (корпоративний каталог, Google, Apple) без створення IAM-користувачів: після перевірки AWS видає тимчасові облікові дані через STS.

 **Шифрування S3 (SSE)** — S3 шифрує об'єкти під час запису. SSE-S3 — ключами, якими керує S3 (увімкнено за замовчуванням для всіх нових об'єктів з 2023 року); SSE-KMS — ключем у KMS з аудитом у CloudTrail і контролем доступу до ключа; DSSE-KMS — два шари шифрування; SSE-C — ключ приносить клієнт з кожним запитом, AWS його не зберігає.

Інтеграція

 **Amazon AppFlow** — Переносить дані між SaaS (Salesforce, SAP, Zendesk) і S3 чи Redshift без коду.

 **Amazon EventBridge** — Шина подій від AWS, твоїх застосунків і SaaS з правилами маршрутизації; Scheduler — запуск за розкладом.

 **Amazon MQ** — Керовані брокери ActiveMQ і RabbitMQ для застосунків на JMS, AMQP, MQTT, STOMP.

 **Amazon SNS** — Pub/sub (push): одне повідомлення — багатьом підписникам (SQS, Lambda, email, SMS, HTTP).

 **Amazon SQS** — Керована черга повідомлень (pull): Standard — майже без лімітів, FIFO — порядок і без дублів; зберігання до 14 днів.

 **AWS Step Functions** — Оркестрація багатокрокових процесів: послідовність, розгалуження, повтори, очікування ручного схвалення.

Аналітика

 **Amazon Athena** — Serverless SQL прямо по файлах у S3; платиш за обсяг прочитаних даних.

 **Amazon Data Firehose** — Керована доставка потоків у S3, Redshift, OpenSearch, Splunk тощо (майже реальний час), з перетворенням у Parquet.

 **Amazon EMR** — Керовані кластери Spark, Hadoop, Hive, Presto для великих даних; task-вузли можна на Spot.

 **Amazon Kinesis Data Streams** — Потік даних у реальному часі з кількома споживачами і перемоткою (replay); зберігання від 24 год до 365 днів.

 **Amazon Kinesis Video Streams** — Приймає відеопотоки з камер для зберігання, аналітики і ML.

 **Amazon MSK** — Керований Apache Kafka (і MSK Serverless) для наявних Kafka-застосунків.

 **Amazon OpenSearch Service** — Повнотекстовий пошук і аналіз логів з дашбордами (сумісний з Elasticsearch).

 **Amazon Quick (QuickSight)** — BI-дашборди для бізнесу (колишній QuickSight): serverless, SPICE, вбудовування в застосунки.

 **Amazon Redshift** — Колонкове сховище даних (OLAP) для складної аналітики на петабайтах; Spectrum читає дані прямо з S3.

 **AWS Data Exchange** — Маркетплейс сторонніх датасетів з доставкою в S3.

 **AWS Glue** — Serverless ETL на Spark, Data Catalog (опис таблиць) і crawlers, що самі визначають схему.

 **AWS Lake Formation** — Будує і захищає data lake на S3: дозволи до окремих колонок і рядків для Athena, Redshift Spectrum, EMR, Glue.

 **Managed Service for Apache Flink** — Аналіз потоків у реальному часі (агрегації у вікнах часу, аномалії) на Apache Flink.

ML / AI

 **Amazon Comprehend** — Аналіз тексту (NLP): тональність, сутності, ключові фрази, мова, персональні дані.

 **Amazon Lex** — Чат-боти з розумінням голосу і тексту.

 **Amazon Polly** — Перетворює текст на природне мовлення.

 **Amazon Rekognition** — Аналіз зображень і відео: об'єкти, обличчя, текст, модерація контенту.

 **Amazon SageMaker AI** — Платформа, щоб будувати, навчати й розгортати власні ML-моделі.

 **Amazon Textract** — Витягує текст, форми і таблиці зі сканів і PDF.

 **Amazon Transcribe** — Перетворює мову на текст (дзвінки, відео, субтитри).

 **Amazon Translate** — Машинний переклад тексту.

Керування

 **Amazon CloudWatch** — Метрики, логи, аларми й дашборди; пам'ять і диск EC2 — лише з CloudWatch agent.

 **AWS CloudFormation** — Інфраструктура як код: шаблони YAML/JSON створюють однакові середовища; StackSets — у багатьох акаунтах і регіонах.

 **AWS CloudTrail** — Журнал усіх викликів API: хто, що, коли і звідки; 90 днів історії безкоштовно.

 **AWS Config** — Інвентар та історія налаштувань ресурсів і правила відповідності з автовиправленням.

 **AWS Health Dashboard** — Події AWS, що зачіпають саме твої ресурси (збої, планове обслуговування); інтеграція з EventBridge.

 **AWS License Manager** — Облік і контроль ліцензій (Microsoft, Oracle, SAP), заборона перевищення лімітів.

 **AWS Service Catalog** — Каталог затверджених шаблонів для самообслуговування користувачів з обмеженими правами.

 **AWS Systems Manager** — Керування автопарком серверів: вхід без SSH (Session Manager), команди (Run Command), патчі (Patch Manager), Parameter Store.

 **AWS Trusted Advisor** — Рекомендації щодо вартості, безпеки, стійкості, продуктивності й лімітів сервісів.

 **AWS X-Ray** — Трасування запитів через мікросервіси: де затримка чи помилка.

 **Managed Grafana i Managed Prometheus** — Керовані Grafana (дашборди) і Prometheus (метрики контейнерів) без власних серверів.

 **SSM Parameter Store** — Сховище налаштувань і секретів (SecureString з KMS) в ієрархії; standard tier безкоштовний; без вбудованої ротації.

Вартість

 **AWS Budgets** — Сповіщення, коли фактичні чи прогнозовані витрати перевищать поріг; Budget Actions — автоматичні дії.

 **AWS Compute Optimizer** — ML-рекомендації правильного розміру для EC2, Auto Scaling, EBS, Lambda, ECS на Fargate, RDS.

 **AWS Cost and Usage Report** — Найдетальніші дані про витрати (кожен ресурс, кожна година) у S3 для аналізу в Athena.

 **AWS Cost Explorer** — Аналіз і прогноз витрат, поради щодо Savings Plans і RI.

 **Cost Anomaly Detection** — ML-сповіщення про незвичні стрибки витрат.

 **On-Demand Instances** — Оплата EC2 посекундно чи погодинно без жодних зобов'язань: найдорожча година, зате максимальна гнучкість. Capacity Reservations — бронь потужності в конкретній AZ за ціною On-Demand.

 **Reserved Instances** — Знижка до ~72% за зобов'язання на 1–3 роки для конкретного типу інстансу; Convertible — можна міняти сімейство.

 **Savings Plans** — Знижка за зобов'язання витратити \$/год 1 або 3 роки: Compute (до ~66%, будь-яке сімейство, Fargate, Lambda), EC2 Instance (до ~72%).

 **Spot Instances** — Вільні потужності EC2 зі знижкою до 90%; AWS може забрати їх з попередженням за 2 хвилини.

Міграція

 **AWS Application Migration Service** — Перенесення серверів «як є» (фізичних, VMware, з інших хмар) в EC2 з безперервною реплікацією.

 **AWS Database Migration Service** — Міграція баз з мінімальним простоем і безперервним копіюванням змін (CDC).

 **AWS DataSync** — Онлайн-перенесення і синхронізація великих обсягів з NFS, SMB, HDFS у S3, EFS, FSx — з перевіркою і за розкладом.

 **AWS Elastic Disaster Recovery** — Відновлення серверів в AWS після аварії: постійна реплікація, RPO — секунди, RTO — хвилини.

 **AWS Schema Conversion Tool** — Конвертує схеми й код баз між різними рушіями (Oracle → PostgreSQL, сховища даних → Redshift).

 **AWS Snow Family** — Фізичні пристрої для офлайн-перенесення терабайтів і петабайтів та обчислень на місці (з 11.2025 недоступні новим клієнтам).

 **AWS Storage Gateway** — Гібридний доступ офісу до сховища AWS з локальним кешем: S3 File, Volume (Cached чи Stored) і Tape Gateway.

 **AWS Transfer Family** — Керований SFTP/FTPS/FTP/AS2-сервер поверх S3 або EFS.

Веб і мобільні

 **AWS Amplify** — Хостинг і бекенд для веб- і мобільних застосунків з автоматичним розгортанням фронтенду.

 **AWS Device Farm** — Тестування застосунків на сотнях реальних телефонів, планшетів і браузерів в AWS.